LOGIX CONNECT

PLC TRAINING WORKSTATION

✓ Compiling 1 routine · 2 rungs
✓ Opening the Start Page
DN · App_Ready = 1

1769-L33ER · VERSION 98.4

CREATED AND DESIGNED BY DEVON CALHOUN

The Programmer’s Guide

Ladder logic from the first rung to a running bottling line — a textbook, a parts catalogue and a step-by-step build of the bottle line from an empty sandbox

How to use this book

This is three books in one. **Part I and II** are the textbook: what a controller does, what a rung is, every instruction you will use, and how to read the machine while it runs. **Part III** is the catalogue: every part in the sandbox palette with its parameters and I/O, generated from the program itself so it cannot drift. **Part IV** is the build: a bottling line laid out, wired and programmed from an empty floor, one routine at a time, with something to run and watch at the end of every lesson. **Part V** points past the bottle line, and the appendices hold the complete program listing, the tag list, the register bit map, the shortcuts and the glossary.

If you have never programmed a PLC, read Parts I and II in order and do the small exercises; they take an afternoon. If you have, skim Part II for the house rules (rung text, tag naming, the alias convention) and go to Part IV. Keep Part III open while you build anything of your own.

Conventions. Rung text is printed exactly as you type it into the editor, in a box like this:

A rung, with its comment above it

```
XIC(Start_PB)XIC(Stop_PB)OTE(Run);
```

Names in `fixed type` are tags, signals and addresses as they appear in the program. Menu paths are written **File → New Project…**. A *Try it* box is an exercise with a known answer; a *Watch for* box is a mistake people make at that point. Everything in this book was checked against Version 98.4 of the page: the program's own regression suite builds the line and runs the lessons' program every release.

Contents

Part I — Getting started

1. What Logix Connect is 6
2. The workbench: a tour 9
3. Projects: new, save, open, export 17

Part II — Ladder logic from zero

4. How a controller thinks: the scan and the rung 20
5. Tags, data types and I/O addresses 25
6. Bit instructions 29
7. Timers and counters 32
8. Compare, move and math 35
9. Routines, programs and the main routine 37
10. Shift registers: tracking parts down a line 42
11. Reading the machine while it runs 45

Part III — The sandbox and its parts

12. The sandbox floor 49
13. Parts catalogue 57
14. Operating the Process HMI 102

Part IV — Building the bottle line from ground zero

15. The line we are going to build 119
16. Lesson 1 — Lay out the machine 122
17. Lesson 2 — Name the I/O: the tag list 130
18. Lesson 3 — Step 1: the run circuit 137
19. Lesson 4 — Step 2: the dwell timer and the feeder 141
20. Lesson 5 — Step 3: the shift registers 144
21. Lesson 6 — Step 4: filling 147
22. Lesson 7 — Step 5: capping and the cap feeder 152
23. Lesson 8 — Step 6: index one pitch 158
24. Lesson 9 — Step 7: the gates, the boxes and the shredder 161
25. Lesson 10 — Step 8: the displays, the preset and Enter 165
26. Lesson 11 — Step 9: the reservoir and the stack light 171
27. Lesson 12 — Step 10: the labeler, FLS logic 176
28. Lesson 13 — Step 11: alarms and machine faults 182
29. Lesson 14 — Step 12: the case packer and the VFD 198

30. Commissioning: the whole line, start to finish 210

Part V — The graded levels

31. The graded levels: the machine, the bins and the score 213
32. Level 1 — Height sort 218
33. Level 2 — Fill by label 229
34. Level 3 — Colour sort 241
35. Level 4 — Two on the line 252
36. Level 5 — Full sort 273
37. Level 6 — Pack it 296
38. Level 7 — Batch Mix 310

Part VI — Beyond the bottle line

39. Save, share and replay 328
40. Where to go next 332

Appendices

Appendix A — The bottle line tag list 334
Appendix B — The complete program 340
Appendix C — The register bit map 356
Appendix D — Sensor reference 357
Appendix E — Keyboard shortcuts 363
Appendix F — Glossary 364
Appendix G — Instruction set 375
Appendix H — Walkthrough log 376

Part I — Getting started

1 What Logix Connect is

Logix Connect is a PLC training workstation that runs in a browser (and, wrapped, as a Windows app and a phone app). It puts three things on one screen that in a plant are three separate things: a **controller** styled on a CompactLogix with Studio 5000-style editing, a **chassis of I/O modules** that the controller scans, and a **machine** wired to that I/O: either one of the graded bottle-line levels or the sandbox, where you lay the machine out yourself from a palette of parts.

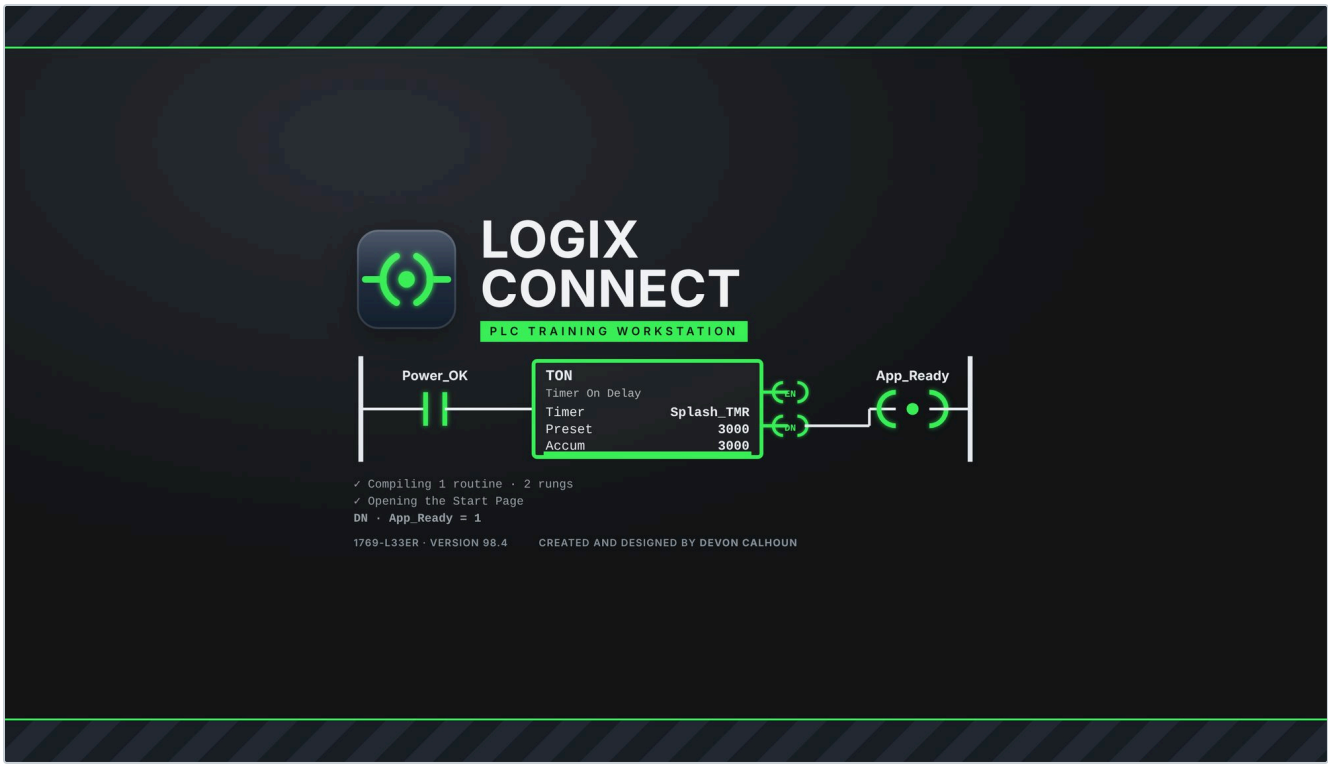


Figure 1. The splash is itself a rung: Power_OK closes, a TON times 3000 ms, and the App_Ready coil opens the workbench. By the end of chapter 7 you will read it without thinking.

The controller is real in the ways that matter for learning. It scans: inputs are read, the ladder is solved rung by rung, top to bottom, left to right, outputs are written, and it does that again every 20 ms. Tags are typed (BOOL, DINT, TIMER…), I/O lives at module addresses (`Local:1:I.Data.0`), alias tags give those addresses names, routines are called from a main routine with JSR, and a rung is written in the same neutral text Studio 5000 uses when you copy a rung. Projects export to L5X and import from it. What it leaves out is the parts of a real controller that teach nothing about logic: no firmware, no download time, no licence.

The machine is also real in the ways that matter. Conveyors carry parts at a speed, sensors see what is physically in front of them, a gate that fires late pushes the wrong bottle, a shredder that runs with no box under it spills glass onto the floor, a capper with an empty chute leaves bottles open, and a tank runs dry if

the inlet valve is never opened. The simulator does not know what you meant; it only knows what your outputs did. That is the whole point.

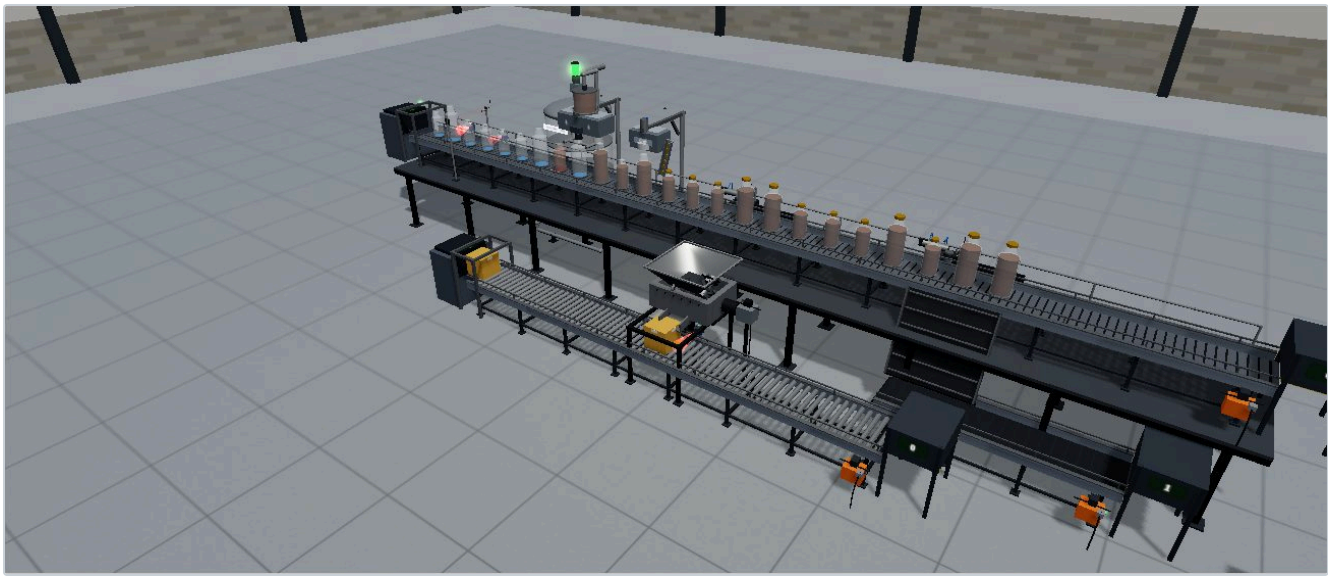


Figure 2. The LogixPro-style bottle line of Part IV, in the 3D view: a raised main conveyor, the fill station and capper, the scrap gate over the shredder and its box conveyor, the divert gate and its incline.

The chassis

Every sandbox project has the same chassis. Slot 0 is the controller, slots 1–3 are 16-point input modules, slots 4–6 are 16-point output modules, slots 7 and 9 are 4-channel analog input modules and slot 8 is a 4-channel analog output module. A point is written the way Studio 5000 writes it: `Local:<slot>:I.Data.<bit>` for an input bit, `Local:<slot>:O.Data.<bit>` for an output bit and `Local:<slot>:I.Ch<n>Data` for an analog channel.

Slot	Module	Name	Points
0	1769-L33ER	LogixConnect	controller
1	1769-IQ16	DI_1	16 digital inputs, Local:1:I.Data.0–15
2	1769-IQ16	DI_2	16 digital inputs, Local:2:I.Data.0–15
3	1769-IQ16	DI_3	16 digital inputs, Local:3:I.Data.0–15
4	1769-OB16	DO_1	16 digital outputs, Local:4:O.Data.0–15
5	1769-OB16	DO_2	16 digital outputs, Local:5:O.Data.0–15
6	1769-OB16	DO_3	16 digital outputs, Local:6:O.Data.0–15
7	1769-IF4	AI_1	4 analog inputs, Local:7:I.Ch0Data–Ch3Data
8	1769-OF4	AO_1	4 analog outputs, Local:8:O.Ch0Data–Ch3Data
9	1769-IF4	AI_2	4 analog inputs, Local:9:I.Ch0Data–Ch3Data

Slot	Module	Name	Points
10	1769-IQ16	DI_4	16 digital inputs, Local:10:I.Data.0–15

That is 64 digital inputs, 48 digital outputs, 8 analog inputs and 4 analog outputs. When you place a part in the sandbox, each of its signals takes the next free point of the right kind; the Scene sheet shows how many are in use. If a kind runs out, the part is placed but the signal has no address, and the status bar says so.

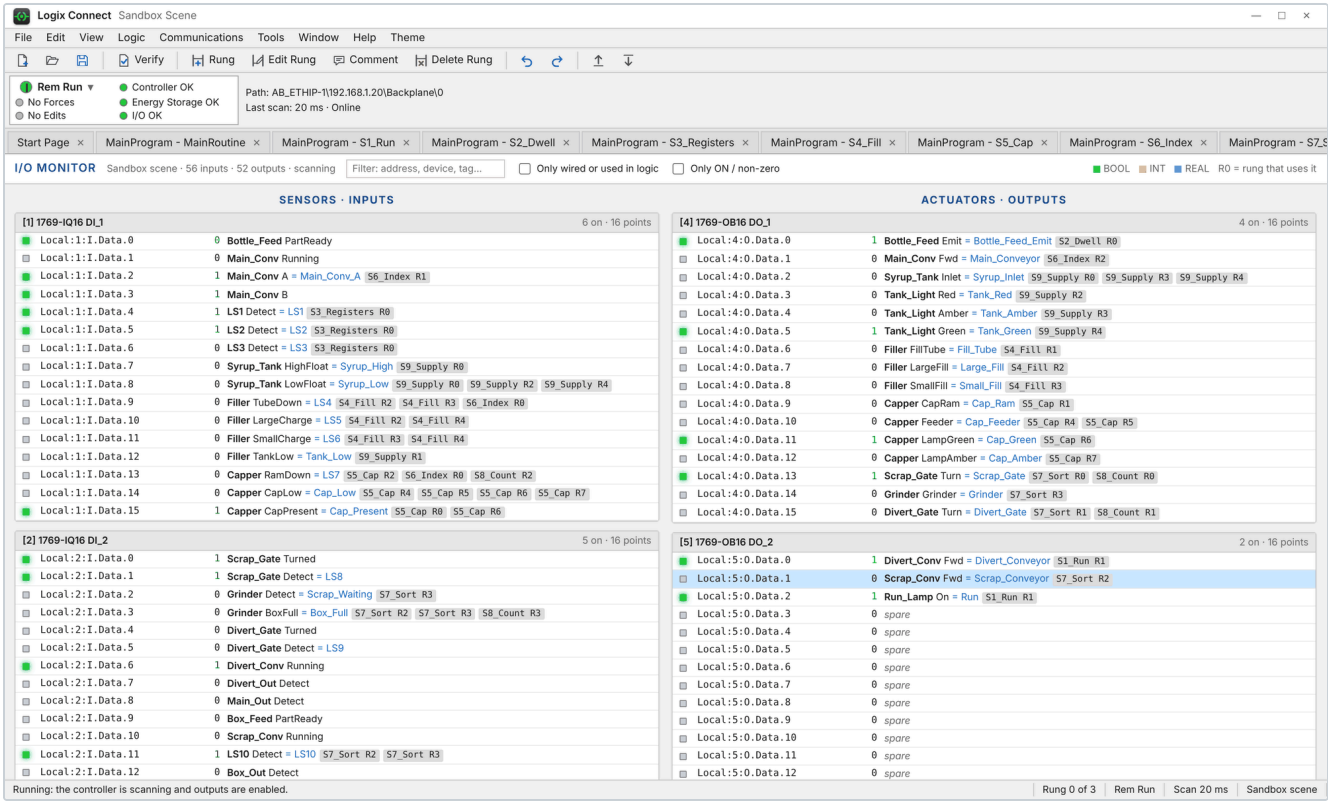


Figure 3. The I/O Monitor: every point of every module, live, with the device and signal wired to it.

Two kinds of project

A **level** is a graded bottle-line puzzle with a fixed machine and a score. The **sandbox** is the open floor: you choose the parts, the I/O and the program. This book is about the sandbox, because that is where you learn to build a machine and not just drive one; the levels are a good place to practise the instructions once you know them.

2 The workbench: a tour

Open the page and the splash gives way to the workbench. It is laid out like Logix Designer because the habits transfer: the **Controller Organizer** on the left, document tabs across the middle, the Errors / Simulator Log / Search Results / Cross Reference / Watch panes at the bottom, and a status bar under everything.

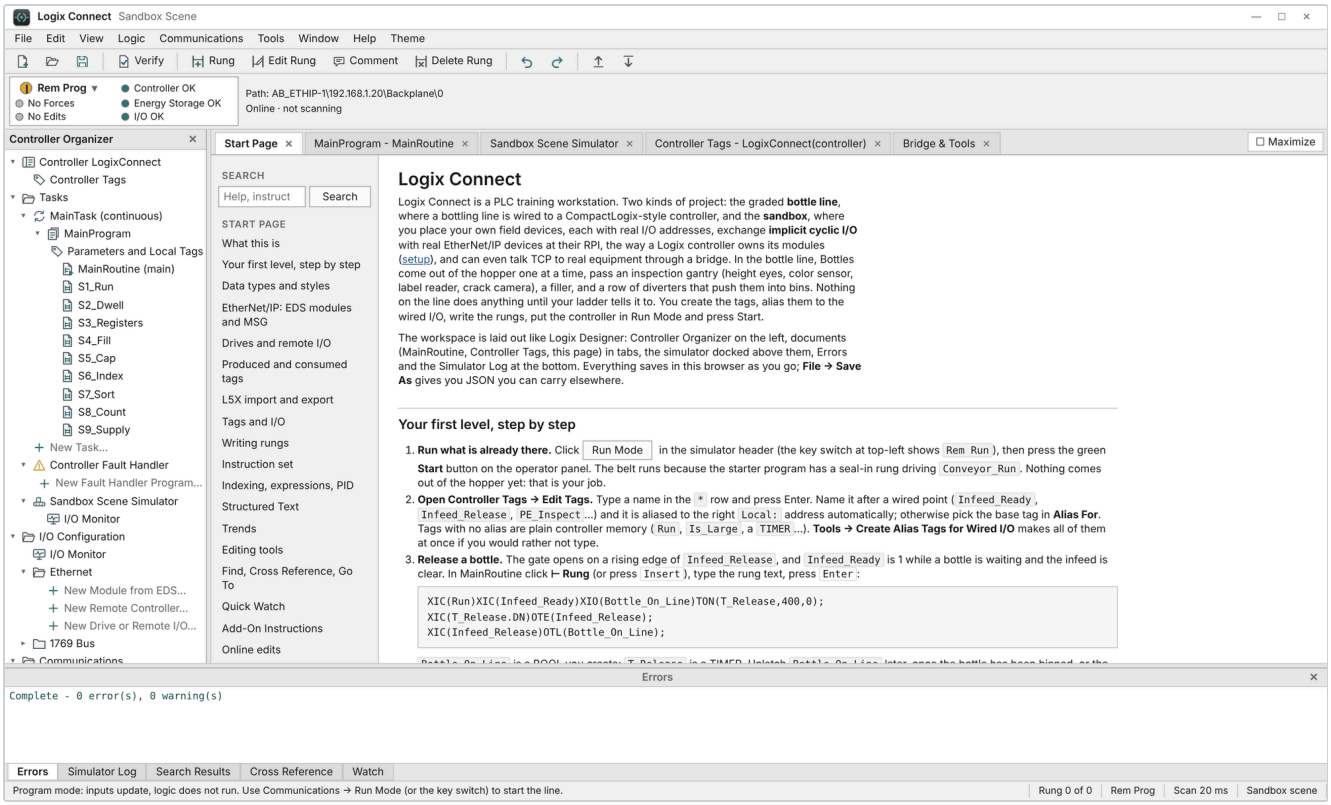


Figure 4. The workbench with the bottle-line project open: menus and toolbar, the key switch and controller status, the Controller Organizer, the document tabs (Start Page showing), the Errors pane and the status bar.

The title bar and the key switch

Top-left is the controller status: the key switch (*Program*, *Run*, *Rem Run*, *Test*), Forces, Edits, Controller OK, Energy Storage OK, I/O OK, and the path (`AB_ETHIP-1\192.168.1.20\Backplane\0`). The same switch is on the simulator's own header: **Running — click for Program Mode** and back. In Program mode inputs still update (you can watch a sensor change in Monitor Tags) but the ladder does not run and outputs are off. In Run mode the ladder scans and outputs drive the machine. Test mode scans without energising outputs.

Figure 5. The title bar, menus, the rung toolbar and the controller status block with the key switch and the path.

The menus

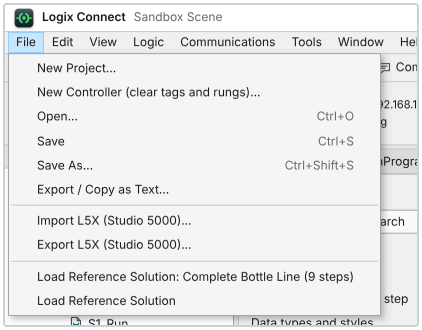


Figure 6. File

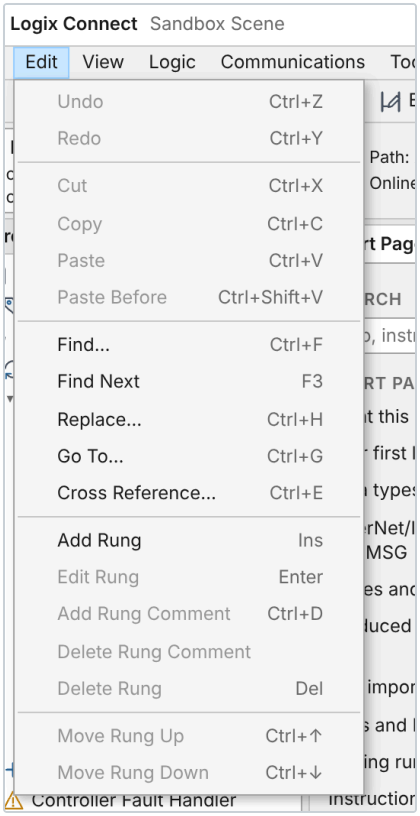


Figure 7. Edit

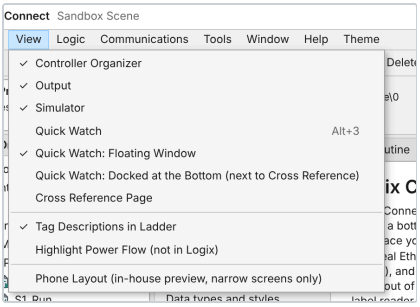


Figure 8. View

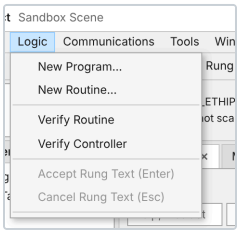


Figure 9. Logic

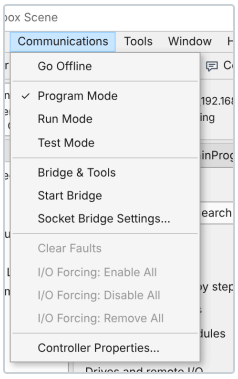


Figure 10. Communications

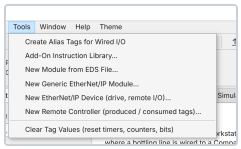


Figure 11. Tools

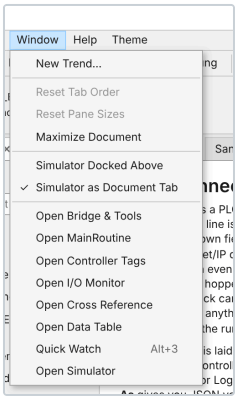


Figure 12. Window

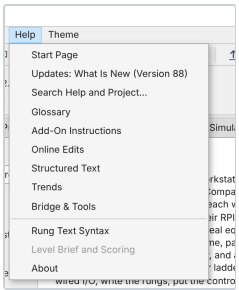


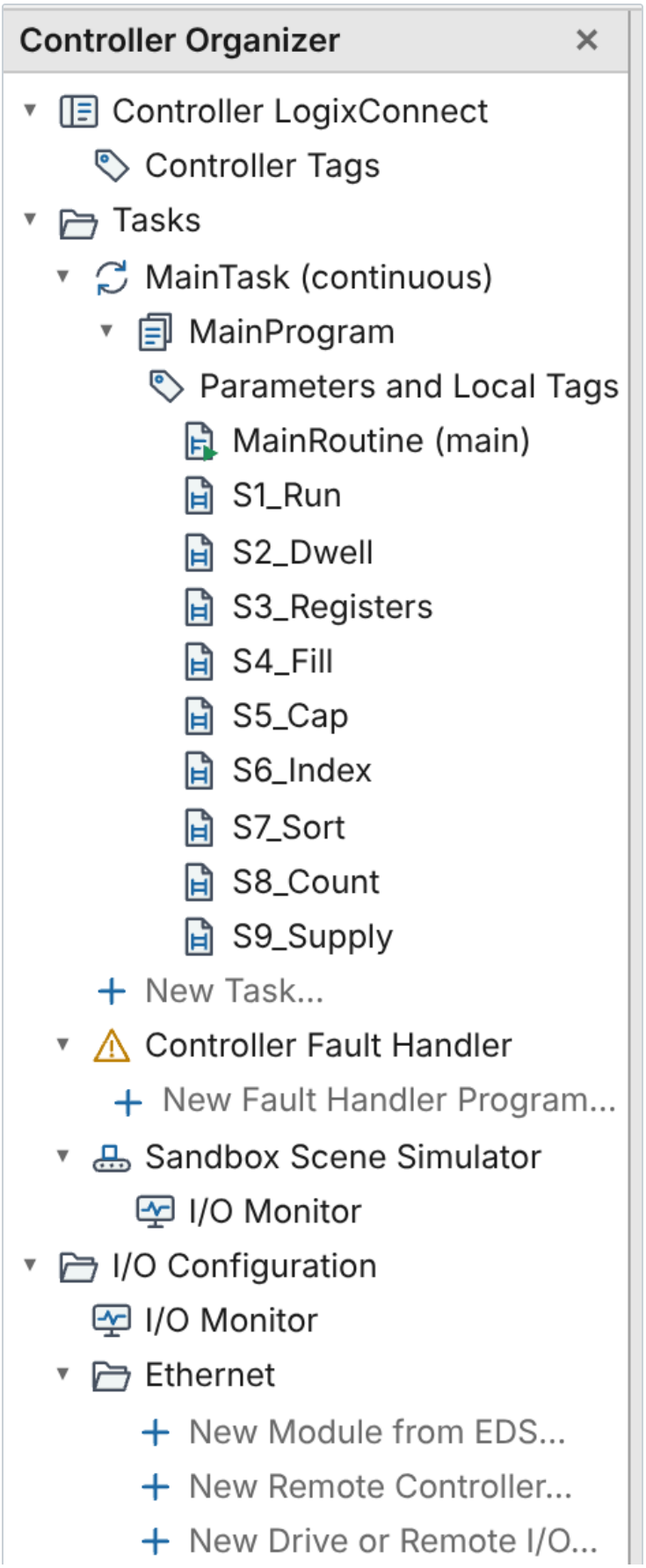
Figure 13. Help

- **File** — *New Project…, New Controller (clear tags and rungs)…, Open…, Save, Save As…, Export / Copy as Text…, Share Link…, Open Link…, Import L5X (Studio 5000)…, Export L5X (Studio 5000)…, Load Reference Solution: Complete Bottle Line (12 steps), Load Reference Solution.*
- **Edit** — *Undo, Redo, Cut, Copy, Paste, Paste Before, Find…, Find Next, Replace…, Go To…, Cross Reference…, Add Rung, Edit Rung, Add Rung Comment, Delete Rung Comment, Delete Rung, Move Rung Up, Move Rung Down.*
- **View** — *Controller Organizer, Output, Simulator, Quick Watch, Quick Watch: Floating Window, Quick Watch: Docked at the Bottom (next to Cross Reference), Cross Reference Page, Tag Descriptions in Ladder, Highlight Power Flow (not in Logix), Phone Layout (in-house preview, narrow screens only).*
- **Logic** — *New Program…, New Routine…, Verify Routine, Verify Controller, Accept Rung Text (Enter), Cancel Rung Text (Esc).*
- **Communications** — *Go Offline, Program Mode, Run Mode, Test Mode, Bridge & Tools, Start Bridge, Socket Bridge Settings…, Clear Faults, I/O Forcing: Enable All, I/O Forcing: Disable All, I/O Forcing: Remove All, Controller Properties….*
- **Tools** — *Inject Machine Fault…, Clear Machine Faults, Create Alias Tags for Wired I/O, Add-On Instruction Library…, New Module from EDS File…, New Generic EtherNet/IP Module…, New EtherNet/IP Device (drive, remote I/O)…, New Remote Controller (produced / consumed tags)…, Clear Tag Values (reset timers, counters, bits).*
- **Window** — *New Trend…, Reset Tab Order, Reset Pane Sizes, Maximize Document, Simulator Docked Above, Simulator as Document Tab, Open Bridge & Tools, Open MainRoutine, Open Controller Tags, Open I/O Monitor, Open Operator View, Open Cross Reference, Open Data Table, Quick Watch, Open Simulator.*
- **Help** — *Start Page, Guided Mode: Build the Bottle Line, Updates: What Is New (Version 98.4), Search Help and Project…, Glossary, Add-On Instructions, Online Edits, Structured Text, Trends, Bridge & Tools, Rung Text Syntax, Level Brief and Scoring, About.*
- **Theme** — *Logix Connect (default), Logix Designer Blue, Graphite (control room), Holiday splash: automatic (Halloween Oct 24–31), Holiday splash: Halloween always, Holiday splash: off, Preview the Halloween splash.*

The Controller Organizer

The tree on the left is the project. **Controller Tags** opens the tag editor. **Tasks → MainTask → MainProgram** holds the routines; the main routine is marked *(main)* and is the only one the controller calls by itself — every other routine runs only when a JSR in a running routine reaches it. Right-click a program for *New Routine…*, right-click a routine to rename it, set it as main or delete it. **Sandbox Scene Simulator**

and **I/O Monitor** open the machine and the live chassis. **I/O Configuration → Ethernet** is where EtherNet/IP devices (a vision camera from its EDS file, a drive, remote I/O, another controller's produced tags) are added. **Add-On Instructions**, **Trends**, **Data Types** and the **Data Table** are below.


Controller Organizer
Controller LogixConnect
Controller Tags
Tasks
MainTask (continuous)
MainProgram
Parameters and Local Tags
MainRoutine (main)
S1_Run
S2_Dwell
S3_Registers
S4_Fill
S5_Cap
S6_Index
S7_Sort
S8_Count
S9_Supply
New Task...
Controller Fault Handler
New Fault Handler Program...
Sandbox Scene Simulator
I/O Monitor
I/O Configuration
I/O Monitor
Ethernet
New Module from EDS...
New Remote Controller...
New Drive or Remote I/O...

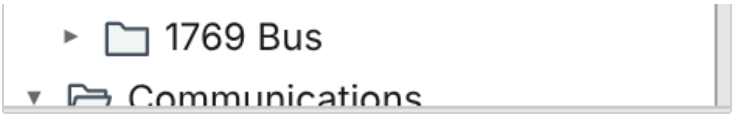


Figure 14. The Controller Organizer: Controller Tags, the task and program with its routines (MainRoutine is main), the simulator and I/O Monitor, the I/O Configuration tree, Communications, Add-On Instructions, Watch, Trends, Data Types and Help.

Document tabs

The **Start Page** is the built-in help (this book is its long form). A **routine** tab is the ladder editor. **Controller Tags** has Monitor and Edit views. The **Sandbox Scene Simulator** is the machine. **Bridge & Tools** is the link to real hardware. Tabs can be dragged into any order (Window → Reset Tab Order puts them back) and the simulator can be docked above the ladder (Window → Simulator Docked Above) so you can watch the machine and the rungs at once — the arrangement most people settle on.

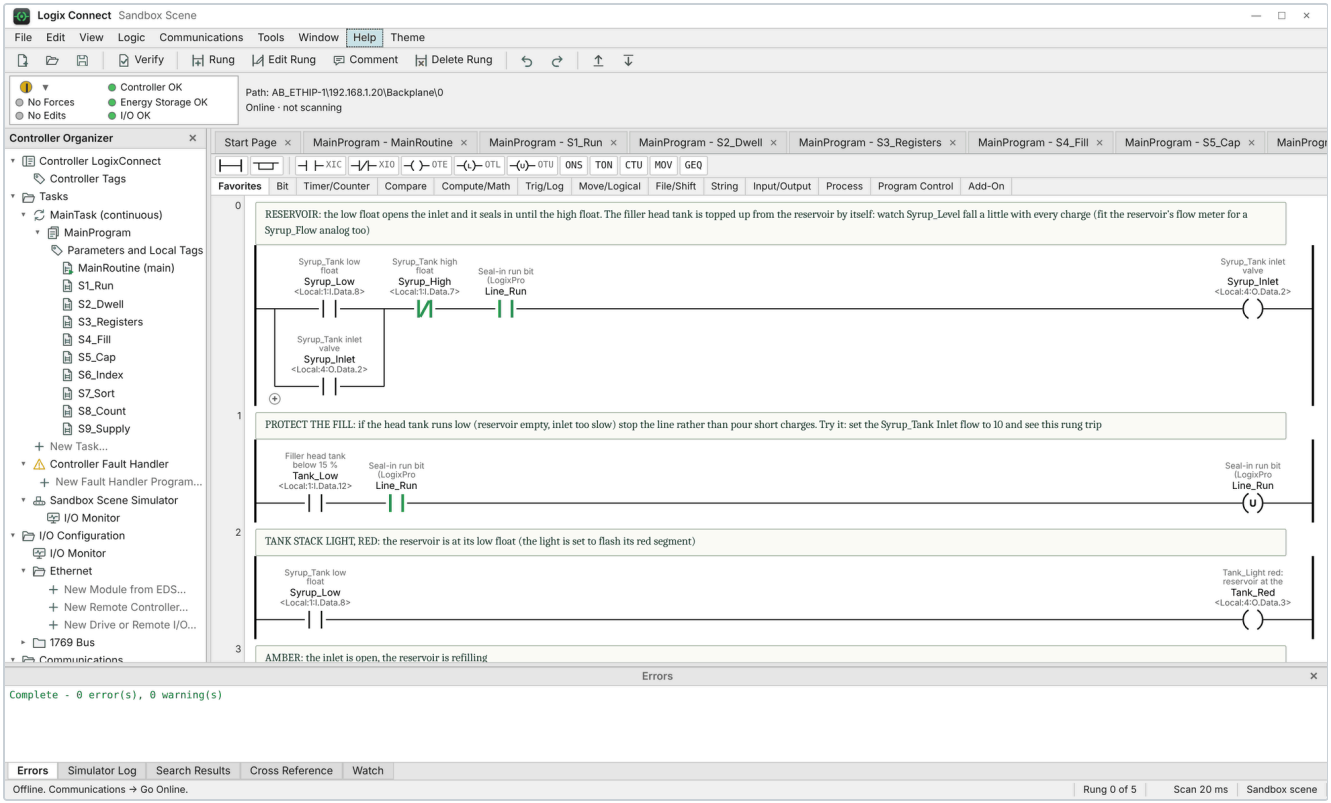


Figure 15. A routine tab with the simulator docked above it (Window → Simulator Docked Above): the machine and the rungs in one view.

The simulator toolbar

Figure 16. The simulator toolbar: mode toggle, Speed, Clear Parts, Reset Sim, Zoom, Floor, View, Labels, the three drawers, and the counters on the right.

Left to right: the Running/Program toggle; **Speed** (½× to 4×: slow the machine to watch a gate, speed it up to fill a box); **Clear Parts** (removes every part, zeroes the simulator's own counters, leaves the program's tags alone); **Reset Sim** (starts over: parts gone, every device back to its starting state — tank levels, cap chute, revolver drum, counters, timers — and every controller tag back to its initial value, in Program mode, with the scene and program untouched); **Zoom**; **Floor** (1600 × 520 up to 4000 × 800 px; the floor grows headroom by itself when a tall device sits near its top edge, and fills the window to the right and below); **View** (Top-down for laying out, Side for watching fills and caps, 3D to walk the line — desktop only); **Labels** (name tags shown, or hidden until you hover). **Palette**, **Properties** and **I/O** open the three side drawers; **?** opens the help for the selected part.

On the right are the counters: **Emitted** (every part the feeders have made), then three counters that read *program tags* — `Large_Bottle_Count`, `Small_Bottle_Count` and `Scrap_Bottle_Count` by default; click one to point it at any DINT tag, and a tag that does not exist shows a dash — then **Total processed** (parts that reached a remover, a bin or the shredder), with **lost** in red only when parts have fallen off the line with nothing to land on, and the simulated clock.

The bottom panes

Errors is where Verify reports (*Complete - 0 error(s), 0 warning(s)* is what you want to see). The **Simulator Log** is the machine's own record: every part made, filled, capped, scrapped, diverted, lost or spilled, with the device that did it and the simulated time — read it before you read your ladder when something goes wrong. **Search Results**, **Cross Reference** (every place a tag is used, click to go there) and **Watch** (Quick Watch: a short list of tags with live values) complete the set.

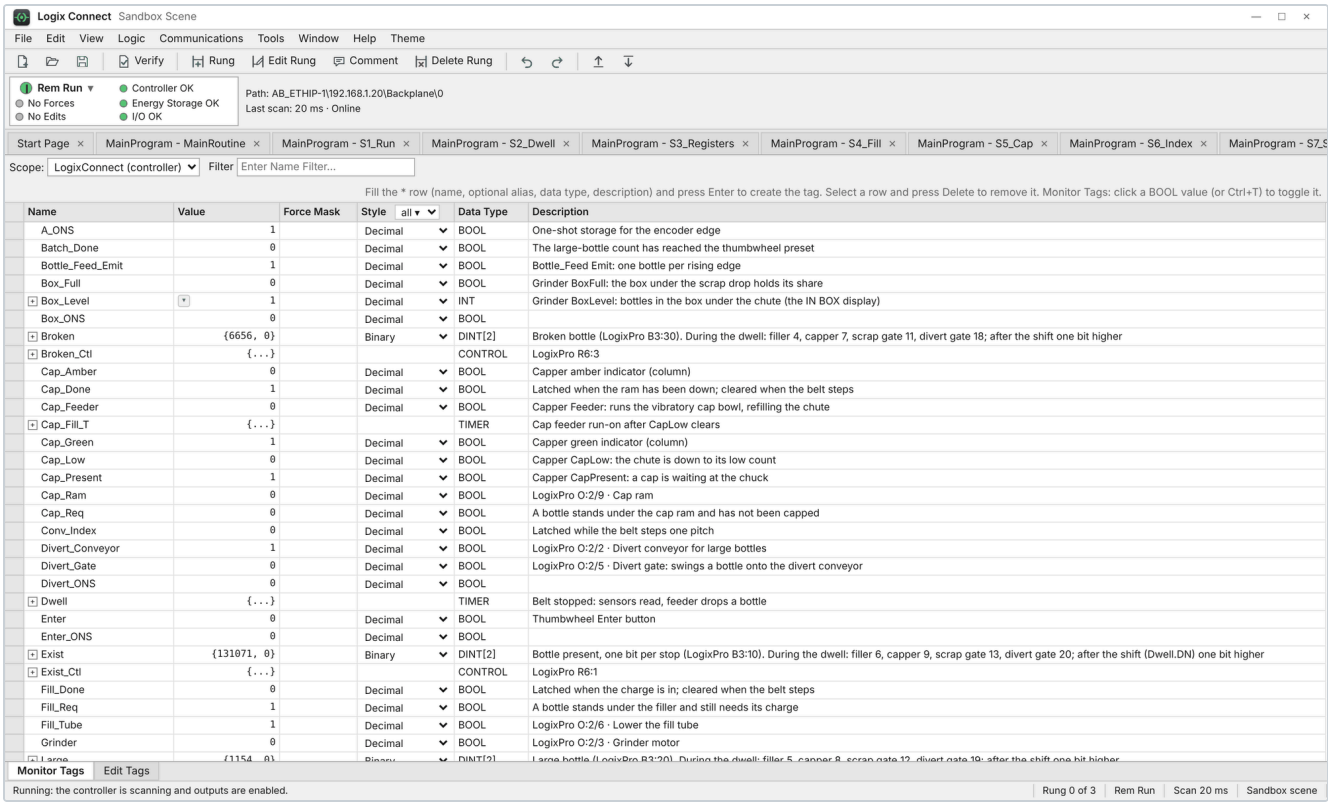


Figure 17. Controller Tags in the Monitor view while the line runs, with a DINT expanded to its bits and timers showing their accumulators.

On a phone

The phone layout has a bottom bar (Start, Ladder, Tags, Sim, Output, Run) and the simulator's own bar (Add Device, Select Several, I/O, Fit, Scene). The **Scene** sheet carries what the small screen has no toolbar room for: View (Top-down / Side), Floor, Name tags, Background, Clear Parts and Reset Sim. 3D is a desktop view; a scene saved in 3D opens Top-down on a phone.

3 Projects: new, save, open, export

File → New Project… offers four starts: an **empty sandbox** (a bare floor and an empty routine, where Part IV begins), the graded **levels**, a **sample** scene, and the **Sandbox: LogixPro-style Bottle Line** — the finished machine of Part IV with its nine-step program already written. The last one is your answer key: build from the empty floor, and when a lesson will not behave, open the finished line in a second tab and compare.

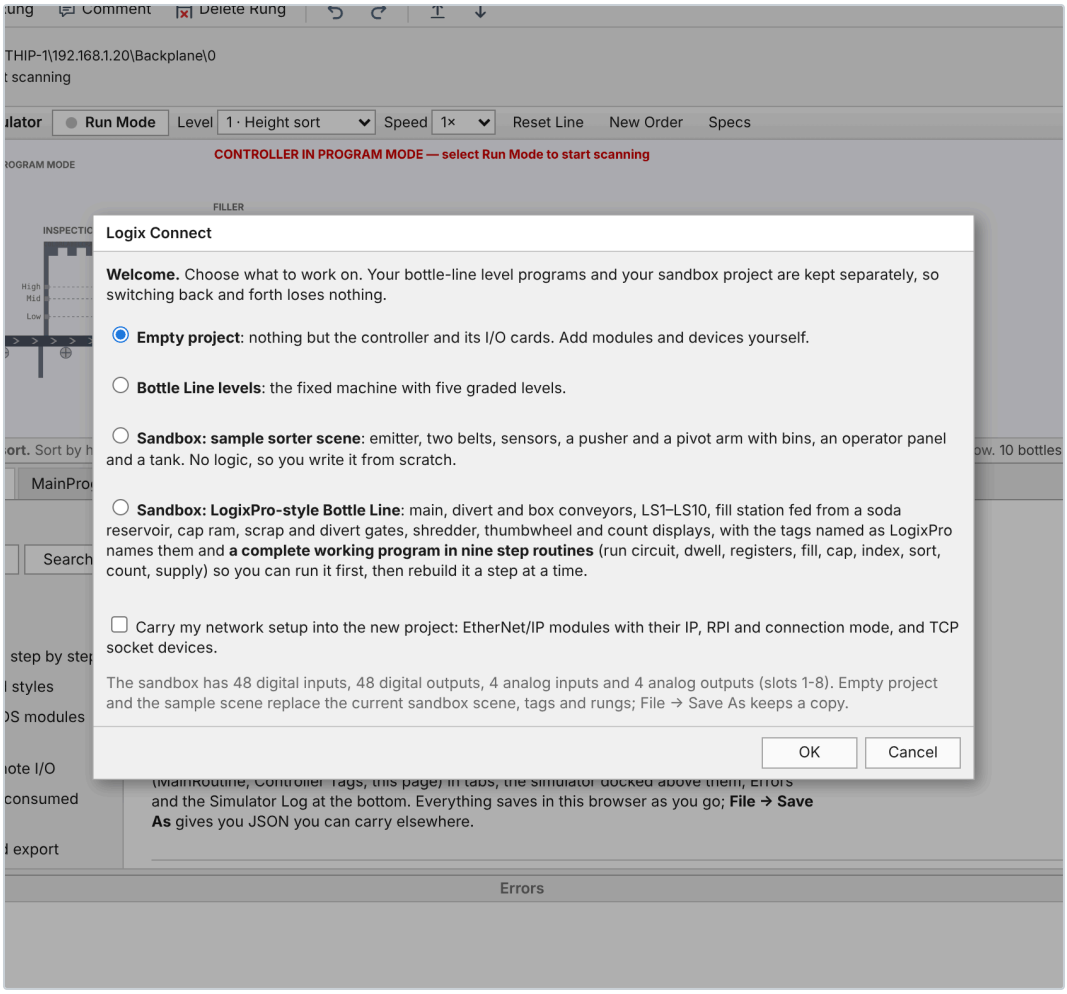


Figure 18. File → New Project...: the empty sandbox, the levels, the sample sorter and the LogixPro-style bottle line with its finished program.

Save writes the whole project — scene, tags, routines, panel layout, trends — as one JSON file; **Open…** reads it back. **Export / Copy as Text…** gives the program as rung text you can paste anywhere, which is also how this book's listings were made. **Export L5X** writes a Studio 5000 import file with your tags and routines; **Import L5X** reads one, including Add-On Instructions. **New Controller** clears tags and rungs but keeps the scene — the fastest way to start Part IV's programming lessons on a machine you have already built.

The page also keeps your work between visits by itself (the browser's storage); Save is for a copy you can carry, send or keep.

> **WATCH FOR**
>
> The two *Load Reference Solution* entries replace your program. *Complete Bottle Line (9 steps)* installs Part IV's nine routines on the LogixPro-style scene; the plain one installs the older single-routine answer. Save first if you have rungs you want to keep.

Part II — Ladder logic from zero

4 How a controller thinks: the scan and the rung

A relay panel is wired; a controller is scanned. Every scan — here every 20 ms — the controller does three things in order: it copies the state of every input point into memory, it solves every rung of the running routines from the top down, and it writes the result of every output rung to the output points. Nothing happens between scans. A sensor that flickers for 2 ms between two scans is never seen; an output written by rung 3 and overwritten by rung 9 is only ever what rung 9 said. Hold on to those two facts and most ladder mysteries dissolve.

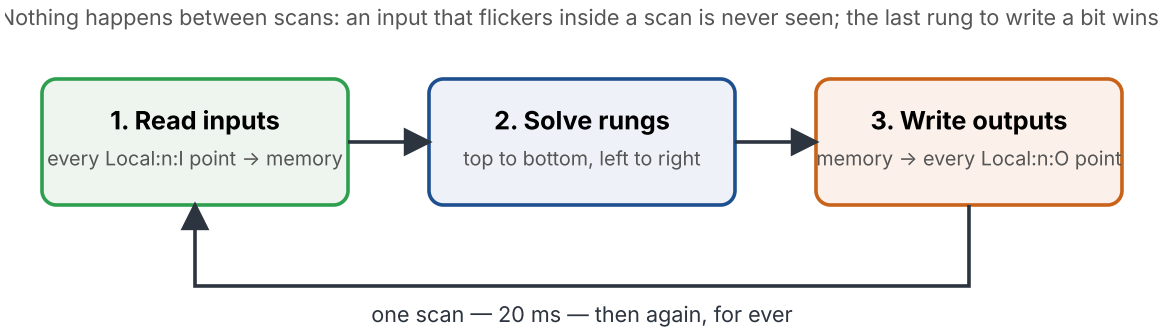


Figure 19. The scan. The controller never "watches" an input; it looks once per scan, decides everything, then writes everything.

Reading a rung

A rung is a path from the left power rail to the right. Instructions on the left (*inputs*, or conditions) either pass power or block it; the instruction on the right (*output*) does something when power reaches it. In Logix Connect you write the rung as text, exactly the way Studio 5000 copies a rung to the clipboard:

```
XIC(Start_PB)XIO(Stop_PB)OTE(Motor);
```

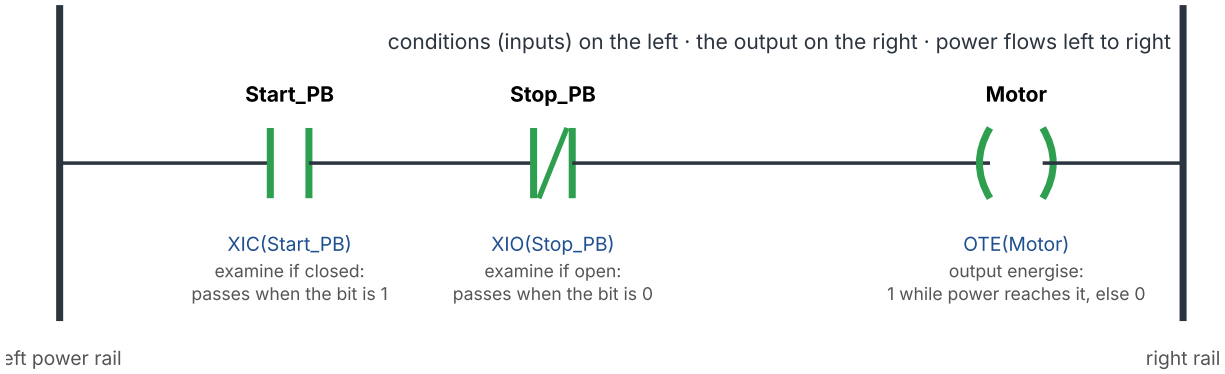


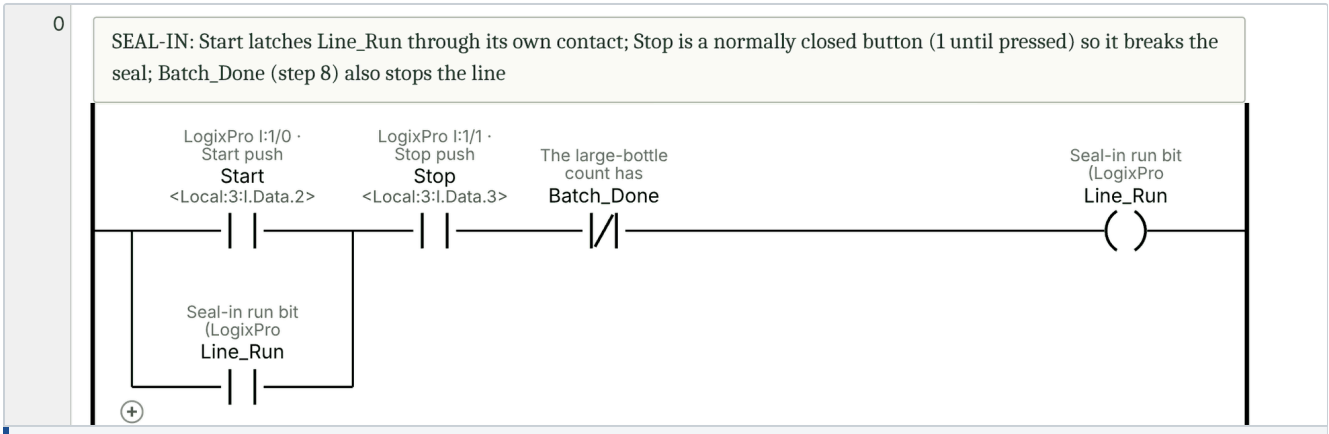
Figure 20. Anatomy of a rung, and the text you type for it: `XIC(Start_PB)XIO(Stop_PB)OTE(Motor);`

Read it left to right: *examine Start_PB if closed* (true when the tag is 1), *examine Stop_PB if open* (true when the tag is 0), *output energise Motor*. Instructions written one after another are in series: all must be true. A branch — instructions that can each complete the path — is written in square brackets with a comma between the legs:

The classic seal-in: Start or Motor itself, and not Stop, keeps Motor on

```
[XIC(Start_PB),XIC(Motor)]XIO(Stop_PB)OTE(Motor);
```

The editor draws it like this (this is the real one from Part IV's step 1, with Stop normally closed and a batch-done condition added; the tag descriptions are printed above the contacts):




```
[XIC(Start),XIC(Line_Run)]XIC(Stop)XIO(Batch_Done)OTE(Line_Run);
```

That rung is the most important one in the book. Press Start: the left leg is true, Motor comes on. Release Start: the right leg, `XIC(Motor)`, is now true because Motor is on, so the rung stays true — the output seals itself in. Press Stop: `XIO(Stop_PB)` goes false, Motor drops, and both legs are now false. Every *Run* bit on every machine you will ever program is this rung.

Entering rungs

Open a routine. Click ⊢ **Rung** (or press **Insert**): a text field opens for the new rung. Type the rung and press **Enter**; **Esc** cancels. Click a rung and choose **Edit Rung** (or double-click) to change it. **Comment** puts a note above the rung — use it; the lessons in Part IV comment every rung and you should too. **Delete Rung**, **Move Rung Up / Down** and the arrow buttons reorder. The semicolon at the end is the rung terminator; the editor adds it if you forget.

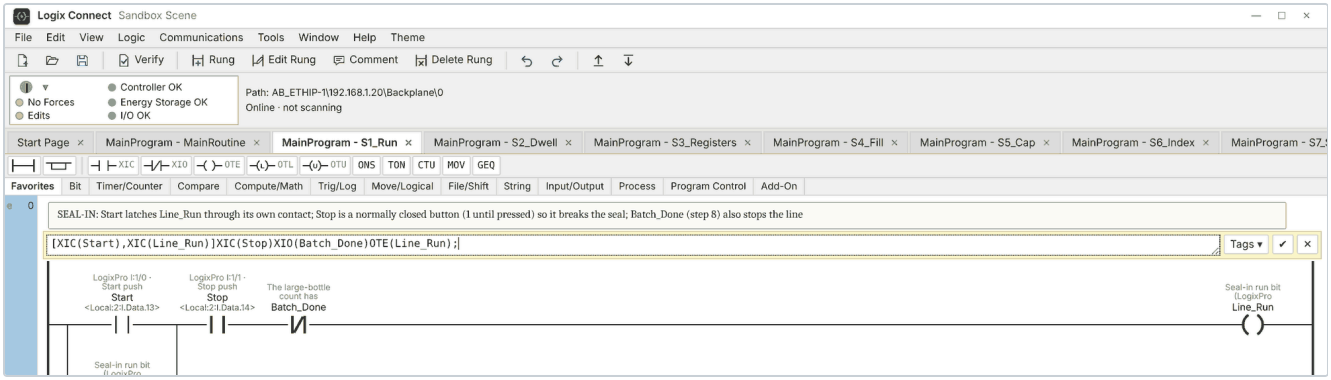


Figure 21. Editing a rung: the text field opens under the rung with Tags ▾ (browse tags for the operand at the cursor), accept and cancel. The rung draws itself as you type.

Logic → Verify Routine (or just Verify on the toolbar) checks every rung: unknown tags, wrong parameter counts, a tag used as the wrong type. Errors appear in the Errors pane with the rung number. A routine that does not verify does not run. Tags you name in a rung before creating them are reported as unknown — create them (chapter 5) and verify again.

Rung text: the syntax on one page

Keep this open while you type. The editor accepts exactly what Studio 5000 puts on the clipboard when you copy a rung, so everything here transfers.

You want	You type	Notes
an instruction	`XIC(Start)`	mnemonic in capitals, parameters in brackets, no spaces needed
instructions in series (AND)	`XIC(A)XIC(B)OTE(C);`	one after another; all must be true
a branch (OR)	`[XIC(A),XIC(B)]OTE(C);`	square brackets around the legs, a comma between legs
a leg with several instructions	`[XIC(A)XIC(B),XIC(C)]OTE(D);`	A and B, or C
a branch inside a branch	`[XIC(A)` `[XIC(B),XIC(C)],XIC(D)]OTE(E);`	nest as deep as you need; count your brackets

You want	You type	Notes
two outputs on one rung	`XIC(A)[OTE(B),OTE(C)];`	an output branch: both energised together
a bit of a word	`XIC(Word.3)` `XIC(Exist[0].12)`	dot, then the bit number
a member of a structure	`XIC(Dwell.DN)` `GEQ(Dwell.ACC,500)`	timers: .PRE .ACC .EN .TT .DN; counters: .CU .DN; control: .DN .UL .POS
an array element	`MOV(Vals[2],Out)`	index in square brackets; a tag used as the index is fine
a number	`GEQ(Count,20)` `TON(T1,1500,0)`	plain digits; a decimal point makes it REAL
a timer / counter	`TON(Dwell,1000,0)` `CTU(Boxes,10,0)`	tag, preset, accumulator (start value)
a shift register	`BSL(Exist[0],Exist_Ctl,LS1,48)`	array, CONTROL tag, source bit, length
the end of the rung	`;`	the editor adds it if you forget

THE MISTAKES EVERYONE MAKES ONCE

- **Unbalanced brackets.** Every `[` needs a `]`, every `(` a `)`. The editor marks the rung with an *e* and the Errors pane says where.
- **A space in a tag name.** `Start PB` is two things; `Start_PB` is one.
- **The same tag on two OTEs.** Only the last one in the scan counts. Use a branch on one rung, or OTL/OTU.
- **An ONS without its own storage bit.** `ONS(X_ONS)` needs a BOOL used nowhere else; two one-shots sharing a bit both fail.
- **Reading a timer bit of a timer that never resets.** A TON resets when its rung goes false; if the rung is always true, .DN comes on once and stays.
- **Forgetting the JSR.** A routine nobody calls does nothing; Verify warns you.

Online

Switch to Run mode and the ladder is *online*: instructions that are true light green, timers and counters show their accumulators, and a change you make to a rung is accepted into the running controller on Enter — like Studio 5000's online edits, with the edited rung marked until you accept it. Values (tag values, timer accumulators) carry over an edit; only Reset Sim or Tools → Clear Tag Values zero them.

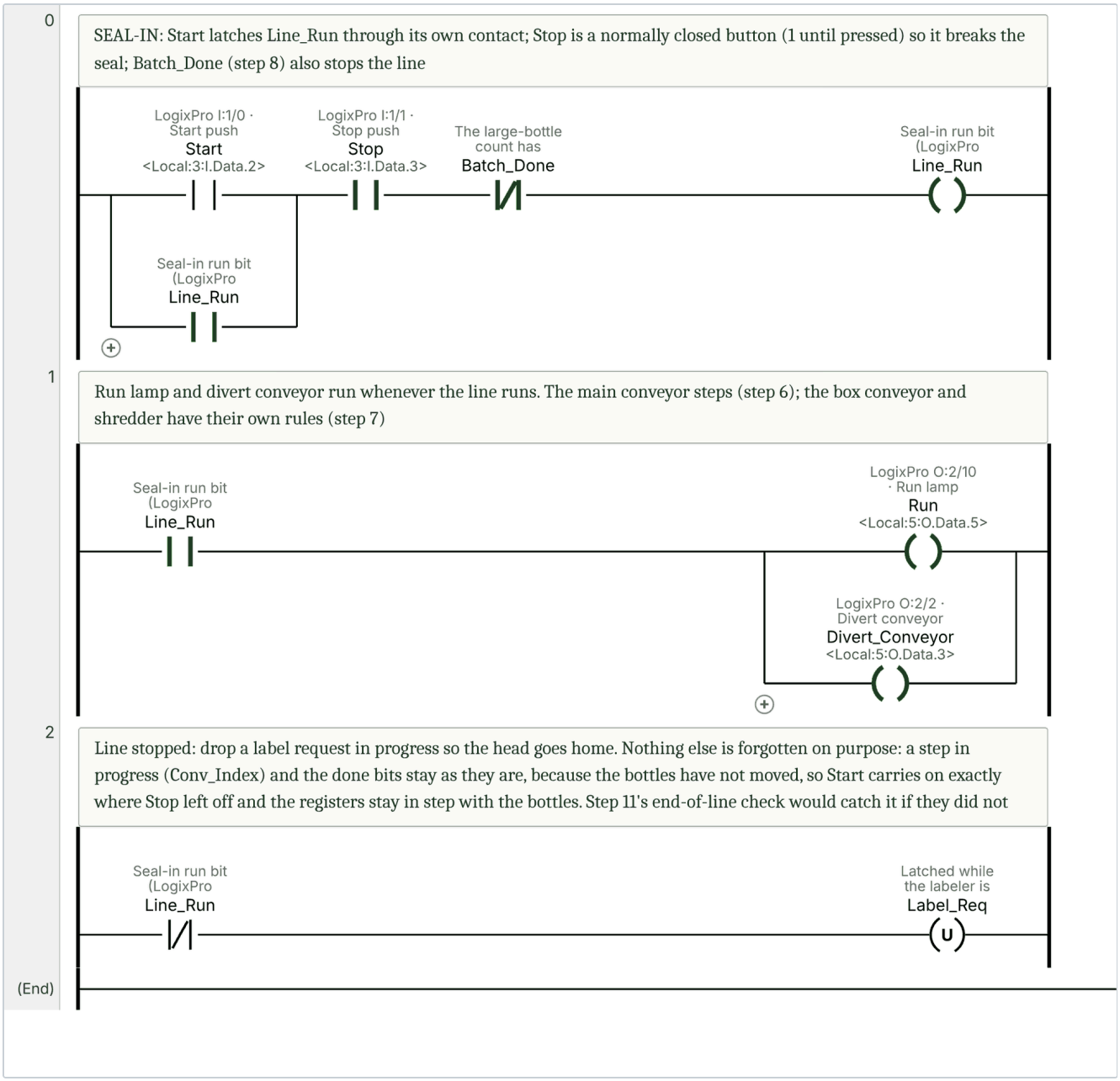


Figure 22. The same routine online, in Run mode: true instructions and energised coils light green; the seal-in is holding Line_Run.

TRY IT

In an empty sandbox place a *Push Button*, a second Push Button with *NC* ticked (its signal is then 1 until pressed, like a real stop button) and an *Indicator Light*. Name them Start_PB, Stop_PB and Lamp. Write the seal-in rung with `Lamp_On` as the output (the alias the light gets by itself, see chapter 5), go to Run mode and press the buttons. Then swap `XIO(Stop_PB_OK)` for `XIC(Stop_PB_OK)` and see why a stop button is wired normally closed.

5 Tags, data types and I/O addresses

A tag is a named piece of controller memory with a type. Logix Connect uses the Logix types you will meet in Studio 5000:

Type	What it holds	Where you use it
`BOOL`	one bit, 0 or 1	every input and output, every latch, every one-shot storage bit
`SINT / INT / DINT`	an 8 / 16 / 32-bit integer	counts, analog values (INT from the modules), shift registers (DINT, one bit per stop)
`REAL`	a floating-point number	scaled analog values, calculations
`TIMER`	a structure: .PRE (preset, ms), .ACC (accumulated, ms), .EN, .TT (timing), .DN (done)	TON / TOF / RTO
`COUNTER`	.PRE, .ACC, .CU, .CD, .DN, .OV, .UN	CTU / CTD
`CONTROL`	.LEN, .POS, .EN, .DN, .EM, .ER, .UL	BSL / BSR / FFL / FFU / SQO
arrays	`Exist[0..1]`: a DINT with Dim 2	a 64-bit shift register, addressed `Exist[0].12` for bit 12

Open **Controller Tags → Edit Tags**. Type a name in the `*` row and press Enter; pick the type; add a description. A description is free and is shown in the ladder when View → Tag Descriptions in Ladder is on — the lessons rely on them. A tag's **style** (Decimal, Binary, Hex) is only how it is displayed; the shift registers in Part IV are Binary so you can watch the bits walk.

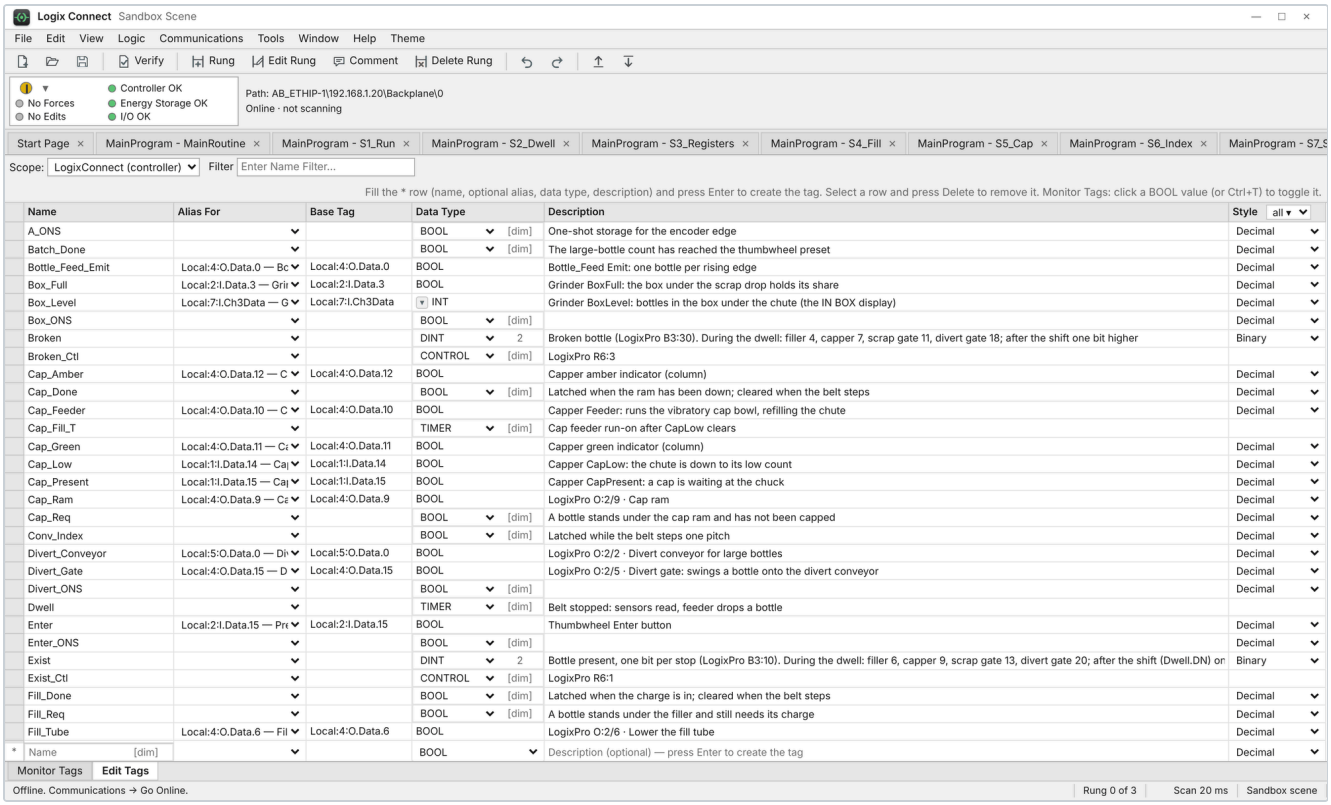


Figure 23. Controller Tags → Edit Tags: name, alias target, data type, style and description. The bottle line's LogixPro-style aliases are at the top.

Base tags: the I/O points

The chassis points exist as tags already: `Local:1:I.Data.0`, `Local:4:O.Data.3`, `Local:7:I.Ch1Data`. You can write them straight into a rung, and the I/O Monitor shows them live, but a program full of slot numbers is unreadable. That is what aliases are for.

Alias tags

An alias is a tag that *is* another tag under a better name: `Start_PB` aliased to `Local:1:I.Data.0`. Reading or writing the alias reads or writes the point. In the tag editor, choose *Alias* and pick the base in *Alias For*.

Logix Connect has a house rule that saves most of that work: **a tag named `Device_Signal` aliases itself.** Place a push button called `Start_PB`; its signal is `OK`; create a tag called `Start_PB_OK` and it is aliased to the button's point automatically. The Properties drawer of every part lists its signals with their addresses and a *tag Device_Signal* link that creates the alias in one click; the Scene sheet's *Alias all I/O* and **Tools → Create Alias Tags for Wired I/O** do it for every signal on the floor at once. Rename a device and its alias tags follow (`Syrup_Tank` to `Milk` turns `Syrup_Inlet` into `Milk_Inlet` and rewrites every rung).

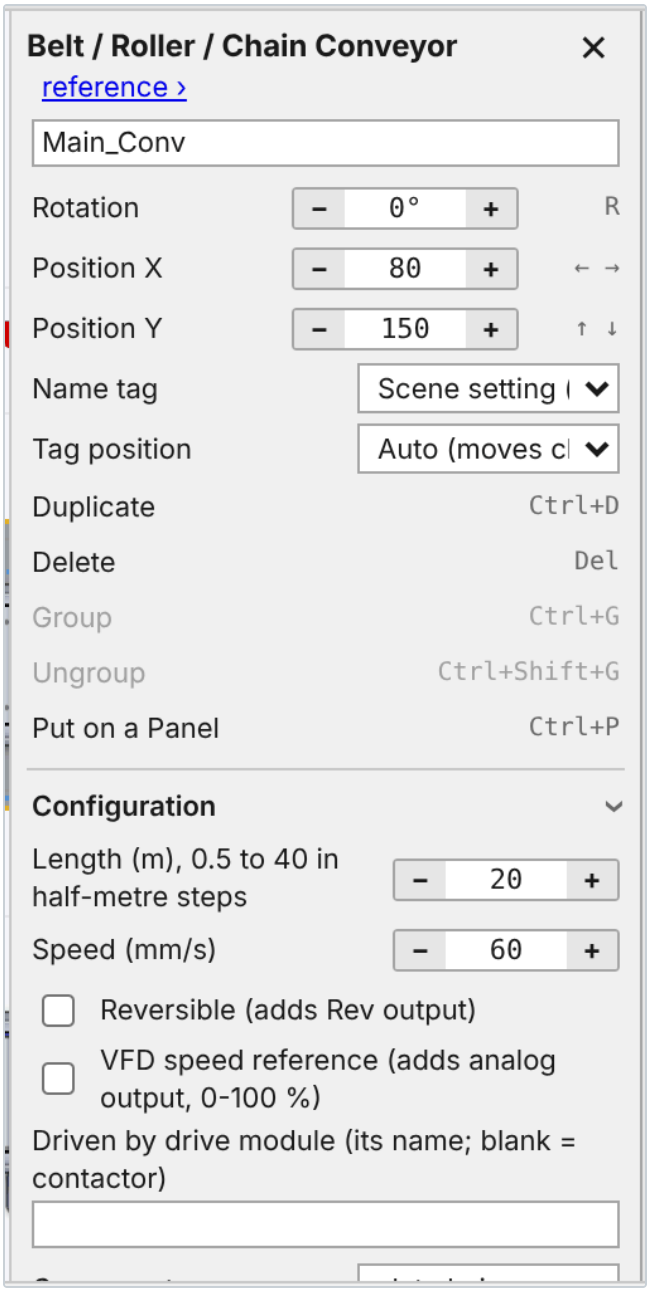


Figure 24. The Properties drawer of LS1: its settings and, under Signals and Addresses, the point each signal has and the tag Device_Signal link that creates the alias.

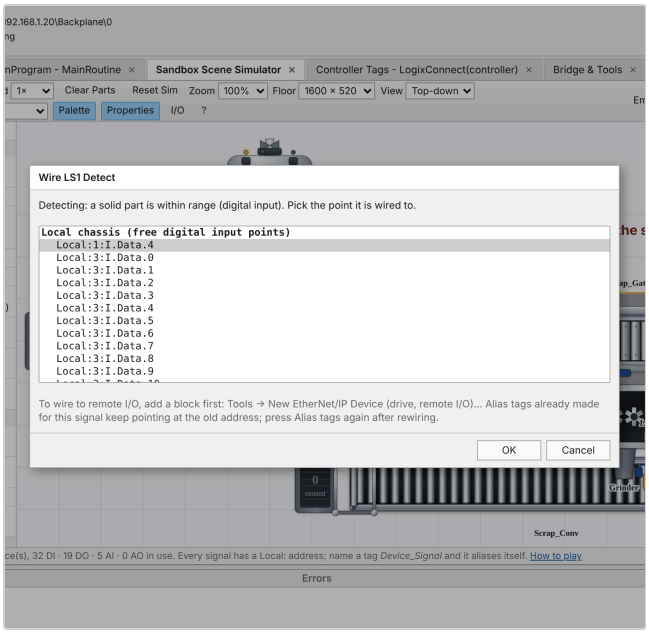


Figure 25. Re-wiring a signal to another point, or binding it straight to a tag of your own.

Part IV uses shorter, LogixPro-style names for the bottle line (`LS1` , `Cap_Ram` , `Syrup_Inlet`), made as explicit aliases; the full list is in Appendix A. Either style is fine; the rule is that a rung should read like the machine, not like the chassis.

Bits of a word

Any integer tag can be used a bit at a time: `Exist[0].12` is bit 12 of the first DINT of the array `Exist`. The Controller Tags browser expands a DINT into its 32 bits and shades the ones the program uses; the Data Table (Window → Open Data Table) shows whole words in the radix of your choice, the way RSLogix's data files did.

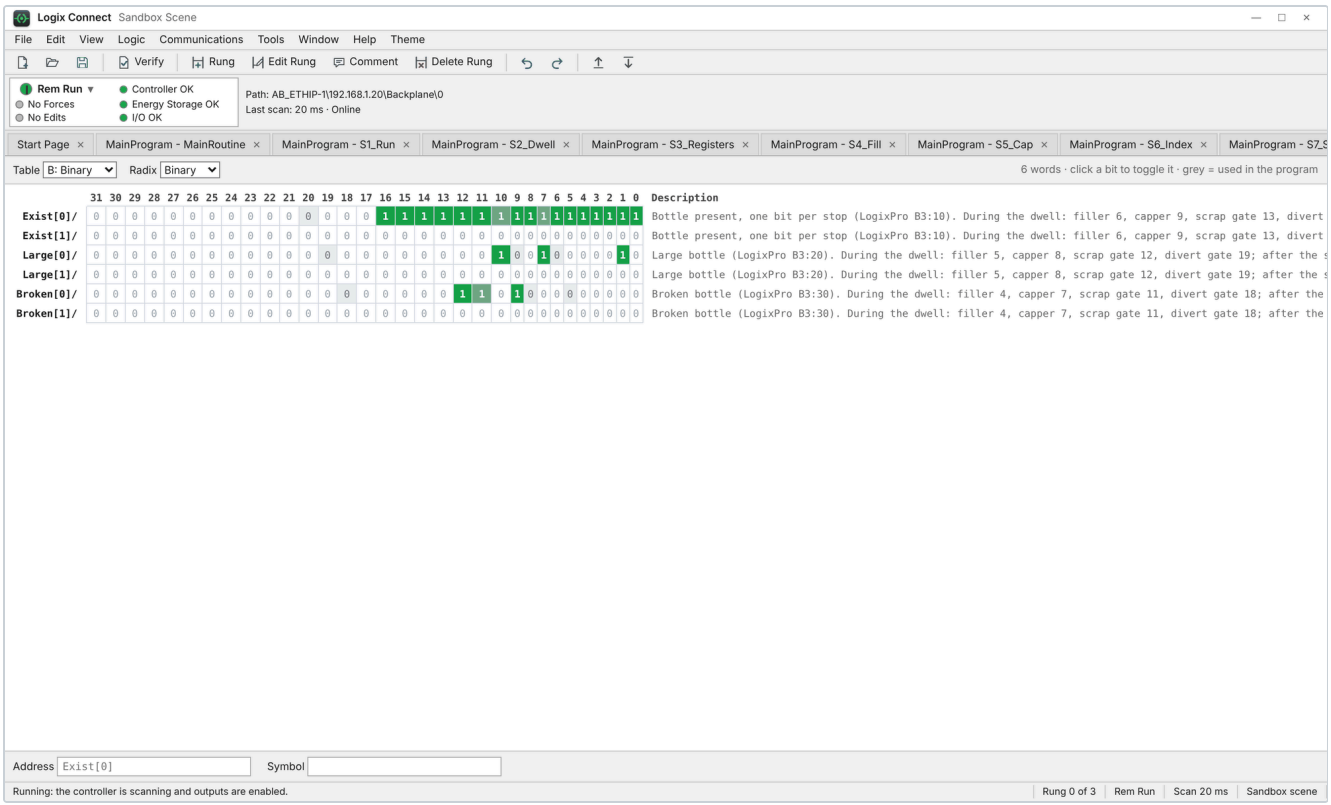


Figure 26. The Data Table (Window → Open Data Table): whole words in the radix of your choice, the way RSLogix data files looked. The shift registers are easiest to read here.

> **WATCH FOR**
>
> Names: letters, digits and underscores, not starting with a digit, up to 40 characters, and no two tags the same ignoring case. `Run` and `run` are the same tag. The editor tidies a device name into that shape when you type one with spaces.

6 Bit instructions

Everything on a machine is, at the bottom, a bit: a sensor sees or does not, a valve is open or shut. These six instructions are the whole language of bits.

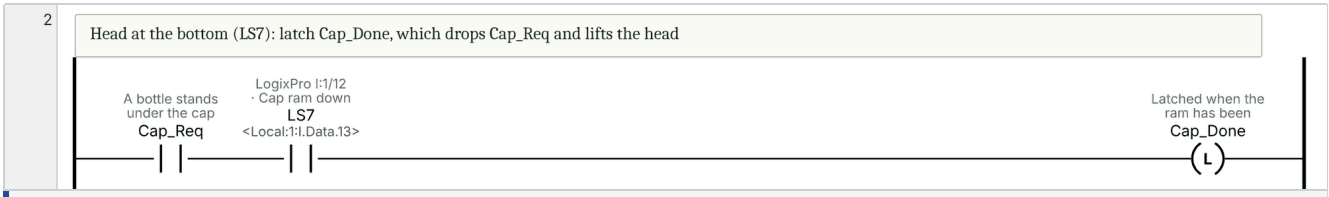
Instruction	Reads	Does
`XIC(tag)`	Examine If Closed	passes power when the bit is 1
`XIO(tag)`	Examine If Open	passes power when the bit is 0
`OTE(tag)`	Output Energise	writes 1 when the rung is true, 0 when it is false — every scan
`OTL(tag)`	Output Latch	writes 1 when the rung is true; leaves it alone when false
`OTU(tag)`	Output Unlatch	writes 0 when the rung is true; leaves it alone when false
`ONS(storage)`	One Shot	passes power for exactly one scan after the rung ahead of it becomes true; the storage bit remembers the previous scan

OTE: the output that follows the rung

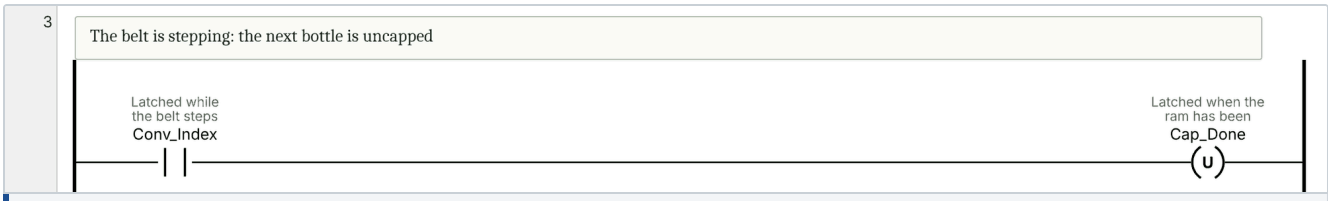
OTE is a mirror. The bit is whatever the rung is, scan after scan. Two rules follow. First, write each bit with OTE in one place only; the last OTE wins and the earlier one is wasted. Second, an OTE rung with a seal-in branch is how you make a bit remember (chapter 4).

OTL / OTU: memory without a seal-in

A latch sets the bit and walks away; nothing turns it off except an unlatch. Use the pair when the set and the reset belong to different events in different places — a request raised by one routine and cleared by another:




```
XIC(Cap_Req)XIC(LS7)OTL(Cap_Done);
```



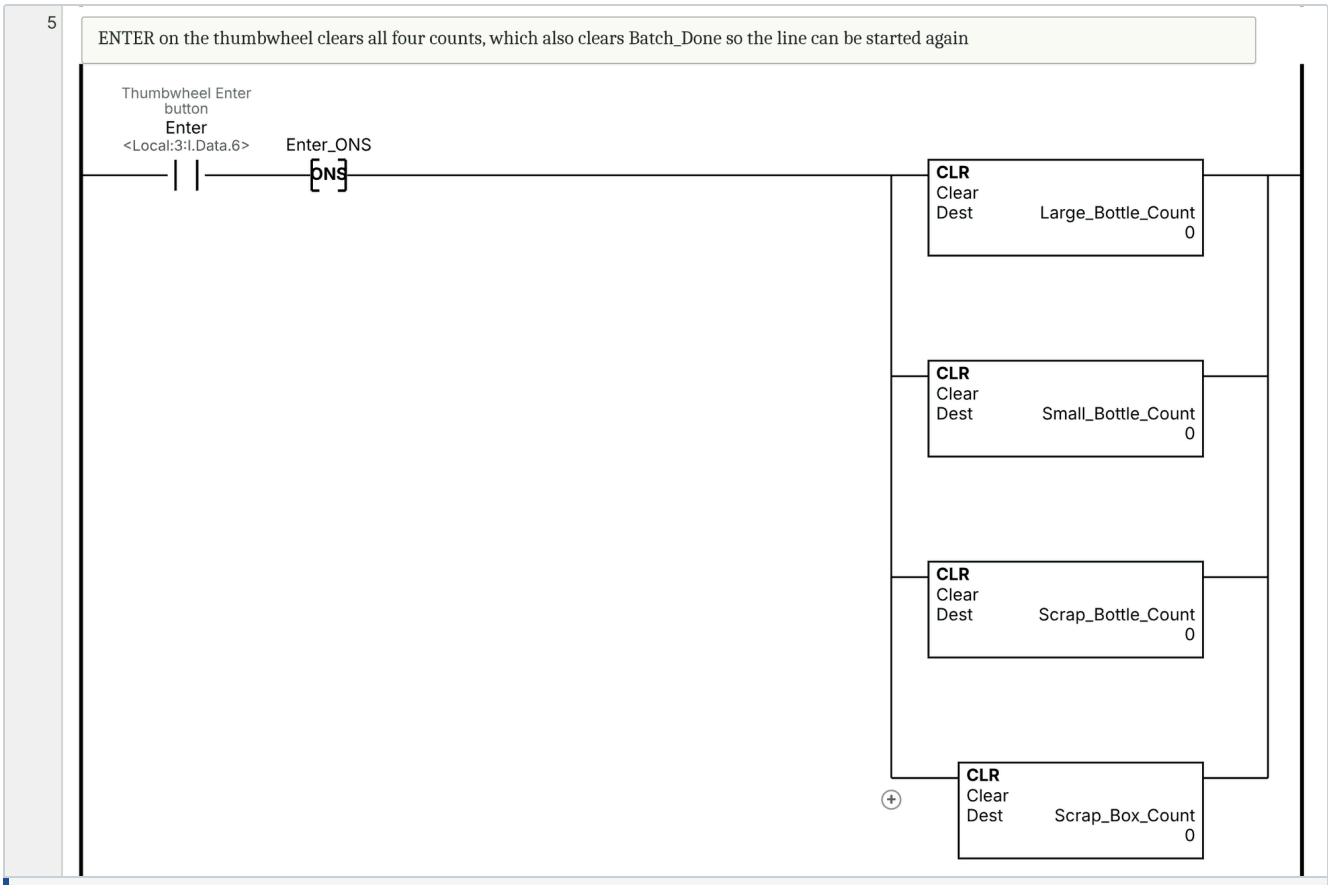

```
XIC(Conv_Index)OTU(Cap_Done);
```

(Part IV, step 5: the chuck reached the bottom, so this bottle has its cap; and when the belt steps, the next bottle is a new bottle.)

Latches survive a power cycle in a real controller and here survive everything short of Reset Sim, which is exactly why they need an unlatch you can find. Put the OTU right after the OTL in the listing when you can, and comment both.

ONS: once, not while

A push button held down is true for fifty scans. If the rung that counts presses were `XIC(PB)ADD(Count,1,Count)`, one press would add fifty. ONS lets the rung through on the first scan only:




```
XIC(Enter)ONS(Enter_ONS)
[CLR(Large_Bottle_Count),CLR(Small_Bottle_Count),CLR(Scrap_Bottle_Count),CLR(Sc
rap_Box_Count)];
```

(Part IV, step 8: one press of Enter clears the counts once, however long it is held.)

The storage bit (`Enter_ONS`) is a BOOL you create and never use anywhere else. OSR and OSF are the same idea as output instructions with their own storage and output bits.

TRY IT

With the button-and-lamp sandbox from chapter 4, add a DINT `Presses` and a BOOL `PB_ONS`. Write `XIC(Start_PB_OK)ONS(PB_ONS)ADD(Presses,1,Presses);` and watch Presses in Quick Watch while you press. Then delete the ONS and press once.

7 Timers and counters

TON, TOF, RTO

A timer tag has a preset (`.PRE`, milliseconds), an accumulator (`.ACC`) and three bits: `.EN` (the rung is true), `.TT` (timing: true while the accumulator is running toward the preset) and `.DN` (done). `TON(Dwell,1000,0)` means: timer `Dwell`, preset 1000 ms, accumulator starts at 0.

Instruction	Behaviour
`TON`	On-delay. Rung true: ACC counts up; at PRE, DN turns on. Rung false: ACC and DN reset. The ordinary "wait a second" timer.
`TOF`	Off-delay. DN turns on the moment the rung is true and stays on for PRE after the rung goes false. A fan that runs on after the heater stops.
`RTO`	Retentive. Like TON but the accumulator holds when the rung goes false and resumes when it comes back; only RES clears it. Run-hour meters, cumulative faults.

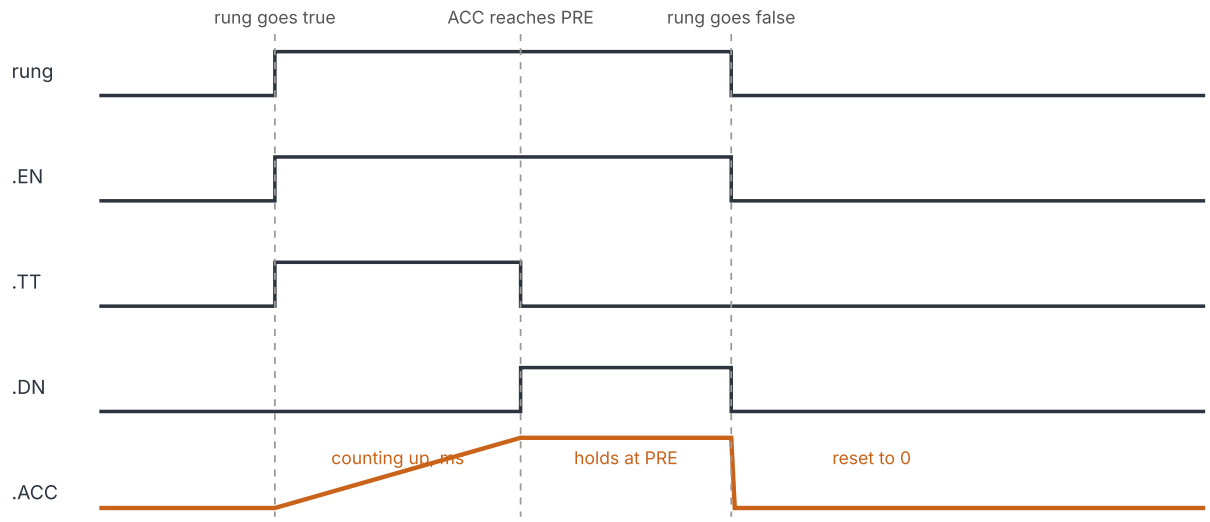
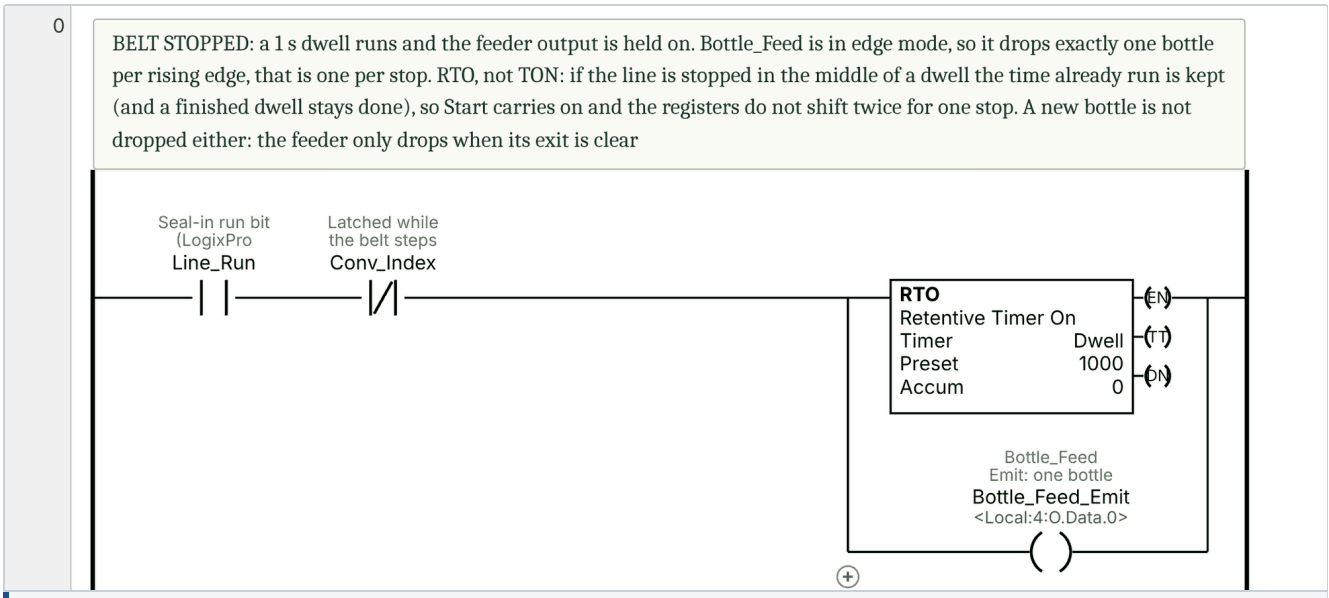


Figure 27. TON timing. .EN follows the rung; .TT is on only while the accumulator is climbing; .DN comes on at the preset and stays on until the rung drops. The dwell rung in Part IV reads .DN to say "the stop is over" and .TT to say "the bottles are standing still".

The timer in Part IV's step 2 is the heartbeat of the whole line:




```
XIC(Line_Run)XIO(Conv_Index)[RTO(Dwell,1000,0),OTE(Bottle_Feed_Emit)];
```

The rung is true whenever the line runs and the belt is not stepping, so the timer runs while the belt stands still; when `Dwell.DN` comes on, step 3 shifts the registers and step 6 steps the belt, `Conv_Index` goes true, the rung goes false and the timer resets for the next stop. The feeder's `Emit` is on the same rung: it fires on the rising edge, once per stop.

A flasher is two timers chasing each other, or one timer that resets itself:

resets itself the scan after it finishes: DN is true for one scan every 500 ms

```
XIO(Flash.DN)TON(Flash,500,0);
```

and that one scan toggles the lamp — though a latch toggled on one branch and unlatched on the next needs the ONS, or both fire

```
XIC(Flash.DN)ONS(F_ONS)[XIO(Lamp)OTL(Lamp),XIC(Lamp)OTU(Lamp)];
```

(An easier flasher: `XIO(T_Off.DN)TON(T_On,500,0); XIC(T_On.DN)TON(T_Off,500,0); XIC(T_On.TT)OTE(Lamp);` — on while T_On times, off while T_Off times. Stack lights in the sandbox also have a *flash* setting that does this for you.)

CTU, CTD, RES

A counter counts rung transitions, false to true, into `.ACC`; `.DN` comes on at `.PRE`. `CTU(Boxes,10,0)` counts up; `CTD` counts down; `RES(Boxes)` zeroes the accumulator (RES also resets timers). Because a counter has its own edge detection you do not need an ONS in front of it — but you do need the rung to go false between counts, which is why a counter fed by a sensor counts parts and a counter fed by a held button counts once.

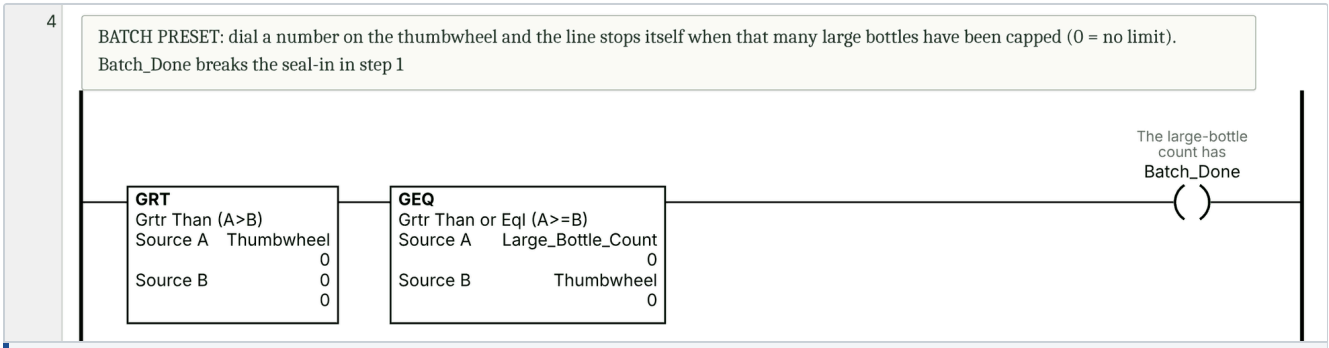
Part IV keeps its counts with ADD rather than CTU, for a reason worth knowing: the four displays show plain DINTs, the thumbwheel compares against a plain DINT, and a CTU's accumulator is just a DINT inside a structure. Both are right; ADD keeps the tag list flat.

> **TRY IT**
>
> Build a belt with a photoeye and a feeder set to automatic. Count the eye with `XIC(PE_1_Detect)CTU(Parts,5,0);` and light a lamp on `Parts.DN`; reset it with a button through RES. Then replace CTU with `ONS(P_ONS)ADD(Count,1,Count)` and show Count on a Digital Display.

8 Compare, move and math

Instruction	True when / does
`EQU(a,b)` `NEQ(a,b)`	a = b / a ≠ b
`GRT(a,b)` `GEQ(a,b)`	a > b / a ≥ b
`LES(a,b)` `LEQ(a,b)`	a < b / a ≤ b
`LIM(low,test,high)`	low ≤ test ≤ high (with low > high the window wraps, as in Logix)
`MEQ(source,mask,compare)`	the bits of source selected by mask equal compare
`MOV(source,dest)`	copies a value; `CLR(dest)` writes zero
`ADD SUB MUL DIV MOD`	dest = a op b, e.g. `ADD(Count,1,Count)`
`CPT(dest,expression)`	evaluates an expression: `CPT(Pct,(Level*100)/Cap)`
`COP / CPS / FLL`	copy arrays, copy synchronously, fill

Compares are input instructions — they sit on the left and pass power — and the math instructions are outputs that act every scan the rung is true, which is why a math rung nearly always wants an ONS, a one-scan timer bit or a counter edge in front of it. The batch preset in Part IV is a compare:




```
GRT(Thumbwheel,0)GEQ(Large_Bottle_Count,Thumbwheel)OTE(Batch_Done);
```

(Step 8: the line stops itself at the dialled number of large bottles; 0 means no limit.)

Scaling an analog input

Analog values arrive as INT counts on a channel of the 1769-IF4. By default a reservoir's `Level` is already 0–100 (percent), an encoder's pulses count up and wrap, a potentiometer is 0–100: the part's signal description (Part III) tells you the range. On a real panel the transmitter is a 4–20 mA loop and the card gives you counts, so the Reservoir and Tank parts have a *Level transmitter signal* setting that hands you what the card would: **Raw counts 0–32767** or **Engineering units 4000–20000** (4 mA = 4000, 20 mA = 20000). Scaling that into a number a person can read is one CPT into a REAL tag:

4–20 mA in engineering units → percent, as a REAL. The 100.0 keeps the arithmetic in floating point

```
CPT(Level_Pct,(Syrup_Level-4000)*100.0/16000);
```

The pattern is always *(raw − raw_min) × (eu_max − eu_min) / (raw_max − raw_min) + eu_min*. For raw counts it is `CPT(Level_Pct,Syrup_Level*100.0/32767)`. Put the result on a Digital Display with *Decimal places* set to 1 or 2 and the console reads `34.9 %`. A compare against a REAL works the same as against a DINT: `LES(Level_Pct,20.0)`.

Two more ways to get a float: a device signal can be bound straight to a tag of yours (*or use a tag directly* under the signal in Properties) and a REAL tag bound that way receives the unrounded value, bypassing the card; and the filler's own head tank, the weighing conveyor and the flow meter report the same way. The card route is the one you will meet in a plant, so the lessons use it.

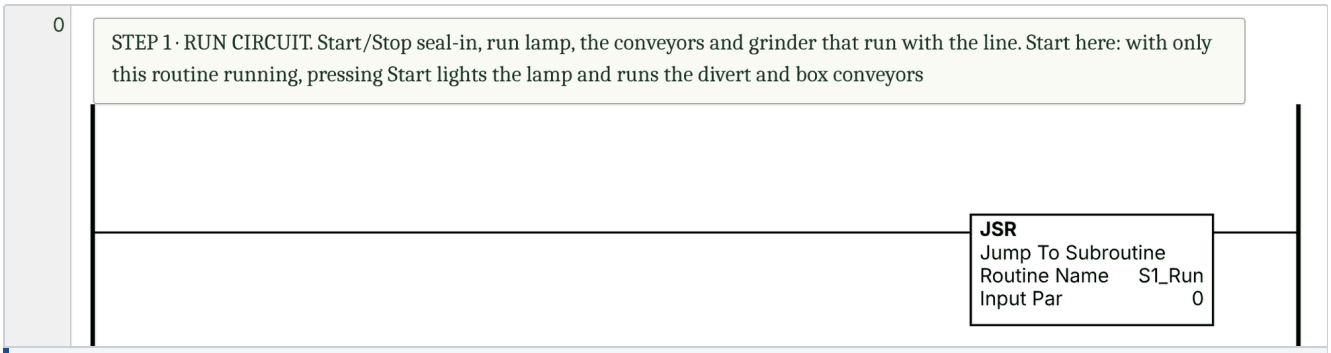
TRY IT

On the bottle line, set Syrup_Tank's *Level transmitter signal* to *Engineering units*, make a REAL tag `Level_Pct`, add the CPT rung above to S9_Supply, and point a Digital Display at `Level_Pct` with two decimals. Watch it fall with every charge. Then try the raw-counts setting and change the rung to match.

9 Routines, programs and the main routine

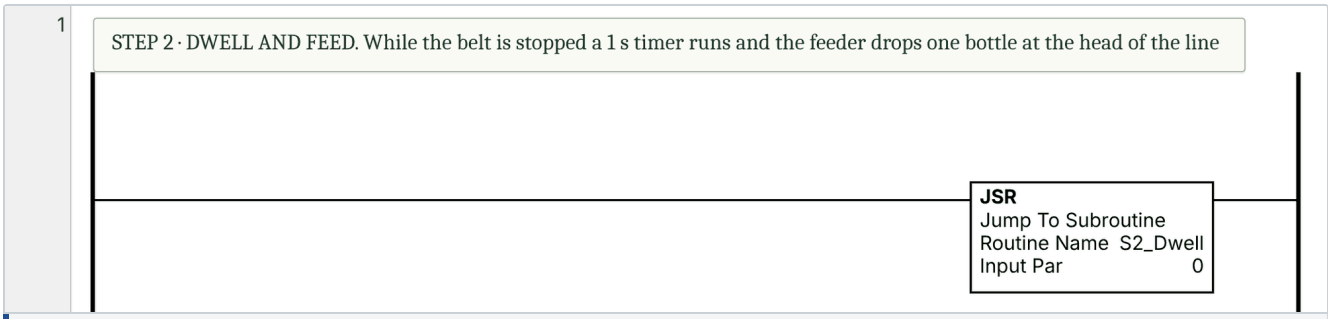
A program is a set of routines; one of them is the **main routine** and is the only one the controller scans on its own. Every other routine runs because a `JSR(Name)` in a running routine jumps to it, scans it top to bottom, and returns. A main routine made of nothing but JSRs is the cleanest way to organise a machine — and it is how Part IV is written:

RUNG 0



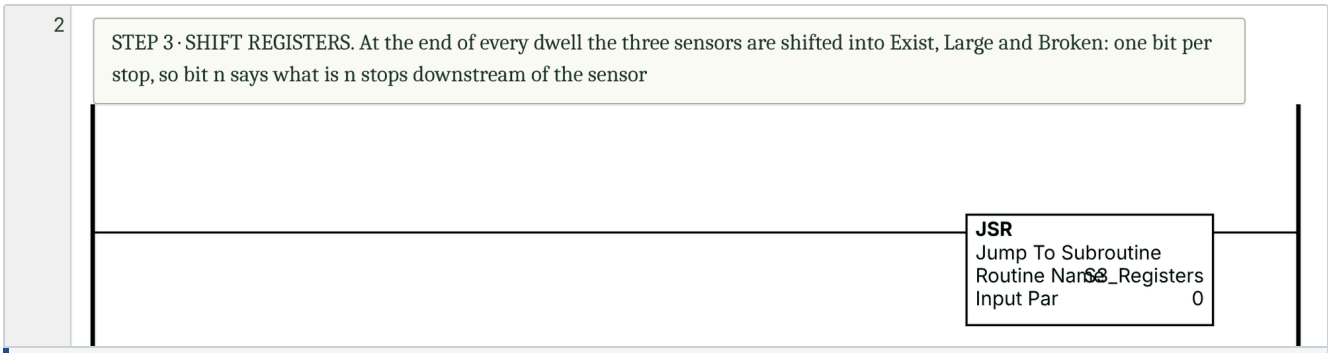

```
JSR(S1_Run);
```

RUNG 1




```
JSR(S2_Dwell);
```

RUNG 2




```
JSR(S3_Registers);
```

RUNG 3

3

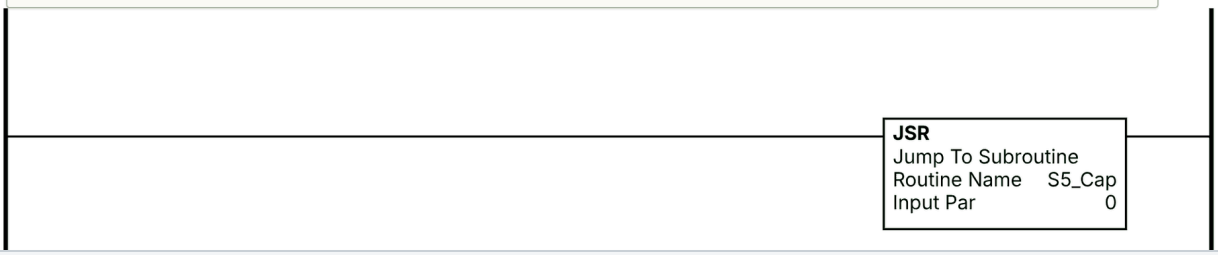
STEP 4 · FILL. A bottle under the nozzle (Exist bit 7 after the shift) gets its large or small charge from the head tank before the belt is allowed to step

```
JSR(S4_Fill);
```

RUNG 4

4

STEP 5 · CAP. A bottle under the chuck (Exist bit 10 after the shift) gets a cap from the chute; the cap feeder keeps the chute full and the two column lights say what the capper is doing

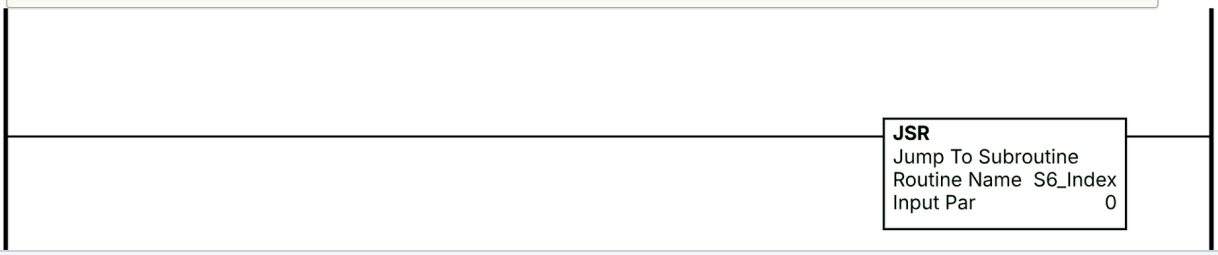



```
JSR(S5_Cap);
```

RUNG 5

5

STEP 6 · INDEX. When the dwell is done and nothing is holding the belt, step exactly one pitch on the encoder. This runs after Fill and Cap so their requests are seen in the same scan




```
JSR(S6_Index);
```

RUNG 6

6

STEP 7 · SORT. During the dwell: a broken bottle at the scrap gate (Broken bit 11) is dropped into the shredder; a small bottle at the divert gate (Exist 20, not Large 19, not Broken 18) goes down the catch incline to the divert conveyor

```
JSR(S7_Sort);
```

RUNG 7

7

STEP 8 · COUNT AND PRESET. The four displays, the thumbwheel batch preset and the Enter button

```
JSR(S8_Count);
```

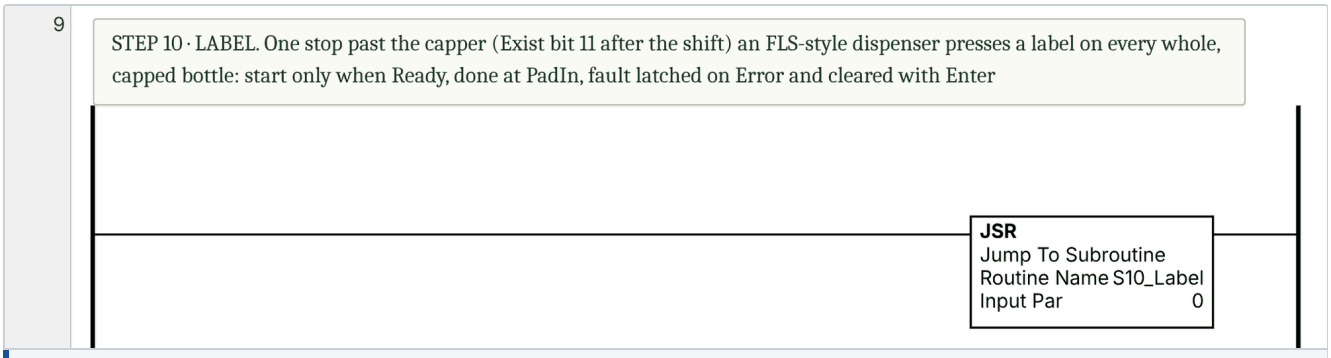
RUNG 8

8

STEP 9 · SUPPLY. The reservoir keeps the filler head tank full; its low float opens the inlet until the high float

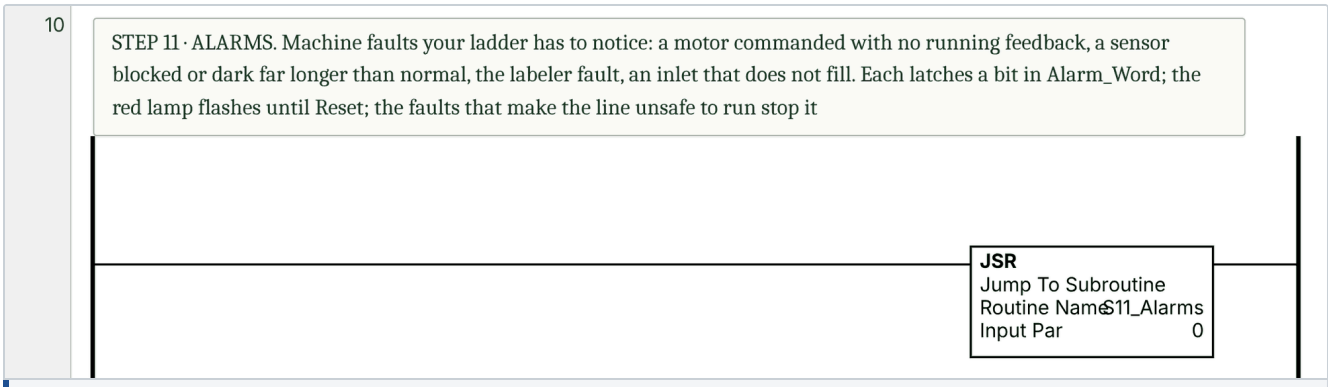
```
JSR(S9_Supply);
```

RUNG 9



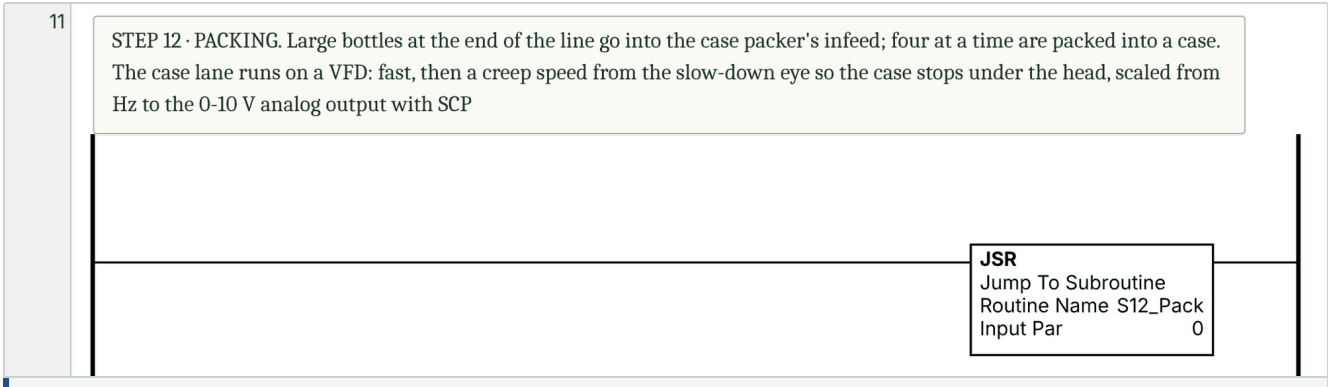

```
JSR(S10_Label);
```

RUNG 10




```
JSR(S11_Alarms);
```

RUNG 11




```
JSR(S12_Pack);
```

Order matters, because of the scan. Step 6 (index the belt) is called after steps 4 and 5 (fill, cap) so that a fill request raised this scan is seen by the index logic in the same scan; put the JSR for step 6 above step 4 and the belt can step before the request is noticed. Exercise 6 in Part IV asks you to do exactly that and watch.

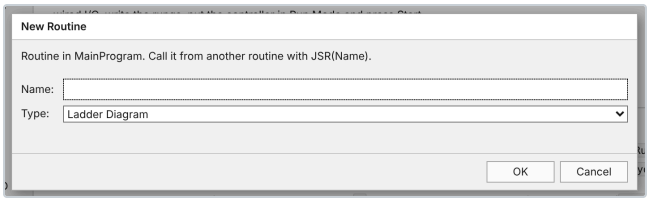


Figure 28. Logic → New Routine...

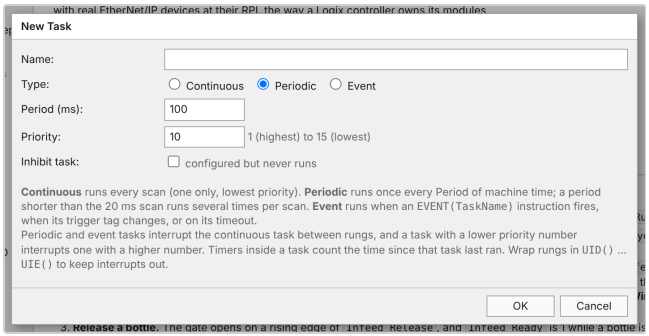
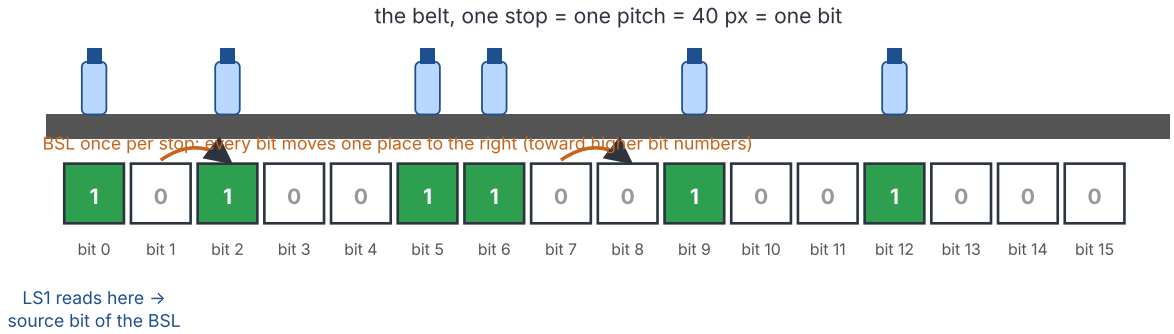


Figure 29. Task properties: continuous or periodic, period and priority.

Logic → New Routine… (or right-click the program) adds a routine; a routine with no JSR to it is dead code and Verify says so. `SBR` / `RET` pass parameters to and from a routine; `JMP(Label)` / `LBL(Label)` skip rungs within one; `AFI` (always false) parks a rung you want to keep but not run; `NOP` is a placeholder. Programs can be scheduled in tasks (continuous or periodic) under *Tasks*, and Add-On Instructions package a routine with its own parameters as a reusable instruction — see Help → Add-On Instructions when you get there. Structured text routines are also supported (Help → Structured Text).

10 Shift registers: tracking parts down a line

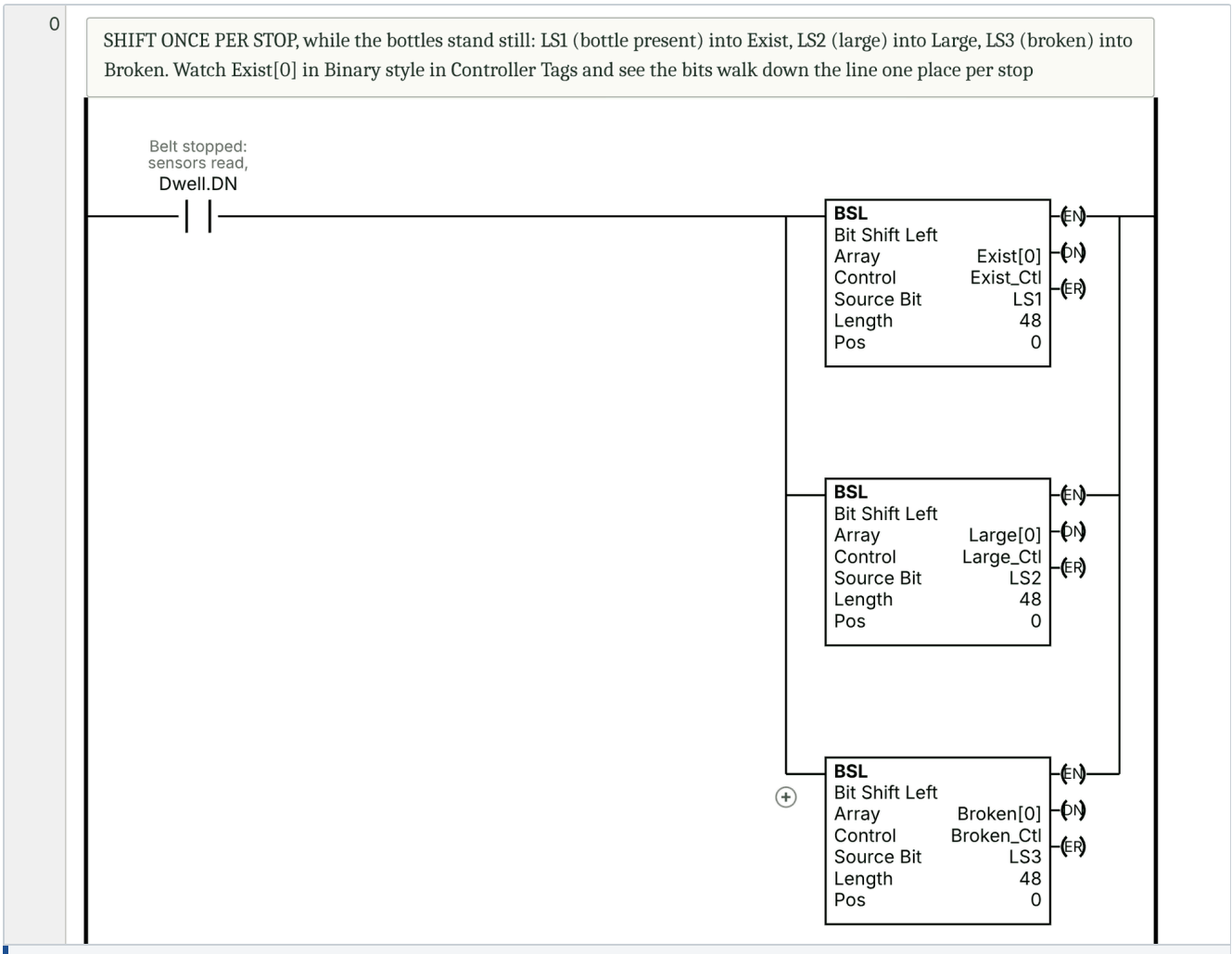
The bottle line's central idea is older than PLCs: an indexing conveyor moves a fixed pitch and stops; a sensor at the head reads what arrived; and a **shift register** — one bit per stop — moves what it knows one place down the line each time the belt steps. Bit 0 is the stop at the sensor; bit *n* is *n* stops downstream. A station at stop 12 reads bit 12 and knows what is standing in front of it without a sensor of its own.



A station n stops past the sensor reads bit n: the register is a model of the belt, one bit per bottle position.

Figure 30. A shift register as a picture of the belt. `Exist` shows a 1 wherever a bottle was seen; each index moves every 1 one stop down the line, in step with the bottles.

`BSL(array,control,source_bit,length)` — Bit Shift Left — does the move. Every time the rung goes true it shifts the whole array up one bit, copies the source bit into bit 0, and reports the bit that fell off the far end in `control.UL`. `BSR` shifts the other way. The control tag is a CONTROL; the length is the number of bits in use.




```
XIC(Dwell.DN)
[BSL(Exist[0],Exist_Ctl,LS1,48),BSL(Large[0],Large_Ctl,LS2,48),BSL(Broken[0],Br
oken_Ctl,LS3,48)];
```

Three registers, three questions about every stop: is there a bottle (`Exist`, from LS1), is it large (`Large`, from LS2, a retro beam set at a height only a large bottle breaks), is it broken (`Broken`, from LS3, a diffuse eye that only reports broken bottles). Downstream, a station asks its question with one XIC.

Which bit is my station?

Count the stops. The belt indexes 40 px. The head sensors sit on the stop grid at x = 160, 200 and 240 — one stop apart — and the stations at 440 (filler), 560 (capper), 720 (scrap gate) and 1000 (divert gate). The scrap gate is (720 − 240) / 40 = 12 stops past LS3. So `Broken[0].12` … almost.

The frame: before or after the shift

The shift happens at the *end* of the dwell, just before the belt steps. During the dwell, while the bottles stand still, the register was last shifted before the previous step: bit *n* describes the bottle that is now at stop *n + 1*. A rung that runs during the dwell — the gates in step 7 — therefore reads one bit *lower* than the count: `Broken[0].11` for the scrap gate. A rung that runs after the shift and before the step — the fill and cap requests in steps 4 and 5, which wait on `Dwell.DN` — reads the count itself. The table every lesson in Part IV refers to:

Station	x	Exist (LS1 @160)	Large (LS2 @200)	Broken (LS3 @240)
Filler	440	6 during the dwell · **7** after the shift	5 · **6**	4 · **5**
Capper	560	9 · **10**	8 · **9**	7 · **8**
Labeler	600	10 · **11**	9 · **10**	8 · **9**
Scrap gate	720	**13** · 14	**12** · 13	**11** · 12
Divert gate	1000	**20** · 21	**19** · 20	**18** · 19

Bold is the frame each station's rung actually uses. These numbers are not assumed: the program's test suite runs the line and correlates every bit against the bottle physically standing at each station, and the comments in the routines say which frame they are in.

WATCH FOR

A sensor that is not on the stop grid sees the same bottle at two stops, and a broken bottle then occupies two bits of the Broken register — the scrap gate fires twice and the good bottle behind the broken one goes into the shredder. The head sensors in the template sit at 160 / 200 / 240 for this reason; if you move one, re-measure your bits with Monitor Tags and a slow belt.

11 Reading the machine while it runs

Programming is mostly watching. Logix Connect gives you six windows onto a running controller, and each answers a different question.

- **The ladder, online.** True instructions light green; a timer shows ACC counting; a rung that should be true and is not shows you which instruction breaks the path. This is where you start.
- **Controller Tags → Monitor Tags**: every tag, live. Expand a DINT to see its bits; open a TIMER to see .ACC and .DN. Right-click a BOOL for *Toggle Bit* (Ctrl+T) to force a value while testing. The toolbar's Toggle button toggles the selected bit.
- **Quick Watch** (View → Quick Watch, floating or docked): a short list of the tags you care about right now, with values.
- **I/O Monitor**: the chassis, point by point, with the device and signal wired to each point. When a sensor "does not work", this is where you find that it is wired to a point your rung does not read.
- **Trends** (Window → New Trend…): pens on a strip chart. The tank level falling with every charge and rising when the inlet opens is a thing you have to see once.
- **The Simulator Log**: what the machine did, in its own words. *Capper cycled with no bottle under it*, *Blue bottle (s) pushed off by Divert_Gate with no bin or conveyor to land on*, *Grinder: ground with no box under it* — each line is a bug report with the device's name in it.

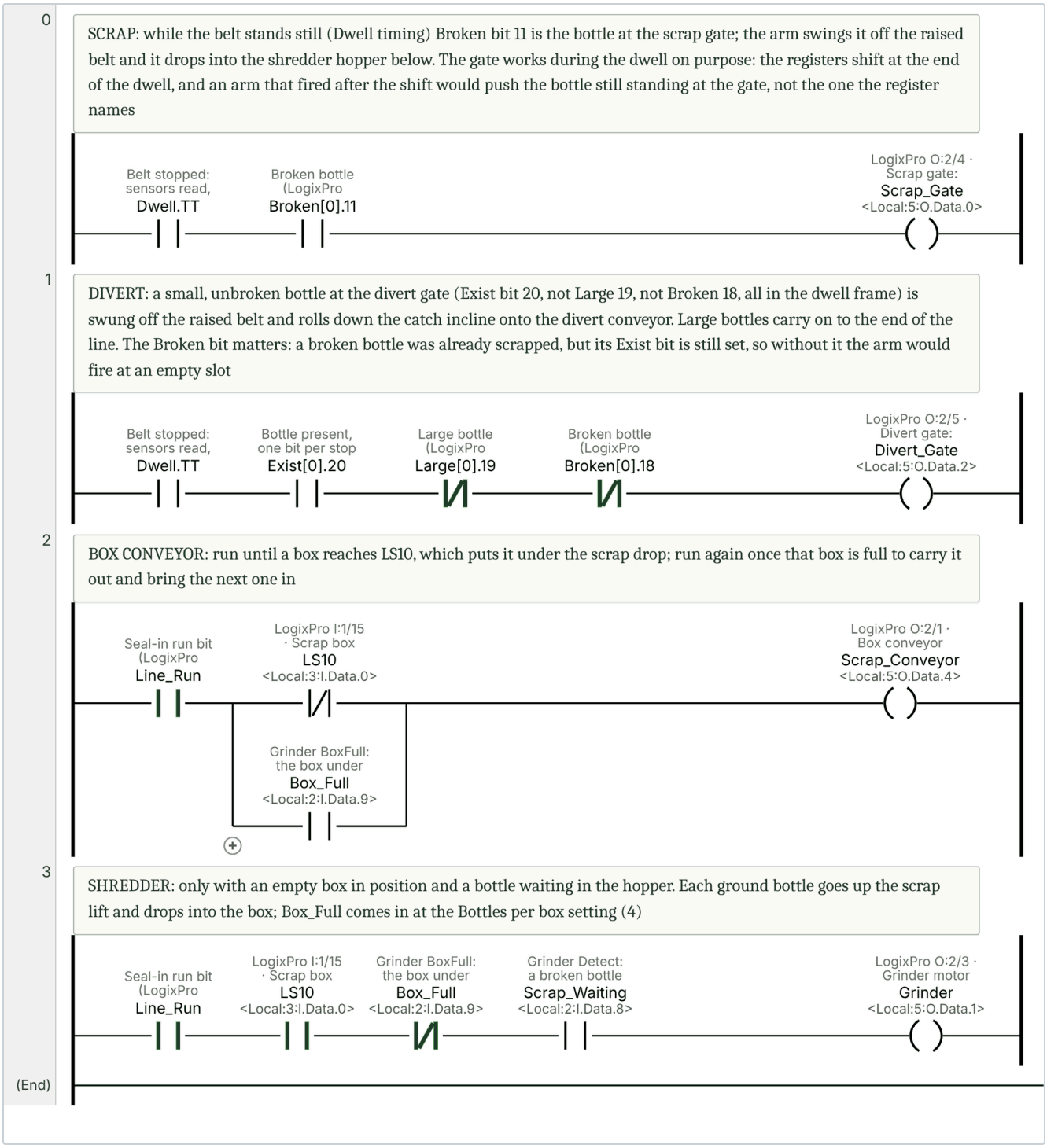


Figure 31. S7_Sort online while the line runs. Green is power; a rung that should be true and is not shows you the instruction that breaks the path.

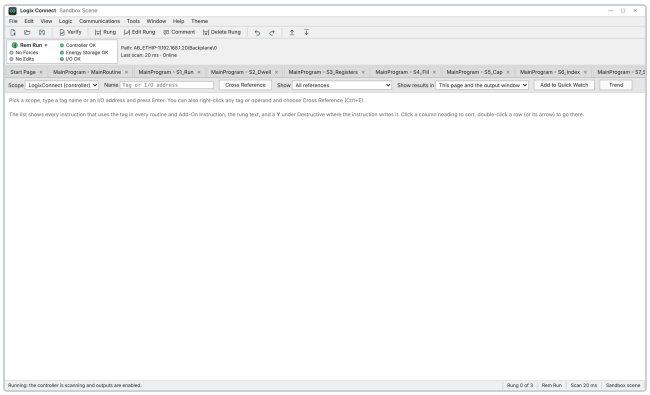


Figure 32. Cross Reference: every rung a tag is used in; click to go there.

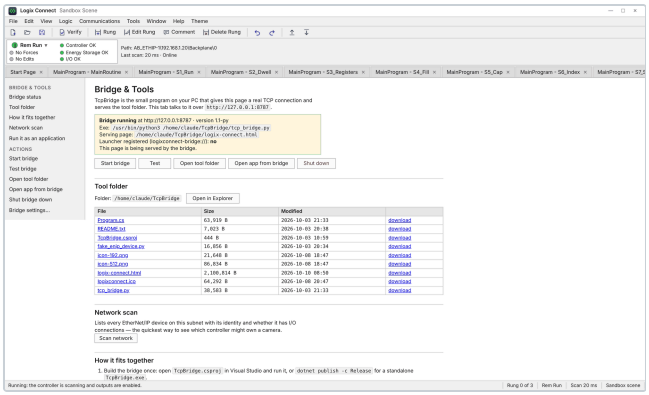


Figure 33. Bridge & Tools: the link to the Python bridge, TCP sockets and EtherNet/IP devices.

Cross Reference (Edit → Cross Reference…, or the bottom tab) lists every rung a tag appears in; click to jump. **Communications → I/O Forcing** lets you force a point regardless of the machine, the way you would on a real controller to test an output rung before the sensor is fitted; *Clear Faults* clears a faulted controller (a major fault stops the scan: an array index out of range, say). **Tools → Clear Tag Values** zeroes timers, counters and bits without touching the machine; **Reset Sim** does the machine as well.

Part III — The sandbox and its parts

12 The sandbox floor

File → New Project… → empty sandbox. The floor is a grid, 1600 × 520 px by default (Floor on the toolbar: up to 4000 × 800). One pixel is 1/60 of a metre in the 3D view, so a 20 m conveyor is 1200 px and the main line of Part IV runs most of the floor.

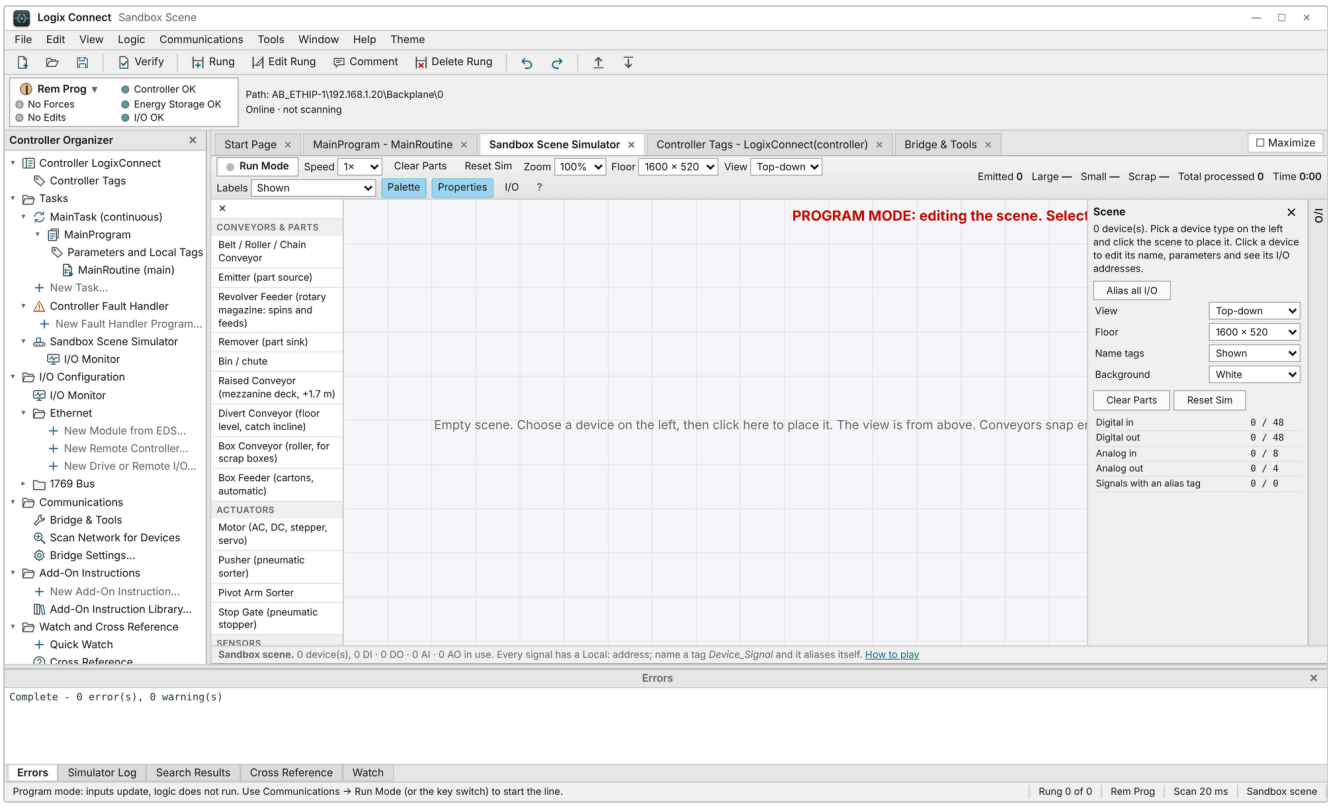
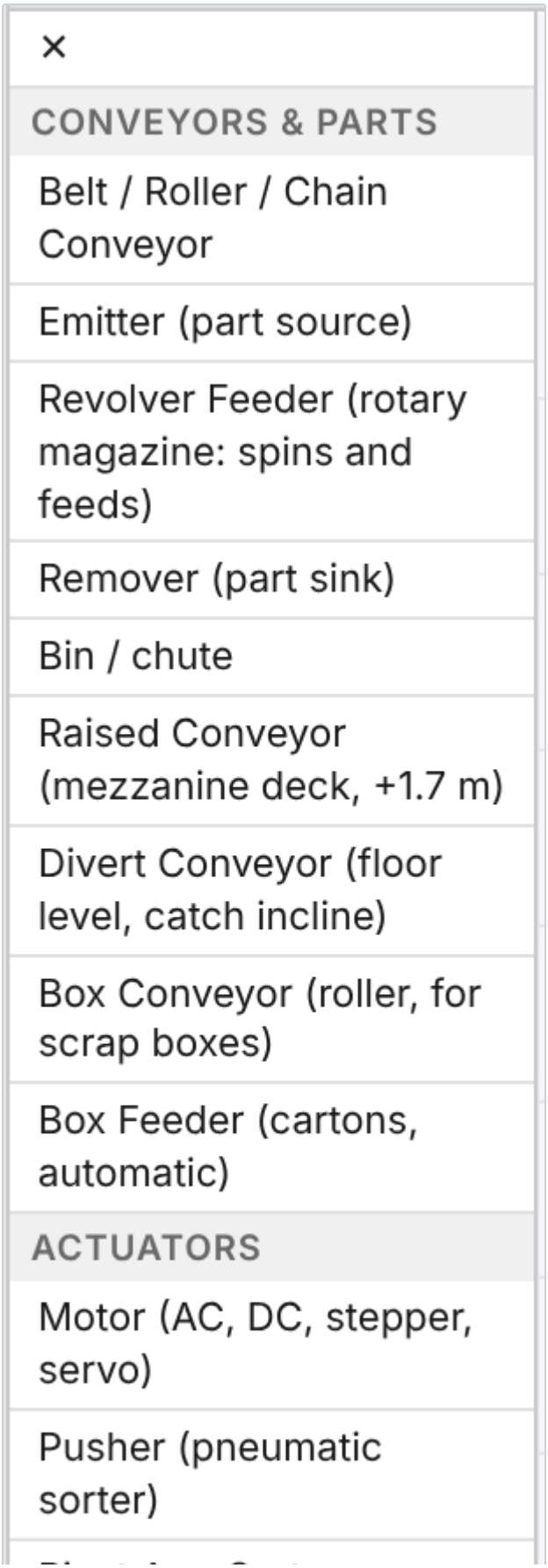


Figure 34. An empty sandbox: the floor, the Palette drawer on the left and the Properties drawer (Scene sheet) on the right.

Placing and arranging

Open the **Palette** drawer (or *Add Device* on a phone), click a part, click the floor: it is placed, named (`PE_1`, `Belt_2`…) and wired — every signal takes the next free point. Hold Shift while clicking to place several. Drag to move; parts snap to a 10 px grid, conveyors snap end to end and line up, and a sensor or station dragged near a conveyor snaps onto its centreline and takes its rotation. **R** rotates the selected part; the Properties drawer has Rotation and Position fields for exact placement, and the arrow keys nudge. Ctrl+click selects several; right-click (or long-press) for Duplicate, Delete, Group, Rotate, Put on a Panel; on a phone the long-press menu starts with Properties…, which opens the part's settings. Program mode only: the floor is locked while the controller runs.



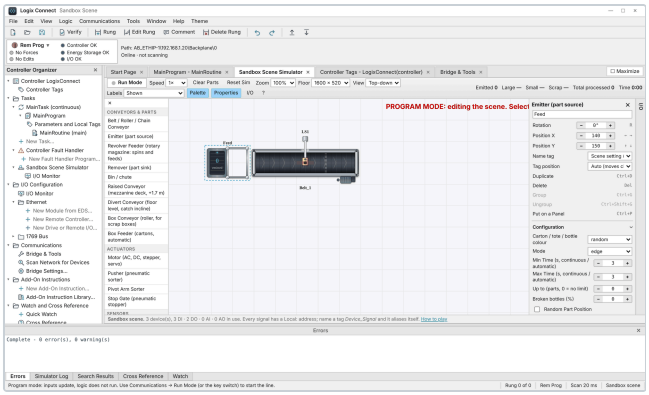


Figure 36. Three parts placed: a feeder, a belt and a photoeye, each wired to the next free point.

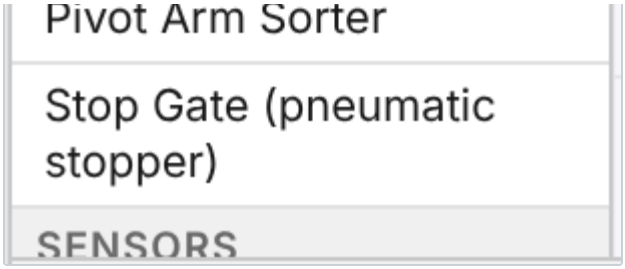


Figure 35. The Palette: every part in chapter 13, by category. Click a part, click the floor.

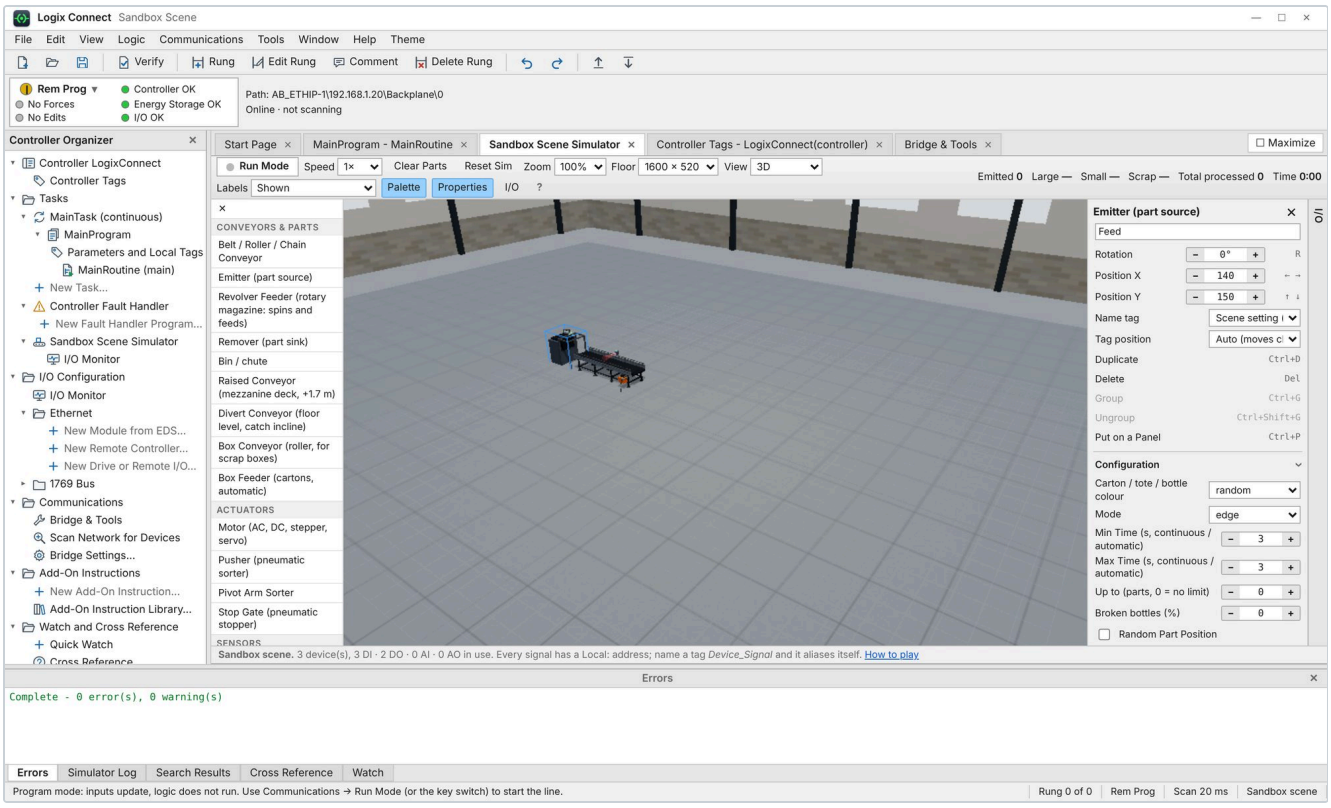


Figure 37. The same three parts in the 3D view.

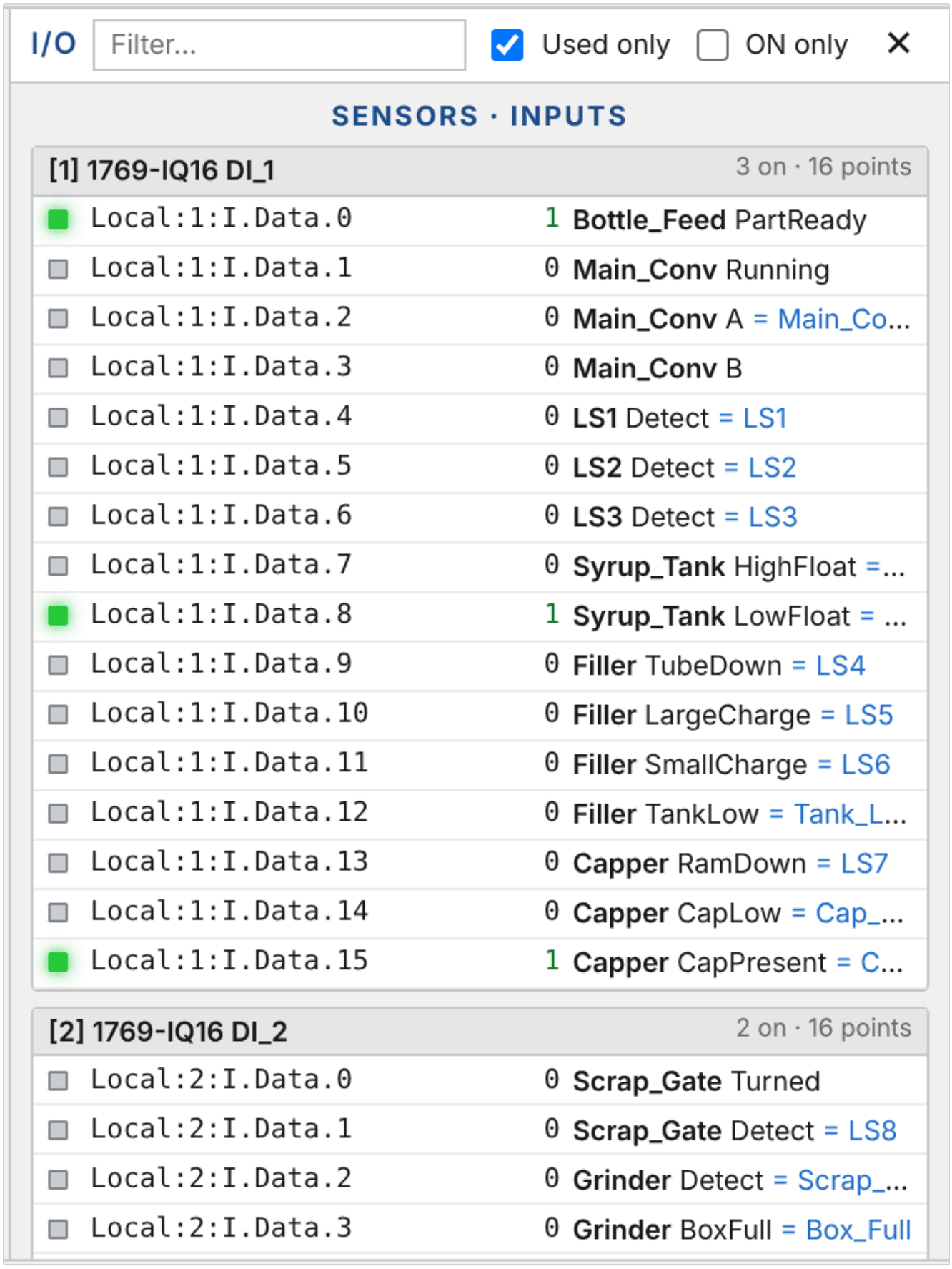

I/O
Filter...
Used only
ON only
SENSORS · INPUTS
[1] 1769-IQ16 DI_1
3 on · 16 points
Local:1:I.Data.0 1 Bottle_Feed PartReady
Local:1:I.Data.1 0 Main_Conv Running
Local:1:I.Data.2 0 Main_Conv A = Main_Co...
Local:1:I.Data.3 0 Main_Conv B
Local:1:I.Data.4 0 LS1 Detect = LS1
Local:1:I.Data.5 0 LS2 Detect = LS2
Local:1:I.Data.6 0 LS3 Detect = LS3
Local:1:I.Data.7 0 Syrup_Tank HighFloat =...
Local:1:I.Data.8 1 Syrup_Tank LowFloat = ...
Local:1:I.Data.9 0 Filler TubeDown = LS4
Local:1:I.Data.10 0 Filler LargeCharge = LS5
Local:1:I.Data.11 0 Filler SmallCharge = LS6
Local:1:I.Data.12 0 Filler TankLow = Tank_L...
Local:1:I.Data.13 0 Capper RamDown = LS7
Local:1:I.Data.14 0 Capper CapLow = Cap_...
Local:1:I.Data.15 1 Capper CapPresent = C...
[2] 1769-IQ16 DI_2
2 on · 16 points
Local:2:I.Data.0 0 Scrap_Gate Turned
Local:2:I.Data.1 0 Scrap_Gate Detect = LS8
Local:2:I.Data.2 0 Grinder Detect = Scrap_...
Local:2:I.Data.3 0 Grinder BoxFull = Box_Full

Figure 38. The I/O drawer: the chassis points in use and the signals on them.

Properties

Click a part and the **Properties** drawer shows its name (edit it there; the alias tags follow), rotation and position, the name-tag setting, the *Configuration* block (the parameters listed in chapter 13) and *Signals and Addresses*: every signal, its kind, its point, and the *tag Device_Signal* link that creates the alias. A signal can also be bound straight to any tag of yours (*or use a tag directly: click to pick*), which bypasses the chassis entirely; the lessons use addresses and aliases because that is what you will meet on a real panel.

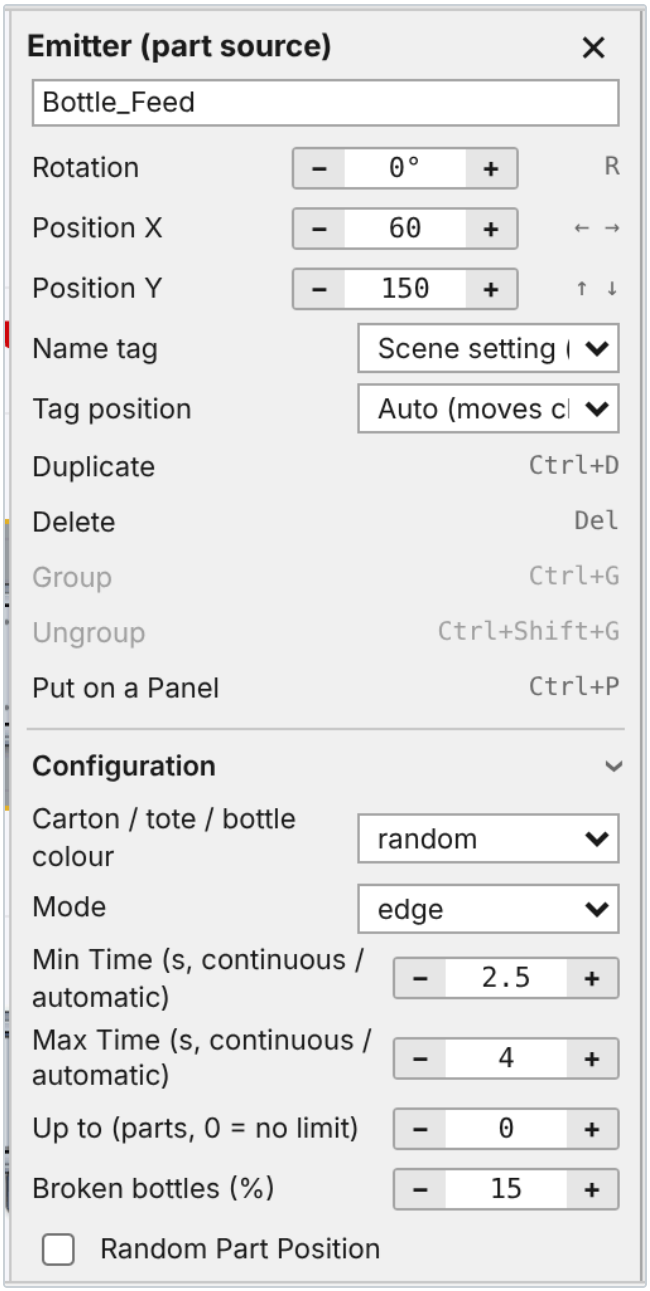


Figure 39. Properties of a conveyor: length, speed, style, Height (floor or raised deck), encoder.

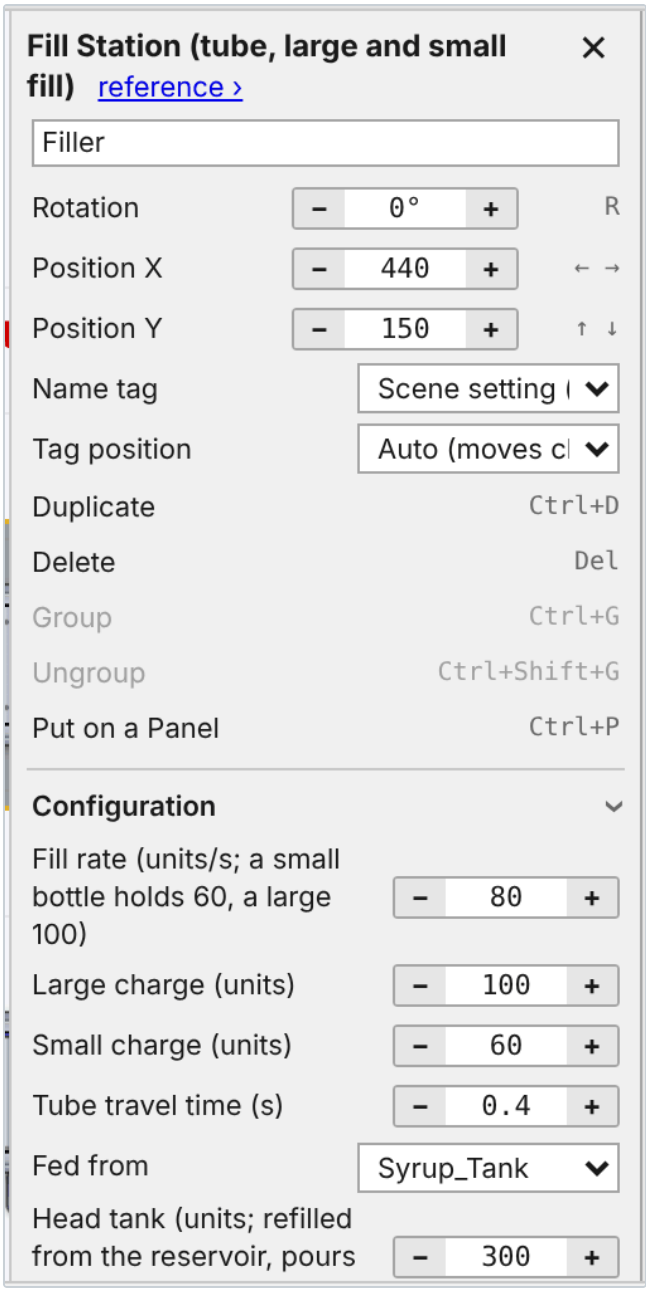


Figure 40. Properties of the Inline Cap Feeder: caps in the chute, magazine rate, the lamp signals.

Panels

A **Panel / Enclosure** is a drawn operator station. Drop buttons, switches, lights, thumbwheels and displays onto it and they move with it; *Put on a Panel* builds one around the selected parts. In 3D the panel stands on a pedestal with the devices on its face. A panel can also be given its own remote I/O block (right-click → its own I/O) so the operator station talks over EtherNet/IP like a real one.

Views

Top-down is the plan: lay the line out here. **Side** is the elevation: bottles stand on the belt, the fill tube comes down, the ram hits, tanks show their level. **3D** (desktop) is the shop floor: drag to orbit, wheel to zoom, right-drag or Shift+drag to pan, double-click a device to look at it, double-click the floor to reset; hover a device for its live I/O tag. All three are the same scene and the same I/O. Tall parts near the top of the floor get headroom automatically, and the floor fills the window.

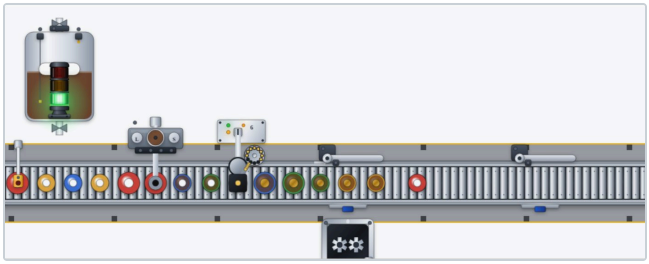


Figure 41. Top-down: the plan.

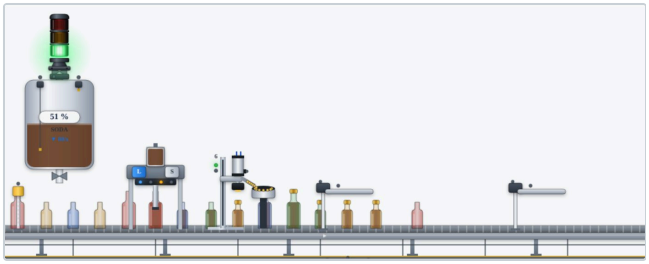


Figure 42. Side: the elevation.

Figure 43. 3D: the raised deck from the head of the line.

Rules of the simulated world

- Parts are bottles (small, 60 units, and large, 100 units, some of them broken when the feeder is set to break some), cartons and totes (S / M / L, which a grinder fills with glass), and metal blocks and raw parts for sorting lines.
- A conveyor moves anything within its lane at its speed; parts queue behind each other; a part that reaches the end of a belt with nothing under it falls and is **lost**. Removers and bins take parts off the line and count them as processed. A Weighing Conveyor reports a weight.
- A pivot arm or pusher knocks a part sideways; it slides up to 200 px and lands on the first conveyor it crosses, into a bin, or into a grinder hopper. A raised conveyor (Height = Raised deck) stands 1.7 m up

on a mezzanine; a gate on it drops bottles into a grinder below or down a catch incline onto a floor-level conveyor beside it. The incline, the rail opening and the latch door appear by themselves from the layout.

- Sensors see what is physically there, within their range and sensitivity and according to what they are set to see (any part, bottles only, broken bottles, filled bottles, capped bottles, boxes only…). A retro beam at a height only sees parts taller than that height.

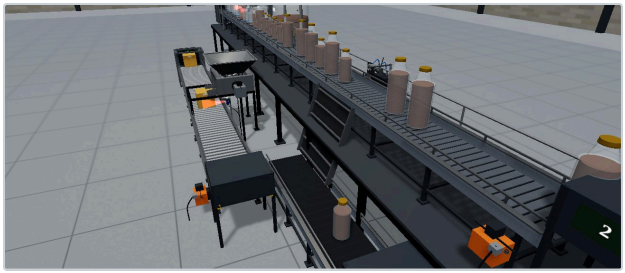


Figure 44. A gate on the deck is a latch door; a small bottle rides the catch incline down to the divert conveyor.

Figure 45. The scrap gate drops a broken bottle into the shredder hopper; the chute fills a box below.

- Fillers draw product from a reservoir through a pipe (the pipe draws itself) into a head tank; a dry tank stops the pour. Cappers take caps from a chute fed by a rotary magazine; an empty chute leaves the bottle open, and a chuck that comes down on a broken bottle crushes it. Grinders need a box under the chute or the glass spills.

13 Parts catalogue

Every part in the palette, in palette order, with its settings and its signals. A signal marked *Digital input* is something the machine tells the controller (it arrives at a `Local:n:I` point); *Digital output* is something the controller tells the machine (a `Local:n:O` point). Some signals only exist when a setting is on (an encoder on a conveyor, a drain valve on a reservoir); the tables show the signals a part has with its default settings. The entries marked *preset of* are the same part with settings already made, listed as parts of their own so you can see every option in the list.

Conveyors & parts

Belt / Roller / Chain Conveyor `belt`

Tick **Incremental encoder** for channels A and B in quadrature (00 → 01 → 11 → 10 running forward, reversed when the belt reverses) plus a Pulses count, one pulse per resolution step of belt travel. Count pulses to position a part without a second sensor.

Figure 46. A slat-chain conveyor on the raised deck (Height = Raised deck): the mezzanine, rails and encoder draw themselves.

Configuration

key	Setting	Default	Range / choices
`length`	Length (m), 0.5 to 40 in half-metre steps	4	0.5 – 40, step 0.5
`speed`	Speed (mm/s)	120	20 – 400, step 10
`reversible`	Reversible (adds Rev output)	False	on / off
`vfd`	VFD speed reference (adds analog output, 0-100 %)	False	on / off
`drive`	Driven by drive module (its name; blank = contactor)		text
`style`	Conveyor type	belt	belt, cleated belt, roller, gravity roller (no motor), modular plastic belt, slat chain, twin chain, wire mesh
`beltColor`	Belt color	Black	Black, Green, Blue, White, Grey
`frame`	Frame	Aluminium	Aluminium, Stainless steel, Painted blue, Painted grey, Safety yellow, Black
`elev`	Height (3D)	Floor level	Floor level, Raised deck (+1.7 m)
`encoder`	Incremental encoder (adds A, B and Pulses)	False	on / off
`encRes`	Encoder resolution (px per pulse)	10	2 – 60, step 1

Signals (with the default settings)

Signal	Kind	Meaning
`Fwd`	Digital output (from the controller)	Motor forward
`Running`	Digital input (to the controller)	Motor contactor aux contact

Emitter (part source) `emitter`

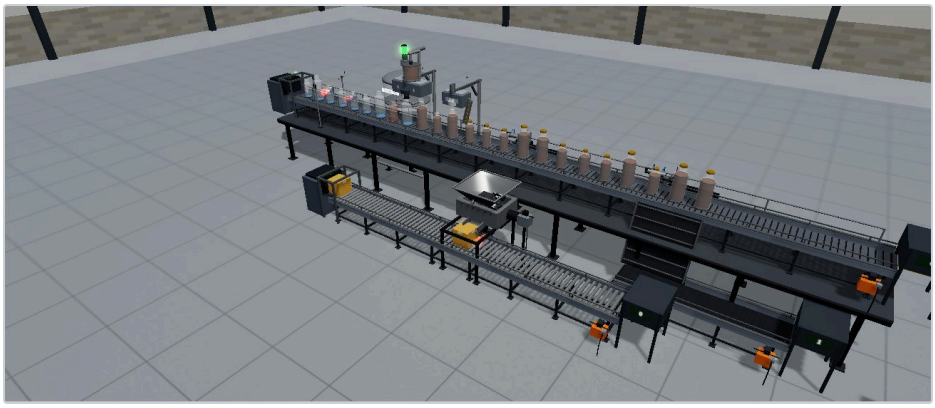


Figure 47. An Emitter at the head of a belt (left) makes parts: bottles, cartons, totes, blocks.

Configuration

key	Setting	Default	Range / choices
`parts`	Part to Emit	cartonS, cartonM, cartonL, metalM	Carton (S), Carton (M), Carton (L), Shipping Crate (XL), Tote, Metal Block (S), Metal Block (M), Metal Block (L), Raw Stock, Blue, Raw Stock, Green, Raw Stock, Metal, Housing Base, Blue, Housing Base, Green, Housing Base, Metal, Housing Lid, Blue, Housing Lid, Green, Housing Lid, Metal, Bottle (S), Can, Bottle (M), Bottle (L)
`color`	Carton / tote / bottle colour	random	random, random (R/B/G), R, B, G, Y
`mode`	Mode	edge	edge, continuous, automatic (no Emit input)
`minTime`	Min Time (s, continuous / automatic)	3	0.5 – 60, step 0.5
`maxTime`	Max Time (s, continuous / automatic)	3	0.5 – 60, step 0.5
`upTo`	Up to (parts, 0 = no limit)	0	0 – 999, step 1
`broken`	Broken bottles (%)	0	0 – 100, step 5
`randPos`	Random Part Position	False	on / off
`randRot`	Random Part Orientation	False	on / off

Signals (with the default settings)

Signal	Kind	Meaning
`Emit`	Digital output (from the controller)	Release a part (rising edge, or continuous while on)
`PartReady`	Digital input (to the controller)	Exit clear, ready to emit

Revolver Feeder (rotary magazine: spins and feeds) `revolver`

A rotary magazine like a revolver cylinder: *Pockets* bores round a drum beside the lane, loaded with *Parts loaded at start*. A rising edge on **Index** turns the drum one pocket (*Index time*) and the part that reaches the drop pocket lands on the lane; **InPosition** is 1 while the drum stands still, **PartReady** while a part sits in the drop pocket with the lane clear, **Empty** when the drum has run out. **Reload** (or *Reload by itself*) refills every pocket in about a second. Pair it with a sensor on the lane: XIC(PartReady) XIO(Lane_Eye) OSR → Index is the classic one-at-a-time feed.

Configuration

key	Setting	Default	Range / choices
`parts`	Part to feed	bottle, bottleL	Carton (S), Carton (M), Carton (L), Shipping Crate (XL), Tote, Metal Block (S), Metal Block (M), Metal Block (L), Raw Stock, Blue, Raw Stock, Green, Raw Stock, Metal, Housing Base, Blue, Housing Base, Green, Housing Base, Metal, Housing Lid, Blue, Housing Lid, Green, Housing Lid, Metal, Bottle (S), Can, Bottle (M), Bottle (L)
`color`	Carton / tote / bottle colour	random	random, random (R/B/G), R, B, G, Y
`pockets`	Pockets on the drum	8	4 – 16, step 1
`loaded`	Parts loaded at start	8	0 – 200, step 1
`indexTime`	Index time (s, one pocket)	0.5	0.1 – 3, step 0.1
`autoReload`	Reload by itself when empty	True	on / off
`broken`	Broken bottles (%)	0	0 – 100, step 5

Signals (with the default settings)

Signal	Kind	Meaning
`Index`	Digital output (from the controller)	Turn the drum one pocket (rising edge); the part in the drop pocket lands on the lane
`Reload`	Digital output (from the controller)	Fill the drum (hold on)
`InPosition`	Digital input (to the controller)	Drum at rest on a pocket
`PartReady`	Digital input (to the controller)	A part sits in the drop pocket and the lane under it is clear

Signal	Kind	Meaning
`Empty`	Digital input (to the controller)	No parts left on the drum

Remover (part sink) `remover`

No settings.

Signals (with the default settings)

Signal	Kind	Meaning
`Detect`	Digital input (to the controller)	Part entering the remover

Bin / chute `bin`

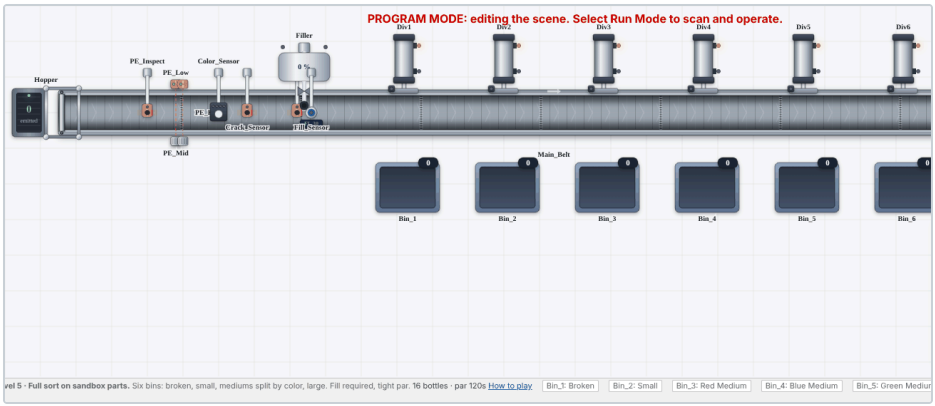


Figure 48. Bins with grading rules under the six pushers of Level 5; the rule shows in the Specs chips.

Configuration

key	Setting	Default	Range / choices
`width`	Width (px)	90	50 – 200, step 10
`capacity`	Full at (parts)	10	1 – 100, step 1
`gKind`	Grading: bottles this bin should get	not graded	not graded, whole bottles, broken bottles
`gSize`	Grading: size	any	any, S, M, L
`gColor`	Grading: colour	any	any, R, B, G, Y
`gFill`	Grading: must be filled	no	no, yes

Signals (with the default settings)

Signal	Kind	Meaning
`Full`	Digital input (to the controller)	Bin holds its capacity
`Count`	Analog input	Parts in the bin

Raised Conveyor (mezzanine deck, +1.7 m) `preset of belt`

A conveyor on a raised deck. Everything placed in its lane stands on the deck; a gate on it drops bottles into a Grinder hopper or down a catch incline onto a floor-level conveyor beside it

Places a Belt / Roller / Chain Conveyor with `elev` = Raised deck (+1.7 m).

Divert Conveyor (floor level, catch incline) `preset of belt`

A floor-level roller conveyor. Lay it alongside a raised conveyor and a cleated catch incline appears between them where a gate kicks bottles down to it

Places a Belt / Roller / Chain Conveyor with `length` = 4, `speed` = 120, `style` = roller.

Box Conveyor (roller, for scrap boxes) `preset of belt`

A roller conveyor for the scrap boxes: run it under the Grinder chute with a photoeye set to boxes only

Places a Belt / Roller / Chain Conveyor with `length` = 10, `speed` = 80, `style` = roller, `beltColor` = Black.

Box Feeder (cartons, automatic) `preset of emitter`

Drops a carton every 4 s at the head of a box conveyor

Places a Emitter (part source) with `parts` = ['cartonM'], `color` = Y, `mode` = automatic (no Emit input), `minTime` = 4, `maxTime` = 4.

Actuators

Motor (AC, DC, stepper, servo) `motor`

Configuration

key	Setting	Default	Range / choices
`kind`	Motor type	Three-phase AC	Three-phase AC, Single-phase AC, DC (PWM drive), Stepper, Servo
`color`	Housing color	Grey	Grey, Blue, Green, Black, Red, Orange, Yellow, White, Stainless steel
`size`	Size	Medium	Small, Medium, Large
`rpm`	Rated speed (rpm)	1750	100 – 6000, step 50
`reversible`	Reversible (adds Rev output)	False	on / off
`vfd`	VFD speed reference (adds analog output, 0-100 %)	False	on / off
`drive`	Driven by drive module (its name; blank = contactor)		text
`brake`	Spring-applied brake (adds Brake output, 1 = release)	False	on / off
`gearbox`	Gearbox on the shaft	False	on / off
`ratio`	Gearbox ratio	10:1	5:1, 10:1, 20:1, 50:1
`tach`	Speed feedback (adds Speed analog input, rpm)	False	on / off
`encoder`	Incremental encoder (adds A, B and Pulses)	False	on / off
`encPPR`	Encoder pulses per revolution	24	1 – 1024, step 1
`control`	Stepper control	Step and direction pulses	Step and direction pulses, Pulse rate (analog) and direction
`stepsRev`	Steps per revolution	200	20 – 6400, step 20
`servoMode`	Servo mode	Position	Position, Velocity
`servoSpeed`	Servo maximum speed (degrees/s)	720	30 – 7200, step 30
`servoAccel`	Servo acceleration (degrees/s²)	3600	90 – 36000, step 90

Signals (with the default settings)

Signal	Kind	Meaning
`Run`	Digital output (from the controller)	Contactor forward
`Running`	Digital input (to the controller)	Aux contact

Signal	Kind	Meaning
`Overload`	Digital input (to the controller)	Overload relay OK (NC, 1 = healthy; click the motor in Run mode to trip or reset it)

Pusher (pneumatic sorter) `pusher`

Configuration

key	Setting	Default	Range / choices
`stroke`	Stroke time (s)	0.15	0.05 – 1, step 0.05

Signals (with the default settings)

Signal	Kind	Meaning
`Extend`	Digital output (from the controller)	Solenoid: extend cylinder
`Extended`	Digital input (to the controller)	Reed switch: cylinder extended
`Retracted`	Digital input (to the controller)	Reed switch: cylinder retracted
`Detect`	Digital input (to the controller)	Photoeye in front of the pusher

Pivot Arm Sorter `pivot`

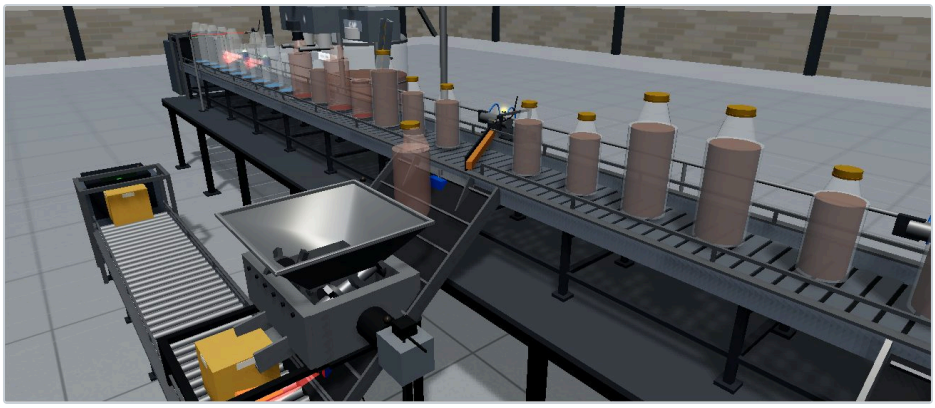


Figure 49. A Pivot Arm Sorter on a raised belt becomes a latch door: the arm swings and the bottle drops through.

Configuration

key	Setting	Default	Range / choices
`stroke`	Swing time (s)	0.12	0.05 – 1, step 0.05

Signals (with the default settings)

Signal	Kind	Meaning
`Turn`	Digital output (from the controller)	Swing the arm into the lane
`Turned`	Digital input (to the controller)	Arm at end of travel
`Detect`	Digital input (to the controller)	Photoeye ahead of the arm

Stop Gate (pneumatic stopper) `gate`

No settings.

Signals (with the default settings)

Signal	Kind	Meaning
`Raise`	Digital output (from the controller)	Raise the stopper into the lane
`Up`	Digital input (to the controller)	Stopper up
`Down`	Digital input (to the controller)	Stopper down

VFD (variable frequency drive, hardwired: 0-10 V speed reference) `vfd`

A variable frequency drive wired to the controller the way most are: two-wire **Run** (and **Rev**) from digital outputs, a **0-10 V speed reference** from an analog output channel, and **Ready**, **Running**, **AtSpeed** and **Fault** back to digital inputs, plus its analog output (**OutputFreq**) for the actual frequency. The analog channels are in engineering units, 0-10000 = 0-10.000 V, and 10 V is the *Frequency at 10 V* setting, so a reference of 5000 asks for 30 Hz on a 60 Hz drive: scale it in the program (SCP or CPT). The drive ramps with its *Accel* and *Decel* times, so a conveyor told to stop from full speed coasts on for a while: slow it down first if it has to stop on a mark. A trip (Tools → Inject Machine Fault) switches the output off and latches until the cause is gone *and* FaultReset rises. To put it on a conveyor, type its name in the belt's *Driven by drive module*: the belt then runs at its *Speed* × output frequency ÷ the 10 V frequency, and its own Fwd output is not used.

Configuration

key	Setting	Default	Range / choices
`maxHz`	Frequency at 10 V (Hz)	60	10 – 120, step 1
`accel`	Accel time, 0 to max (s)	1	0.1 – 30, step 0.1
`decel`	Decel time, max to 0 (s)	0.5	0.1 – 30, step 0.1
`atBand`	At Speed band (± Hz)	0.5	0.1 – 5, step 0.1
`fbk`	Analog output: output frequency (adds OutputFreq)	True	on / off

Signals (with the default settings)

Signal	Kind	Meaning
`Run`	Digital output (from the controller)	Run while 1 (two-wire control)
`Rev`	Digital output (from the controller)	Reverse while 1, with Run
`FaultReset`	Digital output (from the controller)	Clears a trip on the rising edge, once its cause has gone
`SpeedRef`	Analog output	Speed reference: 0-10000 = 0-10.000 V = 0 to the 10 V frequency
`Ready`	Digital input (to the controller)	Powered and not tripped
`Running`	Digital input (to the controller)	Output frequency above zero (also while decelerating)
`AtSpeed`	Digital input (to the controller)	Running and within the At Speed band of the reference
`Fault`	Digital input (to the controller)	Tripped: output off until FaultReset
`OutputFreq`	Analog input	Output frequency: 0-10000 = 0-10 V = 0 to the 10 V frequency

Sensors

Weighing Conveyor `scale`

Configuration

key	Setting	Default	Range / choices
`speed`	Speed (mm/s)	120	20 – 400, step 10

Signals (with the default settings)

Signal	Kind	Meaning
`Fwd`	Digital output (from the controller)	Motor forward
`Weight`	Analog input	Weight on the scale, kg
`Running`	Digital input (to the controller)	Motor aux contact

Fill Level Sensor (liquid in the bottle) `level`

Configuration

key	Setting	Default	Range / choices
`height`	Fill line (px above the belt; a small bottle is 40 tall, a large 56; the liquid tops out at about 80 % of that)	30	2 – 80, step 1
`type`	Sensor	Capacitive (through the glass)	Capacitive (through the glass), Photoelectric (clear-liquid eye)
`analog`	Analog output (adds a Percent analog input: fill of the bottle under the sensor, 0-100 % of its label)	False	on / off

Signals (with the default settings)

Signal	Kind	Meaning
`Detect`	Digital input (to the controller)	Liquid at or above the fill line in the bottle under the sensor

Diffuse Sensor (photoelectric) `pe`

Diffuse photoelectric sensor: its own light reflects off the part, so no reflector is needed. The shaded band on the lane shows the sensing range.

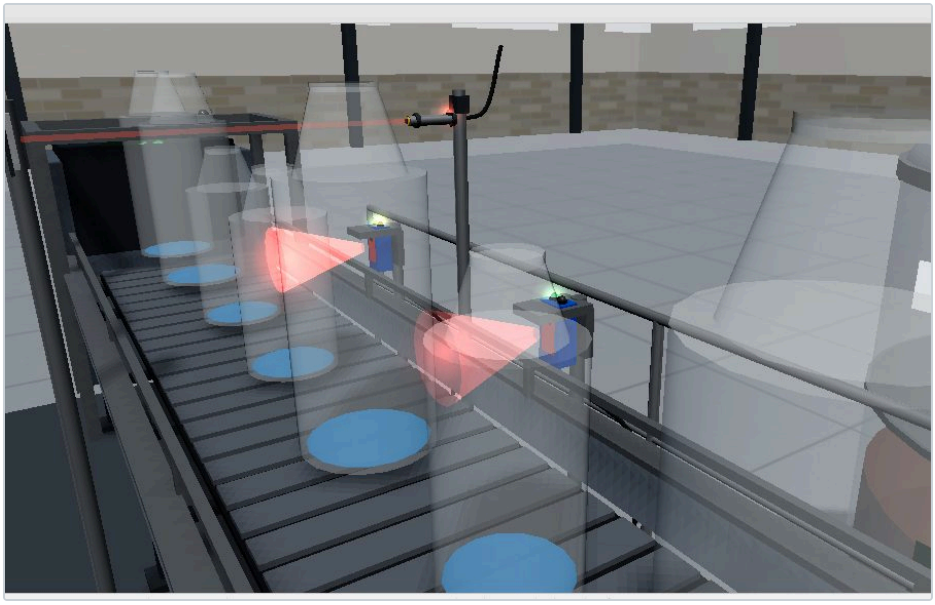


Figure 50. Sensors carry a real indicator LED: lens, bezel and a halo when lit.

Configuration

key	Setting	Default	Range / choices
`range`	Sensing range (px; 60 px = 1 m)	20	2 – 100, step 1
`sensitivity`	Sensitivity (%): the gain. 100 sees every part at full range; lower it and the range shrinks and dull targets (glass bottles, broken glass, dark cartons) drop out before bright ones (metal, cans, white and yellow cartons)	100	10 – 100, step 5
`beam`	Beam	Wide (diffuse)	Wide (diffuse), Narrow (focused spot), Laser (pencil)
`sees`	Sees	any part	any part, bottles only, broken bottles, filled bottles, capped bottles, labeled bottles, empty bottles, boxes only

Signals (with the default settings)

Signal	Kind	Meaning
`Detect`	Digital input (to the controller)	Detecting: a solid part is within range

Retroreflective Sensor and Reflector `retro`

The sensor and its reflector face each other across the conveyor. **Detect** is 1 while the beam is interrupted. Use the beam height to ignore low parts: a lid (8 px) passes under a 12 px beam, a carton does not.

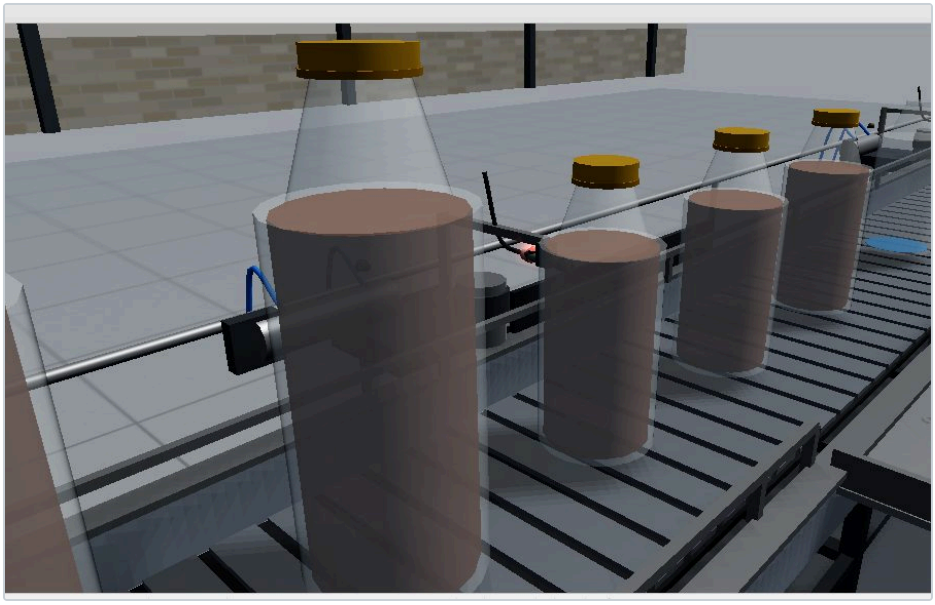


Figure 51. A retroreflective beam at height: only a bottle taller than the beam breaks it.

Configuration

key	Setting	Default	Range / choices
`height`	Beam height (px above the belt)	12	2 – 80, step 1

Signals (with the default settings)

Signal	Kind	Meaning
`Detect`	Digital input (to the controller)	Light beam interrupted

Light Array (8 beams) `array`

Eight parallel beams form a light curtain. Beam 1 is lowest. **Digital** gives one BOOL per beam, **Numerical** packs the beams into one INT (bit 0 = beam 1), **Analog** gives 10 V × interrupted beams ÷ 8 as hundredths of a volt. With the default 6 px spacing a small carton (22 px) breaks beams 1-4, a medium one 5, a large one 7, a crate all 8, a lid only beam 1.

Configuration

key	Setting	Default	Range / choices
`mode`	Output	digital	digital, numerical, analog
`pitch`	Beam spacing (px of part height)	6	2 – 20, step 1

Signals (with the default settings)

Signal	Kind	Meaning
`Beam1`	Digital input (to the controller)	Beam 1 interrupted (3 px above the belt)
`Beam2`	Digital input (to the controller)	Beam 2 interrupted (9 px above the belt)
`Beam3`	Digital input (to the controller)	Beam 3 interrupted (15 px above the belt)
`Beam4`	Digital input (to the controller)	Beam 4 interrupted (21 px above the belt)
`Beam5`	Digital input (to the controller)	Beam 5 interrupted (27 px above the belt)
`Beam6`	Digital input (to the controller)	Beam 6 interrupted (33 px above the belt)
`Beam7`	Digital input (to the controller)	Beam 7 interrupted (39 px above the belt)
`Beam8`	Digital input (to the controller)	Beam 8 interrupted (45 px above the belt)

Inductive Sensor (metal) `inductive`

Proximity sensor for metal. **Digital** gives Detect. **Analog** gives Volts: 0 V with the part at the sensor, rising to 10 V (1000) at the edge of the range, and 1000 when nothing metal is in range.

Configuration

key	Setting	Default	Range / choices
`output`	Output	digital	digital, analog
`range`	Sensing range (px; 60 px = 1 m)	10	2 – 60, step 1

Signals (with the default settings)

Signal	Kind	Meaning
`Detect`	Digital input (to the controller)	Detecting: a conductive (metal) part is within range

Capacitive Sensor (any material) `capacitive`

Short-range proximity sensor for anything. **Digital** gives Detect. **Analog** gives Volts exactly like the inductive sensor, without the material filter.

Configuration

key	Setting	Default	Range / choices
`output`	Output	digital	digital, analog
`range`	Sensing range (px; 60 px = 1 m)	12	2 – 60, step 1

Signals (with the default settings)

Signal	Kind	Meaning
`Detect`	Digital input (to the controller)	Detecting: any solid part is within range

Color Sensor (R/B/G/Y banks) `color`

Four colour banks, one BOOL each.

No settings.

Signals (with the default settings)

Signal	Kind	Meaning
`Red`	Digital input (to the controller)	Bank 1: red
`Blue`	Digital input (to the controller)	Bank 2: blue
`Green`	Digital input (to the controller)	Bank 3: green
`Yellow`	Digital input (to the controller)	Bank 4: yellow

Vision Sensor `vision`

Recognises the part under it. **Digital**: one BOOL per ticked part. **Bits**: the part code as five binary inputs Bit0-Bit4 (Factory I/O's "All Digital"). **Numerical**: one INT with the part code (the number next to each part in the list; 0 = none or not ticked). **ID**: the serial number of the part in view plus its code, usable like a barcode. **Size**: the older Small/Medium/Large/Metal outputs. Tick **Triggered** for a Trigger output with Busy and Complete, like a real smart camera: the result then holds until the next trigger.

Configuration

key	Setting	Default	Range / choices
`mode`	Output	digital	digital, bits, numerical, id, size, camera tags only
`camera`	Camera module (its name in I/O Configuration; blank = wired signals only)		text
`housing`	Housing	Charcoal	Charcoal, Yellow, Blue, Silver
`detect`	Parts to Detect	cartonS, cartonM, cartonL, metalM	1 · Carton (S), 2 · Carton (M), 3 · Carton (L), 4 · Shipping Crate (XL), 5 · Tote, 6 · Metal Block (S), 7 · Metal Block (M), 8 · Metal Block (L), 9 · Raw Stock, Blue, 10 · Raw Stock, Green, 11 · Raw Stock, Metal, 12 · Housing Base, Blue, 13 · Housing Base, Green, 14 · Housing Base, Metal, 15 · Housing Lid, Blue, 16 · Housing Lid, Green, 17 · Housing Lid, Metal, 18 · Bottle (S), 19 · Can, 20 · Bottle (M), 21 · Bottle (L)
`trigger`	Triggered (adds Trigger input, Busy and Complete)	False	on / off
`time`	Inspection time (ms)	150	0 – 2000, step 10

Signals (with the default settings)

Signal	Kind	Meaning
`Detect`	Digital input (to the controller)	A part is in the field of view
`Pass`	Digital input (to the controller)	The part is one of the selected parts
`CartonS`	Digital input (to the controller)	Carton (S) in view
`CartonM`	Digital input (to the controller)	Carton (M) in view
`CartonL`	Digital input (to the controller)	Carton (L) in view
`MetalM`	Digital input (to the controller)	Metal Block (M) in view

RFID Reader `rfid`

Single-trigger command interface: set **Command**, pulse **Execute**; when **CommandID** increments the command has finished and **Status** says how it went (0 = success). Command 0 only checks for a tag, 1 reads the serial into ReadData, 2 reads the word at MemoryIndex, 3 writes WriteData there, 4 clears the memory. Two tags in range give status 2. Write a route code into a part at one station and sort on it at the next.

Configuration

key	Setting	Default	Range / choices
`range`	Reading range (px)	16	4 – 60, step 1
`time`	Command time (ms)	100	0 – 2000, step 10

Signals (with the default settings)

Signal	Kind	Meaning
`Command`	Analog output	Command to execute: 0 = is a tag available?, 1 = read serial number, 2 = read data, 3 = write data, 4 = clear all data
`Execute`	Digital output (from the controller)	Rising edge executes the command
`MemoryIndex`	Analog output	Tag memory index for read/write (0-15)
`WriteData`	Analog output	Data written by command 3
`CommandID`	Analog input	Number of commands executed so far; changes when the current one has finished
`Status`	Analog input	0 = success, 1 = no tag, 2 = too many tags, 3 = invalid index, 4 = invalid command
`ReadData`	Analog input	Result of command 1 (serial) or 2 (data)
`TagPresent`	Digital input (to the controller)	Exactly one tag is within range

Box Photoeye (LS10, boxes only) `preset of pe`

A diffuse eye that only reports boxes: LogixPro LS10 under the grinder chute

Places a Diffuse Sensor (photoelectric) with `range` = 12, `sees` = boxes only.

Broken-bottle Eye (LS3) `preset of pe`

A diffuse eye that only reports broken bottles

Places a Diffuse Sensor (photoelectric) with `range` = 14, `sees` = broken bottles.

Bottling

Fill Station (tube, large and small fill) `filler`

Fill station as on the LogixPro bottle line. **FillTube** lowers the tube (LS4 **TubeDown** when it arrives); with the tube down and a bottle under it, **LargeFill** or **SmallFill** pours at the fill rate. **LargeCharge** (LS5) and **SmallCharge** (LS6) come on when that valve has delivered its charge since it opened, so close the valve on the charge sensor. A small bottle holds 60 units and a large 100; pouring on past the brim overflows, a valve open with no bottle spills, and a broken bottle leaks instead of filling.

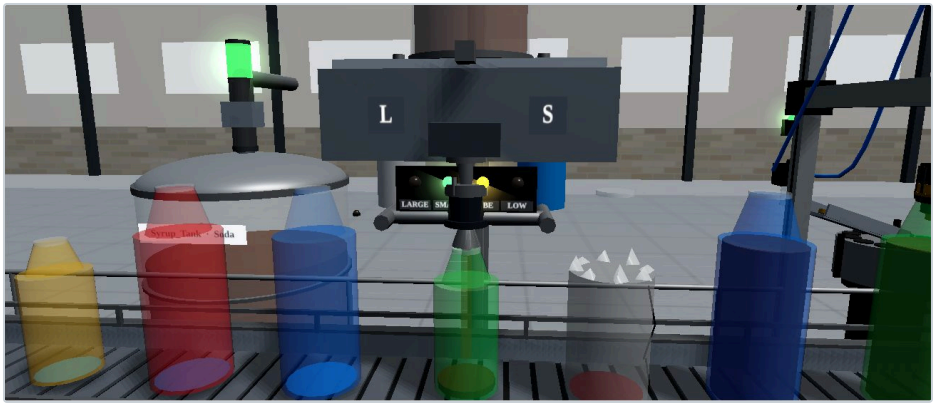


Figure 52. The Fill Station: tube, head tank with level, and the four-lamp indicator panel (L / S charge, tube down, done).

Configuration

key	Setting	Default	Range / choices
`rate`	Fill rate (units/s; a small bottle holds 60, a large 100)	80	10 – 400, step 10
`largeCharge`	Large charge (units)	100	10 – 400, step 10
`smallCharge`	Small charge (units)	60	10 – 400, step 10
`tubeTime`	Tube travel time (s)	0.4	0.1 – 2, step 0.1
`feed`	Fed from		text
`tank`	Head tank (units; refilled from the reservoir, pours from here)	300	50 – 2000, step 50

Signals (with the default settings)

Signal	Kind	Meaning
`FillTube`	Digital output (from the controller)	Lower the fill tube into the bottle
`LargeFill`	Digital output (from the controller)	Large fill valve
`SmallFill`	Digital output (from the controller)	Small fill valve
`TubeDown`	Digital input (to the controller)	Fill tube down (LS4)
`LargeCharge`	Digital input (to the controller)	Large charge delivered since the large valve opened (LS5)

Signal	Kind	Meaning
`SmallCharge`	Digital input (to the controller)	Small charge delivered since the small valve opened (LS6)
`TankLevel`	Analog input	Head tank level 0-100 %
`TankLow`	Digital input (to the controller)	Head tank below 15 %: the reservoir is not keeping up

Inline Cap Feeder (pneumatic capper + cap bowl) `capper`

A pneumatic capping machine as on the LogixPro line: **CapRam** drives the head down, **RamDown** (LS7) reports the bottom of the stroke, and the bottle under the chuck gets a cap from the chute on the way down. The chute holds *Chute holds* caps and starts with *Caps at start*; **CapPresent** says a cap is waiting, **CapLow** comes on at the *CapLow below* count, and **Feeder** runs the rotary magazine that refills the chute at *Cap feeder rate*. A cycle with an empty chute leaves the bottle open and is logged. **LampGreen** and **LampAmber** are the two indicator lights on the column: yours to program (ready / cycling, caps low…).

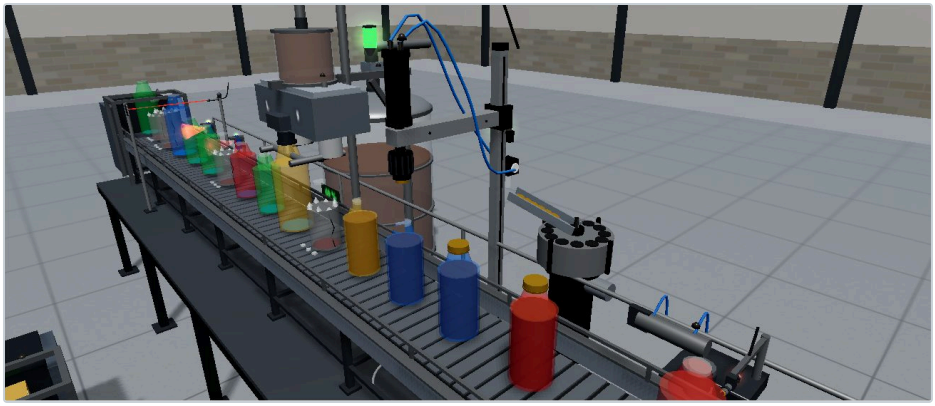


Figure 53. The Inline Cap Feeder: pneumatic ram, the cap chute with its reed switch, the rotary magazine that refills it, and the two column lamps.

Configuration

key	Setting	Default	Range / choices
`stroke`	Ram travel time (s)	0.3	0.1 – 2, step 0.1
`capsStart`	Caps in the chute at start	30	0 – 200, step 1
`chuteCap`	Chute holds (caps)	40	5 – 200, step 1
`lowAt`	CapLow below (caps)	8	1 – 100, step 1
`feedRate`	Cap feeder rate (caps/s)	3	0.5 – 20, step 0.5

Signals (with the default settings)

Signal	Kind	Meaning
`CapRam`	Digital output (from the controller)	Drive the capping head down
`Feeder`	Digital output (from the controller)	Run the rotary cap magazine (refills the chute)
`LampGreen`	Digital output (from the controller)	Green indicator on the column
`LampAmber`	Digital output (from the controller)	Amber indicator on the column
`RamDown`	Digital input (to the controller)	Head at the bottom of its stroke (LS7)
`CapLow`	Digital input (to the controller)	Chute at or below the CapLow count

Signal	Kind	Meaning
`CapPresent`	Digital input (to the controller)	A cap is waiting at the chuck

Inline Labeler (FLS-style dispenser, RH / LH, any applicator) `labeler`

A pressure-sensitive label dispenser in the FLS 204 mould: modular, *RH* or *LH* (which side of the lane it stands on, looking downstream), with an *Applicator* of your choice (tamp pad, wipe-on dispensing edge with a brush, blow-on non-contact, wrap-around roller), a label width of 5-233 mm and a web that follows the conveyor or runs at a fixed speed. **Ready** says the web is loaded and the head is home; **Error** is end of roll or a cycle with no label. The next station after the capper: **Apply** drives the pad in, **PadIn** reports the end of the stroke, and the bottle under the pad gets a label from the roll. The roll holds *Roll holds* labels and starts with *Labels at start*; **LabelPresent** says a label is peeled and waiting, **LabelLow** comes on at the *LabelLow below* count, and **Labeled** says the bottle standing under the pad already carries a label. With *Only label capped bottles* on, an open bottle is skipped and logged. **LampGreen** and **LampAmber** are yours to program. A photoeye set to *labeled bottles* checks the work downstream.

Figure 54. The Inline Labeler one stop past the capper: the roll on its spindle, the web down to the pad, and labeled bottles leaving it.

Configuration

key	Setting	Default	Range / choices
`hand`	Hand (side of the lane the dispenser stands on, looking downstream)	RH	RH, LH
`applicator`	Applicator	Tamp pad	Tamp pad, Wipe-on (dispensing edge + brush), Blow-on (non-contact), Wrap-around (roller)
`labelW`	Label width (mm, 5-233)	60	5 – 233, step 1
`web`	Web speed	Follows the conveyor	Follows the conveyor, Fixed 20 m/min, Fixed 40 m/min, Fixed 70 m/min
`stroke`	Apply time (s)	0.4	0.1 – 2, step 0.1
`labelsStart`	Labels on the roll at start	200	0 – 2000, step 10
`rollCap`	Roll holds (labels)	500	10 – 5000, step 10
`lowAt`	LabelLow below (labels)	25	1 – 500, step 1

key	Setting	Default	Range / choices
`labelColor`	Label colour	White	White, Cream, Blue, Red, Black
`capOnly`	Only label capped bottles (skips an open one and logs it)	True	on / off

Signals (with the default settings)

Signal	Kind	Meaning
`Apply`	Digital output (from the controller)	Start: drive the applicator in against the bottle
`LampGreen`	Digital output (from the controller)	Green indicator on the frame
`LampAmber`	Digital output (from the controller)	Amber indicator on the frame
`PadIn`	Digital input (to the controller)	Applicator at the end of its stroke (label pressed on)
`Ready`	Digital input (to the controller)	Dispenser ready: web loaded, no fault, not cycling
`Error`	Digital input (to the controller)	Dispenser fault: end of roll (empty) or a cycle with no label
`LabelLow`	Digital input (to the controller)	Roll at or below the LabelLow count
`LabelPresent`	Digital input (to the controller)	A label is peeled and waiting at the dispensing edge
`Labeled`	Digital input (to the controller)	The bottle under the applicator carries a label

Case Packer (top-load: infeed buffer, pick head, case lane beside the line) `packer`

A top-load case packer at the end of a line. Bottles that reach the end of the belt are taken into its **infeed** buffer (up to *Infeed buffer*); when it is full they wait on the belt at the backstop. A rising edge on **Drop** picks *Bottles per case* from the infeed (only when **GroupReady**) and places them in the case standing under the head on the case lane, *Case lane offset* px across the line. **CaseInPlace** is a case centred under the head within ±6 px; **CaseFull** is that case once packed. Drop with no case under the head, or onto a full case, and the bottles are lost. The case lane is an ordinary conveyor carrying cartons from an emitter: position the case with the program (a slow-down eye and a creep speed on a VFD is the usual way), stop it, drop, then run it out.

Configuration

key	Setting	Default	Range / choices
`perCase`	Bottles per case	4	1 – 12, step 1
`infeed`	Infeed buffer (bottles)	8	1 – 24, step 1
`cycle`	Pack cycle (s)	1.6	0.5 – 6, step 0.1
`lane`	Case lane offset across the line (px; negative = far side)	-108	-240 – 240, step 2

Signals (with the default settings)

Signal	Kind	Meaning
`Drop`	Digital output (from the controller)	Pack: the rising edge picks a case worth from the infeed and places it in the case under the head
`Ready`	Digital input (to the controller)	Head home, not cycling
`GroupReady`	Digital input (to the controller)	The infeed holds at least a case worth
`InfeedFull`	Digital input (to the controller)	Infeed buffer full: bottles arriving now wait on the belt
`InfeedCount`	Analog input	Bottles in the infeed buffer
`CaseInPlace`	Digital input (to the controller)	A case stands under the head (centre within ±6 px)
`CaseFull`	Digital input (to the controller)	The case under the head has been packed

Grinder (shredder under a raised belt, over a box conveyor) `grinder`

The shredder. Put it beside a **Raised Conveyor** (Height = Raised deck) with a pivot gate on the belt above it: the gate swings a broken bottle off the edge and it drops into the hopper, where it waits (**Detect**) until **Grinder** runs; each bottle is chewed between the shafts and the glass falls down the chute into the box on the conveyor 76 px below the hopper (a **Box Conveyor** with a **Box Feeder**). A box holds *Bottles per box*; **BoxFull** reports a full box under the chute and the level sensor beside the chute mouth gives **BoxLevel**, the number of bottles ground into the box so far (an analog input; the IN BOX display on the template panel shows it). Grinding with no box or a full box under the chute spills the glass and is logged. Put a **Box Photoeye** (boxes only) on the box conveyor under the chute for LS10.

Figure 55. The Grinder under a raised belt: hopper, shredder, chute, and the box conveyor below with its level sensor.

Configuration

key	Setting	Default	Range / choices
`perBox`	Bottles per box	4	1 – 30, step 1
`speed`	Grind time per bottle (s)	0.6	0.1 – 3, step 0.1

Signals (with the default settings)

Signal	Kind	Meaning
`Grinder`	Digital output (from the controller)	Run the grinder
`Detect`	Digital input (to the controller)	A bottle is waiting in the hopper
`BoxFull`	Digital input (to the controller)	The box under the chute holds its share
`BoxLevel`	Analog input	Level sensor on the chute: bottles ground into the box under it (0 with no box)

Dispenser (tank, valve, nozzle) `dispenser`

Configuration

key	Setting	Default	Range / choices
`rate`	Flow (units/s; a small bottle holds 60, a large 100)	60	10 – 400, step 10
`capacity`	Tank holds (units)	1000	100 – 10000, step 100
`start`	Starts at (%)	100	0 – 100, step 5
`refill`	Refill valve (adds Refill output; fills the tank at 4× the flow)	True	on / off
`empty`	Empty below (%)	5	0 – 50, step 1
`feed`	Fed from		text

Signals (with the default settings)

Signal	Kind	Meaning
`Valve`	Digital output (from the controller)	Open the valve: liquid runs out of the nozzle
`Refill`	Digital output (from the controller)	Refill the tank
`Level`	Analog input	Tank level 0-100 %
`Empty`	Digital input (to the controller)	Tank below the Empty level
`Flowing`	Digital input (to the controller)	Liquid is leaving the nozzle

Thumbwheel Switch (digits + Enter) `thumbwheel`

Digit wheels and an Enter button, like the BCD thumbwheel on the LogixPro panel. In Run mode tap the top half of a digit to turn it up and the bottom half to turn it down; **Value** reads the digits as one number and **Enter** is 1 while the button is held.

Configuration

key	Setting	Default	Range / choices
`digits`	Digits	4	1 – 6, step 1
`enter`	Enter button	True	on / off
`text`	Label	PRESET	text

Signals (with the default settings)

Signal	Kind	Meaning
`Value`	Analog input	The digits read as one number
`Enter`	Digital input (to the controller)	Enter button (momentary)

Bottle Feeder (LS1 edge, 15 % broken) `preset of emitter`

LogixPro’s bottle source: small and large bottles on each Emit edge, some of them broken

Places a Emitter (part source) with `parts` = ['bottle', 'bottleL'], `mode` = edge, `minTime` = 2.5, `maxTime` = 4, `broken` = 15.

Process

Supply Tote (an IBC of raw ingredient that tanks pump from) `supply`

A supply tote: an IBC of raw ingredient on its pallet, in its cage, with an outlet valve low on one side. Pipe it to a tank by naming it in that tank's *fed from* setting (a Mix Tank's *Inlet A* or *Inlet B*, a Tank's or Reservoir's *Inlet*, a filler's *Fed from*): the pump or valve on the other end draws from it, and when the tote runs empty the pump runs dry and nothing more goes in. **LowLevel** is the low switch; tick *Level transmitter* for a 0-100 % analog input and *Refill valve* to top it up from the plant header with the **Refill** output. In Run mode, click an empty tote to swap in a full one, as a forklift driver would.

Configuration

key	Setting	Default	Range / choices
`product`	Ingredient	Water	Water, Soda, Juice, Oil, Milk, Syrup, Concentrate
`capacity`	Holds (units)	10000	500 – 100000, step 500
`start`	Starts at (%)	100	0 – 100, step 5
`low`	Low level switch (%)	10	0 – 50, step 1
`locked`	Locked out (LOTO): its valve is locked shut, it gives nothing	False	on / off
`levelAi`	Level transmitter (adds Level analog input, 0-100 %)	False	on / off
`refill`	Refill valve from the plant header (adds Refill output)	False	on / off
`refillRate`	Refill flow (units/s)	200	10 – 5000, step 10

Signals (with the default settings)

Signal	Kind	Meaning
`LowLevel`	Digital input (to the controller)	Low level switch: 1 while the tote is at or below its low level

Reservoir (product supply for fillers) `reservoir`

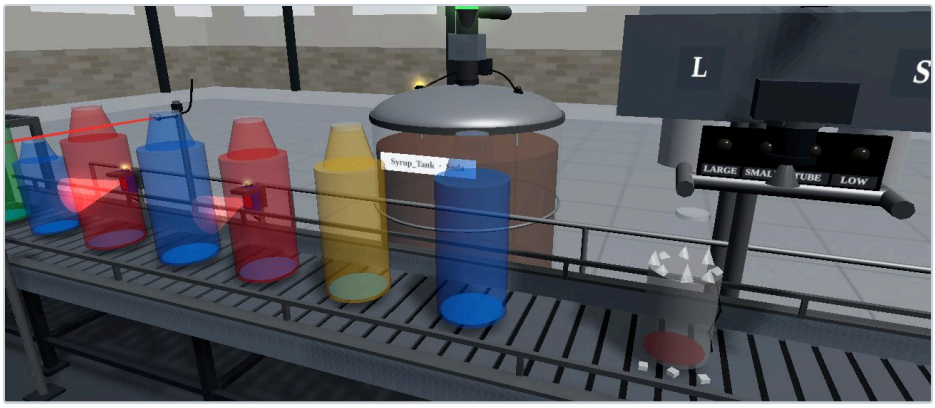


Figure 56. The Reservoir: dome, inlet valve, level transmitter and the high and low float probes; the pipe to the filler draws itself.

Configuration

key	Setting	Default	Range / choices
`product`	Product	Water	Water, Soda, Juice, Oil, Milk, Syrup, Concentrate
`capacity`	Holds (units; a small bottle takes 60, a large 100)	5000	500 – 100000, step 500
`start`	Starts at (%)	100	0 – 100, step 5
`inflow`	Inlet flow (units/s)	200	10 – 5000, step 10
`high`	High float (%)	90	50 – 100, step 1
`low`	Low float (%)	15	0 – 50, step 1
`drain`	Drain valve (adds Drain output)	False	on / off
`flowAi`	Flow meter (adds Flow analog input; the sandbox has four analog inputs)	False	on / off
`levelSig`	Level transmitter signal	0-100 % (INT)	0-100 % (INT), Raw counts 0-32767 (1769-IF4 style, 4-20 mA), Engineering units 4000-20000 (4-20 mA)
`source`	Inlet fed from		text

Signals (with the default settings)

Signal	Kind	Meaning
`Inlet`	Digital output (from the controller)	Inlet valve: fills the reservoir at the inlet flow
`Level`	Analog input	Level 0-100 % (bind it to a REAL tag for the unrounded value)
`HighFloat`	Digital input (to the controller)	High level float switch
`LowFloat`	Digital input (to the controller)	Low level float switch

Mix Tank (batch: pumps, flowmeters, heater, thermostat, mixer) `mixtank`

A batch mixing vessel in the mould of the LogixPro Batch Mix. Ingredients come in through **PumpA** (and **PumpB** with two inlets), each through a **flowmeter** that gives one pulse per *Units per pulse* as a square wave, so a CTU on its input counts the dose. **HiLevel** and **LoLevel** are probes, 1 while the liquid covers them; **Level** is the transmitter. The **Heater** under the tank heats while its element is covered (below LO it scorches the batch); the tank loses heat to the room, and cold ingredient pulls the temperature down. **Temp** is °F; the **Thermostat** is a high-limit switch that closes at its setpoint. The **Mixer** counts mixing time only while the liquid covers the impeller (above LO). **PumpOut** empties the tank through the outlet flowmeter; type the tank's name in a filler's *Feed* and the filler draws its product through that pump instead, so the filler only refills while PumpOut runs.

Configuration

key	Setting	Default	Range / choices
`product`	Product (what a filler fed from it puts in the bottles)	Juice	Water, Soda, Juice, Oil, Milk, Syrup, Concentrate
`capacity`	Holds (units)	2000	500 – 20000, step 100
`start`	Starts at (%)	0	0 – 100, step 5
`inlets`	Ingredient inlets	Two (A and B)	One (A), Two (A and B)
`rateA`	Pump A flow (units/s)	100	10 – 1000, step 10
`rateB`	Pump B flow (units/s)	100	10 – 1000, step 10
`rateOut`	Outlet pump flow (units/s)	150	10 – 1000, step 10
`pulse`	Flowmeters: units per pulse	20	1 – 500, step 1
`high`	HI level probe (%)	90	50 – 100, step 1
`low`	LO level probe (%)	10	0 – 50, step 1
`heat`	Heater: °F per second	3	0.5 – 20, step 0.5
`ambient`	Ambient and ingredient temperature (°F)	70	32 – 120, step 1
`setpoint`	Thermostat setpoint, high limit (°F)	180	80 – 250, step 1
`levelSig`	Level transmitter signal	0-100 % (INT)	0-100 % (INT), Raw counts 0-32767 (1769-IF4 style, 4-20 mA), Engineering units 4000-20000 (4-20 mA)
`spTag`	Panel and HMI: heater setpoint tag (a REAL the operator sets on the TEMP page)		text
`mode`	Inlet mode (interlocks the pumps)	Auto (follow the product)	Auto (follow the product), Mix A + B, A only, B only, Same liquid (B fills from A), Water only (rinse)

key	Setting	Default	Range / choices
`lockA`	Inlet A locked out (LOTO)	False	on / off
`lockB`	Inlet B locked out (LOTO)	False	on / off
`rcpRatio`	Recipe: A share the HMI aims for (%)	50	0 – 100, step 1
`rcpMix`	Recipe: mixing time (s)	10	0 – 600, step 1
`purgeRate`	Purge drain flow (units/s)	250	20 – 2000, step 10
`srcA`	Inlet A fed from		text
`srcB`	Inlet B fed from		text
`rateAi`	Flowmeters also report their rate on analog inputs (units/s; a REAL when bound to a REAL tag)	False	on / off

Signals (with the default settings)

Signal	Kind	Meaning
`PumpA`	Digital output (from the controller)	Ingredient A pump
`PumpB`	Digital output (from the controller)	Ingredient B pump
`PumpOut`	Digital output (from the controller)	Outlet pump: empties the tank, or feeds a filler piped to it
`Heater`	Digital output (from the controller)	Heater element under the tank
`Mixer`	Digital output (from the controller)	Mixer motor
`FlowA`	Digital input (to the controller)	Flowmeter A: one pulse per Units-per-pulse pumped in (square wave)
`FlowB`	Digital input (to the controller)	Flowmeter B: one pulse per Units-per-pulse
`FlowOut`	Digital input (to the controller)	Outlet flowmeter: one pulse per Units-per-pulse pumped out
`HiLevel`	Digital input (to the controller)	HI probe: 1 while the liquid covers it
`LoLevel`	Digital input (to the controller)	LO probe: 1 while the liquid covers it
`Thermostat`	Digital input (to the controller)	High-limit thermostat: closes at its setpoint
`Level`	Analog input	Level 0-100 % (bind it to a REAL tag for the unrounded value)
`Temp`	Analog input	Temperature, °F

Tank with valves & level transmitter `tank`

Configuration

key	Setting	Default	Range / choices
`inflow`	Inlet flow (%/s)	10	1 – 50, step 1
`outflow`	Outlet flow (%/s)	8	1 – 50, step 1
`mixer`	Mixer motor (adds output)	False	on / off
`high`	High float (%)	90	50 – 100, step 1
`low`	Low float (%)	10	0 – 50, step 1
`levelSig`	Level transmitter signal	0-100 % (INT)	0-100 % (INT), Raw counts 0-32767 (1769-IF4 style, 4-20 mA), Engineering units 4000-20000 (4-20 mA)
`product`	Product	Water	Water, Soda, Juice, Oil, Milk, Syrup, Concentrate
`capacity`	Holds (units: what it gives a filler or another tank)	1000	100 – 20000, step 100
`source`	Inlet fed from		text

Signals (with the default settings)

Signal	Kind	Meaning
`Inlet`	Digital output (from the controller)	Inlet valve
`Outlet`	Digital output (from the controller)	Outlet valve
`Level`	Analog input	Level 0-100 % (bind it to a REAL tag for the unrounded value)
`HighFloat`	Digital input (to the controller)	High level float switch
`LowFloat`	Digital input (to the controller)	Low level float switch

Operator

Push Button `pb`

Configuration

key	Setting	Default	Range / choices
`color`	Color	green	green, red, amber, yellow, orange, blue, white, black
`style`	Style	Flush	Flush, Extended, Mushroom, Square, Illuminated ring, Legend button
`size`	Size	Medium	Small, Medium, Large
`action`	Action	Momentary	Momentary, Maintained (push on, push off)
`text`	Legend text (Legend button style)	START	text
`nc`	Normally closed contact	False	on / off
`lamp`	Illuminated (adds Lamp output)	False	on / off

Signals (with the default settings)

Signal	Kind	Meaning
`Pressed`	Digital input (to the controller)	NO contact: 1 while pressed

Emergency Stop `estop`

Configuration

key	Setting	Default	Range / choices
`style`	Style	Round yellow plate	Round yellow plate, Square yellow plate, No plate
`size`	Size	Medium	Small, Medium, Large
`lamp`	Illuminated (adds Lamp output)	False	on / off

Signals (with the default settings)

Signal	Kind	Meaning
`OK`	Digital input (to the controller)	NC contact: 1 while healthy, 0 when pressed

Selector Switch (2-position) `sel2`

Configuration

key	Setting	Default	Range / choices
`handle`	Handle	Lever	Lever, Round knob, Key
`color`	Handle color	black	black, red, green, blue, yellow, white
`size`	Size	Medium	Small, Medium, Large

Signals (with the default settings)

Signal	Kind	Meaning
`Pos2`	Digital input (to the controller)	1 in the right-hand position

Selector Switch (3-position) `sel3`

Configuration

key	Setting	Default	Range / choices
`handle`	Handle	Lever	Lever, Round knob, Key
`color`	Handle color	black	black, red, green, blue, yellow, white
`size`	Size	Medium	Small, Medium, Large

Signals (with the default settings)

Signal	Kind	Meaning
`Left`	Digital input (to the controller)	1 in the left position
`Right`	Digital input (to the controller)	1 in the right position

Potentiometer (analog) `pot`

No settings.

Signals (with the default settings)

Signal	Kind	Meaning
`Value`	Analog input	0-100

Indicator Light / LED `light`

Configuration

key	Setting	Default	Range / choices
`color`	Color	green	green, red, amber, yellow, orange, blue, white
`style`	Style	Pilot light	Pilot light, Panel LED, Square indicator, Dome beacon, Light bar, Legend plate
`size`	Size	Medium	Small, Medium, Large
`effect`	When on	Steady	Steady, Flashing, Fast flash, Pulsing
`text`	Legend text (Legend plate style)	RUN	text

Signals (with the default settings)

Signal	Kind	Meaning
`On`	Digital output (from the controller)	Lamp

Stack Light `stack`

Configuration

key	Setting	Default	Range / choices
`tiers`	Segments	Red / Amber / Green	Red / Amber / Green, Red / Amber / Green / Blue, Red / Amber / Green / Blue / White, Red / Green, Red / Amber
`lens`	Lens	Ribbed	Ribbed, Smooth, Frosted, LED ring
`mount`	Mount	Pole	Pole, Tall pole, Direct
`size`	Size	Medium	Small, Medium, Large
`flash`	Built-in flasher	None	None, Red flashes, Red and amber flash, All flash, Red strobes
`buzzer`	Buzzer (adds output)	False	on / off

Signals (with the default settings)

Signal	Kind	Meaning
`Red`	Digital output (from the controller)	Red segment
`Amber`	Digital output (from the controller)	Amber segment
`Green`	Digital output (from the controller)	Green segment

Alarm Horn `horn`

No settings.

Signals (with the default settings)

Signal	Kind	Meaning
`On`	Digital output (from the controller)	Horn

Process HMI (operator terminal: a live faceplate of a tank) `hmi`

A Process HMI: the operator terminal a tank would have beside it, showing a live faceplate of the vessel you pick in *Shows* (or the nearest one). The faceplate is drawn the way plant HMIs are: grey, dark text, colour only for what is running or wrong. It shows the vessel with its level, HI and LO marks, the mixer and heater, the level as a number, the temperature and the high limit, a one-minute trend of temperature and level, the A:B ratio, the mixing time, the flows, the pumps, heater and mixer (green running, orange heating), the state, and an alarm line. It reads the tank directly and has no I/O of its own; to show a tag of your own, use a Digital Display. In 3D it stands on a pedestal or hangs on a wall bracket.

Configuration

key	Setting	Default	Range / choices
`shows`	Shows		text
`spTag`	Heater setpoint tag (REAL; blank = the tank's own)		text
`startTag`	START button tag (BOOL, pulsed while touched)		text
`stopTag`	STOP button tag (BOOL, pulsed while touched)		text
`runTag`	Running indicator tag (BOOL: lights START)		text
`ratioTag`	Recipe A share tag (REAL, %)		text
`mixTag`	Recipe mixing time tag (REAL, s)		text
`size`	Screen	15 inch	7 inch, 10 inch, 15 inch, 21 inch
`mount`	Mounting (3D)	Pedestal	Pedestal, Wall bracket

No signals.

Digital Display (any tag) `display`

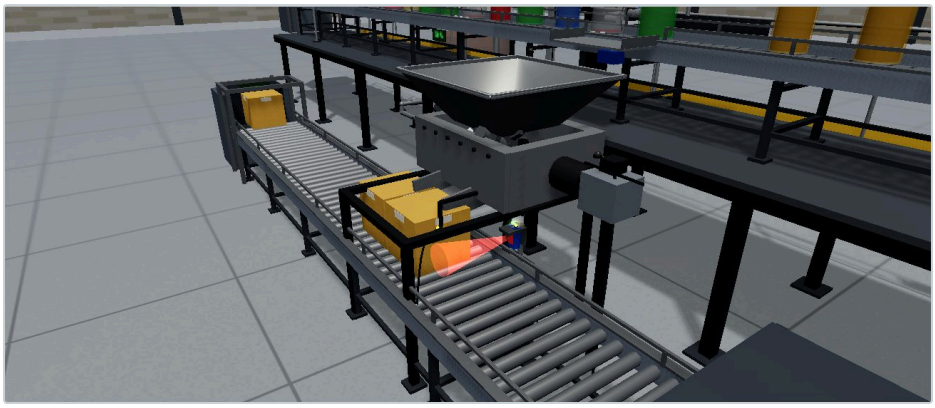


Figure 57. A Digital Display on the console shows any tag; here the IN BOX count from the grinder's box level sensor.

Configuration

key	Setting	Default	Range / choices
`tag`	Tag to show (blank = the wired analog output)		text
`format`	Show as	Auto	Auto, Seconds (from ms), mm:ss (from ms), On / Off, Hex
`decimals`	Decimal places (REAL and seconds)	1	0 – 4, step 1
`units`	Units shown after the value		text
`chars`	Width (characters)	6	4 – 16, step 1
`type`	Display type	LED 7-segment	LED 7-segment, LCD backlit, VFD, Plain text
`color`	Digit color	green	green, red, amber, yellow, orange, blue, cyan, white
`housing`	Housing	Black	Black, Grey, Stainless steel, White, Blue
`size`	Size	Medium	Small, Medium, Large
`caption`	Caption on the bezel (optional)		text
`hi`	Alarm color above this value (blank = never)		text
`lo`	Alarm color below this value (blank = never)		text
`alarm`	Alarm color	red	red, amber, yellow, white

Signals (with the default settings)

Signal	Kind	Meaning
`Value`	Analog output	Number to show

Communications

TCP Client Socket (via bridge) `socket`

Configuration

key	Setting	Default	Range / choices
`host`	Target IP / host	192.168.1.50	text
`port`	Target port	5000	1 – 65535, step 1
`term`	Terminator	CRLF	none, CR, LF, CRLF
`message`	Message on Send ({v} = SendValue)	T1	text
`m1`	Match 1 (sets Match1 when received text contains it)	PASS	text
`m2`	Match 2	FAIL	text
`m3`	Match 3		text
`m4`	Match 4		text
`latch`	Latch matches until the next message	False	on / off

Signals (with the default settings)

Signal	Kind	Meaning
`Connected`	Digital input (to the controller)	TCP connection is up
`Error`	Digital input (to the controller)	Bridge or connection fault
`Match1`	Digital input (to the controller)	Received text matched string 1
`Match2`	Digital input (to the controller)	Received text matched string 2
`Match3`	Digital input (to the controller)	Received text matched string 3
`Match4`	Digital input (to the controller)	Received text matched string 4
`RxValue`	Analog input	First number in the last received message
`RxCount`	Analog input	Messages received
`Send`	Digital output (from the controller)	Rising edge sends the message
`SendValue`	Analog output	Number substituted for {v}

Panels

Panel / Enclosure `panel`

Configuration

key	Setting	Default	Range / choices
`style`	Style	Enclosure door	Enclosure door, Flat plate, Push button station, Operator console
`finish`	Finish	ANSI 61 grey	ANSI 61 grey, Light grey (RAL 7035), Stainless steel, White, Black, Beige, Safety yellow, Blue
`w`	Width (px)	320	60 – 1400, step 10
`h`	Height (px)	220	60 – 900, step 10
`title`	Title plate (blank for none)	CONTROL PANEL	text
`plate`	Title plate	White on black	White on black, Black on white, Black on yellow, White on red, White on blue, White on green, Black on aluminium
`carry`	Devices on the panel move with it	True	on / off
`names`	Show device name tags on this panel	True	on / off

No signals.

Nameplate (engraved legend) `label`

Configuration

key	Setting	Default	Range / choices
`text`	Text	START	text
`text2`	Second line (optional)		text
`plate`	Colors	White on black	White on black, Black on white, Black on yellow, White on red, White on blue, White on green, Black on aluminium
`size`	Size	Medium	Small, Medium, Large
`screws`	Fixing screws	False	on / off

No signals.

14 Operating the Process HMI

A tank on a real plant has a screen beside it. The operator reads the vessel at a glance, sets a temperature, and sees what went wrong last night. Logix Connect has two such screens. The **Mix Tank** carries its own local panel on the shell, and the **Process HMI** is a separate operator terminal you place anywhere and point at any vessel. Both show the same faceplate, both are touch screens (a mouse click is a touch), and both work in the top view and in 3D. This chapter is the operating manual: what every part of each page means, how to change a value, and a worked example where the operator's setpoint drives the heater through a REAL tag.

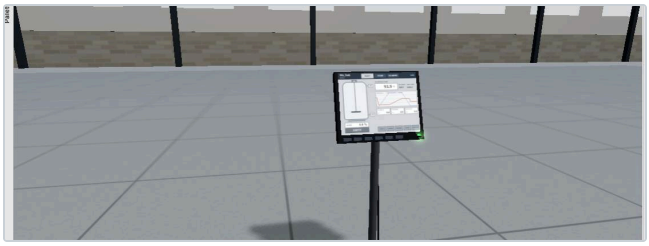


Figure 58. The Process HMI on its pedestal in 3D, a 15-inch terminal.

Figure 59. The mix tank's own panel, on standoffs from the shell.

Placing an HMI and connecting it

1. Program mode. In the palette, *Operator* → **Process HMI**; click the floor to place it.
2. In its properties, **Shows**: pick the vessel (any Mix Tank, Tank, Reservoir or Supply Tote). Left blank, it shows the nearest one.
3. **Heater setpoint tag**: the controller tag the TEMP page writes, normally a REAL. Left blank, the HMI uses the tank's own *Panel and HMI: heater setpoint tag*, so the panel and the HMI set the same tag.
4. **START button tag**, **STOP button tag** and **Running indicator tag**: optional; with them set, START and STOP appear on the MAIN page (below).
5. **Screen**: 7, 10, 15 (the default) or 21 inch. **Mounting**: on a pedestal or a wall bracket (3D).

The HMI has no I/O of its own: it reads the vessel directly and writes only the setpoint tag and the tank's high limit. Its properties end in a **Connections** section, like every part of the liquid family: what it shows and which tag it writes. On the tank, the same section lists every HMI showing it, and on each tote, tank, reservoir, mix tank, filler and dispenser it lists what feeds the part, what it feeds, its product, and whether each link is moving liquid right now. Every name is a link: click it to open that part.

Figure 60. Supply_A: it feeds the mix tank's inlet A, and it holds water.

Figure 61. Mix_Tank: inlets from the two totes, the outlet to the floor drain, its product, shown on Tank_HMI.

Touching the screen

The screen answers in **Run mode**. In the top view, click it; in 3D, click it where it hangs, on the HMI or on the tank's panel. In Program mode a click selects the part instead, so you can move it and edit its settings. The page you are on is remembered for each screen. Across the top are the page tabs, **MAIN**, **TEMP** (on a vessel with a temperature), **RECIPE** (on a mix tank) and **ALARMS**. A red dot on ALARMS means there is an active alarm.

The MAIN page

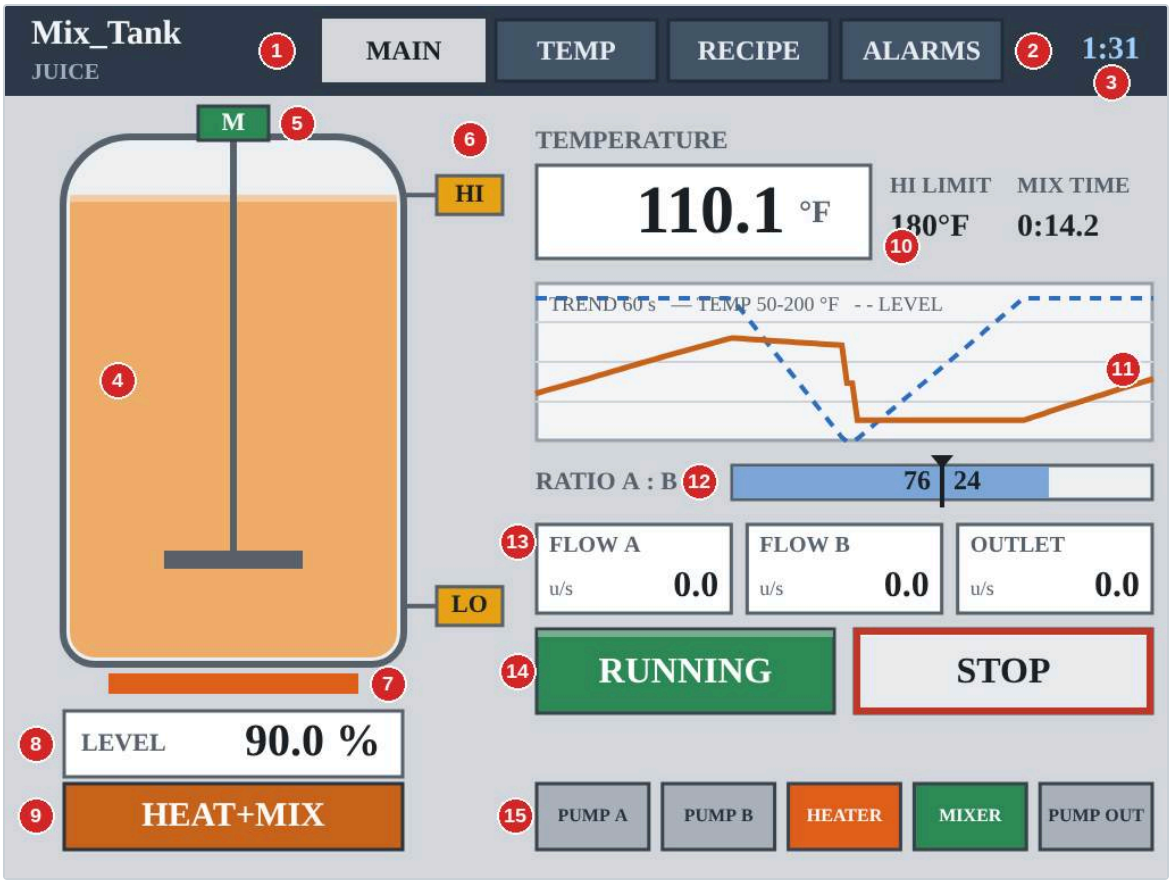


Figure 62. The MAIN page of the Level 7 mix tank during HEAT+MIX, with the batch running from START.

	What it is	What it tells you
1	Name and product	Which vessel this is and what it holds.
2	Page tabs	Touch to flip pages. The one you are on is light. A red dot on ALARMS: an alarm is active.
3	Run clock	Machine time since the run started (m:ss).
4	The vessel	The liquid at its level, in its colour. In the mix tank, clear and pale while the ingredients go in, cloudy as it mixes.
5	Mixer	Green and turning while the mixer runs.
6	HI and LO marks	At the probe heights; amber while the liquid covers them.
7	Heater	Orange while the heater is on.
8	Level	The level in percent.
9	State	FILLING (blue), HEAT+MIX (orange), MIXING, DRAINING (green), HOLDING, EMPTY (grey), PURGING (purple).
10	Temperature, high limit, mix time	The temperature (red once the high limit is made), the thermostat setting, and the mixing time of this batch. Touch the temperature to go to the TEMP page.
11	Trend	The last 60 s: temperature solid orange (50-200 °F), level dashed blue (0-100 %).

	What it is	What it tells you
12	Ratio	The share of A and B in the batch now in the tank. The black marker is the A share set on the RECIPE page.
13	Flows	What each flowmeter reads, units a second, blue while moving.
14	START and STOP	Touch pushbuttons on the START and STOP tags (when they are set). START reads RUNNING, green, while the Running tag is on.
15	Equipment	Each pump, the heater and the mixer: grey stopped, green running, orange heating. A pump on a locked inlet reads A LOCKED or B LOCKED, in red.

The faceplate is drawn the way plant HMIs are (the ISA-101 style): a grey screen and dark text, with colour kept for what is running and what is wrong. A lit heater or a red alarm line stands out because nothing else is bright. When there is an alarm, a red line appears above the equipment buttons; touch it to go to ALARMS.

The TEMP page

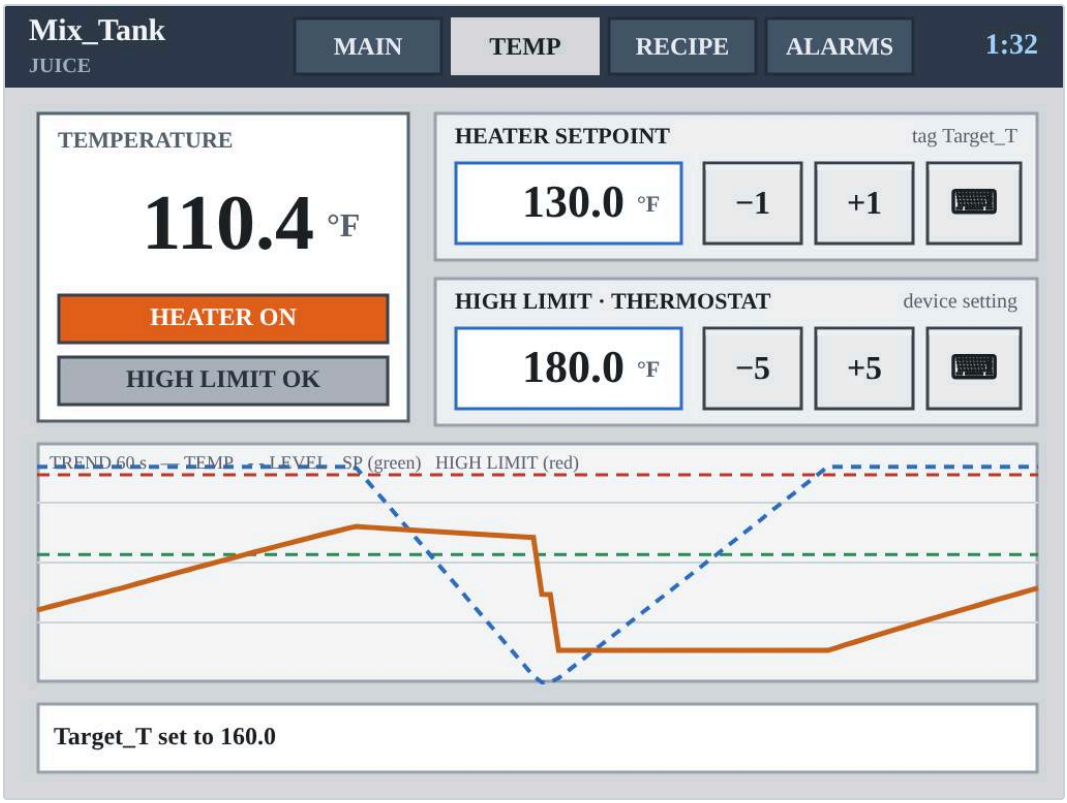


Figure 63. The TEMP page: the temperature, the heater and high-limit status, the two setpoint cards, the trend with the setpoint (green) and the high limit (red), and the message line.

The left card shows the temperature and two status bars: **HEATER ON / OFF** (what the heater output is doing) and **HIGH LIMIT OK / MADE** (the thermostat switch). The two cards on the right are the controls.

- **HEATER SETPOINT** is the controller tag named under the card. Writing it is all the HMI does; what the heater does with it is up to your program, as in the worked example below. **−1** and **+1** step it by one degree. Touching the value, or the keypad button, opens the keypad. The value is kept between 32 and 250 °F.
- **HIGH LIMIT · THERMOSTAT** is the tank's own thermostat setting, the temperature at which its Thermostat input closes. **−5** and **+5** step it, or type it, between 80 and 250 °F. This is the hardware safety, not the control: the program should only ever use it to switch the heater off.

The trend draws both as dashed lines over the temperature, so you can watch the heater hold the setpoint and see how much margin is left below the limit. The message line at the bottom confirms each change (*Heat_SP set to 147.5*), and every change is also written in the Simulator Log with the HMI's name.

Typing a value with the keypad

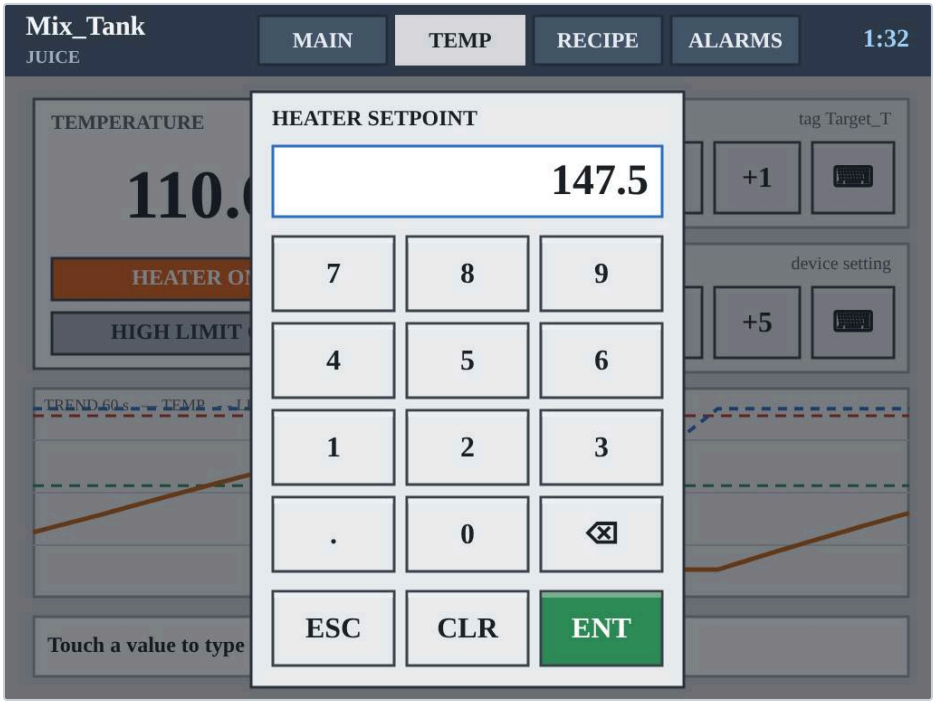


Figure 64. The keypad over the TEMP page, with 147.5 typed.

1. Touch the value box (or the keypad button) of the card you want to change.
2. Touch the digits. **.** adds the decimal point (once), ⌫ deletes the last digit, **CLR** clears them all.
3. **ENT** writes the value and closes the keypad. **ESC** closes it without changing anything.

If the program writes the same tag on every scan, your value lasts one scan and the message line says so in red: *The program wrote Target_T back to 150.0: a rung writes it every scan*. The HMI is not broken: the tag has two writers, and the program always writes last. Give the operator a tag of their own, as the worked example does.

The ALARMS page

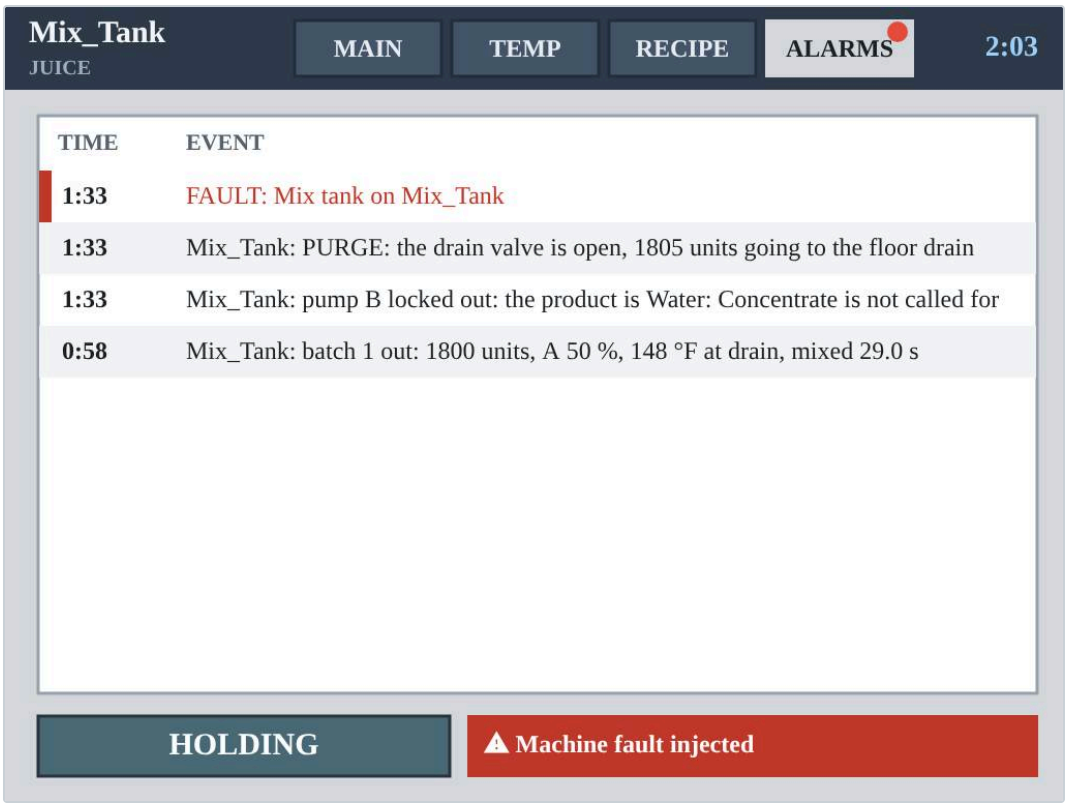


Figure 65. The ALARMS page after an injected pump fault: the newest first, a red bar on anything abnormal, the state and the active alarm at the bottom.

Every Simulator Log line that names this vessel, newest first, with the machine time: batches out, operator changes, pumps running dry, overflows, scorches and injected faults. Anything abnormal has a red bar and red text. The bottom line shows the state and the alarm that is active now.

START and STOP on the screen

Give the HMI a **START button tag** and a **STOP button tag** (BOOLs), and its MAIN page shows two pushbuttons above the alarm line. A touch pulses the tag: on for a quarter of a second of machine time, then off, so the program sees it for several scans, exactly like a momentary pushbutton wired to an input. Give it a **Running indicator tag** too and START lights green and reads RUNNING while that tag is on. Every press goes in the Simulator Log.

The program has to listen for them, and the plant way is in parallel with the hardwired buttons, never instead of them. Level 7 is set up like this: `HMI_Start` and `HMI_Stop` are ordinary BOOL tags (not aliases of inputs, since a real HMI cannot write an input), and the run rung takes either START and either STOP:

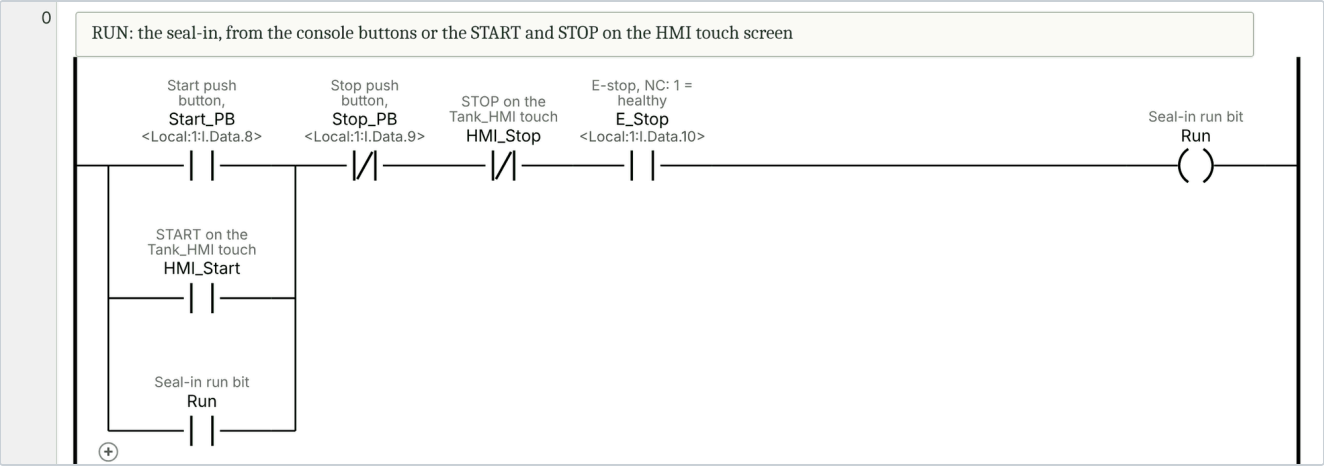


Figure 66. Level 7, rung 0: the seal-in takes the console Start_PB or the HMI's HMI_Start, and drops out on the console Stop_PB or the HMI's HMI_Stop. E_Stop stays hardwired.

The Operator View

The **Operator View** is the same screen as a tab of its own, full size, for running the machine without the scene: **Window → Open Operator View**, the *Operator View* entry under the simulator in the organizer, or **HMI** in the phone's bottom bar. Beside the screen (below it on a phone) is a **control panel** of real-looking pushbuttons:

- **RUNNING** lamp: green while the Running tag is on.
- **ALARM** lamp: red while the tank has an active alarm.
- **START** and **STOP**: pushbuttons that pulse the START and STOP tags, the same as the buttons on the screen. Under each is the tag it writes, in red if there is no such tag.
- A message line: what the last press or change did.

The bar along the top sets what the view shows and its four tags: *Shows* (the mix tank, or the first tank, when blank), *START tag*, *STOP tag*, *Running tag* and *Setpoint tag*. They are kept with the scene and saved with the project; on Level 7 they are already set to Mix_Tank, HMI_Start, HMI_Stop, Run and Target_T. The screen works exactly like the HMI's: touch the tabs, the values and the keypad. The buttons work in Run mode; in Program mode they say so and do nothing.

Figure 67. The Operator View on the PC during Level 7, just after START on the control panel: the RUNNING lamp is lit, START on the screen reads RUNNING, pump A is filling the tank, and the message line confirms the press.

Figure 68. The same view on a phone: the screen on top, the control panel below, HMI selected in the bottom bar.

> **TRY IT: THREE WAYS TO START**
>
> On Level 7 with the reference program, start the batch from the console Start_PB in the top view, stop it with STOP on Tank_HMI, start it again from the Operator View, and stop it from the console. One seal-in rung, three operator stations. Now delete the HMI_Start branch from rung 0 and try START on the Operator View again: the press is logged, the tag pulses (watch it in Quick Watch), and nothing happens, because no rung reads it. An HMI only asks; the program decides.

Recipe mode: what goes in, what stays out, and purge

A real batch tank is never trusted to the program alone to keep the wrong liquid out. When the tank is making water, the concentrate pump must not run, whatever a rung asks for. Oil must never get into milk or water, and water must never get into oil. The mix tank has these interlocks built in, the way a plant puts

them in the pump starter circuit. A pump on a **locked** inlet does not run, even with its output on. The first time the program asks for it, the Simulator Log says which pump and why, and the MAIN page shows **A LOCKED** or **B LOCKED** in red.

The **RECIPE** page is where the operator sets the batch up before START.

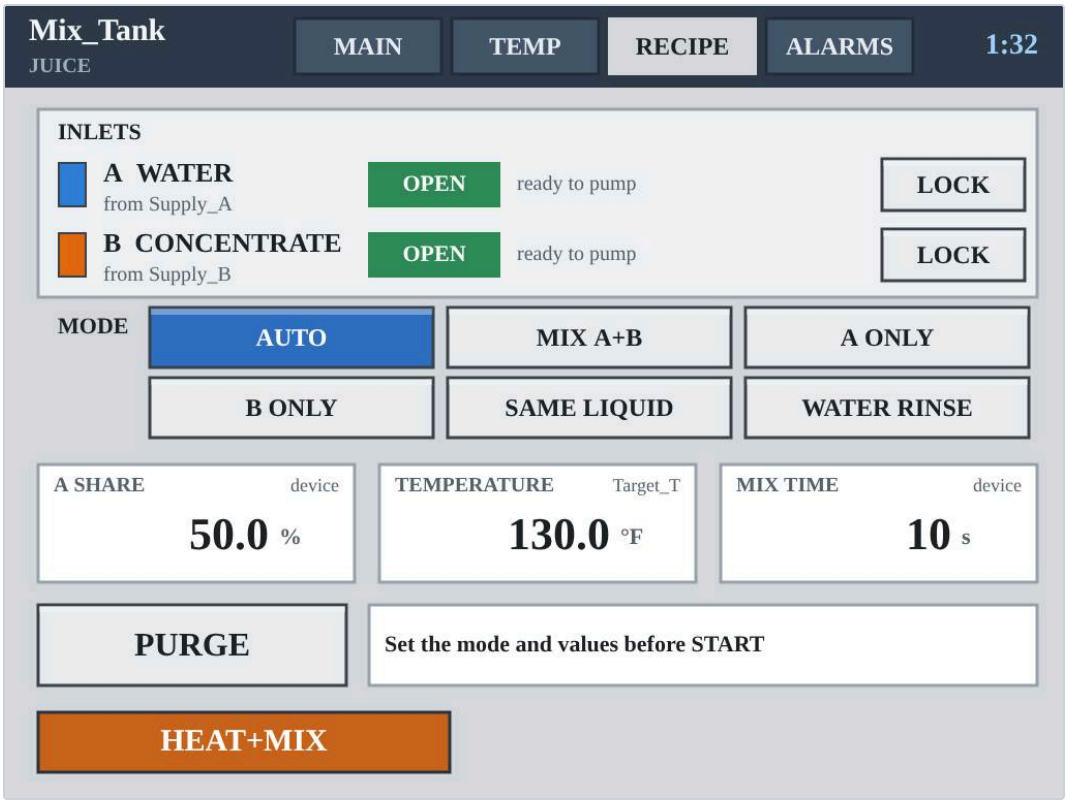


Figure 69. The RECIPE page on Tank_HMI during Level 7: water on inlet A and concentrate on inlet B, both open, mode AUTO, and the recipe values for the batch.

Part of the page	What it does
INLETS	One row per inlet: the colour and name of the liquid it carries, where it is piped from, **OPEN** (green) or **LOCKED** (red) with the reason, and a LOCK / UNLOCK button.
MODE	Which inlets this batch uses (the table below). The one in force is blue.
A SHARE, TEMPERATURE, MIX TIME	Touch a card for the keypad. The temperature is the heater setpoint tag, as on TEMP. The A share and the mixing time are kept on the tank, and written to the Ratio tag and the Mix time tag when those are set on the Process HMI or the Operator View.
PURGE	Drain the tank to the floor drain now (below).
Message line, state, alarm	What the last touch did, what the tank is doing, and the active alarm.

The six modes:

Mode	Inlet A	Inlet B	Use it for
AUTO	runs unless the product is what B carries and A carries something else	the same rule the other way	The default. Set the tank's Product to Water and the concentrate inlet locks; set it to Juice (what Level 7 makes) and both run.
MIX A+B	runs	runs	Two liquids mixed to the A share.
A ONLY	runs	locked	One liquid, from inlet A.
B ONLY	locked	runs	One liquid, from inlet B.
SAME LIQUID	runs	runs from A's source	Both pumps on the same liquid: inlet B draws through a crossover from whatever feeds inlet A, so the tank fills twice as fast. Inlet B's own tote is not touched.
WATER RINSE	runs if it carries water	runs if it carries water	Running the tank with water only, to rinse it between products.

On top of the mode come three locks that no mode lifts:

- **Contamination.** Oil and the water-based liquids (water, juice, milk, soda, concentrate) never meet. An inlet carrying oil is locked when the product is water-based, and an inlet carrying a water-based liquid is locked when the product is oil. The tank also remembers what is in it: until it has been emptied or purged, an inlet that would put the other kind on top is locked with *purge it first*.
- **Lockout / tagout.** LOCK on the page (or *Inlet A locked out* and *Inlet B locked out* in the tank's properties) locks that inlet until someone unlocks it. A Supply Tote has *Locked out* too: its valve gives nothing, and every pump drawing from it is locked.
- **Purging.** While the tank purges, both inlets are locked.

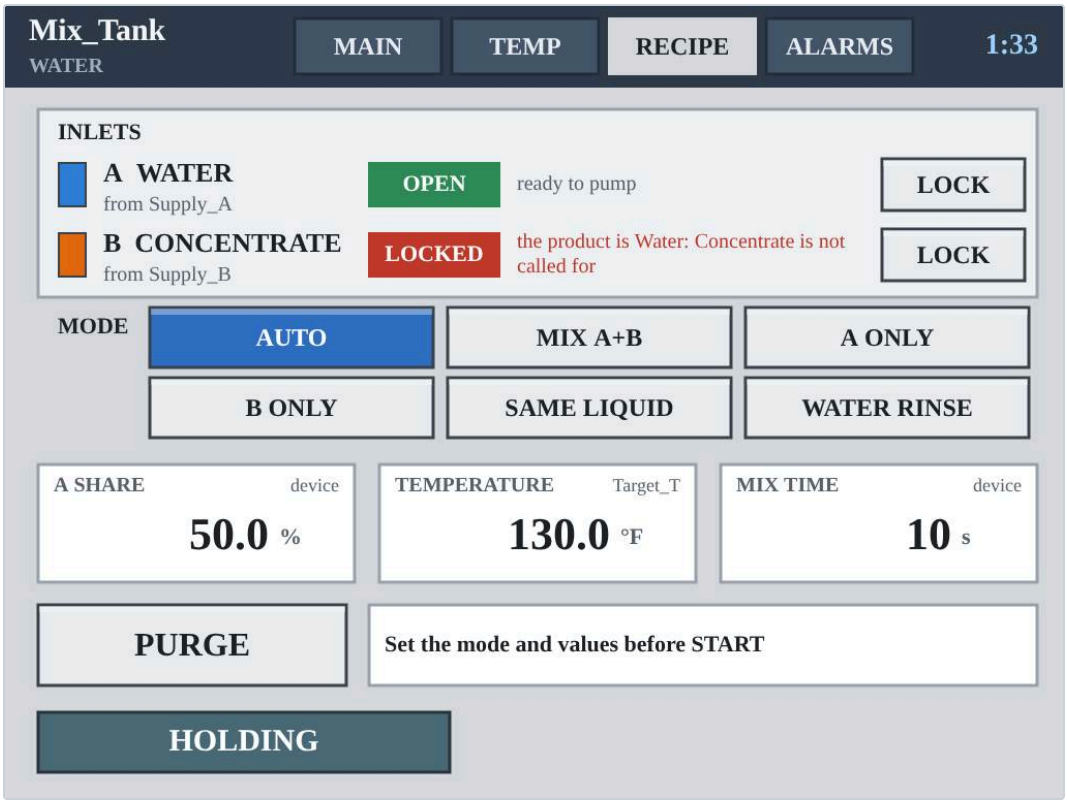


Figure 70. The tank's Product set to Water with the mode on AUTO: the concentrate inlet is locked and says why. Pump B will not run, even though the program has PumpB on.

PURGE opens the drain valve on the bottom of the tank and empties it to the floor drain on demand. It asks first. It is refused while the process is running (the Running tag is on: stop it first) and when the tank is already empty. While it purges, the state reads **PURGING**, the flows show PURGE, and both inlets are locked. When the tank is empty the valve closes and the batch is discarded: it is not counted or graded.

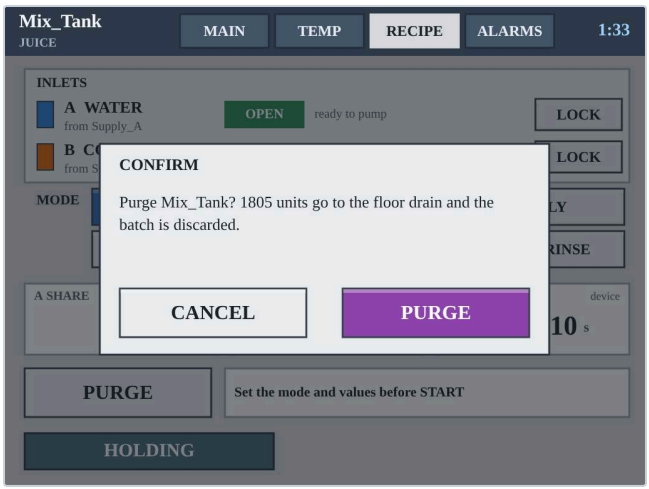


Figure 71. PURGE asks first, with how much will go to the drain.

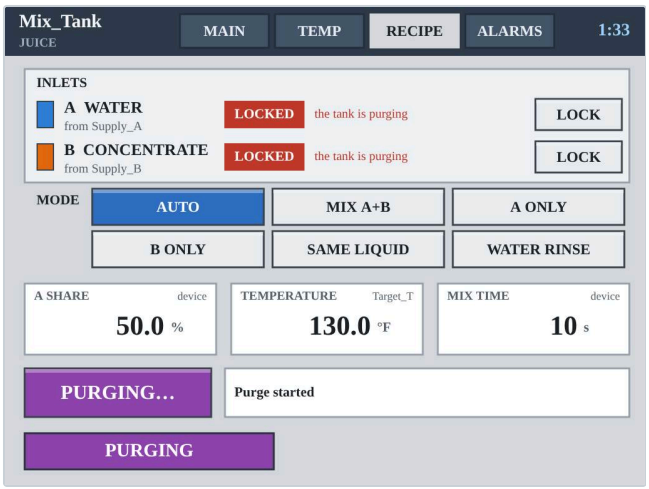


Figure 72. Purging: both inlets locked, the state PURGING.

TRY IT: WATER WHEN WATER IS CALLED FOR

On Level 7 with the reference program, open the Operator View and the RECIPE page. In Program mode set Mix_Tank's *Product* to Water and go back to Run mode: inlet B goes LOCKED. Press START. Step 1 doses water as before, but in step 2 PumpB is on and nothing comes in. Watch FLOW B stay at 0.0, the Simulator Log line, and the tank never reaching HI, so the sequence waits in step 2. The program asked; the interlock said no. Now press STOP, touch **SAME LIQUID** and START again: inlet B runs water from Supply_A and the tank fills. Finish by pressing STOP, then PURGE, then CANCEL; press PURGE again and confirm. Set the product back to Juice and the mode to AUTO.

TRY IT: LOCK OUT A TOTE

Select Supply_B and tick *Locked out*. Run the level: inlet B reads *Supply_B is locked out*, and the RECIPE page cannot unlock it, because the lock is on the tote, not the tank. That is how lockout/tagout works on a real line. Untick it to put the line back.

Worked example: the operator sets the heater

A mix tank, an HMI and five rungs. The operator types the temperature on the HMI; the program heats to it with a 2 °F band and never past the thermostat.

1. File → New Project… → **Empty sandbox**.
2. Place a **Mix Tank**. Set *Ingredient inlets* to One (A), *Starts at* 80 %, and *Panel and HMI: heater setpoint tag* to `Heat_SP`.
3. Place a **Process HMI** beside it. *Shows* can stay blank (the nearest tank is the mix tank).
4. Make the tags: `Heat_SP` REAL (the operator's setpoint), `Heat_Lo` REAL, `Heat_On` BOOL, and alias tags for the tank's `Temp` (Tank_Temp), `LoLevel` (Lo_Level), `Thermostat`, `Heater` and `Mixer`.
5. Enter the five rungs below, Verify, Run mode.

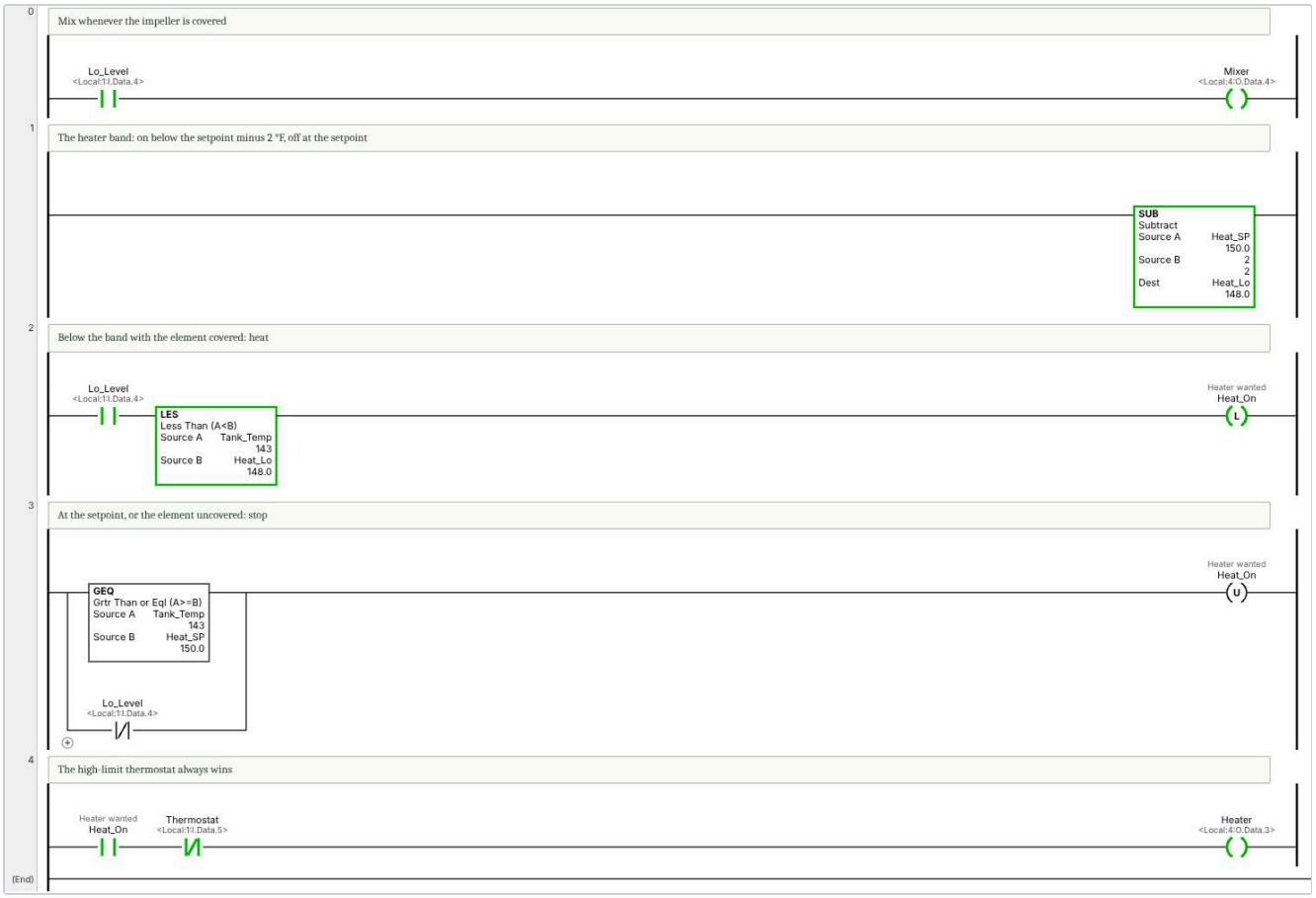


Figure 73. The five rungs, online with Heat_SP at 150.0: the band bottom is 148.0, the tank is at 143 °F, so Heat_On is latched and the heater is on.

- **Rung 0** runs the mixer whenever the impeller is covered.
- **Rung 1** works out the bottom of the band, `Heat_SP − 2`, every scan, so a new setpoint takes effect at once.
- **Rung 2** latches `Heat_On` below the band, but only with the element covered.
- **Rung 3** unlatches it at the setpoint, or as soon as LO uncovers. The two-rung latch is the hysteresis: between 148 and 150 °F the heater keeps doing what it was doing.
- **Rung 4** drives the output, and `XIO(Thermostat)` means the high limit always wins.

Run it. On the TEMP page the setpoint reads 0.0 and the heater stays off: the program does exactly what the operator asked. Touch the value, type 150, ENT. HEATER ON lights, the temperature climbs on the trend toward the green line, and the heater cycles between 148 and 150 °F from then on.

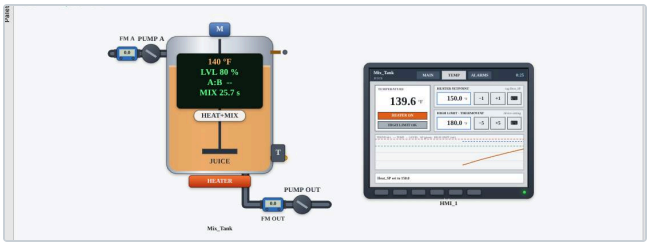


Figure 74. The example in the top view: the mix tank and the HMI on its TEMP page.

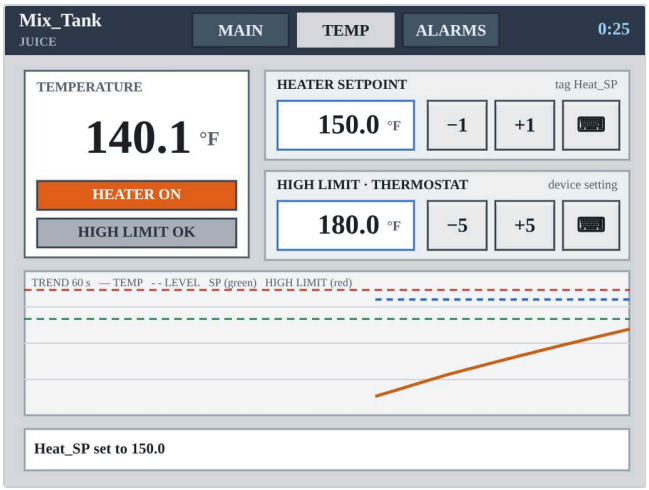


Figure 75. The HMI close up: setpoint 150.0, the heater on, the temperature climbing to the green line.

TRY IT: THE SETPOINT ABOVE THE LIMIT

With the example running, type 190 into HEATER SETPOINT while the high limit is 180. The temperature climbs to 180 and stops there. HIGH LIMIT MADE turns red, the heater drops out, comes back as the tank cools below the limit, and drops out again. The program still wants 190; the thermostat says no. That is what a high limit is for. Now lower the limit to 160 on the same page and watch it take over at 160.

TRY IT: TWO WRITERS, ONE TAG

Open Level 7 and its HMI (Tank_HMI points at the recipe tag Target_T). On the TEMP page, type 160. The message line turns red within a scan: the recipe rungs MOV the recipe temperature into Target_T on every scan, so the operator loses. Fix it the plant way. Add a REAL tag Temp_Trim, point the HMI at it, and change the heater rungs to compare against Target_T plus the trim (ADD them into a third tag). Now the recipe sets the base and the operator adjusts it, and nobody overwrites anybody.

TRY IT: HOW WIDE A BAND

In rung 1, change the 2.0 to 0.2 and watch the HEATER button on MAIN and the trend: the heater switches on and off every second or two, which on a real contactor is wear. Then try 5.0: hardly any switching, but the temperature swings 5 °F. The band is a trade between wear and accuracy, and it is the operator who sees the result on this screen.

WATCH FOR

A click in Program mode selects the HMI rather than pressing the screen: switch to Run mode to operate it. A setpoint tag that does not exist (a typo in the properties) shows "----" on the card; touch it and the message line names the missing tag. The Connections section shows it in red too. The setpoint is written as a number: point it at a REAL (or a DINT if whole degrees will do), not a BOOL or a TIMER.

Where the liquid comes from: the supply under the floor

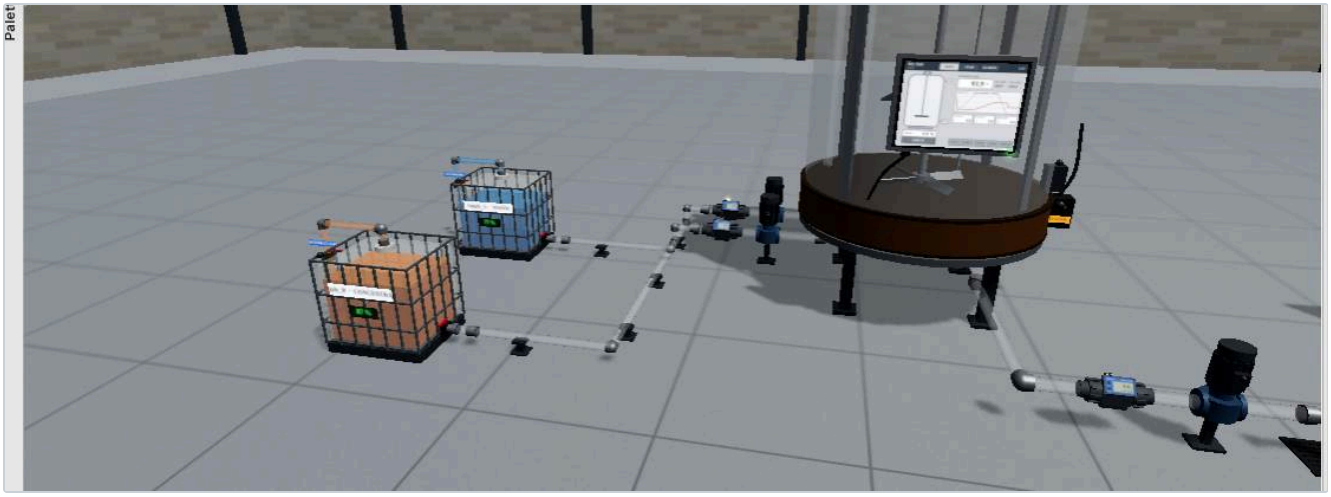


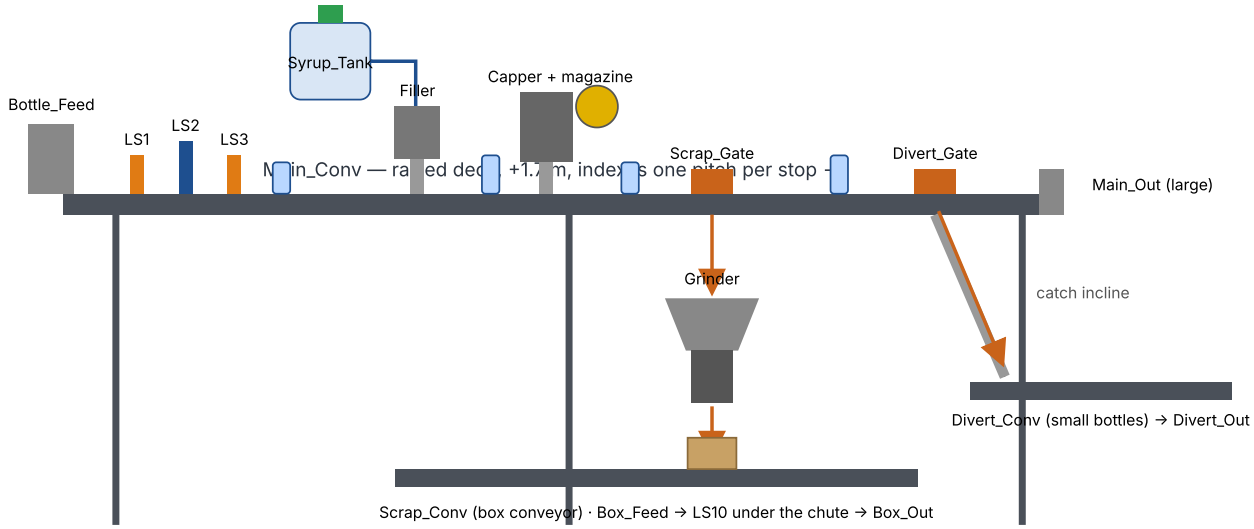
Figure 76. Level 7 in 3D: water and concentrate rise out of the floor beside the two totes and turn down into their fill caps; the totes feed the mix tank's inlets; the HMI stands to the right.

Every supply in the scene now starts somewhere. A supply tote is filled through a clear line that comes up out of a flanged connection in the floor beside it, turns over the cage and drops into the fill cap, with a ball valve at hand height; the line stands full and moves while the tote refills. An inlet on the *plant supply header* (a reservoir, a tank or a mix tank inlet with nothing piped to it) runs down into the floor the same way: the plant's supply mains run under the floor. Once you pipe the inlet to a tote or another tank, that line goes away and the pipe from the other part takes its place.

Part IV — Building the bottle line from ground zero

15 The line we are going to build

The machine is the LogixPro bottle line, the exercise generations of technicians learned shift registers on, rebuilt as a plant. A main conveyor on a raised deck indexes bottles one 40 px pitch at a time past three head sensors, a fill station fed from a soda reservoir, a capper with a rotary cap magazine, a scrap gate and a divert gate. Broken bottles are dropped off the deck into a shredder whose chute fills boxes on a box conveyor below; small bottles go down a catch incline onto a divert conveyor; large bottles run on to a case packer at the end of the line, which packs them four to a case on a case lane driven by a variable frequency drive. An operator console has Start, Stop, a Run lamp, a red fault lamp and a blue Reset, a thumbwheel preset with Enter, and six displays: SMALL, LARGE, SCRAP, BOXES, IN BOX and CASES.



Side elevation (not to scale). Broken bottles drop into the shredder; small ones ride the incline; large ones run to the end.

Figure 77. The bottle line in one picture. Everything the gates do is a *drop*: the main conveyor is a raised deck, so a gate is a latch door onto the level below.

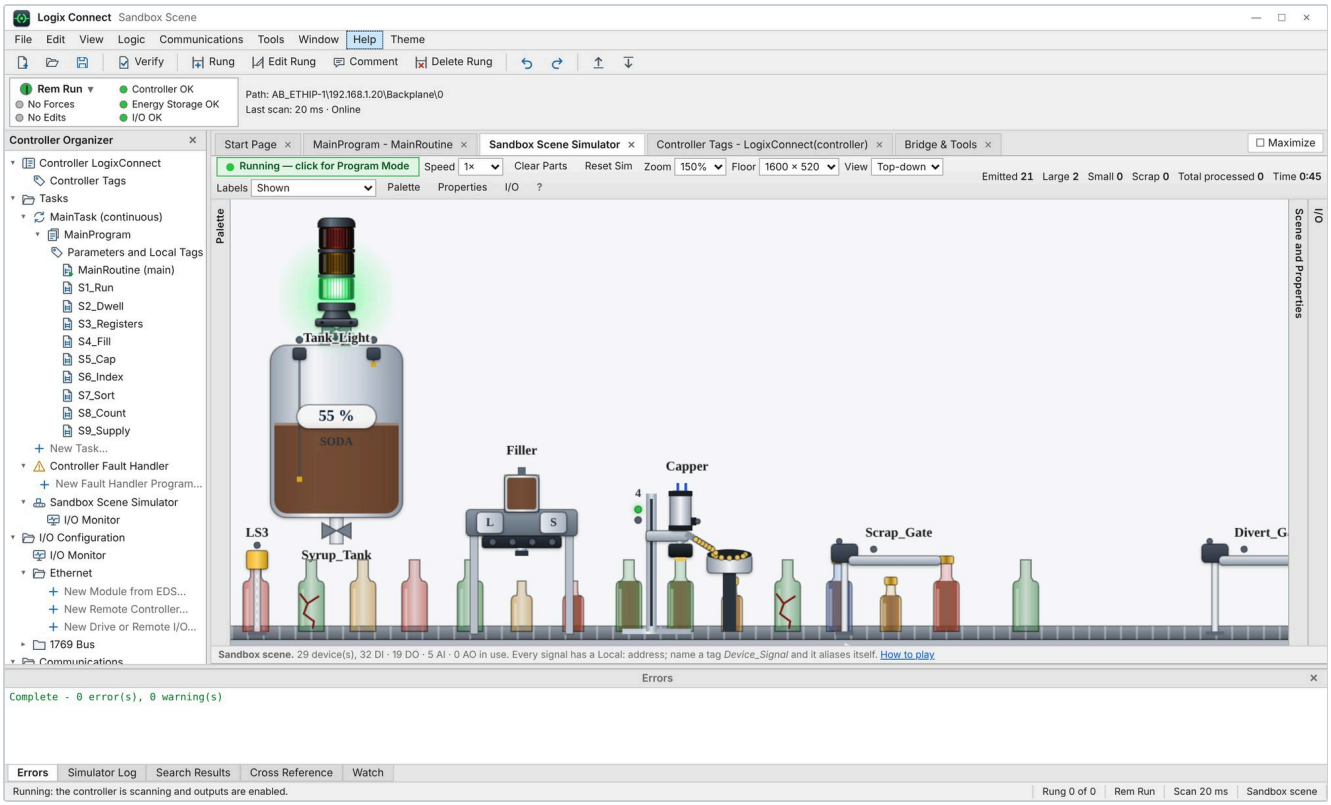


Figure 78. The finished line running in the Side view: bottles standing at the stops, the tank and filler, the capper, the gates over the lower level, the console on the right.

The program is twelve routines called from the main routine, one per step, and each step is a lesson that leaves you with something that runs:

1. **Run circuit** — Start/Stop seal-in, the run lamp, the conveyors that run with the line.
2. **Dwell and feed** — a one-second timer while the belt stands, and one bottle per stop from the feeder.
3. **Shift registers** — three registers take the three head sensors once per stop.
4. **Fill** — a bottle under the nozzle gets its large or small charge from the head tank.
5. **Cap** — a cap from the chute, the cap feeder, the column lights.
6. **Index** — step exactly one pitch on the encoder when nothing holds the belt.
7. **Sort** — the scrap and divert gates, the box conveyor and the shredder.
8. **Count and preset** — the displays, the thumbwheel batch stop, Enter.
9. **Supply** — the reservoir's inlet on its floats, the head-tank protection, the tank stack light.
10. **Label** — an FLS-style dispenser one stop past the capper: Start / Ready / Error handshake, a latched fault, lamps.
11. **Alarms** — the program watching the machine: conveyor overloads, an end-of-line check of the tracking, a dirty eye, the labeler, the reservoir inlet; a fault lamp and Reset.

12. **Pack** — the case packer and the VFD on its case lane: fast, a creep onto the mark, a speed reference scaled to 0-10 V with SCP, four bottles a case.

Guided mode

You can do Part IV with the book open beside the app, or let the app walk you through it. **File → New Project… → Guided: build the bottle line** (or **Help → Guided Mode** on an empty sandbox) opens a panel that follows these chapters one lesson at a time. Each lesson lists what it checks — parts placed at the book's positions, alias tags made, the routine and its JSR in place, the routine verifying — and, for the steps that do something, what it watches for while you run the line: Line_Run coming on when you press Start, a bottle filled and capped, a label going on. A check turns green when the page sees it happen. The reference rungs for the lesson sit under a fold with *Copy* and *Add* buttons; type the rungs yourself and use the buttons only when you are stuck. Progress saves with the project; drag the panel by its header out of your way.

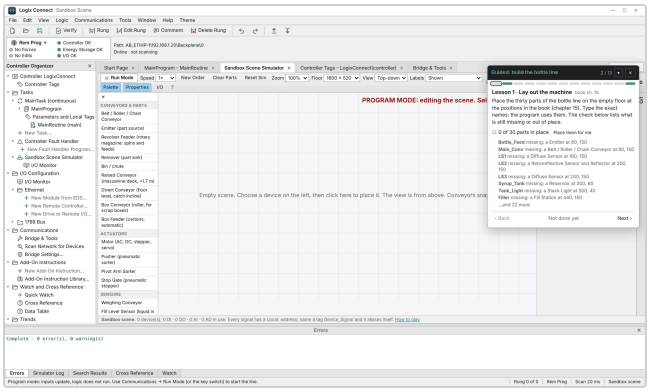


Figure 79. Guided mode on a new project: an empty floor and lesson 1.

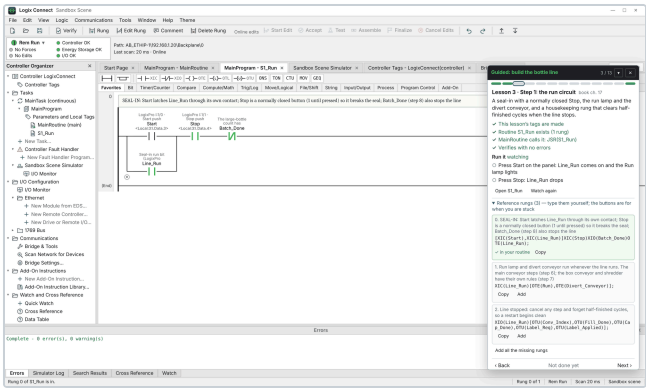


Figure 80. Lesson 3: the routine is in, Start pressed, the run checks watching.

Chapter 15 lays the machine out, chapter 16 makes the tags, chapters 17 to 28 write the twelve routines, and chapter 29 commissions the line. Every coordinate and setting below is the one the finished template uses, so your machine and the answer key are the same machine.

16 Lesson 1 — Lay out the machine

> **IN THE APP**
>
> Guided mode lesson 1 is this chapter. It counts the parts you have placed against this list — name, type and position within 10 px — and names the ones still missing or out of place. *Place them for me* does the whole list in one click when you would rather get to the programming.

File → New Project… → **empty sandbox**. Set the floor to **1600 × 520** (it is the default) and the view to Top-down. Open the Palette. Place the parts below in order, then set each one's position in the Properties drawer (type the numbers; dragging is for later). Names: type them in the name field at the top of Properties; every name here is used by the program, so use them exactly.

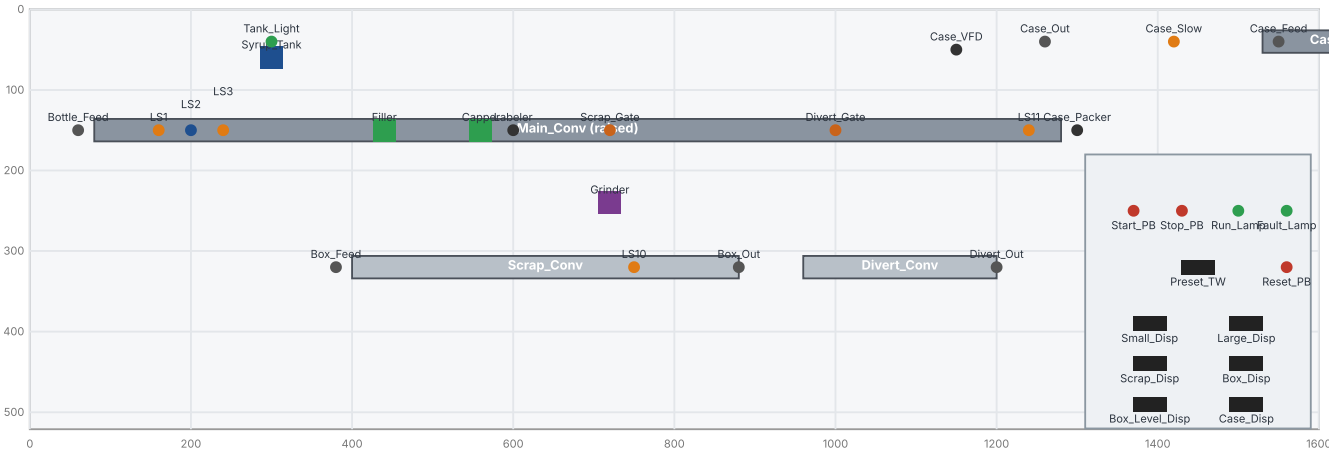


Figure 81. The plan, drawn from the template's own coordinates (grid in floor pixels; x to the right, y down). Belts are drawn from their start point; everything else at its centre.

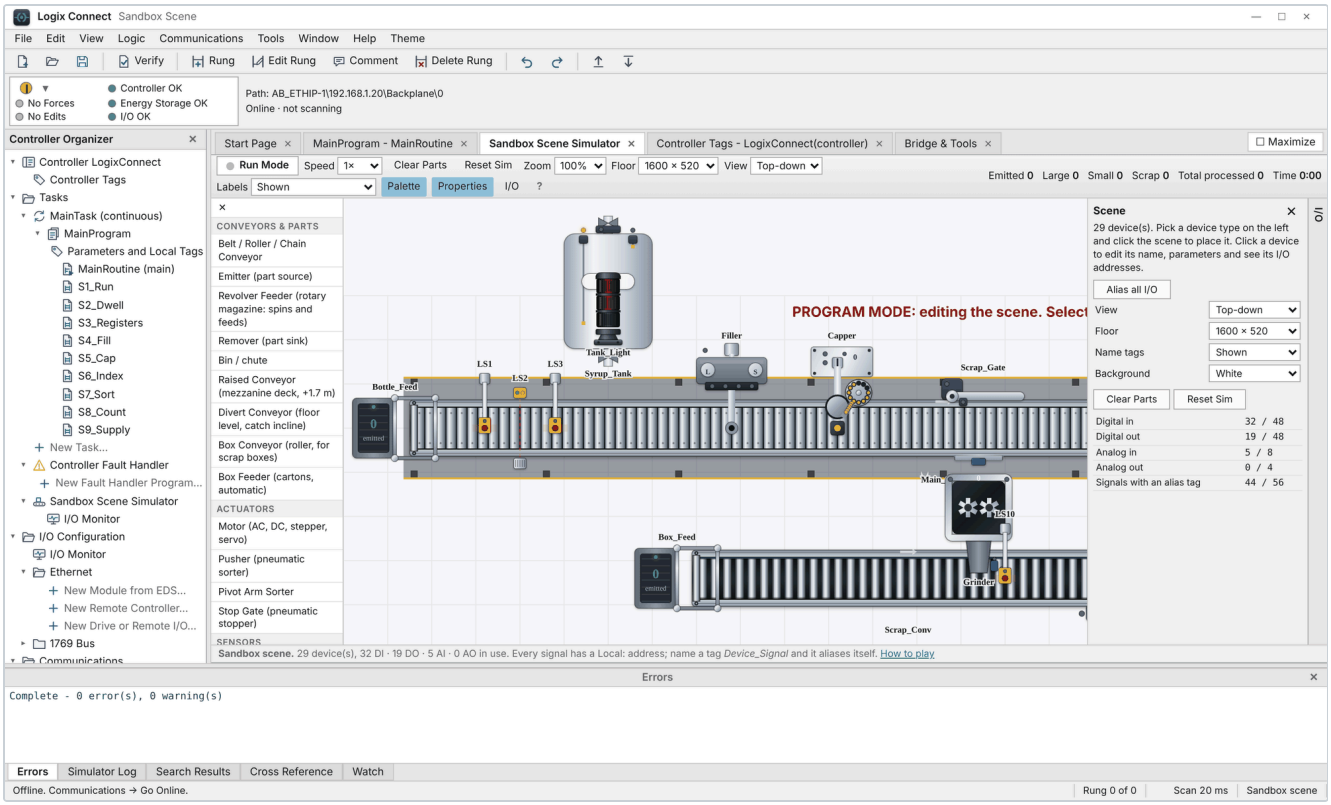


Figure 82. The finished layout in Top-down view with name tags on. Yours should look like this when the lists below are placed.

The main line

- **Emitter (part source)** at **x 60, y 150**, named `Bottle_Feed`: *Part to Emit* = **bottle, bottleL**; *Min Time (s, continuous / automatic)* = **2.5**; *Max Time (s, continuous / automatic)* = **4**; *Broken bottles (%)* = **15**. Edge mode: one bottle per rising edge of Emit. Fifteen per cent of them come out broken; that is the scrap the line has to deal with.
- **Belt / Roller / Chain Conveyor** at **x 80, y 150**, named `Main_Conv`: *Length (m), 0.5 to 40 in half-metre steps* = **20**; *Speed (mm/s)* = **60**; *Conveyor type* = **slat chain**; *Belt color* = **Grey**; *Height (3D)* = **Raised deck (+1.7 m)**; *Incremental encoder (adds A, B and Pulses)* = **True**; *Encoder resolution (px per pulse)* = **40**. Twenty metres of slat chain on a raised deck, with an encoder of 40 px per pulse: one pulse is one stop. The deck is what lets the gates drop bottles to the conveyors below.
- **Diffuse Sensor (photoelectric)** at **x 160, y 150**, named `LS1`: *Sensing range (px; 60 px = 1 m)* = **14**. Bottle present at the head of the line. On the 40 px stop grid, one stop before LS2.
- **Retroreflective Sensor and Reflector** at **x 200, y 150**, named `LS2`: *Beam height (px above the belt)* = **48**. A retro beam 48 px up: only a large bottle (56 px tall) breaks it; a small one (40 px) passes under.
- **Diffuse Sensor (photoelectric)** at **x 240, y 150**, named `LS3`: *Sensing range (px; 60 px = 1 m)* = **14**; *Sees* = **broken bottles**. Set to see broken bottles only.

- **Fill Station (tube, large and small fill)** at **x 440, y 150**, named `Filler`: *Fed from* = **Syrup_Tank**. Fed from Syrup_Tank; its own 300-unit head tank.
- **Inline Cap Feeder (pneumatic capper + cap bowl)** at **x 560, y 150**, named `Capper` (default settings). Thirty caps in the chute to start; the magazine refills it at 3 caps a second when Feeder runs.
- **Inline Labeler (FLS-style dispenser, RH / LH, any applicator)** at **x 600, y 150**, named `Labeler`: *Applicator* = **Wrap-around (roller)**. One stop past the capper. RH, wrap-around roller: the dispenser stands on the far side of the lane; 200 labels on the roll to start.
- **Pivot Arm Sorter** at **x 720, y 150**, named `Scrap_Gate` (default settings). A pivot arm. Its push side is +y, toward the grinder.
- **Pivot Arm Sorter** at **x 1000, y 150**, named `Divert_Gate` (default settings). Pushes toward the divert conveyor, 166 px below on the lower level.
- **Diffuse Sensor (photoelectric)** at **x 1240, y 150**, named `LS11`: *Sees* = **bottles only**. Set to see bottles only. One stop before the end of the line: the program checks here that the bottle it expects at this stop is really there (step 11).

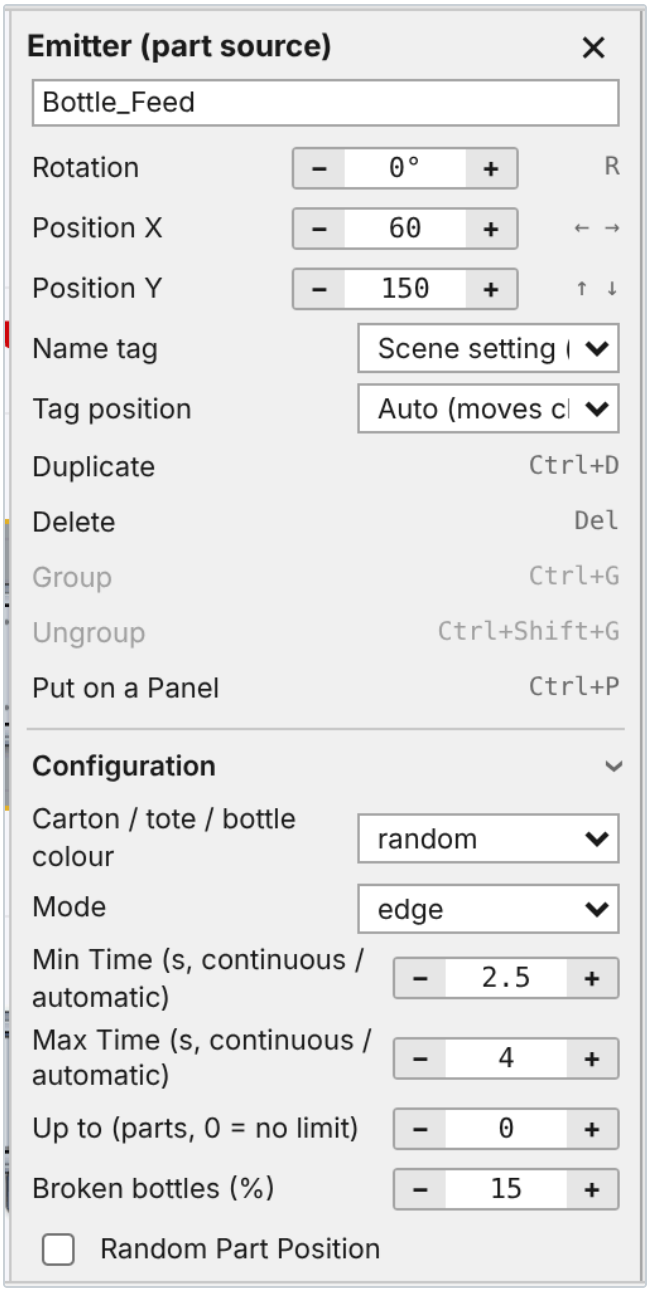


Figure 83. Main_Conv's settings: 20 m slat chain, raised deck, encoder 40 px per pulse.

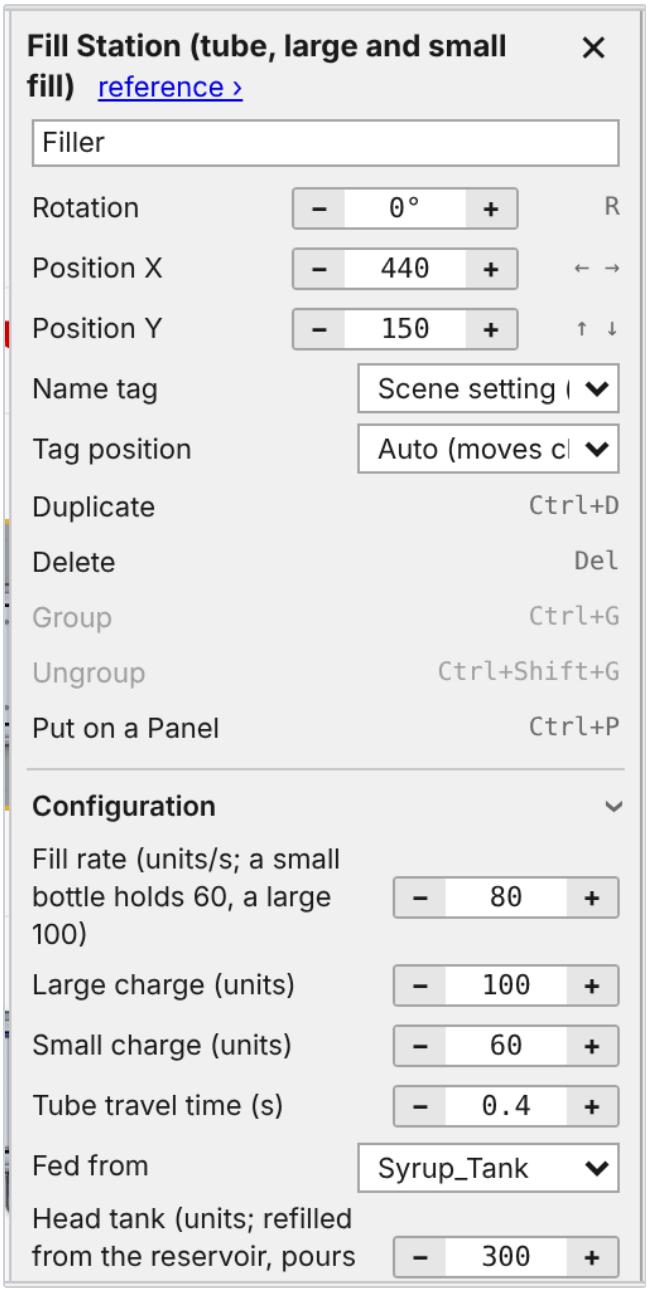


Figure 84. The Capper's settings.

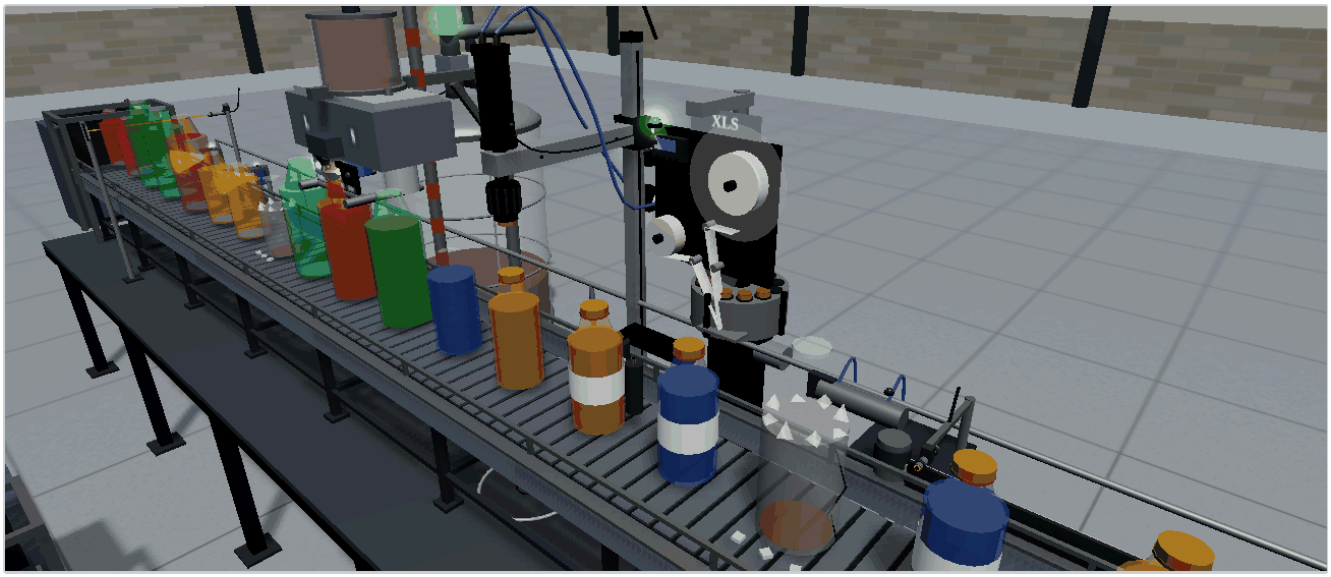


Figure 85. The Labeler on the line, one stop past the capper: the FLS-style dispenser on its column, the unwind reel, the web to the dispensing edge, the wrap-around roller at bottle height.

Packing

- **Case Packer (top-load: infeed buffer, pick head, case lane beside the line)** at **x 1300, y 150**, named `Case_Packer` (default settings). At the end of the main line: its infeed takes the bottles off the end of the deck, and its pick head reaches across to the case lane 108 px away on the far side.
- **Belt / Roller / Chain Conveyor** at **x 1530, y 40**, rotation 180°, named `Case_Conv`: *Length (m), 0.5 to 40 in half-metre steps* = **4.5**; *Speed (mm/s)* = **150**; *Driven by drive module (its name; blank = contactor)* = **Case_VFD**; *Conveyor type* = **roller**; *Height (3D)* = **Raised deck (+1.7 m)**. The case lane: a raised roller conveyor running right to left (rotation 180°) past the packer head. Its *Driven by drive module* is Case_VFD, so it has no Fwd output of its own: the drive runs it.
- **Emitter (part source)** at **x 1550, y 40**, rotation 180°, named `Case_Feed`: *Part to Emit* = **cartonL**; *Carton / tote / bottle colour* = **Y**; *Mode* = **automatic (no Emit input)**. An empty carton every 3 s at the upstream (right-hand) end, whenever there is room.
- **Diffuse Sensor (photoelectric)** at **x 1420, y 40**, named `Case_Slow`: *Sensing range (px; 60 px = 1 m)* = **12**; *Sees* = **boxes only**. Boxes only, 100 px before the packer head: the slow-down eye.
- **Remover (part sink)** at **x 1260, y 40**, rotation 180°, named `Case_Out` (default settings). At the downstream end of the lane: packed cases leave here.
- **VFD (variable frequency drive, hardwired: 0-10 V speed reference)** at **x 1150, y 50**, named `Case_VFD` (default settings). The drive for the case lane: 60 Hz at 10 V, 1 s accel, 0.5 s decel.

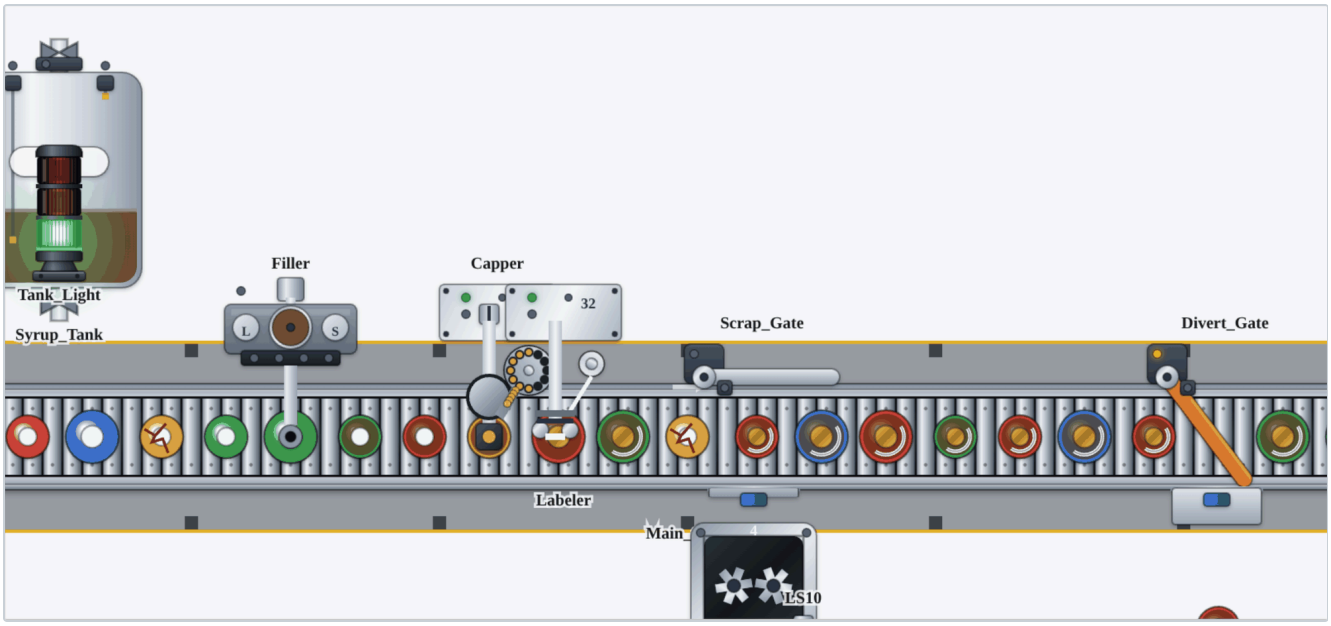


Figure 86. The end of the line in Top-down view: the packer's infeed table at the end of the deck, the gantry reaching over the case lane, a case under the head, the slow-down eye, the case feeder, and the VFD beside the lane.

Supply

- **Reservoir (product supply for fillers)** at **x 300, y 60**, named `Syrup_Tank`: *Product* = **Soda**; *Holds (units; a small bottle takes 60, a large 100)* = **2000**; *Starts at (%)* = **35**; *Inlet flow (units/s)* = **150**; *Low float (%)* = **20**. Soda, 2000 units, starting at 35 % so you see the inlet work early. High float 90 %, low float 20 %.
- **Stack Light** at **x 300, y 40**, named `Tank_Light`: *Mount* = **Direct**; *Built-in flasher* = **Red flashes**. Mounted directly on the tank dome; its red segment flashes when driven.

The lower level

- **Grinder (shredder under a raised belt, over a box conveyor)** at **x 720, y 240**, named `Grinder` (default settings). Placed 90 px below the scrap gate so a bottle pushed off the deck lands in its hopper; its chute reaches the box conveyor 76 px further down.
- **Emitter (part source)** at **x 380, y 320**, named `Box_Feed`: *Part to Emit* = **cartonM**; *Carton / tote / bottle colour* = **Y**; *Mode* = **automatic (no Emit input)**; *Min Time (s, continuous / automatic)* = **4**; *Max Time (s, continuous / automatic)* = **4**. A carton every 4 s, by itself.
- **Belt / Roller / Chain Conveyor** at **x 400, y 320**, named `Scrap_Conv`: *Length (m), 0.5 to 40 in half-metre steps* = **8**; *Speed (mm/s)* = **80**; *Conveyor type* = **roller**. The box conveyor, LogixPro's "scrap conveyor". Eight metres of roller.
- **Diffuse Sensor (photoelectric)** at **x 750, y 320**, named `LS10`: *Sensing range (px; 60 px = 1 m)* = **12**; *Sees* = **boxes only**. Boxes only, placed so a carton stops centred under the chute.

- **Remover (part sink)** at **x 880, y 320**, named `Box_Out` (default settings). At the end of the box conveyor.
- **Belt / Roller / Chain Conveyor** at **x 960, y 320**, named `Divert_Conv`: *Conveyor type* = **roller**. Starts under the divert gate; the catch incline from the deck lands on it.
- **Remover (part sink)** at **x 1200, y 320**, named `Divert_Out` (default settings)

The operator console

- **Panel / Enclosure** at **x 1450, y 350**, named `Panel_1`: *Style* = **Operator console**; *Width (px)* = **280**; *Height (px)* = **340**; *Title plate (blank for none)* = **BOTTLE LINE**
- **Push Button** at **x 1370, y 250**, named `Start_PB` (default settings)
- **Push Button** at **x 1430, y 250**, named `Stop_PB`: *Color* = **red**; *Legend text (Legend button style)* = **STOP**; *Normally closed contact* = **True**. Normally closed: its OK signal is 1 until it is pressed.
- **Indicator Light / LED** at **x 1500, y 250**, named `Run_Lamp` (default settings)
- **Indicator Light / LED** at **x 1560, y 250**, named `Fault_Lamp`: *Color* = **red**. Flashes while an alarm is unacknowledged, steady once acknowledged (step 11).
- **Push Button** at **x 1560, y 320**, named `Reset_PB`: *Color* = **blue**; *Legend text (Legend button style)* = **RESET**. Acknowledges alarms and clears those whose cause has gone (step 11).
- **Thumbwheel Switch (digits + Enter)** at **x 1450, y 320**, named `Preset_TW` (default settings)
- **Digital Display (any tag)** at **x 1390, y 390**, named `Small_Disp`: *Tag to show (blank = the wired analog output)* = **Small_Bottle_Count**; *Width (characters)* = **4**; *Digit color* = **amber**; *Caption on the bezel (optional)* = **SMALL**. Shows the tag named in its settings; the tag does not exist yet, so the display reads dashes until chapter 16.
- **Digital Display (any tag)** at **x 1510, y 390**, named `Large_Disp`: *Tag to show (blank = the wired analog output)* = **Large_Bottle_Count**; *Width (characters)* = **4**; *Digit color* = **amber**; *Caption on the bezel (optional)* = **LARGE**
- **Digital Display (any tag)** at **x 1390, y 440**, named `Scrap_Disp`: *Tag to show (blank = the wired analog output)* = **Scrap_Bottle_Count**; *Width (characters)* = **4**; *Digit color* = **amber**; *Caption on the bezel (optional)* = **SCRAP**
- **Digital Display (any tag)** at **x 1510, y 440**, named `Box_Disp`: *Tag to show (blank = the wired analog output)* = **Scrap_Box_Count**; *Width (characters)* = **4**; *Digit color* = **amber**; *Caption on the bezel (optional)* = **BOXES**
- **Digital Display (any tag)** at **x 1390, y 490**, named `Box_Level_Disp`: *Tag to show (blank = the wired analog output)* = **Box_Level**; *Width (characters)* = **2**; *Caption on the bezel (optional)* = **IN BOX**

- **Digital Display (any tag)** at **x 1510, y 490**, named `Case_Disp`: *Tag to show (blank = the wired analog output)* = **Case_Count**; *Width (characters)* = **4**; *Digit color* = **amber**; *Caption on the bezel (optional)* = **CASES**. CASES: the case count from step 12.

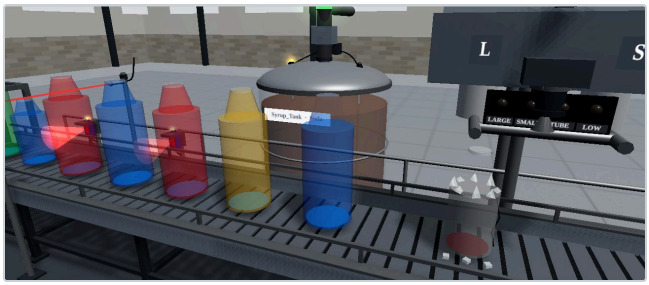


Figure 87. Syrup_Tank with Tank_Light on its dome, and the pipe to the Filler.

Figure 88. The Grinder between the deck and the box conveyor, with LS10 and a box under the chute.

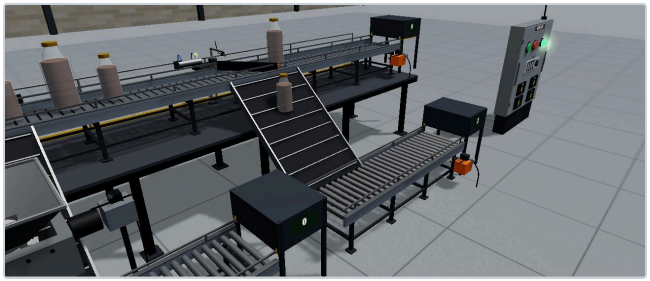


Figure 89. The Divert_Gate, its incline and Divert_Conv.

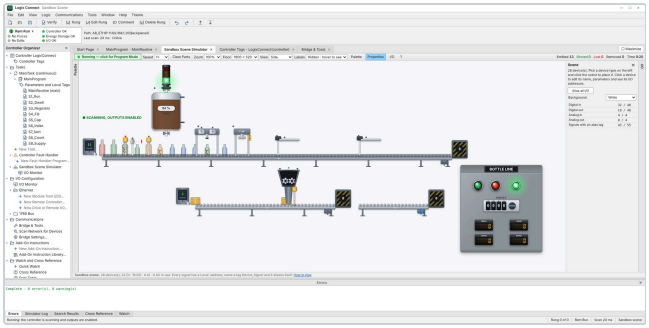


Figure 90. The console at the right-hand end of the floor.

That is 39 devices. Open the Scene sheet (click empty floor, Properties): it should read about 52 digital inputs, 29 digital outputs and 7 analog inputs in use. Switch to Side view and then, on a desktop, 3D, and walk along it: the pipe from the tank to the filler draws itself, the incline from the deck lands on the divert conveyor, the gates have their latch doors.

RUN IT WITH NO PROGRAM

Go to Run mode. Nothing moves — every output is off — but the feeders set to automatic are not outputs: boxes start arriving on the box conveyor and queue at its head, because the conveyor is not running either. The Simulator Log says nothing yet. Back to Program mode.

WATCH FOR

Positions are the centre of a conveyor's start and the centre of every other part. A sensor must be on the conveyor's centreline (y = 150 on the main line) to see bottles; a station must be at a stop position (x = 160 + 40n) to meet a bottle standing still. If you place by dragging, the snap does both for you.

17 Lesson 2 — Name the I/O: the tag list

> **IN THE APP**
>
> Guided mode lesson 2 is this chapter: it lists the alias tags still to make or to fix, and *Make them for me* makes them all.

Every point on the machine now has an address; the program needs names. This chapter is the complete list, so you can see the whole shape of it and look any tag up later — **you do not have to create them all now**. Each lesson from here on begins with *Tags this lesson needs*, a short table of the tags its rungs use for the first time; make those as you go and you have something running at the end of every chapter. Open **Controller Tags → Edit Tags**. The fastest route is **Tools → Create Alias Tags for Wired I/O**, which makes a `Device_Signal` alias for every signal on the floor — and then rename the ones the program calls by LogixPro names (click the name in the grid). Or create them by hand: name, type, *Alias*, and pick the point in *Alias For*. The Properties drawer of each part shows which point each signal has, so you are never guessing.

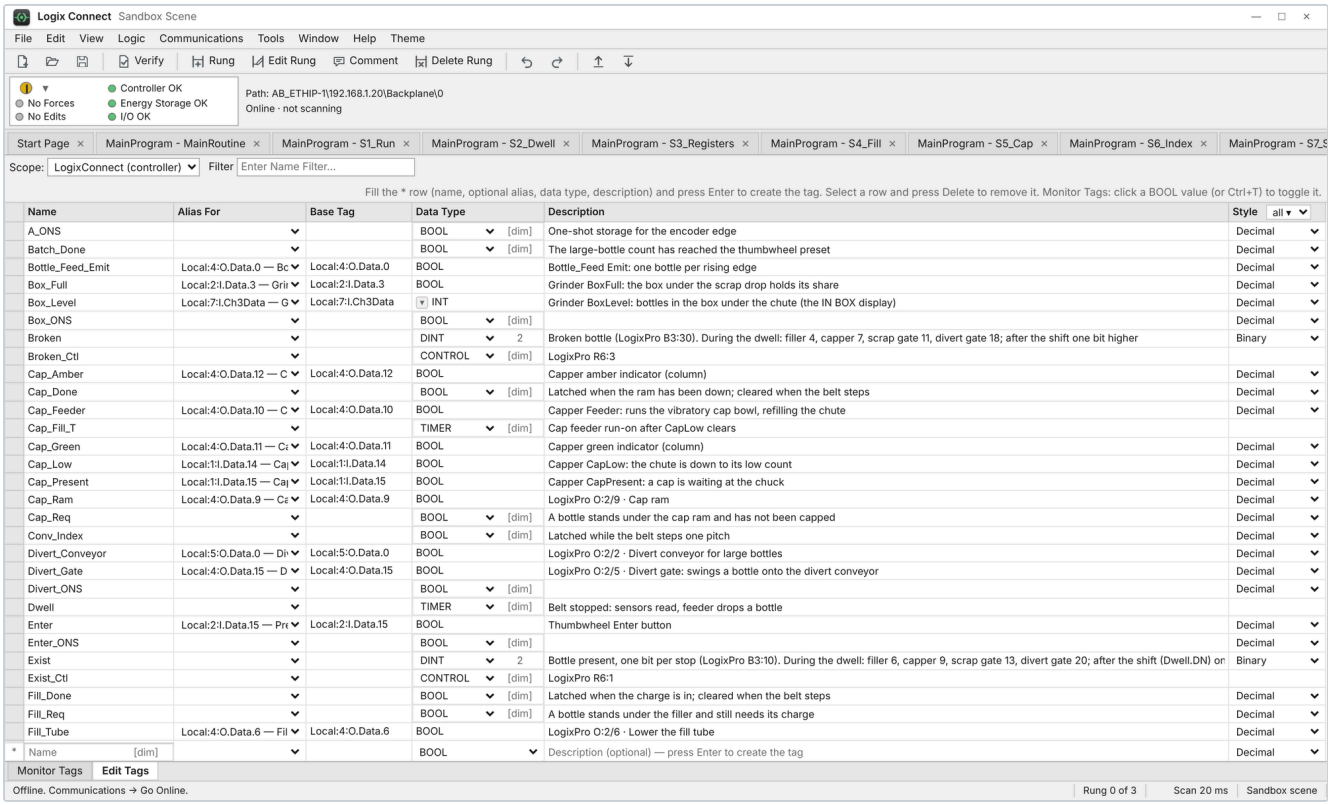


Figure 91. The finished tag list in Edit Tags. Make yours match this table.

Alias tags

Tag	Type	Alias for	Description
`Start`	BOOL	`Local:3:I.Data.2`	LogixPro I:1/0 · Start push button, NO
`Stop`	BOOL	`Local:3:I.Data.3`	LogixPro I:1/1 · Stop push button, NC (1 until pressed)
`LS1`	BOOL	`Local:1:I.Data.4`	LogixPro I:1/6 · Exist sensor: a bottle at the start of the line
`LS2`	BOOL	`Local:1:I.Data.5`	LogixPro I:1/7 · Size check: beam set so only a large bottle breaks it
`LS3`	BOOL	`Local:1:I.Data.6`	LogixPro I:1/8 · Broken check: sees broken bottles only
`LS4`	BOOL	`Local:1:I.Data.9`	LogixPro I:1/9 · Fill tube down
`LS5`	BOOL	`Local:1:I.Data.10`	LogixPro I:1/10 · Large charge delivered
`LS6`	BOOL	`Local:1:I.Data.11`	LogixPro I:1/11 · Small charge delivered
`LS7`	BOOL	`Local:1:I.Data.13`	LogixPro I:1/12 · Cap ram down
`LS8`	BOOL	`Local:2:I.Data.7`	LogixPro I:1/13 · Bottle at the scrap gate
`LS9`	BOOL	`Local:2:I.Data.11`	LogixPro I:1/14 · Bottle at the divert gate
`LS10`	BOOL	`Local:3:I.Data.0`	LogixPro I:1/15 · Scrap box under the grinder chute
`Main_Conveyor`	BOOL	`Local:4:O.Data.1`	LogixPro O:2/0 · Main conveyor motor
`Scrap_Conveyor`	BOOL	`Local:5:O.Data.4`	LogixPro O:2/1 · Box conveyor under the grinder
`Divert_Conveyor`	BOOL	`Local:5:O.Data.3`	LogixPro O:2/2 · Divert conveyor for large bottles
`Grinder`	BOOL	`Local:5:O.Data.1`	LogixPro O:2/3 · Grinder motor
`Scrap_Gate`	BOOL	`Local:5:O.Data.0`	LogixPro O:2/4 · Scrap gate: swings a bottle into the grinder
`Divert_Gate`	BOOL	`Local:5:O.Data.2`	LogixPro O:2/5 · Divert gate: swings a bottle onto the divert conveyor
`Fill_Tube`	BOOL	`Local:4:O.Data.6`	LogixPro O:2/6 · Lower the fill tube
`Large_Fill`	BOOL	`Local:4:O.Data.7`	LogixPro O:2/7 · Large fill valve
`Small_Fill`	BOOL	`Local:4:O.Data.8`	LogixPro O:2/8 · Small fill valve
`Cap_Ram`	BOOL	`Local:4:O.Data.9`	LogixPro O:2/9 · Cap ram
`Run`	BOOL	`Local:5:O.Data.5`	LogixPro O:2/10 · Run lamp
`Cap_Feeder`	BOOL	`Local:4:O.Data.10`	Capper Feeder: runs the vibratory cap bowl, refilling the chute
`Label_Apply`	BOOL	`Local:4:O.Data.13`	Labeler Start: the applicator comes in and presses a label on

Tag	Type	Alias for	Description
`Label_PadIn`	BOOL	`Local:2:I.Data.0`	Labeler applicator at the end of its stroke (label pressed on)
`Label_Ready`	BOOL	`Local:2:I.Data.1`	Labeler ready: web loaded, head home, no fault
`Label_Error`	BOOL	`Local:2:I.Data.2`	Labeler fault: end of roll, or the last cycle found no label
`Label_Low`	BOOL	`Local:2:I.Data.3`	Labeler roll low
`Label_Present`	BOOL	`Local:2:I.Data.4`	A label is peeled and waiting at the dispensing edge
`Label_Done`	BOOL	`Local:2:I.Data.5`	The bottle under the applicator already carries a label
`Label_Green`	BOOL	`Local:4:O.Data.14`	Labeler green indicator
`Label_Amber`	BOOL	`Local:4:O.Data.15`	Labeler amber indicator
`Cap_Green`	BOOL	`Local:4:O.Data.11`	Capper green indicator (column)
`Cap_Amber`	BOOL	`Local:4:O.Data.12`	Capper amber indicator (column)
`Cap_Low`	BOOL	`Local:1:I.Data.14`	Capper CapLow: the chute is down to its low count
`Cap_Present`	BOOL	`Local:1:I.Data.15`	Capper CapPresent: a cap is waiting at the chuck
`Thumbwheel`	INT	`Local:9:I.Ch0Data`	LogixPro I:3 · Thumbwheel switch, read as a number (BCD on the real one)
`Enter`	BOOL	`Local:3:I.Data.6`	Thumbwheel Enter button
`LS11`	BOOL	`Local:3:I.Data.4`	End-of-line eye at 1240: verifies the tracking (only large, whole bottles reach it)
`Main_Conv_Running`	BOOL	`Local:1:I.Data.1`	Main conveyor running feedback (motor contactor)
`Divert_Conv_Running`	BOOL	`Local:2:I.Data.12`	Divert conveyor running feedback
`Fault_Lamp`	BOOL	`Local:5:O.Data.6`	Red fault lamp on the console: flashing = unacknowledged alarm, steady = alarm still active
`Reset_PB`	BOOL	`Local:3:I.Data.5`	Blue Reset button: acknowledge alarms and clear the ones whose cause is gone
`Case_Run`	BOOL	`Local:5:O.Data.9`	Case lane drive: run (two-wire)
`Case_VFD_Reset`	BOOL	`Local:5:O.Data.11`	Case lane drive: fault reset (rising edge)
`Case_Speed_Ref`	INT	`Local:8:O.Ch0Data`	Case lane drive speed reference: 0-10000 = 0-10 V = 0-60 Hz
`Case_VFD_Ready`	BOOL	`Local:10:I.Data.0`	Case lane drive ready (not tripped)
`Case_VFD_Running`	BOOL	`Local:10:I.Data.1`	Case lane drive output frequency above zero
`Case_At_Speed`	BOOL	`Local:10:I.Data.2`	Case lane drive at the commanded speed
`Case_VFD_Fault`	BOOL	`Local:10:I.Data.3`	Case lane drive tripped

Tag	Type	Alias for	Description
`Case_Hz_Fbk`	INT	`Local:9:I.Ch2Data`	Case lane drive output frequency: 0-10000 = 0-10 V = 0-60 Hz
`Case_Slow`	BOOL	`Local:3:I.Data.15`	Slow-down eye on the case lane, 100 px before the packer head
`Case_In_Place`	BOOL	`Local:3:I.Data.10`	A case is centred under the packer head
`Case_Full`	BOOL	`Local:3:I.Data.11`	The case under the head has been packed
`Group_Ready`	BOOL	`Local:3:I.Data.8`	The packer infeed holds a case worth (4)
`Infeed_Full`	BOOL	`Local:3:I.Data.9`	Packer infeed buffer full: bottles arriving now wait on the belt
`Infeed_Count`	INT	`Local:9:I.Ch1Data`	Bottles in the packer infeed
`Packer_Ready`	BOOL	`Local:3:I.Data.7`	Packer head home, not cycling
`Pack_Drop`	BOOL	`Local:5:O.Data.7`	Packer: pick a case worth from the infeed and place it in the case (rising edge)
`Main_Conv_A`	BOOL	`Local:1:I.Data.2`	Main_Conv encoder channel A: one rising edge per 40 px pitch
`Bottle_Feed_Emit`	BOOL	`Local:4:O.Data.0`	Bottle_Feed Emit: one bottle per rising edge
`Box_Full`	BOOL	`Local:2:I.Data.9`	Grinder BoxFull: the box under the scrap drop holds its share
`Box_Level`	INT	`Local:7:I.Ch3Data`	Grinder BoxLevel: bottles in the box under the chute (the IN BOX display)
`Scrap_Waiting`	BOOL	`Local:2:I.Data.8`	Grinder Detect: a broken bottle is waiting in the hopper
`Tank_Level`	INT	`Local:7:I.Ch2Data`	Filler head tank, 0-100 %
`Tank_Low`	BOOL	`Local:1:I.Data.12`	Filler head tank below 15 %
`Syrup_Low`	BOOL	`Local:1:I.Data.8`	Syrup_Tank low float
`Syrup_High`	BOOL	`Local:1:I.Data.7`	Syrup_Tank high float
`Syrup_Inlet`	BOOL	`Local:4:O.Data.2`	Syrup_Tank inlet valve
`Syrup_Level`	INT	`Local:7:I.Ch1Data`	Syrup_Tank level, 0-100 %
`Tank_Red`	BOOL	`Local:4:O.Data.3`	Tank_Light red: reservoir at the low float (flashes)
`Tank_Amber`	BOOL	`Local:4:O.Data.4`	Tank_Light amber: inlet open, refilling
`Tank_Green`	BOOL	`Local:4:O.Data.5`	Tank_Light green: level healthy

Plain tags

These are the program's own memory. The three registers are DINT arrays of two elements (Dim 2 — 64 bits) shown in Binary; the ONS storage bits are BOOLs the one-shots own; the timers and the CONTROL tags belong to their instructions.

Tag	Type	Style	Description
`Cap_Fill_T`	TIMER		Cap feeder run-on after CapLow clears
`Large_Bottle_Count`	DINT		LogixPro O:4/0 · Large bottle count, shown on the LARGE display
`Small_Bottle_Count`	DINT		LogixPro O:4/1 · Small bottle count, shown on the SMALL display
`Scrap_Bottle_Count`	DINT		LogixPro O:4/2 · Scrap bottle count, shown on the SCRAP display
`Scrap_Box_Count`	DINT		LogixPro O:4/3 · Scrap box count, shown on the BOXES display
`Line_Run`	BOOL		Seal-in run bit (LogixPro B3:0/0)
`Conv_Index`	BOOL		Latched while the belt steps one pitch
`A_ONS`	BOOL		One-shot storage for the encoder edge
`Dwell`	TIMER		Belt stopped: sensors read, feeder drops a bottle
`Exist`	DINT[2]	Binary	Bottle present, one bit per stop (LogixPro B3:10). During the dwell: filler 6, capper 9, scrap gate 13, divert gate 20; after the shift (Dwell.DN) one bit higher
`Large`	DINT[2]	Binary	Large bottle (LogixPro B3:20). During the dwell: filler 5, capper 8, scrap gate 12, divert gate 19; after the shift one bit higher
`Broken`	DINT[2]	Binary	Broken bottle (LogixPro B3:30). During the dwell: filler 4, capper 7, scrap gate 11, divert gate 18; after the shift one bit higher
`Exist_Ctl`	CONTROL		LogixPro R6:1
`Large_Ctl`	CONTROL		LogixPro R6:2
`Broken_Ctl`	CONTROL		LogixPro R6:3
`Fill_Req`	BOOL		A bottle stands under the filler and still needs its charge
`Fill_Done`	BOOL		Latched when the charge is in; cleared when the belt steps
`Cap_Req`	BOOL		A bottle stands under the cap ram and has not been capped
`Cap_Done`	BOOL		Latched when the ram has been down; cleared when the belt steps
`Label_Req`	BOOL		Latched while the labeler is asked to apply; cleared at PadIn
`Label_Applied`	BOOL		Latched when the label is pressed on; cleared when the belt steps
`Label_Fault`	BOOL		Latched on the labeler Error input (end of roll or a missed label); Enter clears it

Tag	Type	Style	Description
`Alarm_Word`	DINT	Binary	Latched alarms, one bit each: 0 main conveyor overload, 1 divert conveyor overload, 2 end-of-line check failed (a jam, a lost bottle, tracking out of step), 3 LS1 dark too long (dirty lens or feeder empty), 4 labeler fault, 5 reservoir inlet open but level not rising, 6 case lane drive tripped, 7 packer infeed full
`Alarm_Unack`	BOOL		New alarm, not yet acknowledged (Reset)
`Main_OL_T`	TIMER		Main conveyor overload timer (no running feedback)
`Div_OL_T`	TIMER		Divert conveyor overload timer (no running feedback)
`EOL_Missing`	BOOL		LS11 dark when the registers said a large, whole bottle was there
`EOL_Extra`	BOOL		LS11 saw a bottle the registers did not expect
`EOL_Inhibit`	BOOL		Tracking cleared by Reset: the end-of-line check waits until the first bottle tracked since then reaches LS11
`EOL_ONS`	BOOL		
`LS1_Dark_T`	TIMER		LS1 dark too long timer
`Inlet_T`	TIMER		Inlet not filling timer
`Flash_T`	TIMER		Fault lamp flasher
`OL_ONS`	BOOL		
`DOL_ONS`	BOOL		
`BLK_ONS`	BOOL		
`Reg_Clear`	BOOL		
`DRK_ONS`	BOOL		
`LBL_ONS`	BOOL		
`INL_ONS`	BOOL		
`RST_ONS`	BOOL		
`VFD_ONS`	BOOL		
`INF_ONS`	BOOL		
`Case_Hz`	REAL		Case lane speed the program wants, in Hz
`Case_Hz_Actual`	REAL		Case lane drive output frequency read back, in Hz
`Case_Count`	DINT		Cases packed, shown on the CASES display
`Case_Creep`	BOOL		Latched from the slow-down eye until an empty case stands under the head
`Drop_ONS`	BOOL		

Tag	Type	Style	Description
`Case_ONS`	BOOL		
`Scrap_ONS`	BOOL		
`Divert_ONS`	BOOL		
`Small_ONS`	BOOL		
`Large_ONS`	BOOL		
`Box_ONS`	BOOL		
`Enter_ONS`	BOOL		
`Batch_Done`	BOOL		The large-bottle count has reached the thumbwheel preset

Addresses in the alias column are the ones the template's parts get when placed in chapter 15's order; if yours differ (you placed something twice and deleted it, say) use the address in your Properties drawer — the name is what matters to the program.

CHECK IT

Run mode, Controller Tags → Monitor Tags. Push Start_PB on the panel and watch `Start` go to 1. Hold a hand over nothing: `Stop` is already 1 — it is a normally-closed button. Put a bottle in front of LS1 by hand: there is no belt running, so instead toggle `Bottle_Feed_Emit` (Ctrl+T) and watch a bottle appear at the head of the line and LS1 go to 1 a scan later. Back to Program mode and Reset Sim.

18 Lesson 3 — Step 1: the run circuit

A machine has one bit that says it is running, and everything else hangs off it. Make a routine called `S1_Run` (Logic → New Routine…) and give MainRoutine a single rung, `JSR(S1_Run);`. Then write these three rungs in S1_Run:

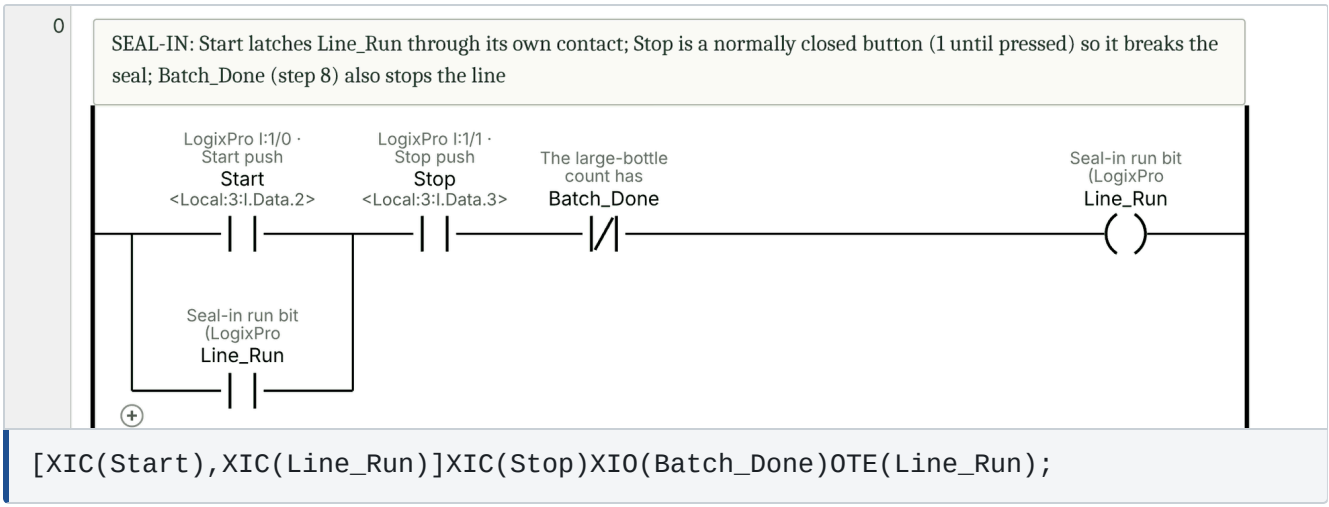
> **IN THE APP**
>
> Guided mode lesson 3 is this chapter: it checks that this lesson's tags are made, that `S1_Run` exists, is called from MainRoutine and verifies, and then watches the line while you run it and ticks each *Run it* result below as it happens. Help → Guided Mode, or start from File → New Project… → Guided: build the bottle line.

Tags this lesson needs

Create these before you type the rungs (Verify reports any you miss as unknown). Tags from earlier lessons are not repeated.

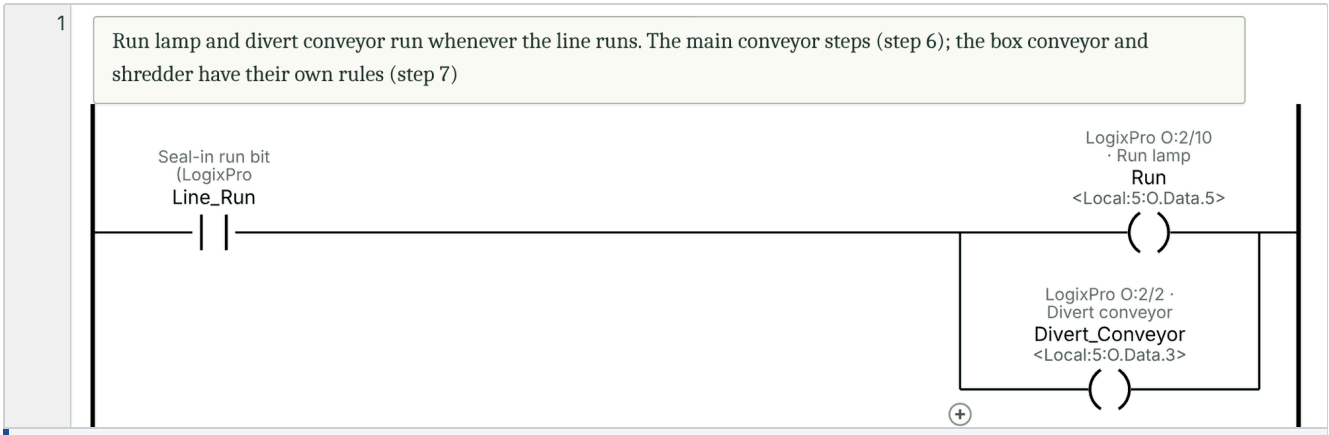
Tag	Type	How to make it	Description
`Start`	BOOL	alias for `Local:3:I.Data.2` — the Properties drawer's *tag* link makes it, then rename it	LogixPro I:1/0 · Start push button, NO
`Line_Run`	BOOL	Edit Tags: type the name, pick the type	Seal-in run bit (LogixPro B3:0/0)
`Stop`	BOOL	alias for `Local:3:I.Data.3` — the Properties drawer's *tag* link makes it, then rename it	LogixPro I:1/1 · Stop push button, NC (1 until pressed)
`Batch_Done`	BOOL	Edit Tags: type the name, pick the type	The large-bottle count has reached the thumbwheel preset
`Run`	BOOL	alias for `Local:5:O.Data.5` — the Properties drawer's *tag* link makes it, then rename it	LogixPro O:2/10 · Run lamp
`Divert_Conveyor`	BOOL	alias for `Local:5:O.Data.3` — the Properties drawer's *tag* link makes it, then rename it	LogixPro O:2/2 · Divert conveyor for large bottles
`Label_Req`	BOOL	Edit Tags: type the name, pick the type	Latched while the labeler is asked to apply; cleared at PadIn

RUNG 0



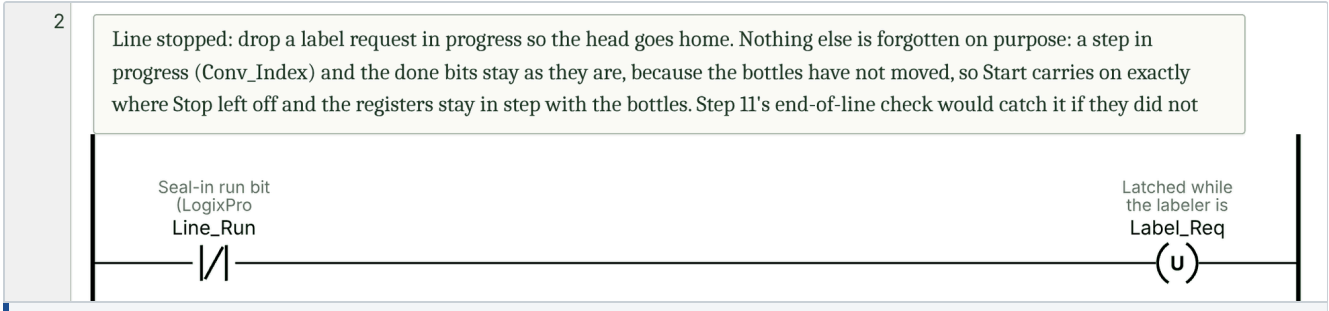

```
[XIC(Start),XIC(Line_Run)]XIC(Stop)XIO(Batch_Done)OTE(Line_Run);
```

RUNG 1




```
XIC(Line_Run)[OTE(Run),OTE(Divert_Conveyor)];
```

RUNG 2




```
XIO(Line_Run)OTU(Label_Req);
```

Rung 0 is chapter 4's seal-in with two extra conditions: `XIC(Stop)` rather than XIO because the stop button is normally closed — pressing it makes the tag 0 — and `XIO(Batch_Done)`, a bit step 8 will set when the batch is complete. Until then it is 0 and the XIO passes. **Rung 1** drives the run lamp and the divert conveyor whenever the line runs; the main conveyor is deliberately not here, because it steps (step 6), and

the box conveyor and the shredder have rules of their own (step 7). **Rung 2** is housekeeping: when the line stops for any reason, drop a label request that is in progress so the labeler's head goes home (step 10 explains the request; make `Label_Req` now, a BOOL, and Verify). Notice what this rung does *not* do. A stopped line keeps the step in progress, the dwell's time and every "done" bit, because the bottles have not moved: Start must carry on exactly where Stop left off, or the shift registers of step 3 and the bottles they describe drift apart. Step 11 has the check that proves it.

Run it

Run mode. Press Start on the panel: the Run lamp lights, the divert conveyor's rollers turn, `Line_Run` is 1 in Monitor Tags. Press Stop: everything drops. Press Start again and hold Stop while pressing Start: nothing, because the stop contact is open. That is the whole of step 1.

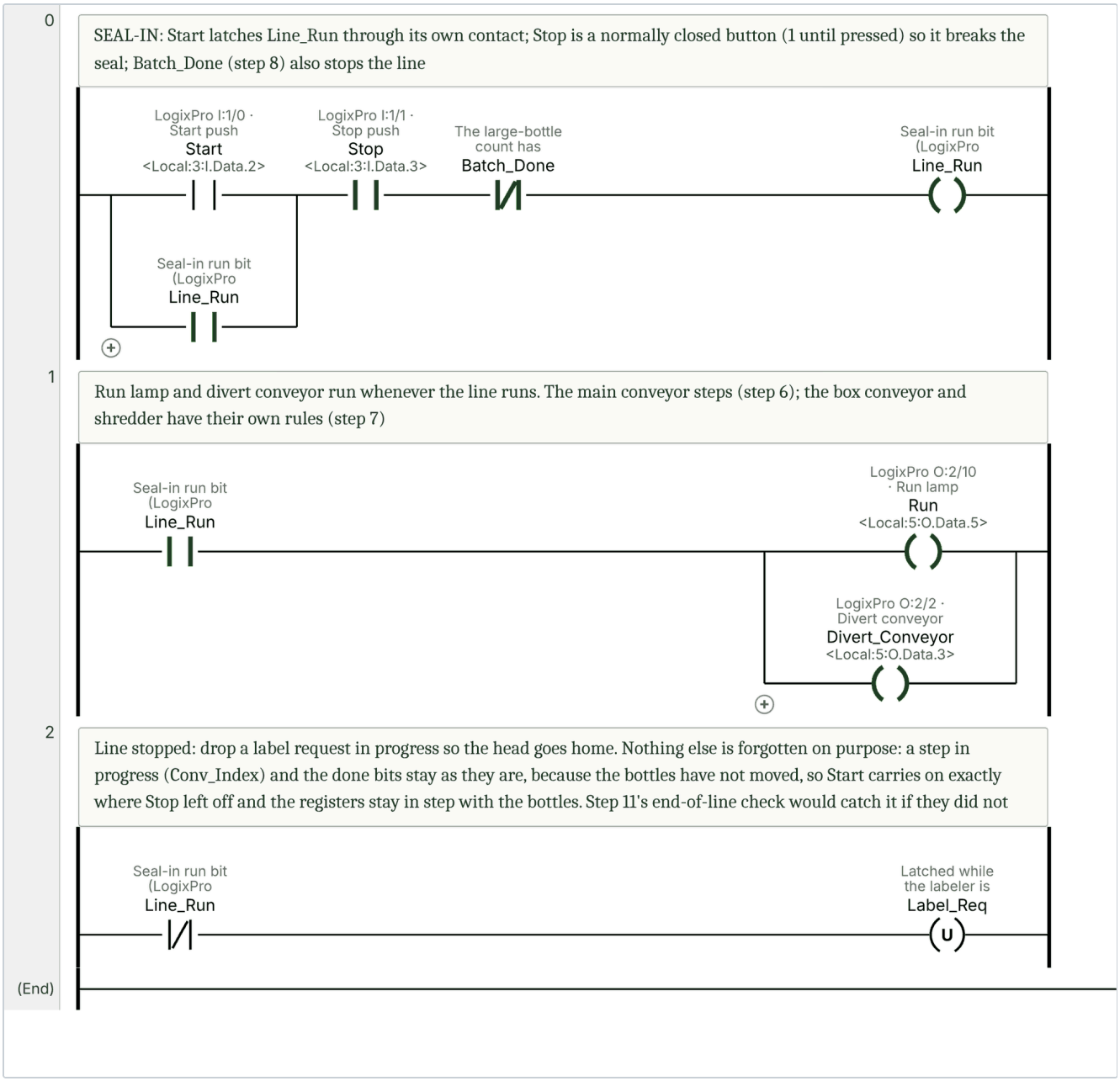


Figure 97. S1_Run online, with the finished line running: this is what your routine should look like once every step is in. Green is power flow.

TRY IT

Give the Run lamp a flash while the line is stopped but the controller is in Run mode: a TON that resets itself (chapter 7) and `XIO(Line_Run)XIC(Flash.TT)OTE(Run)` on a branch. Then take it out again — the lessons assume the lamp means running.

19 Lesson 4 — Step 2: the dwell timer and the feeder

The line indexes: belt stops, sensors read, bottle dropped, belt steps. The dwell is the stop. Add `JSR(S2_Dwell);` to MainRoutine under step 1 and write the routine:

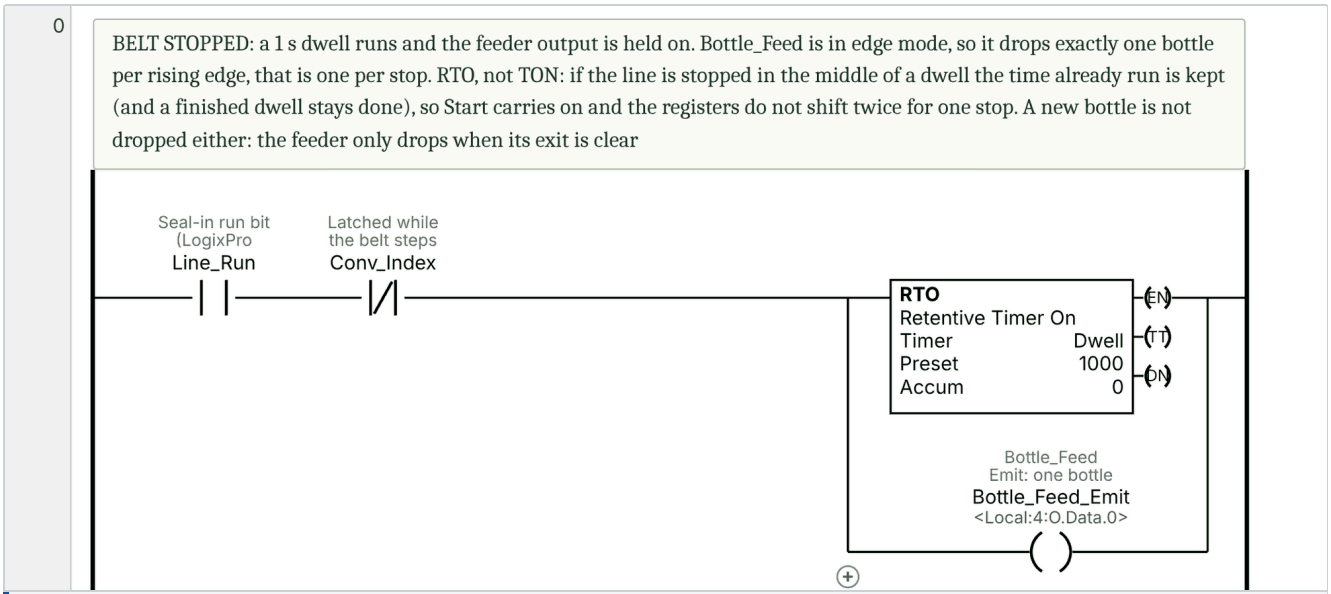
> **IN THE APP**
>
> Guided mode lesson 4 is this chapter: it checks that this lesson's tags are made, that `S2_Dwell` exists, is called from MainRoutine and verifies, and then watches the line while you run it and ticks each *Run it* result below as it happens. Help → Guided Mode, or start from File → New Project… → Guided: build the bottle line.

Tags this lesson needs

Create these before you type the rungs (Verify reports any you miss as unknown). Tags from earlier lessons are not repeated.

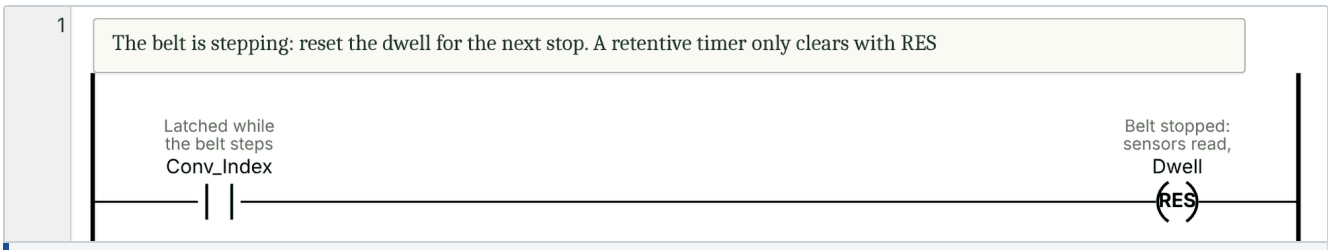
Tag	Type	How to make it	Description
`Conv_Index`	BOOL	Edit Tags: type the name, pick the type	Latched while the belt steps one pitch
`Dwell`	TIMER	Edit Tags: type the name, pick the type	Belt stopped: sensors read, feeder drops a bottle
`Bottle_Feed_Emit`	BOOL	alias for `Local:4:O.Data.0` — the Properties drawer's *tag* link makes it, then rename it	Bottle_Feed Emit: one bottle per rising edge

RUNG 0




```
XIC(Line_Run)XIO(Conv_Index)[RTO(Dwell,1000,0),OTE(Bottle_Feed_Emit)];
```

RUNG 1




```
XIC(Conv_Index)RES(Dwell);
```

While the line runs and the belt is not stepping (`Conv_Index`, which step 6 will latch while the belt moves), the `Dwell` timer counts a second, and on the same rung the feeder's `Emit` is energised; the feeder is in edge mode, so one bottle appears at the head of the line on the rising edge, once per stop. When the dwell is done, steps 3 and 6 act and `Conv_Index` goes true; **rung 1** resets the timer for the next stop.

Why an **RTO** and not a TON (chapter 7)? A TON resets the moment its rung goes false, and the rung goes false whenever the line stops. Press Stop half-way through a dwell, Start again, and a TON would time the whole second again; press Stop after the dwell was done and the registers (step 3, which shift when `Dwell.DN` comes on) would shift a *second* time for the same stop. The retentive timer keeps what it has: a stopped dwell resumes where it was, and a finished one stays done. A retentive timer only clears with `RES`, which is what rung 1 is for. The feeder does not drop a second bottle on Start either: an edge-mode feeder only drops when its exit is clear, and the bottle it dropped is still standing there.

Run it

Start the line. Exactly one bottle drops at the head of the belt and sits there, because nothing steps the belt yet; `Dwell.DN` comes on after a second and stays on (nothing resets it until the belt steps). Watch `Dwell.ACC` in Monitor Tags count to 1000 and stop. That is correct for now.

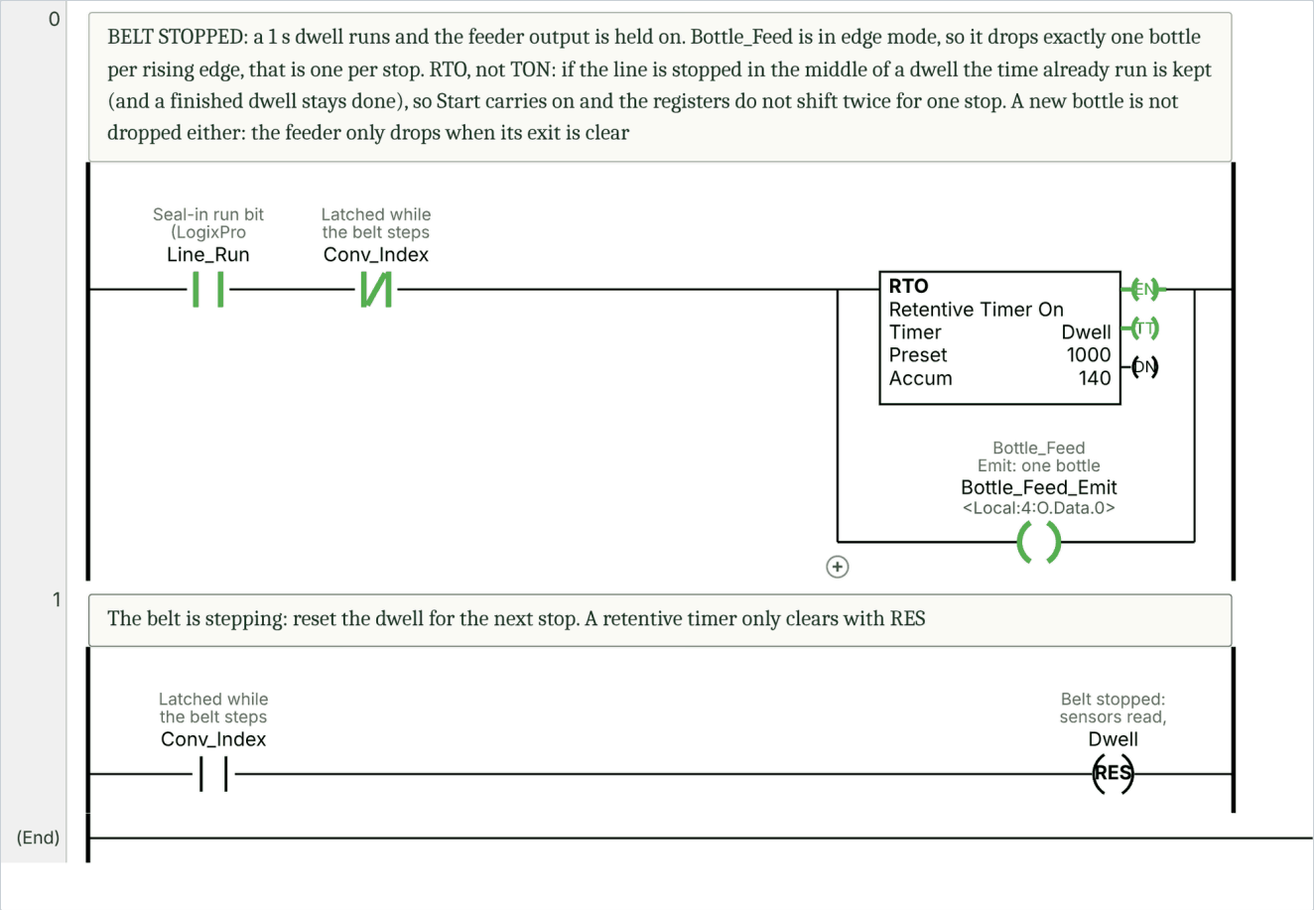


Figure 98. S2_Dwell online, with the finished line running: this is what your routine should look like once every step is in. Green is power flow.

TRY IT

Change the preset to 3000, Start, and press Stop when the accumulator reads about 1500: it holds 1500. Start again and it carries on to 3000. Set it back to 1000. In the finished line the dwell is the one number that sets the line's speed — the belt steps in about 0.7 s, so the line runs a bottle every 1.7 s.

20 Lesson 5 — Step 3: the shift registers

Chapter 10 is the theory; this is the rung. Add the JSR under step 2, and in `S3_Registers`:

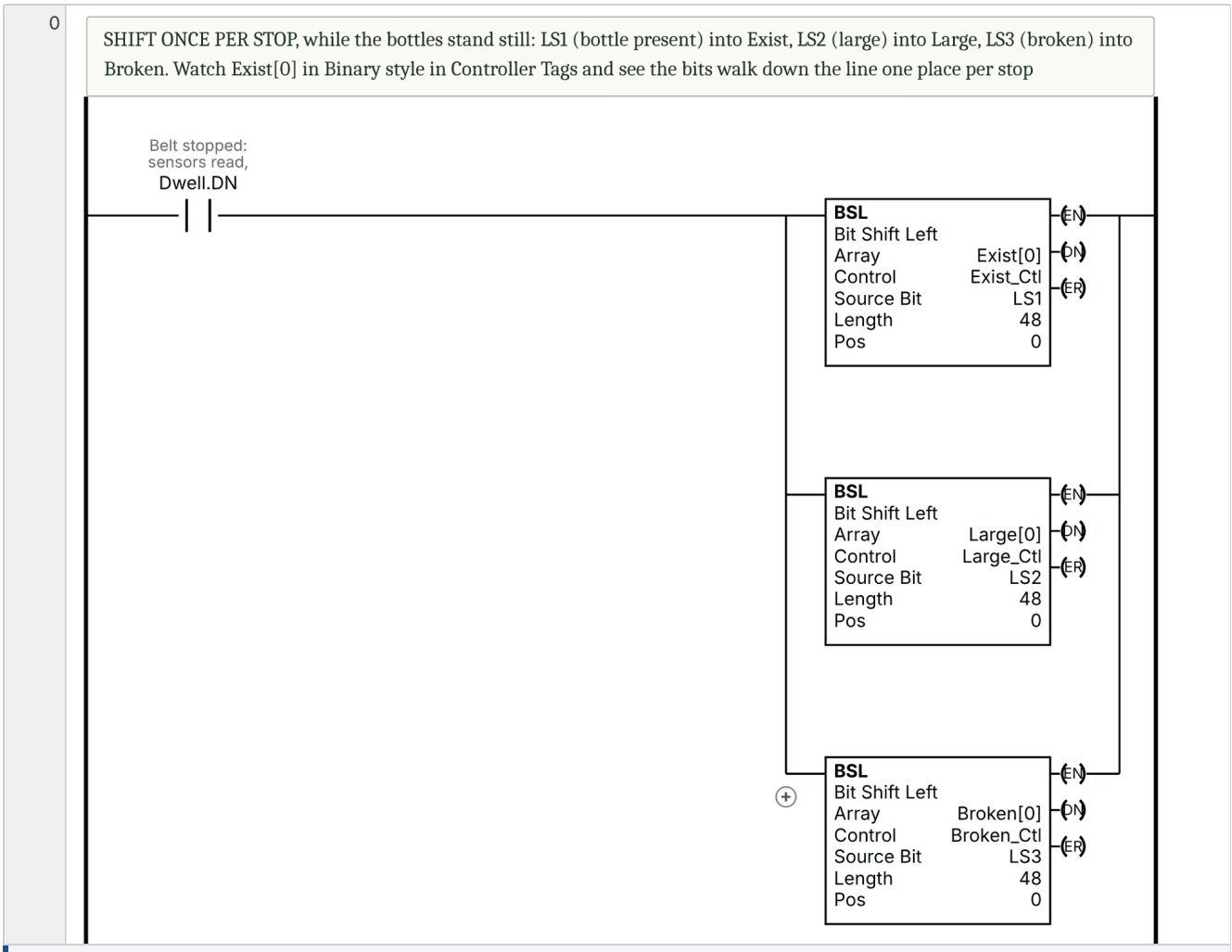
> **IN THE APP**
>
> Guided mode lesson 5 is this chapter: it checks that this lesson's tags are made, that `S3_Registers` exists, is called from MainRoutine and verifies, and then watches the line while you run it and ticks each *Run it* result below as it happens. Help → Guided Mode, or start from File → New Project… → Guided: build the bottle line.

Tags this lesson needs

Create these before you type the rungs (Verify reports any you miss as unknown). Tags from earlier lessons are not repeated.

Tag	Type	How to make it	Description
`Exist`	DINT[2]	Edit Tags: type the name, pick the type, Style Binary, Dim 2	Bottle present, one bit per stop (LogixPro B3:10). During the dwell: filler 6, capper 9, scrap gate 13, divert gate 20; after the shift (Dwell.DN) one bit higher
`Exist_Ctl`	CONTROL	Edit Tags: type the name, pick the type	LogixPro R6:1
`LS1`	BOOL	alias for `Local:1:I.Data.4` — the Properties drawer's *tag* link makes it, then rename it	LogixPro I:1/6 · Exist sensor: a bottle at the start of the line
`Large`	DINT[2]	Edit Tags: type the name, pick the type, Style Binary, Dim 2	Large bottle (LogixPro B3:20). During the dwell: filler 5, capper 8, scrap gate 12, divert gate 19; after the shift one bit higher
`Large_Ctl`	CONTROL	Edit Tags: type the name, pick the type	LogixPro R6:2
`LS2`	BOOL	alias for `Local:1:I.Data.5` — the Properties drawer's *tag* link makes it, then rename it	LogixPro I:1/7 · Size check: beam set so only a large bottle breaks it
`Broken`	DINT[2]	Edit Tags: type the name, pick the type, Style Binary, Dim 2	Broken bottle (LogixPro B3:30). During the dwell: filler 4, capper 7, scrap gate 11, divert gate 18; after the shift one bit higher
`Broken_Ctl`	CONTROL	Edit Tags: type the name, pick the type	LogixPro R6:3
`LS3`	BOOL	alias for `Local:1:I.Data.6` — the Properties drawer's *tag* link makes it, then rename it	LogixPro I:1/8 · Broken check: sees broken bottles only

RUNG 0




```
XIC(Dwell.DN)
[BSL(Exist[0],Exist_Ctl,LS1,48),BSL(Large[0],Large_Ctl,LS2,48),BSL(Broken[0],Br
oken_Ctl,LS3,48)];
```

At the end of every dwell the three registers shift once, each taking its head sensor into bit 0. The three CONTROL tags (`Exist_Ctl`, `Large_Ctl`, `Broken_Ctl`) belong to the BSLs; 48 is the length, more than the 33 stops on the belt, so a bottle is tracked all the way to the end and off.

Run it

Start the line and open Monitor Tags with `Exist`, `Large` and `Broken` expanded (they are Binary). After the first dwell, bit 0 of Exist is 1 — a bottle is at the head. Nothing moves, so the picture freezes. Toggle `Conv_Index` on and off by hand (Ctrl+T, twice): the dwell runs again, the registers shift, and the 1 walks to bit 1. The belt did not move, so the register is now wrong about the world — which is exactly why step 6 ties the shift to the encoder. Reset Sim before the next lesson.

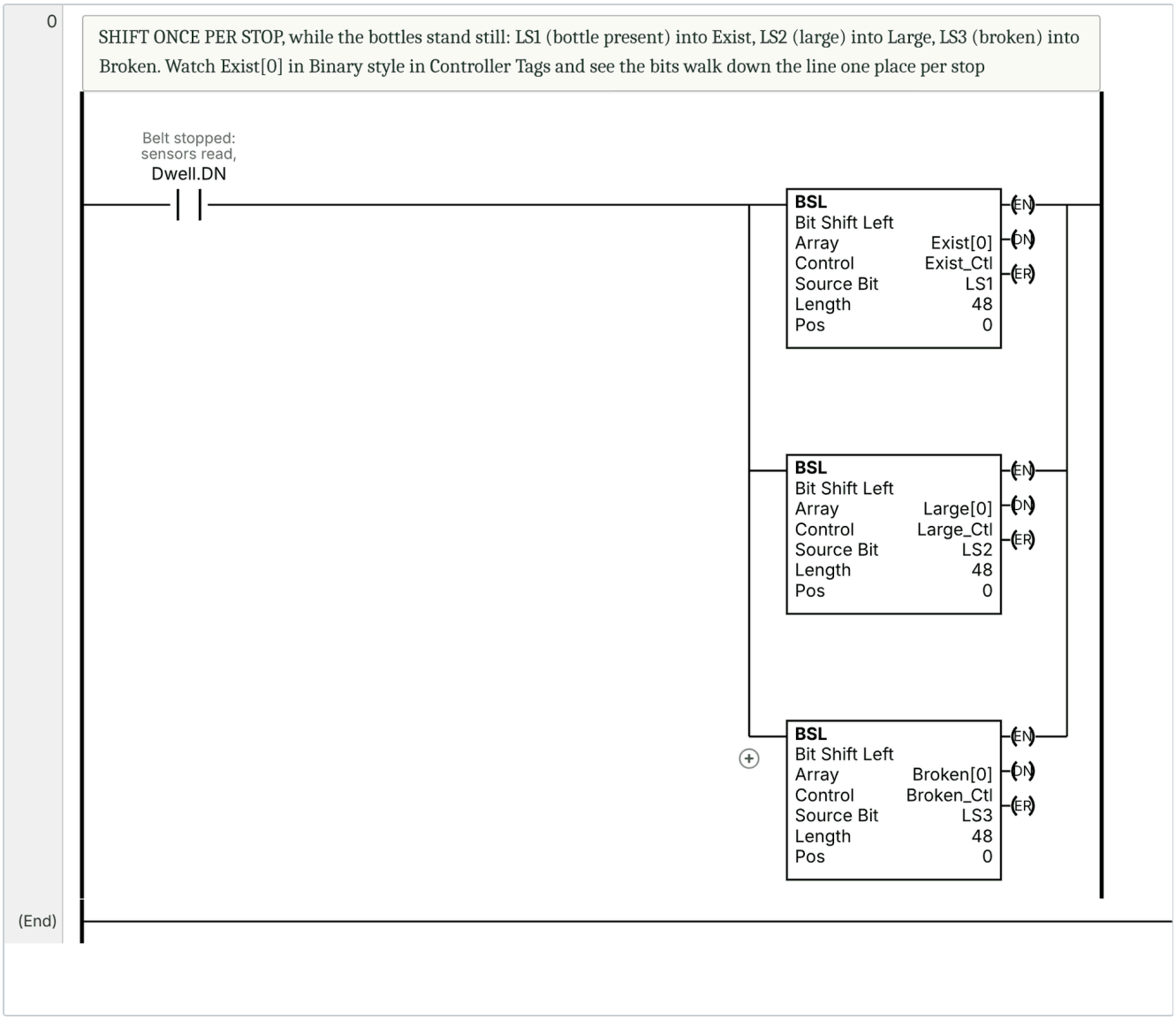


Figure 99. S3_Registers online, with the finished line running: this is what your routine should look like once every step is in. Green is power flow.

TRY IT

Set Bottle_Feed's *Broken bottles* to 50 % for a run and watch the Broken register fill; then back to 15.

21 Lesson 6 — Step 4: filling

The fill station needs five rungs: a request, two valves, a done latch and a clear. The request is written in the *after the shift* frame — it waits on `Dwell.DN`, which means the registers have just shifted and the belt is about to step — so it reads Exist bit 7 and Large bit 6 (chapter 10's table). Add the JSR under step 3 and write:

> **IN THE APP**
>
> Guided mode lesson 6 is this chapter: it checks that this lesson's tags are made, that `S4_Fill` exists, is called from MainRoutine and verifies, and then watches the line while you run it and ticks each *Run it* result below as it happens. Help → Guided Mode, or start from File → New Project… → Guided: build the bottle line.

Tags this lesson needs

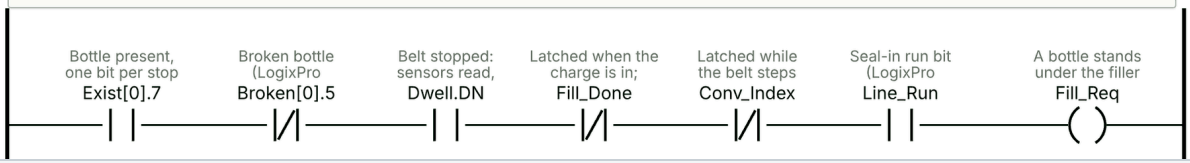
Create these before you type the rungs (Verify reports any you miss as unknown). Tags from earlier lessons are not repeated.

Tag	Type	How to make it	Description
`Fill_Done`	BOOL	Edit Tags: type the name, pick the type	Latched when the charge is in; cleared when the belt steps
`Fill_Req`	BOOL	Edit Tags: type the name, pick the type	A bottle stands under the filler and still needs its charge
`Fill_Tube`	BOOL	alias for `Local:4:O.Data.6` — the Properties drawer's *tag* link makes it, then rename it	LogixPro O:2/6 · Lower the fill tube
`LS4`	BOOL	alias for `Local:1:I.Data.9` — the Properties drawer's *tag* link makes it, then rename it	LogixPro I:1/9 · Fill tube down
`LS5`	BOOL	alias for `Local:1:I.Data.10` — the Properties drawer's *tag* link makes it, then rename it	LogixPro I:1/10 · Large charge delivered
`Large_Fill`	BOOL	alias for `Local:4:O.Data.7` — the Properties drawer's *tag* link makes it, then rename it	LogixPro O:2/7 · Large fill valve
`LS6`	BOOL	alias for `Local:1:I.Data.11` — the Properties drawer's *tag* link makes it, then rename it	LogixPro I:1/11 · Small charge delivered
`Small_Fill`	BOOL	alias for `Local:4:O.Data.8` — the Properties drawer's *tag* link makes it, then rename it	LogixPro O:2/8 · Small fill valve

RUNG 0

0

FILL REQUEST: the registers have just shifted (Dwell.DN), Exist bit 7 says a bottle stands under the nozzle and Broken bit 5 says it is whole; it has not had its charge yet. The shift happens at the END of the dwell, so a rung that runs after it reads one bit higher than a rung that runs during the dwell. A broken bottle gets no charge: it would only run out onto the belt




```
XIC(Exist[0].7)XIO(Broken[0].5)XIC(Dwell.DN)XIO(Fill_Done)XIO(Conv_Index)XIC(Line_Run)OTE(Fill_Req);
```

RUNG 1

1

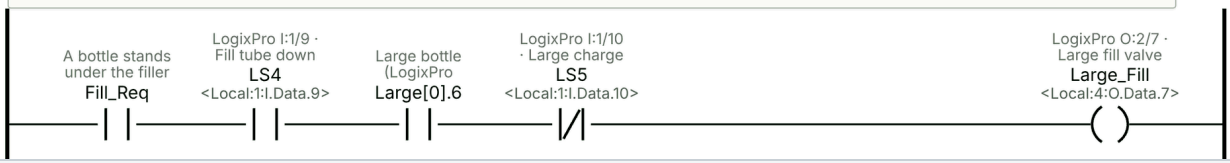
Tube down while a fill is requested; it comes back up the moment Fill_Done latches

```
XIC(Fill_Req)OTE(Fill_Tube);
```

RUNG 2

2

LARGE BOTTLE: open the large valve once the tube is down (LS4) until the large charge is in (LS5). Large[0].6, not .9: LS2 sits 40 px past LS1 and sees the bottle earlier, so the Large register runs three bits ahead of Exist at the same stop

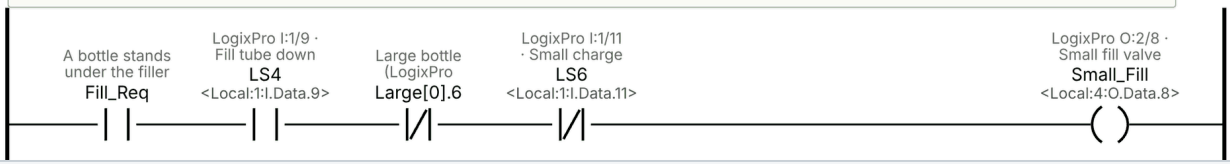



```
XIC(Fill_Req)XIC(LS4)XIC(Large[0].6)XIO(LS5)OTE(Large_Fill);
```

RUNG 3

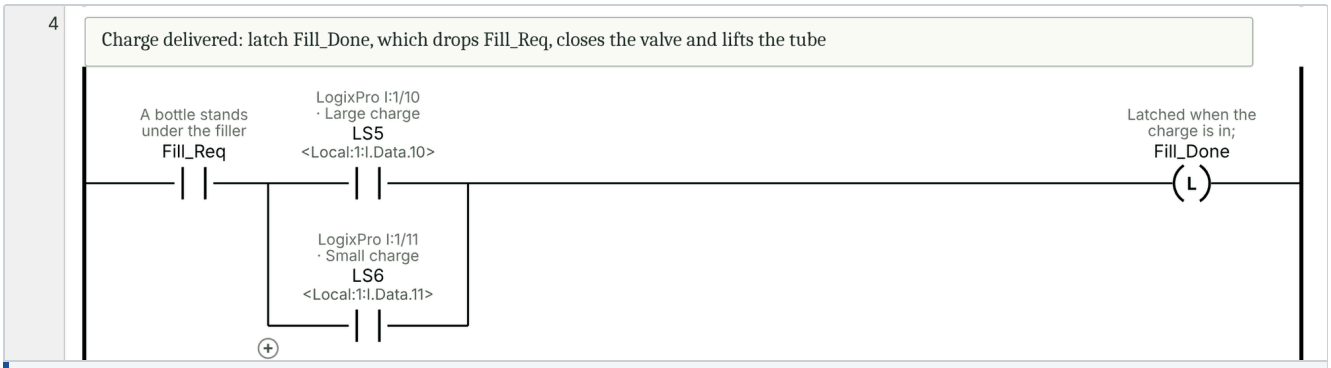
3

SMALL BOTTLE: the small valve until the small charge is in (LS6)



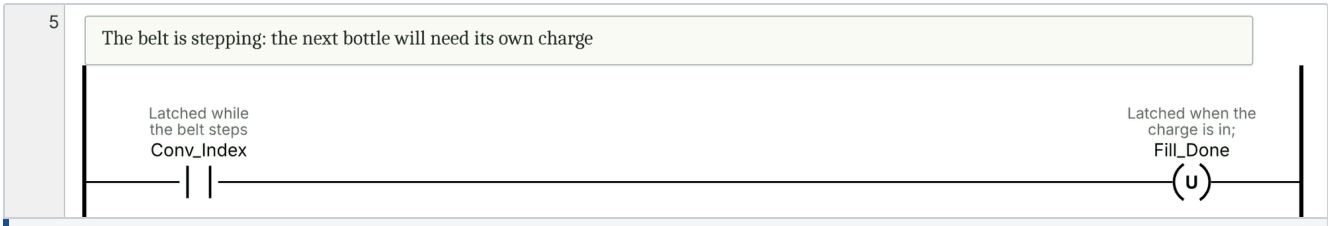

```
XIC(Fill_Req)XIC(LS4)XIO(Large[0].6)XIO(LS6)OTE(Small_Fill);
```

RUNG 4




```
XIC(Fill_Req)[XIC(LS5),XIC(LS6)]OTL(Fill_Done);
```

RUNG 5




```
XIC(Conv_Index)OTU(Fill_Done);
```

Rung 0: a bottle under the nozzle, not broken, not yet filled, the registers shifted, the belt not stepping, the line running → `Fill_Req`. The request is an OTE, so it drops the moment `Fill_Done` latches. **Rung 1** lowers the tube while requested; the filler reports `LS4` (tube down). **Rungs 2 and 3** open the large or the small valve, chosen by the Large register, until the filler's own float (`LS5` large charge, `LS6` small) says the charge is in. **Rung 4** latches `Fill_Done` at that moment, which drops the request and lifts the tube, and **rung 5** unlatches it when the belt steps so the next bottle is treated as new. Step 6 will not let the belt step while the tube is down.

Run it

Nothing fills yet, because no bottle ever reaches the filler: the belt does not step. This step is written now because step 6 depends on it. Verify it, and read the Side view of the filler: the head tank is 100 % (it fills from the reservoir by itself), the four lamps on the head block are dark.

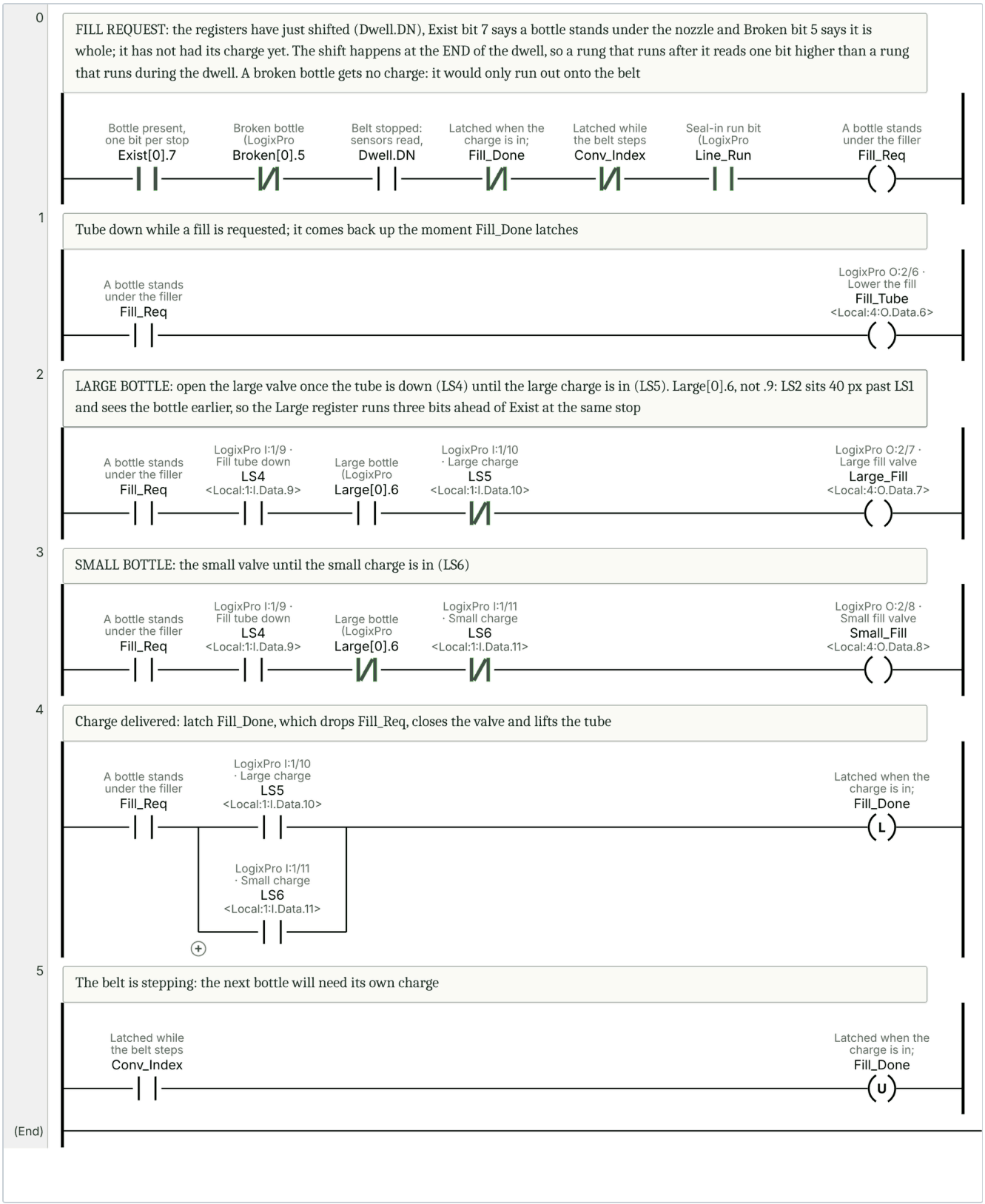


Figure 100. S4_Fill online, with the finished line running: this is what your routine should look like once every step is in. Green is power flow.

TRY IT

Why is the bottle Exist 7 but Large 6? Count the sensors: LS2 is one stop past LS1, so the same bottle is one bit further along in Exist than in Large. Move LS2 one stop in the scene and re-measure after step 6 is running.

22 Lesson 7 — Step 5: capping and the cap feeder

Same shape as the fill, plus the cap supply. Add the JSR and write:

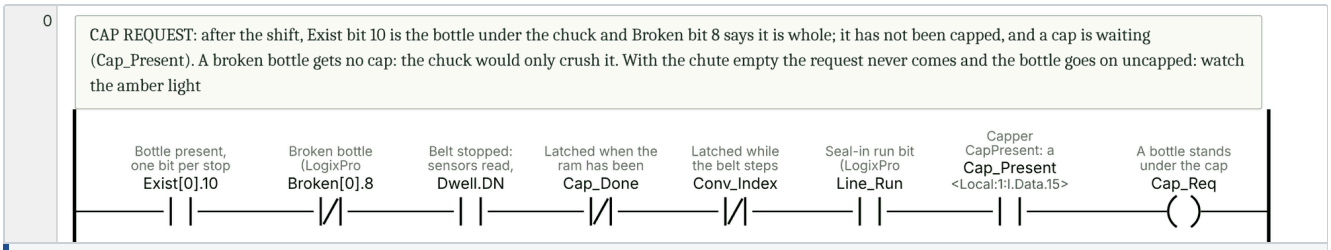
> **IN THE APP**
>
> Guided mode lesson 7 is this chapter: it checks that this lesson's tags are made, that `S5_Cap` exists, is called from MainRoutine and verifies, and then watches the line while you run it and ticks each *Run it* result below as it happens. Help → Guided Mode, or start from File → New Project… → Guided: build the bottle line.

Tags this lesson needs

Create these before you type the rungs (Verify reports any you miss as unknown). Tags from earlier lessons are not repeated.

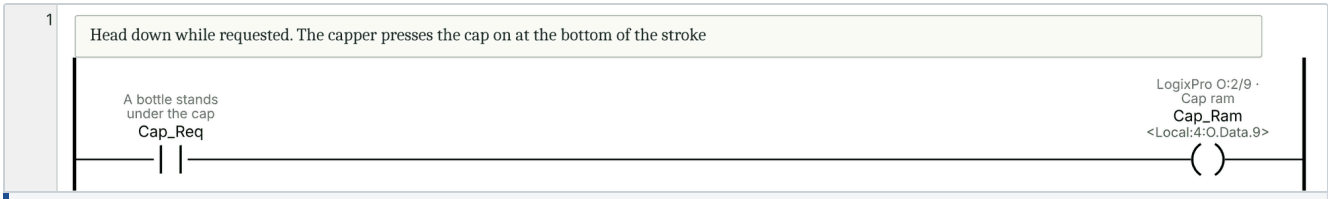
Tag	Type	How to make it	Description
`Cap_Done`	BOOL	Edit Tags: type the name, pick the type	Latched when the ram has been down; cleared when the belt steps
`Cap_Present`	BOOL	alias for `Local:1:I.Data.15` — the Properties drawer's *tag* link makes it, then rename it	Capper CapPresent: a cap is waiting at the chuck
`Cap_Req`	BOOL	Edit Tags: type the name, pick the type	A bottle stands under the cap ram and has not been capped
`Cap_Ram`	BOOL	alias for `Local:4:O.Data.9` — the Properties drawer's *tag* link makes it, then rename it	LogixPro O:2/9 · Cap ram
`LS7`	BOOL	alias for `Local:1:I.Data.13` — the Properties drawer's *tag* link makes it, then rename it	LogixPro I:1/12 · Cap ram down
`Cap_Low`	BOOL	alias for `Local:1:I.Data.14` — the Properties drawer's *tag* link makes it, then rename it	Capper CapLow: the chute is down to its low count
`Cap_Feeder`	BOOL	alias for `Local:4:O.Data.10` — the Properties drawer's *tag* link makes it, then rename it	Capper Feeder: runs the vibratory cap bowl, refilling the chute
`Cap_Fill_T`	TIMER	Edit Tags: type the name, pick the type	Cap feeder run-on after CapLow clears
`Cap_Green`	BOOL	alias for `Local:4:O.Data.11` — the Properties drawer's *tag* link makes it, then rename it	Capper green indicator (column)
`Cap_Amber`	BOOL	alias for `Local:4:O.Data.12` — the Properties drawer's *tag* link makes it, then rename it	Capper amber indicator (column)

RUNG 0



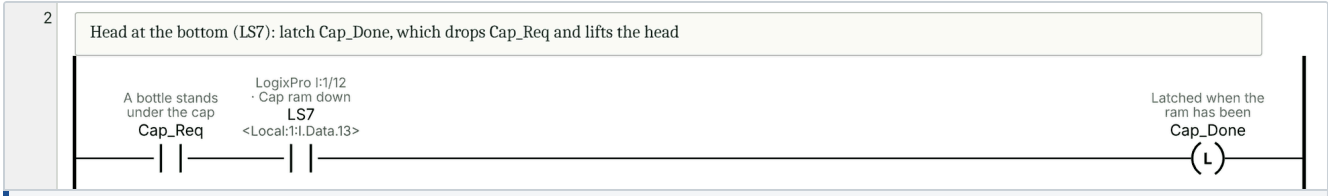

```
XIC(Exist[0].10)XIO(Broken[0].8)XIC(Dwell.DN)XIO(Cap_Done)XIO(Conv_Index)XIC(Line_Run)XIC(Cap_Present)OTE(Cap_Req);
```

RUNG 1



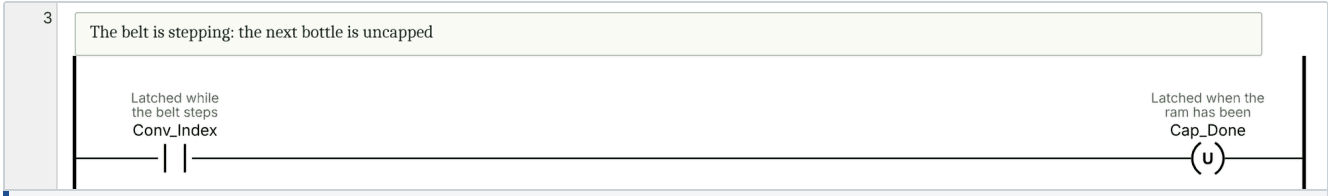

```
XIC(Cap_Req)OTE(Cap_Ram);
```

RUNG 2



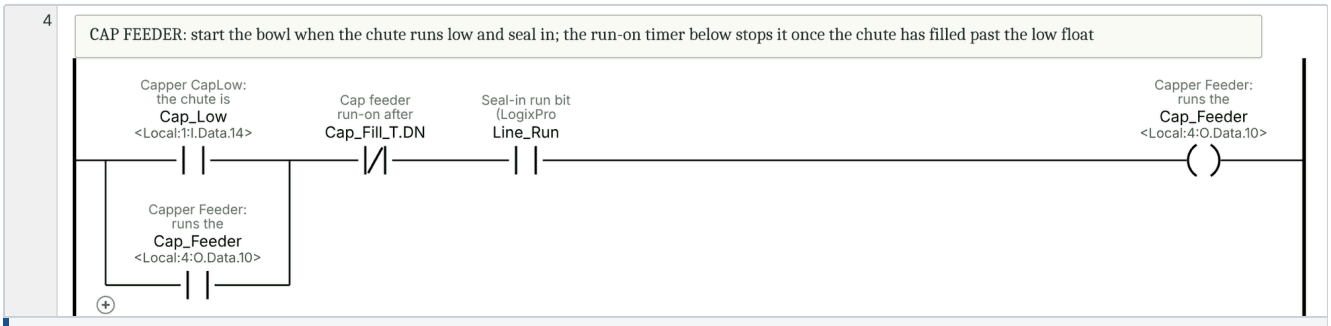

```
XIC(Cap_Req)XIC(LS7)OTL(Cap_Done);
```

RUNG 3



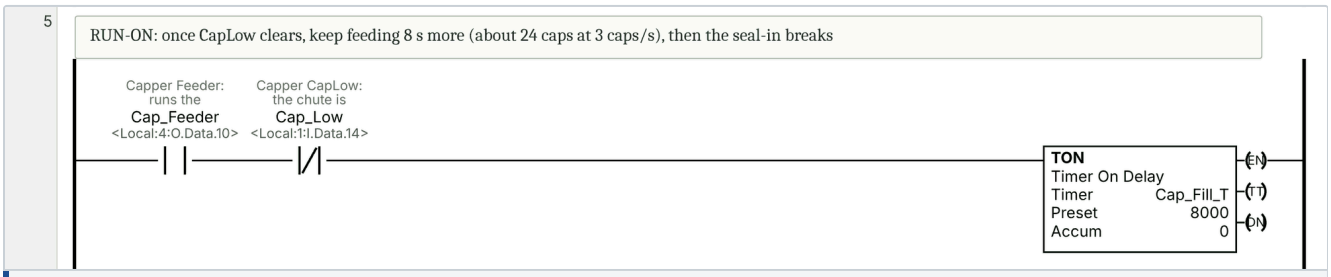

```
XIC(Conv_Index)OTU(Cap_Done);
```

RUNG 4



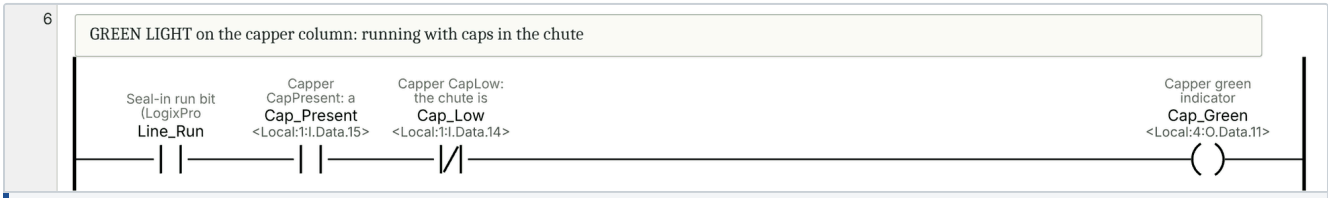

```
[XIC(Cap_Low),XIC(Cap_Feeder)]XIO(Cap_Fill_T.DN)XIC(Line_Run)OTE(Cap_Feeder);
```

RUNG 5



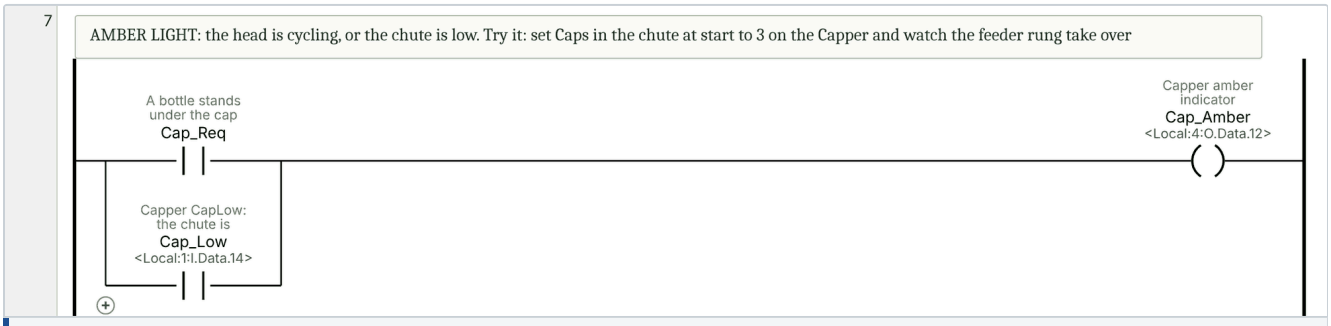

```
XIC(Cap_Feeder)XIO(Cap_Low)TON(Cap_Fill_T,8000,0);
```

RUNG 6




```
XIC(Line_Run)XIC(Cap_Present)XIO(Cap_Low)OTE(Cap_Green);
```

RUNG 7




```
[XIC(Cap_Req),XIC(Cap_Low)]OTE(Cap_Amber);
```

Rungs 0–3 mirror the fill station: a whole bottle under the chuck (Exist 10, not Broken 8), a cap waiting (`Cap_Present`), not yet capped → `Cap_Req`; the chuck comes down; `LS7` at the bottom of the stroke latches `Cap_Done`; the belt stepping clears it. The Broken bit matters here: a chuck that comes down on a

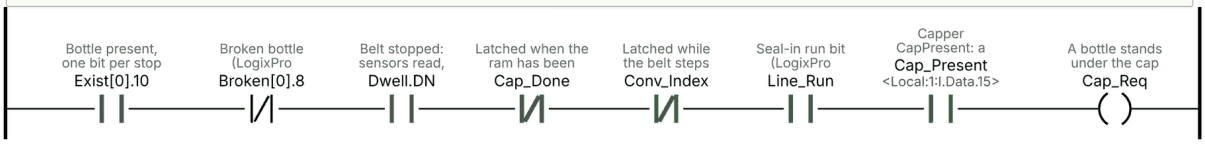
broken bottle crushes it (try it — exercise below). **Rungs 4–5** are the cap feeder: when the chute runs low, `Cap_Feeder` starts the rotary magazine and seals in; once `Cap_Low` clears, the run-on timer gives it eight more seconds (about 24 caps) and then the seal-in breaks. **Rungs 6–7** drive the two lights on the column: green while running with caps in the chute, amber while cycling or low.

Run it

Still nothing reaches the capper. But the lights work now: Start the line and the green column lamp comes on (Side view or 3D, or `Cap_Green` in Monitor Tags).

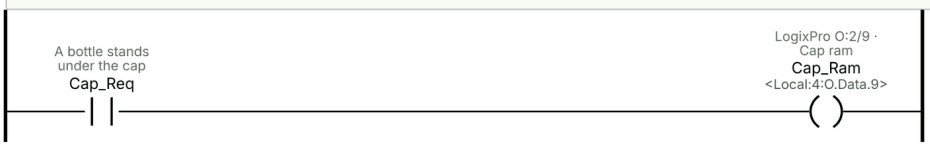
0

CAP REQUEST: after the shift, Exist bit 10 is the bottle under the chuck and Broken bit 8 says it is whole; it has not been capped, and a cap is waiting (Cap_Present). A broken bottle gets no cap: the chuck would only crush it. With the chute empty the request never comes and the bottle goes on uncapped: watch the amber light



1

Head down while requested. The capper presses the cap on at the bottom of the stroke



2

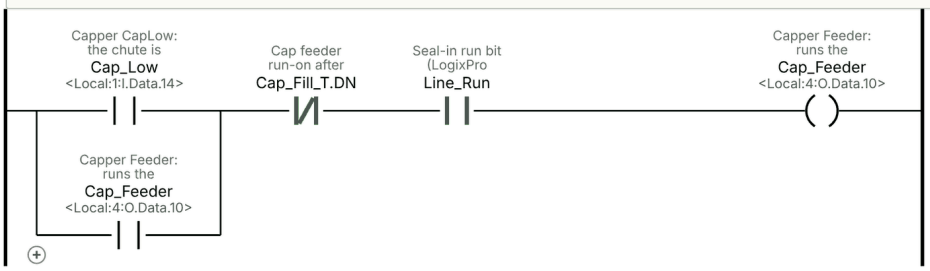
Head at the bottom (LS7): latch Cap_Done, which drops Cap_Req and lifts the head

3

The belt is stepping: the next bottle is uncapped

4

CAP FEEDER: start the bowl when the chute runs low and seal in; the run-on timer below stops it once the chute has filled past the low float



5

RUN-ON: once CapLow clears, keep feeding 8 s more (about 24 caps at 3 caps/s), then the seal-in breaks

6

GREEN LIGHT on the capper column: running with caps in the chute

7

AMBER LIGHT: the head is cycling, or the chute is low. Try it: set Caps in the chute at start to 3 on the Capper and watch the feeder rung take over

Figure 101. S5_Cap online, with the finished line running: this is what your routine should look like once every step is in. Green is power flow.

TRY IT

Set the Capper's *Caps in the chute at start* to 3, run the finished line later, and watch the amber lamp, the feeder rung and the magazine spin as the chute refills. Then take `XIO(Broken[0].8)` out of rung 0 and watch what the chuck does to a broken bottle.

23 Lesson 8 — Step 6: index one pitch

This is the step that makes it a line. The belt must move exactly one pitch and stop: the encoder on `Main_Conv` gives one rising edge of channel A per 40 px, so the rule is *latch Conv_Index, run the belt, unlatch on the next edge*. Add the JSR under step 5 — after fill and cap, so their requests are seen in the same scan — and write:

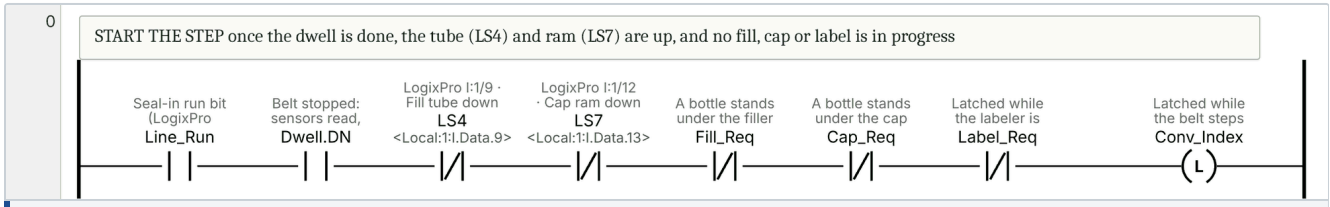
> **IN THE APP**
>
> Guided mode lesson 8 is this chapter: it checks that this lesson's tags are made, that `S6_Index` exists, is called from MainRoutine and verifies, and then watches the line while you run it and ticks each *Run it* result below as it happens. Help → Guided Mode, or start from File → New Project… → Guided: build the bottle line.

Tags this lesson needs

Create these before you type the rungs (Verify reports any you miss as unknown). Tags from earlier lessons are not repeated.

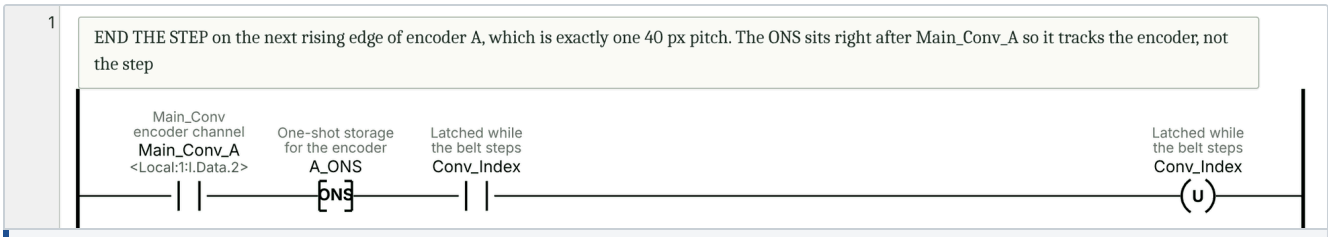
Tag	Type	How to make it	Description
`Main_Conv_A`	BOOL	alias for `Local:1:I.Data.2` — the Properties drawer's *tag* link makes it, then rename it	Main_Conv encoder channel A: one rising edge per 40 px pitch
`A_ONS`	BOOL	Edit Tags: type the name, pick the type	One-shot storage for the encoder edge
`Main_Conveyor`	BOOL	alias for `Local:4:O.Data.1` — the Properties drawer's *tag* link makes it, then rename it	LogixPro O:2/0 · Main conveyor motor

RUNG 0



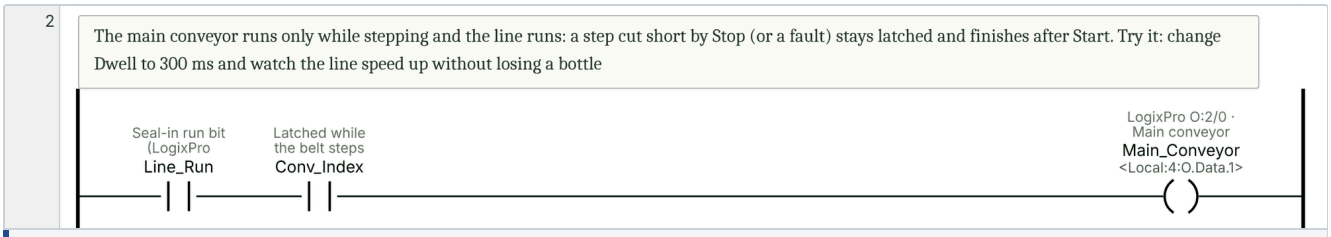

```
XIC(Line_Run)XIC(Dwell.DN)XIO(LS4)XIO(LS7)XIO(Fill_Req)XIO(Cap_Req)XIO(Label_Req)OTL(Conv_Index);
```

RUNG 1




```
XIC(Main_Conv_A)ONS(A_ONS)XIC(Conv_Index)OTU(Conv_Index);
```

RUNG 2




```
XIC(Line_Run)XIC(Conv_Index)OTE(Main_Conveyor);
```

Rung 0 starts a step: the dwell is done, no fill or cap is in progress (the request bits are off), and neither the tube (`LS4`) nor the chuck (`LS7`) is down. **Rung 1** ends it: the next rising edge of encoder A — the ONS sits right after `Main_Conv_A` so it tracks the encoder, not the step — unlatches `Conv_Index`. **Rung 2** runs the motor only while stepping and only while the line runs. A Stop in the middle of a step leaves `Conv_Index` latched (step 1 no longer touches it) and stops the motor; Start runs the motor again and the step ends on the same encoder edge it would have, one pitch from where it began. That is what keeps the registers and the bottles together.

Run it

Start the line. The belt steps, stops, a bottle drops, a second passes, the belt steps. Bottles walk down the line one pitch at a time; the registers walk with them; at stop 7 the tube comes down and the bottle fills (large charge for a large bottle, small for a small — watch the head-block lamps); at stop 10 it is capped, and the magazine indexes one pocket per cap. Open the Simulator Log: every fill and cap is there. Bottles reach the end of the line and are removed — all of them, broken ones included, which step 7 will fix.

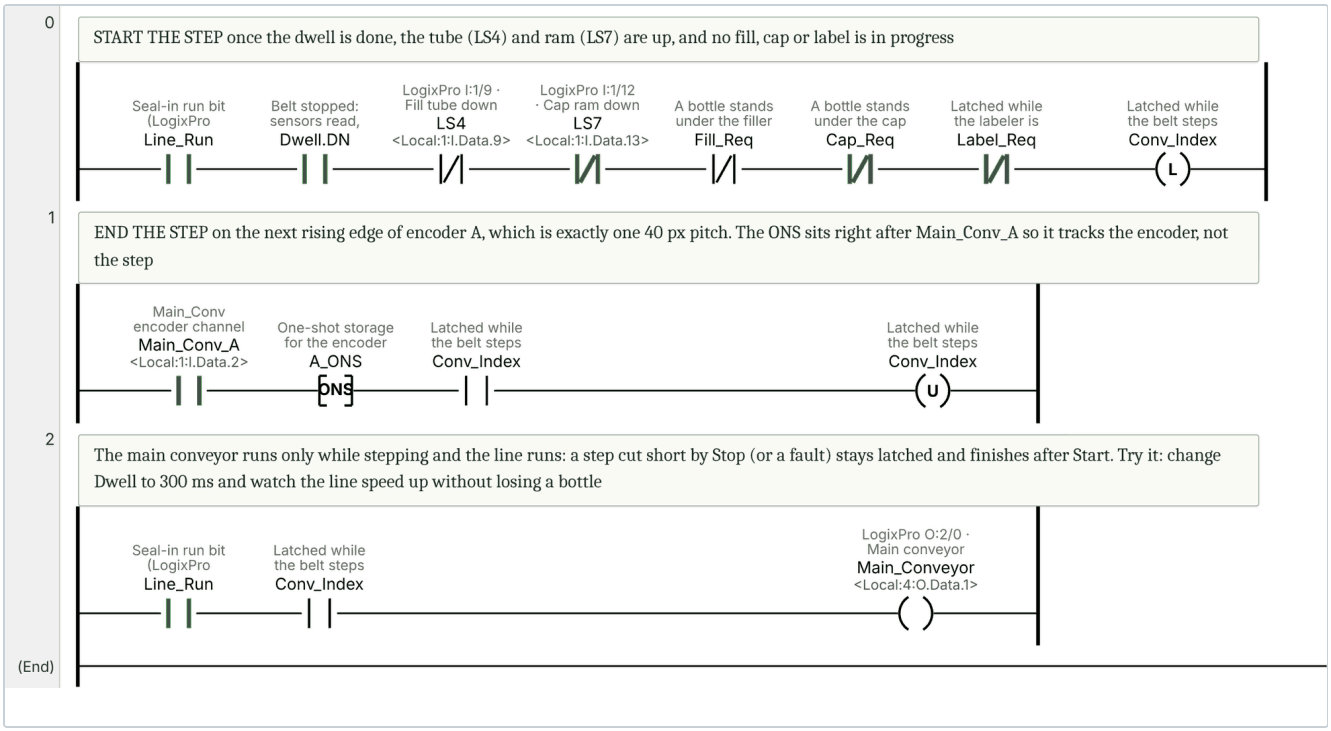


Figure 102. S6_Index online, with the finished line running: this is what your routine should look like once every step is in. Green is power flow.

TRY IT

Move `JSR(S6_Index)` above `JSR(S4_Fill)` in MainRoutine. The belt now sometimes steps with the tube down, because the fill request raised in step 4 is not seen until the next scan. Put it back. This is the scan order lesson in one move.

24 Lesson 9 — Step 7: the gates, the boxes and the shredder

Two gates, and the machinery below them. The gate rungs are written in the *during the dwell* frame — qualified by `Dwell.TT`, the timer timing — so they read the bits for bottles standing on the belt now: Broken 11 at the scrap gate; Exist 20, Large 19, Broken 18 at the divert gate. Add the JSR and write:

> **IN THE APP**
>
> Guided mode lesson 9 is this chapter: it checks that this lesson's tags are made, that `S7_Sort` exists, is called from MainRoutine and verifies, and then watches the line while you run it and ticks each *Run it* result below as it happens. Help → Guided Mode, or start from File → New Project… → Guided: build the bottle line.

Tags this lesson needs

Create these before you type the rungs (Verify reports any you miss as unknown). Tags from earlier lessons are not repeated.

Tag	Type	How to make it	Description
`Scrap_Gate`	BOOL	alias for `Local:5:O.Data.0` — the Properties drawer's *tag* link makes it, then rename it	LogixPro O:2/4 · Scrap gate: swings a bottle into the grinder
`Divert_Gate`	BOOL	alias for `Local:5:O.Data.2` — the Properties drawer's *tag* link makes it, then rename it	LogixPro O:2/5 · Divert gate: swings a bottle onto the divert conveyor
`LS10`	BOOL	alias for `Local:3:I.Data.0` — the Properties drawer's *tag* link makes it, then rename it	LogixPro I:1/15 · Scrap box under the grinder chute
`Box_Full`	BOOL	alias for `Local:2:I.Data.9` — the Properties drawer's *tag* link makes it, then rename it	Grinder BoxFull: the box under the scrap drop holds its share
`Scrap_Conveyor`	BOOL	alias for `Local:5:O.Data.4` — the Properties drawer's *tag* link makes it, then rename it	LogixPro O:2/1 · Box conveyor under the grinder
`Scrap_Waiting`	BOOL	alias for `Local:2:I.Data.8` — the Properties drawer's *tag* link makes it, then rename it	Grinder Detect: a broken bottle is waiting in the hopper
`Grinder`	BOOL	alias for `Local:5:O.Data.1` — the Properties drawer's *tag* link makes it, then rename it	LogixPro O:2/3 · Grinder motor

RUNG 0

0

SCRAP: while the belt stands still (Dwell timing) Broken bit 11 is the bottle at the scrap gate; the arm swings it off the raised belt and it drops into the shredder hopper below. The gate works during the dwell on purpose: the registers shift at the end of the dwell, and an arm that fired after the shift would push the bottle still standing at the gate, not the one the register names

```
XIC(Dwell.TT)XIC(Broken[0].11)OTE(Scrap_Gate);
```

RUNG 1

1

DIVERT: a small, unbroken bottle at the divert gate (Exist bit 20, not Large 19, not Broken 18, all in the dwell frame) is swung off the raised belt and rolls down the catch incline onto the divert conveyor. Large bottles carry on to the end of the line. The Broken bit matters: a broken bottle was already scrapped, but its Exist bit is still set, so without it the arm would fire at an empty slot

```
XIC(Dwell.TT)XIC(Exist[0].20)XIO(Large[0].19)XIO(Broken[0].18)OTE(Divert_Gate);
```

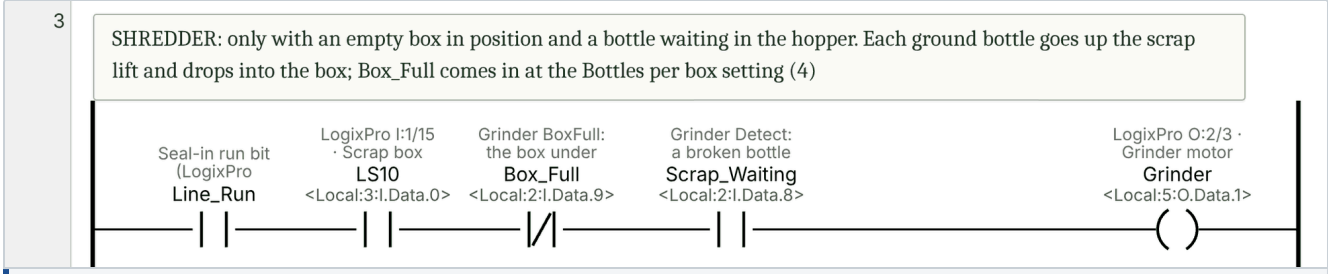
RUNG 2

2

BOX CONVEYOR: run until a box reaches LS10, which puts it under the scrap drop; run again once that box is full to carry it out and bring the next one in

```
XIC(Line_Run)[XIO(LS10),XIC(Box_Full)]OTE(Scrap_Conveyor);
```

RUNG 3




```
XIC(Line_Run)XIC(LS10)XIO(Box_Full)XIC(Scrap_Waiting)OTE(Grinder);
```

Rung 0: a broken bottle at the scrap gate → the arm swings it off the deck and it drops into the shredder hopper. **Rung 1**: a small, unbroken bottle at the divert gate → down the catch incline to the divert conveyor; large bottles carry on to the end. The Exist bit stops the gate firing at empty slots at start-up; the Broken bit stops it firing at the empty slot a scrapped bottle left behind, because its Exist bit is still set. **Rung 2** runs the box conveyor until a box reaches `LS10` under the chute, and again once the box is full to carry it out and bring the next one. **Rung 3** runs the shredder only with an empty box in position and a bottle waiting in the hopper.

Why the dwell frame? A gate arm acts on whatever is standing at the gate. The registers shift at the end of the dwell, before the belt moves; a rung that fired on the shifted bit would swing the arm while the *previous* bottle still stands there and push the wrong one. Qualifying with `Dwell.TT` fires the arm only while the belt is still and the bits describe the bottles in front of it.

Run it

Start the line with the Side view on the grinder. A broken bottle reaches the scrap gate; the door swings, the bottle drops into the hopper; the shredder waits (red lamp on the frame) until a box is under the chute, then runs — glass falls into the box, the IN BOX count rises; at four the box conveyor carries the box out and brings the next. Small bottles go down the incline and run to Divert_Out; large ones to Main_Out. The Simulator Log should show no *no box* or *pushed off with nothing to land on* lines. (In Part IV's own test the line runs three minutes with nothing lost.)

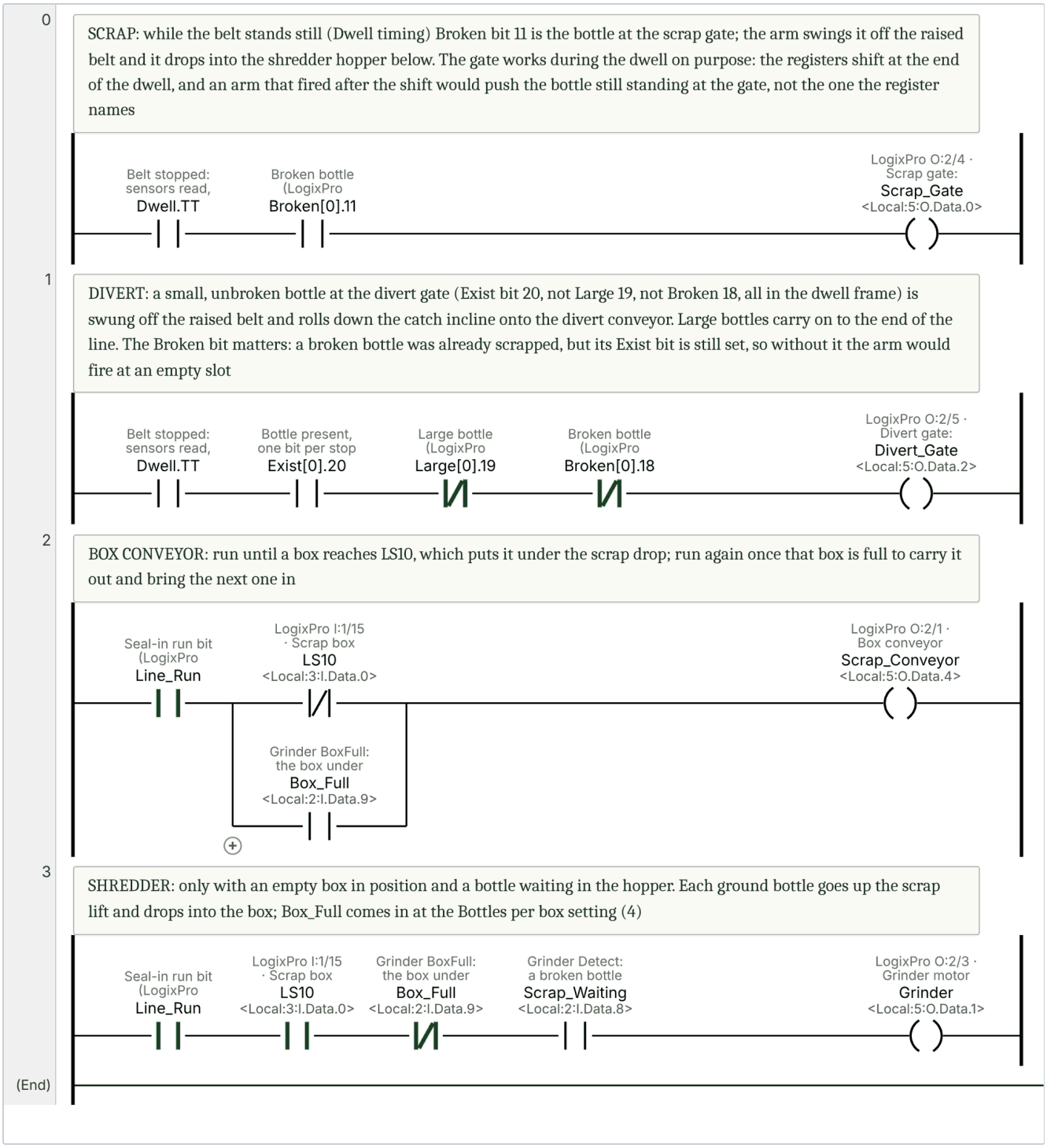


Figure 103. S7_Sort online, with the finished line running: this is what your routine should look like once every step is in. Green is power flow.

TRY IT

Take `XIC(Dwell.TT)` off rung 1 and watch which bottle the arm pushes. Then change *Bottles per box* on the Grinder to 6 and make the BOXES display (step 8) still count right.

25 Lesson 10 — Step 8: the displays, the preset and Enter

Four counts and a batch stop. Add the JSR and write:

> **IN THE APP**
>
> Guided mode lesson 10 is this chapter: it checks that this lesson's tags are made, that `S8_Count` exists, is called from MainRoutine and verifies, and then watches the line while you run it and ticks each *Run it* result below as it happens. Help → Guided Mode, or start from File → New Project… → Guided: build the bottle line.

Tags this lesson needs

Create these before you type the rungs (Verify reports any you miss as unknown). Tags from earlier lessons are not repeated.

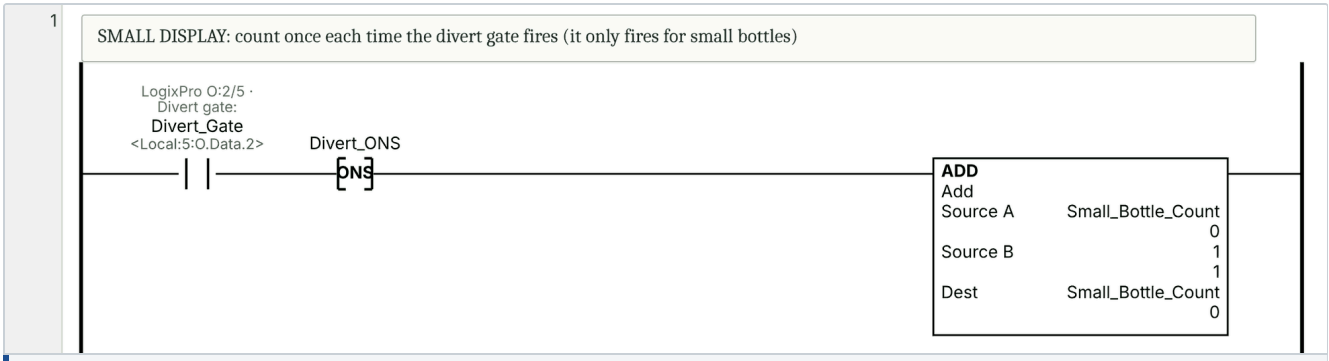
Tag	Type	How to make it	Description
`Scrap_ONS`	BOOL	Edit Tags: type the name, pick the type	
`Scrap_Bottle_Count`	DINT	Edit Tags: type the name, pick the type	LogixPro O:4/2 · Scrap bottle count, shown on the SCRAP display
`Divert_ONS`	BOOL	Edit Tags: type the name, pick the type	
`Small_Bottle_Count`	DINT	Edit Tags: type the name, pick the type	LogixPro O:4/1 · Small bottle count, shown on the SMALL display
`Large_ONS`	BOOL	Edit Tags: type the name, pick the type	
`Large_Bottle_Count`	DINT	Edit Tags: type the name, pick the type	LogixPro O:4/0 · Large bottle count, shown on the LARGE display
`Box_ONS`	BOOL	Edit Tags: type the name, pick the type	
`Scrap_Box_Count`	DINT	Edit Tags: type the name, pick the type	LogixPro O:4/3 · Scrap box count, shown on the BOXES display
`Thumbwheel`	INT	alias for `Local:9:I.Ch0Data` — the Properties drawer's *tag* link makes it, then rename it	LogixPro I:3 · Thumbwheel switch, read as a number (BCD on the real one)
`Enter`	BOOL	alias for `Local:3:I.Data.6` — the Properties drawer's *tag* link makes it, then rename it	Thumbwheel Enter button

Tag	Type	How to make it	Description
`Enter_ONS`	BOOL	Edit Tags: type the name, pick the type	

RUNG 0

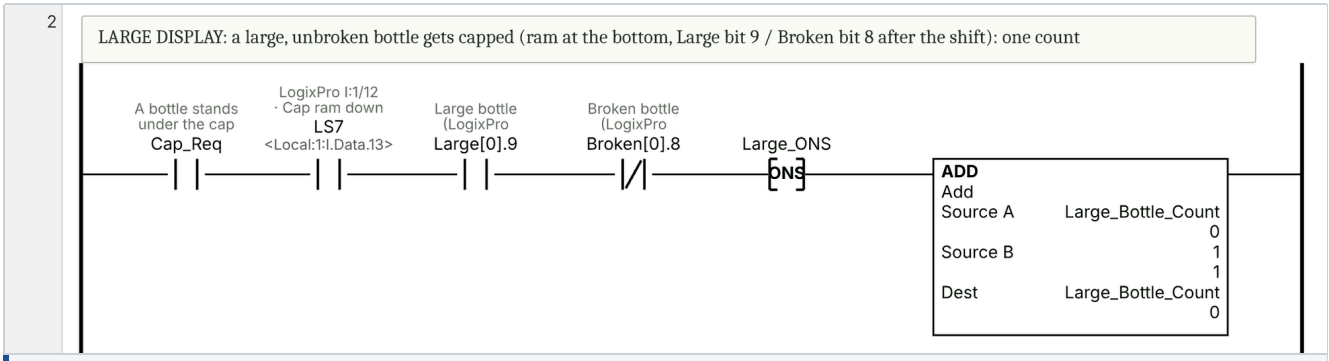
```
XIC(Scrap_Gate)ONS(Scrap_ONS)ADD(Scrap_Bottle_Count,1,Scrap_Bottle_Count);
```

RUNG 1




```
XIC(Divert_Gate)ONS(Divert_ONS)ADD(Small_Bottle_Count,1,Small_Bottle_Count);
```

RUNG 2




```
XIC(Cap_Req)XIC(LS7)XIC(Large[0].9)XIO(Broken[0].8)ONS(Large_ONS)ADD(Large_Bott
le_Count,1,Large_Bottle_Count);
```

RUNG 3

3

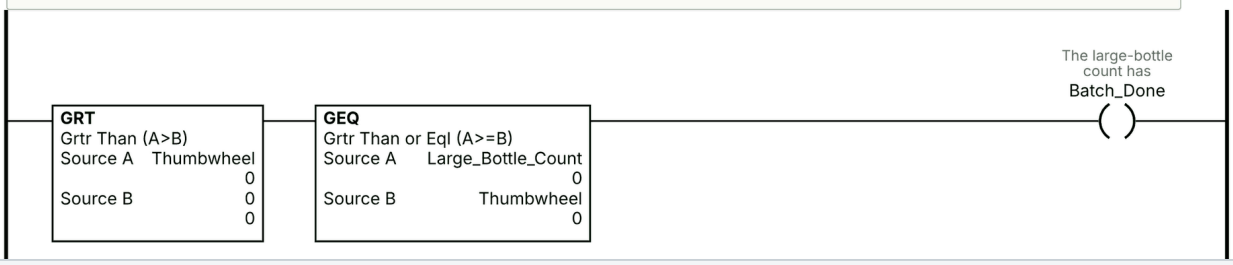
BOXES DISPLAY: one count each time a box fills

```
XIC(Box_Full)ONS(Box_ONS)ADD(Scrap_Box_Count,1,Scrap_Box_Count);
```

RUNG 4

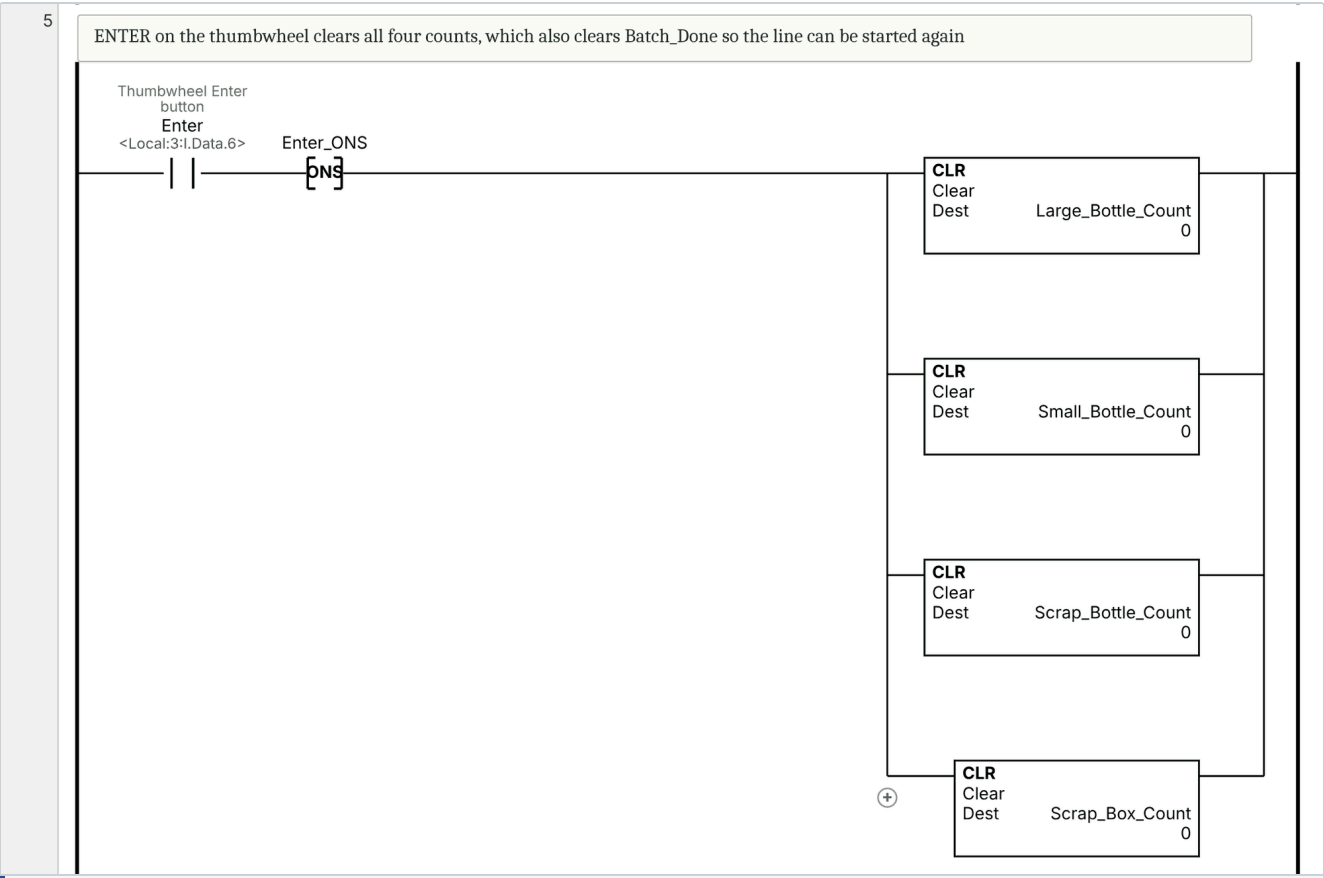
4

BATCH PRESET: dial a number on the thumbwheel and the line stops itself when that many large bottles have been capped (0 = no limit). Batch_Done breaks the seal-in in step 1




```
GRT(Thumbwheel,0)GEQ(Large_Bottle_Count,Thumbwheel)OTE(Batch_Done);
```

RUNG 5




```
XIC(Enter)ONS(Enter_ONS)
[CLR(Large_Bottle_Count),CLR(Small_Bottle_Count),CLR(Scrap_Bottle_Count),CLR(Sc
rap_Box_Count)];
```

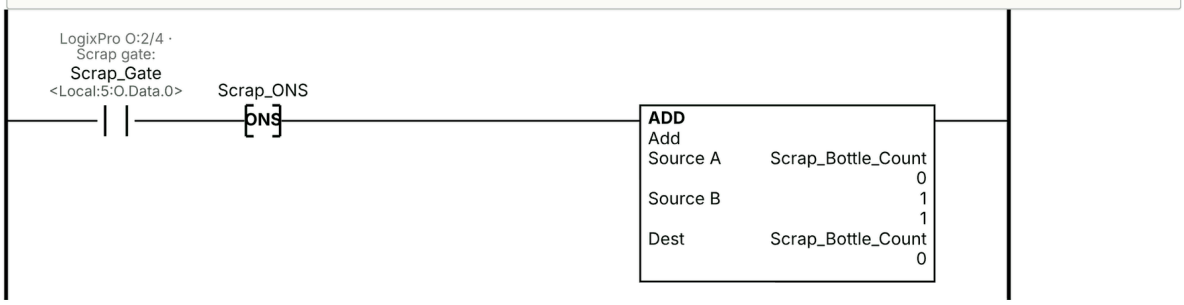
The four ADD rungs count once per event with a one-shot: the scrap gate firing, the divert gate firing (small bottles), a large unbroken bottle being capped (`Cap_Req` with `LS7`, Large 9, not Broken 8 — after the shift), and a box filling (`Box_Full`). The displays on the panel show those tags, and the toolbar counters read them too. **Rung 4** is the batch preset: a number dialled on the thumbwheel greater than zero, and the large count reaching it, sets `Batch_Done`, which step 1's seal-in breaks on; **rung 5** clears all four counts on Enter, which also clears `Batch_Done` so Start works again.

Run it

Dial 0020 on the thumbwheel and press Start. The line stops itself when the LARGE display reaches 20, the Run lamp goes out. Press Enter: the counts zero. Press Start: it runs again.

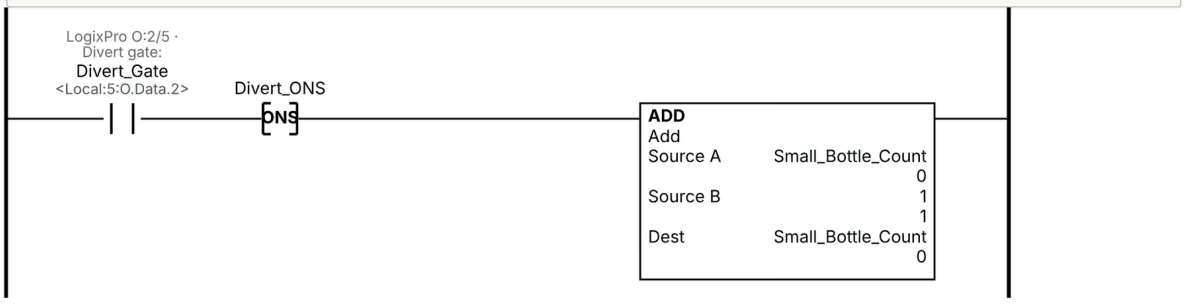
0

SCRAP DISPLAY: count once each time the scrap gate fires



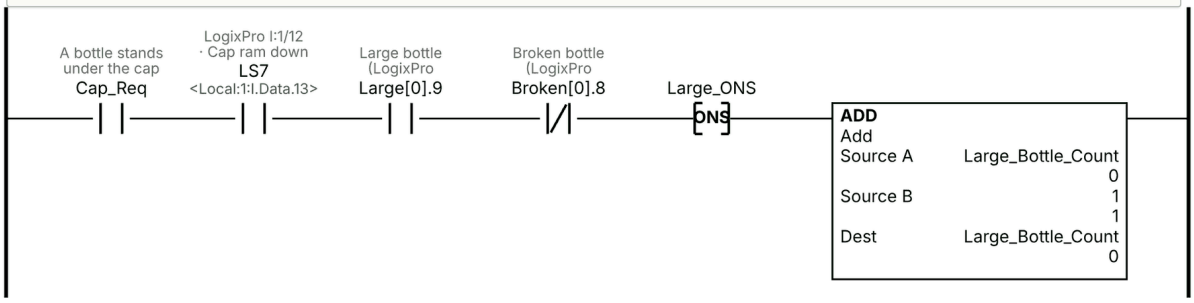
1

SMALL DISPLAY: count once each time the divert gate fires (it only fires for small bottles)



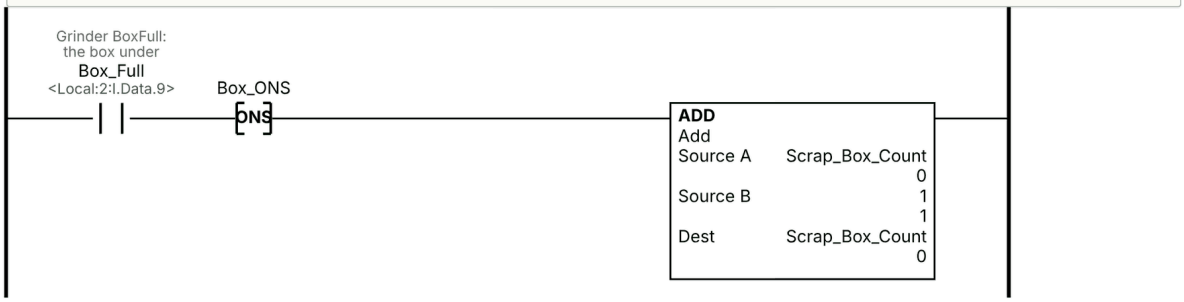
2

LARGE DISPLAY: a large, unbroken bottle gets capped (ram at the bottom, Large bit 9 / Broken bit 8 after the shift): one count



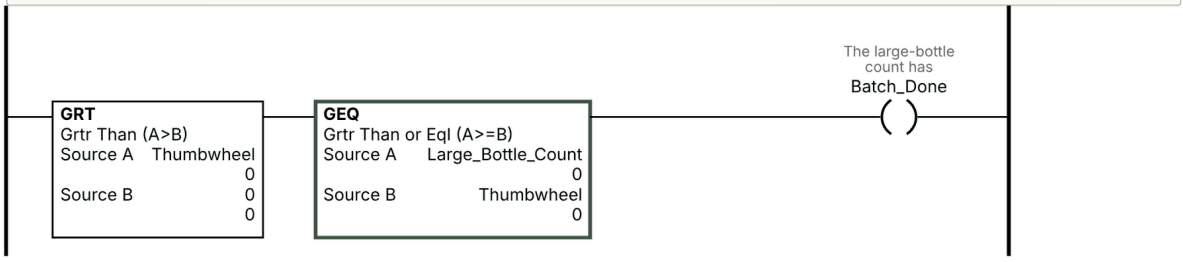
3

BOXES DISPLAY: one count each time a box fills



4

BATCH PRESET: dial a number on the thumbwheel and the line stops itself when that many large bottles have been capped (0 = no limit). Batch_Done breaks the seal-in in step 1



5

ENTER on the thumbwheel clears all four counts, which also clears Batch_Done so the line can be started again

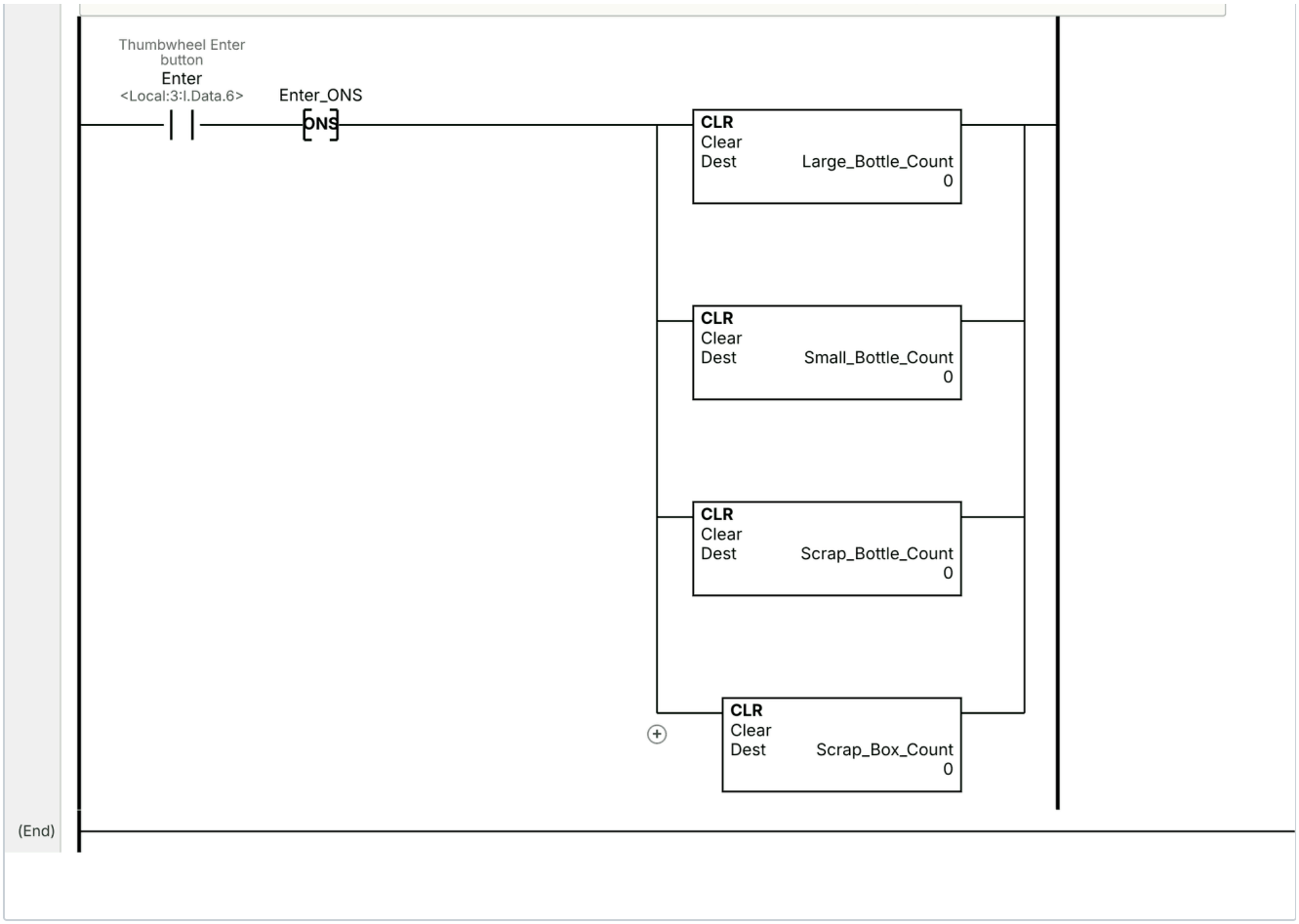


Figure 104. S8_Count online, with the finished line running: this is what your routine should look like once every step is in. Green is power flow.

TRY IT

Count large bottles at the end of the line (Main_Out's Detect) instead of at the capper, and compare the two counts over a batch.

26 Lesson 11 — Step 9: the reservoir and the stack light

The last step keeps the line fed. Add the JSR and write:

> **IN THE APP**
>
> Guided mode lesson 11 is this chapter: it checks that this lesson's tags are made, that `S9_Supply` exists, is called from MainRoutine and verifies, and then watches the line while you run it and ticks each *Run it* result below as it happens. Help → Guided Mode, or start from File → New Project… → Guided: build the bottle line.

Tags this lesson needs

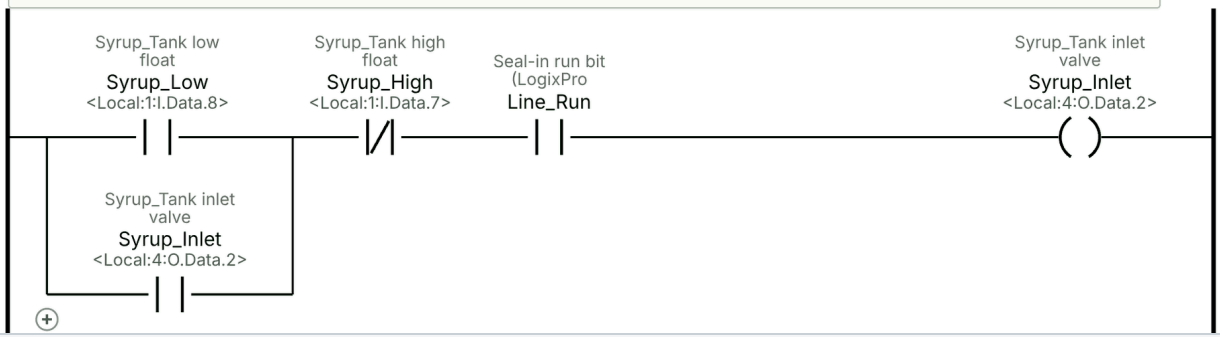
Create these before you type the rungs (Verify reports any you miss as unknown). Tags from earlier lessons are not repeated.

Tag	Type	How to make it	Description
`Syrup_Low`	BOOL	alias for `Local:1:I.Data.8` — the Properties drawer's *tag* link makes it, then rename it	Syrup_Tank low float
`Syrup_Inlet`	BOOL	alias for `Local:4:O.Data.2` — the Properties drawer's *tag* link makes it, then rename it	Syrup_Tank inlet valve
`Syrup_High`	BOOL	alias for `Local:1:I.Data.7` — the Properties drawer's *tag* link makes it, then rename it	Syrup_Tank high float
`Tank_Low`	BOOL	alias for `Local:1:I.Data.12` — the Properties drawer's *tag* link makes it, then rename it	Filler head tank below 15 %
`Tank_Red`	BOOL	alias for `Local:4:O.Data.3` — the Properties drawer's *tag* link makes it, then rename it	Tank_Light red: reservoir at the low float (flashes)
`Tank_Amber`	BOOL	alias for `Local:4:O.Data.4` — the Properties drawer's *tag* link makes it, then rename it	Tank_Light amber: inlet open, refilling
`Tank_Green`	BOOL	alias for `Local:4:O.Data.5` — the Properties drawer's *tag* link makes it, then rename it	Tank_Light green: level healthy

RUNG 0

0

RESERVOIR: the low float opens the inlet and it seals in until the high float. The filler head tank is topped up from the reservoir by itself: watch Syrup_Level fall a little with every charge (fit the reservoir's flow meter for a Syrup_Flow analog too)




```
[XIC(Syrup_Low),XIC(Syrup_Inlet)]XIO(Syrup_High)XIC(Line_Run)OTE(Syrup_Inlet);
```

RUNG 1

1

PROTECT THE FILL: if the head tank runs low (reservoir empty, inlet too slow) stop the line rather than pour short charges. Try it: set the Syrup_Tank Inlet flow to 10 and see this rung trip

```
XIC(Tank_Low)XIC(Line_Run)OTU(Line_Run);
```

RUNG 2

2

TANK STACK LIGHT, RED: the reservoir is at its low float (the light is set to flash its red segment)

```
XIC(Syrup_Low)OTE(Tank_Red);
```

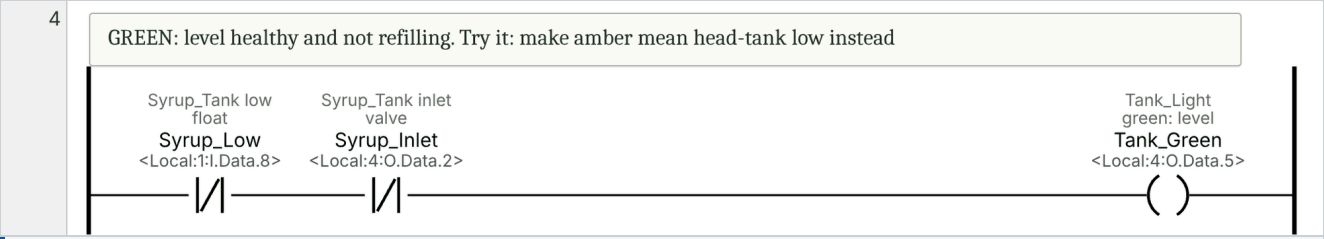
RUNG 3

3

AMBER: the inlet is open, the reservoir is refilling

```
XIC(Syrup_Inlet)OTE(Tank_Amber);
```

RUNG 4




```
XIO(Syrup_Low)XIO(Syrup_Inlet)OTE(Tank_Green);
```

Rung 0 is a seal-in on the floats: the low probe opens the inlet valve, and the valve stays open until the high probe. **Rung 1** protects the fill: if the filler's head tank runs low (the reservoir empty, the inlet too slow) the line is stopped rather than pouring short charges. **Rungs 2–4** drive the stack light on the tank: red flashing at the low float (the light is set to flash its red segment), amber while refilling, green when healthy and not refilling.

Run it

The tank starts at 35 %: watch it fall a little with every charge (Trends: pen `Syrup_Level`), reach 20 %, the red flash, the inlet valve on the tank turn green and the level climb to 90 %. Set the reservoir's *Inlet flow* to 10 and the line will eventually stop on rung 1 — the Simulator Log says why.

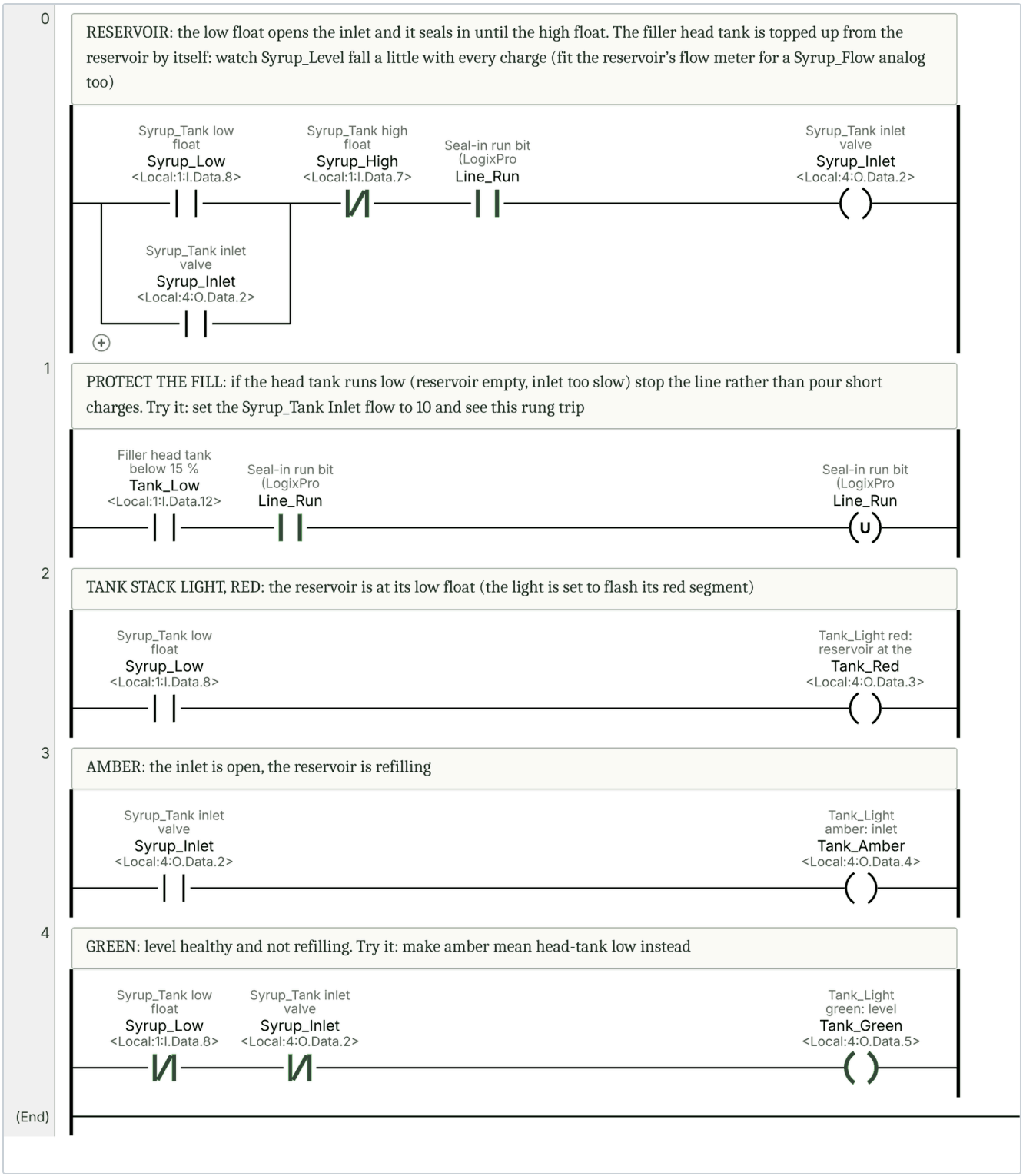


Figure 105. S9_Supply online, with the finished line running: this is what your routine should look like once every step is in. Green is power flow.

TRY IT

Make amber mean head-tank low instead of refilling. Then fit the reservoir's flow meter (a setting; it adds an analog input) and trend `Syrup_Flow` against the charges.

27 Lesson 12 — Step 10: the labeler, FLS logic

One stop past the capper stands an Inline Labeler, a pressure-sensitive dispenser in the mould of a Novexx FLS 204: a web of labels on a liner runs from the unwind reel over the dispensing edge to the rewind, and an applicator presses the peeled label onto the bottle. Its handshake is the one every dispenser has — **Start** (our `Apply`), **Ready**, **Error**, **Label low** — and the rules are the same as on a real panel: start only when Ready, treat Error as a fault that needs a person, never step the belt mid-cycle. Add the JSR and write:

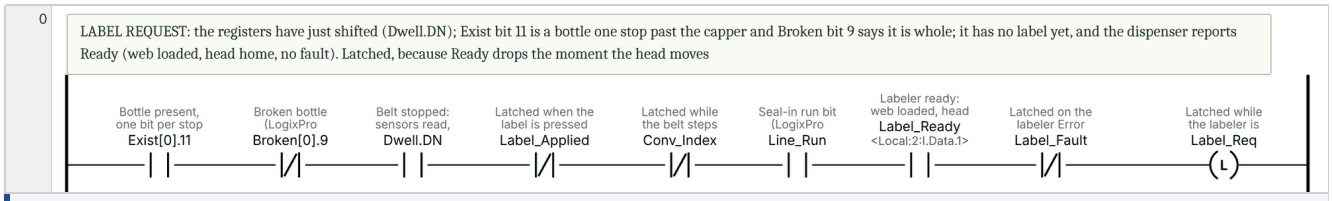
> **IN THE APP**
>
> Guided mode lesson 12 is this chapter: it checks that this lesson's tags are made, that `S10_Label` exists, is called from MainRoutine and verifies, and then watches the line while you run it and ticks each *Run it* result below as it happens. Help → Guided Mode, or start from File → New Project… → Guided: build the bottle line.

Tags this lesson needs

Create these before you type the rungs (Verify reports any you miss as unknown). Tags from earlier lessons are not repeated.

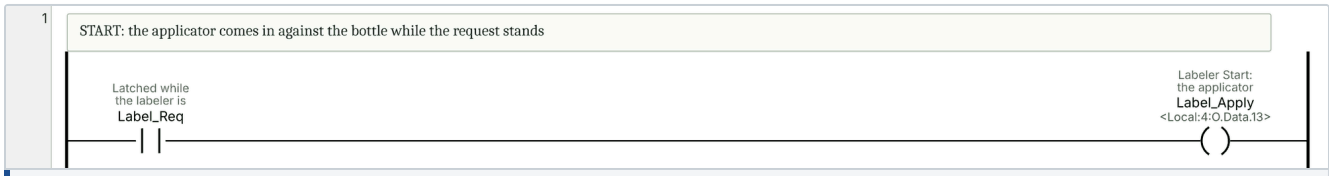
Tag	Type	How to make it	Description
`Label_Applied`	BOOL	Edit Tags: type the name, pick the type	Latched when the label is pressed on; cleared when the belt steps
`Label_Ready`	BOOL	alias for `Local:2:I.Data.1` — the Properties drawer's *tag* link makes it, then rename it	Labeler ready: web loaded, head home, no fault
`Label_Fault`	BOOL	Edit Tags: type the name, pick the type	Latched on the labeler Error input (end of roll or a missed label); Enter clears it
`Label_Apply`	BOOL	alias for `Local:4:O.Data.13` — the Properties drawer's *tag* link makes it, then rename it	Labeler Start: the applicator comes in and presses a label on
`Label_PadIn`	BOOL	alias for `Local:2:I.Data.0` — the Properties drawer's *tag* link makes it, then rename it	Labeler applicator at the end of its stroke (label pressed on)
`Label_Error`	BOOL	alias for `Local:2:I.Data.2` — the Properties drawer's *tag* link makes it, then rename it	Labeler fault: end of roll, or the last cycle found no label
`Label_Low`	BOOL	alias for `Local:2:I.Data.3` — the Properties drawer's *tag* link makes it, then rename it	Labeler roll low
`Label_Green`	BOOL	alias for `Local:4:O.Data.14` — the Properties drawer's *tag* link makes it, then rename it	Labeler green indicator
`Label_Amber`	BOOL	alias for `Local:4:O.Data.15` — the Properties drawer's *tag* link makes it, then rename it	Labeler amber indicator

RUNG 0



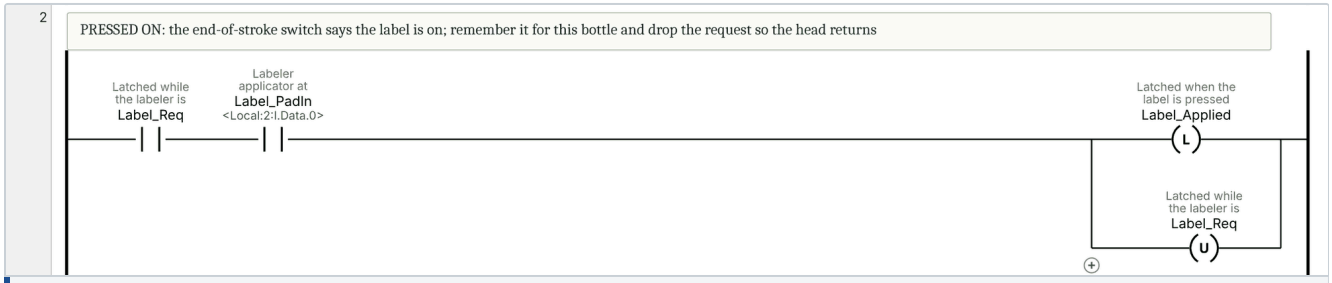

```
XIC(Exist[0].11)XIO(Broken[0].9)XIC(Dwell.DN)XIO(Label_Applied)XIO(Conv_Index)X
IC(Line_Run)XIC(Label_Ready)XIO(Label_Fault)OTL(Label_Req);
```

RUNG 1



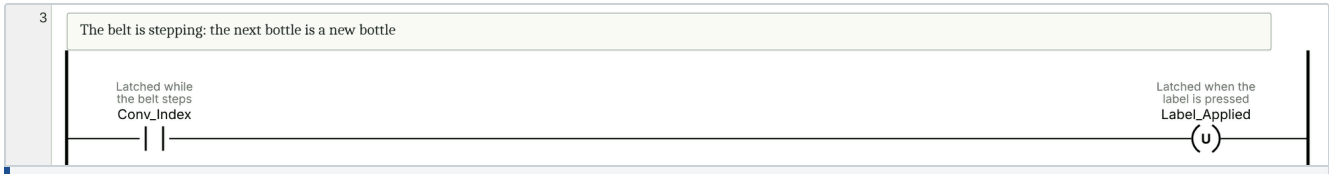

```
XIC(Label_Req)OTE(Label_Apply);
```

RUNG 2



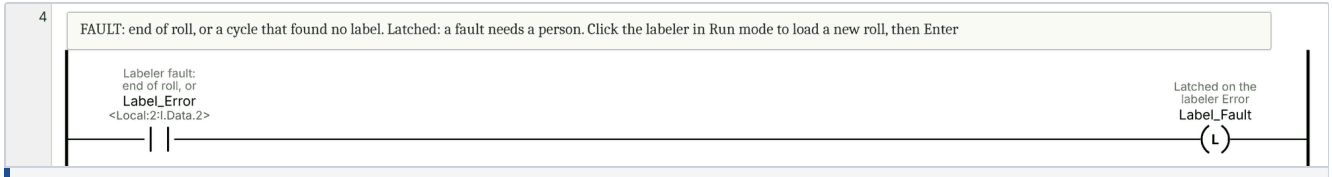

```
XIC(Label_Req)XIC(Label_PadIn)[OTL(Label_Applied),OTU(Label_Req)];
```

RUNG 3



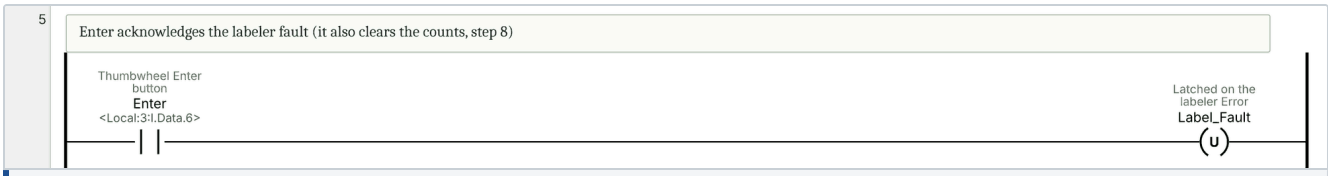

```
XIC(Conv_Index)OTU(Label_Applied);
```

RUNG 4



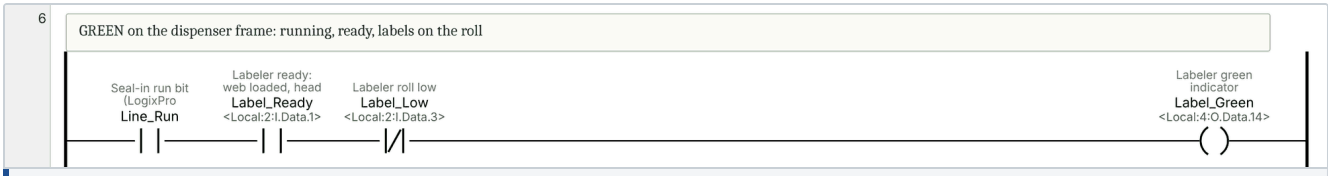

```
XIC(Label_Error)OTL(Label_Fault);
```

RUNG 5




```
XIC(Enter)OTU(Label_Fault);
```

RUNG 6




```
XIC(Line_Run)XIC(Label_Ready)XIO(Label_Low)OTE(Label_Green);
```

RUNG 7

```
[XIC(Label_Low),XIC(Label_Fault),XIC(Label_Req)]OTE(Label_Amber);
```

Rung 0 asks for a label in the *after the shift* frame, one bit past the capper (Exist 11, Broken 9), only when the dispenser says Ready — and it is a **latch**, because Ready drops the moment the head leaves home; an OTE here would start the cycle and cancel it in the next scan. **Rung 1** drives Start while the request stands; **rung 2** takes the end-of-stroke switch as "pressed on", remembers it for this bottle and drops the request so the head returns; **rung 3** forgets it when the belt steps. **Rung 4** latches a fault on the dispenser's Error — end of roll, or a cycle that found no label; **rung 5** lets Enter acknowledge it. **Rungs 6–7** are the frame lights. Step 6 gained `XIO(Label_Req)` so the belt never steps with the applicator in, and step 1's housekeeping rung drops the request when the line stops (the head goes home; the bottle is labeled after Start). `Label_Applied` is kept: that bottle has its label.

Run it

Every capped bottle leaves with a label (Side or 3D view: the band on the body; the unwind reel shrinks as the roll goes down). Set the Labeler's *Labels on the roll at start* to 3 and run: three labels, then the Error input, the amber lamp, and `Label_Fault` latched — and the line keeps running, bottles leaving bare,

because a labeler fault is not a reason to stop a filler. Click the labeler in Run mode to load a new roll (Ready comes back), press Enter, and labeling resumes.

0

LABEL REQUEST: the registers have just shifted (Dwell.DN); Exist bit 11 is a bottle one stop past the capper and Broken bit 9 says it is whole; it has no label yet, and the dispenser reports Ready (web loaded, head home, no fault). Latched, because Ready drops the moment the head moves

1

START: the applicator comes in against the bottle while the request stands

2

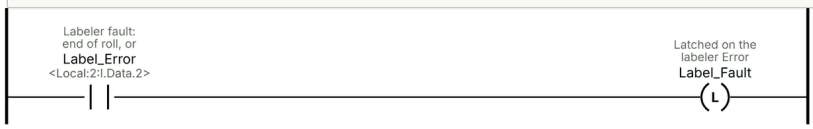
PRESSED ON: the end-of-stroke switch says the label is on; remember it for this bottle and drop the request so the head returns

3

The belt is stepping: the next bottle is a new bottle

4

FAULT: end of roll, or a cycle that found no label. Latched: a fault needs a person. Click the labeler in Run mode to load a new roll, then Enter

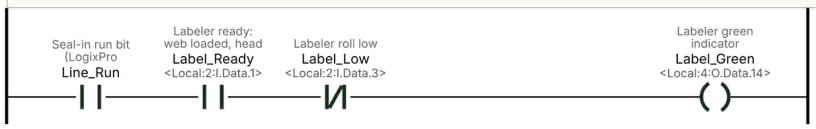


5

Enter acknowledges the labeler fault (it also clears the counts, step 8)

6

GREEN on the dispenser frame: running, ready, labels on the roll



7

AMBER: roll low, faulted, or a cycle in progress

(End)

Figure 106. S10_Label online, with the finished line running: this is what your routine should look like once every step is in. Green is power flow.

TRY IT

Make the fault stop the line instead (add `XIO(Label_Fault)` to step 1's seal-in) and decide which you would want on a real line. Then switch the Labeler's *Hand* to LH and *Applicator* to Tamp pad in Properties and watch the machine rebuild on the other side of the lane — the program does not change.

28 Lesson 13 — Step 11: alarms and machine faults

Everything so far assumes the machine does what it is told. Real machines do not. A bottle jams; a lens gets dirty and the eye stays dark, or stays made; a motor overload trips and the conveyor stops although the output is still on; a label web breaks; a valve sticks shut. The controller is fine through all of this — the scan goes on, no instruction faults — so nothing tells you unless your program checks. This step is the checks.

The template has three parts for it (chapter 15): `LS11`, an eye one stop before the end of the line, a red `Fault_Lamp` and a blue `Reset_PB` on the console. Each belt also reports **Running** — the auxiliary contact on its motor contactor, made when the motor really turns — which the template aliases as `Main_Conv_Running` and `Divert_Conv_Running`. The alarms are the bits of one DINT, `Alarm_Word`, shown in Binary:

Bit	Alarm	Raised when	Stops the line
0	Main conveyor overload	Main_Conveyor commanded, Main_Conv_Running missing for 0.5 s	yes
1	Divert conveyor overload	Divert_Conveyor commanded, Divert_Conv_Running missing for 0.5 s	yes
2	End-of-line check	once per stop, LS11 disagrees with the tracking for that stop: a bottle missing or one extra	yes
3	LS1 dark too long	the line running and LS1 dark for 8 s	yes
4	Labeler fault	Label_Fault (step 10) latched	no
5	Inlet not filling	Syrup_Inlet open with Syrup_Low still made for 10 s	yes
6	Case lane drive tripped	Case_VFD_Fault (the drive of step 12)	no
7	Packer infeed full	Infeed_Full: eight bottles waiting to be packed	yes

Bits 6 and 7 watch the case packer and its drive, which step 12 programs (chapter 28); their alias tags are already in your list from chapter 16, so type the rungs now and they will make sense two chapters on.

Add the JSR under step 10 and write:

> **IN THE APP**
>
> Guided mode lesson 13 is this chapter: it checks that this lesson's tags are made, that `S11_Alarms` exists, is called from MainRoutine and verifies, and then watches the line while you run it and ticks each *Run it* result below as it happens. Help → Guided Mode, or start from File → New Project… → Guided: build the bottle line.

Tags this lesson needs

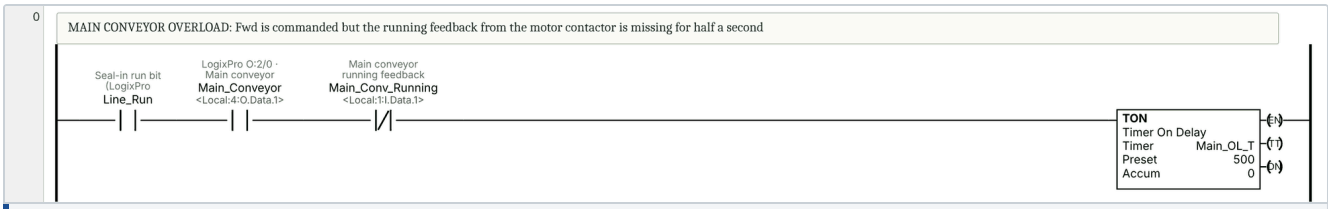
Create these before you type the rungs (Verify reports any you miss as unknown). Tags from earlier lessons are not repeated.

Tag	Type	How to make it	Description
`Main_Conv_Running`	BOOL	alias for `Local:1:I.Data.1` — the Properties drawer's *tag* link makes it, then rename it	Main conveyor running feedback (motor contactor)
`Main_OL_T`	TIMER	Edit Tags: type the name, pick the type	Main conveyor overload timer (no running feedback)
`OL_ONS`	BOOL	Edit Tags: type the name, pick the type	
`Alarm_Word`	DINT	Edit Tags: type the name, pick the type, Style Binary	Latched alarms, one bit each: 0 main conveyor overload, 1 divert conveyor overload, 2 end-of-line check failed (a jam, a lost bottle, tracking out of step), 3 LS1 dark too long (dirty lens or feeder empty), 4 labeler fault, 5 reservoir inlet open but level not rising, 6 case lane drive tripped, 7 packer infeed full
`Alarm_Unack`	BOOL	Edit Tags: type the name, pick the type	New alarm, not yet acknowledged (Reset)
`Divert_Conv_Running`	BOOL	alias for `Local:2:I.Data.12` — the Properties drawer's *tag* link makes it, then rename it	Divert conveyor running feedback
`Div_OL_T`	TIMER	Edit Tags: type the name, pick the type	Divert conveyor overload timer (no running feedback)
`DOL_ONS`	BOOL	Edit Tags: type the name, pick the type	
`EOL_Inhibit`	BOOL	Edit Tags: type the name, pick the type	Tracking cleared by Reset: the end-of-line check waits until the first bottle tracked since then reaches LS11
`EOL_ONS`	BOOL	Edit Tags: type the name, pick the type	

Tag	Type	How to make it	Description
`LS11`	BOOL	alias for `Local:3:I.Data.4` — the Properties drawer's *tag* link makes it, then rename it	End-of-line eye at 1240: verifies the tracking (only large, whole bottles reach it)
`EOL_Missing`	BOOL	Edit Tags: type the name, pick the type	LS11 dark when the registers said a large, whole bottle was there
`EOL_Extra`	BOOL	Edit Tags: type the name, pick the type	LS11 saw a bottle the registers did not expect
`BLK_ONS`	BOOL	Edit Tags: type the name, pick the type	
`LS1_Dark_T`	TIMER	Edit Tags: type the name, pick the type	LS1 dark too long timer
`DRK_ONS`	BOOL	Edit Tags: type the name, pick the type	
`LBL_ONS`	BOOL	Edit Tags: type the name, pick the type	
`Inlet_T`	TIMER	Edit Tags: type the name, pick the type	Inlet not filling timer
`INL_ONS`	BOOL	Edit Tags: type the name, pick the type	
`Case_VFD_Fault`	BOOL	alias for `Local:10:I.Data.3` — the Properties drawer's *tag* link makes it, then rename it	Case lane drive tripped
`VFD_ONS`	BOOL	Edit Tags: type the name, pick the type	
`Infeed_Full`	BOOL	alias for `Local:3:I.Data.9` — the Properties drawer's *tag* link makes it, then rename it	Packer infeed buffer full: bottles arriving now wait on the belt
`INF_ONS`	BOOL	Edit Tags: type the name, pick the type	
`Reset_PB`	BOOL	alias for `Local:3:I.Data.5` — the Properties drawer's *tag* link makes it, then rename it	Blue Reset button: acknowledge alarms and clear the ones whose cause is gone
`RST_ONS`	BOOL	Edit Tags: type the name, pick the type	
`Flash_T`	TIMER	Edit Tags: type the name, pick the type	Fault lamp flasher

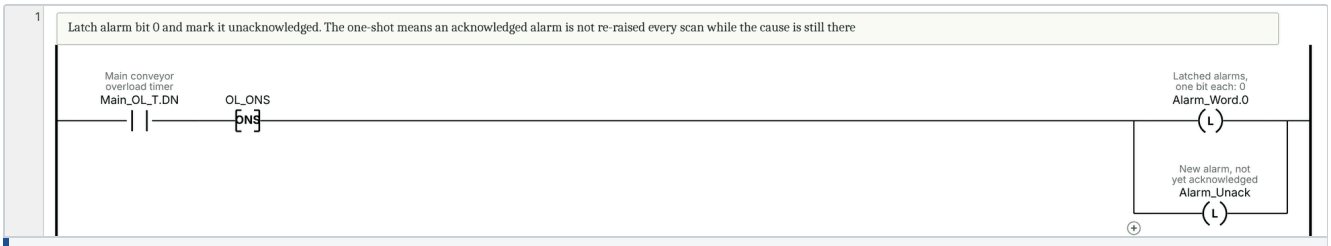
Tag	Type	How to make it	Description
`Fault_Lamp`	BOOL	alias for `Local:5:O.Data.6` — the Properties drawer's *tag* link makes it, then rename it	Red fault lamp on the console: flashing = unacknowledged alarm, steady = alarm still active

RUNG 0



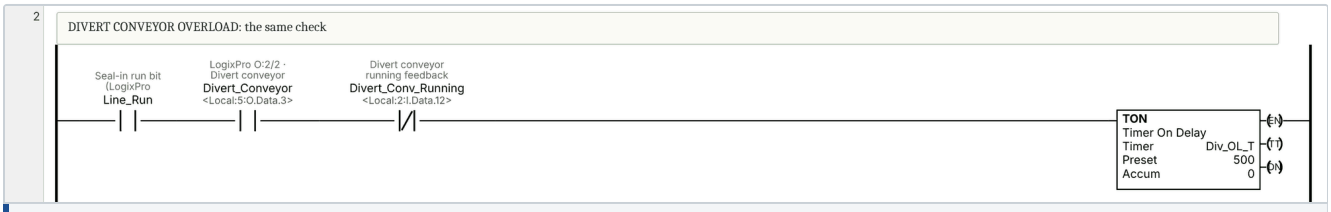

```
XIC(Line_Run)XIC(Main_Conveyor)XIO(Main_Conv_Running)TON(Main_OL_T,500,0);
```

RUNG 1




```
XIC(Main_OL_T.DN)ONS(OL_ONS)[OTL(Alarm_Word.0),OTL(Alarm_Unack)];
```

RUNG 2

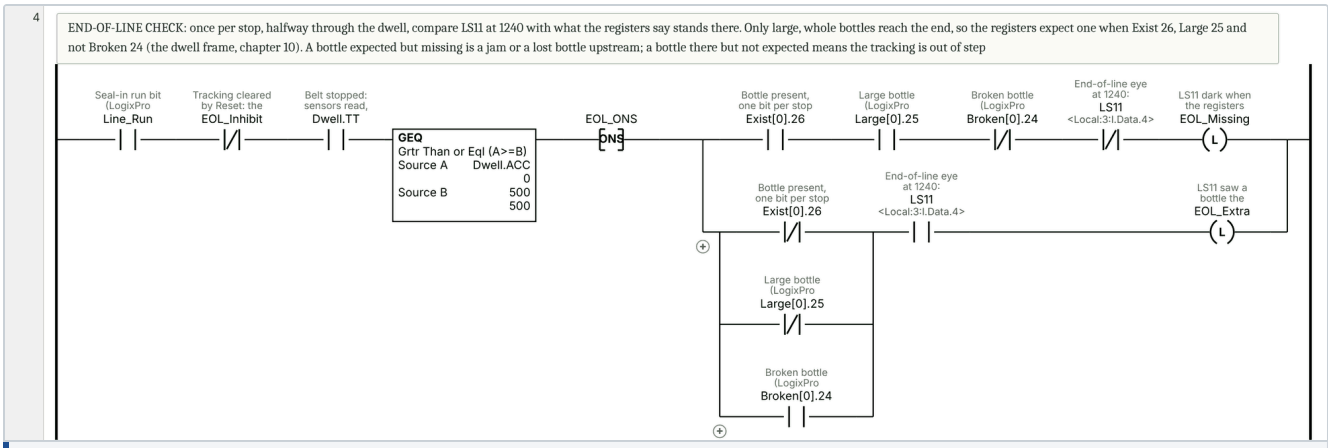



```
XIC(Line_Run)XIC(Divert_Conveyor)XIO(Divert_Conv_Running)TON(Div_OL_T,500,0);
```

RUNG 3

```
XIC(Div_OL_T.DN)ONS(DOL_ONS)[OTL(Alarm_Word.1),OTL(Alarm_Unack)];
```

RUNG 4

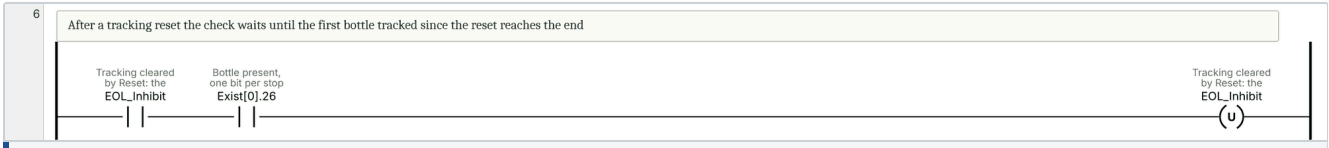



```
XIC(Line_Run)XIO(EOL_Inhibit)XIC(Dwell.TT)GEQ(Dwell.ACC,500)ONS(EOL_ONS)
[XIC(Exist[0].26)XIC(Large[0].25)XIO(Broken[0].24)XIO(LS11)OTL(EOL_Missing),
[XIO(Exist[0].26),XIO(Large[0].25),XIC(Broken[0].24)]XIC(LS11)OTL(EOL_Extra)];
```

RUNG 5

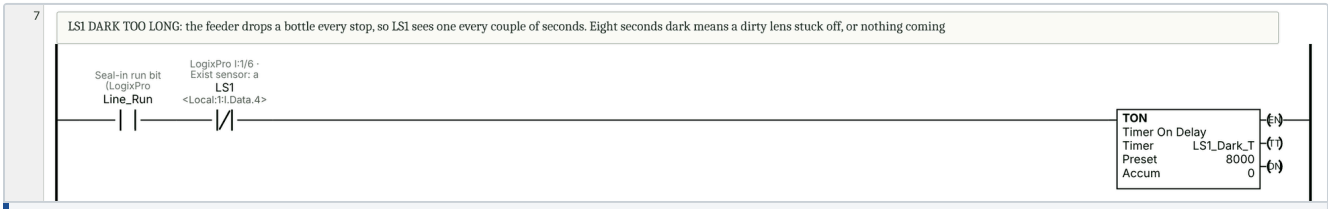
```
[XIC(EOL_Missing),XIC(EOL_Extra)]ONS(BLK_ONS)
[OTL(Alarm_Word.2),OTL(Alarm_Unack)];
```

RUNG 6



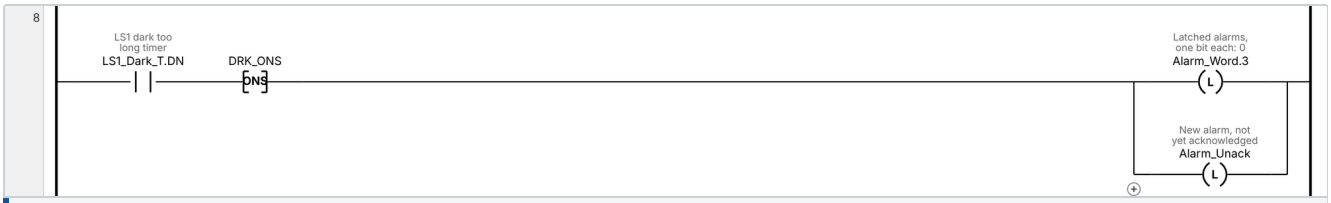

```
XIC(EOL_Inhibit)XIC(Exist[0].26)OTU(EOL_Inhibit);
```

RUNG 7



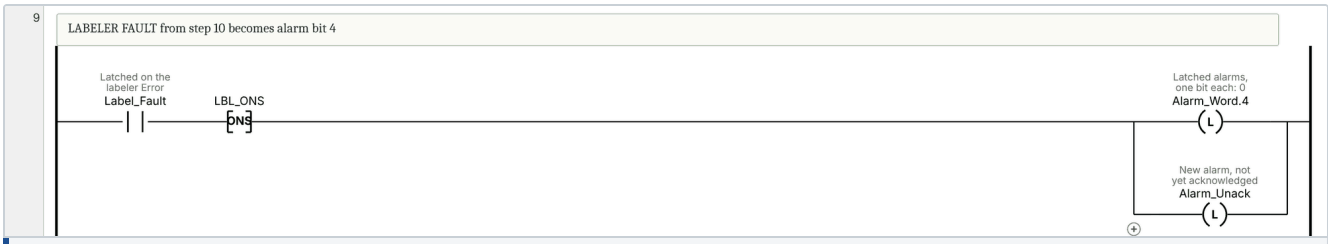

```
XIC(Line_Run)XIO(LS1)TON(LS1_Dark_T,8000,0);
```

RUNG 8



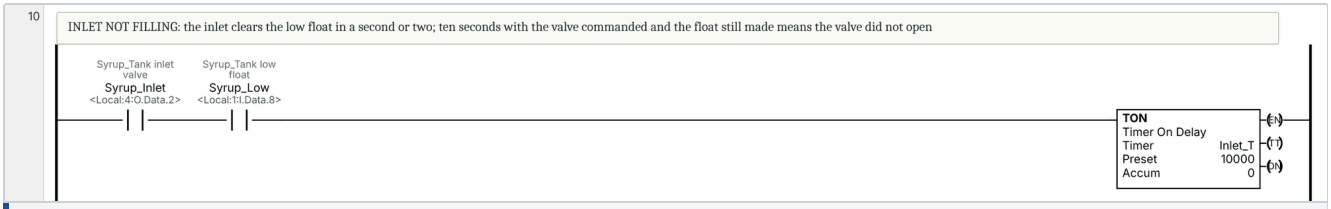

```
XIC(LS1_Dark_T.DN)ONS(DRK_ONS)[OTL(Alarm_Word.3),OTL(Alarm_Unack)];
```

RUNG 9



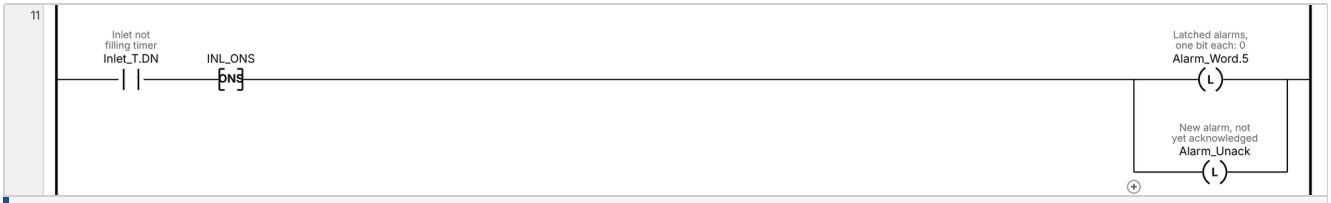

```
XIC(Label_Fault)ONS(LBL_ONS)[OTL(Alarm_Word.4),OTL(Alarm_Unack)];
```

RUNG 10



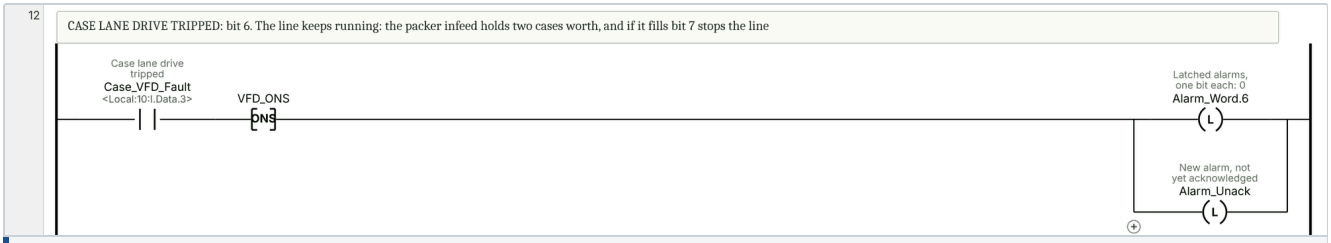

```
XIC(Syrup_Inlet)XIC(Syrup_Low)TON(Inlet_T,10000,0);
```

RUNG 11



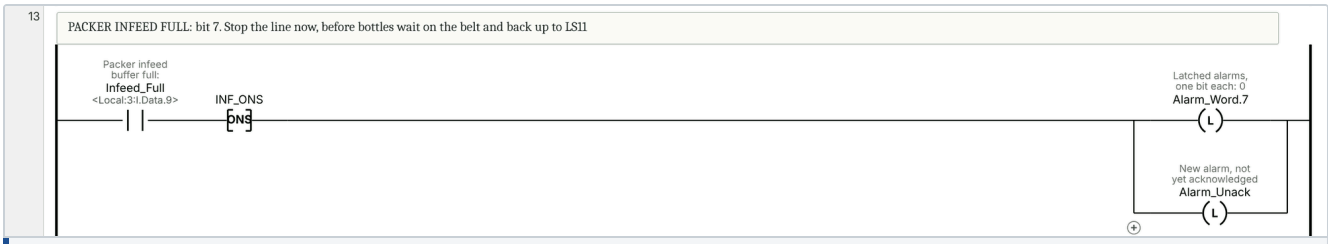

```
XIC(Inlet_T.DN)ONS(INL_ONS)[OTL(Alarm_Word.5),OTL(Alarm_Unack)];
```

RUNG 12




```
XIC(Case_VFD_Fault)ONS(VFD_ONS)[OTL(Alarm_Word.6),OTL(Alarm_Unack)];
```

RUNG 13

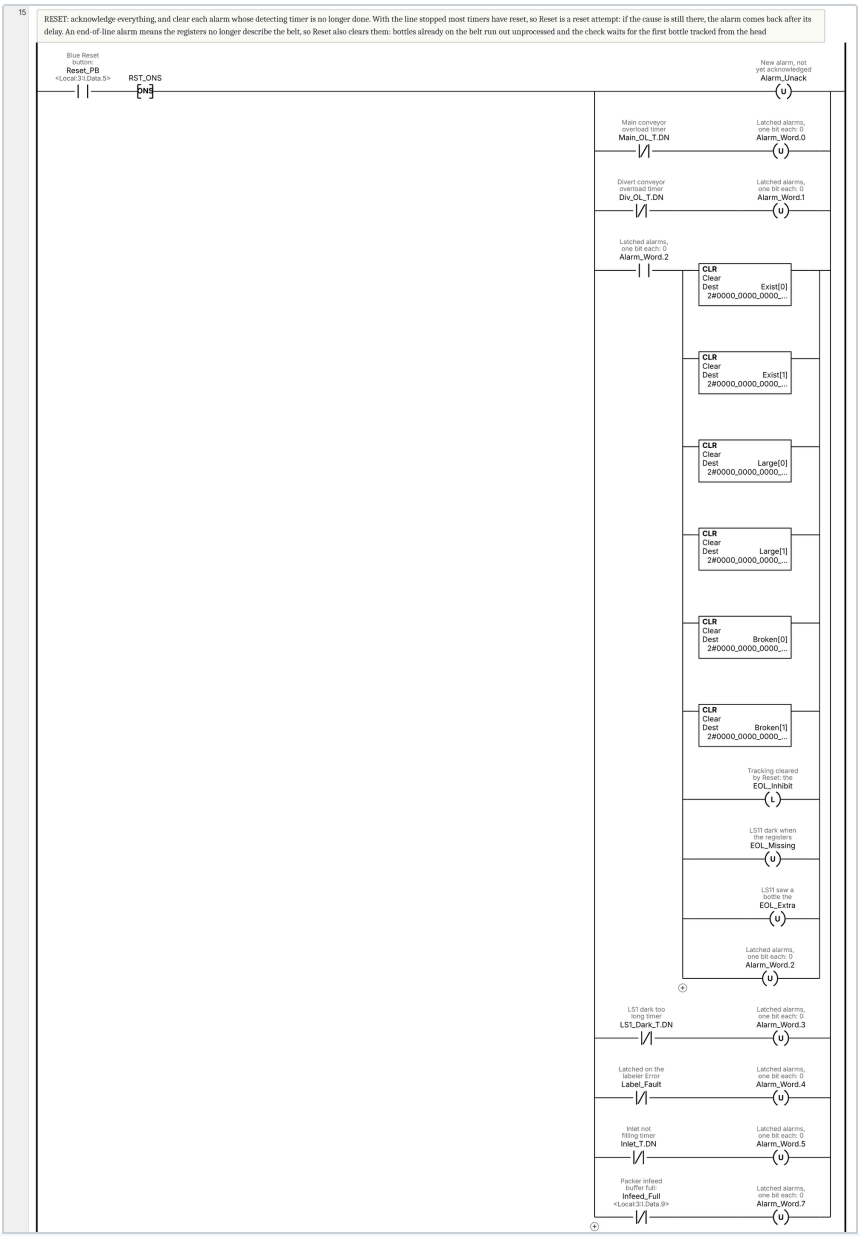



```
XIC(Infeed_Full)ONS(INF_ONS)[OTL(Alarm_Word.7),OTL(Alarm_Unack)];
```

RUNG 14

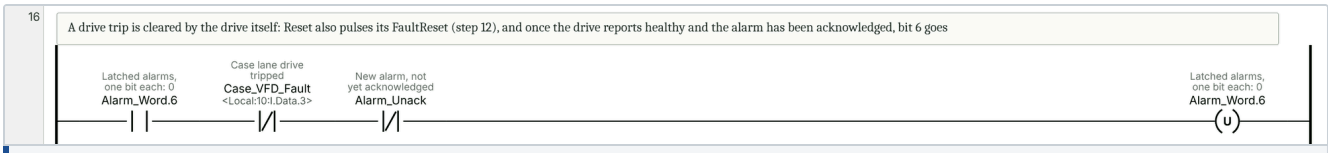
```
[XIC(Alarm_Word.0),XIC(Alarm_Word.1),XIC(Alarm_Word.2),XIC(Alarm_Word.3),XIC(Alarm_Word.5),XIC(Alarm_Word.7)]OTU(Line_Run);
```

RUNG 15



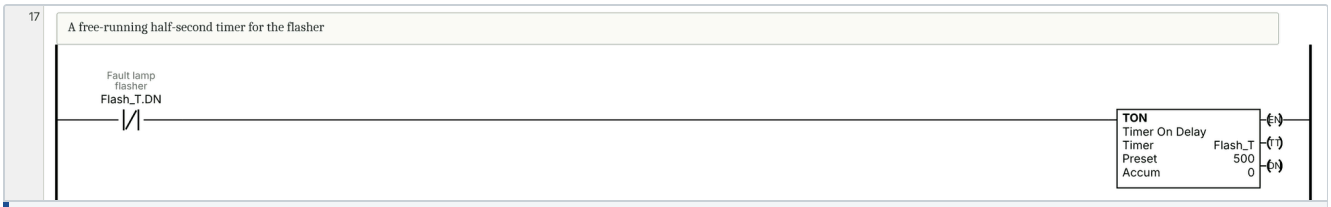

```
XIC(Reset_PB)ONS(RST_ONS)
[OTU(Alarm_Unack),XIO(Main_OL_T.DN)OTU(Alarm_Word.0),XIO(Div_OL_T.DN)OTU(Alarm_
Word.1),XIC(Alarm_Word.2)
[CLR(Exist[0]),CLR(Exist[1]),CLR(Large[0]),CLR(Large[1]),CLR(Broken[0]),CLR(Bro
ken[1]),OTL(EOL_Inhibit),OTU(EOL_Missing),OTU(EOL_Extra),OTU(Alarm_Word.2)],XIO
(LS1_Dark_T.DN)OTU(Alarm_Word.3),XIO(Label_Fault)OTU(Alarm_Word.4),XIO(Inlet_T.
DN)OTU(Alarm_Word.5),XIO(Infeed_Full)OTU(Alarm_Word.7)];
```

RUNG 16



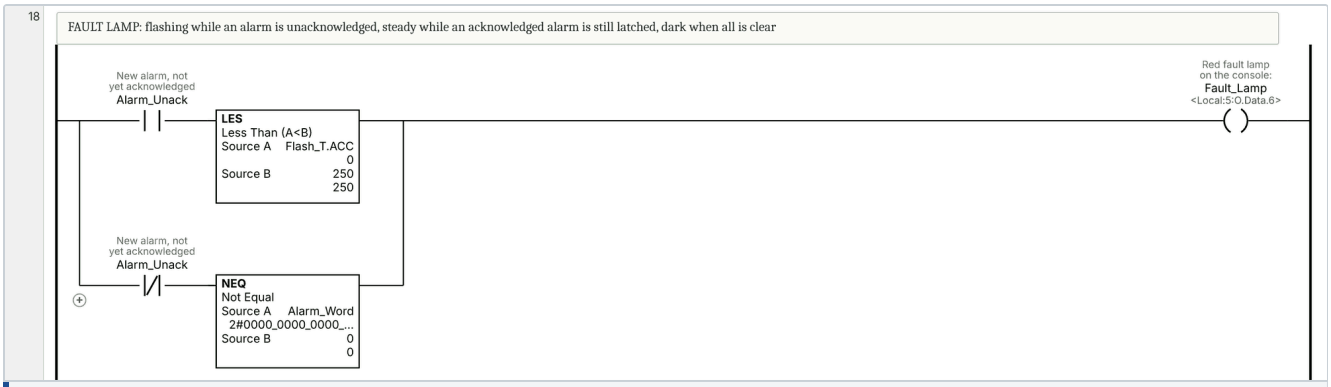

```
XIC(Alarm_Word.6)XIO(Case_VFD_Fault)XIO(Alarm_Unack)OTU(Alarm_Word.6);
```

RUNG 17




```
XIO(Flash_T.DN)TON(Flash_T,500,0);
```

RUNG 18




```
[XIC(Alarm_Unack)LES(Flash_T.ACC,250),XIO(Alarm_Unack)NEQ(Alarm_Word,0)]OTE(Fault_Lamp);
```

Rungs 0–3, overloads. The output says run; the feedback says not running. Half a second allows for the contactor to pull in. When the timer is done, latch the bit and `Alarm_Unack`. The one-shot matters: without it an alarm the operator has acknowledged would be re-raised every scan while the cause is still there.

Rungs 4–6, the end-of-line check. This is the shift register's own proof. Once per stop — halfway through the dwell (`Dwell.TT` and `ACC ≥ 500`, so the belt is standing still and LS11 has settled), once only (`ONS`) — compare LS11 with what the registers say stands in front of it. LS11 at x 1240 is 27 stops after LS1, so in the dwell frame (chapter 10) that is `Exist[0].26`, `Large[0].25`, `Broken[0].24`, and a bottle should be there only if it exists, is large (small ones were diverted at stop 20) and is whole (broken ones were scrapped). Expected and dark is `EOL_Missing`; not expected and made is `EOL_Extra`. Either is bit 2. A jammed bottle shows up here a few stops after it jams: the bottles behind it pile up, the stops they should be at stay empty, the registers walk on regardless. So does a bottle that slipped past LS1 untracked. Rung 6 is for after a reset (below).

Rungs 7–8, LS1 dark too long. The feeder drops a bottle every stop, so LS1 sees one every couple of seconds. Eight seconds dark with the line running means the eye is dirty and stuck off, or nothing is coming.

Rung 9 turns the labeler's fault into bit 4, and **rungs 10–11** watch the reservoir: the inlet clears the low float in a second or two, so ten seconds commanded open with the float still made means the valve did not open.

Rungs 12–13 are the packer: a drive trip is bit 6, a full infeed bit 7. **Rung 14** stops the line on bits 0, 1, 2, 3, 5 and 7 — an `OTU(Line_Run)` after step 1's seal-in, so Start cannot hold while one of them is set. Bits 4 and 6 do not: bottles can leave unlabeled while someone loads a roll (step 10's Try it asks you to decide that for yourself), and the packer's infeed holds two cases worth while the drive is reset — if it fills, bit 7 stops the line before bottles back up to LS11.

Rung 15, Reset. One press, one-shot, does two things. It acknowledges — `Alarm_Unack` off — and it clears every alarm whose cause has gone: an overload bit only once its timer is no longer done, LS1 dark only once LS1 has seen something, the labeler bit only once `Label_Fault` is off. An end-of-line alarm is different: its cause is that the registers no longer describe the belt, so Reset clears all six register words and sets `EOL_Inhibit`, and the check stays off until the first bottle tracked since the reset reaches LS11 (rung 6). That is how a real line recovers after a jam: clear the jam, reset the tracking, purge the line.

Rung 16 lets a drive trip clear itself from the alarm word: Reset also pulses the drive's FaultReset (step 12), and once the drive reports healthy and the alarm has been acknowledged, bit 6 goes. **Rungs 17–18, the lamp.** A free-running half-second timer; the lamp flashes on the first half while an alarm is unacknowledged, and is steady while an acknowledged alarm is still set. Off means healthy.

> **A MACHINE FAULT IS NOT A CONTROLLER FAULT**
>
> Chapter 11's *major fault* is the controller stopping itself: an instruction it cannot execute — an array index out of range, an integer divide by zero. The scan stops, Controller Properties says why, and Communications → Clear Faults clears it. A *machine* fault is everything in this chapter: the controller is healthy, the scan runs, and the only way anyone finds out is a rung like these. The first kind is the PLC protecting itself; the second is your program protecting the machine — and the product, and the people near it.

Inject a fault

The simulator can break the machine on purpose. **Tools → Inject Machine Fault…** lists, for the parts on your floor: a bottle **jammed** on a conveyor (it stops where it is; the bottles behind it pile up), a sensor with a **dirty lens** stuck on or stuck off, a conveyor **overload** (the motor stops and Running drops), a labeler **web break** (Ready drops, Error comes on), a reservoir **inlet stuck shut**. Each is logged in the Simulator Log and

drawn with a red dashed outline. **Tools → Clear Machine Faults** or Reset Sim takes them away; click a jammed bottle in Run mode to free it. The *Drill* box injects a random one every few minutes of run time, which is how you practise finding a fault you did not choose.

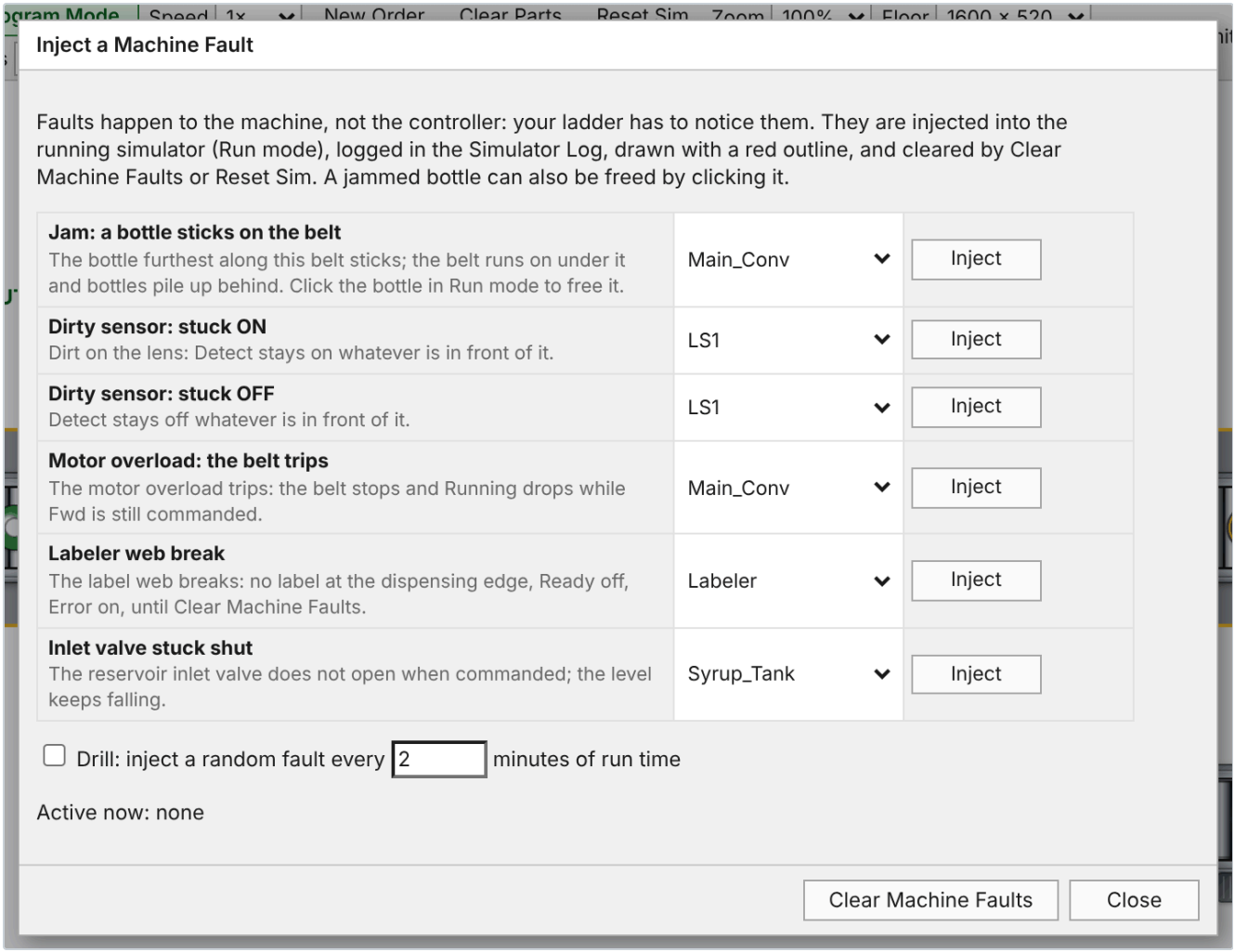


Figure 92. Tools → Inject Machine Fault: one row per kind of fault, the parts on this floor it can be injected into, and the drill.

Run it

Start the line and leave it three minutes: `Alarm_Word` stays 0 and the lamp stays dark. Press Stop and Start at any moment you like, mid-step, mid-dwell, mid-fill — still 0. (Before this step was written the stop rung in step 1 cancelled the step in progress and the dwell started again from zero; the end-of-line check caught the drift on the first try. That is why steps 1, 2 and 6 read the way they do.)

Now break things, one at a time, Clear Machine Faults, Reset and Start between each:

- **Overload on Main_Conv.** At the next step the motor does not turn: bit 0 in under four seconds (the conveyor only runs during a step), the line stops, the lamp flashes. Press Reset without clearing the

fault: the lamp goes out (the overload timer is no longer done, because nothing is commanded) — but Start trips it again at the next step. Clear, Reset, Start: the line runs.

- **Jam on Main_Conv.** The furthest bottle along stops. A few stops later LS11 disagrees with the registers: bit 2, the line stops. Free the bottle (click it), Reset — the registers clear and the check waits — Start, and a minute later the check is armed again with no alarm.
- **Dirty LS1, stuck off.** Eight seconds later, bit 3. Clear, Reset, Start — and about a minute later bit 2: the bottles that passed LS1 while it was blind were never entered in the registers, and they arrive at LS11 as extras. Reset once more.
- **Labeler web break.** Bit 4 at once, the line keeps running, bottles leave bare. Reset acknowledges: the lamp goes steady. Clear the fault (the web is rethreaded), press Enter, Reset: the lamp goes out.
- **Inlet stuck shut.** Nothing happens until the reservoir falls to the low float; ten seconds after that, bit 5.

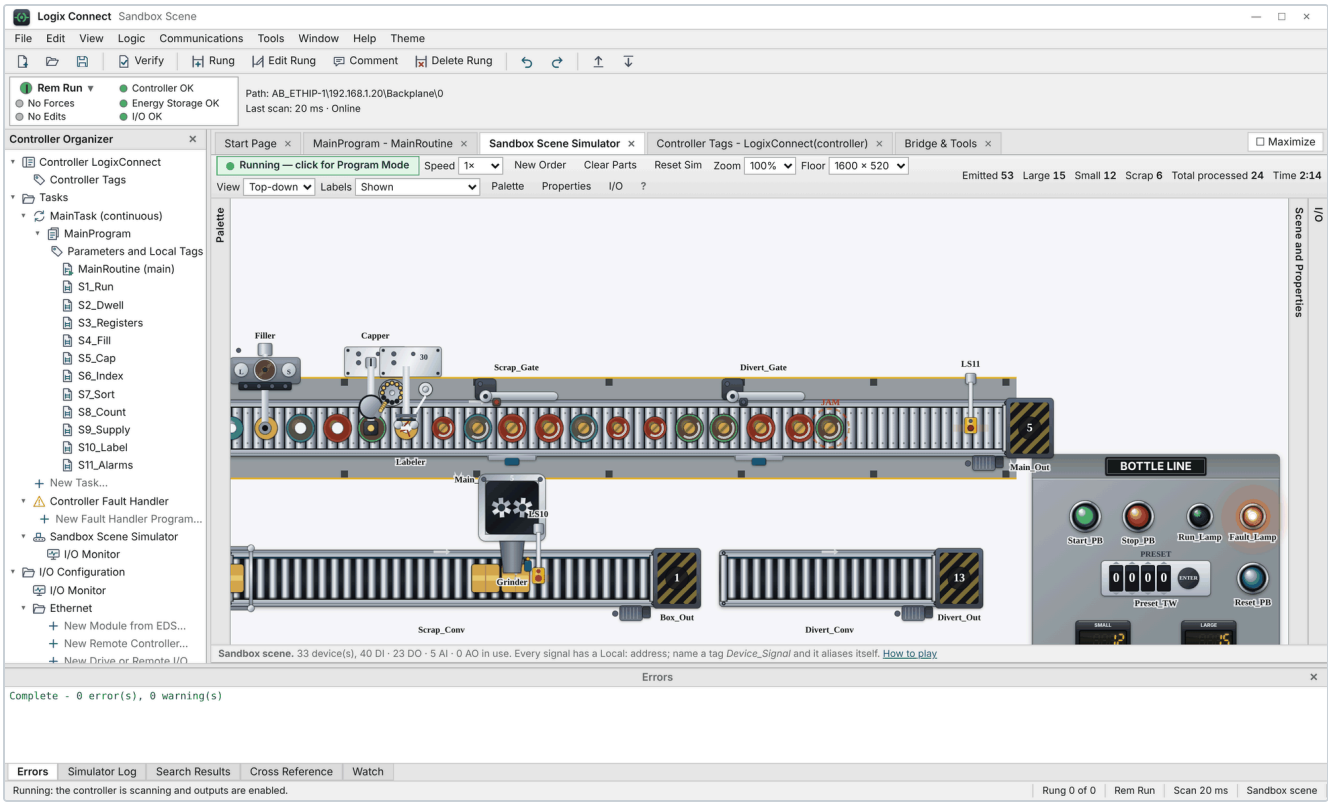
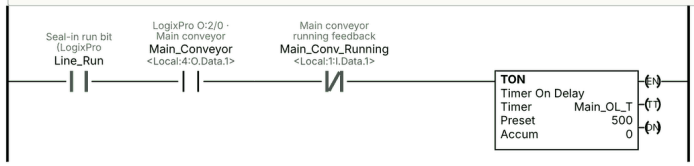


Figure 93. A jam caught. The bottle with the red dashed outline is stuck; the bottles behind it have piled up, so the stops the registers say are full stand empty when they reach LS11. Bit 2 latched, the line stopped, Fault_Lamp lit on the console. In this run it took 13 s from the jam to the alarm.

0

MAIN CONVEYOR OVERLOAD: Fwd is commanded but the running feedback from the motor contactor is missing for half a second

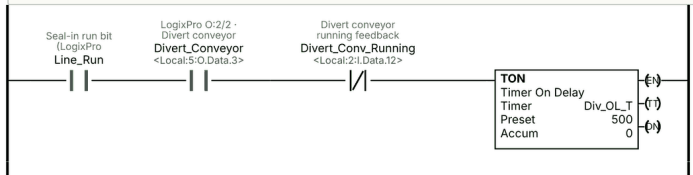


1

Latch alarm bit 0 and mark it unacknowledged. The one-shot means an acknowledged alarm is not re-raised every scan while the cause is still there

2

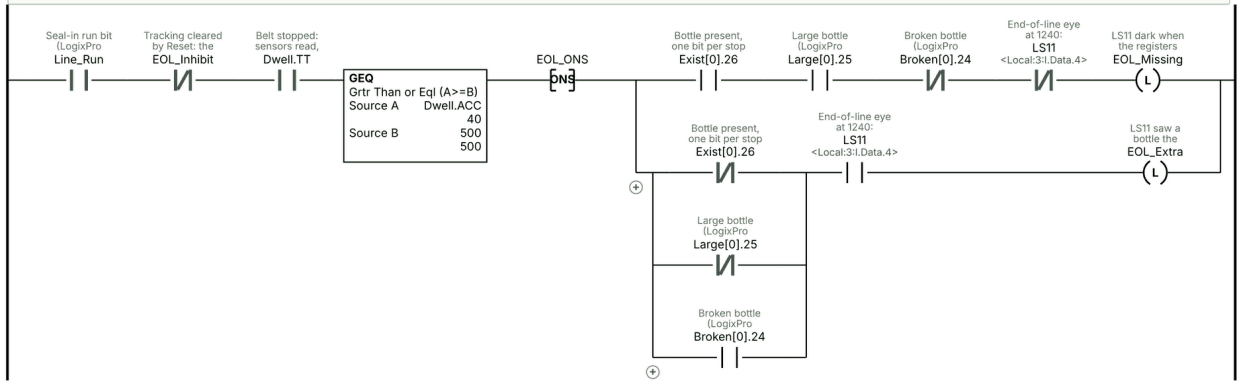
DIVERT CONVEYOR OVERLOAD: the same check



3

4

END-OF-LINE CHECK: once per stop, halfway through the dwell, compare LS11 at 1240 with what the registers say stands there. Only large, whole bottles reach the end, so the registers expect one when Exist 26, Large 25 and not Broken 24 (the dwell frame, chapter 10). A bottle expected but missing is a jam or a lost bottle upstream; a bottle there but not expected means the tracking is out of step



5

Either one is alarm bit 2

6

After a tracking reset the check waits until the first bottle tracked since the reset reaches the end

7

LS1 DARK TOO LONG: the feeder drops a bottle every stop, so LS1 sees one every couple of seconds. Eight seconds dark means a dirty lens stuck off, or nothing coming

9 LABELER FAULT from step 10 becomes alarm bit 4

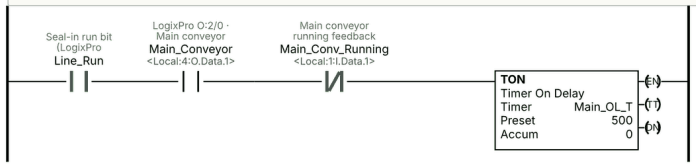
10 INLET NOT FILLING: the inlet clears the low float in a second or two; ten seconds with the valve commanded and the float still made means the valve did not open

11

12 CASE LANE DRIVE TRIPPED: bit 6. The line keeps running: the packer infeed holds two cases worth, and if it fills bit 7 stops the line

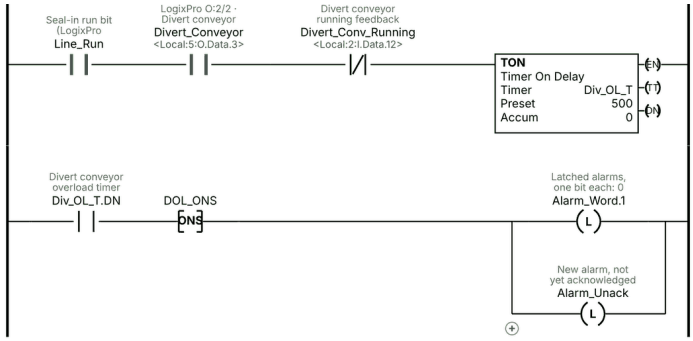
13 PACKER INFEED FULL: bit 7. Stop the line now, before bottles wait on the belt and back up to LS11

0 MAIN CONVEYOR OVERLOAD: Fwd is commanded but the running feedback from the motor contactor is missing for half a second

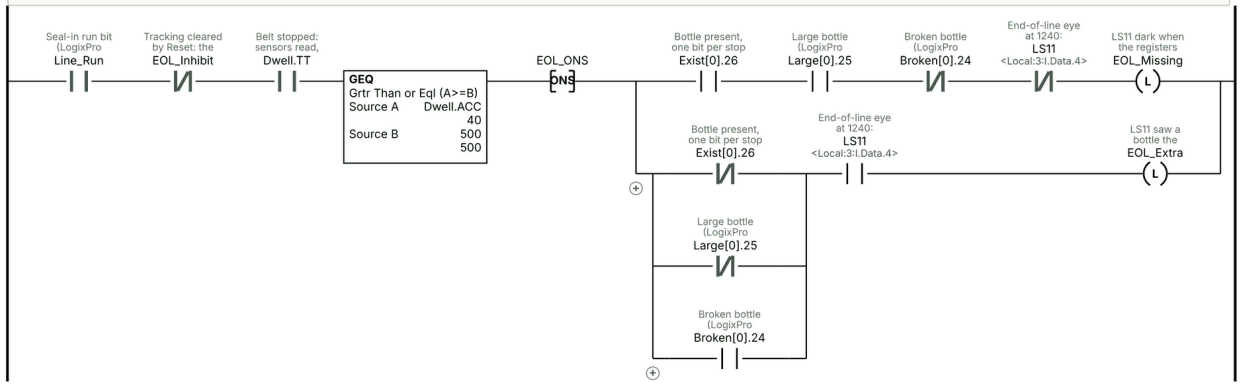


1 Latch alarm bit 0 and mark it unacknowledged. The one-shot means an acknowledged alarm is not re-raised every scan while the cause is still there

2 DIVERT CONVEYOR OVERLOAD: the same check



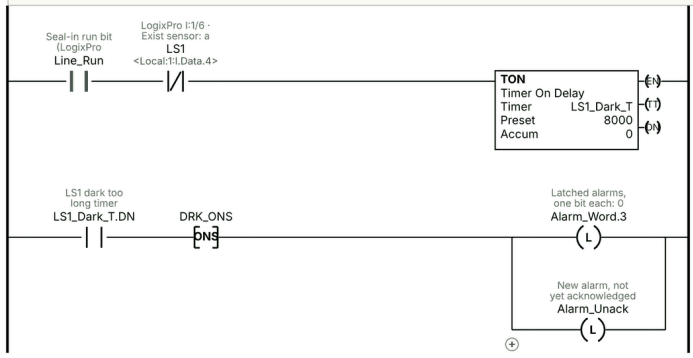
4 END-OF-LINE CHECK: once per stop, halfway through the dwell, compare LS11 at 1240 with what the registers say stands there. Only large, whole bottles reach the end, so the registers expect one when Exist 26, Large 25 and not Broken 24 (the dwell frame, chapter 10). A bottle expected but missing is a jam or a lost bottle upstream; a bottle there but not expected means the tracking is out of step



5 Either one is alarm bit 2

6 After a tracking reset the check waits until the first bottle tracked since the reset reaches the end

7 LS1 DARK TOO LONG: the feeder drops a bottle every stop, so LS1 sees one every couple of seconds. Eight seconds dark means a dirty lens stuck off, or nothing coming



9 LABELER FAULT from step 10 becomes alarm bit 4

10 INLET NOT FILLING: the inlet clears the low float in a second or two; ten seconds with the valve commanded and the float still made means the valve did not open

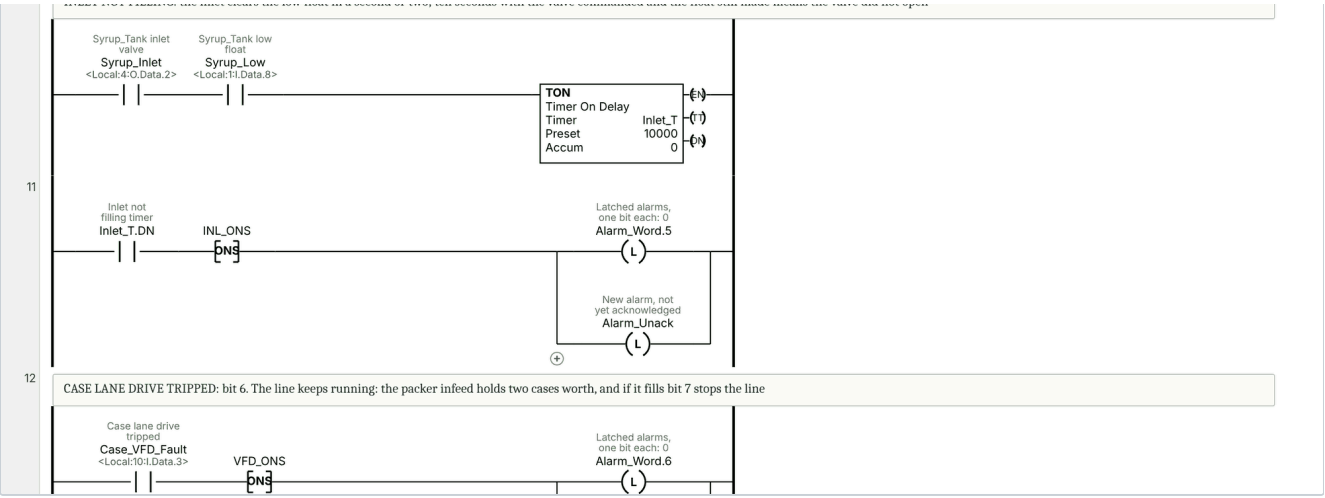


Figure 107. S11_Alarms online, with the finished line running: this is what your routine should look like once every step is in. Green is power flow.

TRY IT

Add a seventh alarm: the head tank (`Tank_Low`) — step 9 already stops the line on it, but nothing tells the operator why. Give it bit 6, latch it like the others, and make Reset clear it once the tank has recovered. Then turn on the drill at one fault every two minutes, run for ten, and keep a log of what you found and how long each took.

29 Lesson 14 — Step 12: the case packer and the VFD

The last station packs the finished bottles. A **top-load case packer** stands at the end of the main line: bottles that reach the end of the deck go into its **infeed** (it holds eight), and on a rising edge of `Pack_Drop` its pick head lifts four of them across to the case lane beside the line and lowers them into the case standing under the head. The case lane is the new thing in this lesson. Its motor is not switched by a contactor like every other conveyor on the line: it runs on a **variable frequency drive**, and the program tells the drive how fast through an **analog output**.

How the drive is wired

This is how most drives are wired to a CompactLogix. Three digital outputs: `Case_Run` (two-wire: the drive runs while it is 1), Rev (not used here) and `Case_VFD_Reset`. One analog output, the **speed reference**: a 0-10 V signal from channel 0 of the 1769-OF4 to the drive's analog input, which the drive is set to read as 0-60 Hz. Four digital inputs back: `Case_VFD_Ready`, `Case_VFD_Running` (output frequency above zero, also while it slows down), `Case_At_Speed` (running and within ½ Hz of the reference) and `Case_VFD_Fault`. And one analog input: the drive's own analog output, set to the output frequency, 0-10 V for 0-60 Hz, into the 1769-IF4 — `Case_Hz_Fbk`.

The analog channels are in **engineering units**: the number in the tag is millivolts, 0-10000 for 0-10.000 V. So the program cannot write "15 Hz" to the drive; it writes the voltage that the drive will read as 15 Hz: 15 ÷ 60 × 10000 = 2500. That conversion is the whole of analog output, and Logix has an instruction for it: **SCP**, Scale with Parameters — *Input*, the input's range (*Input Min*, *Input Max*), the range to map it onto (*Scaled Min*, *Scaled Max*), and the *Output*. It draws a straight line through two points; nothing more.

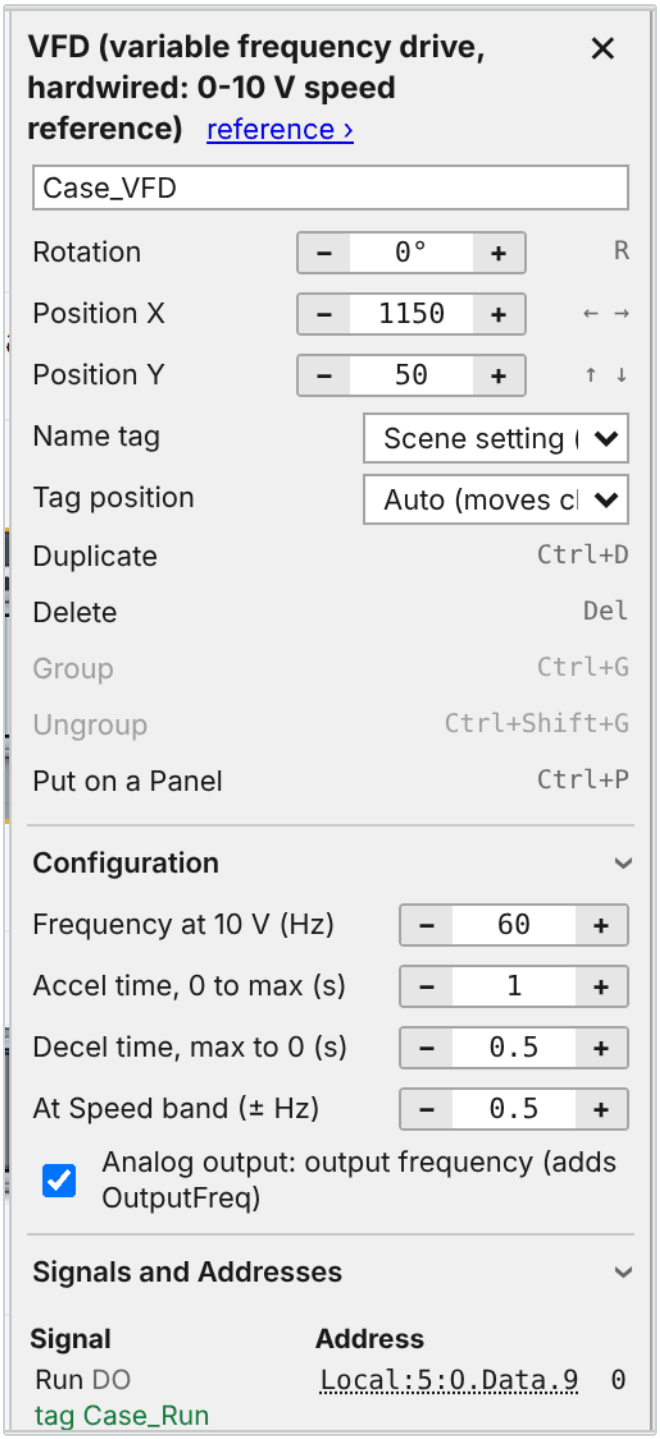


Figure 94. Case_VFD in the Properties drawer: the 10 V frequency, the accel and decel times, the At Speed band, and its signals with their I/O points.

Why a creep speed

A drive does not stop when Run drops: it ramps down on its *decel* time, 0.5 s from 60 Hz to 0 on this one. The case lane runs at 150 px/s at 60 Hz, so a case that is told to stop at full speed travels on for about 37 px — and the packer's `Case_In_Place` window is ±6 px. The cure every packaging line uses: a **slow-down**

eye a little before the mark (`Case_Slow`, 100 px before the head), a **creep speed** from there (15 Hz, 37.5 px/s), and the stop on the mark. From 15 Hz the drive is down in an eighth of a second and the case stops within about 2 px.

Add the JSR under step 11 and write:

> **IN THE APP**
>
> Guided mode lesson 14 is this chapter: it checks that this lesson's tags are made, that `S12_Pack` exists, is called from MainRoutine and verifies, and then watches the line while you run it and ticks each *Run it* result below as it happens. Help → Guided Mode, or start from File → New Project… → Guided: build the bottle line.

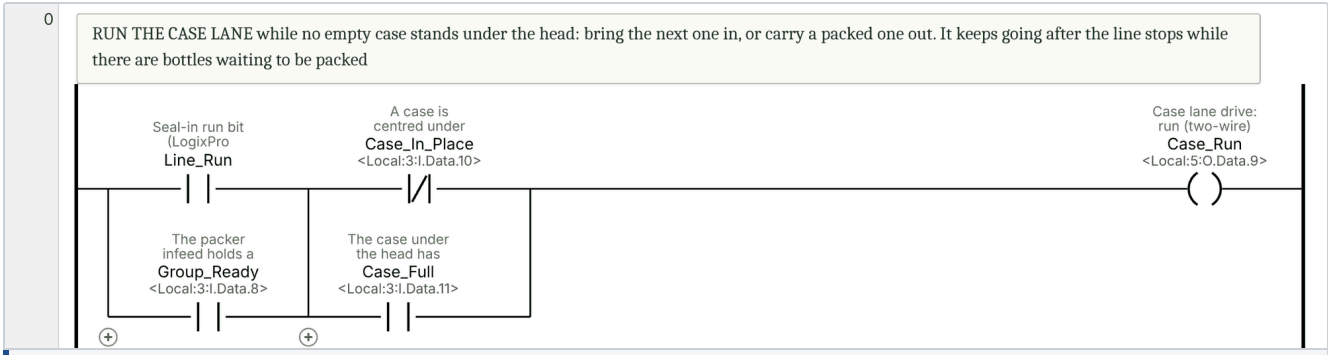
Tags this lesson needs

Create these before you type the rungs (Verify reports any you miss as unknown). Tags from earlier lessons are not repeated.

Tag	Type	How to make it	Description
`Group_Ready`	BOOL	alias for `Local:3:I.Data.8` — the Properties drawer's *tag* link makes it, then rename it	The packer infeed holds a case worth (4)
`Case_In_Place`	BOOL	alias for `Local:3:I.Data.10` — the Properties drawer's *tag* link makes it, then rename it	A case is centred under the packer head
`Case_Full`	BOOL	alias for `Local:3:I.Data.11` — the Properties drawer's *tag* link makes it, then rename it	The case under the head has been packed
`Case_Run`	BOOL	alias for `Local:5:O.Data.9` — the Properties drawer's *tag* link makes it, then rename it	Case lane drive: run (two-wire)
`Case_Slow`	BOOL	alias for `Local:3:I.Data.15` — the Properties drawer's *tag* link makes it, then rename it	Slow-down eye on the case lane, 100 px before the packer head
`Case_Creep`	BOOL	Edit Tags: type the name, pick the type	Latched from the slow-down eye until an empty case stands under the head
`Case_Hz`	REAL	Edit Tags: type the name, pick the type	Case lane speed the program wants, in Hz
`Case_Speed_Ref`	INT	alias for `Local:8:O.Ch0Data` — the Properties drawer's *tag* link makes it, then rename it	Case lane drive speed reference: 0-10000 = 0-10 V = 0-60 Hz
`Case_Hz_Fbk`	INT	alias for `Local:9:I.Ch2Data` — the Properties drawer's *tag* link makes it, then rename it	Case lane drive output frequency: 0-10000 = 0-10 V = 0-60 Hz
`Case_Hz_Actual`	REAL	Edit Tags: type the name, pick the type	Case lane drive output frequency read back, in Hz

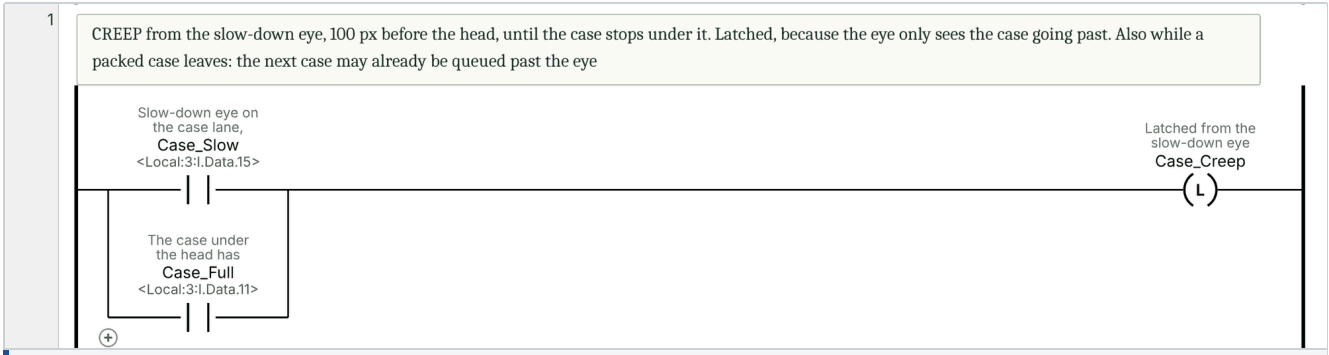
Tag	Type	How to make it	Description
`Case_VFD_Running`	BOOL	alias for `Local:10:I.Data.1` — the Properties drawer's *tag* link makes it, then rename it	Case lane drive output frequency above zero
`Packer_Ready`	BOOL	alias for `Local:3:I.Data.7` — the Properties drawer's *tag* link makes it, then rename it	Packer head home, not cycling
`Drop_ONS`	BOOL	Edit Tags: type the name, pick the type	
`Pack_Drop`	BOOL	alias for `Local:5:O.Data.7` — the Properties drawer's *tag* link makes it, then rename it	Packer: pick a case worth from the infeed and place it in the case (rising edge)
`Case_ONS`	BOOL	Edit Tags: type the name, pick the type	
`Case_Count`	DINT	Edit Tags: type the name, pick the type	Cases packed, shown on the CASES display
`Case_VFD_Reset`	BOOL	alias for `Local:5:O.Data.11` — the Properties drawer's *tag* link makes it, then rename it	Case lane drive: fault reset (rising edge)

RUNG 0



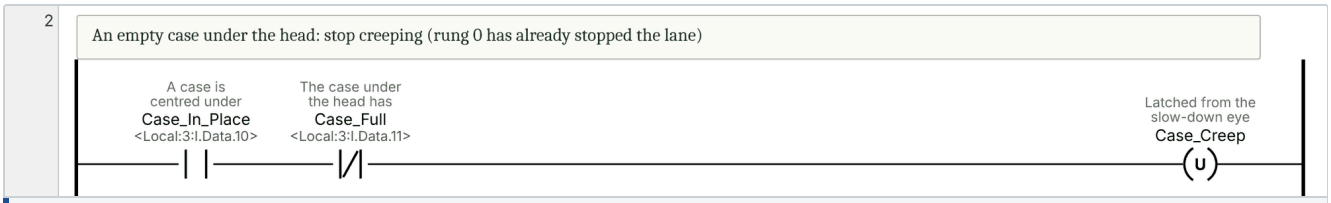

```
[XIC(Line_Run),XIC(Group_Ready)]
[XIO(Case_In_Place),XIC(Case_Full)]OTE(Case_Run);
```

RUNG 1




```
[XIC(Case_Slow),XIC(Case_Full)]OTL(Case_Creep);
```

RUNG 2

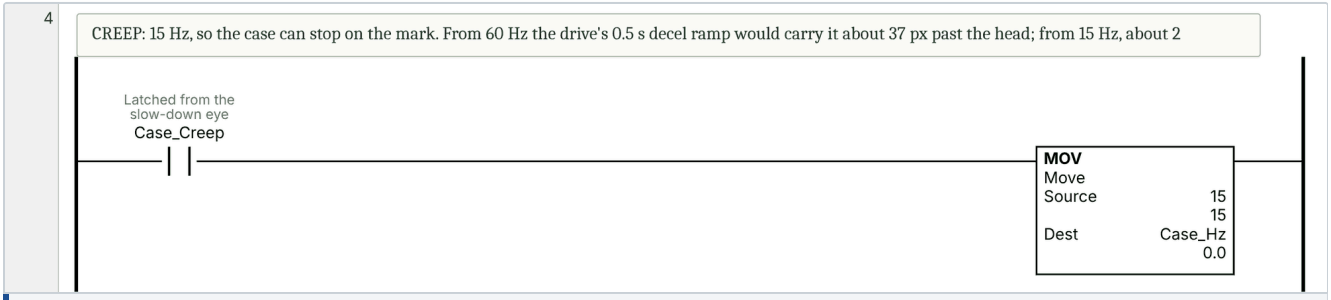



```
XIC(Case_In_Place)XIO(Case_Full)OTU(Case_Creep);
```

RUNG 3

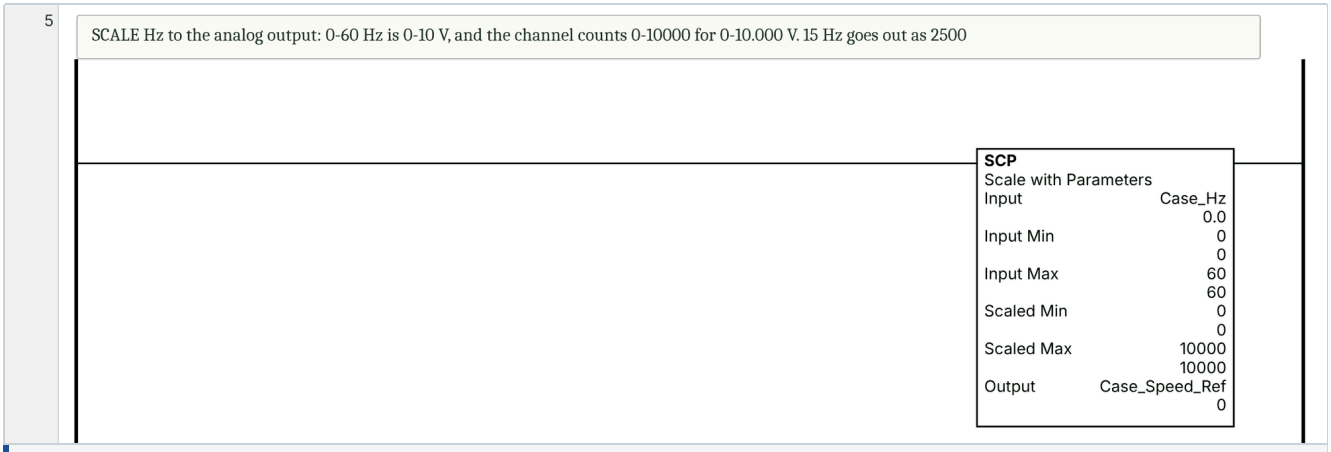
```
MOV(60,Case_Hz);
```

RUNG 4



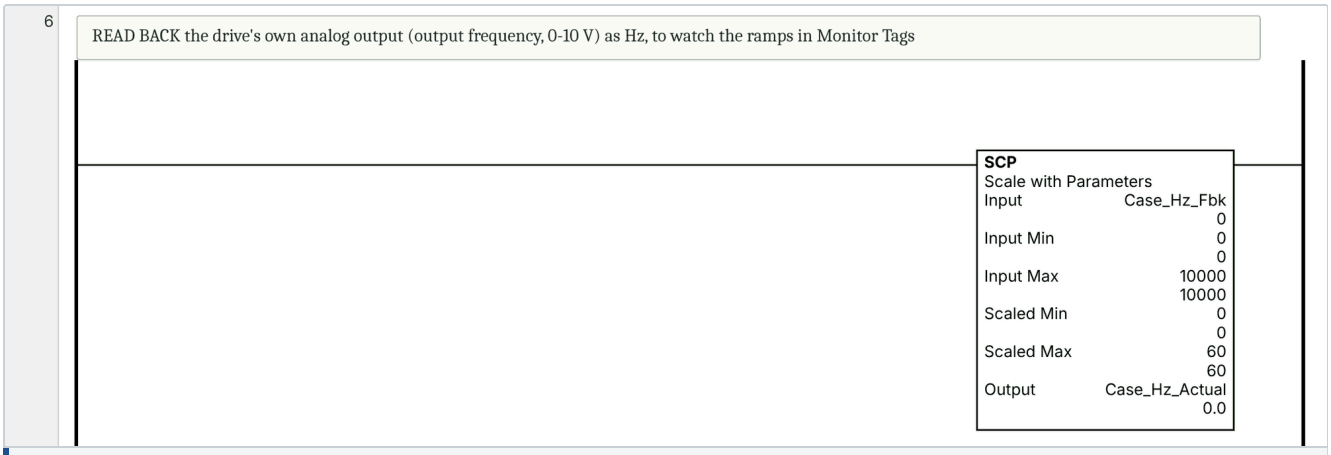

```
XIC(Case_Creep)MOV(15,Case_Hz);
```

RUNG 5



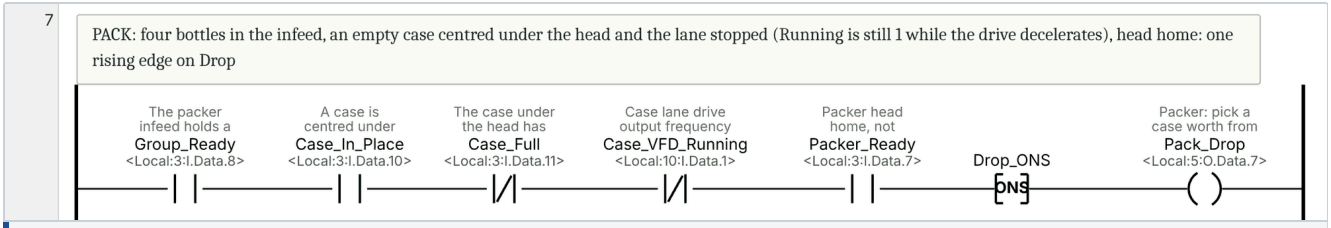

```
SCP(Case_Hz,0,60,0,10000,Case_Speed_Ref);
```

RUNG 6




```
SCP(Case_Hz_Fbk,0,10000,0,60,Case_Hz_Actual);
```

RUNG 7

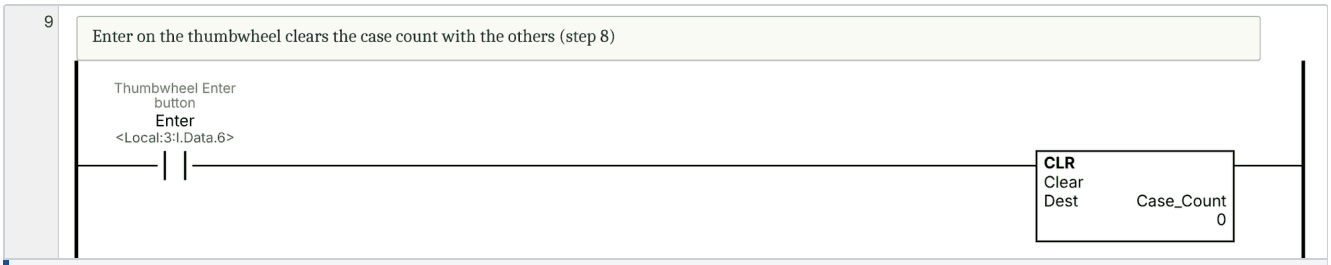



```
XIC(Group_Ready)XIC(Case_In_Place)XIO(Case_Full)XIO(Case_VFD_Running)XIC(Packer
_Ready)ONS(Drop_ONS)OTE(Pack_Drop);
```

RUNG 8

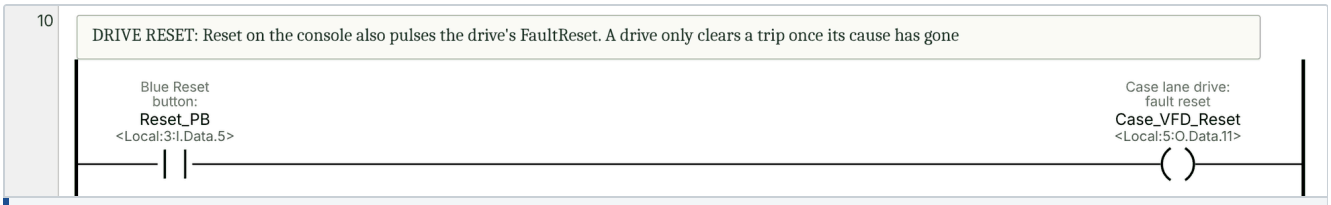
```
XIC(Case_Full)ONS(Case_ONS)ADD(Case_Count,1,Case_Count);
```

RUNG 9




```
XIC(Enter)CLR(Case_Count);
```

RUNG 10




```
XIC(Reset_PB)OTE(Case_VFD_Reset);
```

Rung 0 runs the lane whenever no *empty* case stands under the head: either nothing is in place (bring the next case in) or the case in place is full (carry it out). It runs with the line, and also after the line has stopped as long as a case worth of bottles is waiting — or the infeed could never empty. **Rungs 1–2** are the creep, **latched**: the slow-down eye only sees the case going past it, and the case still has 60 px to go when the eye clears. The latch is also set while a packed case leaves, because the next case may already be queued past the eye and must not start for the head at full speed. An empty case under the head ends the creep.

Rungs 3–4 choose the speed in Hz with the overwrite pattern: `MOV(60,…)` every scan, then `MOV(15,…)` if creeping. The second rung wins because it runs later in the same scan, and the drive only ever sees one value. **Rung 5** is the analog output: SCP maps 0-60 Hz onto 0-10000, so 60 Hz goes out as 10000 (10 V) and 15 Hz as 2500 (2.5 V). **Rung 6** is the same instruction the other way round: the drive's 0-10 V output frequency back to Hz in `Case_Hz_Actual`, a REAL, so you can watch the ramps in Monitor Tags.

Rung 7 packs: a case worth in the infeed (`Group_Ready`), an empty case on the mark, the head home, and the lane *stopped*. `XIO(Case_VFD_Running)`, not `XIO(Case_Run)`: Run drops the moment the case is in place, but the drive is still ramping down for a few scans, and a head that comes down on a moving case misses it. **Rung 8** counts the case on the CASES display, **rung 9** lets Enter clear it with the other counts, and **rung 10** sends Reset to the drive's FaultReset: a drive clears a trip only on that rising edge, and only once the cause has gone.

Run it

Start the line and put `Case_Hz`, `Case_Speed_Ref`, `Case_Hz_Actual`, `Case_In_Place` and `Infeed_Count` in Monitor Tags (or a Watch). The first carton comes in at 60 Hz — `Case_Speed_Ref` 10000, `Case_Hz_Actual` climbing on the 1 s accel ramp — reaches the slow-down eye, drops to 2500 and

creeps, and stops under the head: `Case_In_Place` 1, the drive's keypad reads 0.0. Large bottles collect in the infeed; at the fourth, the head swings out, lowers them into the case, swings back, and the case leaves for Case_Out with four bottle tops showing. CASES counts 1. In the test run the case stopped 2.6 px from the head, and five minutes packed eleven cases with nothing lost.

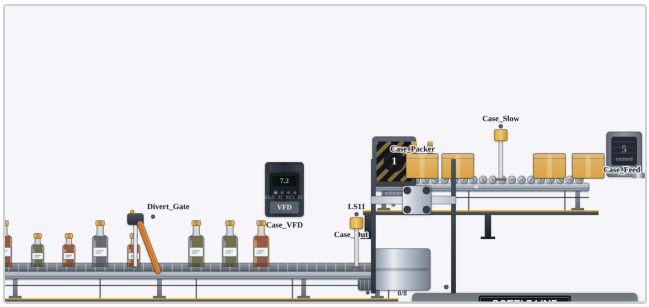


Figure 95. The packer in Side view: the infeed at the end of the deck, the head over a case on the lane.

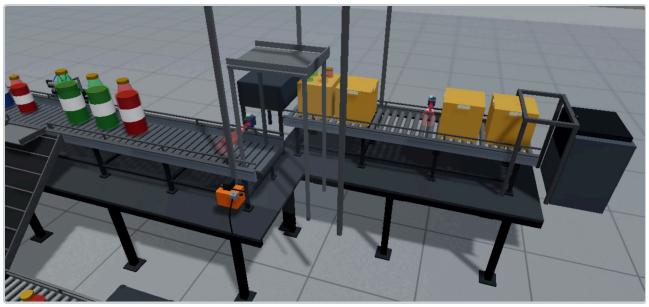


Figure 96. In 3D: the gantry over the end of the line and the case lane, a packed case leaving.

WATCH FOR: THE STOP THAT IS NOT A STOP

Take rung 4 out (or set the creep to 60 Hz) and run: the case reaches the mark, `Case_Run` drops, and the case sails on 30-odd px on the decel ramp, out of the window; `Case_In_Place` goes back to 0, rung 0 runs the lane again, and the case leaves empty. Nothing in the program is wrong except the physics. Every positioning problem with a drive is this one: the stopping distance is speed × decel time ÷ 2.

0

RUN THE CASE LANE while no empty case stands under the head: bring the next one in, or carry a packed one out. It keeps going after the line stops while there are bottles waiting to be packed

1

CREEP from the slow-down eye, 100 px before the head, until the case stops under it. Latched, because the eye only sees the case going past. Also while a packed case leaves: the next case may already be queued past the eye

2

An empty case under the head: stop creeping (rung 0 has already stopped the lane)

3

FAST: 60 Hz unless the next rung says otherwise

4

CREEP: 15 Hz, so the case can stop on the mark. From 60 Hz the drive's 0.5 s decel ramp would carry it about 37 px past the head; from 15 Hz, about 2

5

SCALE Hz to the analog output: 0-60 Hz is 0-10 V, and the channel counts 0-10000 for 0-10.000 V. 15 Hz goes out as 2500

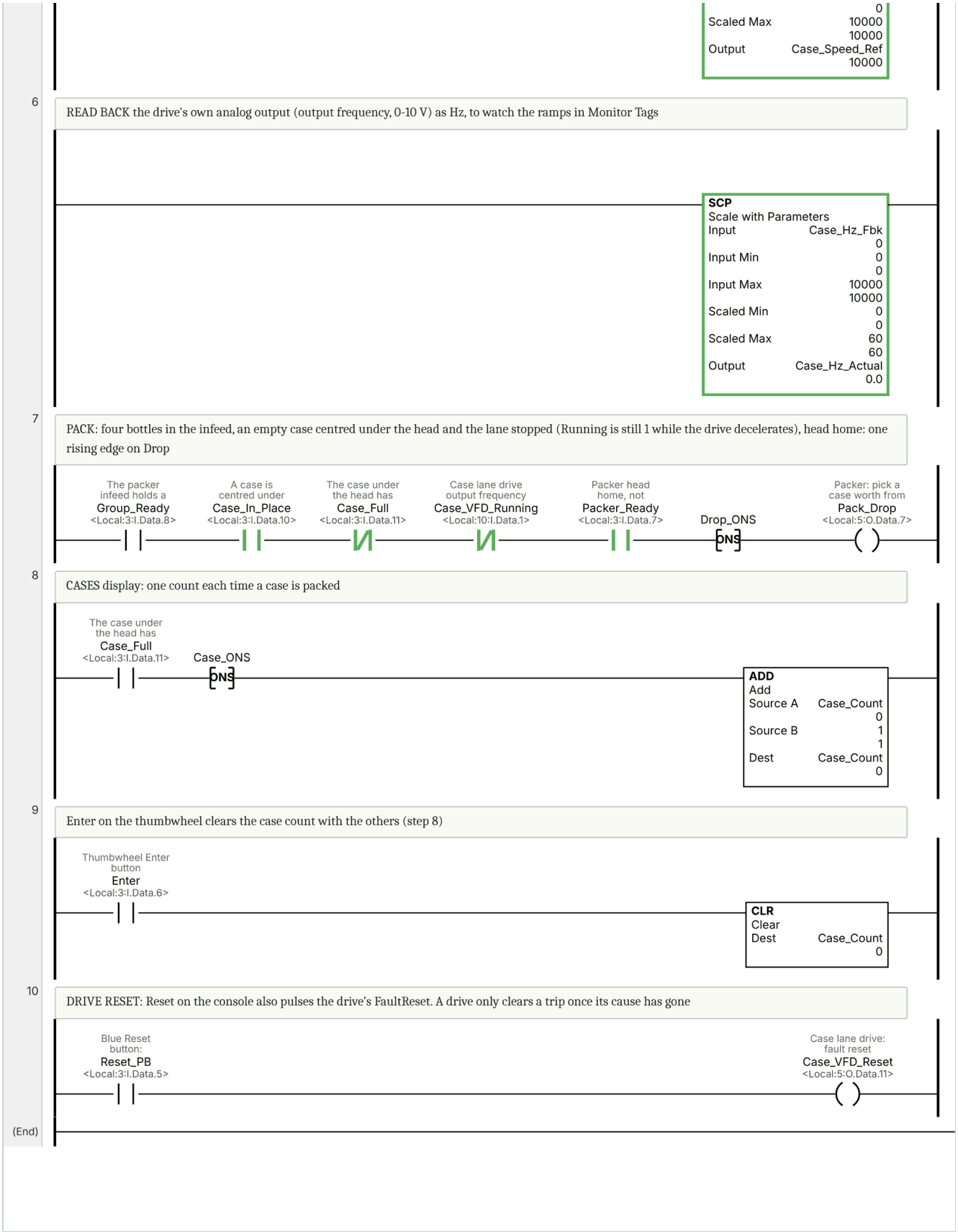


Figure 108. S12_Pack online, with the finished line running: this is what your routine should look like once every step

is in. Green is power flow.

TRY IT

Set the creep to 30 Hz (5000) and count how often the case still lands in the window. Then set Case_VFD's *Decel* to 2 s: the creep has to come down to make it work again. Last, Tools → Inject Machine Fault → Drive trip on Case_VFD: bit 6 latches, the lane coasts to a stop, the infeed fills — and when it reaches eight, bit 7 stops the line. Clear Machine Faults, press Reset, watch the drive come back and the infeed drain, Reset again, Start.

30 Commissioning: the whole line, start to finish

With all twelve routines in place, MainRoutine is twelve JSRs in step order. Verify Controller (Logic menu): *0 error(s)*. Reset Sim. Then run this checklist, the one the program's own test runs on every release:

1. Run mode. Press **Start**. Run lamp on; divert conveyor turning; the belt steps and a bottle drops every stop.
2. Within a minute bottles are filling at stop 7 (large charge for large bottles — the LARGE lamp on the head block; small — the SMALL lamp) and capping at stop 10 (the magazine indexes a pocket per cap).
3. Broken bottles are dropped into the shredder; the box conveyor positions a box under the chute; IN BOX counts to 4; the box leaves and the next arrives; BOXES counts 1.
4. Small bottles ride the incline to the divert conveyor and are counted on SMALL; large bottles run to the end and are counted on LARGE; SCRAP counts the broken ones.
5. Nothing is **lost** (the toolbar shows no red number) and the Simulator Log has no *no bottle under it*, *no box* or *nothing to land on* lines.
6. The tank falls to 20 %, the red light flashes, the inlet opens, the level climbs to 90 %.
7. Dial 0010 on the thumbwheel; the line stops at ten large bottles; Enter clears; Start runs again.
8. Every capped bottle leaves the labeler with a label; `Label_Fault` stays off for the whole run.
9. `Alarm_Word` stays 0 and the fault lamp dark for the whole run, including when you press Stop and Start at random moments.
10. Every few large bottles a case is brought in, stops under the packer head, is packed with four and leaves at Case_Out; CASES counts them, and no empty case is sent out.
11. Each fault of chapter 27's *Run it* (and a drive trip on Case_VFD), injected with Tools → Inject Machine Fault, is caught by the bit the table says, stops the line if it should, and clears with Clear Machine Faults, Reset and Start.

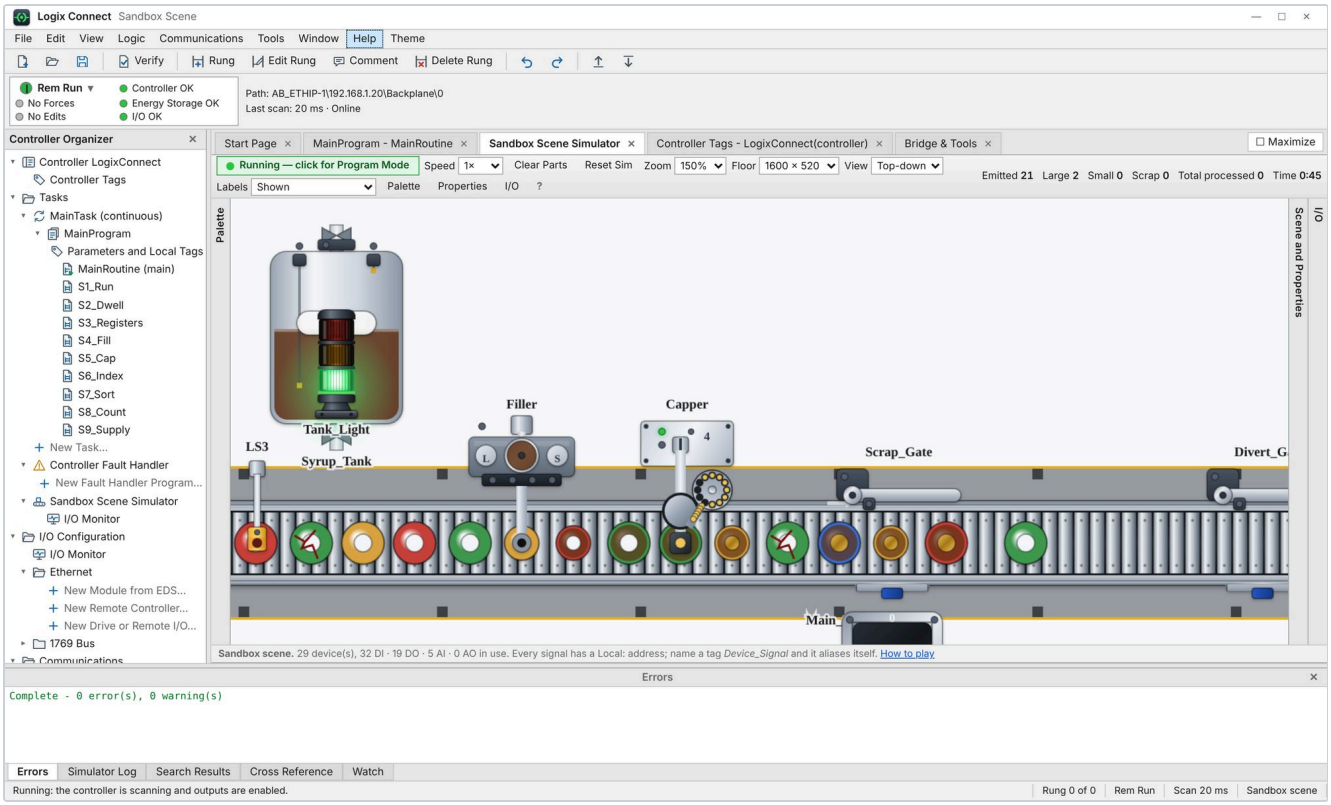


Figure 109. The whole line running in Top-down view: the toolbar counters read the program's own tags.

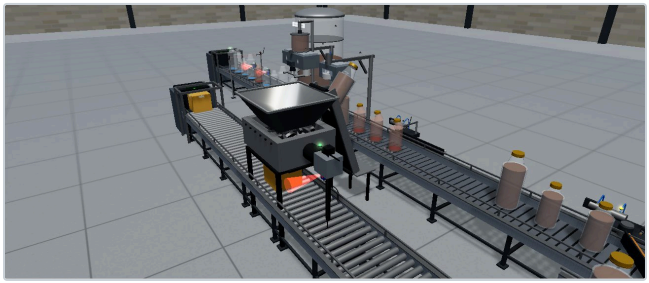


Figure 110. Glass into the box: IN BOX counts up.

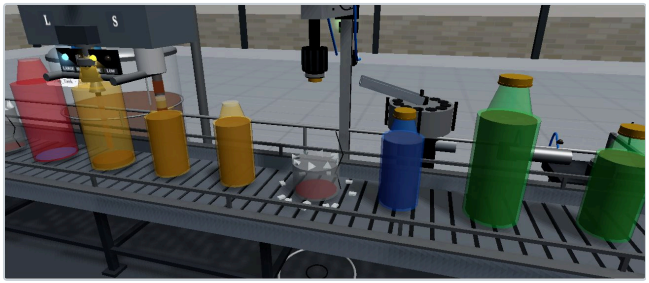


Figure 111. What happens when a chuck comes down on a broken bottle — take the Broken bit out of the cap request and see.

The regression runs every line of that list on every release, plus two dozen random Stop / Start presses. If any line fails for you, the Simulator Log names the device, Monitor Tags shows the bit, and the matching lesson's *Run it* paragraph says what should have happened. The finished template (File → New Project… → Sandbox: LogixPro-style Bottle Line) is the answer key; File → Export / Copy as Text… on both and compare.

Appendix B is the complete listing; Appendix C the bit map. Save the project. You have built a bottling line from an empty floor and written every rung that runs it.

Part V — The graded levels

31 The graded levels: the machine, the bins and the score

Part IV built one machine and one program. Part V is seven puzzles on a second machine, each a little harder than the last, with a score: the graded levels. They began as a fixed machine of their own; from Version 91 each level is built from the same palette parts as everything else in this book, and from Version 93 every one of them has been solved by the programs printed in the next seven chapters — every program here ran to completion, every bottle in the right bin, before it was printed.

Building a level

File → New Project… → Level on sandbox parts, choose the level, OK. You get an empty program and this machine:

- **Hopper** — an Emitter in edge mode, limited to the level's number of bottles. `Infeed_Release` drops one bottle on each rising edge; `Infeed_Ready` is 1 while the exit is clear.
- **Main_Belt** — one belt the length of the floor; `Conveyor_Run` runs it.
- **The inspection gantry** — `PE_Inspect` (a bottle is here), three retro beams at 30, 44 and 52 px (`PE_Low`, `PE_Mid`, `PE_High`: a small bottle is 40 px tall and breaks only the low one, a medium is 48 and breaks two, a large is 56 and breaks all three), the colour sensor's three banks (`Color_Red`, `Color_Blue`, `Color_Green`) and the crack eye (`Crack_Sensor`, broken bottles only).
- **The filler** — a dispenser over the belt (`Fill_Valve`), the fill-level sensor beside the nozzle with its analog percent (`Fill_Level`, 0-100 % of the bottle's own label) and the presence eye under the nozzle (`PE_Fill`, short range, placed so a bottle that stops on it stands under the nozzle).
- **One pusher and one bin per lane** — `Div1` … `Div6` extend the pushers; each pusher has its own eye, `PE_Div1` …, just ahead of the arm.
- **The end of the line** — `PE_LineEnd` and a remover that takes anything nobody pushed. On Level 6 the belt is shorter and ends in a **case packer** with a VFD-driven case lane, the same parts and tag names as step 12 of the bottle line (chapter 28). Level 7 is a different machine altogether: a batch mix tank and its console, no belt at all.
- **The console** — `Start_PB`, `Stop_PB` (a plain button on this machine), `E_Stop` (1 while healthy), `Lamp_Run`, `Lamp_Fault`.

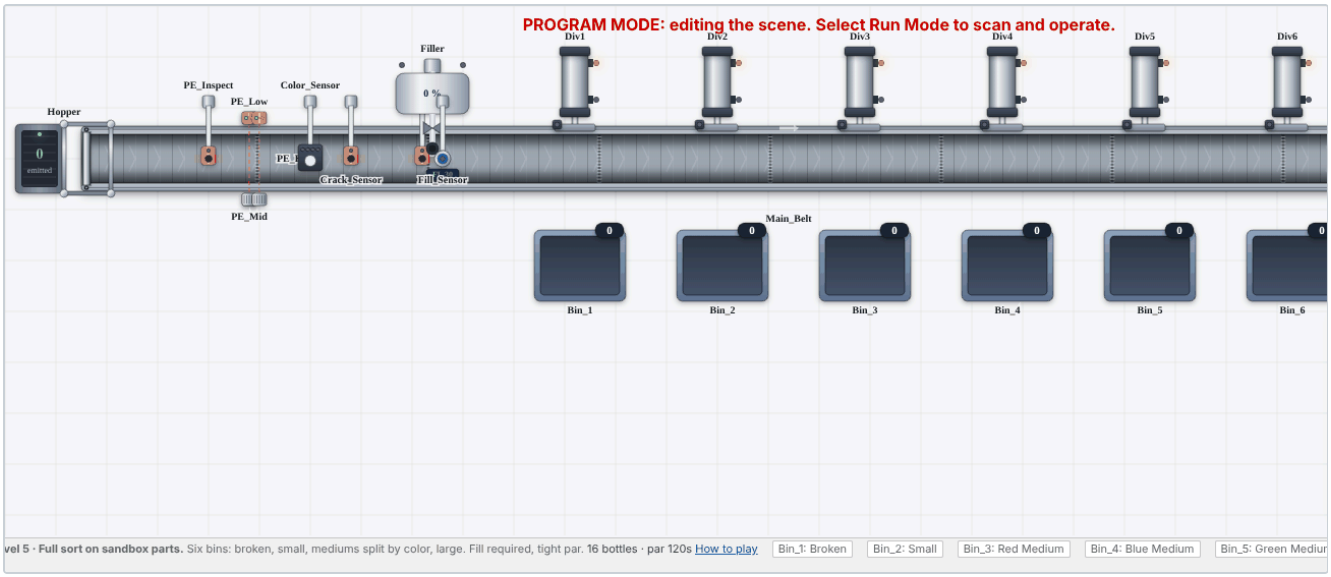


Figure 112. Level 5 on sandbox parts in the Top-down view with Specs on: the hopper, the gantry, the filler, six pushers over six graded bins, the console.

The tags are already made

Every name in the list above is an alias tag the level makes for you, pointing at the part's I/O point. They are the names the levels have always used, so a program written for the old fixed machine runs here unchanged:

Tag	Alias for	Description
`Start_PB`	`Local:3:I.Data.9`	Start push button, momentary
`Stop_PB`	`Local:3:I.Data.10`	Stop push button, momentary
`E_Stop`	`Local:3:I.Data.11`	E-stop, NC: 1 = healthy
`Infeed_Ready`	`Local:1:I.Data.0`	Hopper exit clear: a bottle can be released
`Infeed_Release`	`Local:4:O.Data.0`	Release one bottle onto the belt (rising edge)
`Conveyor_Run`	`Local:4:O.Data.1`	Belt motor
`PE_Inspect`	`Local:1:I.Data.2`	Presence eye at the inspection gantry
`PE_Low`	`Local:1:I.Data.3`	Height beam at 30 px: blocked by every size
`PE_Mid`	`Local:1:I.Data.4`	Height beam at 44 px: blocked by M and L
`PE_High`	`Local:1:I.Data.5`	Height beam at 52 px: blocked by L only
`Color_Red`	`Local:1:I.Data.6`	Colour sensor bank 1: red
`Color_Blue`	`Local:1:I.Data.7`	Colour sensor bank 2: blue
`Color_Green`	`Local:1:I.Data.8`	Colour sensor bank 3: green

Tag	Alias for	Description
`Crack_Sensor`	`Local:1:I.Data.10`	Crack eye: the bottle in front of it is broken
`PE_Fill`	`Local:1:I.Data.14`	Presence eye under the fill nozzle
`Fill_Valve`	`Local:4:O.Data.2`	Fill valve
`Fill_Level`	`Local:7:I.Ch1Data`	Fill of the bottle under the nozzle, 0-100 % of its label
`Fill_Done`	`Local:1:I.Data.13`	Liquid at the fill line of the bottle under the nozzle
`PE_Div1`	`Local:2:I.Data.1`	Presence eye ahead of diverter 1
`Div1`	`Local:4:O.Data.4`	Diverter 1 cylinder
`PE_Div2`	`Local:2:I.Data.5`	Presence eye ahead of diverter 2
`Div2`	`Local:4:O.Data.5`	Diverter 2 cylinder
`PE_Div3`	`Local:2:I.Data.9`	Presence eye ahead of diverter 3
`Div3`	`Local:4:O.Data.6`	Diverter 3 cylinder
`PE_Div4`	`Local:2:I.Data.13`	Presence eye ahead of diverter 4
`Div4`	`Local:4:O.Data.7`	Diverter 4 cylinder
`PE_Div5`	`Local:3:I.Data.1`	Presence eye ahead of diverter 5
`Div5`	`Local:4:O.Data.8`	Diverter 5 cylinder
`PE_Div6`	`Local:3:I.Data.5`	Presence eye ahead of diverter 6
`Div6`	`Local:4:O.Data.9`	Diverter 6 cylinder
`PE_LineEnd`	`Local:3:I.Data.7`	Presence eye at the end of the belt
`Lamp_Run`	`Local:4:O.Data.10`	Run lamp
`Lamp_Fault`	`Local:4:O.Data.11`	Fault lamp

(`Label_Water`, `Label_Soda` and `Label_Juice` are plain tags that stay 0: the label reader of the old machine is not fitted.)

Graded bins and the score

Each bin carries a *Grading* rule — whole or broken bottles, a size, a colour, must be filled — set in its Properties, and it scores what lands in it: **+100** for the right bottle, **−50** for the wrong one, **−40** for the right bin with a wrong fill (or a filled broken bottle). A bottle that falls off the line is −50. When the hopper is empty and the belt is clear, the level completes: **+200** if you are under par, and a card with your score and stars — three for no mistakes under par, two for no mistakes, one for 70 % right. **Specs** on the simulator toolbar shows each bin's rule. You can give any bin of your own a grading rule, too.

The seven levels

Level	Bins	Bottles	Fill	Par	The new idea
1 Height sort	broken, S, M, L	10	no	150 s	latching what a sensor saw; one bottle at a time
2 Fill by label	broken, S, M, L	10	yes	170 s	stopping on a sensor and an analog fill
3 Colour sort	broken, R, B, G	12	yes	190 s	a different sensor, the same pattern
4 Two on the line	broken, S, M, L	14	yes	110 s	tracking more than one bottle: FIFO queues
5 Full sort	broken, S, R/B/G medium, L	16	yes	120 s	a class word carrying size and colour
6 Pack it	broken; the rest packed 4 to a case	16	yes	140 s	an analog speed reference, positioning on a drive, the end of a batch
7 Batch Mix	three batches, recipes A, B, C	—	—	240 s	a step sequencer, dosing by counting, temperature with hysteresis

Guided mode on a level

A level project opens with the guided-mode panel on, following that level's chapter: the tags to make, every rung of the chapter's program under a fold, a check that MainRoutine verifies, and three results it ticks while you run the level — complete, every bottle right, under par. *Next level* on the score card builds the next level and keeps the panel on. Help → Guided Mode turns it off or back on.

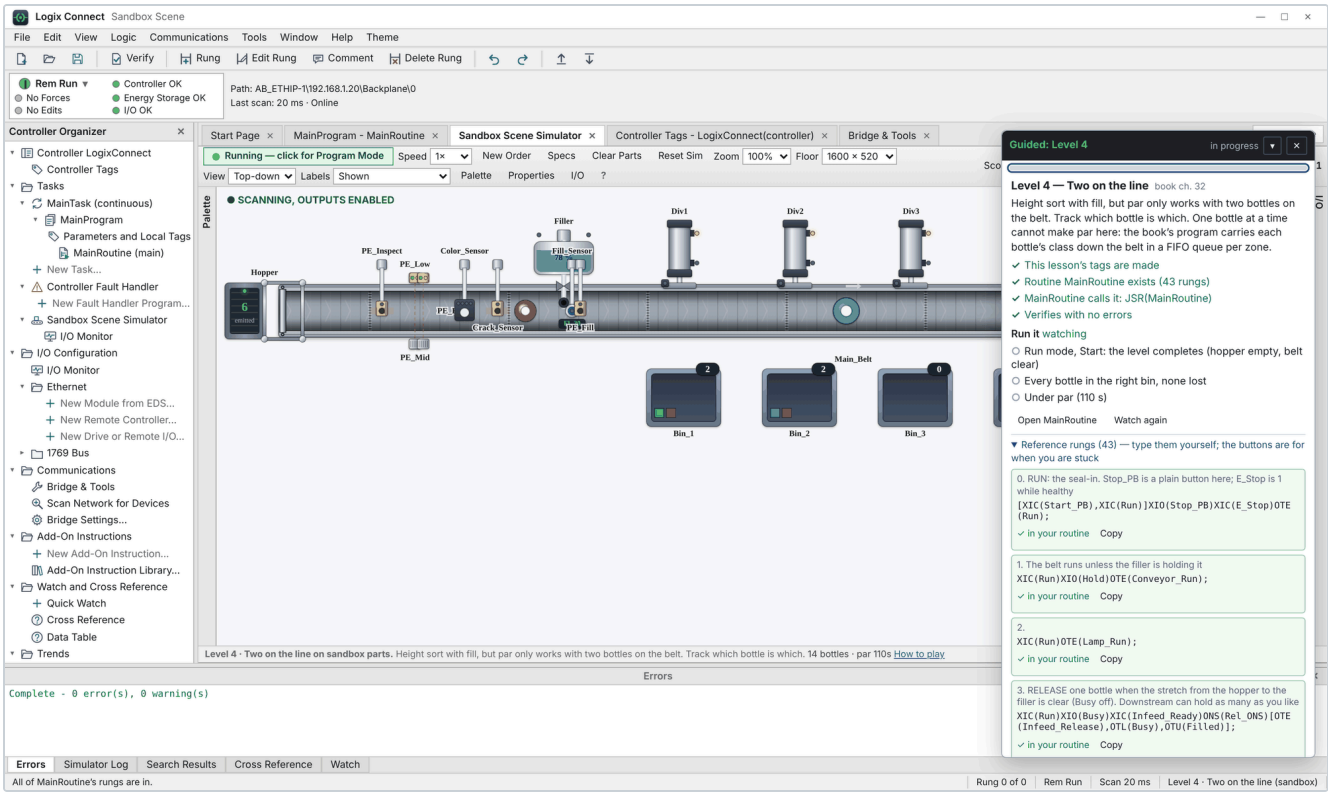


Figure 113. Level 4 with the guided panel: the program in, the line running, the run checks waiting for the level to complete.

Each chapter is built like the lessons of Part IV: what the level asks, the tags, every rung as the editor draws it with the text you type, what the rungs do, *Run it*, the routine online, and the traps. Write each program into MainRoutine. **File → Load Reference Solution: Level N on sandbox parts** installs the Level 4/5 program for whichever level you are in, and works on Levels 1-3 as well; use it as an answer key in a second tab, as chapter 3 describes.

32 Level 1 — Height sort

> **IN THE APP**
>
> A level project opens with guided mode on this chapter: the panel lists the tags the program needs (*Make them*), shows every rung below under its fold with *Copy* and *Add*, checks that MainRoutine verifies, and while you run the level ticks three results — the level completes, every bottle in the right bin, under par.

Ten bottles — small, medium and large, in three colours, some of them broken — come out of the hopper one per release. Sort them by height into four bins: broken bottles to bin 1, small to 2, medium to 3, large to 4. The filler is there but not scored on this level; leave it alone. Par is 150 s.

The plan is the simplest one that works: release a bottle, let it run past the gantry, **remember** what the gantry saw, fire the right pusher when the bottle reaches it, wait for it to land, release the next. One bottle on the belt at a time means one set of memory bits, and that is the whole idea of this level: a sensor tells you something for a fraction of a second, and the program has to keep it until it is needed further down the belt.

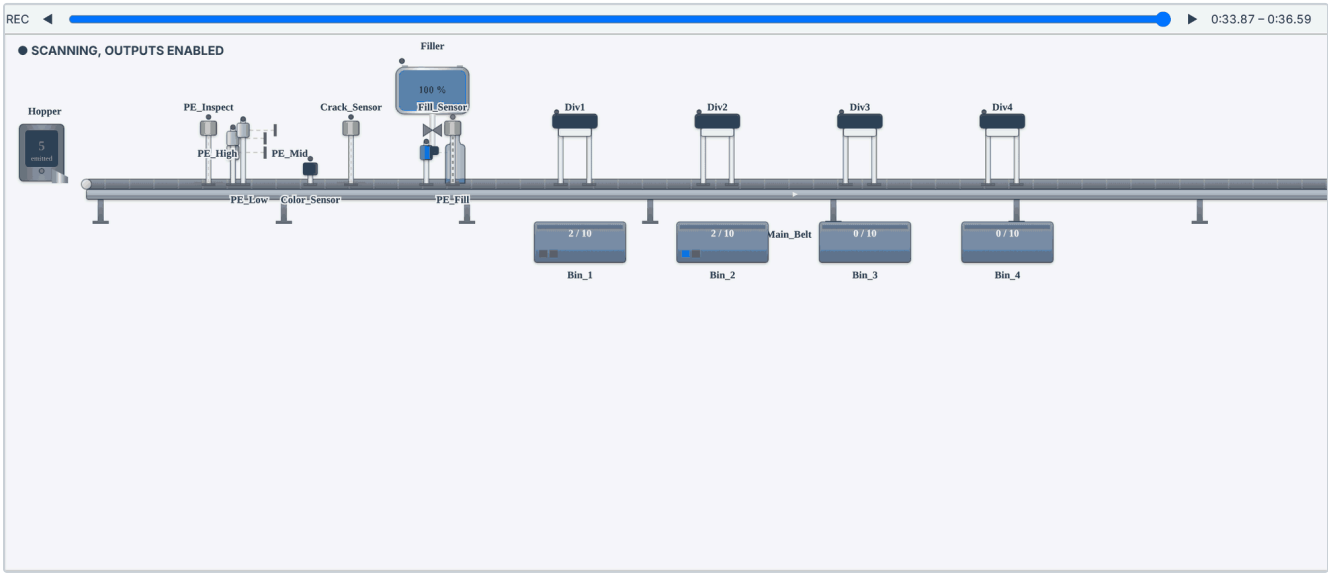


Figure 114. Level 1 running in the Side view: a bottle on its way from the gantry to the pushers.

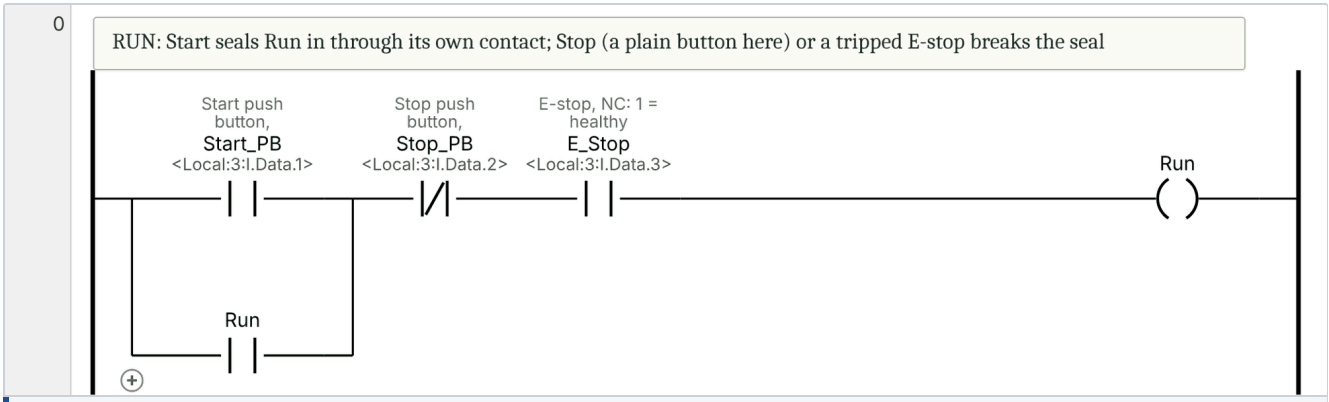
Tags this lesson needs

The level's alias tags are already made for you (chapter 30's table). Create these plain tags before you type the rungs; Verify reports any you miss.

Tag	Type	How to make it	Description
`Run`	BOOL	Edit Tags: name, type	Seal-in run bit
`Busy`	BOOL	Edit Tags: name, type	A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)
`Rel_ONS`	BOOL	Edit Tags: name, type	One-shot storage for the release
`Is_Low`	BOOL	Edit Tags: name, type	Latched: the low height beam was broken
`Is_Mid`	BOOL	Edit Tags: name, type	Latched: the middle beam was broken
`Is_High`	BOOL	Edit Tags: name, type	Latched: the high beam was broken
`Is_Broken`	BOOL	Edit Tags: name, type	Latched: the crack eye saw a broken bottle
`Done`	BOOL	Edit Tags: name, type	A pusher has fired for this bottle
`After`	TIMER	Edit Tags: name, type	Landing time after a push
`DN`	BOOL	Edit Tags: name, type	
`Trip`	TIMER	Edit Tags: name, type	Watchdog: the bottle ran the whole belt

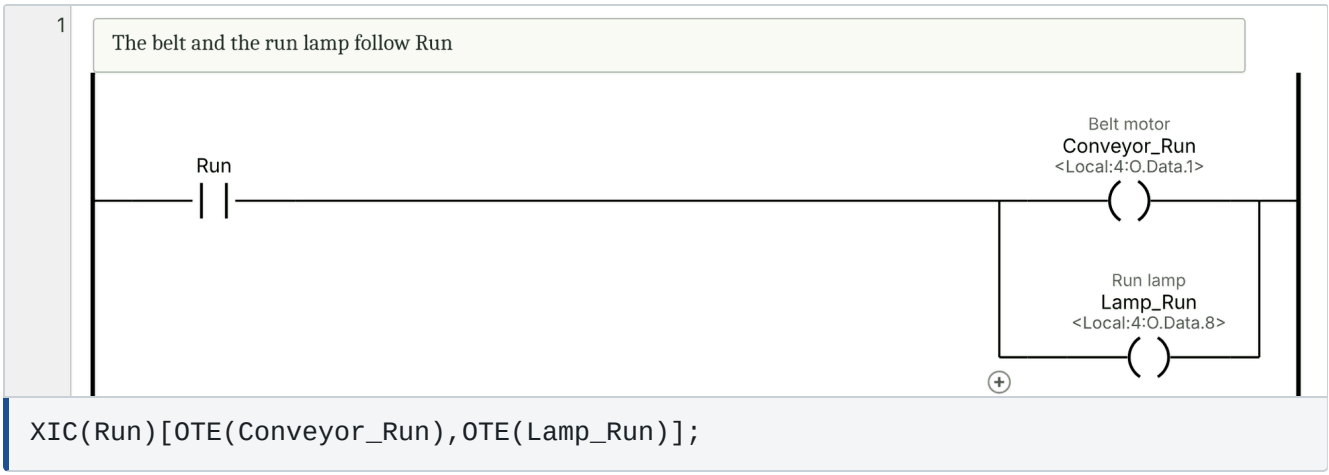
The run circuit

RUNG 0




```
[XIC(Start_PB),XIC(Run)]XIO(Stop_PB)XIC(E_Stop)OTE(Run);
```

RUNG 1

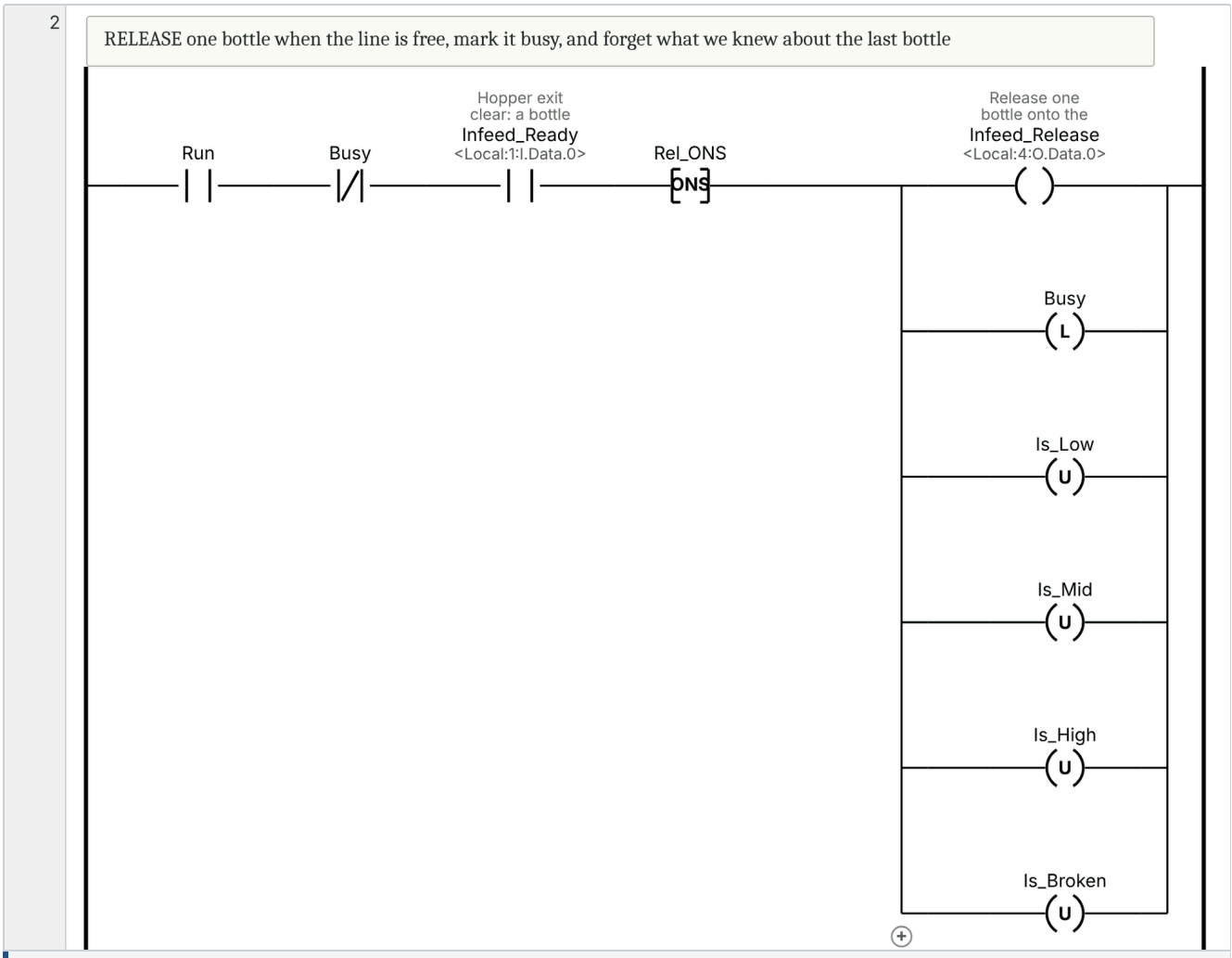



```
XIC(Run)[OTE(Conveyor_Run),OTE(Lamp_Run)];
```

Rung 0 is chapter 4's seal-in. On this machine Stop is a plain normally-open button, so it is `XIO(Stop_PB)`, not the `XIC(Stop)` of the bottle line, whose stop button is normally closed — read the device's settings, not your habits. `E_Stop` is 1 while the E-stop is healthy. **Rung 1** runs the belt and lights the run lamp whenever the line runs; on this level the belt never stops.

Releasing one bottle

RUNG 2

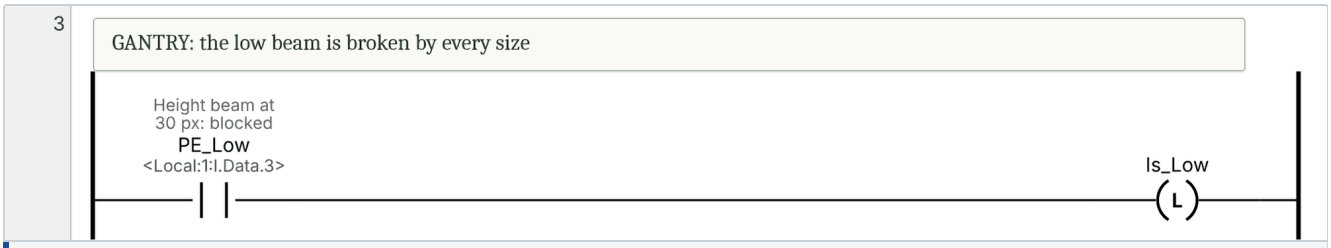



```
XIC(Run)XIO(Busy)XIC(Infeed_Ready)ONS(Rel_ONS)
[OTE(Infeed_Release),OTL(Busy),OTU(Is_Low),OTU(Is_Mid),OTU(Is_High),OTU(Is_Brok
en)];
```

The release rung fires once — the one-shot — when the line runs, no bottle is in progress (`XIO(Busy)`) and the hopper's exit is clear. On the same scan it latches `Busy`, so nothing else is released until this bottle is dealt with, and unlatches the four memory bits, so this bottle starts with a clean slate. The hopper is in edge mode: one rising edge of `Infeed_Release`, one bottle.

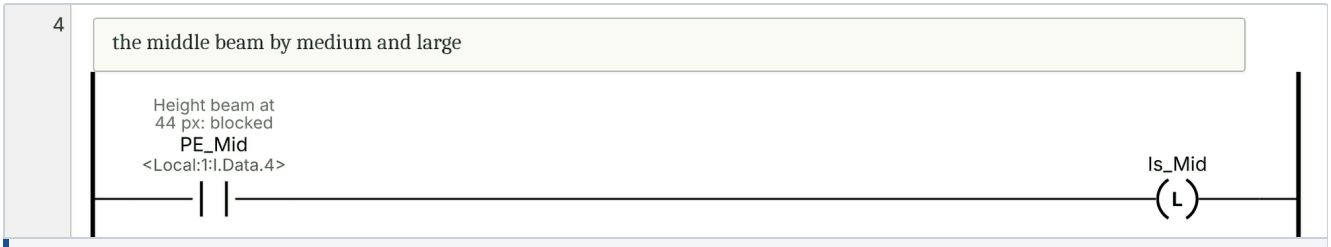
Remembering what the gantry saw

RUNG 3



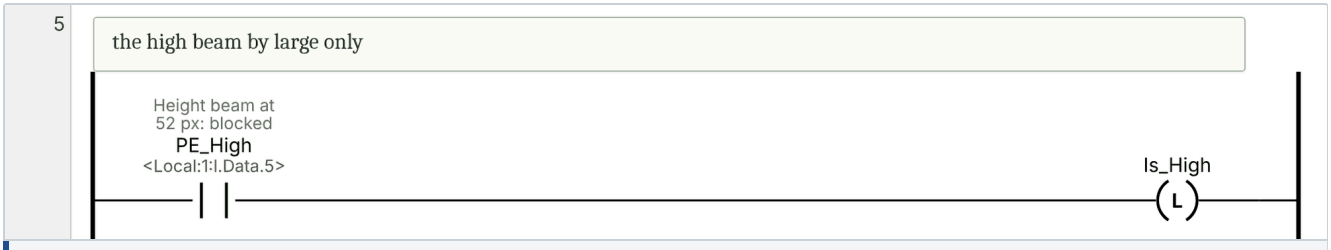

```
XIC(PE_Low)OTL(Is_Low);
```

RUNG 4



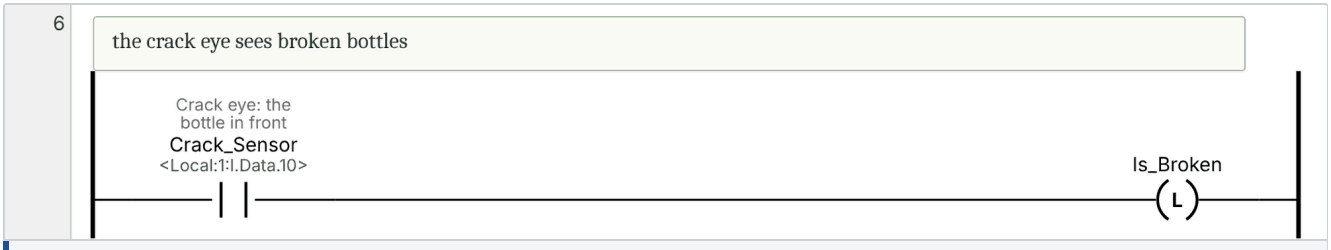

```
XIC(PE_Mid)OTL(Is_Mid);
```

RUNG 5




```
XIC(PE_High)OTL(Is_High);
```

RUNG 6

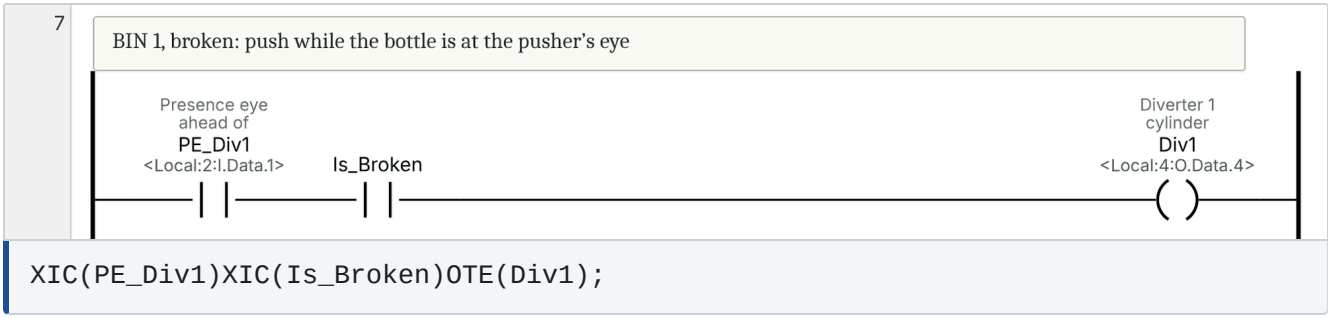



```
XIC(Crack_Sensor)OTL(Is_Broken);
```

Four latches. Each beam is broken only while the bottle passes it — about half a second — but the pushers are metres downstream, so the program latches what it saw. A small bottle sets only `Is_Low`; a medium sets `Is_Low` and `Is_Mid`; a large sets all three; the crack eye sets `Is_Broken`. Nothing clears them except the next release.

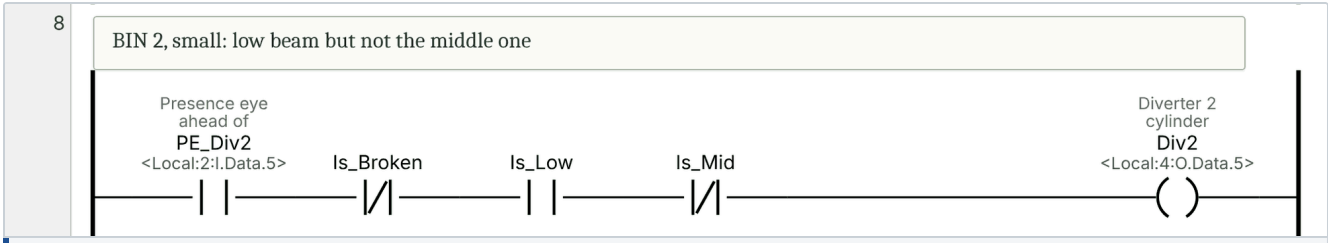
Firing the right pusher

RUNG 7



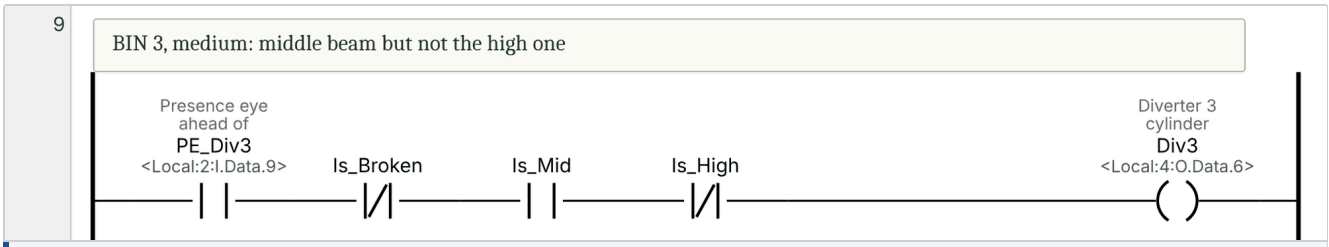

```
XIC(PE_Div1)XIC(Is_Broken)OTE(Div1);
```

RUNG 8



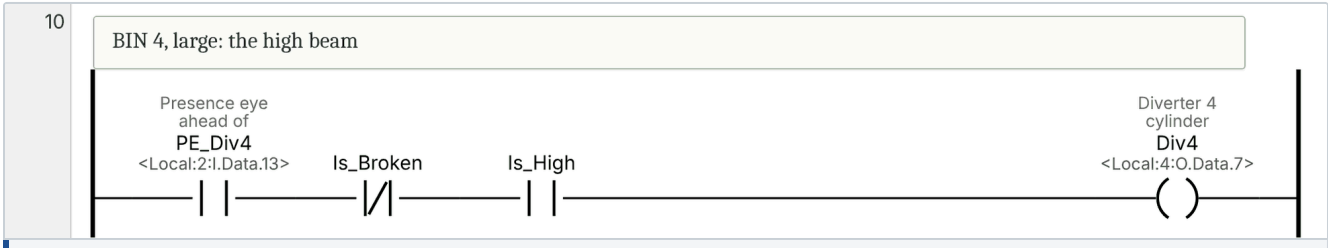

```
XIC(PE_Div2)XIO(Is_Broken)XIC(Is_Low)XIO(Is_Mid)OTE(Div2);
```

RUNG 9




```
XIC(PE_Div3)XIO(Is_Broken)XIC(Is_Mid)XIO(Is_High)OTE(Div3);
```

RUNG 10



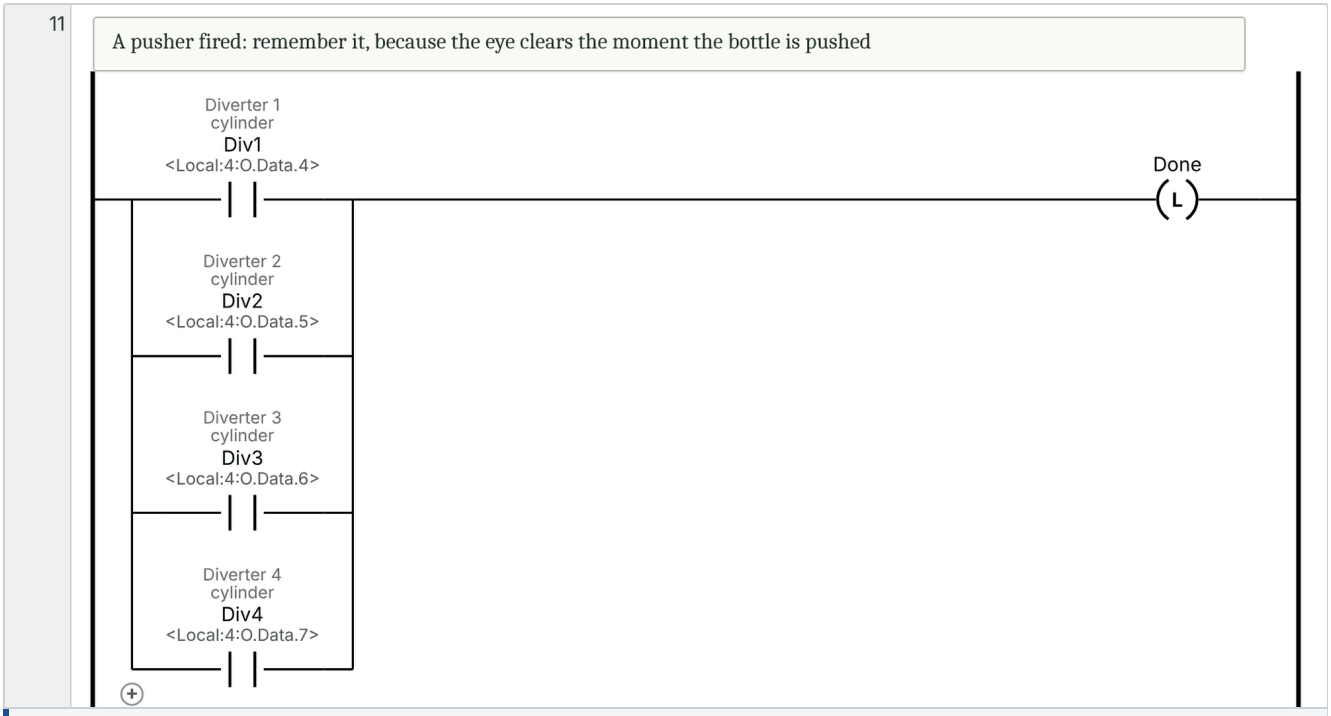

```
XIC(PE_Div4)XIO(Is_Broken)XIC(Is_High)OTE(Div4);
```

Each pusher has its own eye just ahead of the arm; the rung pushes while that eye sees the bottle and the memory says the bottle belongs in this bin. The size tests are written as *this beam and not the next one up*: small is low-not-mid, medium is mid-not-high, large is high. Every whole-bottle bin tests

`XIO(Is_Broken)` first: a broken bottle is broken whatever its size, and bin 1 is upstream, so it has already gone by the time a broken bottle could reach bin 2 — but the test costs nothing and saves you when the bins are reordered.

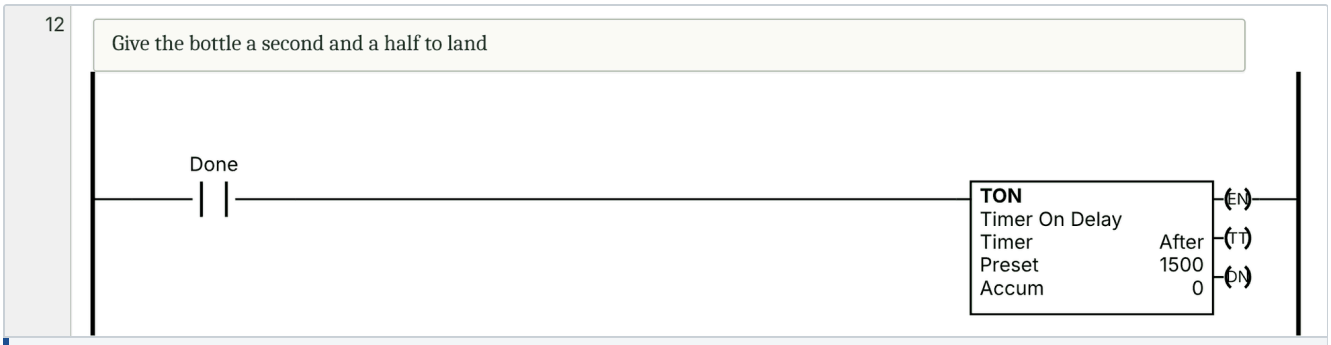
Freeing the line

RUNG 11



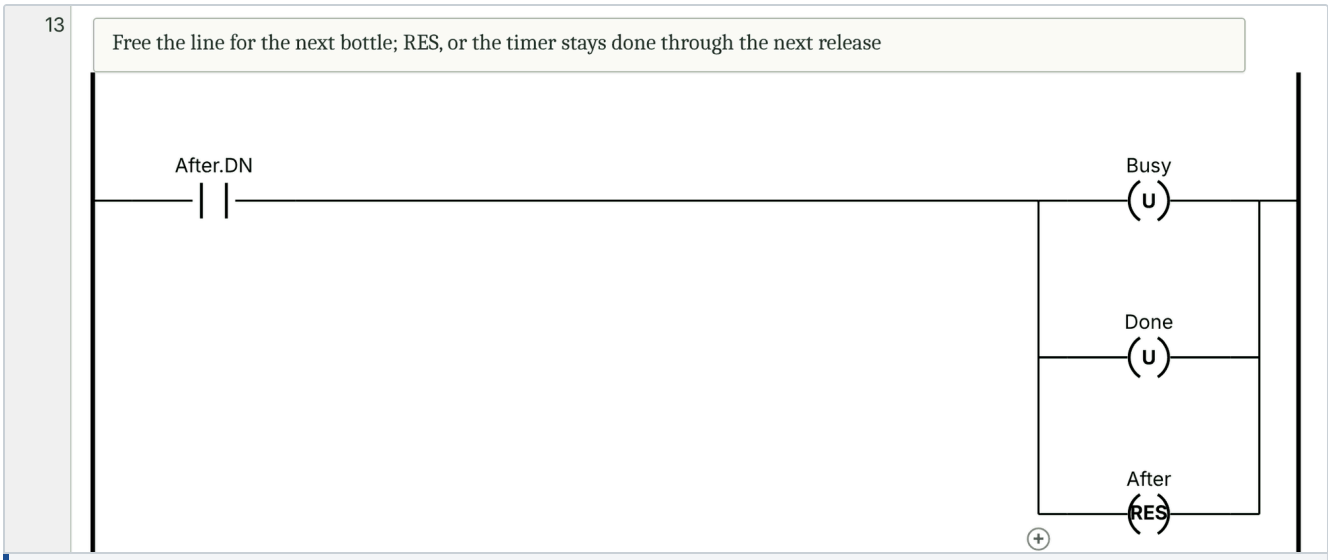

```
[XIC(Div1),XIC(Div2),XIC(Div3),XIC(Div4)]OTL(Done);
```

RUNG 12



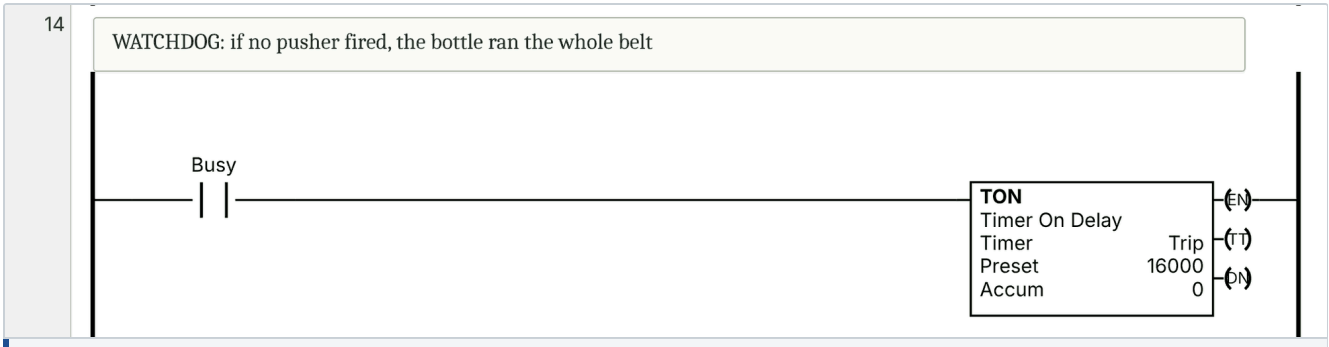

```
XIC(Done)TON(After,1500,0);
```

RUNG 13



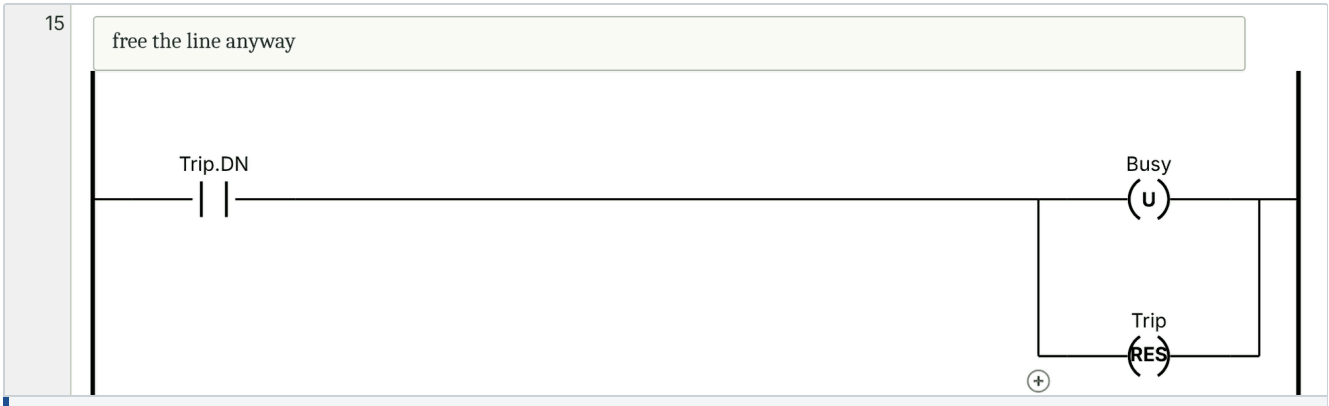

```
XIC(After.DN)[OTU(Busy),OTU(Done),RES(After)];
```

RUNG 14




```
XIC(Busy)TON(Trip,16000,0);
```

RUNG 15




```
XIC(Trip.DN)[OTU(Busy),RES(Trip)];
```

Rung 11 latches `Done` when any pusher fires, because the pusher's eye — and so the pusher rung — goes false the moment the bottle is pushed off the belt. A timer fed straight from `Div2` would see half a second, never 1.5. **Rungs 12-13** give the bottle a second and a half to land and then free the line. **Rungs 14-15** are the watchdog: if no pusher fires — a bottle nobody wanted, or a bug — the bottle runs to the end of the belt and the line is freed after 16 s anyway.

> **WATCH FOR: THE RES**
>
> The first draft of this program had `XIC(After.DN)OTU(Busy);` with no `RES`, and released bottles in pairs. Walk the scan: the timer finishes and rung 13 unlatches Busy. Next scan, rung 2 sees Busy off, releases a bottle and latches Busy again — so rung 12's timer sees its rung *true*, as it was last scan. It never saw the rung go false, so it never reset: it is still done, and rung 13 unlatches Busy again, and the next bottle is released right behind the first. A timer resets only when its rung is false for a whole scan. When the thing that ends a cycle also starts the next one, reset the timer yourself.

Run it

Run mode, press Start on the console. A bottle drops, runs past the gantry (watch the four `Is_` bits in Monitor Tags latch), reaches its pusher and goes into its bin; a moment later the next one drops. The header counts Correct and Wrong; the Simulator Log names every bottle and the bin it landed in. The reference run of this program: 10 of 10 bottles in the right bin, nothing lost, in 94 s against a par of 150 s: 1200 points and three stars.

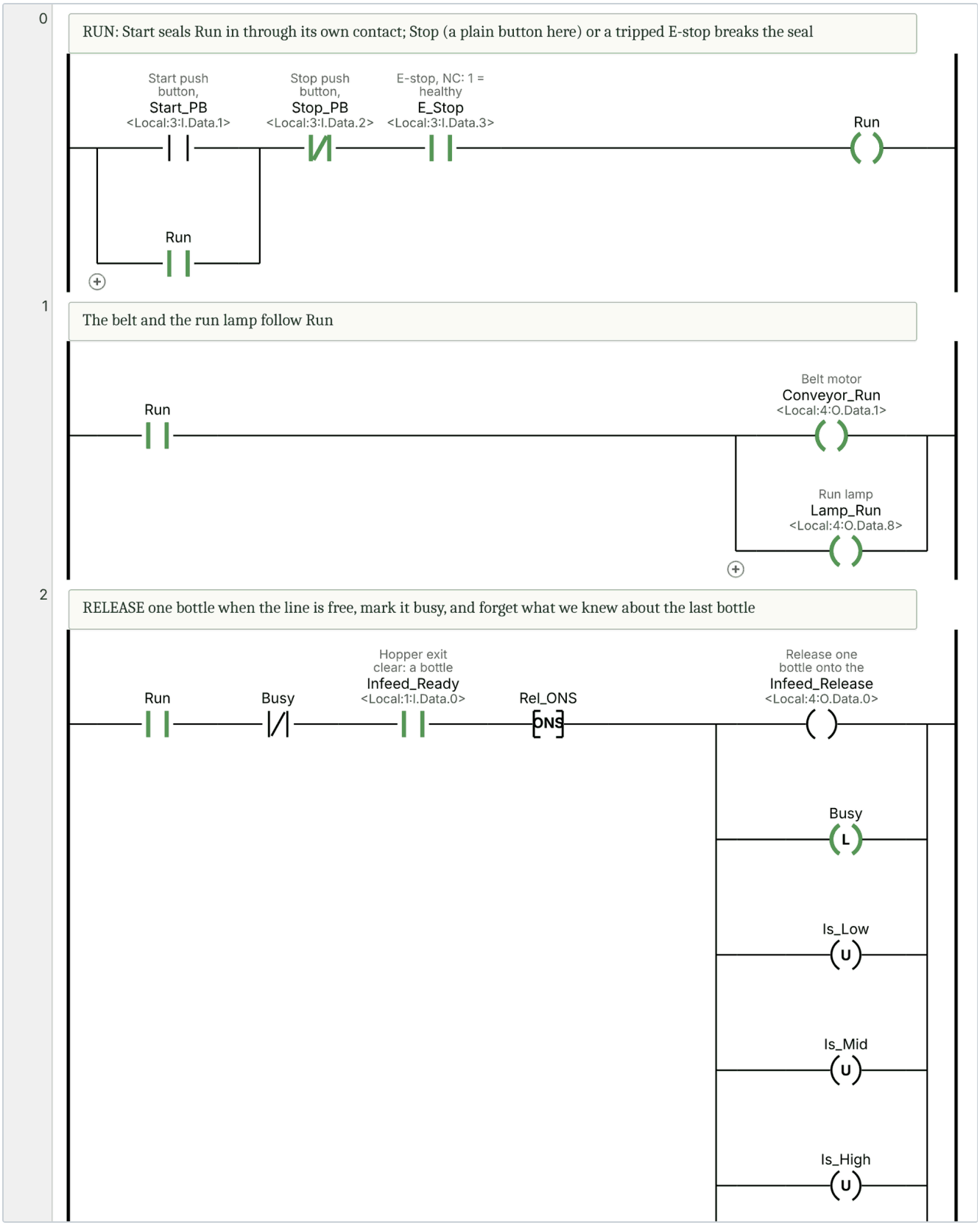


Figure 115. Level 1's program online mid-run: Busy latched, the bottle's height bits latched, the belt running.

TRY IT

Take `XIO(Is_Broken)` out of the small-bin rung and set the hopper's *Broken bottles* to 60 % — then move bin 1 to the far end of the line (swap the pushers' names). A broken small bottle now reaches bin 2 first. Put the test back. Then shorten `After` to 300 ms and see what the next bottle does to a bottle still sliding into its bin.

33 Level 2 — Fill by label

> **IN THE APP**
>
> A level project opens with guided mode on this chapter: the panel lists the tags the program needs (*Make them*), shows every rung below under its fold with *Copy* and *Add*, checks that MainRoutine verifies, and while you run the level ticks three results — the level completes, every bottle in the right bin, under par.

The same bins and the same ten bottles, but every whole bottle must be filled before it is binned: a bin graded *must be filled* gives −40 for a bottle that arrives short or overfilled, and a broken bottle that is filled leaks and costs −40 too. Par 170 s. Everything from Level 1 stays; the new problem is **stopping the belt in the right place, filling to a measured level, and going again**.

Three things on the machine do the work: `PE_Fill`, an eye with a short range placed 20 px past the nozzle, so a bottle that stops the belt on it is standing under the nozzle; `Fill_Valve`, the dispenser's valve; and `Fill_Level`, the fill-level sensor's analog input, the bottle's fill as a percent of its *own* label — 60 units for a small bottle, 80 for a medium, 100 for a large all read 100 when full, so one number fills all three sizes.

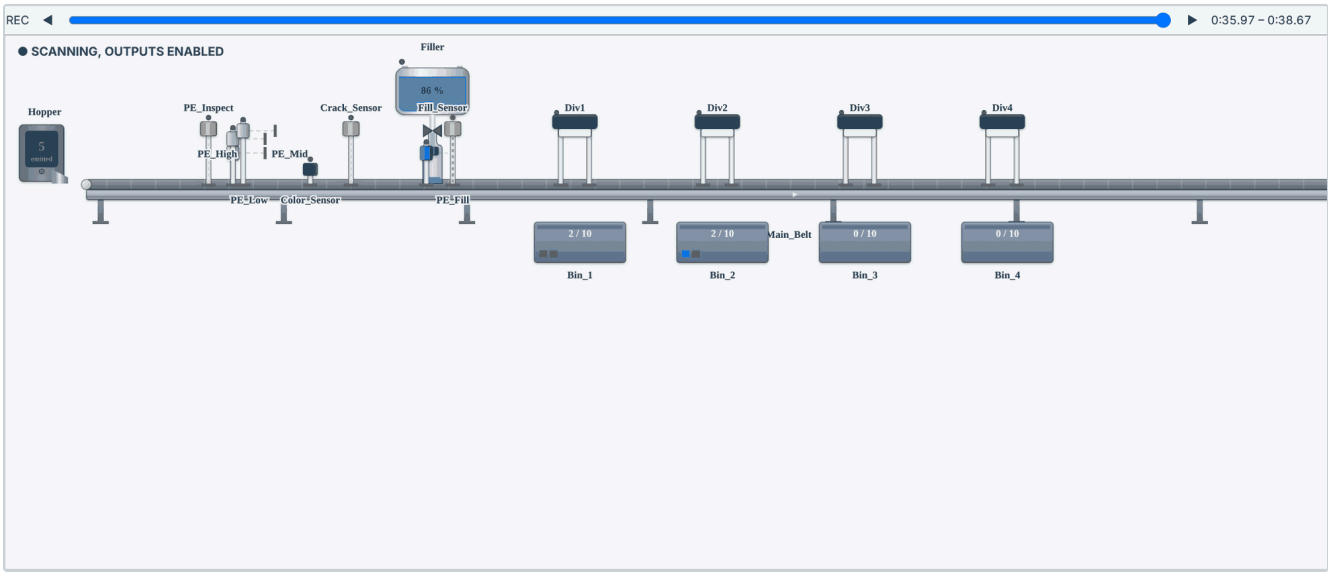


Figure 116. Level 2 in the Side view: a bottle stopped under the nozzle, filling.

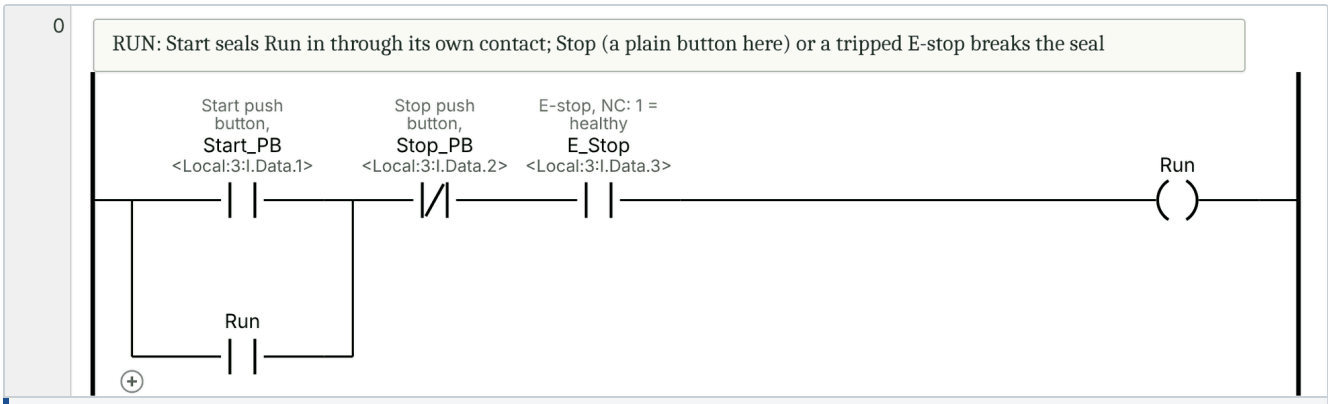
Tags this lesson needs

The level's alias tags are already made for you (chapter 30's table). Create these plain tags before you type the rungs; Verify reports any you miss.

Tag	Type	How to make it	Description
`Run`	BOOL	Edit Tags: name, type	Seal-in run bit
`Hold`	BOOL	Edit Tags: name, type	The filler holds the belt
`Busy`	BOOL	Edit Tags: name, type	A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)
`Rel_ONS`	BOOL	Edit Tags: name, type	One-shot storage for the release
`Is_Low`	BOOL	Edit Tags: name, type	Latched: the low height beam was broken
`Is_Mid`	BOOL	Edit Tags: name, type	Latched: the middle beam was broken
`Is_High`	BOOL	Edit Tags: name, type	Latched: the high beam was broken
`Is_Broken`	BOOL	Edit Tags: name, type	Latched: the crack eye saw a broken bottle
`Filled`	BOOL	Edit Tags: name, type	This bottle has its charge
`Done`	BOOL	Edit Tags: name, type	A pusher has fired for this bottle
`After`	TIMER	Edit Tags: name, type	Landing time after a push
`DN`	BOOL	Edit Tags: name, type	
`Trip`	TIMER	Edit Tags: name, type	Watchdog: the bottle ran the whole belt

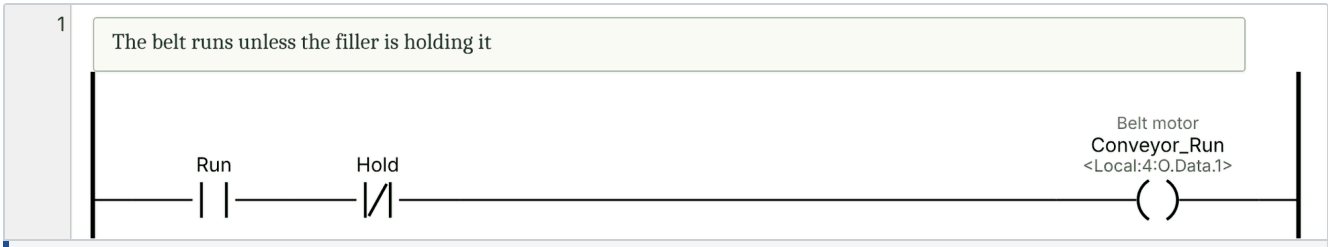
The run circuit, now with a hold

RUNG 0



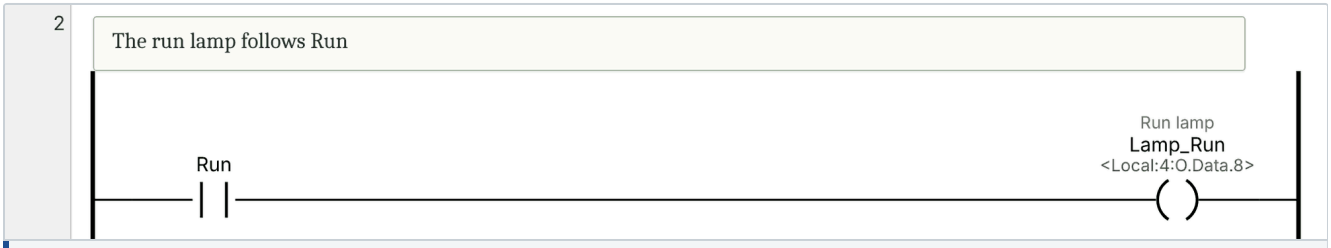

```
[XIC(Start_PB),XIC(Run)]XIO(Stop_PB)XIC(E_Stop)OTE(Run);
```

RUNG 1




```
XIC(Run)XIO(Hold)OTE(Conveyor_Run);
```

RUNG 2




```
XIC(Run)OTE(Lamp_Run);
```

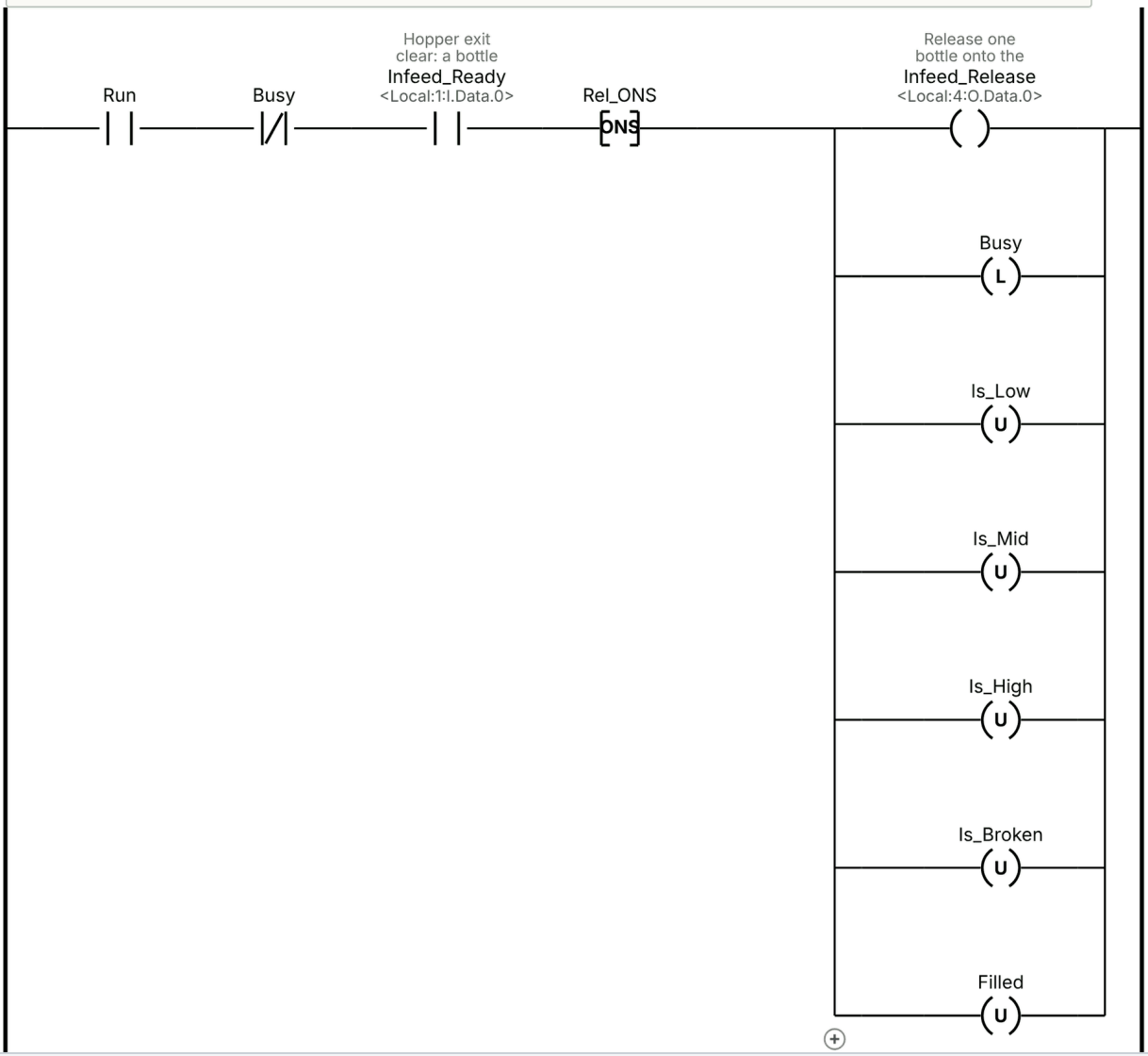
Rung 1 changes: the belt runs unless `Hold` is on. The lamp gets a rung of its own, because it should stay lit while the belt is held for a fill — the line is still running.

Release, gantry, pushers, freeing the line

RUNG 3

3

RELEASE: as before, and the new bottle is not filled yet




```
XIC(Run)XIO(Busy)XIC(Infeed_Ready)ONS(Rel_ONS)
[OTE(Infeed_Release),OTL(Busy),OTU(Is_Low),OTU(Is_Mid),OTU(Is_High),OTU(Is_Brok
en),OTU(Filled)];
```

RUNG 4

4

GANTRY: the low beam is broken by every size

```
XIC(PE_Low)OTL(Is_Low);
```

RUNG 5

5

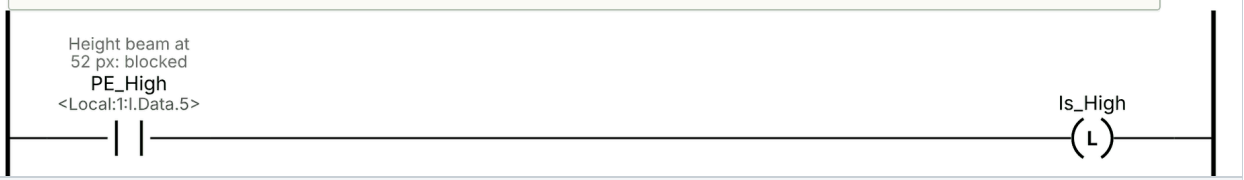
the middle beam by medium and large

```
XIC(PE_Mid)OTL(Is_Mid);
```

RUNG 6

6

the high beam by large only

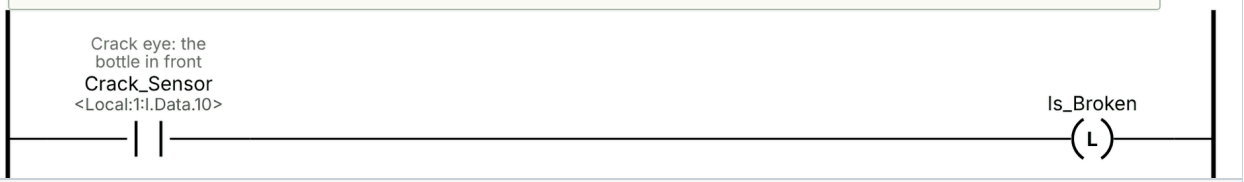



```
XIC(PE_High)OTL(Is_High);
```

RUNG 7

7

the crack eye sees broken bottles




```
XIC(Crack_Sensor)OTL(Is_Broken);
```

RUNG 8

8

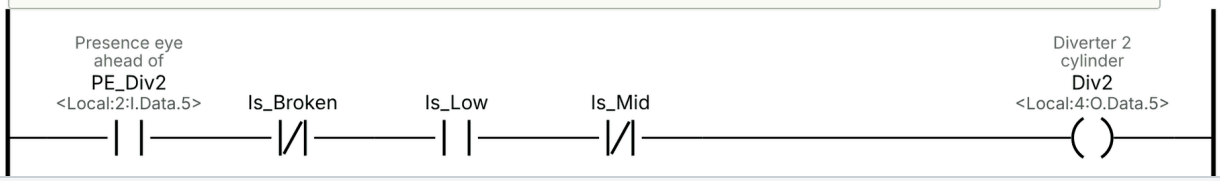
BIN 1, broken: push while the bottle is at the pusher's eye

```
XIC(PE_Div1)XIC(Is_Broken)OTE(Div1);
```

RUNG 9

9

BIN 2, small: low beam but not the middle one

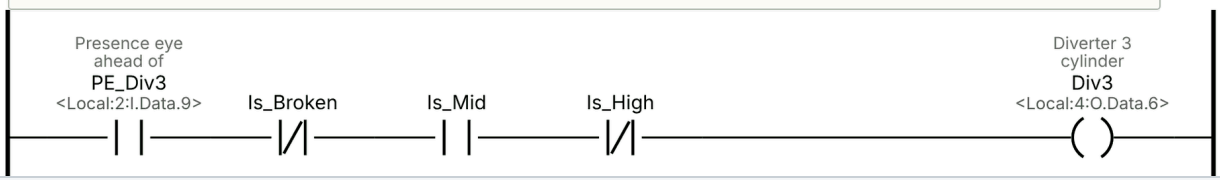



```
XIC(PE_Div2)XIO(Is_Broken)XIC(Is_Low)XIO(Is_Mid)OTE(Div2);
```

RUNG 10

10

BIN 3, medium: middle beam but not the high one

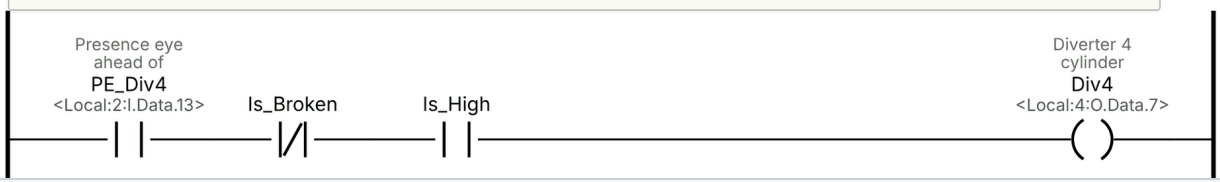



```
XIC(PE_Div3)XIO(Is_Broken)XIC(Is_Mid)XIO(Is_High)OTE(Div3);
```

RUNG 11

11

BIN 4, large: the high beam

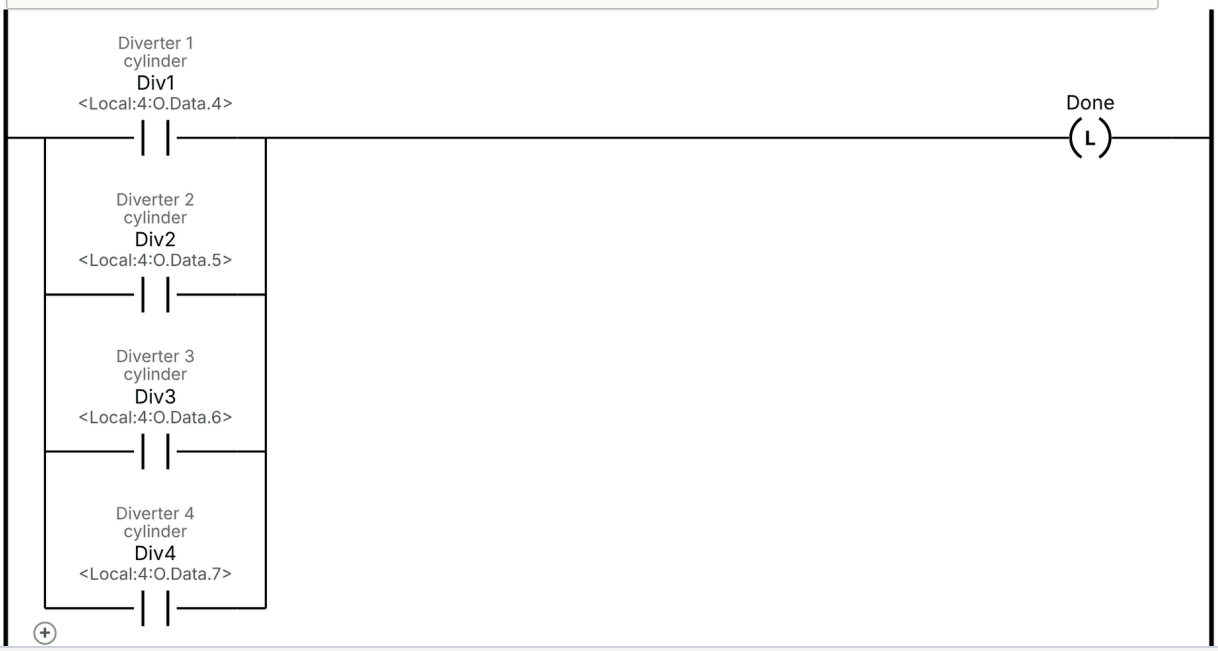



```
XIC(PE_Div4)XIO(Is_Broken)XIC(Is_High)OTE(Div4);
```

RUNG 12

12

A pusher fired: remember it, because the eye clears the moment the bottle is pushed




```
[XIC(Div1),XIC(Div2),XIC(Div3),XIC(Div4)]OTL(Done);
```

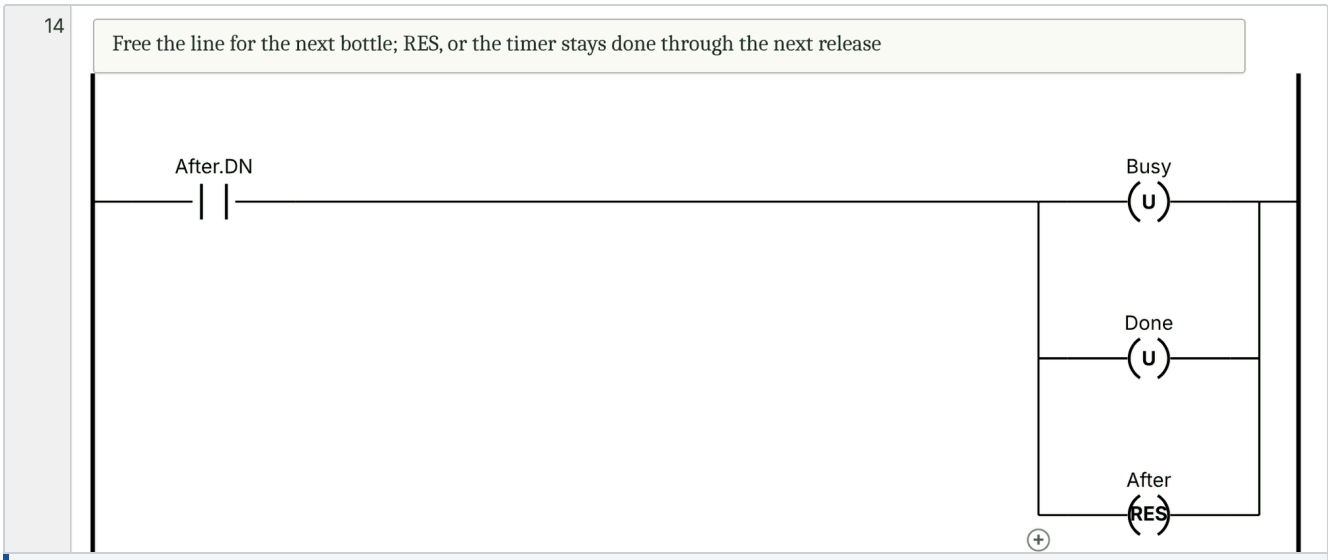
RUNG 13

13

Give the bottle a second and a half to land

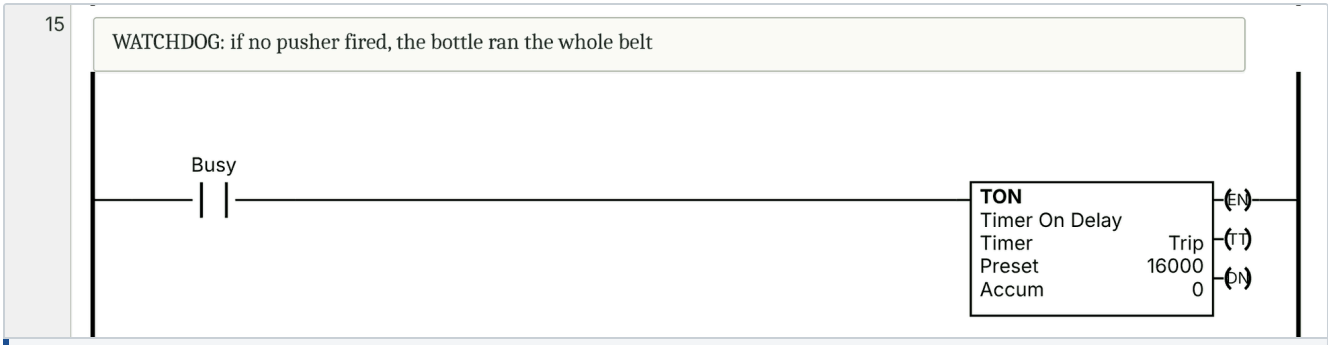
```
XIC(Done)TON(After,1500,0);
```

RUNG 14



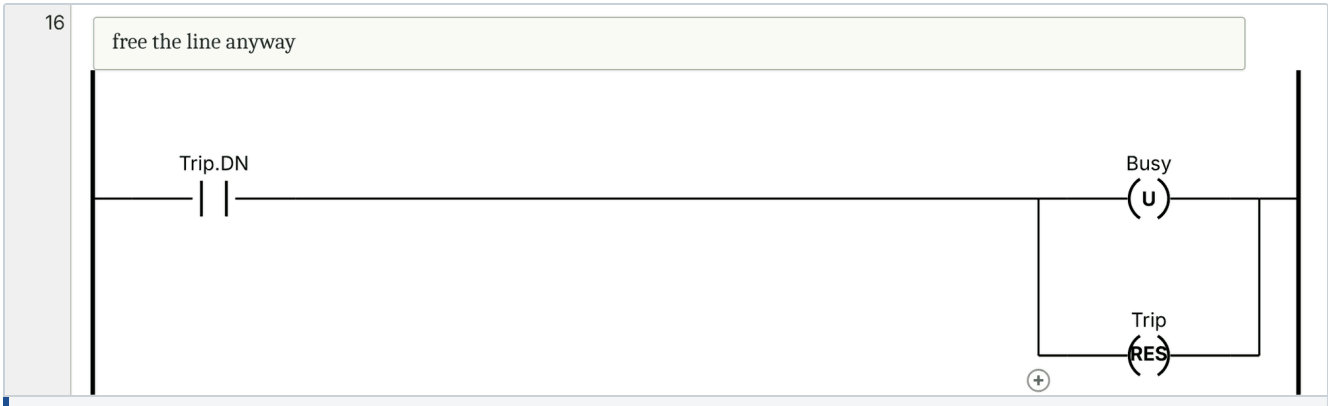

```
XIC(After.DN)[OTU(Busy),OTU(Done),RES(After)];
```

RUNG 15




```
XIC(Busy)TON(Trip,16000,0);
```

RUNG 16

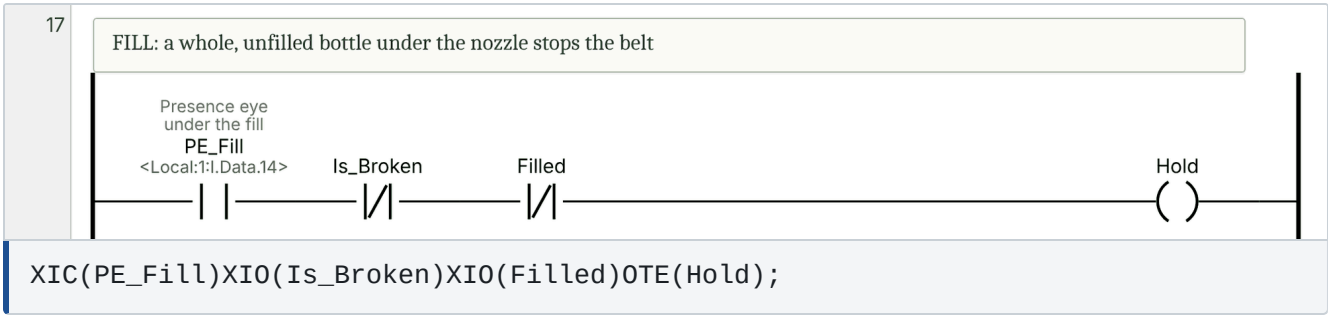



```
XIC(Trip.DN)[OTU(Busy),RES(Trip)];
```

As in Level 1, with one addition to the release rung: `OTU(Filled)`, so the new bottle starts unfilled.

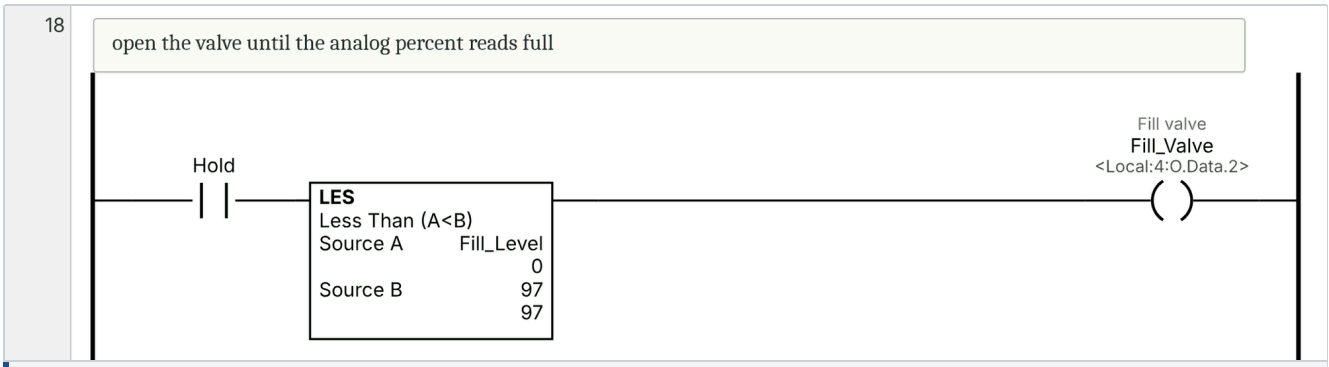
The fill

RUNG 17



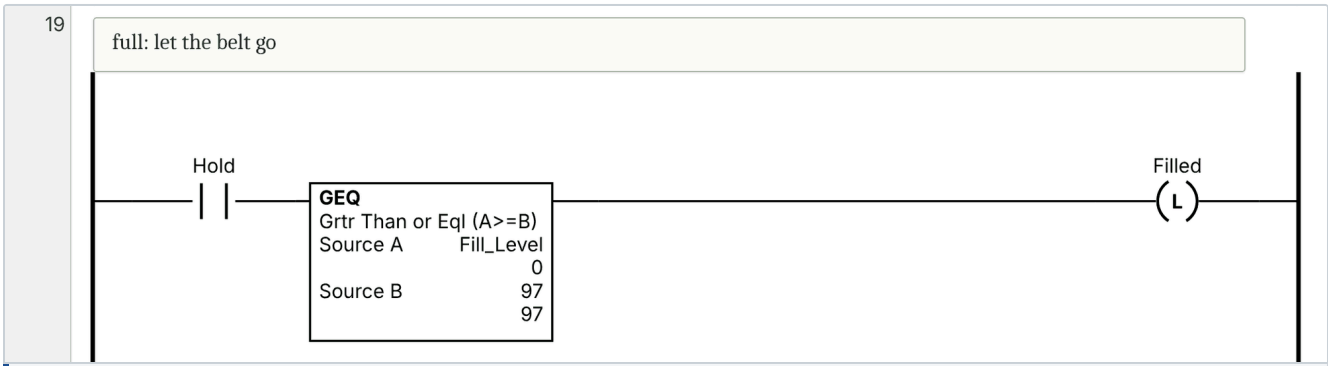

```
XIC(PE_Fill)XIO(Is_Broken)XIO(Filled)OTE(Hold);
```

RUNG 18




```
XIC(Hold)LES(Fill_Level,97)OTE(Fill_Valve);
```

RUNG 19




```
XIC(Hold)GEQ(Fill_Level,97)OTL(Filled);
```

Rung 17 holds the belt when `PE_Fill` sees a bottle that is whole and not yet filled. It is an OTE, so the hold lasts exactly as long as those three things are true. **Rung 18** opens the valve while held and the percent is under 97 — an LES compare on an analog input, chapter 8. **Rung 19** latches `Filled` once it reaches 97, which drops the hold (rung 17's `XIO(Filled)`), which restarts the belt. A broken bottle is never held: `XIO(Is_Broken)` lets it ride through to bin 1 dry.

Why 97 and not 100? The valve closes on the scan *after* the compare goes true, and liquid is still in the air. Stop a fraction short and the last of it brings the bottle to its label; aim at 100 and you overfill. Every real filler has this *in-flight* allowance.

> **WATCH FOR: WHERE THE EYE IS**
>
> The first draft of this program used an eye 8 px before the nozzle with a long range. The belt stopped with the bottle short of the nozzle, the valve opened, and the dispenser's tank emptied onto the floor while `Fill_Level` sat at 0 — the sensor reads the bottle under *it*, and there was none. The level's `PE_Fill` sits past the nozzle with a 6 px range for that reason. Before you trust any stop position, watch it once in the Side view.

Run it

Each whole bottle stops under the nozzle, the liquid rises in the Side view (watch `Fill_Level` climb in Monitor Tags), the belt goes again, and the bottle is sorted as in Level 1. Broken bottles pass the filler without stopping. The reference run: 10 of 10 bottles in the right bin, nothing lost, in 104 s against a par of 170 s: 1200 points and three stars.

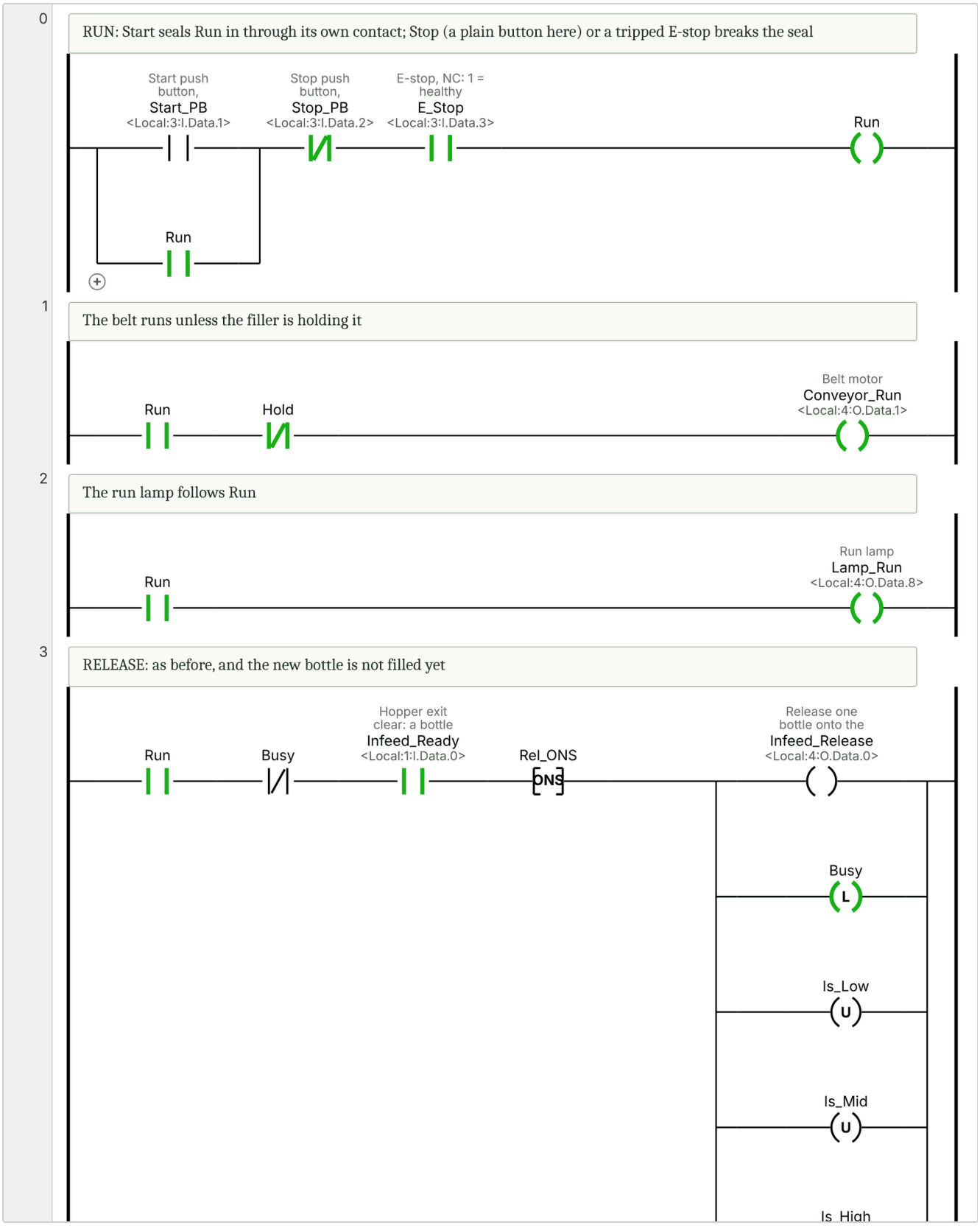


Figure 117. Level 2 online: the hold, the valve and the fill latch are the new rungs at the bottom.

TRY IT

Change 97 to 100 and run: the score card shows −40s for overfilled bottles. Then try 90 and see the short fills. Then take `XIO(Is_Broken)` out of the hold rung and watch a broken bottle filled and leaking into bin 1.

34 Level 3 — Colour sort

> **IN THE APP**
>
> A level project opens with guided mode on this chapter: the panel lists the tags the program needs (*Make them*), shows every rung below under its fold with *Copy* and *Add*, checks that MainRoutine verifies, and while you run the level ticks three results — the level completes, every bottle in the right bin, under par.

Twelve bottles, bins by **colour** now — broken, red, blue, green — and every whole bottle filled. Par 190 s. The machine is the same; only the question the gantry answers changes, and this lesson is about how little of a good program has to change when the question does.

The colour sensor has three banks, one per colour, each on its own input: `Color_Red`, `Color_Blue`, `Color_Green`. Like the height beams, a bank is on only while the bottle is in front of the sensor, so the pattern is the same: latch it, test the latch at the pusher.

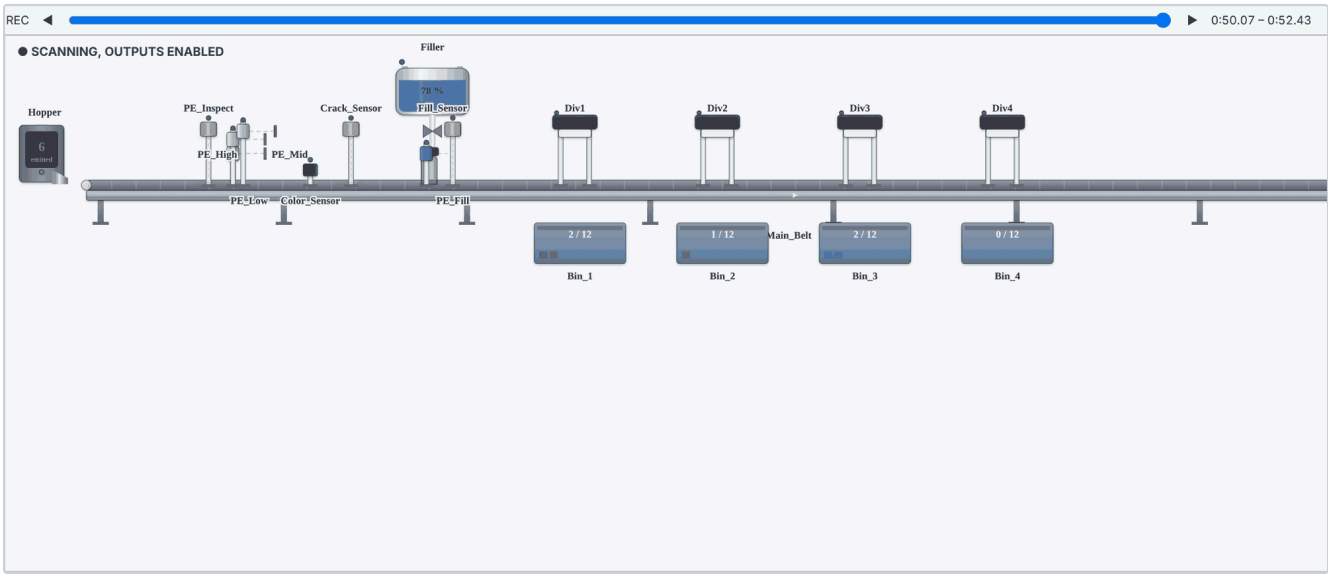


Figure 118. Level 3 running: the colour sensor on the gantry, bins graded by colour.

Tags this lesson needs

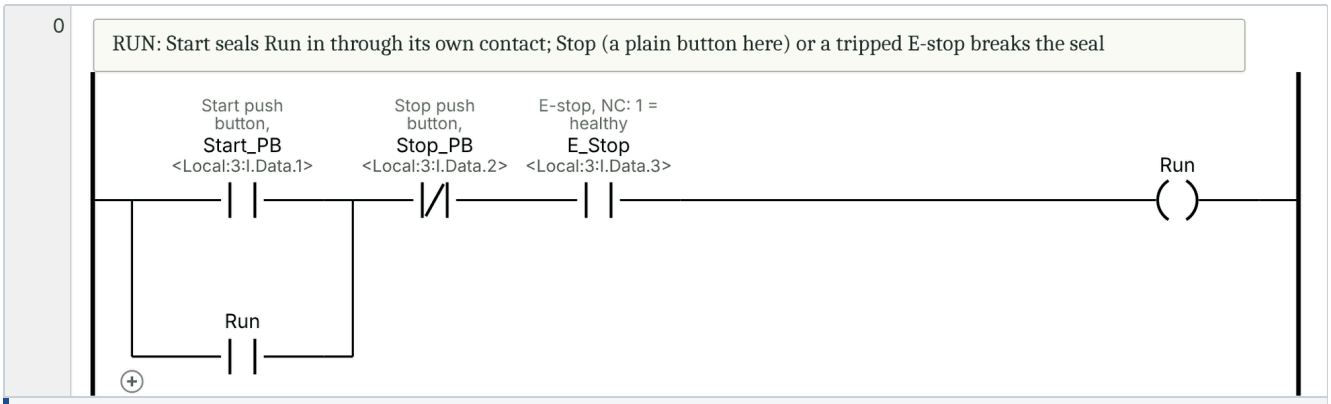
The level's alias tags are already made for you (chapter 30's table). Create these plain tags before you type the rungs; Verify reports any you miss.

Tag	Type	How to make it	Description
`Run`	BOOL	Edit Tags: name, type	Seal-in run bit

Tag	Type	How to make it	Description
`Hold`	BOOL	Edit Tags: name, type	The filler holds the belt
`Busy`	BOOL	Edit Tags: name, type	A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)
`Rel_ONS`	BOOL	Edit Tags: name, type	One-shot storage for the release
`Is_R`	BOOL	Edit Tags: name, type	Latched: the colour sensor saw red
`Is_B`	BOOL	Edit Tags: name, type	Latched: the colour sensor saw blue
`Is_G`	BOOL	Edit Tags: name, type	Latched: the colour sensor saw green
`Is_Broken`	BOOL	Edit Tags: name, type	Latched: the crack eye saw a broken bottle
`Filled`	BOOL	Edit Tags: name, type	This bottle has its charge
`Done`	BOOL	Edit Tags: name, type	A pusher has fired for this bottle
`After`	TIMER	Edit Tags: name, type	Landing time after a push
`DN`	BOOL	Edit Tags: name, type	
`Trip`	TIMER	Edit Tags: name, type	Watchdog: the bottle ran the whole belt

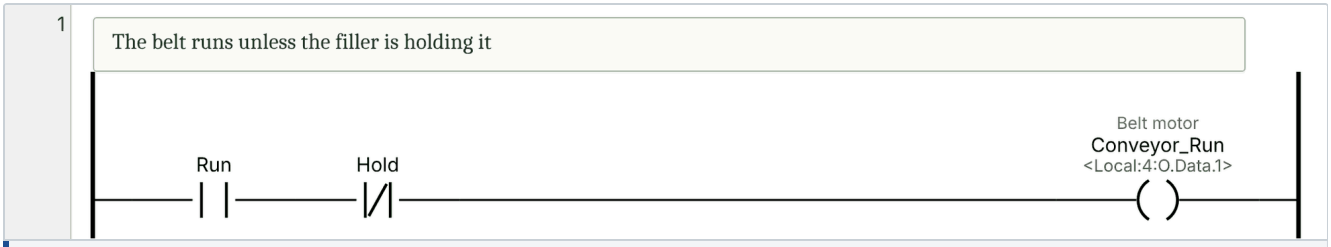
The run circuit and the release

RUNG 0



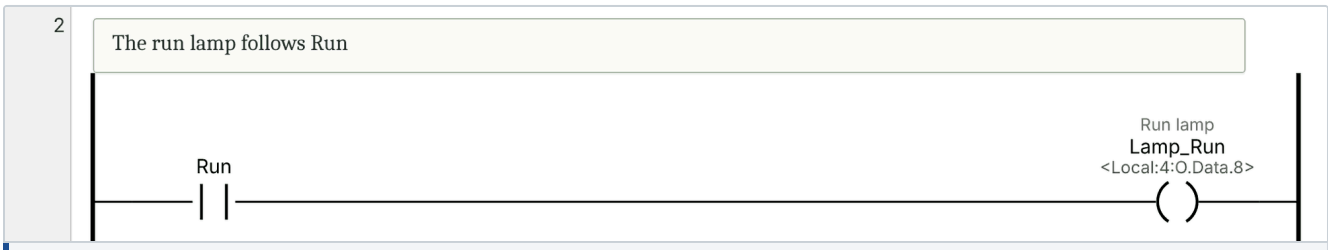

```
[XIC(Start_PB),XIC(Run)]XIO(Stop_PB)XIC(E_Stop)OTE(Run);
```

RUNG 1



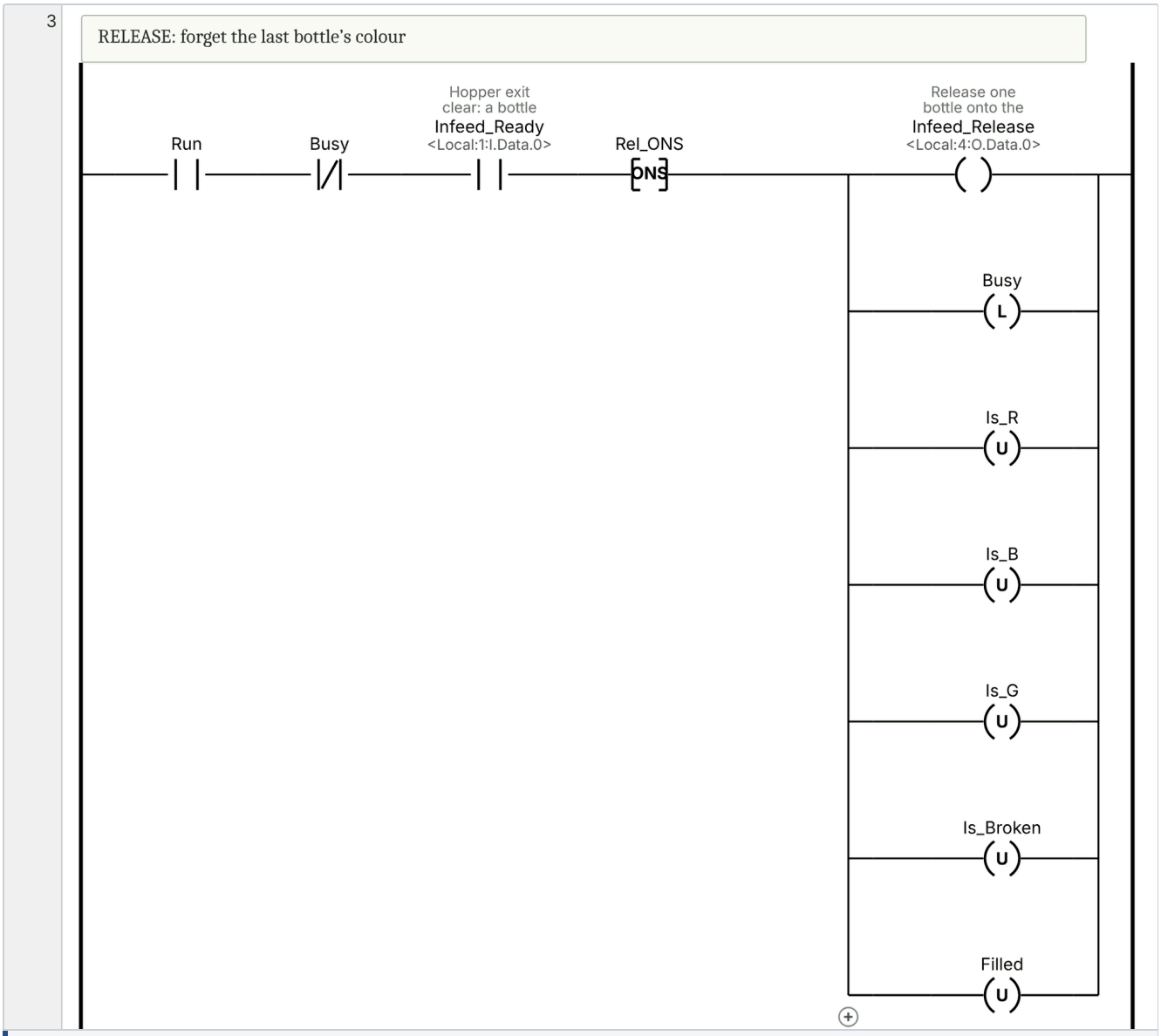

```
XIC(Run)XIO(Hold)OTE(Conveyor_Run);
```

RUNG 2




```
XIC(Run)OTE(Lamp_Run);
```

RUNG 3

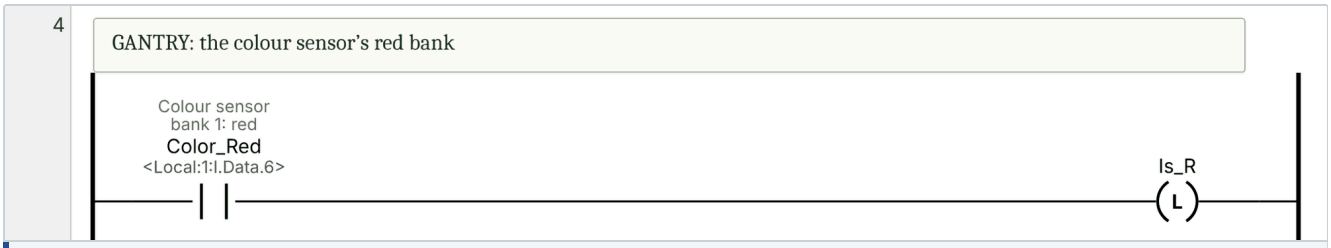



```
XIC(Run)XIO(Busy)XIC(Infeed_Ready)ONS(Rel_ONS)
[OTE(Infeed_Release),OTL(Busy),OTU(Is_R),OTU(Is_B),OTU(Is_G),OTU(Is_Broken),OTU
(Filled)];
```

The release now unlatches the colour bits instead of the height bits.

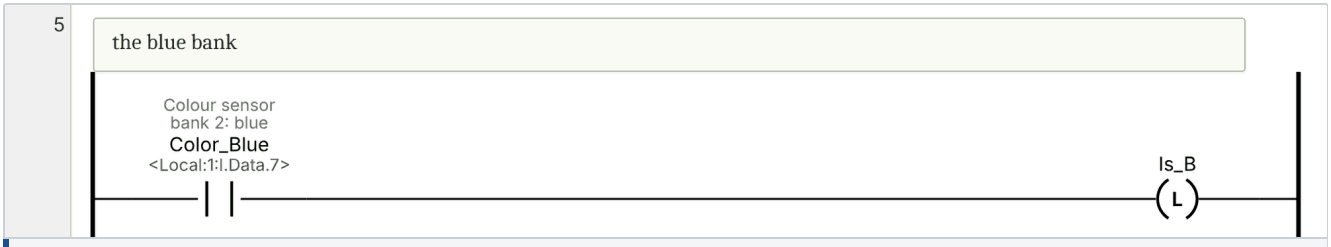
Latching the colour

RUNG 4



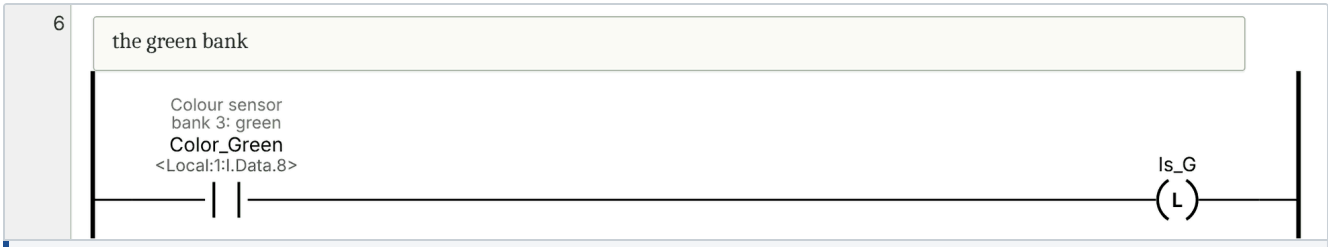

```
XIC(Color_Red)OTL(Is_R);
```

RUNG 5



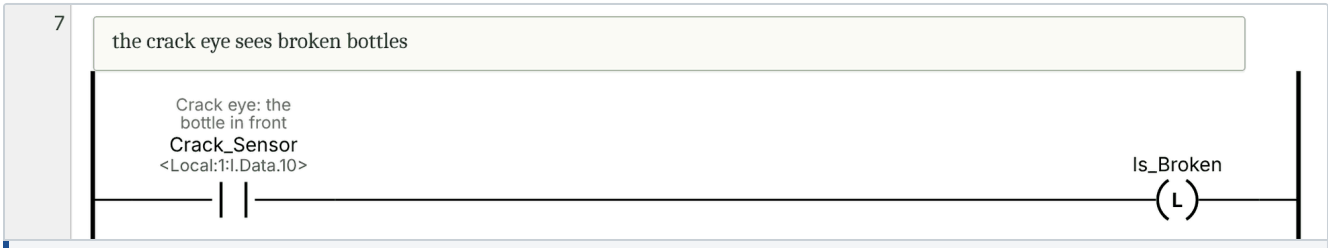

```
XIC(Color_Blue)OTL(Is_B);
```

RUNG 6




```
XIC(Color_Green)OTL(Is_G);
```

RUNG 7

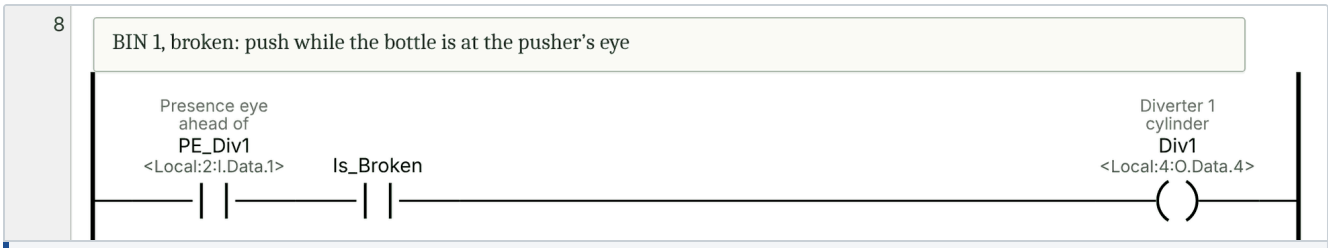



```
XIC(Crack_Sensor)OTL(Is_Broken);
```

Three latches replace the three height latches; the crack eye stays.

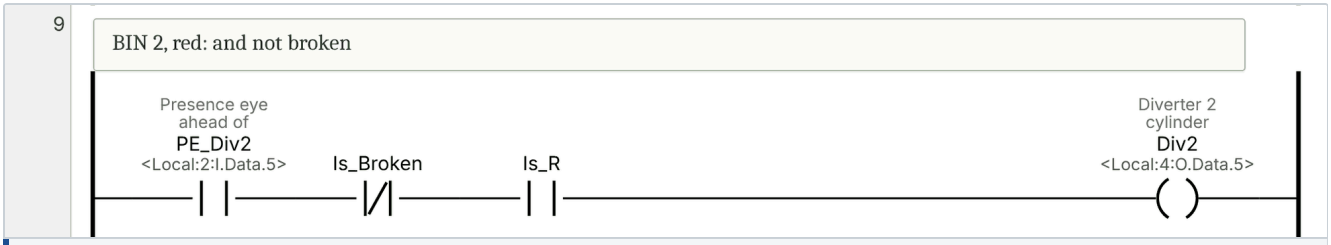
The pushers

RUNG 8



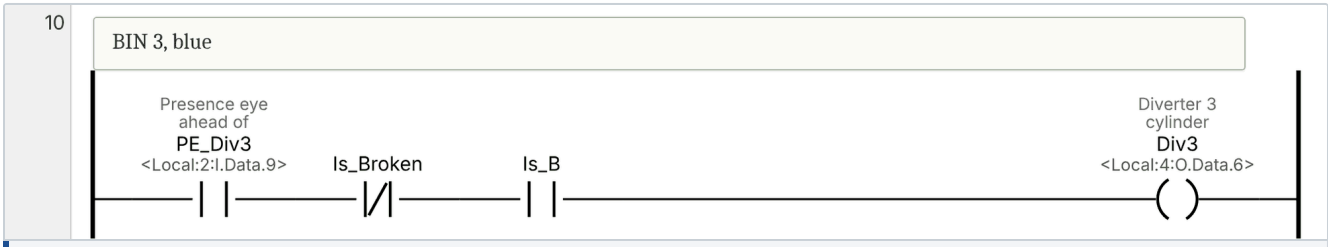

```
XIC(PE_Div1)XIC(Is_Broken)OTE(Div1);
```

RUNG 9



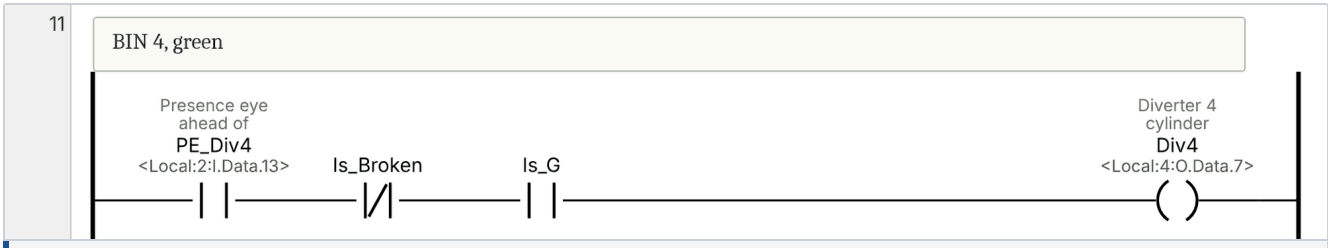

```
XIC(PE_Div2)XIO(Is_Broken)XIC(Is_R)OTE(Div2);
```

RUNG 10




```
XIC(PE_Div3)XIO(Is_Broken)XIC(Is_B)OTE(Div3);
```

RUNG 11



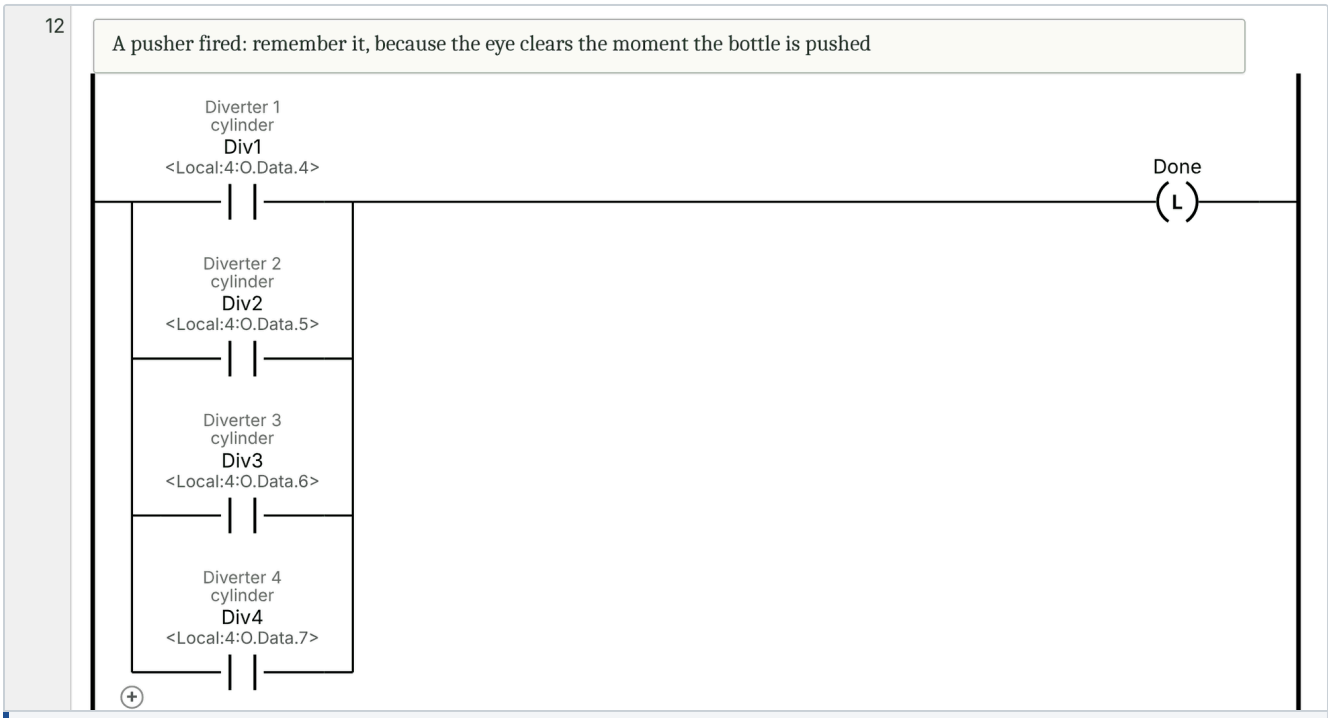

```
XIC(PE_Div4)XIO(Is_Broken)XIC(Is_G)OTE(Div4);
```

Bin 1 is still broken bottles; bins 2, 3 and 4 test one colour each.

> **WATCH FOR: A BROKEN BOTTLE HAS A COLOUR TOO**
>
> The colour sensor does not know a bottle is broken; a broken red bottle turns `Color_Red` on like any other. If the red bin's rung tested only `Is_R`, a broken red bottle that somehow got past bin 1 would score −50 in bin 2. Every whole-bottle rung keeps `XIO(Is_Broken)`.

Freeing the line and the fill

RUNG 12

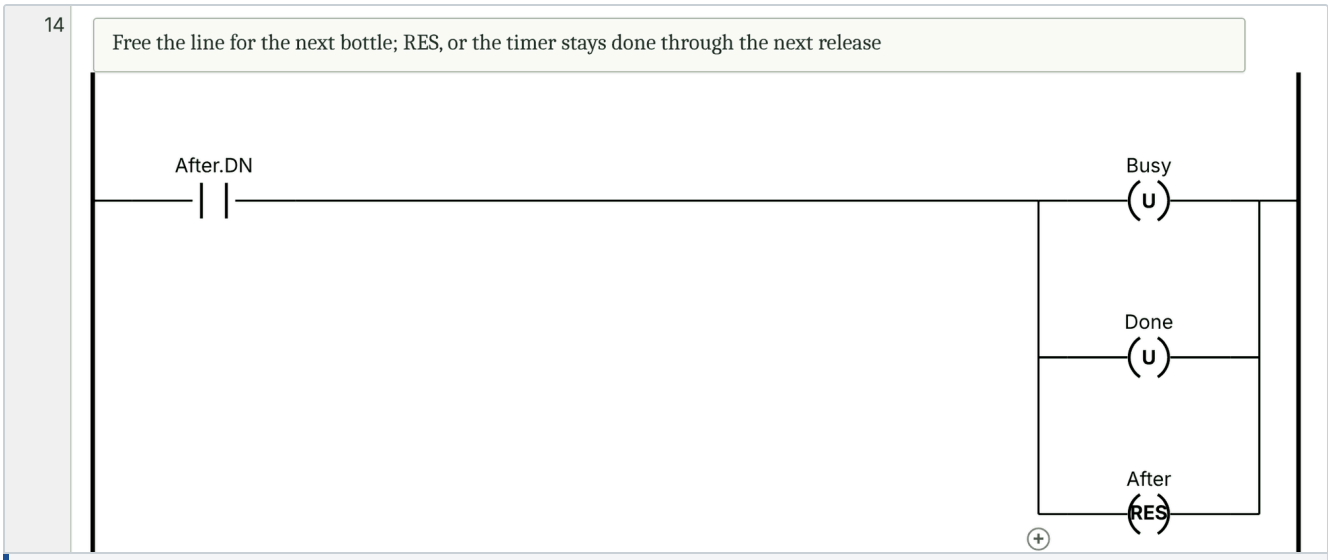



```
[XIC(Div1),XIC(Div2),XIC(Div3),XIC(Div4)]OTL(Done);
```

RUNG 13

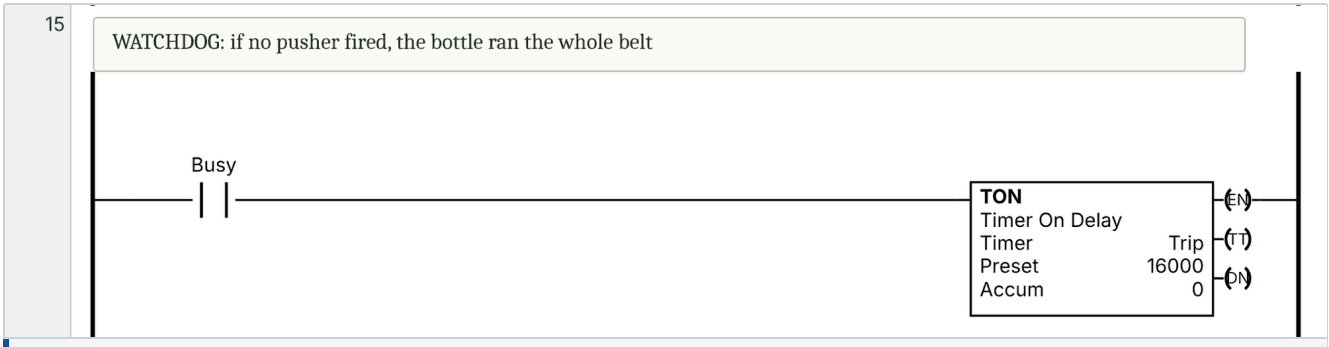
```
XIC(Done)TON(After,1500,0);
```

RUNG 14



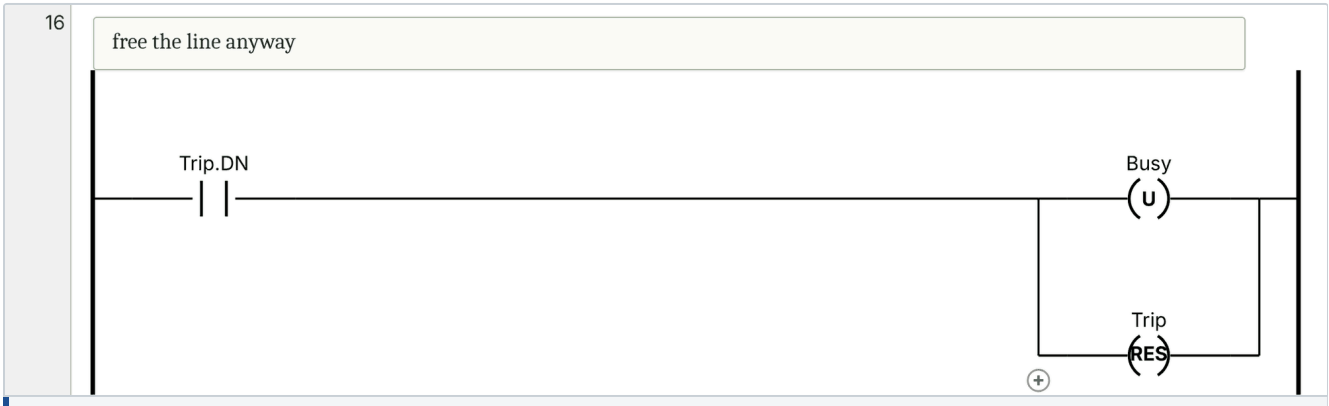

```
XIC(After.DN)[OTU(Busy),OTU(Done),RES(After)];
```

RUNG 15



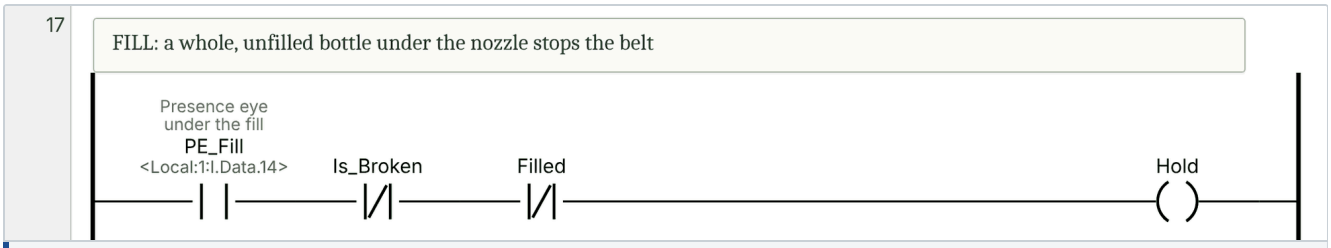

```
XIC(Busy)TON(Trip,16000,0);
```

RUNG 16



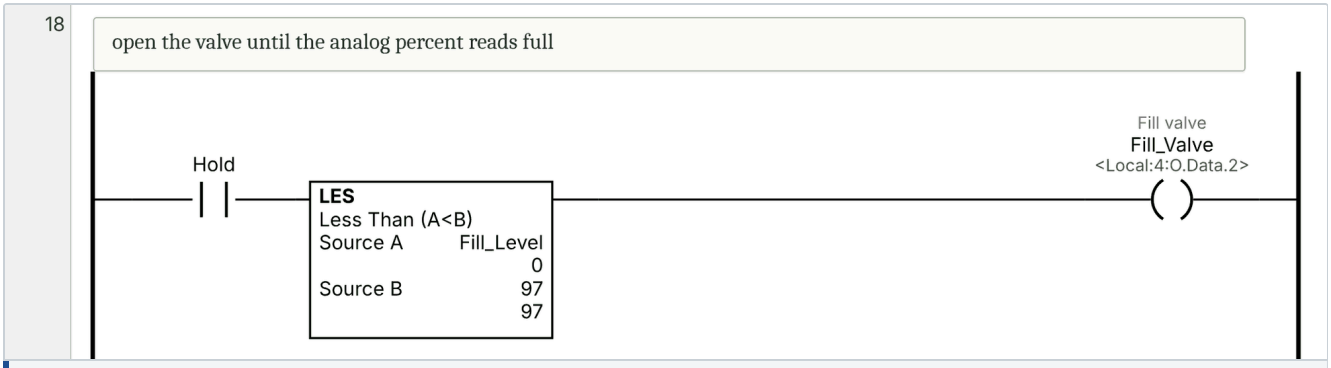

```
XIC(Trip.DN)[OTU(Busy),RES(Trip)];
```

RUNG 17



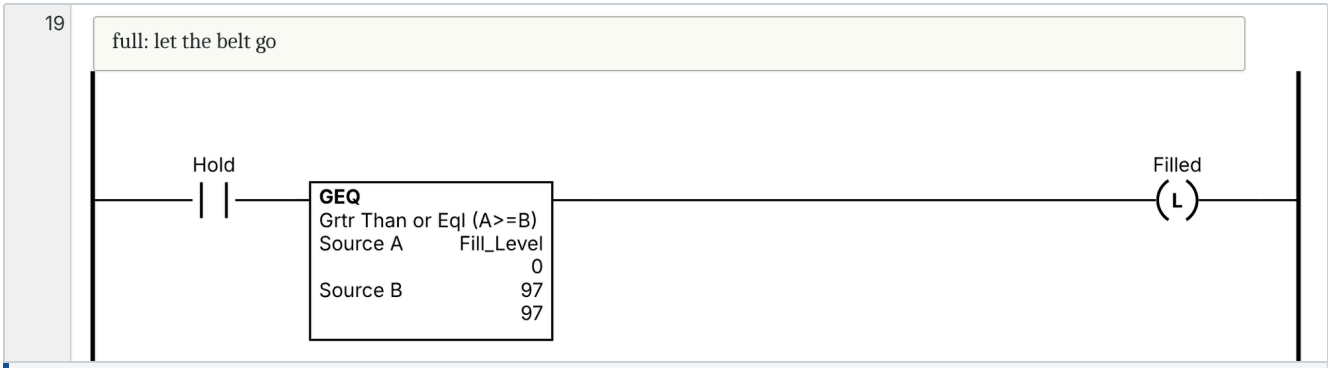

```
XIC(PE_Fill)XIO(Is_Broken)XIO(Filled)OTE(Hold);
```

RUNG 18




```
XIC(Hold)LES(Fill_Level,97)OTE(Fill_Valve);
```

RUNG 19




```
XIC(Hold)GEQ(Fill_Level,97)OTL(Filled);
```

Unchanged from Level 2: the landing timer, the watchdog, the hold and the fill.

Run it

Twelve bottles, sorted red, blue and green, broken ones to bin 1, every whole one filled. The reference run: 12 of 12 bottles in the right bin, nothing lost, in 125 s against a par of 190 s: 1400 points and three stars. Look at the time: one bottle at a time is now taking most of two minutes, and the next level asks for fourteen bottles in 110 seconds.

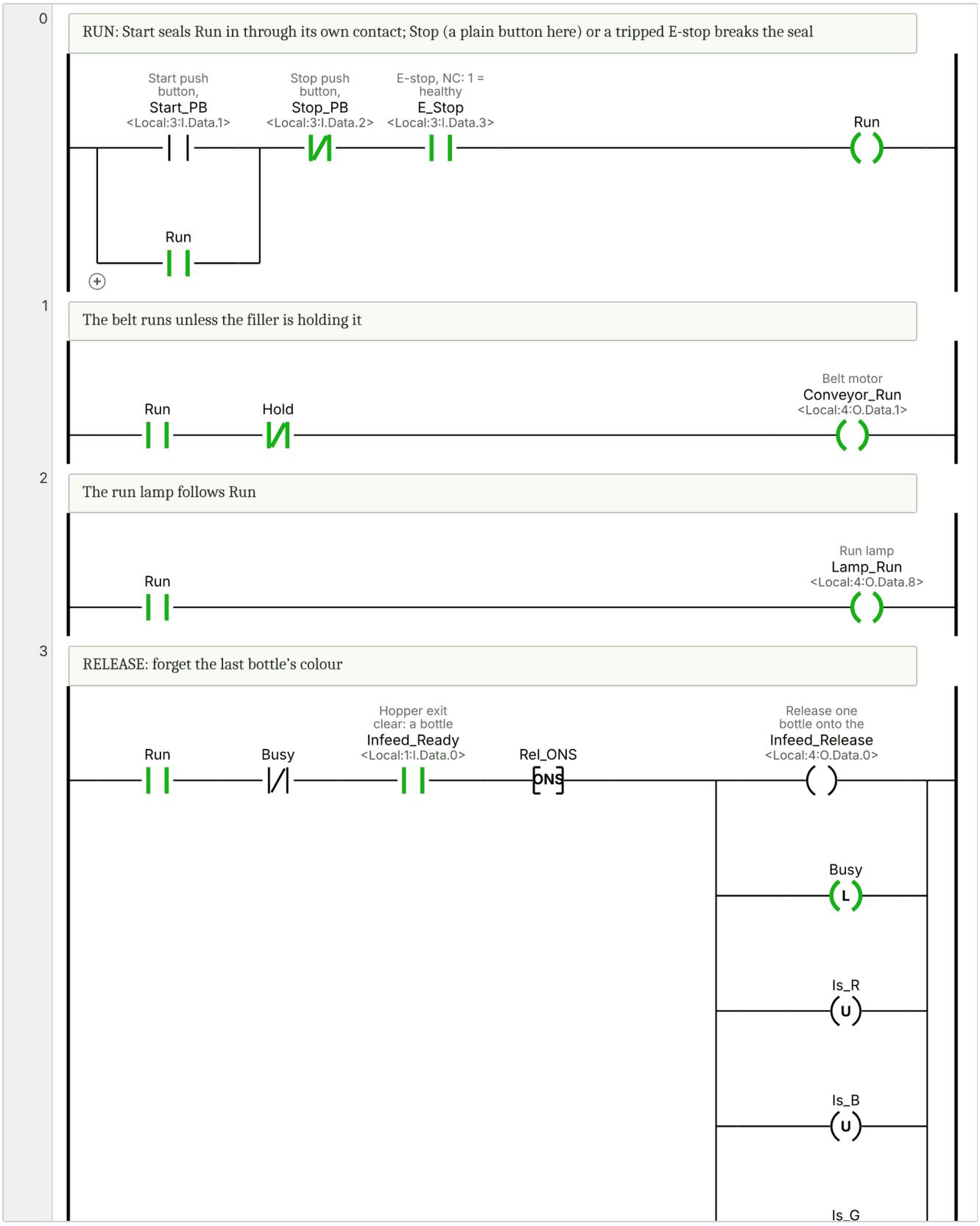


Figure 119. Level 3 online: the colour latches in place of the height latches.

TRY IT

Set the hopper's colour mix to *random* instead of *random (R/B/G)*: some bottles are now yellow. Nobody wants them, so they ride to the end of the belt and the watchdog frees the line after 16 s. Count what that costs against par, then write a rung that frees the line as soon as `PE_LineEnd` sees the bottle.

35 Level 4 — Two on the line

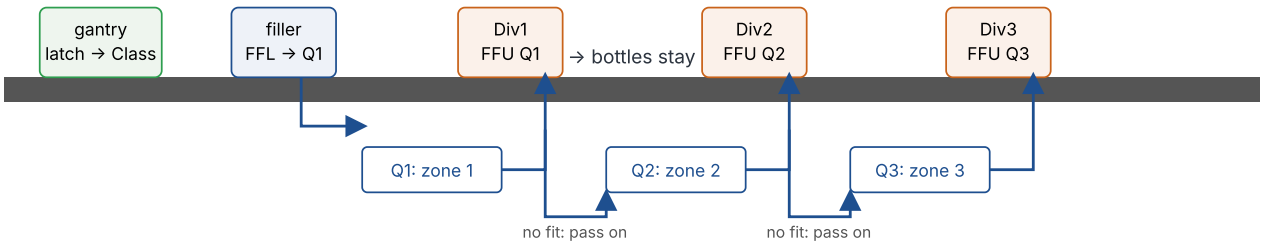
> **IN THE APP**
>
> A level project opens with guided mode on this chapter: the panel lists the tags the program needs (*Make them*), shows every rung below under its fold with *Copy* and *Add*, checks that MainRoutine verifies, and while you run the level ticks three results — the level completes, every bottle in the right bin, under par.

Height sort with fill again — Level 2's bins — but fourteen bottles and a par of 110 seconds. Level 2's program takes about 10 s a bottle; fourteen bottles is over two minutes. No setting fixes that. **The trap is the program's shape**: as long as the next bottle waits for the last one to reach its bin, the belt runs mostly empty. The next bottle has to leave the hopper while the last one is still on the belt, and as soon as there are two bottles on the belt, one set of `Is_` bits is not enough memory: by the time the first bottle reaches its pusher, the gantry has overwritten what it saw.

The idea: a queue per stretch of belt

On the indexing line of Part IV, the memory that moved with the bottles was a shift register, one bit per stop, shifted once per index. This belt does not index — bottles arrive at the pushers whenever they arrive — but it keeps them **in order**: nothing on a belt overtakes anything else. Memory that keeps things in order is a **FIFO**, first in, first out. So:

- Keep one bottle at a time only in the *front* of the line — hopper to filler — where the gantry has to latch what it sees. Release the next bottle the moment the last one leaves the filler.
- Fold what the gantry saw into one number, the **class**: bit 0 small, 1 medium, 2 large, 3 broken (and 4-6 for colour, used in Level 5).
- When a bottle leaves the filler, load its class into **zone 1's queue** — the stretch between the filler and diverter 1.
- When a bottle reaches diverter *n*'s eye, **unload** the head of zone *n*'s queue: that is this bottle's class, because it is the oldest bottle in that stretch. If the class fits the bin, push. If not, the bottle rides on into zone *n*+1, so **load its class into zone *n*+1's queue**.



Each diverter unloads the head of its own zone's queue when a bottle reaches its eye: fit → push; no fit → load the class into the next zone's queue

Figure 120. Zone queues. The class of every bottle travels through FIFOs in step with the bottle itself — a shift register for a belt that does not index.

`FFL(source,array,control,length)` — FIFO Load — copies *source* to the tail of the queue in *array* on a rising edge of its rung; `FFU(array,dest,control,length)` — FIFO Unload — copies the head into *dest* and moves everything up. The CONTROL tag's `.POS` says how many are in the queue (chapter 5). Eight entries per zone is far more than this belt holds.

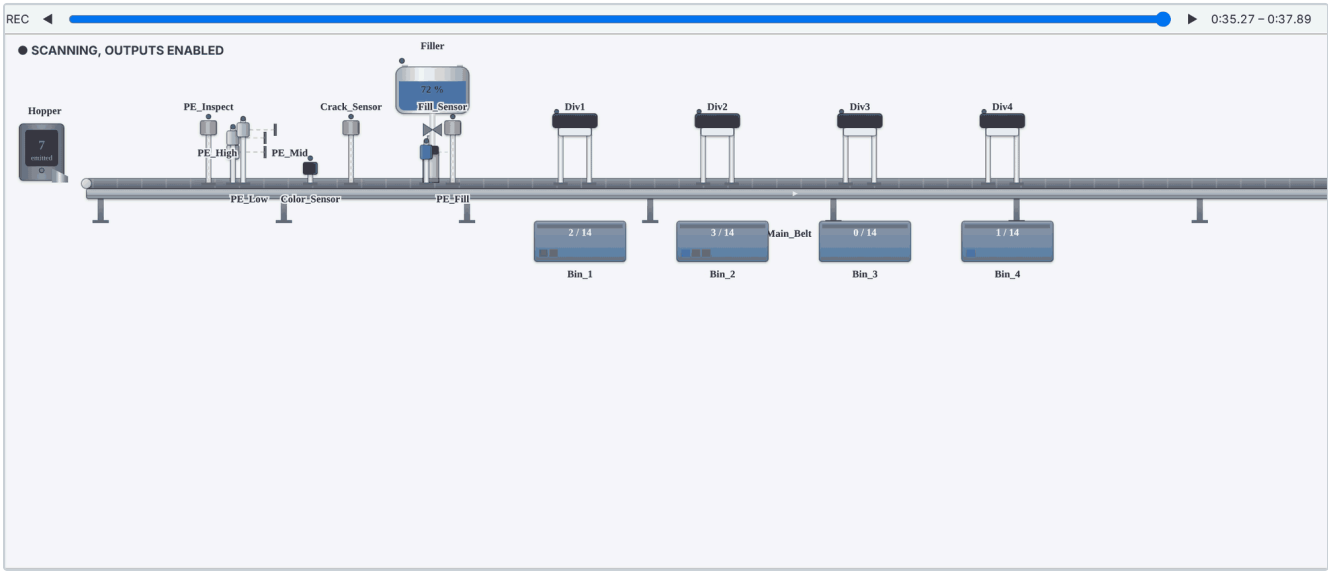


Figure 121. Level 4 running: several bottles on the belt at once, each with its class in a queue.

Tags this lesson needs

The level's alias tags are already made for you (chapter 30's table). Create these plain tags before you type the rungs; Verify reports any you miss.

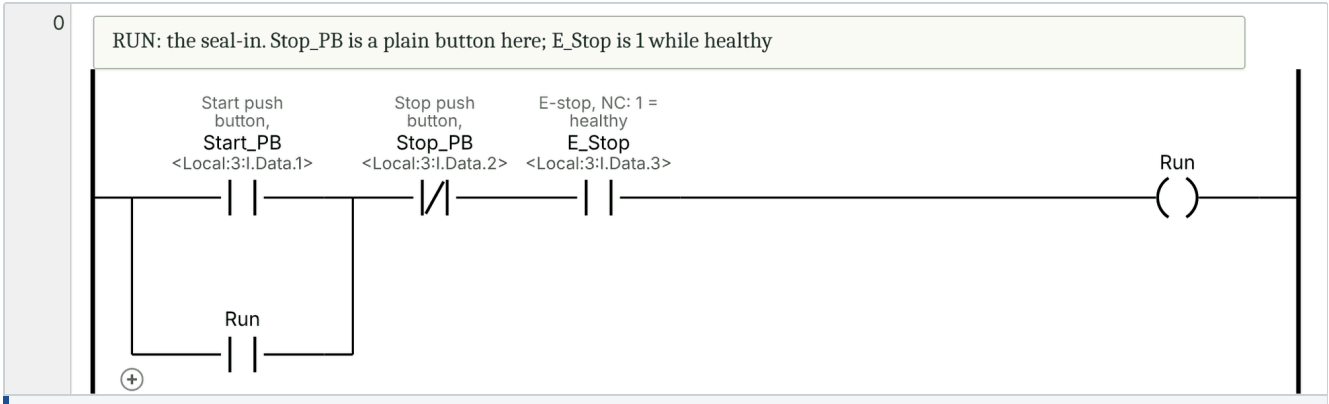
Tag	Type	How to make it	Description
`Run`	BOOL	Edit Tags: name, type	Seal-in run bit

Tag	Type	How to make it	Description
`Busy`	BOOL	Edit Tags: name, type	A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)
`Rel_ONS`	BOOL	Edit Tags: name, type	One-shot storage for the release
`Hold`	BOOL	Edit Tags: name, type	The filler holds the belt
`Filled`	BOOL	Edit Tags: name, type	This bottle has its charge
`AtFill`	BOOL	Edit Tags: name, type	The eye under the nozzle has seen this bottle
`Leave_ONS`	BOOL	Edit Tags: name, type	One-shot storage: the bottle left the filler
`Is_Low`	BOOL	Edit Tags: name, type	Latched: the low height beam was broken
`Is_Mid`	BOOL	Edit Tags: name, type	Latched: the middle beam was broken
`Is_High`	BOOL	Edit Tags: name, type	Latched: the high beam was broken
`Is_Broken`	BOOL	Edit Tags: name, type	Latched: the crack eye saw a broken bottle
`Is_R`	BOOL	Edit Tags: name, type	Latched: the colour sensor saw red
`Is_B`	BOOL	Edit Tags: name, type	Latched: the colour sensor saw blue
`Is_G`	BOOL	Edit Tags: name, type	Latched: the colour sensor saw green
`Class`	DINT	Edit Tags: name, type, Style Binary	The bottle in the gantry: bit 0 S, 1 M, 2 L, 3 broken, 4 red, 5 blue, 6 green
`Cur`	DINT	Edit Tags: name, type, Style Binary	The class unloaded from a zone queue at a diverter
`Q1`	DINT[8]	Edit Tags: name, type, Dim 8	Zone 1 queue: classes of the bottles between the filler and diverter 1
`Q1_Ctl`	CONTROL	Edit Tags: name, type	Queue control
`D1_ONS`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`D1_ONS2`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`D1_ONS3`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`Fire_1`	BOOL	Edit Tags: name, type	Armed: this diverter will push the bottle at its eye
`Q2`	DINT[8]	Edit Tags: name, type, Dim 8	Zone 2 queue: classes of the bottles between diverter 1 and diverter 2
`Q2_Ctl`	CONTROL	Edit Tags: name, type	Queue control
`D2_ONS`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`D2_ONS2`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`D2_ONS3`	BOOL	Edit Tags: name, type	One-shot storage for a diverter

Tag	Type	How to make it	Description
`Fire_2`	BOOL	Edit Tags: name, type	Armed: this diverter will push the bottle at its eye
`Q3`	DINT[8]	Edit Tags: name, type, Dim 8	Zone 3 queue: classes of the bottles between diverter 2 and diverter 3
`Q3_Ctl`	CONTROL	Edit Tags: name, type	Queue control
`D3_ONS`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`D3_ONS2`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`D3_ONS3`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`Fire_3`	BOOL	Edit Tags: name, type	Armed: this diverter will push the bottle at its eye
`Q4`	DINT[8]	Edit Tags: name, type, Dim 8	Zone 4 queue: classes of the bottles between diverter 3 and diverter 4
`Q4_Ctl`	CONTROL	Edit Tags: name, type	Queue control
`D4_ONS`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`D4_ONS2`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`D4_ONS3`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`Fire_4`	BOOL	Edit Tags: name, type	Armed: this diverter will push the bottle at its eye

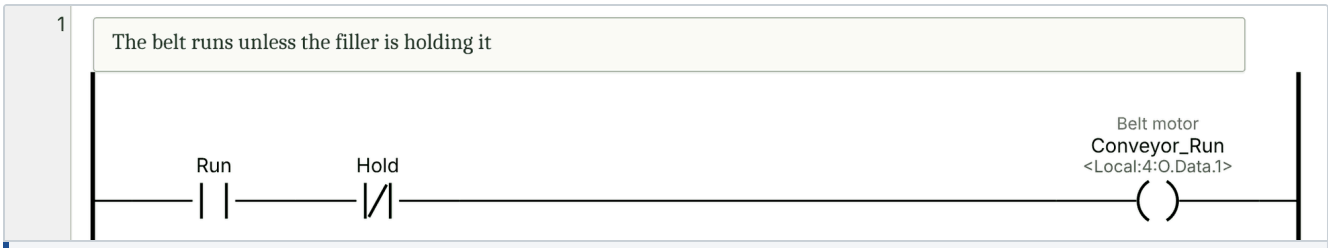
The run circuit and the release

RUNG 0




```
[XIC(Start_PB),XIC(Run)]XIO(Stop_PB)XIC(E_Stop)OTE(Run);
```

RUNG 1

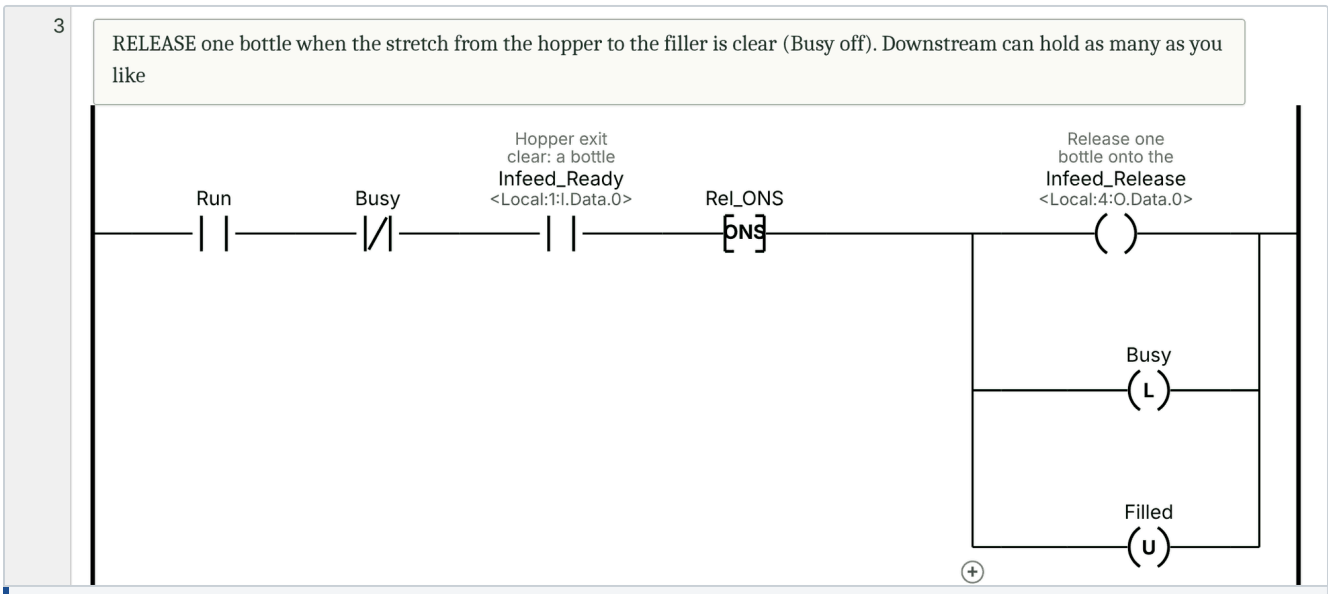



```
XIC(Run)XIO(Hold)OTE(Conveyor_Run);
```

RUNG 2

```
XIC(Run)OTE(Lamp_Run);
```

RUNG 3

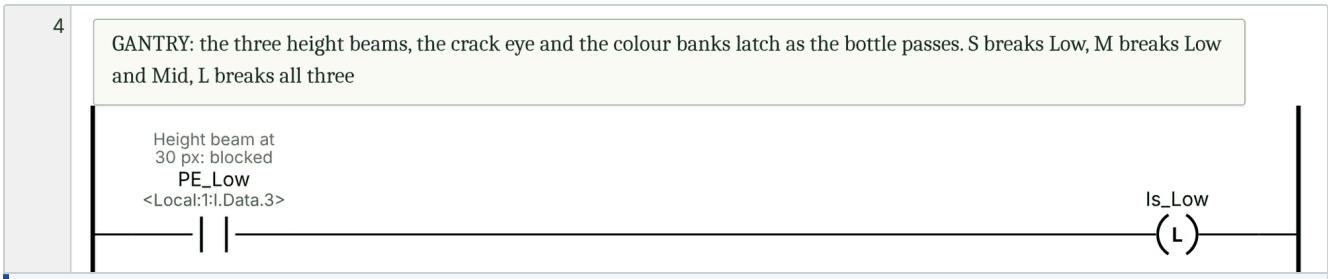



```
XIC(Run)XIO(Busy)XIC(Infeed_Ready)ONS(Rel_ONS)
[OTE(Infeed_Release),OTL(Busy),OTU(Filled)];
```

As in Level 2, with one change in meaning: `Busy` now means *a bottle is between the hopper and the filler*, not *a bottle is somewhere on the line*.

The gantry and the class word

RUNG 4

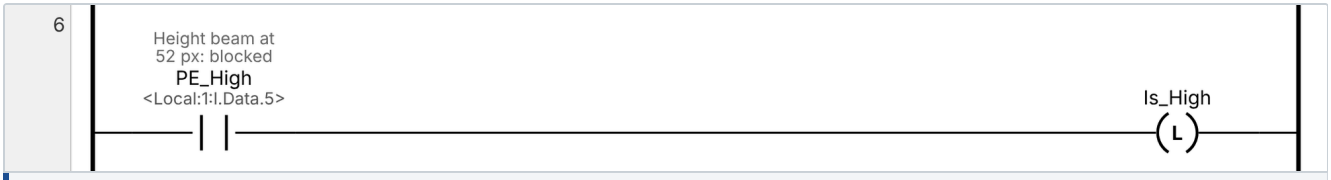



```
XIC(PE_Low)OTL(Is_Low);
```

RUNG 5

```
XIC(PE_Mid)OTL(Is_Mid);
```

RUNG 6




```
XIC(PE_High)OTL(Is_High);
```

RUNG 7

```
XIC(Crack_Sensor)OTL(Is_Broken);
```

RUNG 8

```
XIC(Color_Red)OTL(Is_R);
```

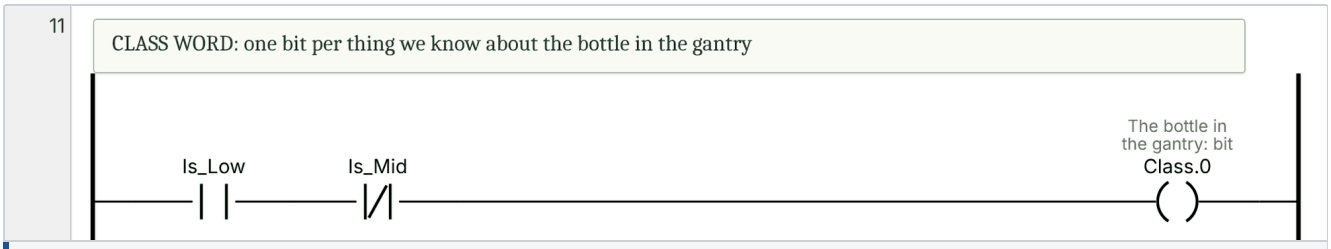
RUNG 9

```
XIC(Color_Blue)OTL(Is_B);
```

RUNG 10

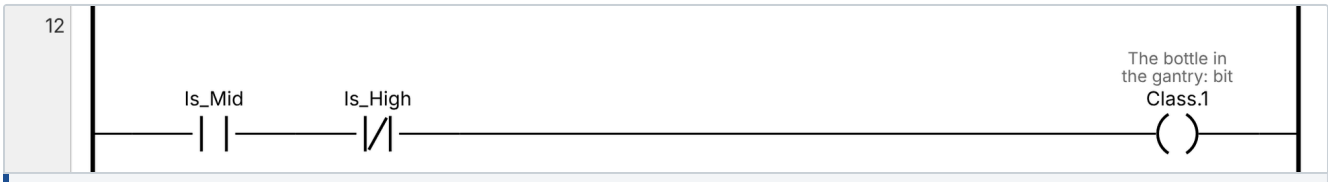
```
XIC(Color_Green)OTL(Is_G);
```

RUNG 11



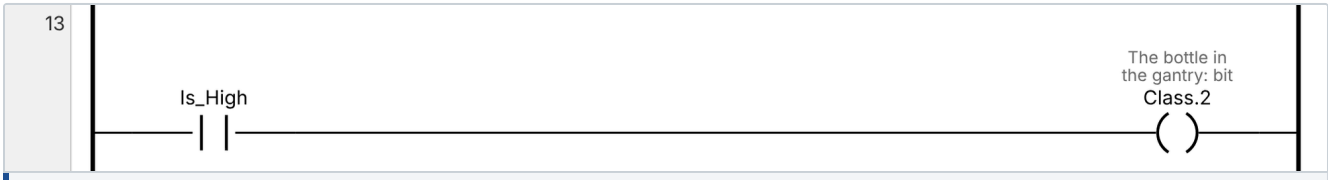

```
XIC(Is_Low)XIO(Is_Mid)OTE(Class.0);
```

RUNG 12



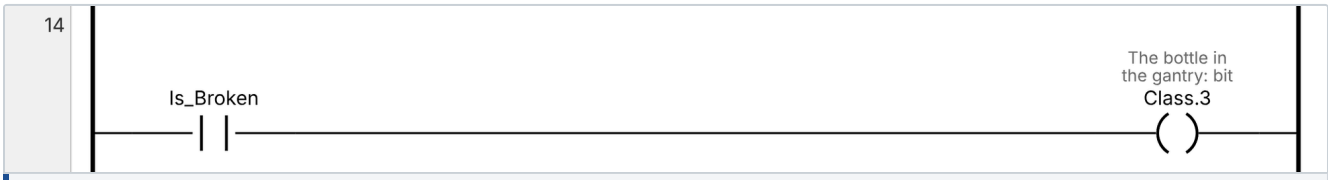

```
XIC(Is_Mid)XIO(Is_High)OTE(Class.1);
```

RUNG 13



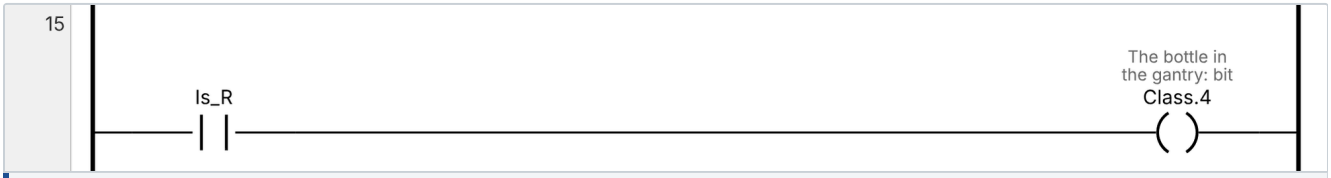

```
XIC(Is_High)OTE(Class.2);
```

RUNG 14



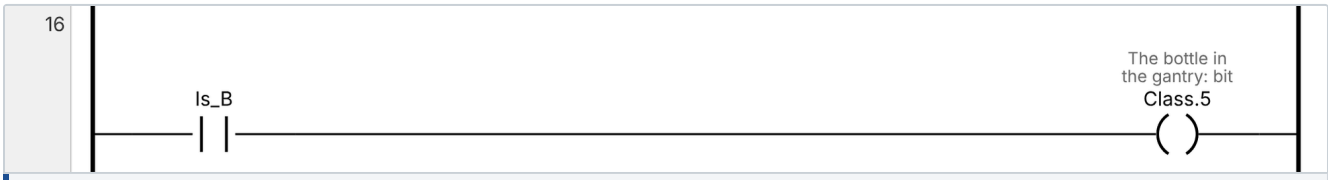

```
XIC(Is_Broken)OTE(Class.3);
```

RUNG 15




```
XIC(Is_R)OTE(Class.4);
```

RUNG 16




```
XIC(Is_B)OTE(Class.5);
```

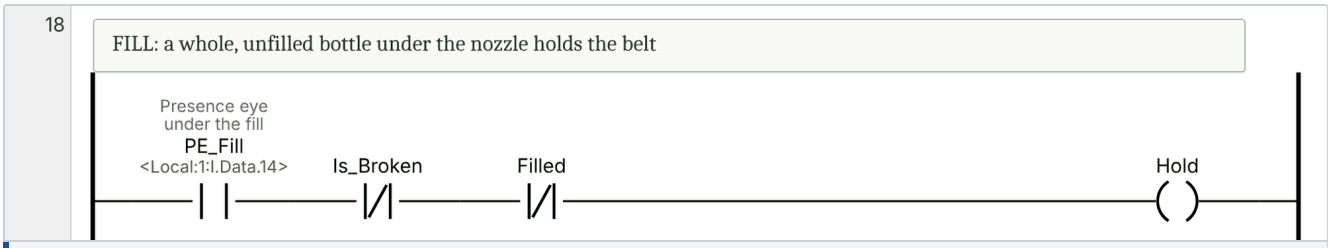
RUNG 17

```
XIC(Is_G)OTE(Class.6);
```

The seven latches are Level 1's and Level 3's together (the colour bits are not needed on this level; they are there because this is the one program for all five, and they cost nothing). **Rungs 11-17** fold them into the bits of the DINT `Class` with OTEs — a tag used a bit at a time, chapter 5. `Class` is Binary style, so Monitor Tags shows the bits.

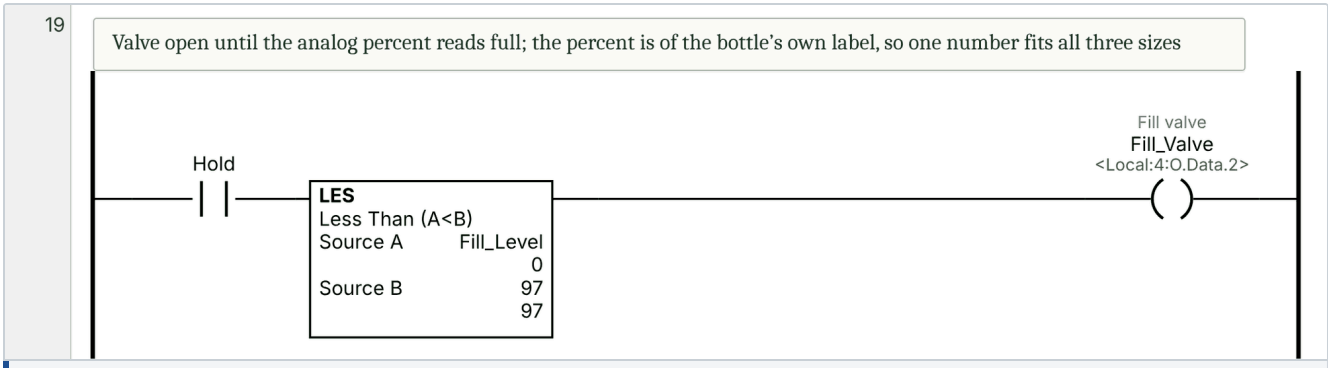
The fill

RUNG 18



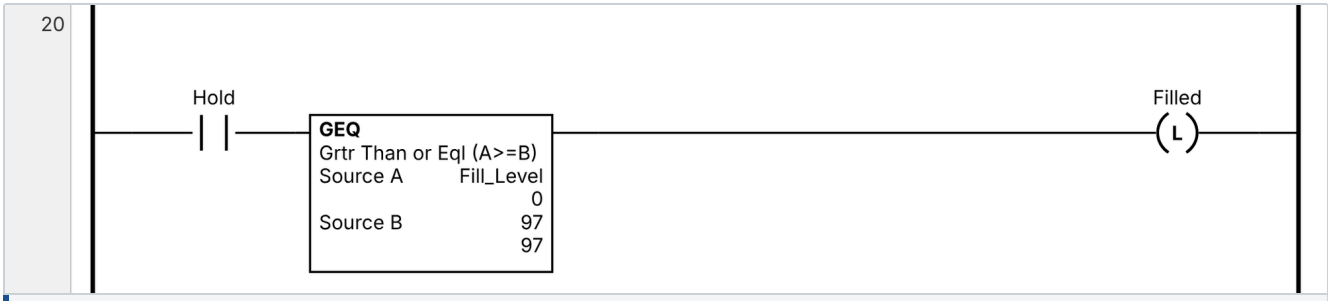

```
XIC(PE_Fill)XIO(Is_Broken)XIO(Filled)OTE(Hold);
```

RUNG 19




```
XIC(Hold)LES(Fill_Level,97)OTE(Fill_Valve);
```

RUNG 20




```
XIC(Hold)GEQ(Fill_Level,97)OTL(Filled);
```

Level 2's fill, unchanged.

Leaving the filler

RUNG 21

21

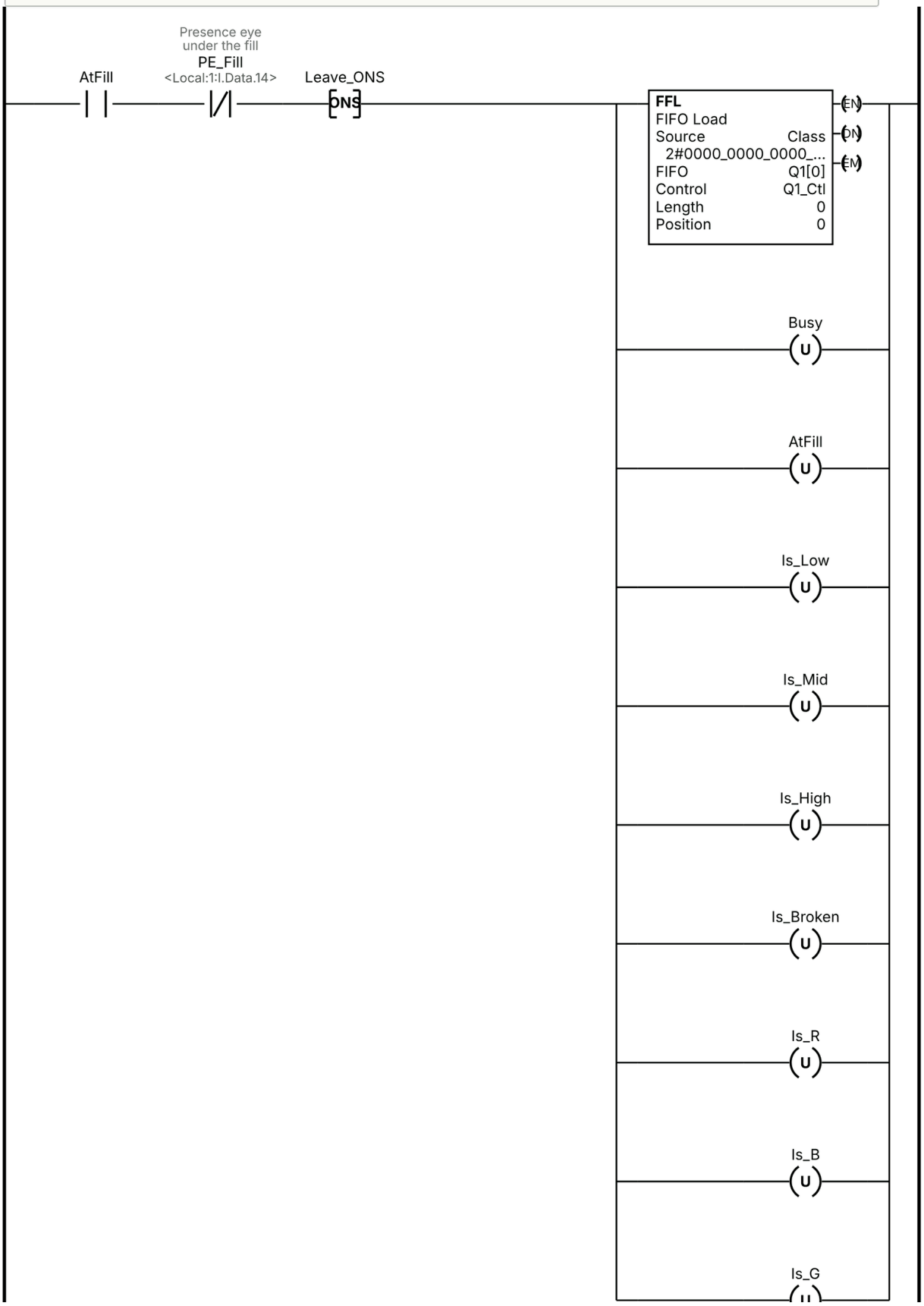
LEAVING THE FILLER: when the eye under the nozzle has seen the bottle and then lost it, the bottle is on its way to the diverters

```
XIC(PE_Fill)OTL(AtFill);
```

RUNG 22

22

Load its class into zone 1's queue, free the front of the line for the next bottle, and forget what the gantry saw

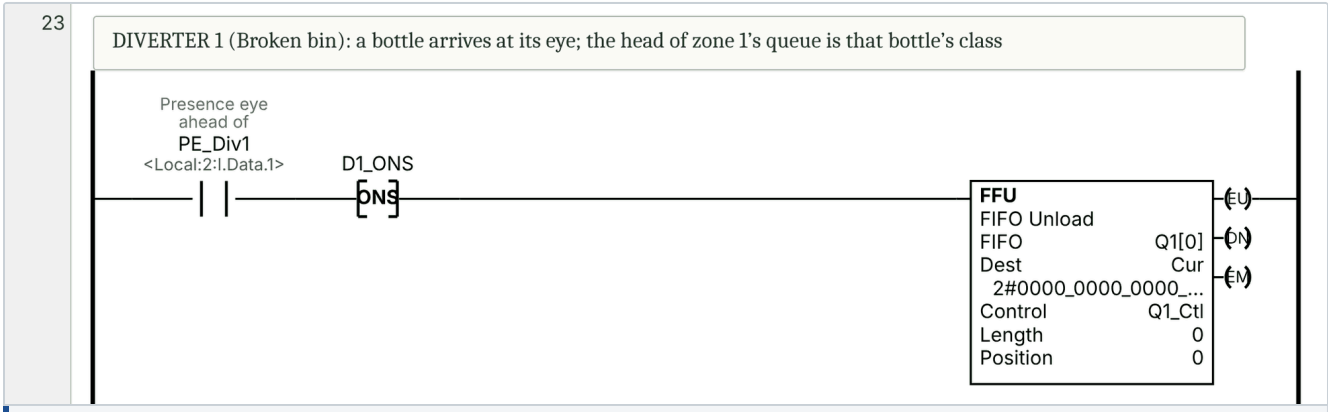


```
XIC(AtFill)XIO(PE_Fill)ONS(Leave_ONS)
[FFL(Class,Q1[0],Q1_Ctl,8),OTU(Busy),OTU(AtFill),OTU(Is_Low),OTU(Is_Mid),OTU(Is
_High),OTU(Is_Broken),OTU(Is_R),OTU(Is_B),OTU(Is_G)];
```

Rung 21 latches `AtFill` when the eye under the nozzle sees the bottle. **Rung 22** fires once when that eye then loses it — the bottle has left — and does four things in one scan: loads `Class` into zone 1's queue, frees the front of the line (`OTU(Busy)`, so rung 3 releases the next bottle), clears `AtFill`, and forgets the gantry's latches. The order inside the branch does not matter: the FFL copies `Class` as it is at the start of the rung, and the latches feed `Class` through rungs 11-17, which do not run again until the next scan.

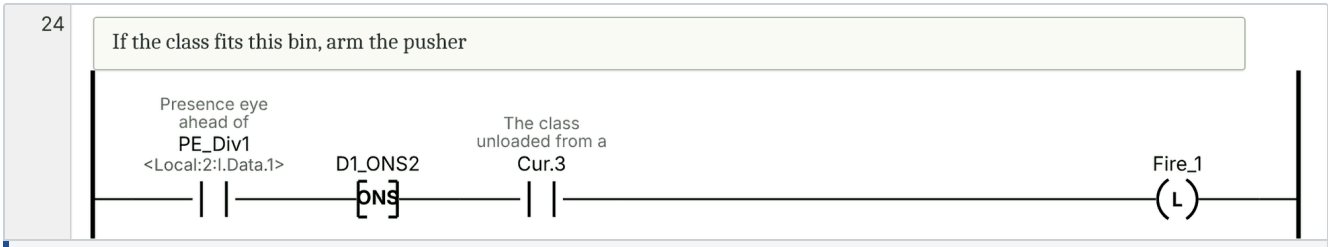
Diverter 1

RUNG 23



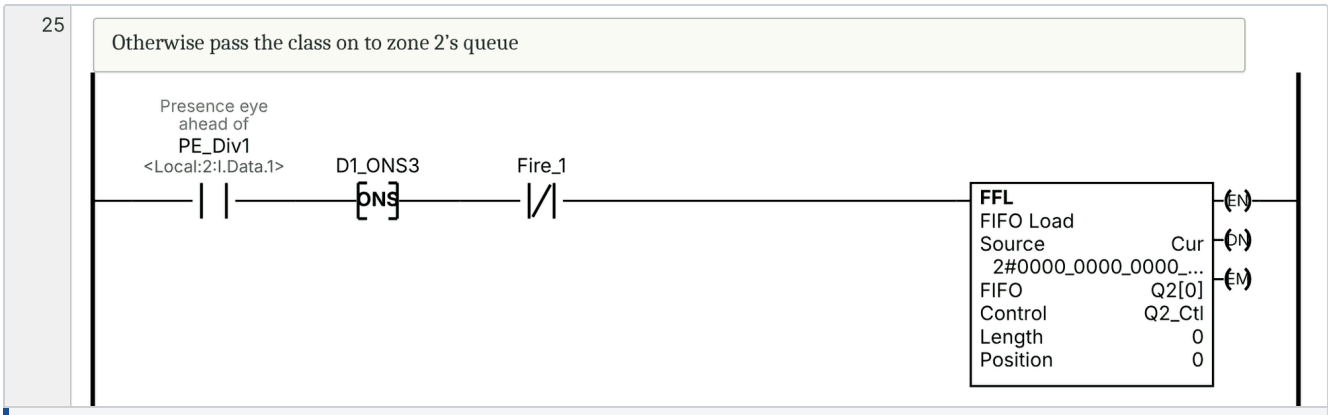

```
XIC(PE_Div1)ONS(D1_ONS)FFU(Q1[0],Cur,Q1_Ctl,8);
```

RUNG 24



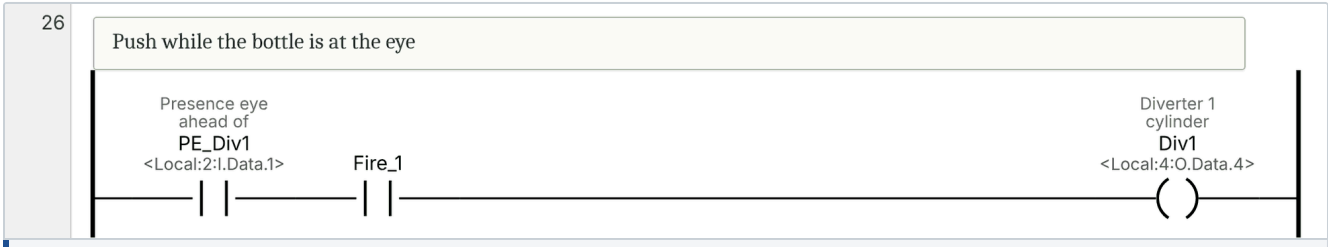

```
XIC(PE_Div1)ONS(D1_ONS2)XIC(Cur.3)OTL(Fire_1);
```

RUNG 25



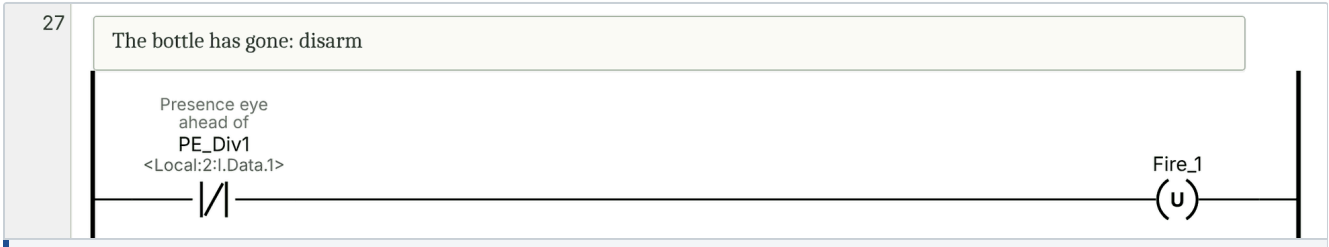

```
XIC(PE_Div1)ONS(D1_ONS3)XIO(Fire_1)FFL(Cur,Q2[0],Q2_Ctl,8);
```

RUNG 26




```
XIC(PE_Div1)XIC(Fire_1)OTE(Div1);
```

RUNG 27



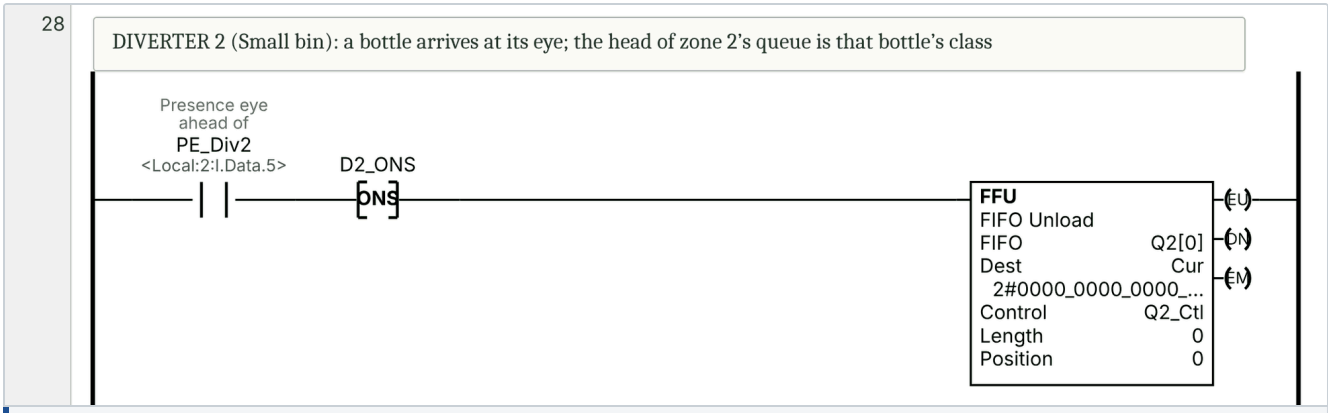

```
XIO(PE_Div1)OTU(Fire_1);
```

Five rungs per diverter, and the same five for every lane. **Rung 23**: a bottle arrives at the eye — unload the head of zone 1's queue into `Cur`. **Rung 24**: if `Cur` fits this bin (here: bit 3, broken), arm the pusher. **Rung 25**: if it did not fit, pass the class on to zone 2's queue. **Rung 26**: push while the bottle is at the eye and the pusher is armed. **Rung 27**: the bottle has gone, disarm.

The three one-shots on the same eye (`D1_ONS`, `D1_ONS2`, `D1_ONS3`) all fire on the same scan, in rung order — that is the scan of chapter 4 doing the work. Rung 23 unloads, so `Cur` is fresh when rung 24 tests it; rung 24 arms, so when rung 25 asks `XIO(Fire_1)` it already knows the answer. That is how *otherwise* is written without repeating the match in reverse. Each needs its own storage bit: a one-shot remembers last scan's rung state, and three rungs sharing one bit would see each other's.

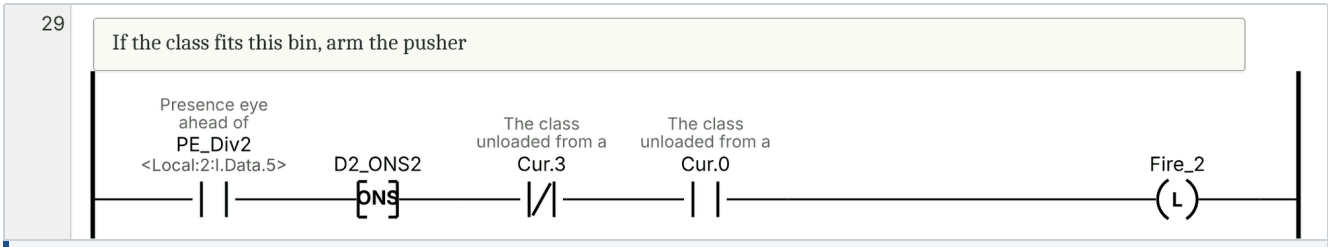
Diverters 2, 3 and 4

RUNG 28



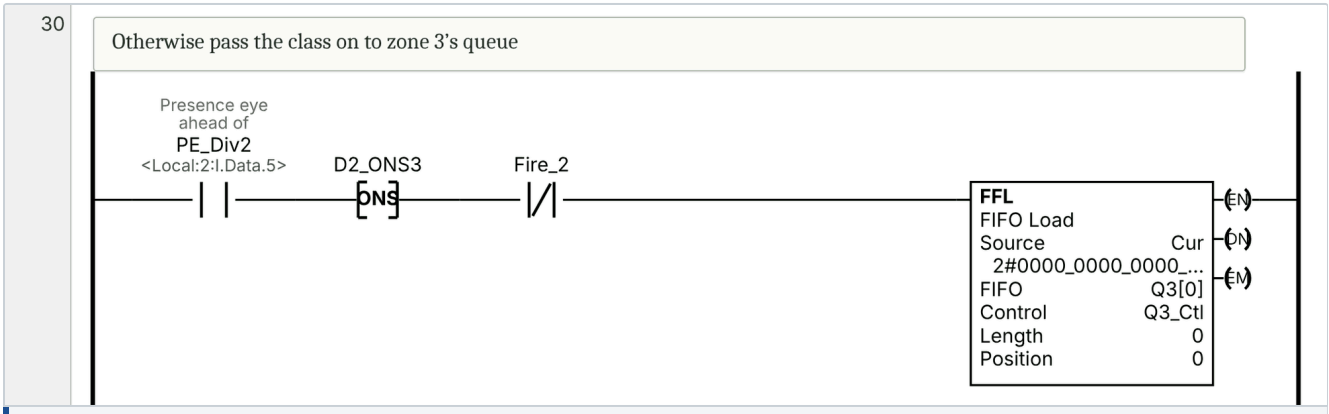

```
XIC(PE_Div2)ONS(D2_ONS)FFU(Q2[0],Cur,Q2_Ctl,8);
```

RUNG 29




```
XIC(PE_Div2)ONS(D2_ONS2)XIO(Cur.3)XIC(Cur.0)OTL(Fire_2);
```

RUNG 30




```
XIC(PE_Div2)ONS(D2_ONS3)XIO(Fire_2)FFL(Cur,Q3[0],Q3_Ctl,8);
```

RUNG 31

31

Push while the bottle is at the eye

```
XIC(PE_Div2)XIC(Fire_2)OTE(Div2);
```

RUNG 32

32

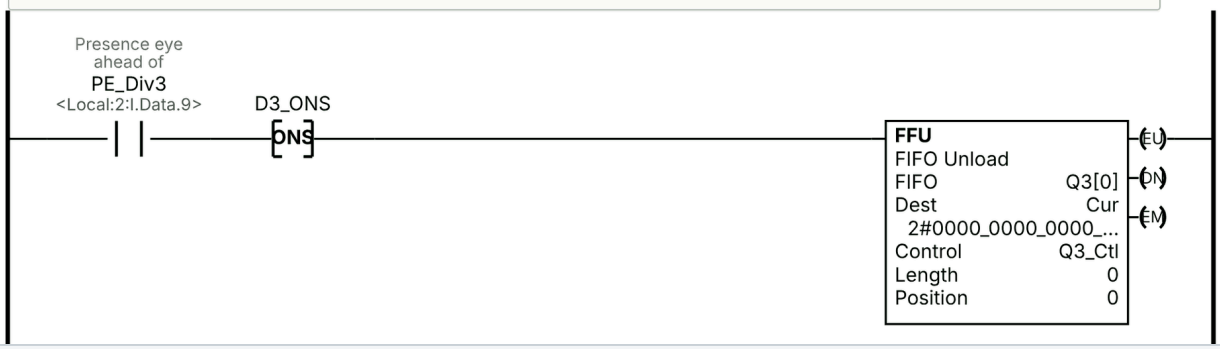
The bottle has gone: disarm

```
XIO(PE_Div2)OTU(Fire_2);
```

RUNG 33

33

DIVERTER 3 (Medium bin): a bottle arrives at its eye; the head of zone 3's queue is that bottle's class

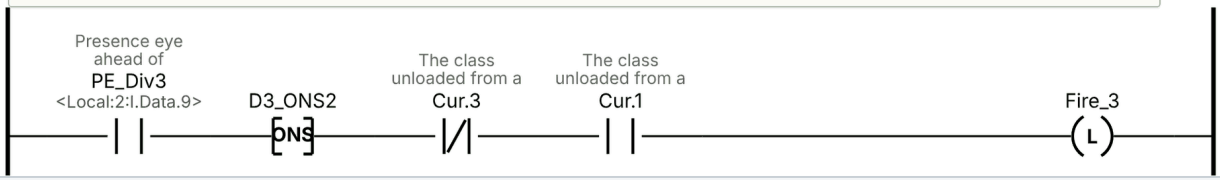



```
XIC(PE_Div3)ONS(D3_ONS)FFU(Q3[0],Cur,Q3_Ctl,8);
```

RUNG 34

34

If the class fits this bin, arm the pusher

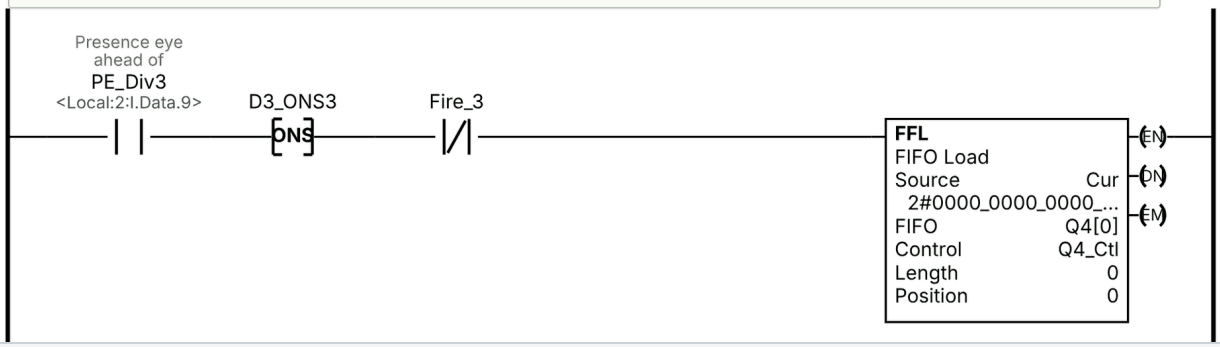



```
XIC(PE_Div3)ONS(D3_ONS2)XIO(Cur.3)XIC(Cur.1)OTL(Fire_3);
```

RUNG 35

35

Otherwise pass the class on to zone 4's queue




```
XIC(PE_Div3)ONS(D3_ONS3)XIO(Fire_3)FFL(Cur,Q4[0],Q4_Ctl,8);
```

RUNG 36

36

Push while the bottle is at the eye

```
XIC(PE_Div3)XIC(Fire_3)OTE(Div3);
```

RUNG 37

37

The bottle has gone: disarm

```
XIO(PE_Div3)OTU(Fire_3);
```

RUNG 38

38

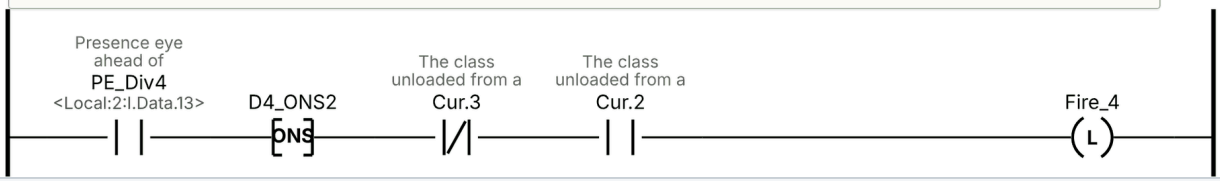
DIVERTER 4 (Large bin): a bottle arrives at its eye; the head of zone 4's queue is that bottle's class

```
XIC(PE_Div4)ONS(D4_ONS)FFU(Q4[0],Cur,Q4_Ctl,8);
```

RUNG 39

39

If the class fits this bin, arm the pusher

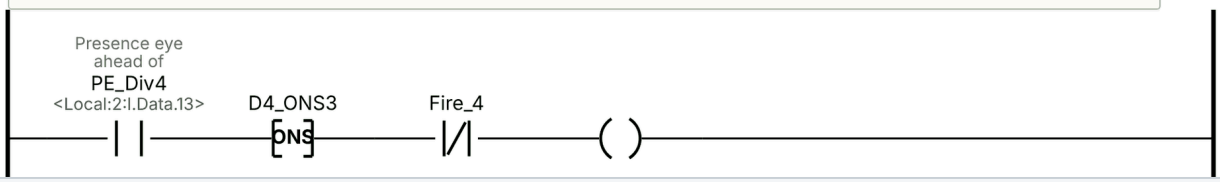



```
XIC(PE_Div4)ONS(D4_ONS2)XIO(Cur.3)XIC(Cur.2)OTL(Fire_4);
```

RUNG 40

40

Otherwise it runs on to the end of the belt




```
XIC(PE_Div4)ONS(D4_ONS3)XIO(Fire_4)NOP();
```

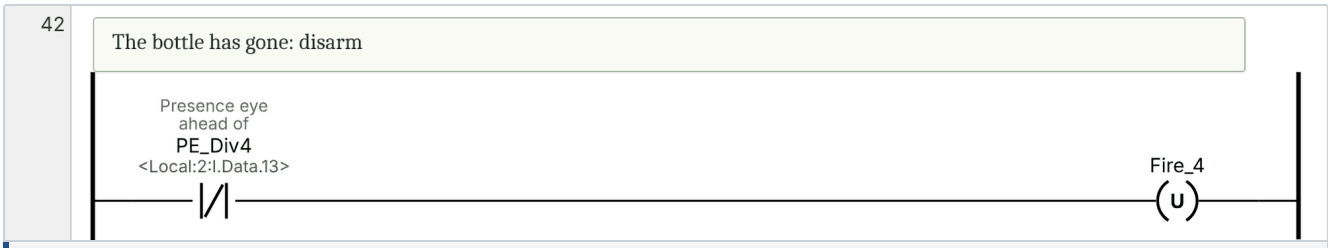
RUNG 41

41

Push while the bottle is at the eye

```
XIC(PE_Div4)XIC(Fire_4)OTE(Div4);
```

RUNG 42




```
XIO(PE_Div4)OTU(Fire_4);
```

The same five rungs, with the match changed: small is `XIO(Cur.3)XIC(Cur.0)`, medium `XIO(Cur.3)XIC(Cur.1)`, large `XIO(Cur.3)XIC(Cur.2)`. The last diverter's pass-on rung ends in `NOP()`: a bottle no bin wanted rides to the remover at the end of the belt, and there is no queue after the last zone.

WATCH FOR: A SLIPPED QUEUE

Everything depends on every bottle that enters a zone being loaded into its queue exactly once, and unloaded exactly once. Miss one load — a pass-on rung with the wrong condition — and every class in the next zone is one bottle late: the next diverter sorts the wrong bottle, and so does every diverter after it, until the line is emptied. That is the same failure as a shift register that misses a pulse in Part IV, and the same check finds it: Monitor Tags, `Q1_Ctl.POS` and `Q2_Ctl.POS`, against the bottles you can see in each stretch of belt.

Run it

Bottles leave the hopper every few seconds; several are on the belt at once, and each still goes to the right bin. Expand `Q1` and `Q2` in Monitor Tags and watch the classes move through the queues as the bottles move down the belt. The reference run: 14 of 14 bottles in the right bin, nothing lost, in 85 s against a par of 110 s: 1600 points and three stars.

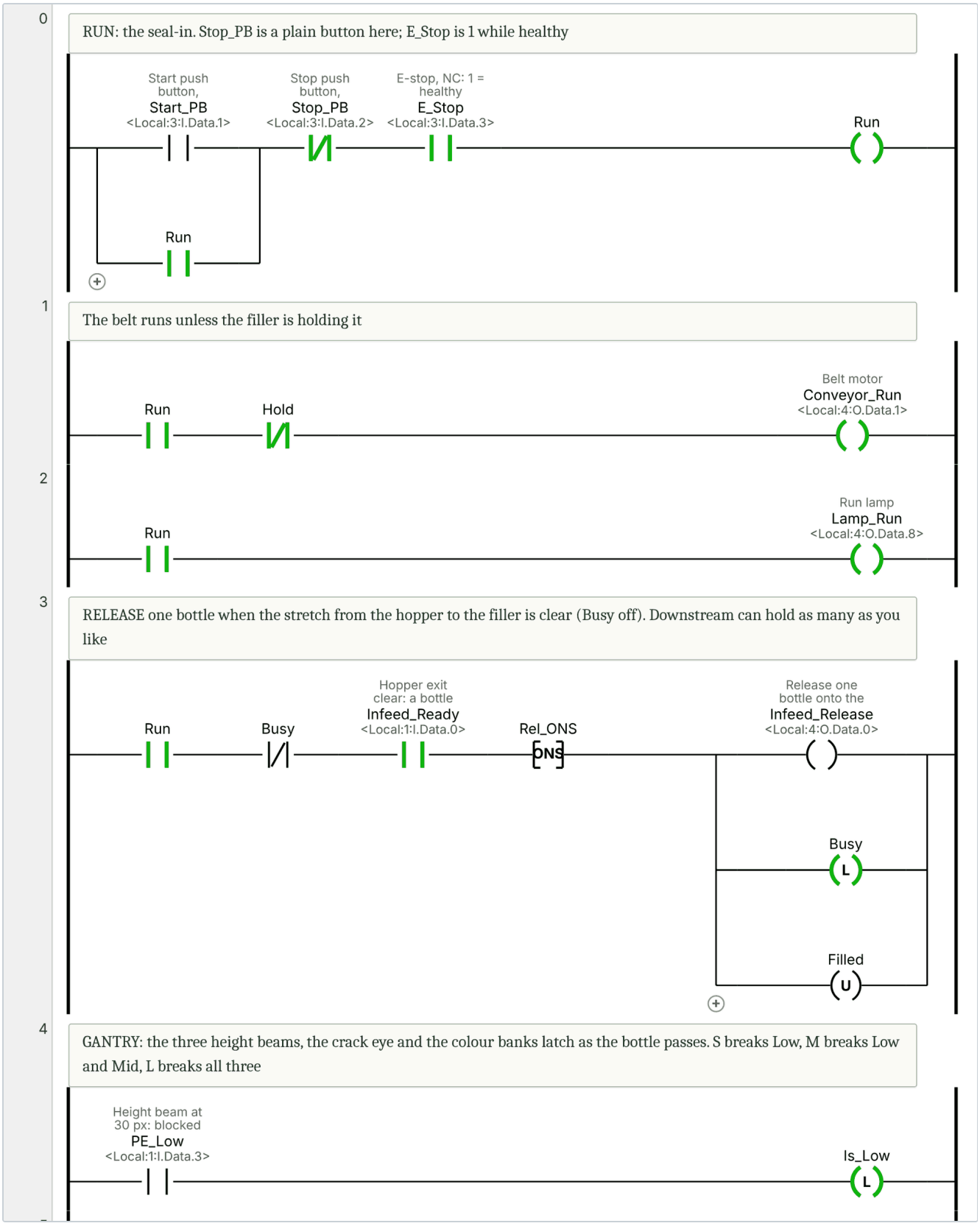


Figure 122. Level 4 online, the top of the routine: release, gantry and class word.

TRY IT

Delete diverter 1's pass-on rung (rung 25) and run: from the first non-broken bottle on, diverter 2 unloads nothing (its queue is empty) and nothing is sorted. Put it back and delete rung 27 instead: `Fire_1` stays armed and diverter 1 pushes the next bottle too, whatever it is.

36 Level 5 — Full sort

> **IN THE APP**
>
> A level project opens with guided mode on this chapter: the panel lists the tags the program needs (*Make them*), shows every rung below under its fold with *Copy* and *Add*, checks that MainRoutine verifies, and while you run the level ticks three results — the level completes, every bottle in the right bin, under par.

Sixteen bottles, six bins: broken; small; mediums split three ways by colour — red, blue, green; and large. Every whole bottle filled. Par 120 s. The method is Level 4's, unchanged; what grows is the question each diverter asks, and this lesson is about writing a match that tests **everything** it depends on.

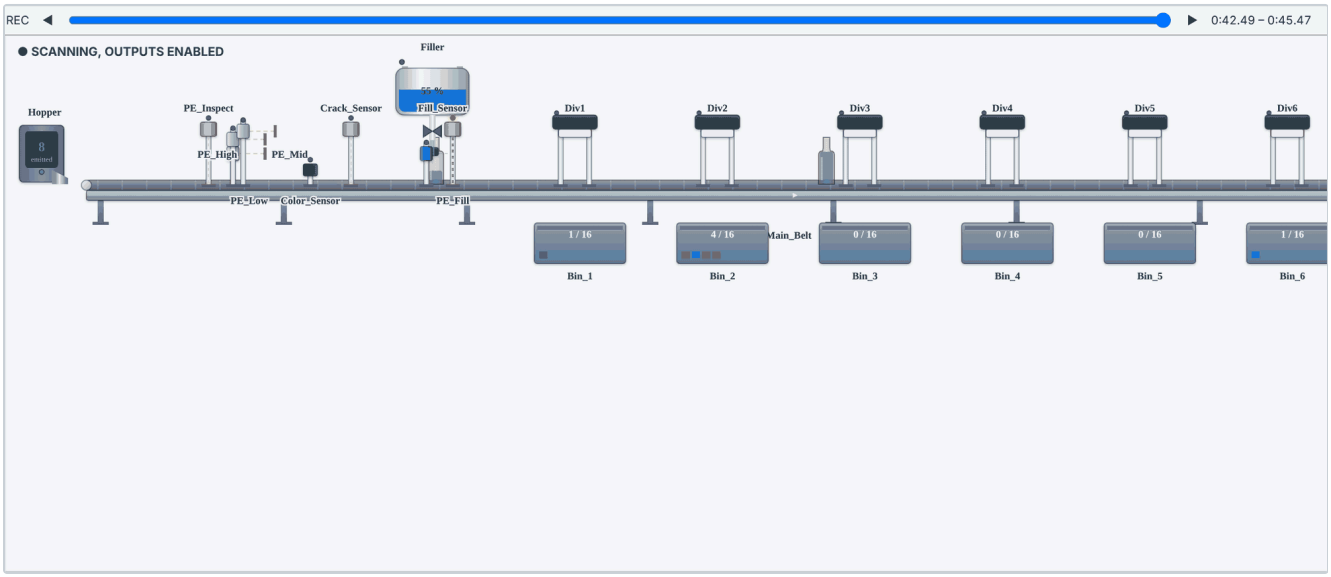


Figure 123. Level 5 running: six pushers, six graded bins.

Tags this lesson needs

The level's alias tags are already made for you (chapter 30's table). Create these plain tags before you type the rungs; Verify reports any you miss.

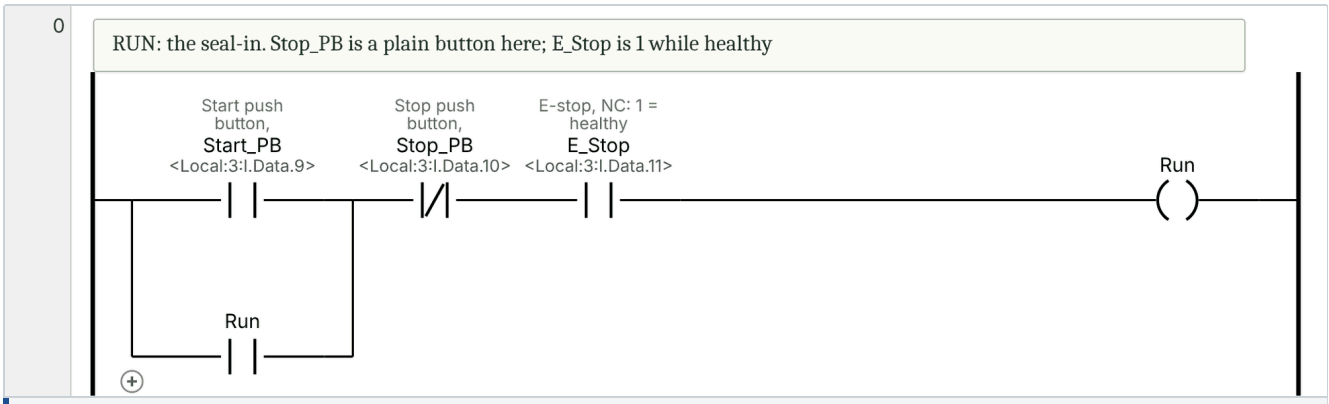
Tag	Type	How to make it	Description
`Run`	BOOL	Edit Tags: name, type	Seal-in run bit
`Busy`	BOOL	Edit Tags: name, type	A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)
`Rel_ONS`	BOOL	Edit Tags: name, type	One-shot storage for the release
`Hold`	BOOL	Edit Tags: name, type	The filler holds the belt

Tag	Type	How to make it	Description
`Filled`	BOOL	Edit Tags: name, type	This bottle has its charge
`AtFill`	BOOL	Edit Tags: name, type	The eye under the nozzle has seen this bottle
`Leave_ONS`	BOOL	Edit Tags: name, type	One-shot storage: the bottle left the filler
`Is_Low`	BOOL	Edit Tags: name, type	Latched: the low height beam was broken
`Is_Mid`	BOOL	Edit Tags: name, type	Latched: the middle beam was broken
`Is_High`	BOOL	Edit Tags: name, type	Latched: the high beam was broken
`Is_Broken`	BOOL	Edit Tags: name, type	Latched: the crack eye saw a broken bottle
`Is_R`	BOOL	Edit Tags: name, type	Latched: the colour sensor saw red
`Is_B`	BOOL	Edit Tags: name, type	Latched: the colour sensor saw blue
`Is_G`	BOOL	Edit Tags: name, type	Latched: the colour sensor saw green
`Class`	DINT	Edit Tags: name, type, Style Binary	The bottle in the gantry: bit 0 S, 1 M, 2 L, 3 broken, 4 red, 5 blue, 6 green
`Cur`	DINT	Edit Tags: name, type, Style Binary	The class unloaded from a zone queue at a diverter
`Q1`	DINT[8]	Edit Tags: name, type, Dim 8	Zone 1 queue: classes of the bottles between the filler and diverter 1
`Q1_Ctl`	CONTROL	Edit Tags: name, type	Queue control
`D1_ONS`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`D1_ONS2`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`D1_ONS3`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`Fire_1`	BOOL	Edit Tags: name, type	Armed: this diverter will push the bottle at its eye
`Q2`	DINT[8]	Edit Tags: name, type, Dim 8	Zone 2 queue: classes of the bottles between diverter 1 and diverter 2
`Q2_Ctl`	CONTROL	Edit Tags: name, type	Queue control
`D2_ONS`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`D2_ONS2`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`D2_ONS3`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`Fire_2`	BOOL	Edit Tags: name, type	Armed: this diverter will push the bottle at its eye
`Q3`	DINT[8]	Edit Tags: name, type, Dim 8	Zone 3 queue: classes of the bottles between diverter 2 and diverter 3
`Q3_Ctl`	CONTROL	Edit Tags: name, type	Queue control

Tag	Type	How to make it	Description
`D3_ONS`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`D3_ONS2`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`D3_ONS3`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`Fire_3`	BOOL	Edit Tags: name, type	Armed: this diverter will push the bottle at its eye
`Q4`	DINT[8]	Edit Tags: name, type, Dim 8	Zone 4 queue: classes of the bottles between diverter 3 and diverter 4
`Q4_Ctl`	CONTROL	Edit Tags: name, type	Queue control
`D4_ONS`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`D4_ONS2`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`D4_ONS3`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`Fire_4`	BOOL	Edit Tags: name, type	Armed: this diverter will push the bottle at its eye
`Q5`	DINT[8]	Edit Tags: name, type, Dim 8	Zone 5 queue: classes of the bottles between diverter 4 and diverter 5
`Q5_Ctl`	CONTROL	Edit Tags: name, type	Queue control
`D5_ONS`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`D5_ONS2`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`D5_ONS3`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`Fire_5`	BOOL	Edit Tags: name, type	Armed: this diverter will push the bottle at its eye
`Q6`	DINT[8]	Edit Tags: name, type, Dim 8	Zone 6 queue: classes of the bottles between diverter 5 and diverter 6
`Q6_Ctl`	CONTROL	Edit Tags: name, type	Queue control
`D6_ONS`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`D6_ONS2`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`D6_ONS3`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`Fire_6`	BOOL	Edit Tags: name, type	Armed: this diverter will push the bottle at its eye

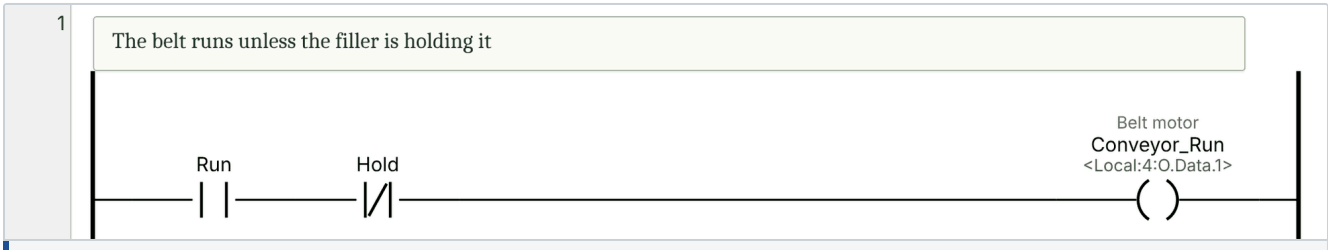
Run circuit, release, gantry, class word, fill and leaving the filler

RUNG 0




```
[XIC(Start_PB),XIC(Run)]XIO(Stop_PB)XIC(E_Stop)OTE(Run);
```

RUNG 1




```
XIC(Run)XIO(Hold)OTE(Conveyor_Run);
```

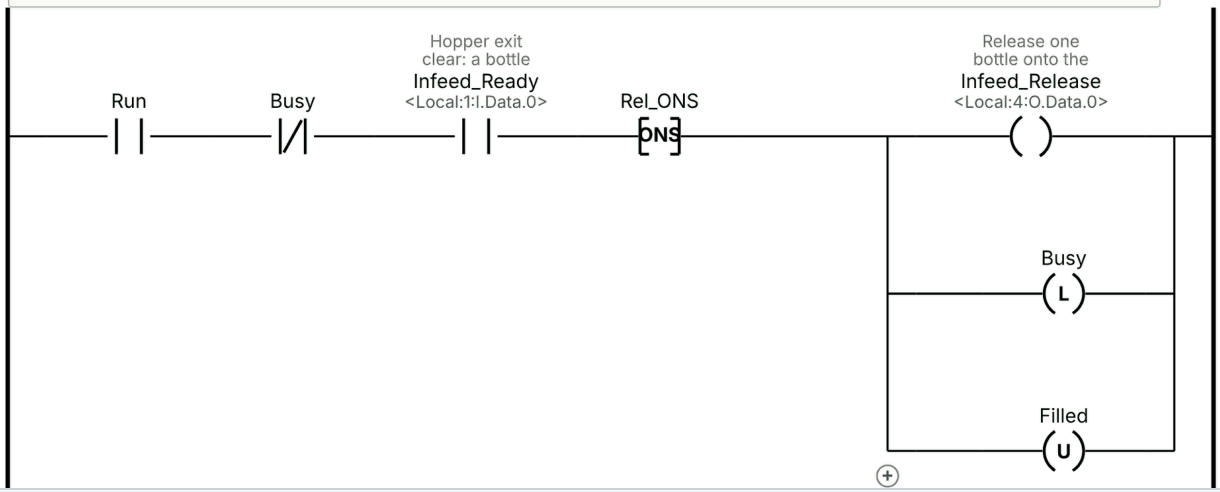
RUNG 2

```
XIC(Run)OTE(Lamp_Run);
```

RUNG 3

3

RELEASE one bottle when the stretch from the hopper to the filler is clear (Busy off). Downstream can hold as many as you like




```
XIC(Run)XIO(Busy)XIC(Infeed_Ready)ONS(Rel_ONS)
[OTE(Infeed_Release),OTL(Busy),OTU(Filled)];
```

RUNG 4

4

GANTRY: the three height beams, the crack eye and the colour banks latch as the bottle passes. S breaks Low, M breaks Low and Mid, L breaks all three

```
XIC(PE_Low)OTL(Is_Low);
```

RUNG 5

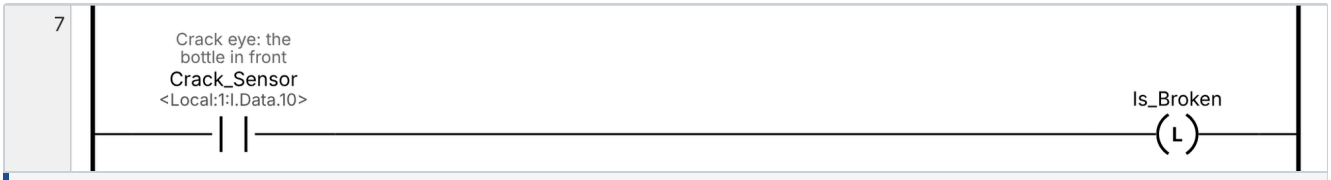
5

```
XIC(PE_Mid)OTL(Is_Mid);
```

RUNG 6

```
XIC(PE_High)OTL(Is_High);
```

RUNG 7




```
XIC(Crack_Sensor)OTL(Is_Broken);
```

RUNG 8

```
XIC(Color_Red)OTL(Is_R);
```

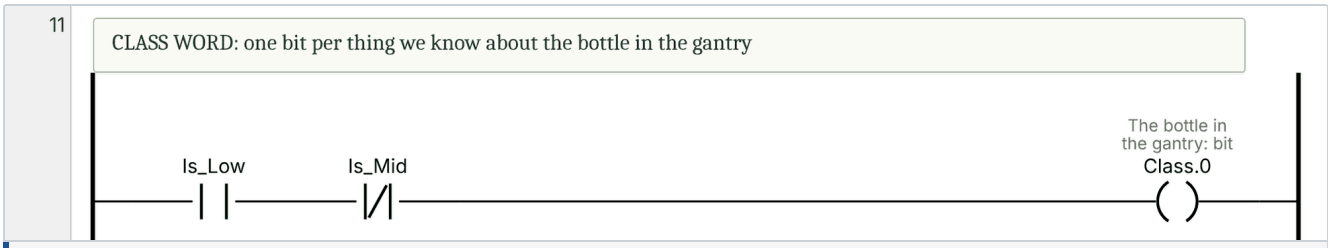
RUNG 9

```
XIC(Color_Blue)OTL(Is_B);
```

RUNG 10

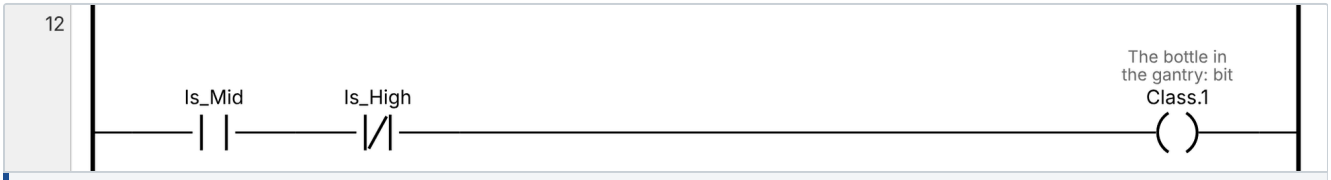
```
XIC(Color_Green)OTL(Is_G);
```

RUNG 11



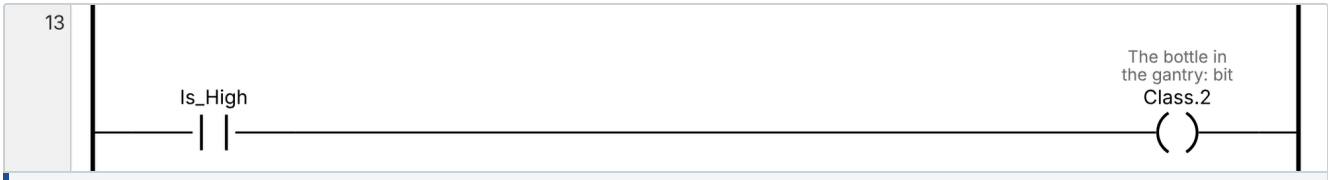

```
XIC(Is_Low)XIO(Is_Mid)OTE(Class.0);
```

RUNG 12



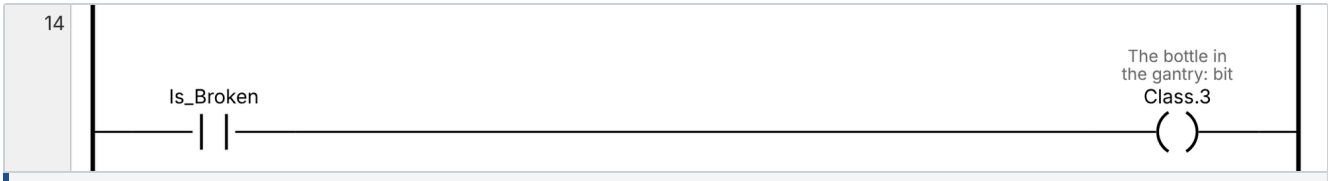

```
XIC(Is_Mid)XIO(Is_High)OTE(Class.1);
```

RUNG 13



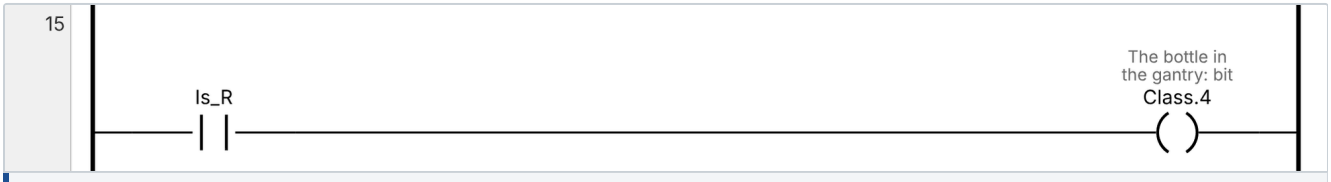

```
XIC(Is_High)OTE(Class.2);
```

RUNG 14




```
XIC(Is_Broken)OTE(Class.3);
```

RUNG 15




```
XIC(Is_R)OTE(Class.4);
```

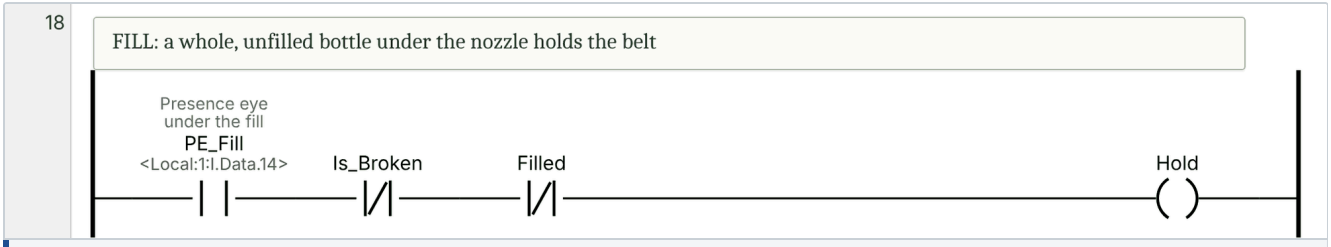
RUNG 16

```
XIC(Is_B)OTE(Class.5);
```

RUNG 17

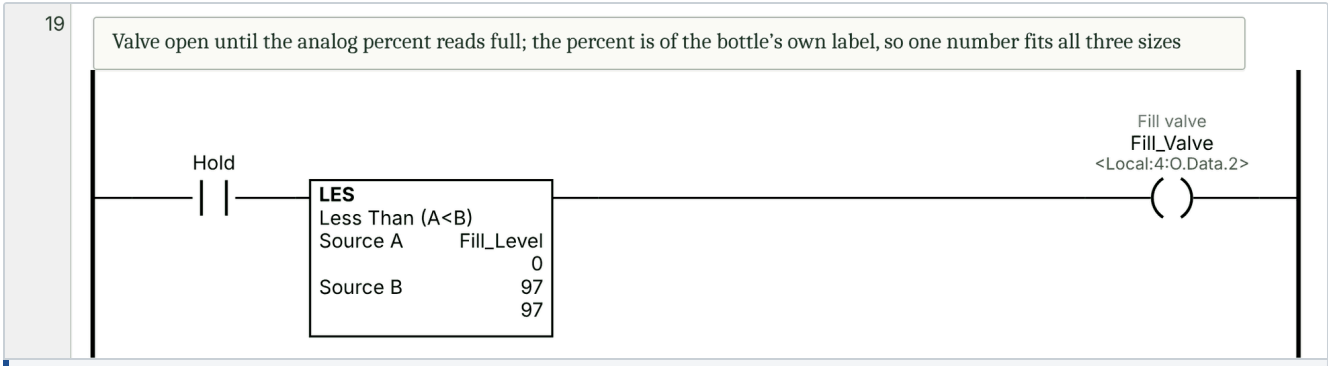
```
XIC(Is_G)OTE(Class.6);
```

RUNG 18



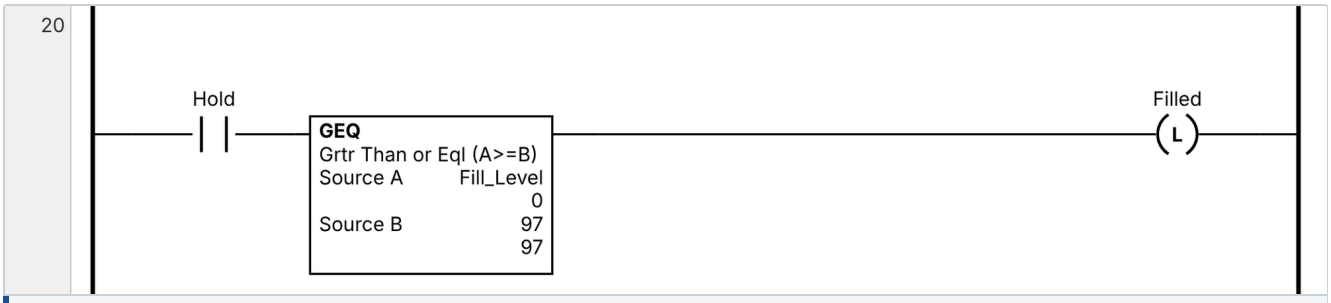

```
XIC(PE_Fill)XIO(Is_Broken)XIO(Filled)OTE(Hold);
```

RUNG 19



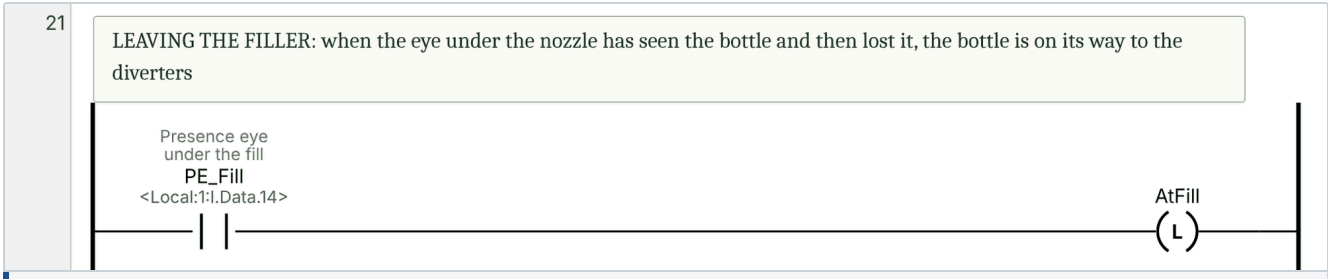

```
XIC(Hold)LES(Fill_Level,97)OTE(Fill_Valve);
```

RUNG 20




```
XIC(Hold)GEQ(Fill_Level,97)OTL(Filled);
```

RUNG 21

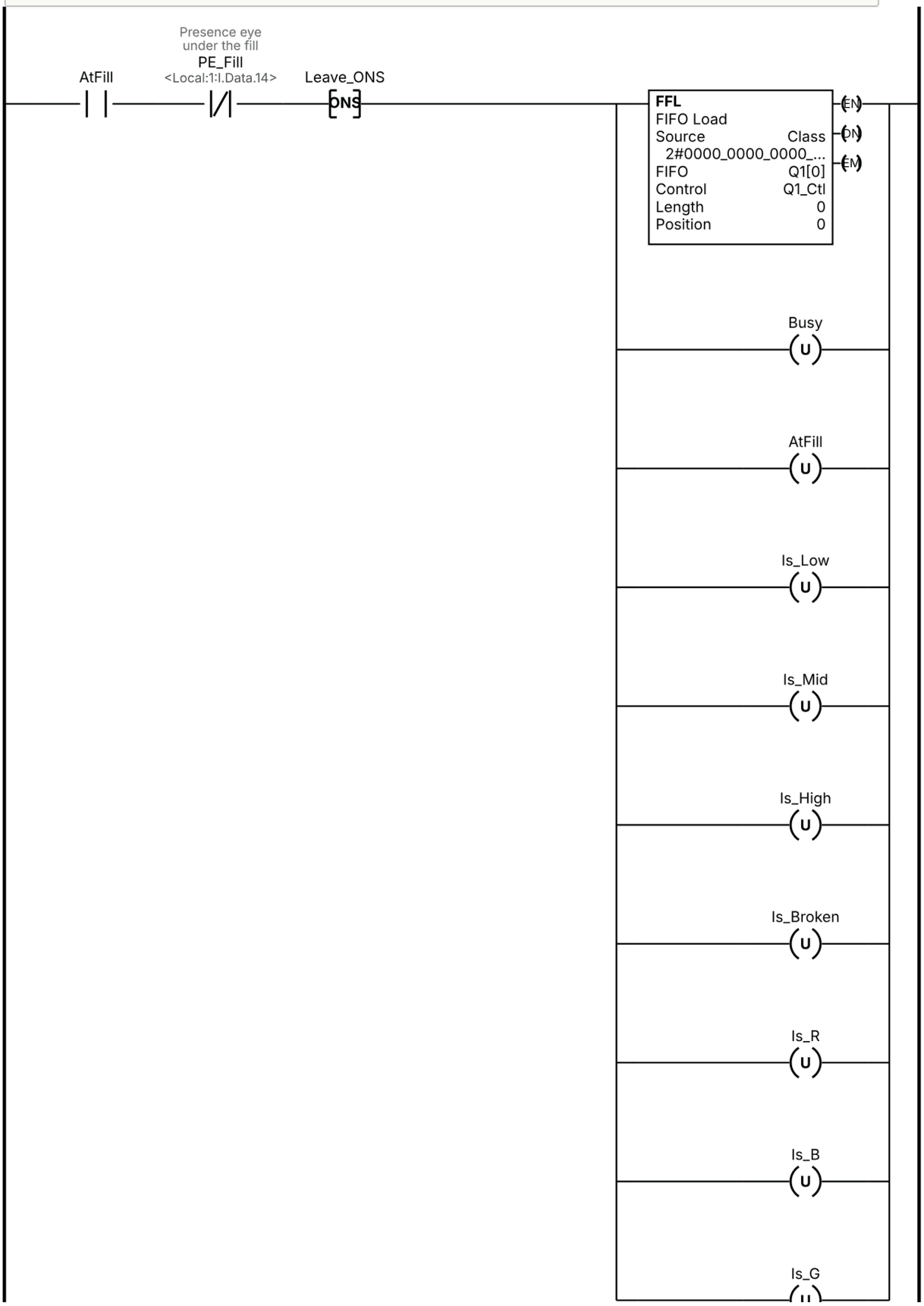



```
XIC(PE_Fill)OTL(AtFill);
```

RUNG 22

22

Load its class into zone 1's queue, free the front of the line for the next bottle, and forget what the gantry saw

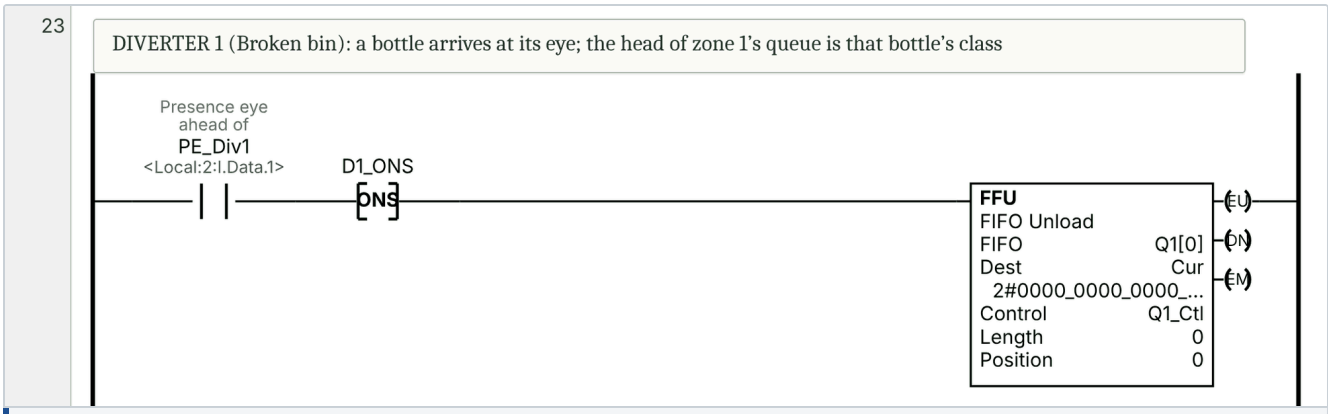


```
XIC(AtFill)XIO(PE_Fill)ONS(Leave_ONS)
[FFL(Class,Q1[0],Q1_Ctl,8),OTU(Busy),OTU(AtFill),OTU(Is_Low),OTU(Is_Mid),OTU(Is
_High),OTU(Is_Broken),OTU(Is_R),OTU(Is_B),OTU(Is_G)];
```

Identical to Level 4. Now the colour bits of `Class` — 4 red, 5 blue, 6 green — are needed.

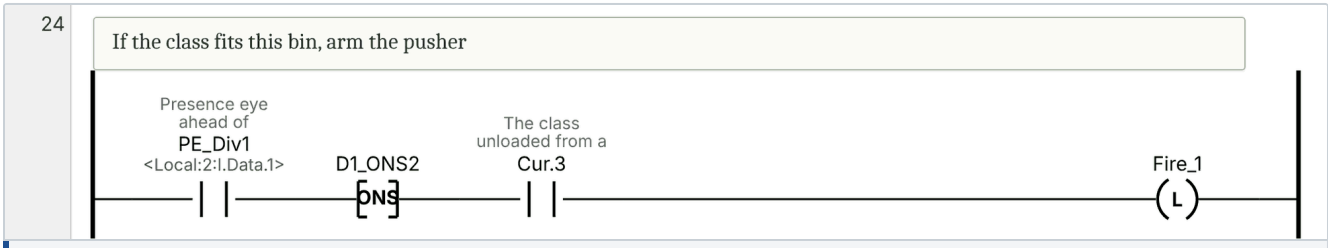
Diverter 1: broken

RUNG 23



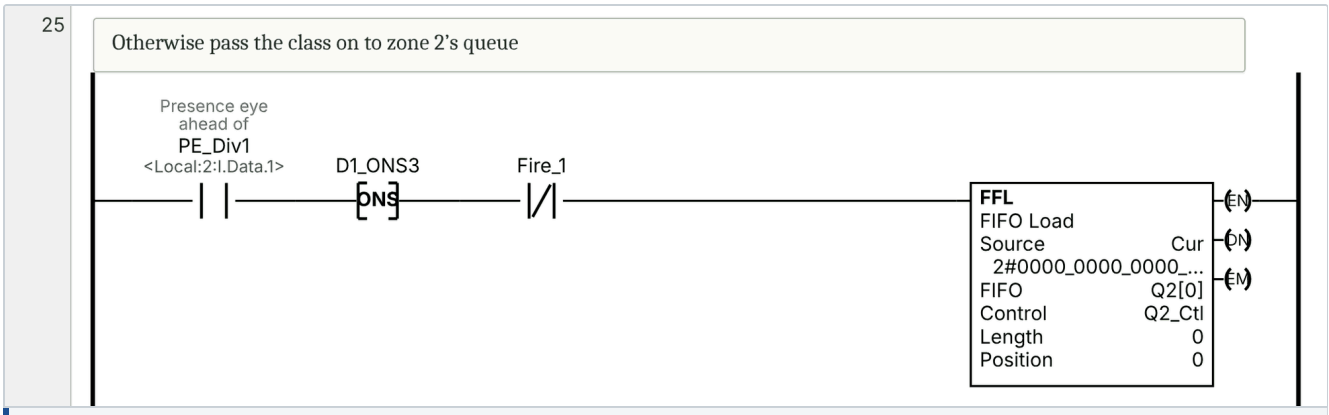

```
XIC(PE_Div1)ONS(D1_ONS)FFU(Q1[0],Cur,Q1_Ctl,8);
```

RUNG 24



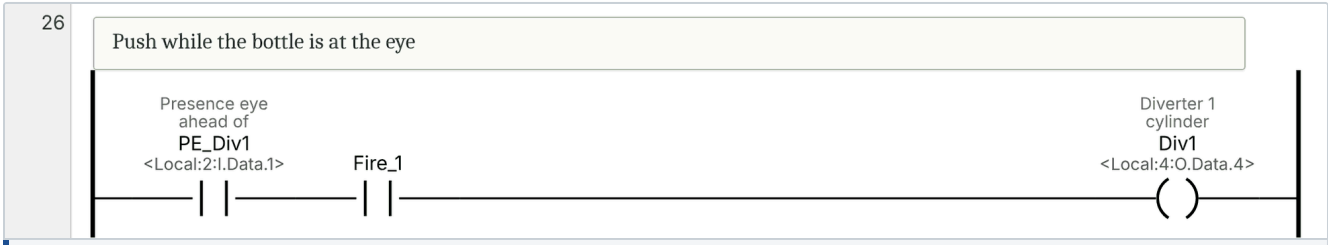

```
XIC(PE_Div1)ONS(D1_ONS2)XIC(Cur.3)OTL(Fire_1);
```

RUNG 25



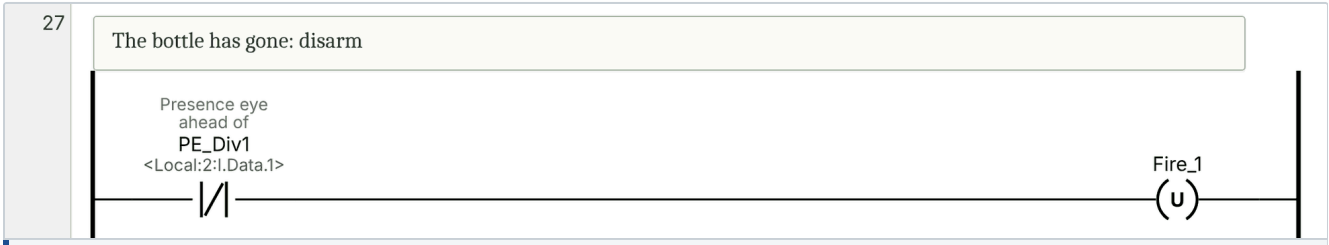

```
XIC(PE_Div1)ONS(D1_ONS3)XIO(Fire_1)FFL(Cur,Q2[0],Q2_Ctl,8);
```

RUNG 26




```
XIC(PE_Div1)XIC(Fire_1)OTE(Div1);
```

RUNG 27




```
XIO(PE_Div1)OTU(Fire_1);
```

Diverter 2: small

RUNG 28

28

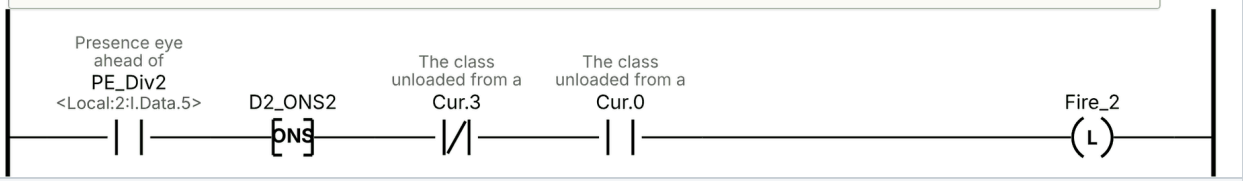
DIVERTER 2 (Small bin): a bottle arrives at its eye; the head of zone 2's queue is that bottle's class

```
XIC(PE_Div2)ONS(D2_ONS)FFU(Q2[0],Cur,Q2_Ctl,8);
```

RUNG 29

29

If the class fits this bin, arm the pusher

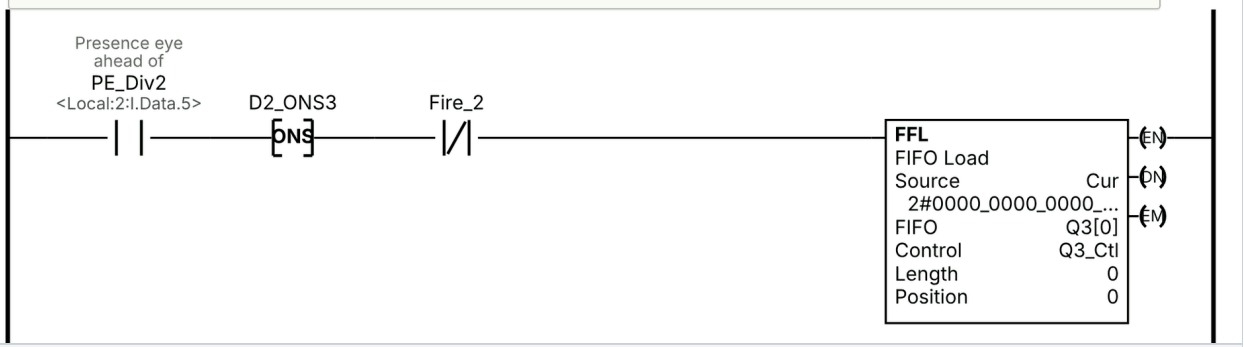



```
XIC(PE_Div2)ONS(D2_ONS2)XIO(Cur.3)XIC(Cur.0)OTL(Fire_2);
```

RUNG 30

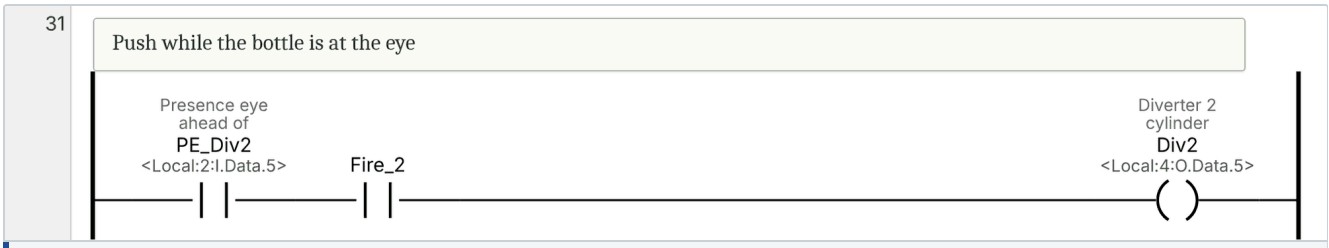
30

Otherwise pass the class on to zone 3's queue



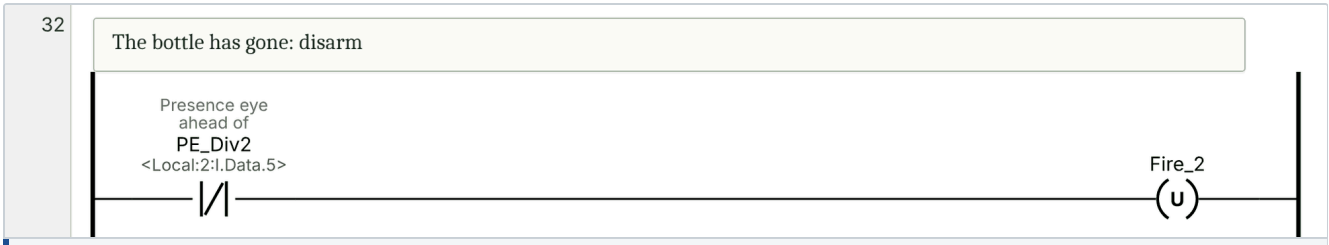

```
XIC(PE_Div2)ONS(D2_ONS3)XIO(Fire_2)FFL(Cur,Q3[0],Q3_Ctl,8);
```

RUNG 31




```
XIC(PE_Div2)XIC(Fire_2)OTE(Div2);
```

RUNG 32

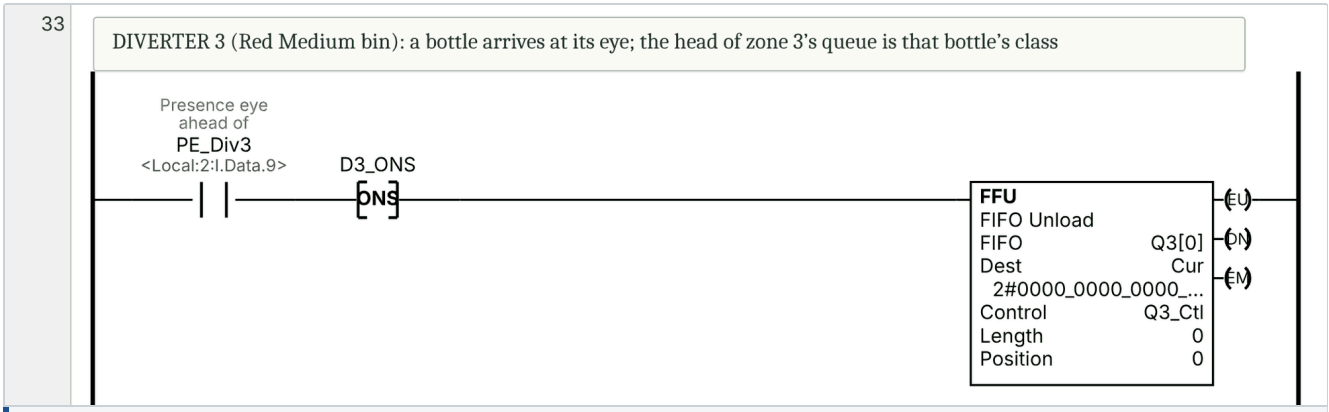



```
XIO(PE_Div2)OTU(Fire_2);
```

Any small, whole bottle, any colour: `XIO(Cur.3)XIC(Cur.0)`.

Diverters 3, 4 and 5: mediums by colour

RUNG 33

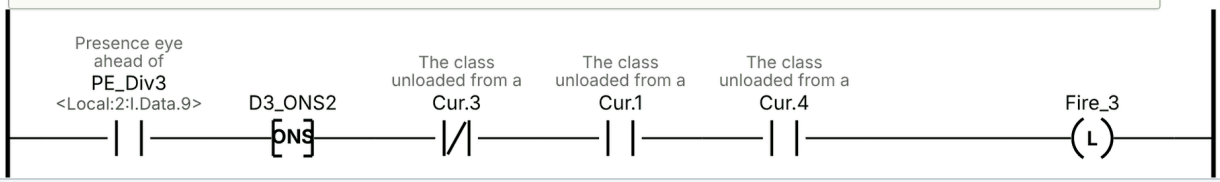



```
XIC(PE_Div3)ONS(D3_ONS)FFU(Q3[0],Cur,Q3_Ctl,8);
```

RUNG 34

34

If the class fits this bin, arm the pusher

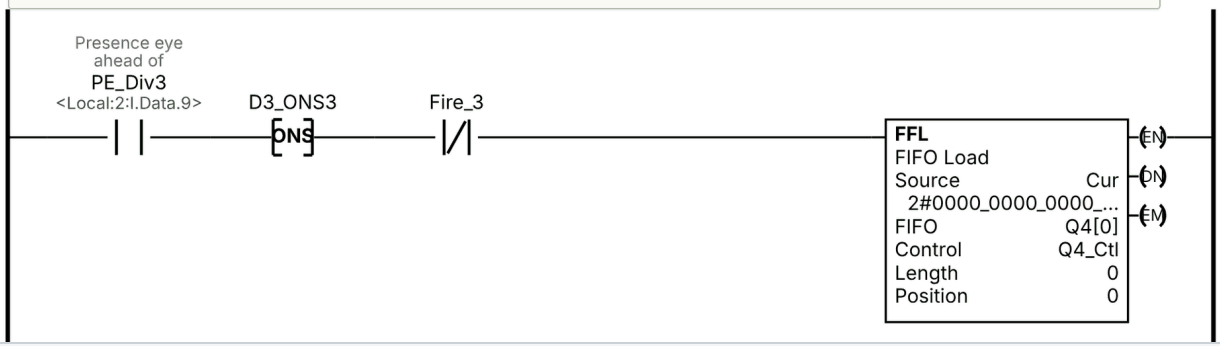



```
XIC(PE_Div3)ONS(D3_ONS2)XIO(Cur.3)XIC(Cur.1)XIC(Cur.4)OTL(Fire_3);
```

RUNG 35

35

Otherwise pass the class on to zone 4's queue

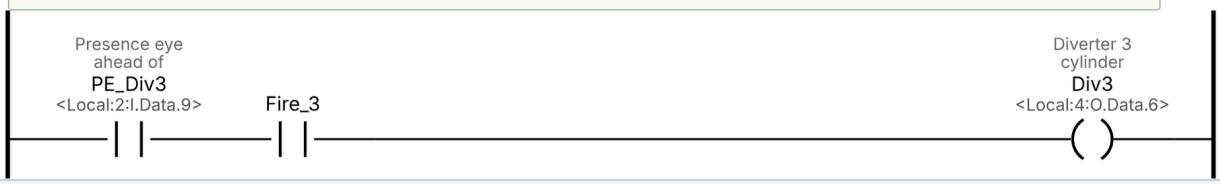



```
XIC(PE_Div3)ONS(D3_ONS3)XIO(Fire_3)FFL(Cur,Q4[0],Q4_Ctl,8);
```

RUNG 36

36

Push while the bottle is at the eye




```
XIC(PE_Div3)XIC(Fire_3)OTE(Div3);
```

RUNG 37

37

The bottle has gone: disarm

```
XIO(PE_Div3)OTU(Fire_3);
```

RUNG 38

38

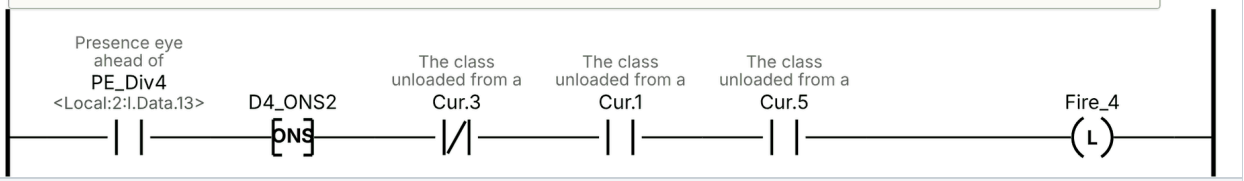
DIVERTER 4 (Blue Medium bin): a bottle arrives at its eye; the head of zone 4's queue is that bottle's class

```
XIC(PE_Div4)ONS(D4_ONS)FFU(Q4[0],Cur,Q4_Ctl,8);
```

RUNG 39

39

If the class fits this bin, arm the pusher

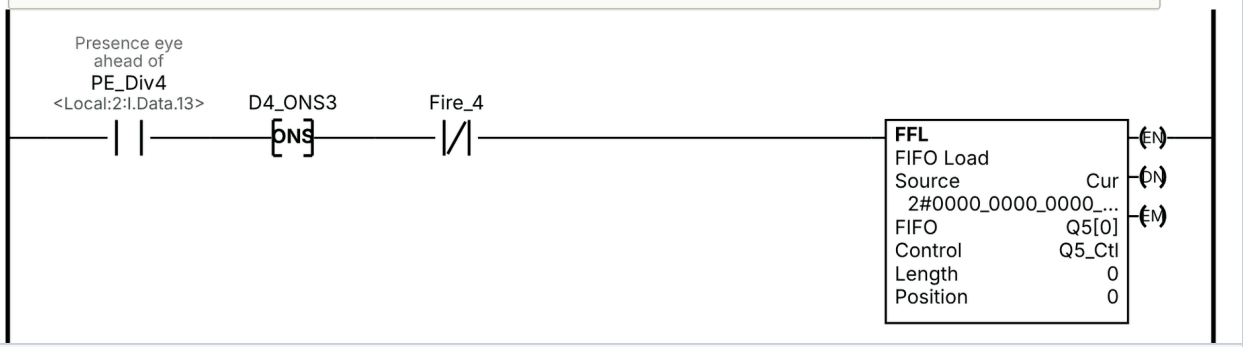



```
XIC(PE_Div4)ONS(D4_ONS2)XIO(Cur.3)XIC(Cur.1)XIC(Cur.5)OTL(Fire_4);
```

RUNG 40

40

Otherwise pass the class on to zone 5's queue

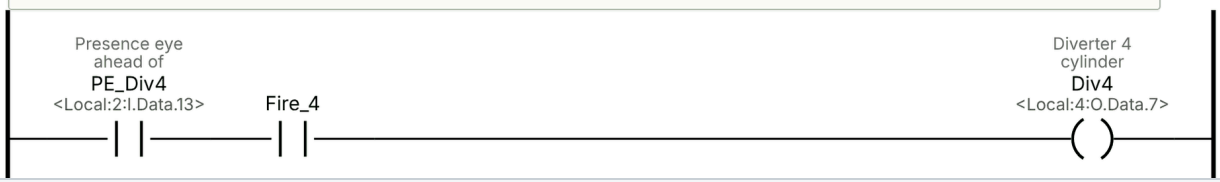



```
XIC(PE_Div4)ONS(D4_ONS3)XIO(Fire_4)FFL(Cur,Q5[0],Q5_Ctl,8);
```

RUNG 41

41

Push while the bottle is at the eye




```
XIC(PE_Div4)XIC(Fire_4)OTE(Div4);
```

RUNG 42

42

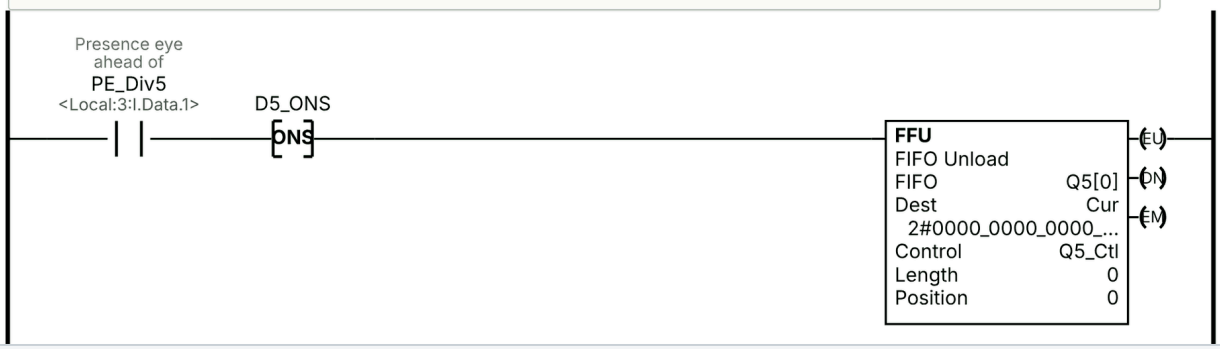
The bottle has gone: disarm

```
XIO(PE_Div4)OTU(Fire_4);
```

RUNG 43

43

DIVERTER 5 (Green Medium bin): a bottle arrives at its eye; the head of zone 5's queue is that bottle's class

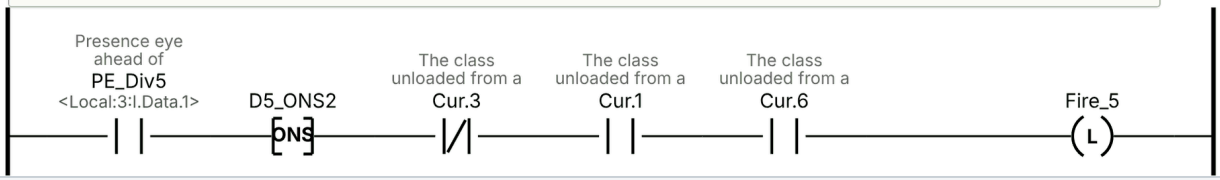



```
XIC(PE_Div5)ONS(D5_ONS)FFU(Q5[0],Cur,Q5_Ctl,8);
```

RUNG 44

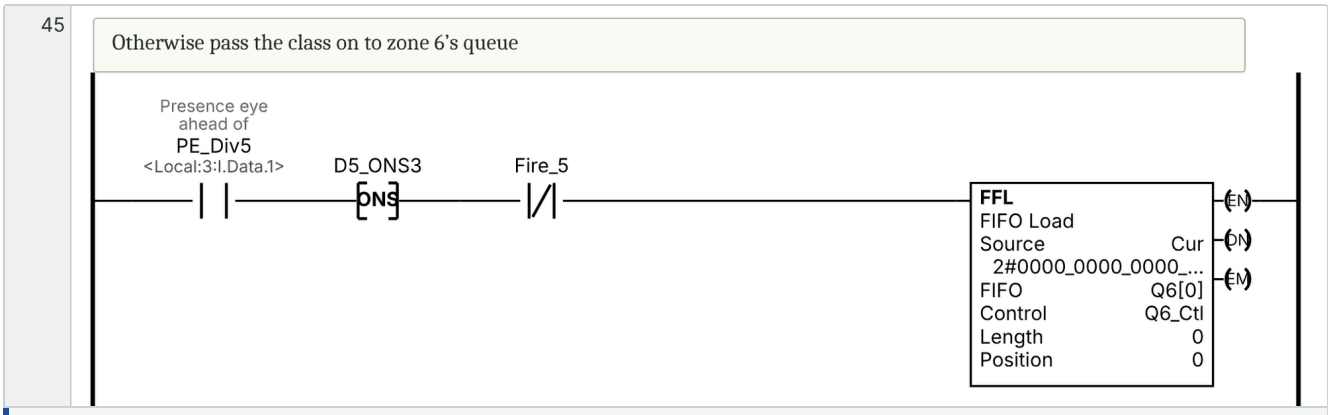
44

If the class fits this bin, arm the pusher



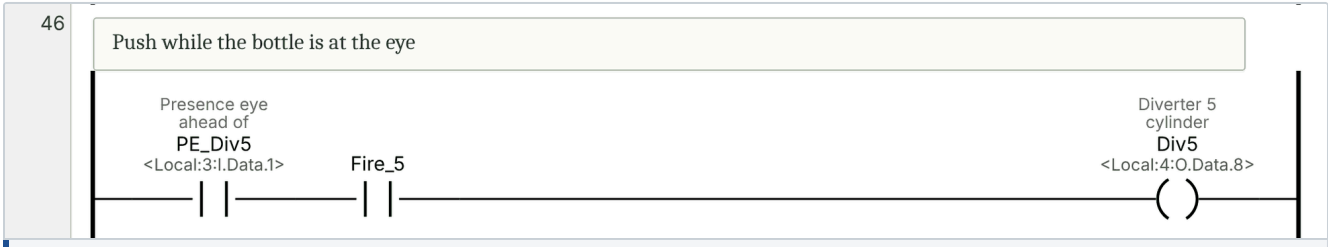

```
XIC(PE_Div5)ONS(D5_ONS2)XIO(Cur.3)XIC(Cur.1)XIC(Cur.6)OTL(Fire_5);
```

RUNG 45



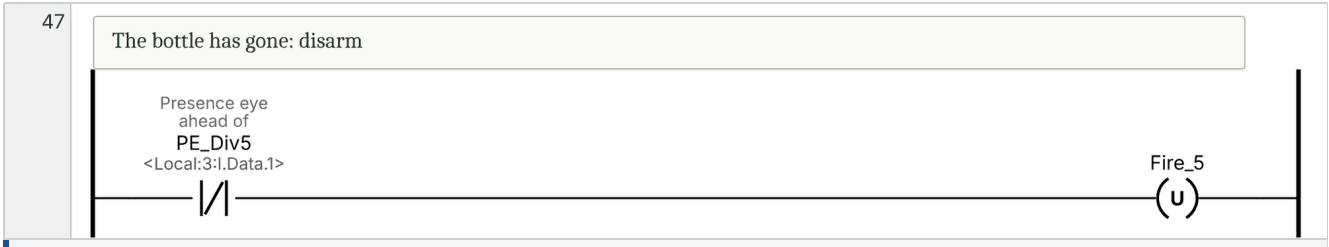

```
XIC(PE_Div5)ONS(D5_ONS3)XIO(Fire_5)FFL(Cur,Q6[0],Q6_Ctl,8);
```

RUNG 46




```
XIC(PE_Div5)XIC(Fire_5)OTE(Div5);
```

RUNG 47




```
XIO(PE_Div5)OTU(Fire_5);
```

Each tests three bits: not broken, medium, and its colour — `XIO(Cur.3)XIC(Cur.1)XIC(Cur.4)` for red, `…XIC(Cur.5)` for blue, `…XIC(Cur.6)` for green. A medium bottle that is not the colour of bin 3 is passed on to zone 4, and so on, until the bin that wants it.

Diverter 6: large

RUNG 48

48

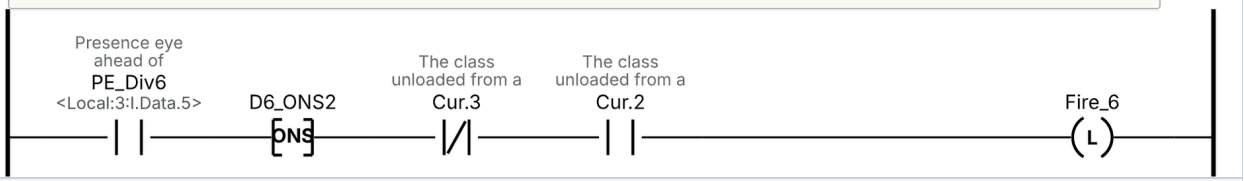
DIVERTER 6 (Large bin): a bottle arrives at its eye; the head of zone 6's queue is that bottle's class

```
XIC(PE_Div6)ONS(D6_ONS)FFU(Q6[0],Cur,Q6_Ctl,8);
```

RUNG 49

49

If the class fits this bin, arm the pusher

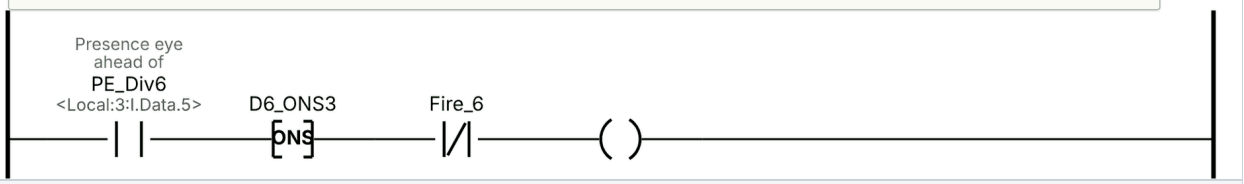



```
XIC(PE_Div6)ONS(D6_ONS2)XIO(Cur.3)XIC(Cur.2)OTL(Fire_6);
```

RUNG 50

50

Otherwise it runs on to the end of the belt




```
XIC(PE_Div6)ONS(D6_ONS3)XIO(Fire_6)NOP();
```

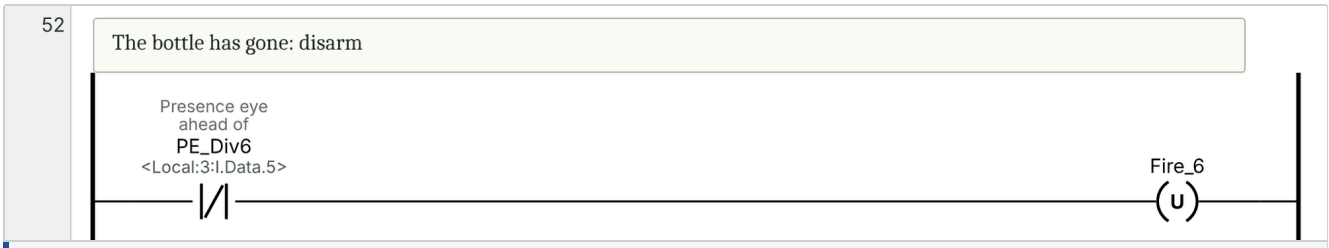
RUNG 51

51

Push while the bottle is at the eye

```
XIC(PE_Div6)XIC(Fire_6)OTE(Div6);
```

RUNG 52




```
XIO(PE_Div6)OTU(Fire_6);
```

WATCH FOR: A MATCH THAT TESTS TOO LITTLE

Write the red-medium rung as `XIC(Cur.4)` alone and it looks right in a test with only medium bottles. Then a *large* red bottle comes along — and bin 3 is upstream of bin 6, so it gets there first and takes it: −50. Bin 2 already took every small bottle, so a small red bottle would never reach bin 3 in this order — but reorder the bins and that hides a second bug. Write every match with every bit it depends on, and the order of the bins stops mattering.

Run it

Sixteen bottles into six bins. Turn on Specs to see each bin's rule, and keep the Simulator Log open: every line names the bottle, its colour and size, and the bin. The reference run: 16 of 16 bottles in the right bin, nothing lost, in 95 s against a par of 120 s: 1800 points and three stars.

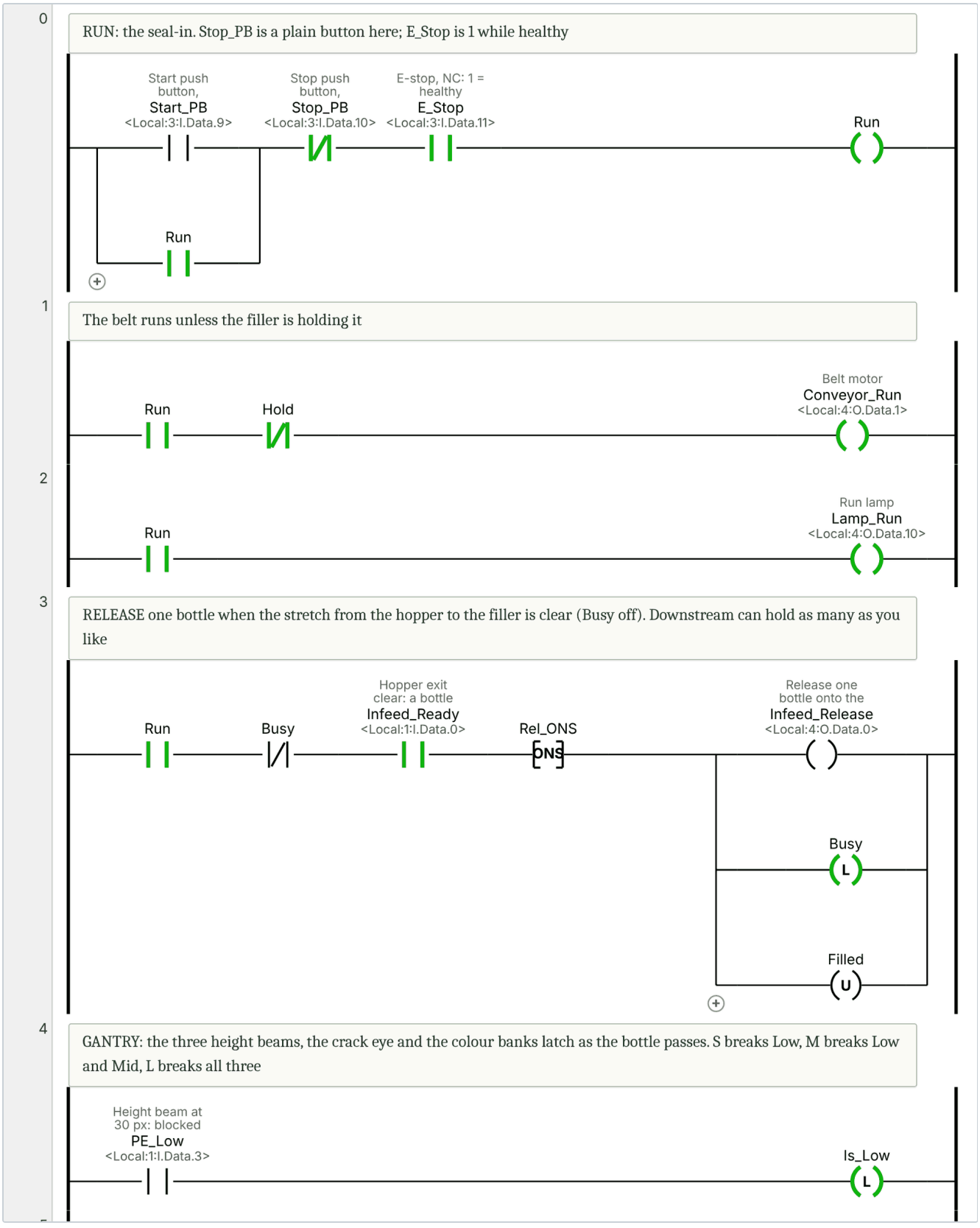


Figure 124. Level 5 online, the top of the routine.

TRY IT

Give a bin of your own a grading rule: drop a seventh pusher and bin at the end of the line for *yellow* bottles, set the hopper's colour mix to *random*, and extend the program — a colour bank for yellow, a bit 7 in the class word, five rungs for the new diverter. Then the result table below has a row that is yours.

37 Level 6 — Pack it

> **IN THE APP**
>
> File → New Project… → Level on sandbox parts → **6**. The level opens with guided mode on this chapter: the tags to make, every rung below under its fold, a Verify check, and five results it ticks while you run — a case stops on the mark, a case is packed, the level completes, every bottle right, under par.

Sixteen bottles. The broken ones go into the scrap bin as before; every other bottle is filled and then **packed**: the belt ends in the case packer of step 12 (chapter 28), and a case lane on a VFD runs beside it. Four bottles to a case, and when the hopper is empty whatever is left in the infeed goes out as a part case. Par 140 s.

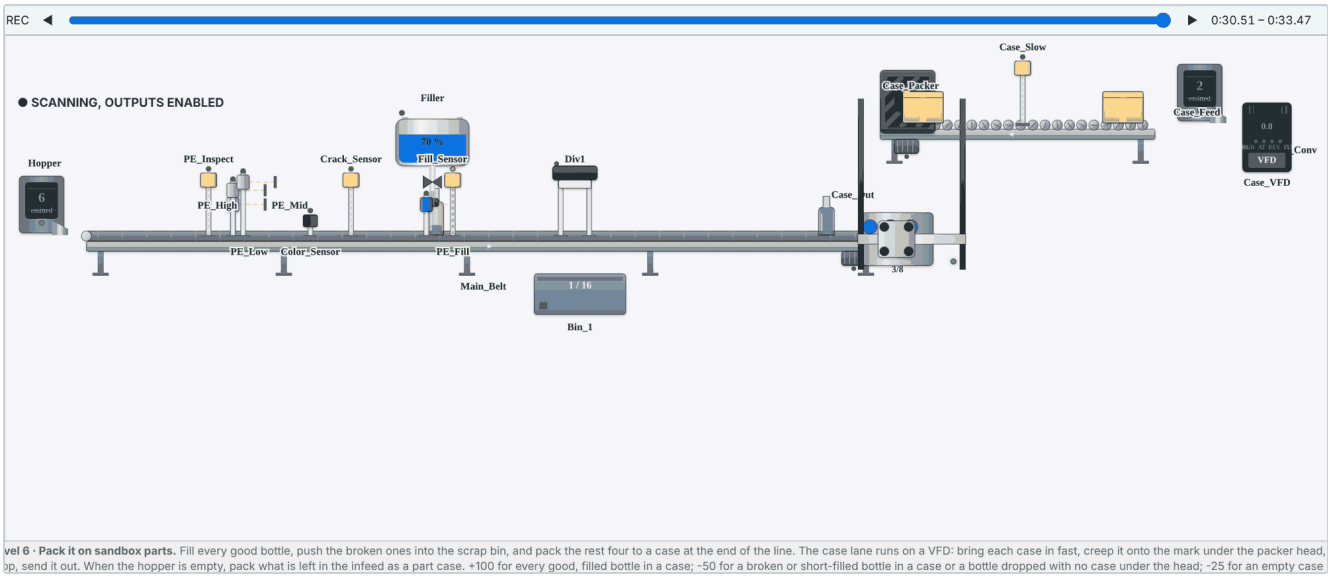


Figure 125. Level 6: the hopper, the gantry, the filler, one scrap pusher and bin, the packer at the end of the belt, its case lane on the far side with the slow-down eye and the case feeder, the VFD, the console.

How the level is scored

What happens	Points
A good, filled bottle packed into a case	+100
A broken bottle in the scrap bin	+100
A broken or short-filled bottle packed into a case	−50
A whole bottle in the scrap bin	−50
Bottles dropped by the packer with no case under the head (each)	−50

What happens	Points
An empty case shipped at Case_Out	−25
Complete under par (140 s)	+200

The level is complete when the hopper is empty, no bottle is left on the belt or in the packer's infeed, and every packed case has reached Case_Out. Three stars: no mistakes and under par.

How to pass it, step by step

1. **Start from Level 5.** Run circuit, release one bottle at a time to the filler, latch the gantry into a class word, fill to 97 %, and load the class into zone 1's queue as the bottle leaves the filler. Copy those 23 rungs; they are unchanged (the colour and size bits are not needed here, but they do no harm).
2. **One diverter only.** Diverter 1 takes broken bottles into the scrap bin. A whole bottle is not pushed: it runs on to the end of the belt, where the packer's infeed takes it. Nothing else is needed for the infeed — it fills itself.
3. **Run the case lane.** `Case_Run` while no empty case is under the head: `[XIO(Case_In_Place),XIC(Case_Full)]`. Let it run while the line runs *or* while a full group waits, so the last cases still come in after you stop.
4. **Two speeds and a latch.** 60 Hz normally; 15 Hz from the slow-down eye until an empty case is on the mark, latched, and also while a packed case leaves. Without the creep the case overshoots on the drive's decel ramp and leaves empty: −25 every time.
5. **Scale to the analog output.** `SCP(Case_Hz,0,60,0,10000,Case_Speed_Ref)`. Writing 60 or 15 straight to Case_Speed_Ref gives the drive 0.06 V or 0.015 V — it barely moves.
6. **Drop only on a stopped, empty case.** Group ready, case in place, case not full, `XIO(Case_VFD_Running)`, packer ready, one-shot. Dropping while the case is still ramping down misses it: −50 per bottle.
7. **Finish the batch.** Count the releases with a CTU preset 16; when it is done and the last bottle has left the filler, wait 15 s for it to reach the packer, then drop whatever is in the infeed as a part case. Without this the last two or three bottles sit in the infeed and the level never completes.

Tags this lesson needs

The level's alias tags are already made for you (chapter 30's table). Create these plain tags before you type the rungs; Verify reports any you miss.

Tag	Type	How to make it	Description
`Run`	BOOL	Edit Tags: name, type	Seal-in run bit
`Busy`	BOOL	Edit Tags: name, type	A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)

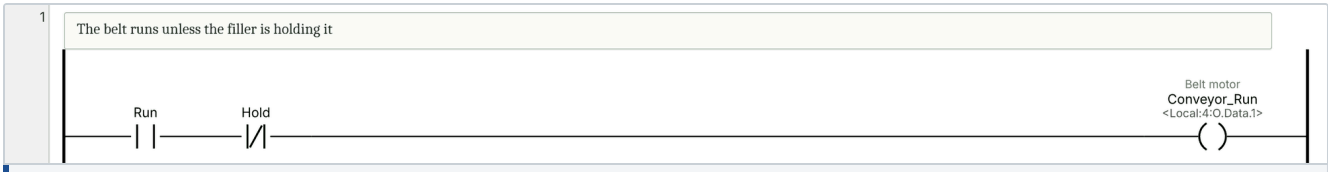
Tag	Type	How to make it	Description
`Rel_ONS`	BOOL	Edit Tags: name, type	One-shot storage for the release
`Hold`	BOOL	Edit Tags: name, type	The filler holds the belt
`Filled`	BOOL	Edit Tags: name, type	This bottle has its charge
`AtFill`	BOOL	Edit Tags: name, type	The eye under the nozzle has seen this bottle
`Leave_ONS`	BOOL	Edit Tags: name, type	One-shot storage: the bottle left the filler
`Is_Low`	BOOL	Edit Tags: name, type	Latched: the low height beam was broken
`Is_Mid`	BOOL	Edit Tags: name, type	Latched: the middle beam was broken
`Is_High`	BOOL	Edit Tags: name, type	Latched: the high beam was broken
`Is_Broken`	BOOL	Edit Tags: name, type	Latched: the crack eye saw a broken bottle
`Is_R`	BOOL	Edit Tags: name, type	Latched: the colour sensor saw red
`Is_B`	BOOL	Edit Tags: name, type	Latched: the colour sensor saw blue
`Is_G`	BOOL	Edit Tags: name, type	Latched: the colour sensor saw green
`Class`	DINT	Edit Tags: name, type, Style Binary	The bottle in the gantry: bit 0 S, 1 M, 2 L, 3 broken, 4 red, 5 blue, 6 green
`Cur`	DINT	Edit Tags: name, type, Style Binary	The class unloaded from a zone queue at a diverter
`Q1`	DINT[8]	Edit Tags: name, type, Dim 8	Zone 1 queue: classes of the bottles between the filler and diverter 1
`Q1_Ctl`	CONTROL	Edit Tags: name, type	Queue control
`D1_ONS`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`D1_ONS2`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`D1_ONS3`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`Fire_1`	BOOL	Edit Tags: name, type	Armed: this diverter will push the bottle at its eye
`Case_Creep`	BOOL	Edit Tags: name, type	Latched from the slow-down eye until an empty case stands under the head
`Case_Hz`	REAL	Edit Tags: name, type	Case lane speed wanted, Hz
`Case_Hz_Actual`	REAL	Edit Tags: name, type	Case lane drive output frequency read back, Hz
`Drop_ONS`	BOOL	Edit Tags: name, type	
`Case_ONS`	BOOL	Edit Tags: name, type	
`Released`	COUNTER	Edit Tags: name, type	Bottles released from the hopper
`Tail_T`	TIMER	Edit Tags: name, type	After the last release: time for the last bottle to reach the packer

Run, release, gantry, class word, fill and leaving the filler

RUNG 0

```
[XIC(Start_PB),XIC(Run)]XIO(Stop_PB)XIC(E_Stop)OTE(Run);
```

RUNG 1

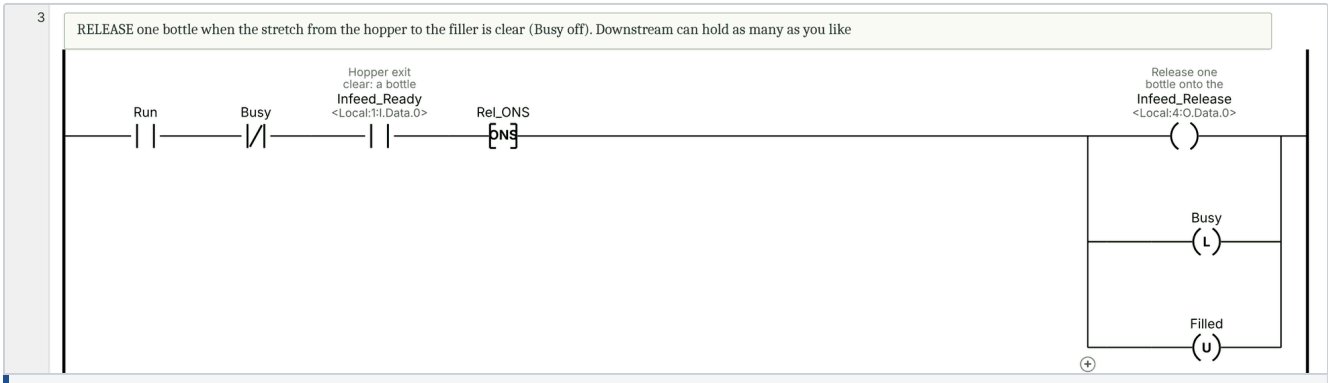



```
XIC(Run)XIO(Hold)OTE(Conveyor_Run);
```

RUNG 2

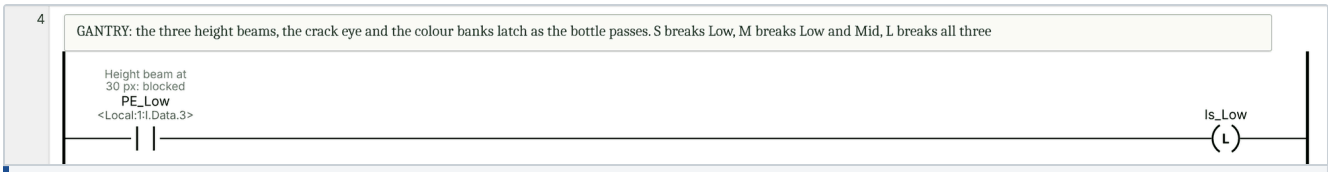
```
XIC(Run)OTE(Lamp_Run);
```

RUNG 3




```
XIC(Run)XIO(Busy)XIC(Infeed_Ready)ONS(Rel_ONS)
[OTE(Infeed_Release),OTL(Busy),OTU(Filled)];
```

RUNG 4




```
XIC(PE_Low)OTL(Is_Low);
```

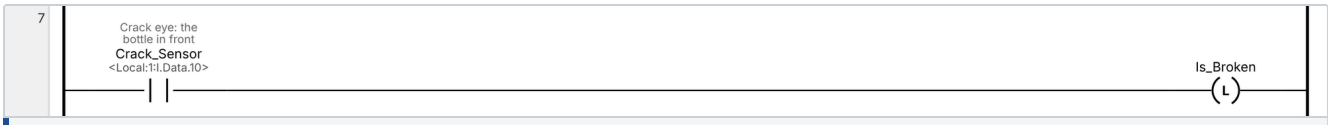
RUNG 5

```
XIC(PE_Mid)OTL(Is_Mid);
```

RUNG 6

```
XIC(PE_High)OTL(Is_High);
```

RUNG 7




```
XIC(Crack_Sensor)OTL(Is_Broken);
```

RUNG 8

```
XIC(Color_Red)OTL(Is_R);
```

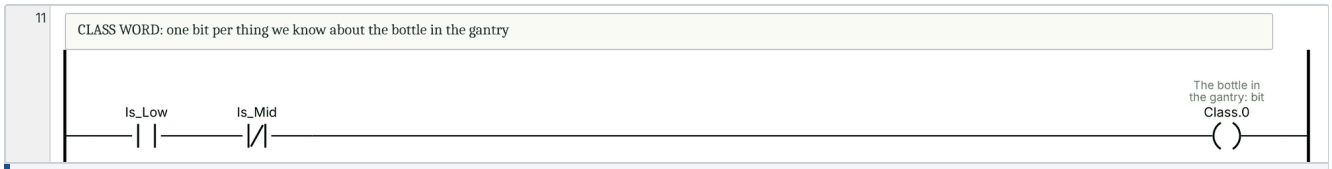
RUNG 9

```
XIC(Color_Blue)OTL(Is_B);
```

RUNG 10

```
XIC(Color_Green)OTL(Is_G);
```

RUNG 11

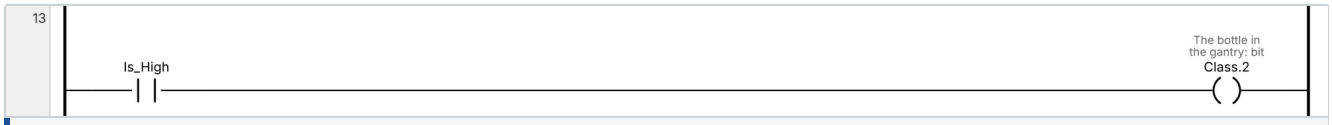



```
XIC(Is_Low)XIO(Is_Mid)OTE(Class.0);
```

RUNG 12

```
XIC(Is_Mid)XIO(Is_High)OTE(Class.1);
```

RUNG 13




```
XIC(Is_High)OTE(Class.2);
```

RUNG 14

```
XIC(Is_Broken)OTE(Class.3);
```

RUNG 15

```
XIC(Is_R)OTE(Class.4);
```

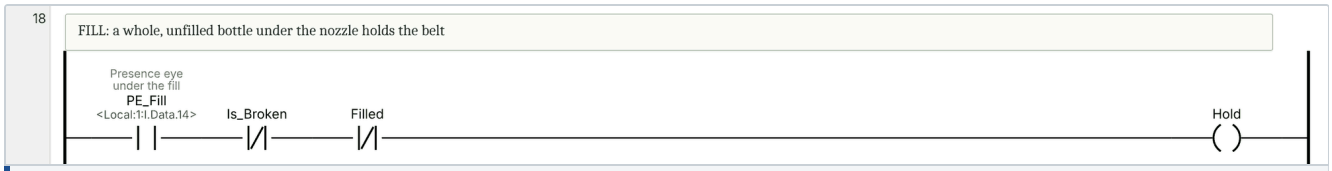
RUNG 16

```
XIC(Is_B)OTE(Class.5);
```

RUNG 17

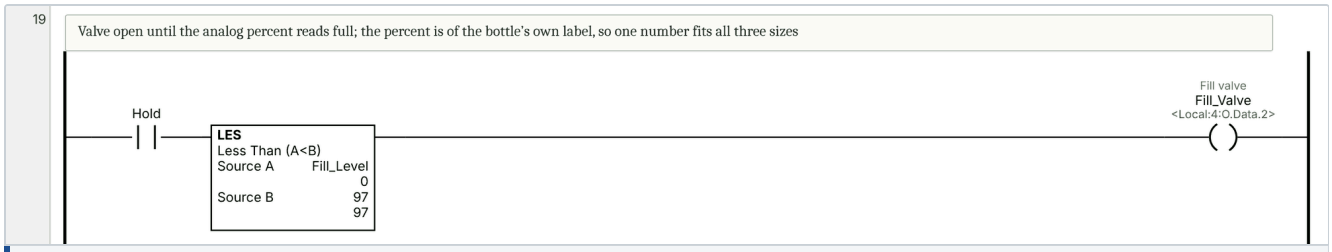
```
XIC(Is_G)OTE(Class.6);
```

RUNG 18




```
XIC(PE_Fill)XIO(Is_Broken)XIO(Filled)OTE(Hold);
```

RUNG 19

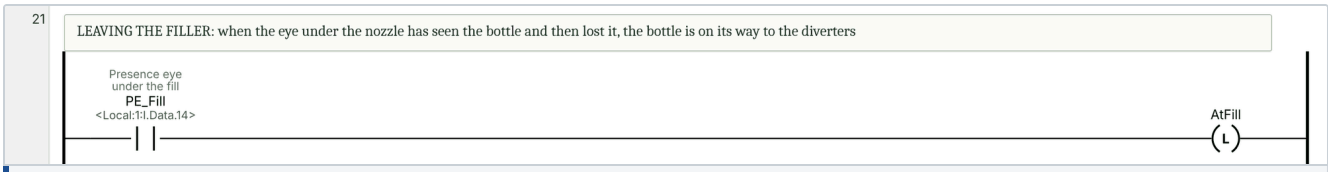



```
XIC(Hold)LES(Fill_Level,97)OTE(Fill_Valve);
```

RUNG 20

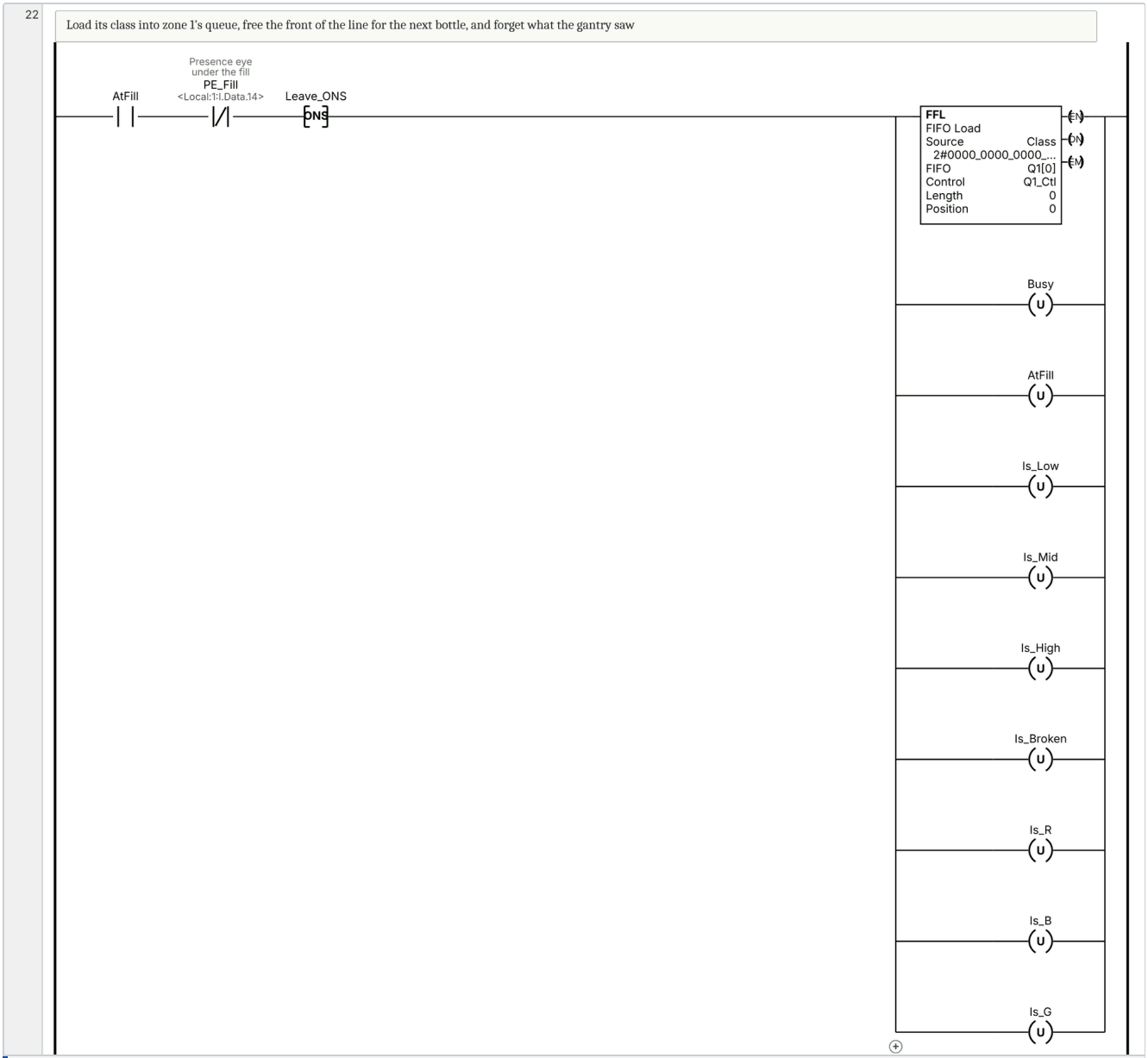
```
XIC(Hold)GEQ(Fill_Level,97)OTL(Filled);
```

RUNG 21




```
XIC(PE_Fill)OTL(AtFill);
```

RUNG 22

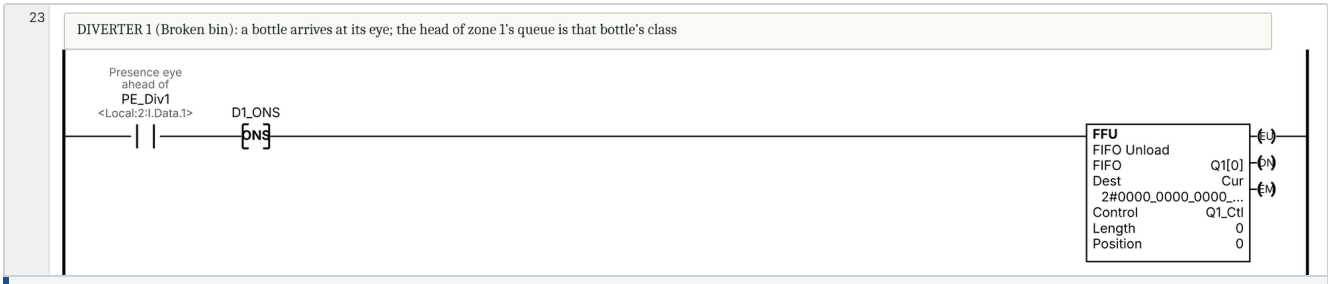



```
XIC(AtFill)XIO(PE_Fill)ONS(Leave_ONS)
[FFL(Class,Q1[0],Q1_Ctl,8),OTU(Busy),OTU(AtFill),OTU(Is_Low),OTU(Is_Mid),OTU(Is
_High),OTU(Is_Broken),OTU(Is_R),OTU(Is_B),OTU(Is_G)];
```

Level 5's first 23 rungs, word for word.

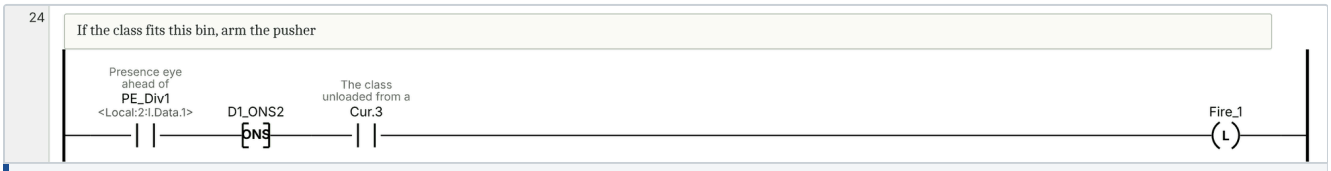
Diverter 1: broken bottles to the scrap bin

RUNG 23



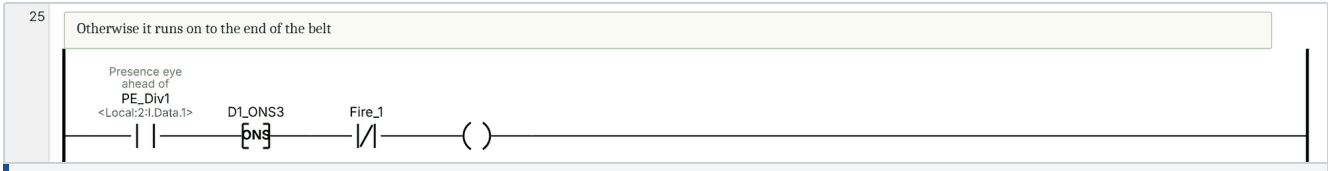

```
XIC(PE_Div1)ONS(D1_ONS)FFU(Q1[0],Cur,Q1_Ctl,8);
```

RUNG 24



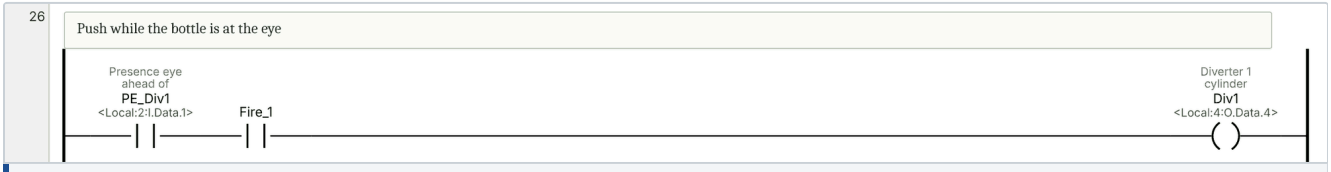

```
XIC(PE_Div1)ONS(D1_ONS2)XIC(Cur.3)OTL(Fire_1);
```

RUNG 25



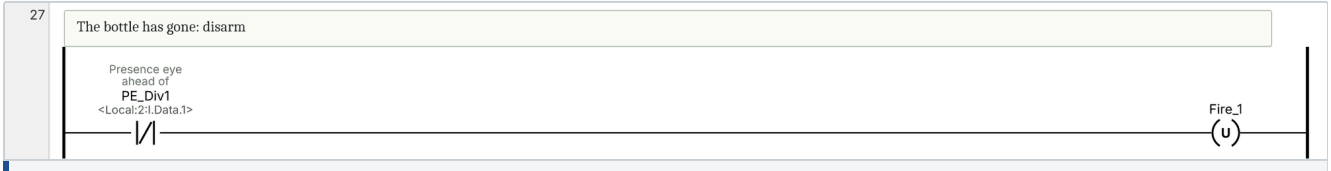

```
XIC(PE_Div1)ONS(D1_ONS3)XIO(Fire_1)NOP();
```

RUNG 26




```
XIC(PE_Div1)XIC(Fire_1)OTE(Div1);
```

RUNG 27

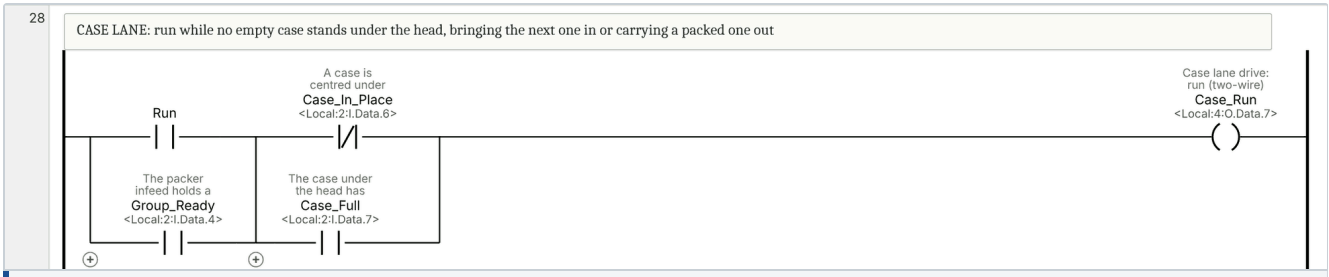



```
XIO(PE_Div1)OTU(Fire_1);
```

The last diverter's "otherwise" rung passes nothing on: a whole bottle simply runs to the end of the belt.

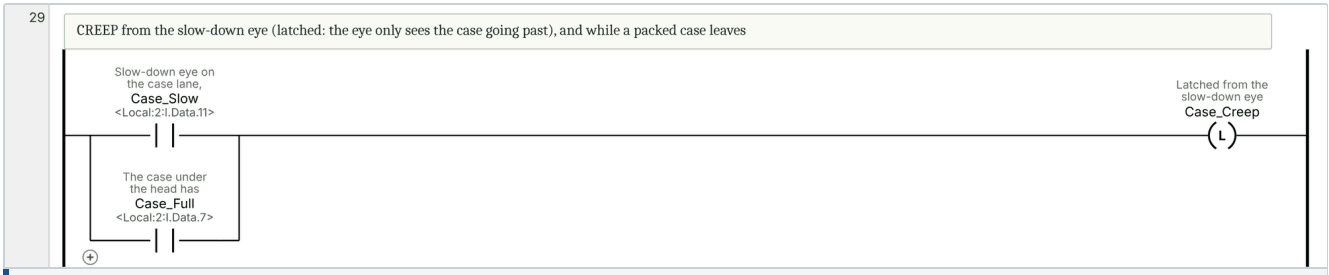
The case lane: run, creep, speed and scaling

RUNG 28



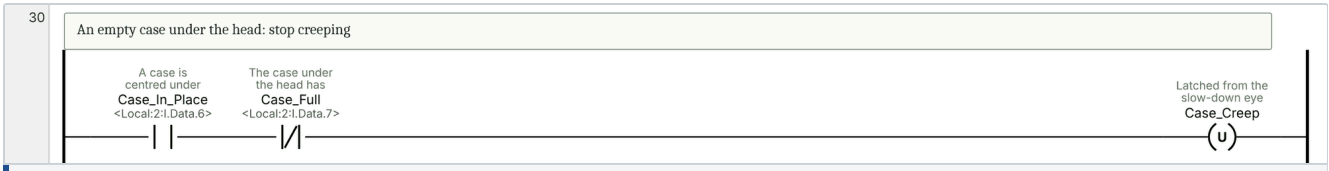

```
[XIC(Run),XIC(Group_Ready)][XIO(Case_In_Place),XIC(Case_Full)]OTE(Case_Run);
```

RUNG 29




```
[XIC(Case_Slow),XIC(Case_Full)]OTL(Case_Creep);
```

RUNG 30

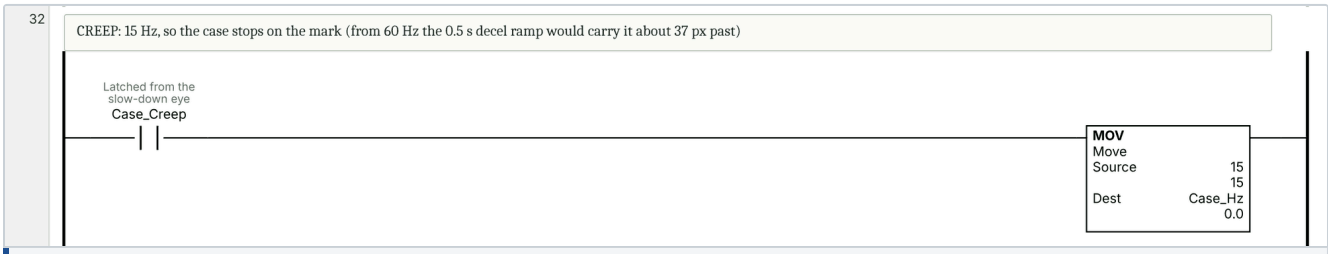



```
XIC(Case_In_Place)XIO(Case_Full)OTU(Case_Creep);
```

RUNG 31

```
MOV(60,Case_Hz);
```

RUNG 32

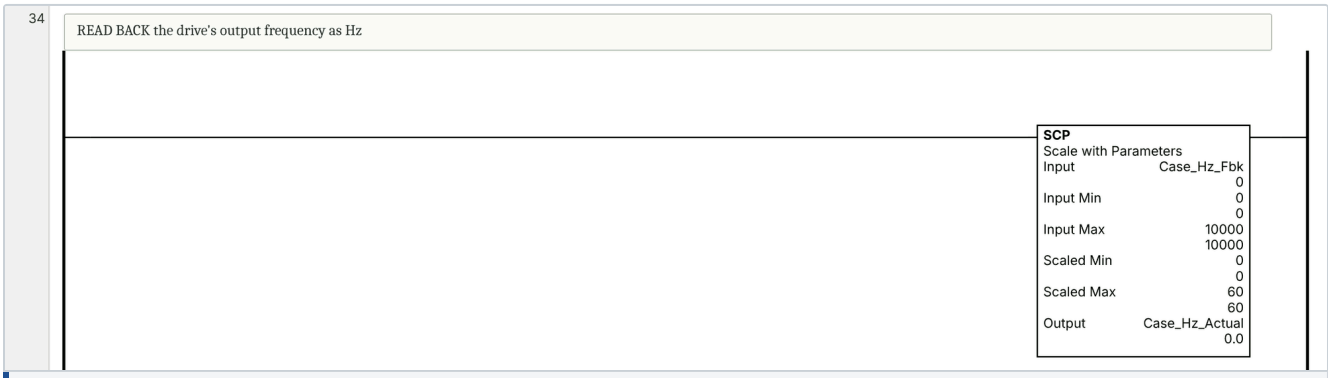



```
XIC(Case_Creep)MOV(15,Case_Hz);
```

RUNG 33

```
SCP(Case_Hz,0,60,0,10000,Case_Speed_Ref);
```

RUNG 34

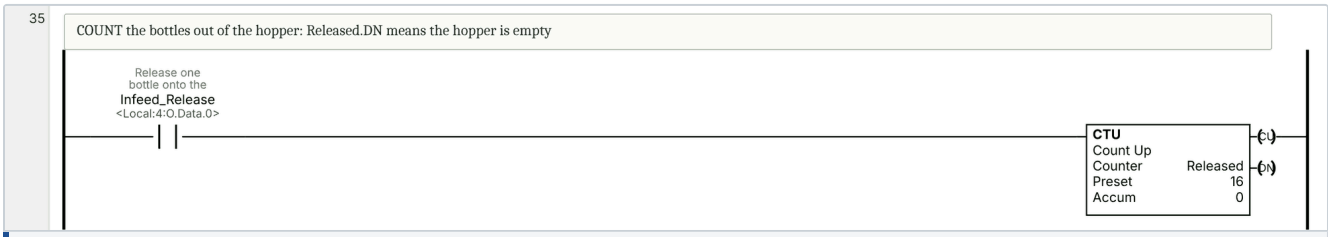



```
SCP(Case_Hz_Fbk,0,10000,0,60,Case_Hz_Actual);
```

The same seven rungs as the bottle line's step 12, with `Run` in place of `Line_Run`: run while no empty case is on the mark, a latched creep from the slow-down eye, 60 Hz or 15 Hz by the overwrite pattern, SCP out to the 0-10 V reference and back from the drive's output frequency.

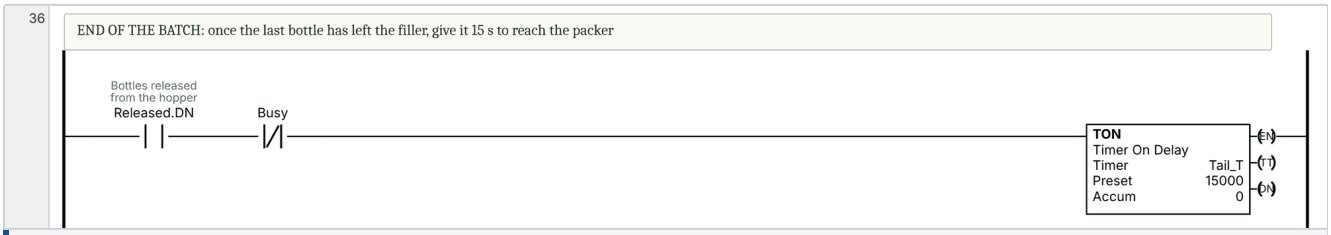
Counting the batch and packing

RUNG 35



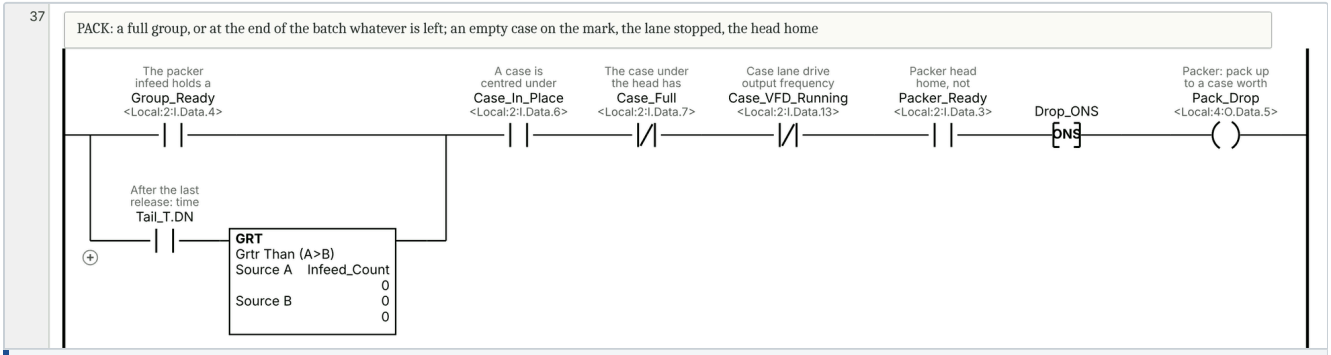

```
XIC(Infeed_Release)CTU(Released,16,0);
```

RUNG 36



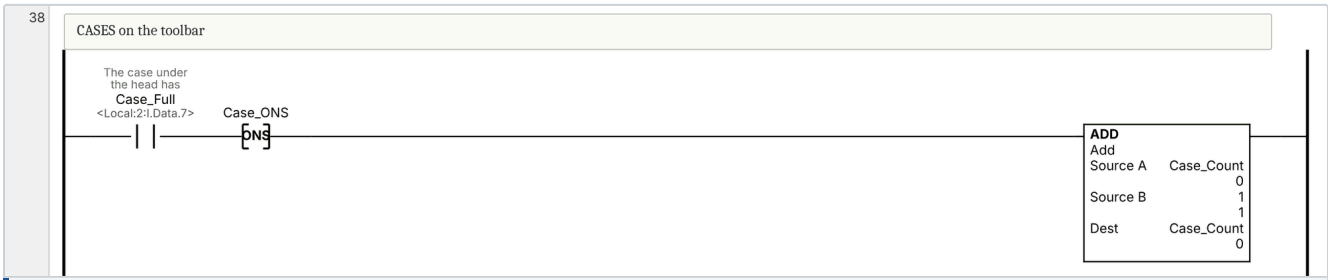

```
XIC(Released.DN)XIO(Busy)TON(Tail_T,15000,0);
```

RUNG 37




```
[XIC(Group_Ready),XIC(Tail_T.DN)GRT(Infeed_Count,0)]XIC(Case_In_Place)XIO(Case_Full)XIO(Case_VFD_Running)XIC(Packer_Ready)ONS(Drop_ONS)OTE(Pack_Drop);
```

RUNG 38




```
XIC(Case_Full)ONS(Case_ONS)ADD(Case_Count,1,Case_Count);
```

The CTU counts rising edges of `Infeed_Release`; at 16 the hopper is empty. **The TON** starts when the last bottle has left the filler (`Busy` off) and gives it 15 s to run down the belt — the belt is 780 px long at 80 px/s, so about 5 s is needed; 15 s is generous and costs nothing, because the timer only starts at the very end. **The PACK rung** drops on a full group as usual, or, once the tail timer is done, on whatever is left in the infeed (`GRT(Infeed_Count,0)`): the packer takes up to four, so the part case gets the last one, two or three.

> **WATCH FOR: THE LEVEL THAT NEVER ENDS**
>
> Sixteen bottles minus the broken ones is rarely a multiple of four. Pack only on `Group_Ready` and the last bottles wait in the infeed for a group that never comes: the hopper is empty, the belt is clear, and the level does not complete. A real line has the same problem at the end of every batch — the last case is a part case, and someone has to decide to send it.

Run it

Load your program, Run mode, Start. Watch the first case come in fast, slow at the eye, stop on the mark; bottles collect in the infeed (the count on the packer's table), and the fourth starts the head. The Simulator Log shows every pack and every scrap-bin push, with points. The reference run: 16 of 16 bottles in the right bin, nothing lost, in 111 s against a par of 140 s: 1800 points and three stars.

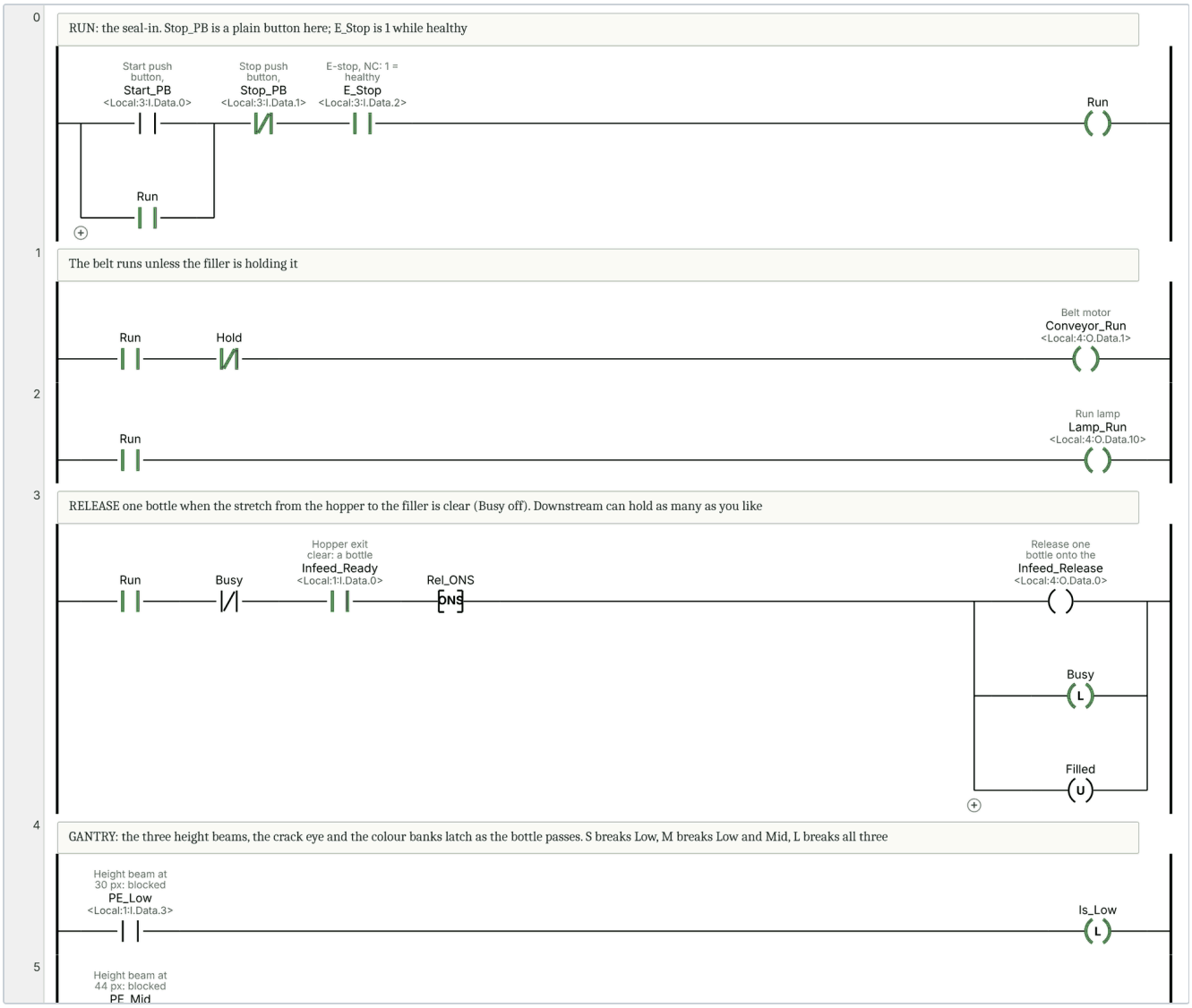


Figure 126. Level 6 online, the top of the routine.

TRY IT

Go for a faster time: the line stops at the filler for every fill, so the packer is never the bottleneck — or is it? Set the case feeder's interval to 8 s and find out. Then pack six to a case (the packer's *Bottles per case*, and the CTU and part-case logic still work unchanged).

38 Level 7 — Batch Mix

> **IN THE APP**
>
> File → New Project… → Level on sandbox parts → **7**. Guided mode opens on this chapter: the tags, every rung below under its fold, a Verify check, and five results — pump A stops on its count and B tops up to HI, the batch heats with the mixer running, all three batches made, every batch on spec, under par.

No belt, no bottles. One **Mix Tank** — the new part of Version 97, built after the LogixPro Batch Mix — and a console. Make three batches, one of each recipe, in the order the **recipe selector** shows: the level turns the selector to the next recipe each time a batch drains. Par 240 s.

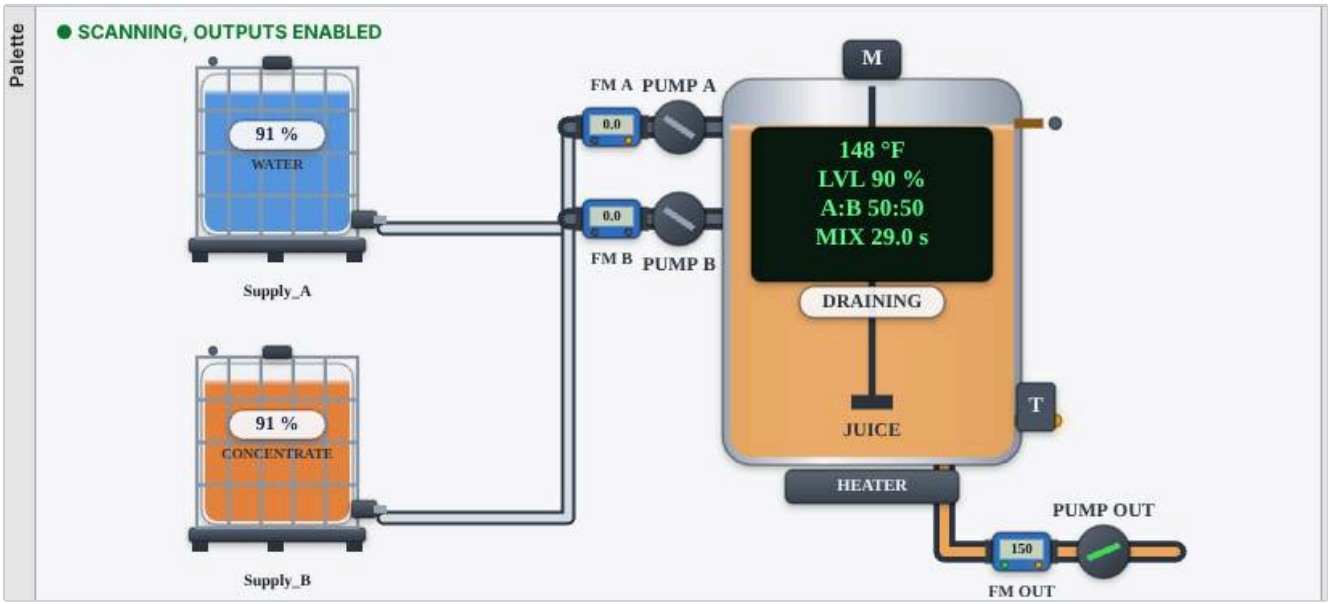


Figure 127. Level 7: the two supply totes piped to the mix tank's inlet pumps and flowmeters (the pipes light in the liquid's colour while it moves), the heater under the tank, the mixer on the lid, HI and LO probes and the thermostat on the wall, the outlet pump and flowmeter.

The tank

- **Pump_A**, **Pump_B** — the two ingredients, 100 units/s each. Each runs through a flowmeter: **Flow_A**, **Flow_B** give one pulse per 20 units, a square wave, so a CTU on the input counts the dose.
- **Hi_Level**, **Lo_Level** — probes, 1 while the liquid covers them (90 % and 10 %). **Tank_Level** is the transmitter, 0-100 %; the tank holds 2000 units, so HI is 1800.
- **Heater** — under the tank, about 3 °F a second while the element is covered. The tank loses heat to the room all the time, faster the hotter it is. Run it below LO and the element is uncovered: the batch is scorched.

- **Tank_Temp** — °F, an analog input. **Thermostat** is a separate high-limit switch that closes at 180 °F: a safety, not the control.
- **Mixer** — mixing time counts only while the impeller is covered.
- **Pump_Out** — empties the tank at 150 units/s through its own flowmeter. (Pipe the tank to a filler and the filler draws through this pump instead — see Try it.)
- The operator stations: the console and **Tank_HMI**, whose START and STOP pulse **HMI_Start** and **HMI_Stop**, also on the Operator View tab (chapter 14).
- The console: **Start_PB**, **Stop_PB**, **E_Stop**, **Recipe_A** and **Recipe_C** from the three-position selector (neither means B), **Lamp_Run**, **Lamp_Idle**, **Lamp_Full**, **Lamp_Fault**, and the BATCHES and °F displays.

Where the ingredients come from

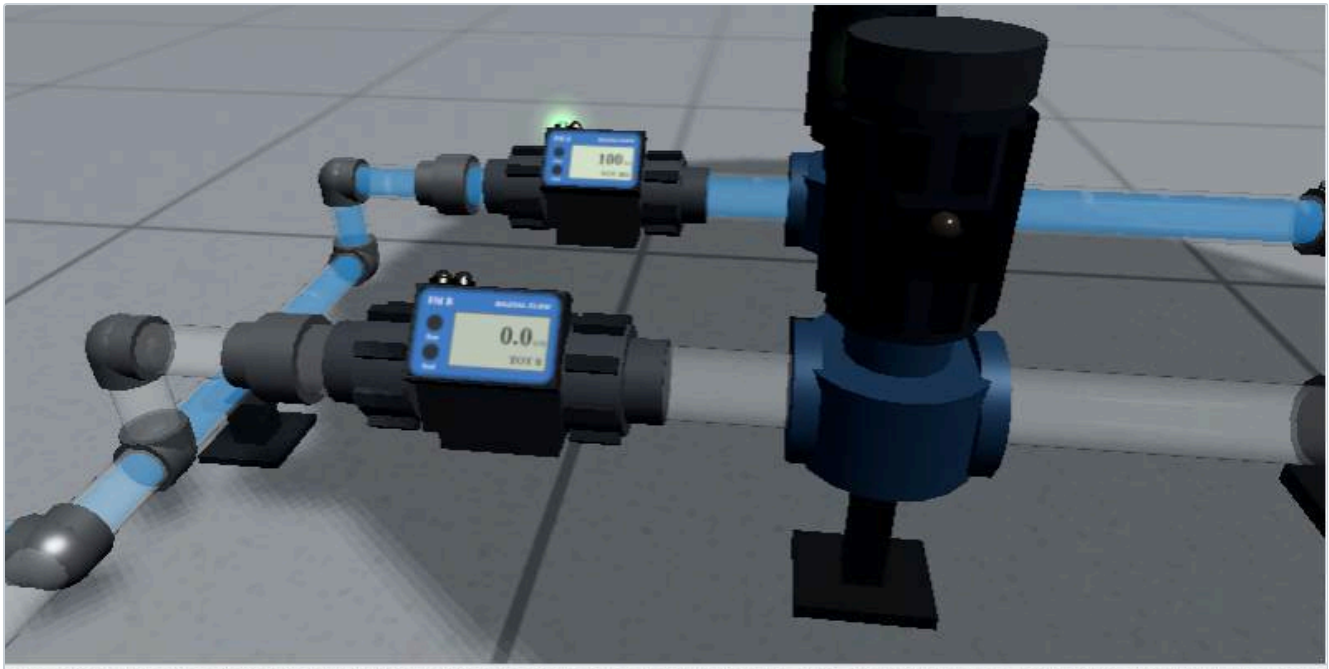
Each inlet is piped to a **Supply Tote**: **Supply_A** holds water, **Supply_B** concentrate, 10,000 units each. The pipes are part of the scene. The Mix Tank's *Inlet A fed from* and *Inlet B fed from* settings name the totes, and the pumps take what they move out of them. An empty tote runs its pump dry: no flow, no flowmeter pulses, and a line in the Simulator Log. An inlet fed from nothing is on the *plant supply header*, drawn in 3D as a riser out of the floor with a ball valve. The outlet, with nothing piped to it, runs to a floor drain.

The flowmeters are inline digital meters. The LCD shows the rate in units a second and the running total; the green lamp is on while liquid moves and the amber one blinks with each pulse the meter sends the controller. Tick *Flowmeters also report their rate* and each meter adds an analog input, `FlowA_Rate` and so on. Bind that signal to a REAL tag in its properties and the program reads 87.5, not 87. While the ingredients go in, the tank's contents look clear and pale; mixing, faster when warm, turns them cloudy, and the next batch starts clear again.

The tank has a local panel on its shell, and the console side of the floor has a **Process HMI** on a pedestal, **Tank_HMI**. Both show the same live faceplate: the vessel with its level and probes, the temperature against the high limit, the mixing time, a one-minute trend, the ratio, the flows, which pumps and heater are on, the state and an alarm line. It is drawn the way plant HMIs are, grey with colour only for what is running or wrong, so a lit heater or an alarm stands out at a glance. The HMI reads the tank directly; it is a window for you, not an input to the program.

vel 7 · Batch Mix on sandbox parts. Make three batches in the mix tank, one of each recipe, in the order the recipe selector shows (the level turns it after
tch). A: 50 % ingredient A, 150 °F, mix 10 s. B: 75 % A, 130 °F, mix 15 s. C: 25 % A, 170 °F, mix 8 s. Dose A by counting flowmeter A, top up with B to the HI
at and mix, drain. Each batch is graded as it drains: +100 each for the ratio (±5 %), the temperature at drain (−3 to +8 °F) and the mixing time; −50 for eac
ssed, −50 for a short batch, −100 for an overflow or a scorched batch (heater on with the element uncovered), −25 if the outlet pump runs dry more than 3

Figure 128. The mix tank's local panel during HEAT+MIX: 125 °F against a 180 °F high limit, 90 % level with both probes covered, the trend of temperature (solid) and level (dashed), the 50:50 ratio, and the heater and mixer lit.



vel 7 · Batch Mix on sandbox parts. Make three batches in the mix tank, one of each recipe, in the order the recipe selector shows (the level turns it after
tch). A: 50 % ingredient A, 150 °F, mix 10 s. B: 75 % A, 130 °F, mix 15 s. C: 25 % A, 170 °F, mix 8 s. Dose A by counting flowmeter A, top up with B to the HI
at and mix, drain. Each batch is graded as it drains: +100 each for the ratio (±5 %), the temperature at drain (−3 to +8 °F) and the mixing time; −50 for eac
ssed, −50 for a short batch, −100 for an overflow or a scorched batch (heater on with the element uncovered), −25 if the outlet pump runs dry more than 3

Figure 129. The inlet runs in 3D: the clear pipe from Supply_A carries water to flowmeter A, whose LCD reads 100 units/s with its green lamp lit, then through pump A; flowmeter B reads 0.0 with pump B stopped.

How each batch is scored

Recipe	Ingredient A	Temperature	Mix at least
A	50 %	150 °F	10 s
B	75 %	130 °F	15 s
C	25 %	170 °F	8 s

Graded when the batch drains	Points
A share within ±5 % of the recipe	+100 (−50 if not)
Temperature when the outlet pump started: recipe −3 to +8 °F	+100 (−50 if not)
Mixed at least the recipe time	+100 (−50 if not)
Less than 80 % of the tank (a short batch)	−50
Overflowed past the top	−100
Scorched (heater on with the element uncovered)	−100
Outlet pump ran dry more than 3 s	−25
All three batches under par (240 s)	+200

How to pass it, step by step

1. **Think in steps, not in rungs.** One DINT, `Step`, says what the tank is doing: 0 waiting for an empty tank, 1 dosing A, 2 topping up with B, 3 heating and mixing, 4 draining. Every output rung is `XIC(Run)EQU(Step,n)OTE(…)`, and every transition is a rung that moves Step on.
2. **Read the recipe into tags.** Three rungs: the selector position sets `Target_A` (pulses of A), `Target_T` (°F) and `Mix_Pre` (ms). A full tank at HI is 1800 units, so 50 % A is 900 units — 45 pulses; 75 % is 68; 25 % is 23.
3. **Dose A by counting.** Pump A runs in step 1; a CTU counts Flow_A; at `Target_A` move to step 2. Reset the counter when a batch starts.
4. **Top up with B to HI.** Pump B runs in step 2 until Hi_Level, and the HI probe ends it — B is "whatever is left", so the ratio is set by the count of A alone.
5. **Heat with hysteresis.** Latch `Heat_On` below the target minus 2 °F, unlatch it at the target. Gate the Heater output with `XIC(Lo_Level)` (never an uncovered element) and `XIO(Thermostat)` (the high limit). Heat on `LES(Tank_Temp,Target_T)` alone and the heater switches on and off every scan at the setpoint.
6. **Mix on a timer with a preset tag.** `TON(Mix_T,Mix_Pre,0)` in step 3 — the preset operand can be a tag, so one timer serves all three recipes. Leave step 3 when the timer is done *and* the temperature is in the band.

7. **Drain to empty.** Pump_Out in step 4. The level reads 0 % with a few units still in the tank, so run the pump on for a second (`Empty_T`) and then go back to step 0 and count the batch. Any longer and the pump runs dry.

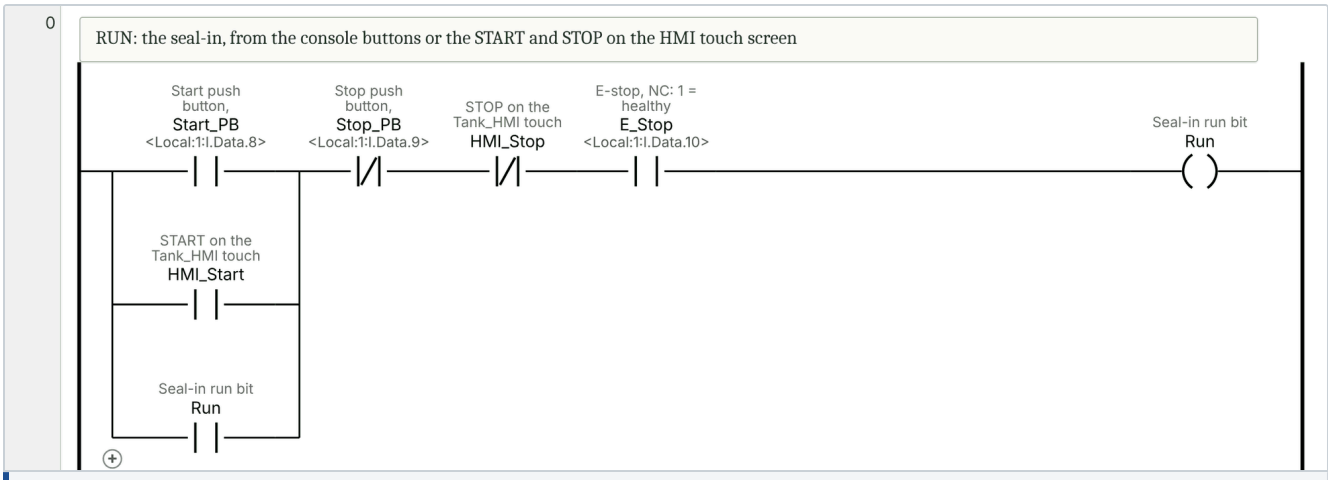
Tags this lesson needs

The level's alias tags are already made for you (chapter 30's table). Create these plain tags before you type the rungs; Verify reports any you miss.

Tag	Type	How to make it	Description
`Run`	BOOL	Edit Tags: name, type	Seal-in run bit
`HMI_Start`	BOOL	Edit Tags: name, type	START on the Tank_HMI touch screen (pulsed by the HMI)
`HMI_Stop`	BOOL	Edit Tags: name, type	STOP on the Tank_HMI touch screen (pulsed by the HMI)
`Step`	DINT	Edit Tags: name, type	Batch step: 0 wait for an empty tank, 1 dose A, 2 top up with B, 3 heat and mix, 4 drain
`A_Count`	COUNTER	Edit Tags: name, type	Flowmeter A pulses this batch
`Target_A`	DINT	Edit Tags: name, type	Pulses of A for the selected recipe
`Target_T`	DINT	Edit Tags: name, type	Temperature for the selected recipe, °F
`Target_Lo`	DINT	Edit Tags: name, type	Heater comes back on below this, °F
`Mix_T`	TIMER	Edit Tags: name, type	Mixing time this batch
`Mix_Pre`	DINT	Edit Tags: name, type	Mixing time for the selected recipe, ms
`Heat_On`	BOOL	Edit Tags: name, type	Heater wanted (with hysteresis)
`Empty_T`	TIMER	Edit Tags: name, type	Run the outlet pump on a moment after the level reads 0
`Done_ONS`	BOOL	Edit Tags: name, type	

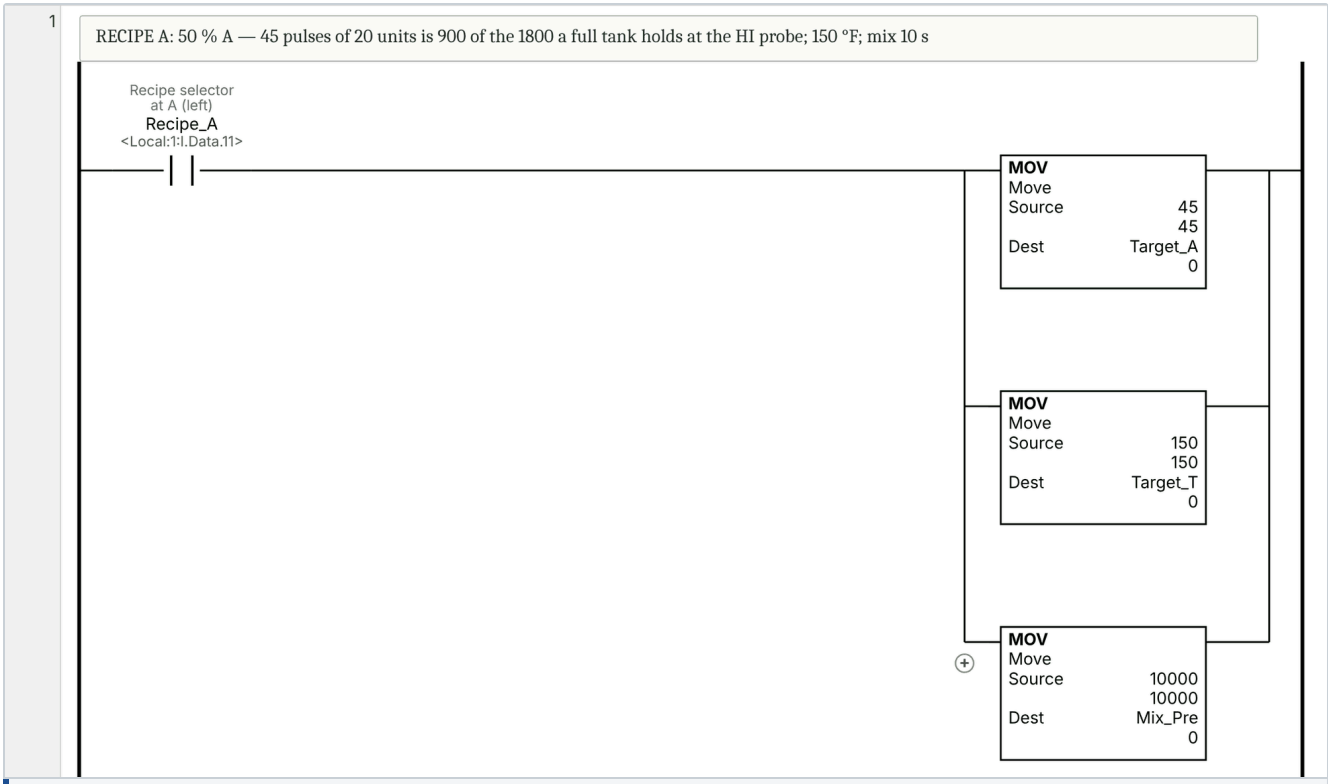
Run, recipes and the start of a batch

RUNG 0




```
[XIC(Start_PB),XIC(HMI_Start),XIC(Run)]XIO(Stop_PB)XIO(HMI_Stop)XIC(E_Stop)OTE(
Run);
```

RUNG 1

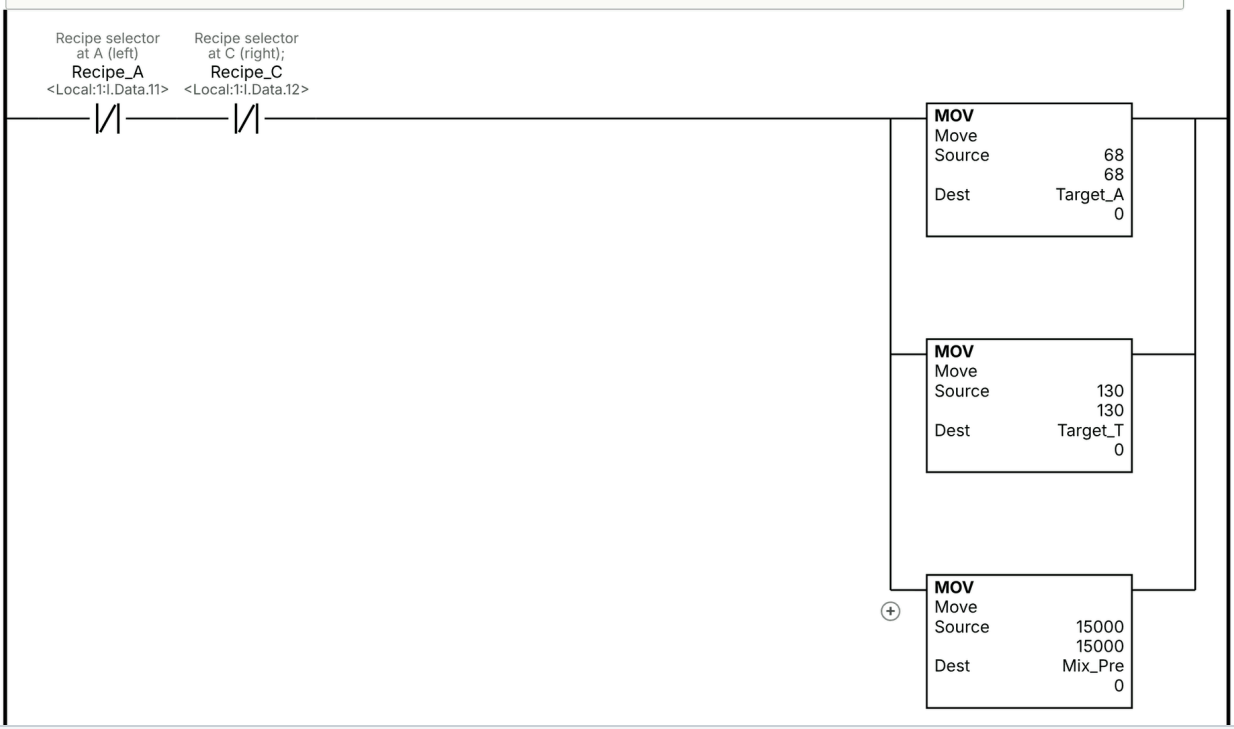



```
XIC(Recipe_A)[MOV(45,Target_A),MOV(150,Target_T),MOV(10000,Mix_Pre)];
```

RUNG 2

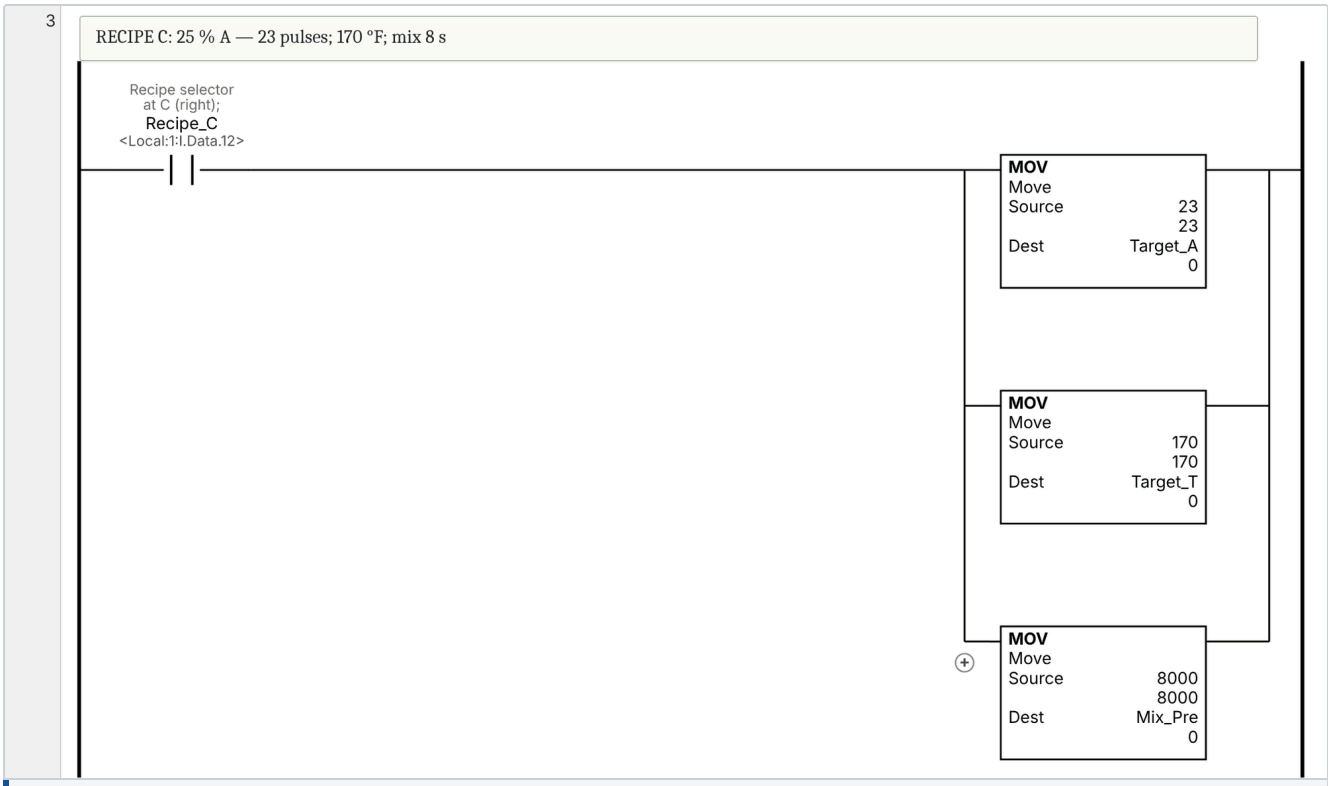
2

RECIPE B (selector in the middle): 75 % A — 68 pulses; 130 °F; mix 15 s



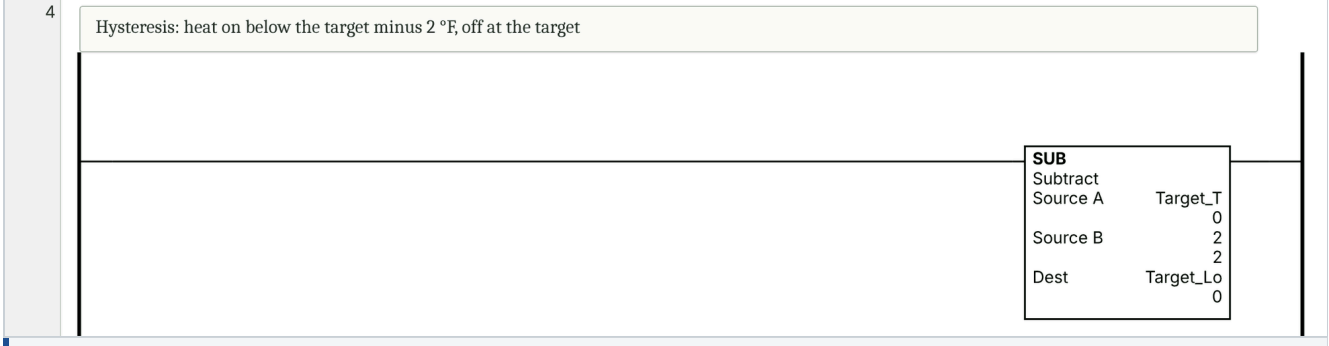

```
XIO(Recipe_A)XIO(Recipe_C)
[MOV(68,Target_A),MOV(130,Target_T),MOV(15000,Mix_Pre)];
```

RUNG 3



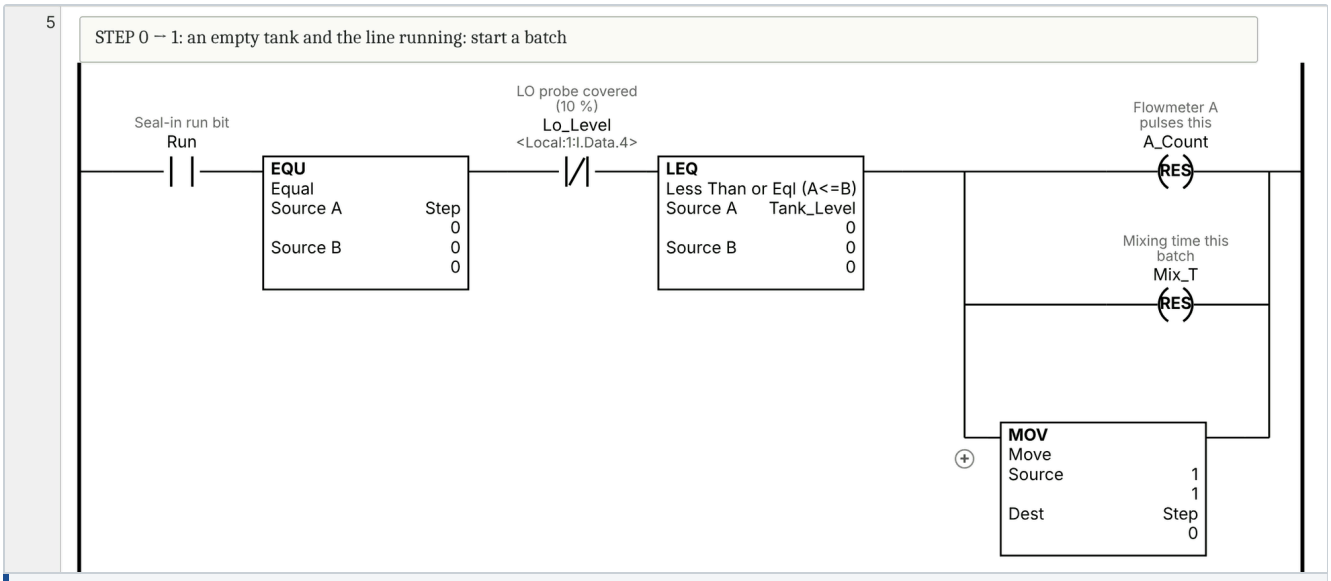

```
XIC(Recipe_C)[MOV(23,Target_A),MOV(170,Target_T),MOV(8000,Mix_Pre)];
```

RUNG 4




```
SUB(Target_T,2,Target_Lo);
```

RUNG 5

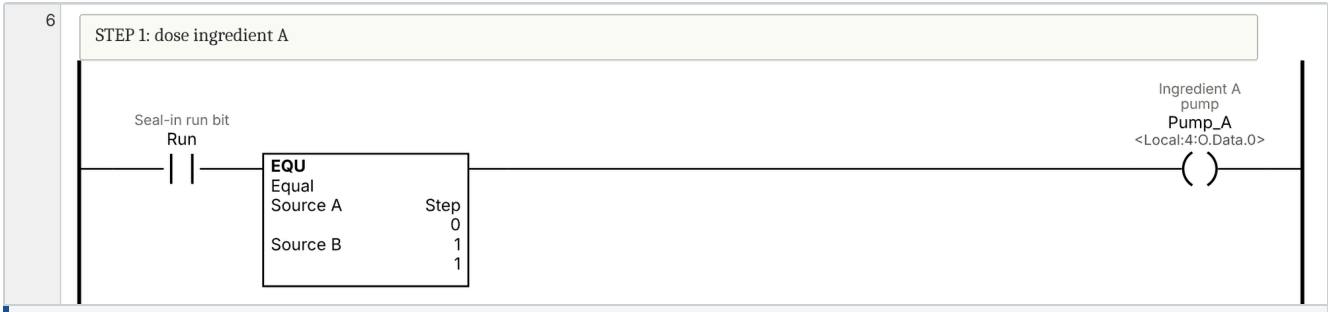



```
XIC(Run)EQU(Step,0)XIO(Lo_Level)LEQ(Tank_Level,0)
[RES(A_Count),RES(Mix_T),MOV(1,Step)];
```

The recipe rungs run every scan; the level only turns the selector when a batch has drained, so the values never change in the middle of one. `SUB(Target_T,2,Target_Lo)` is the bottom of the heater's band. A batch starts only on an empty tank: `XIO(Lo_Level)` and `LEQ(Tank_Level,0)`.

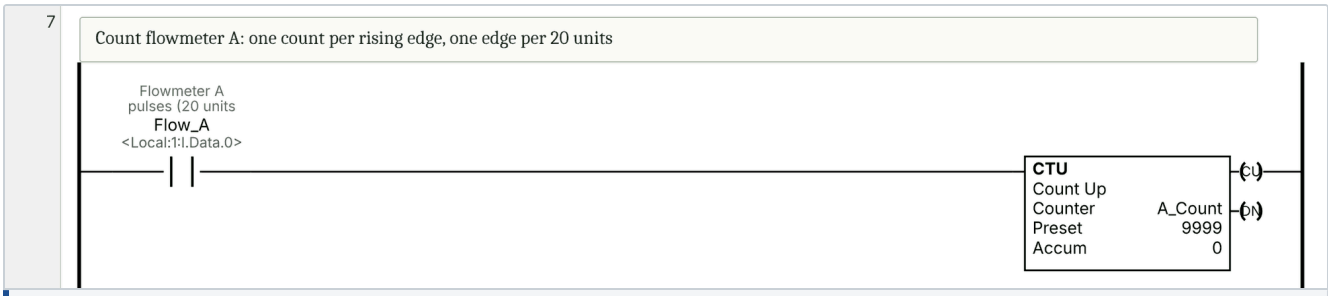
Dosing: A by count, B to HI

RUNG 6



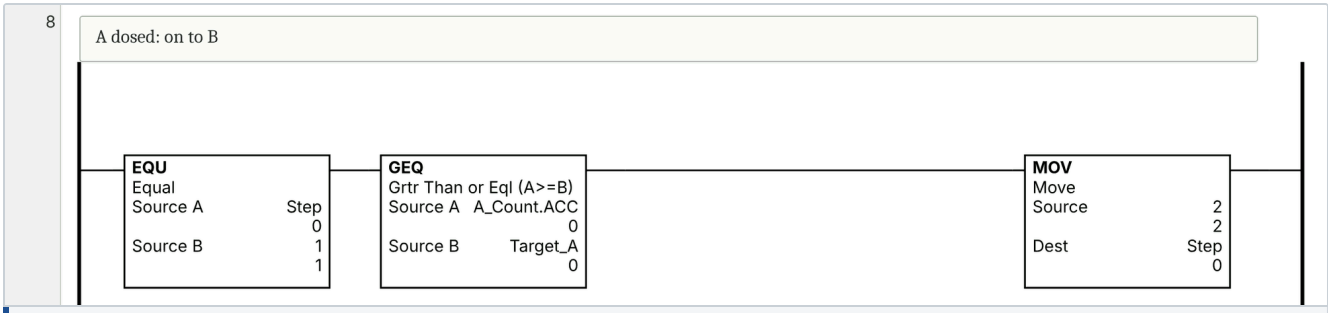

```
XIC(Run)EQU(Step,1)OTE(Pump_A);
```

RUNG 7




```
XIC(Flow_A)CTU(A_Count,9999,0);
```

RUNG 8

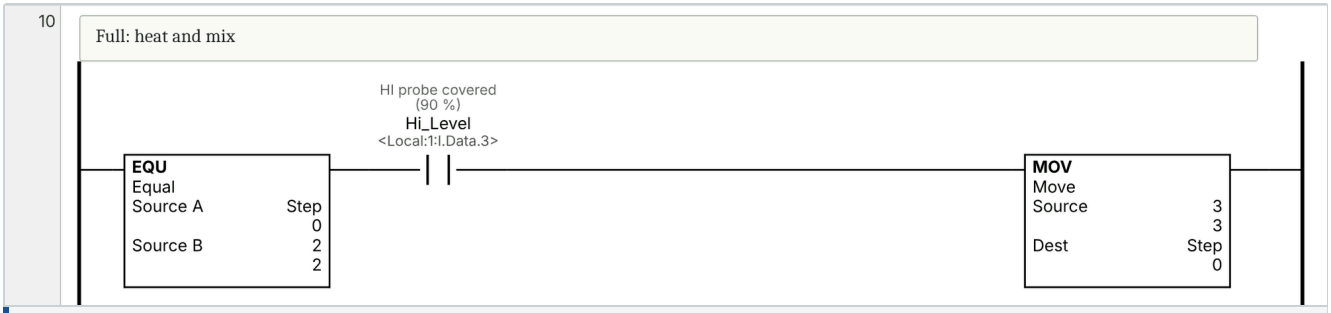



```
EQU(Step,1)GEQ(A_Count.ACC,Target_A)MOV(2,Step);
```

RUNG 9

```
XIC(Run)EQU(Step,2)XIO(Hi_Level)OTE(Pump_B);
```

RUNG 10




```
EQU(Step,2)XIC(Hi_Level)MOV(3,Step);
```

The CTU counts every rising edge of Flow_A, whatever step it is in — fine, because only step 1 runs pump A, and the batch start resets it.

Heating and mixing

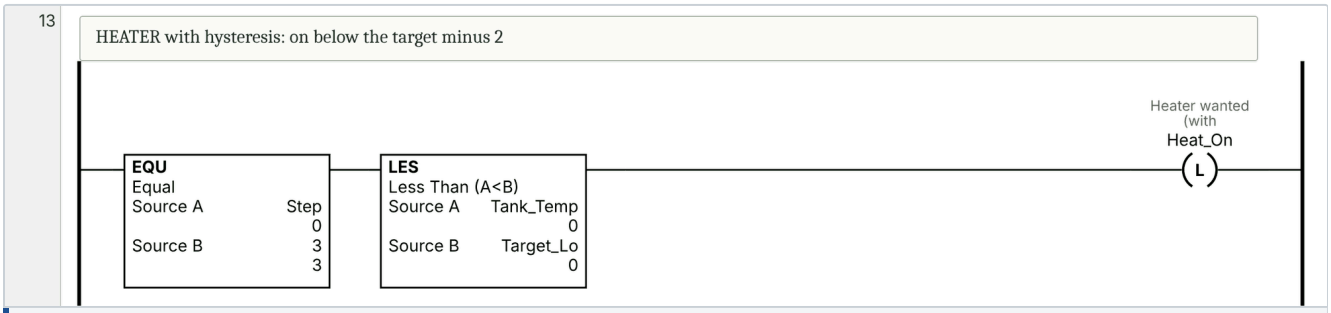
RUNG 11

```
XIC(Run)EQU(Step,3)OTE(Mixer);
```

RUNG 12

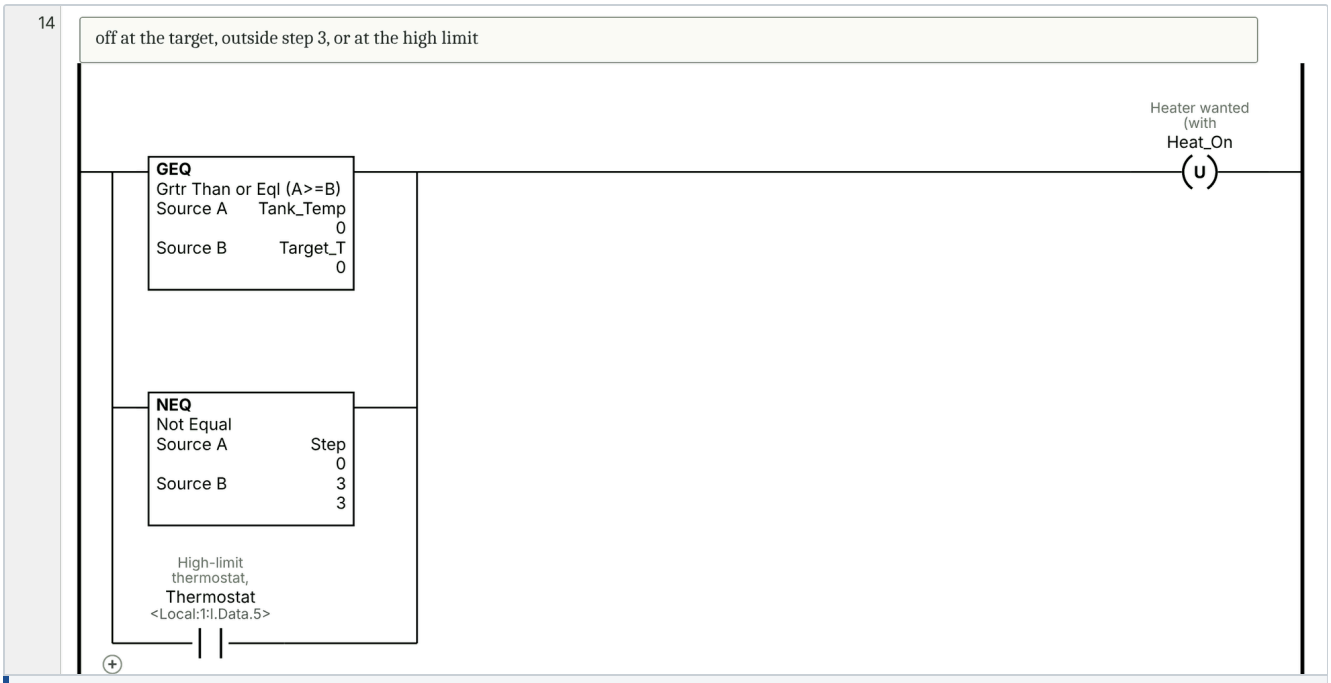
```
XIC(Run)EQU(Step,3)TON(Mix_T,Mix_Pre,0);
```

RUNG 13



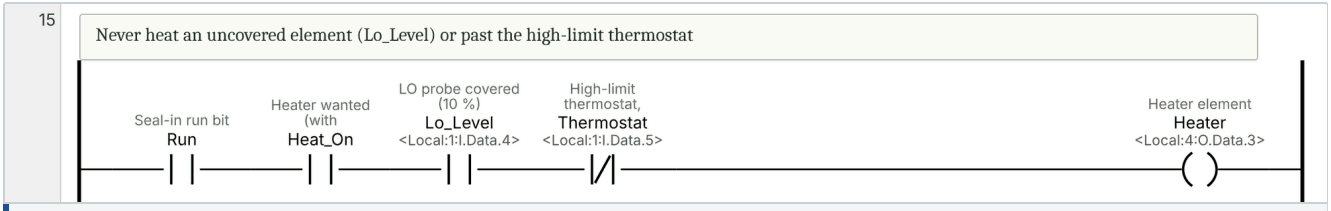

```
EQU(Step,3)LES(Tank_Temp,Target_Lo)OTL(Heat_On);
```

RUNG 14



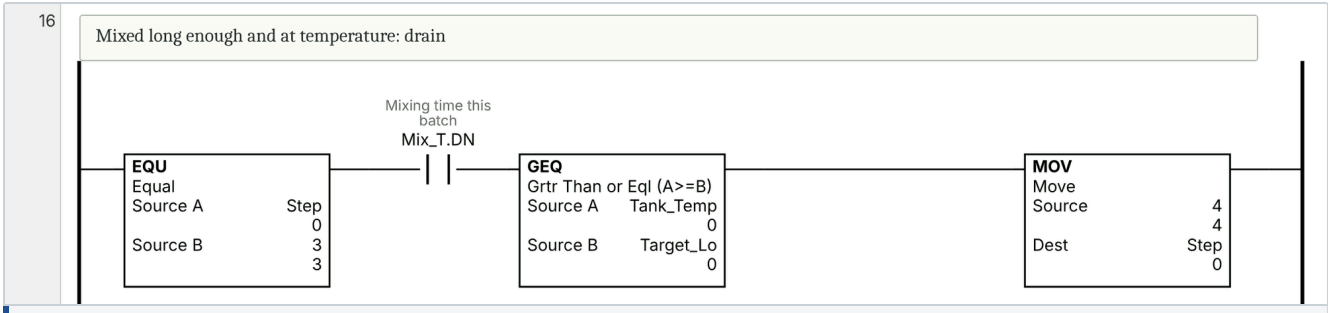

```
[GEQ(Tank_Temp,Target_T),NEQ(Step,3),XIC(Thermostat)]OTU(Heat_On);
```

RUNG 15




```
XIC(Run)XIC(Heat_On)XIC(Lo_Level)XIO(Thermostat)OTE(Heater);
```

RUNG 16




```
EQU(Step,3)XIC(Mix_T.DN)GEQ(Tank_Temp,Target_Lo)MOV(4,Step);
```

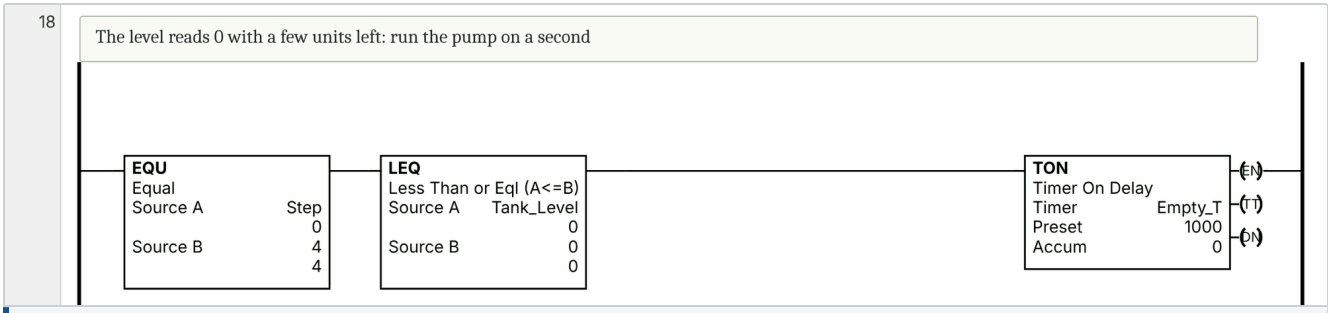
Rung 14 unlatches `Heat_On` at the target, outside step 3, or at the high limit; rung 15 is the only rung that drives the heater, and it refuses an uncovered element. Rung 16 leaves step 3 when the mixing timer is done and the batch is at least at the bottom of its band.

Draining, counting, lamps

RUNG 17

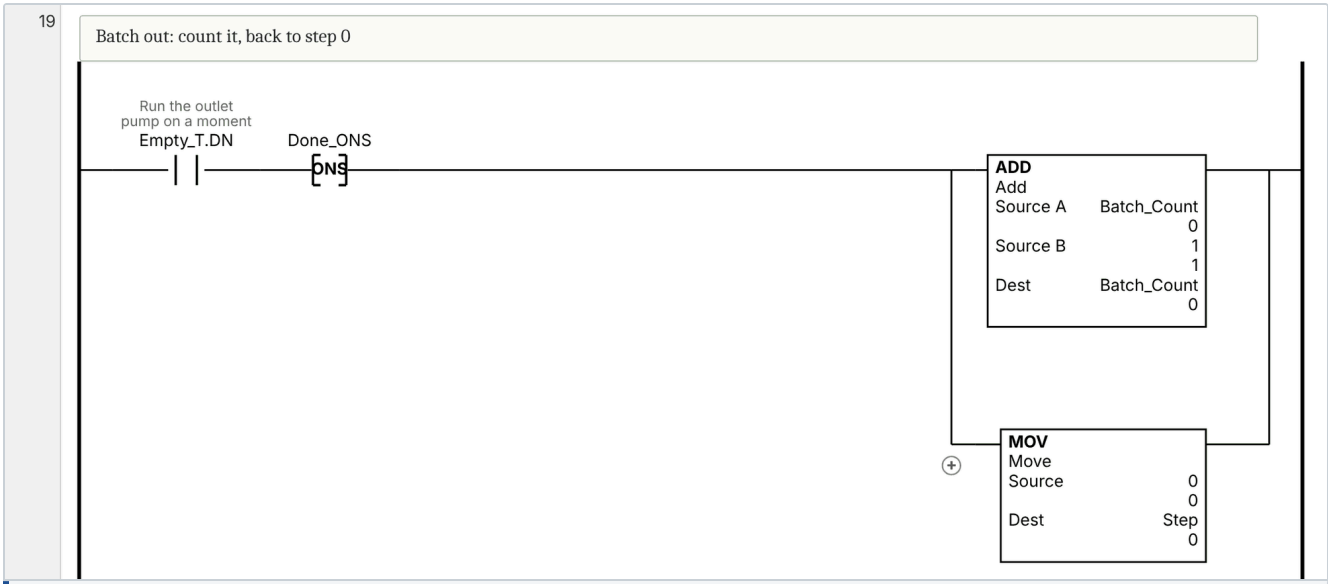
```
XIC(Run)EQU(Step,4)OTE(Pump_Out);
```

RUNG 18



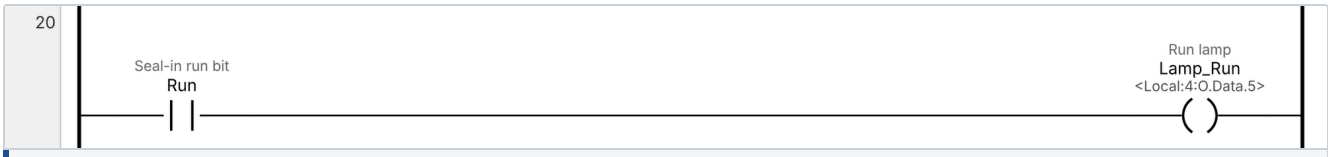

```
EQU(Step,4)LEQ(Tank_Level,0)TON(Empty_T,1000,0);
```

RUNG 19



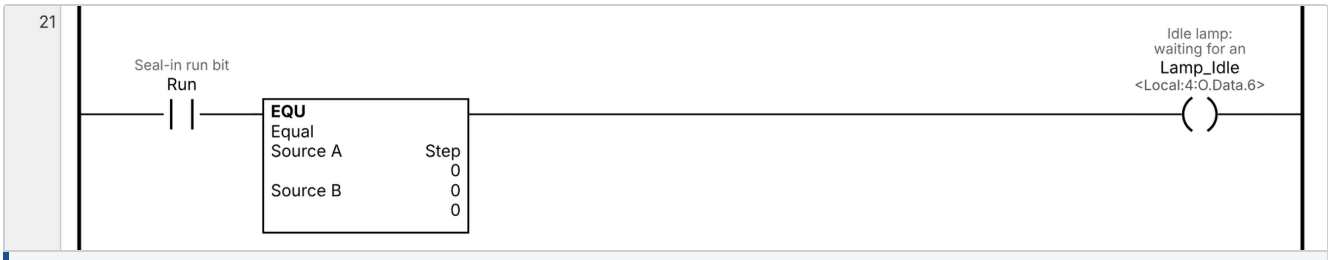

```
XIC(Empty_T.DN)ONS(Done_ONS)[ADD(Batch_Count,1,Batch_Count),MOV(0,Step)];
```

RUNG 20



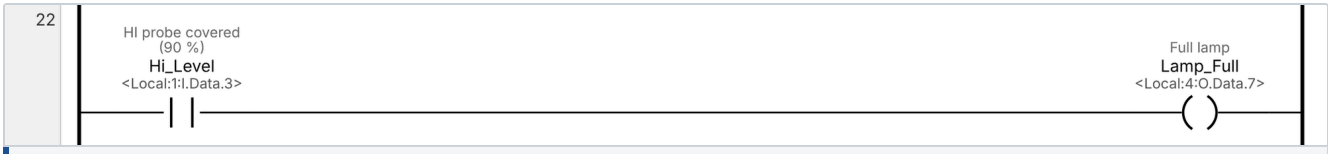

```
XIC(Run)OTE(Lamp_Run);
```

RUNG 21



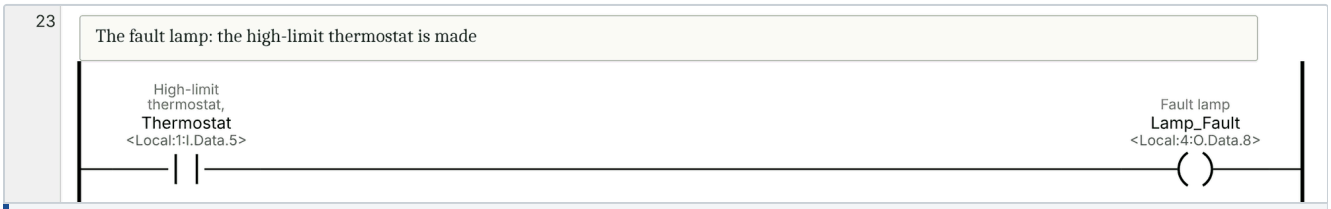

```
XIC(Run)EQU(Step,0)OTE(Lamp_Idle);
```

RUNG 22




```
XIC(Hi_Level)OTE(Lamp_Full);
```

RUNG 23




```
XIC(Thermostat)OTE(Lamp_Fault);
```

WATCH FOR: COUNTING THE WRONG THING

Count `Flow_A` with an XIC and an ADD instead of a CTU and the count races: the pulse is high for a whole scan or more, and the ADD adds on every one of them. A CTU (or an ONS in front of the ADD) counts edges. Then the opposite trap: count `Flow_B` by mistake and pump A never stops — exactly what a dead flowmeter does (Tools → Inject Machine Fault → flowmeter A dead): the tank fills to the top and overflows. A dose that ends on a count needs a second way to end — a HI probe, a time limit — or a dead meter floods the floor.

Run it

Load your program, Run mode, Start. Watch the display: FILLING with the A share climbing, then B pulling it down to the recipe; HEAT+MIX with the temperature climbing at a little over 2 °F a second; DRAINING; EMPTY; and the selector turns. The Simulator Log grades each batch as it drains. The reference run: 3 of 3 bottles in the right bin, nothing lost, in 179 s against a par of 240 s: 1100 points and three stars.

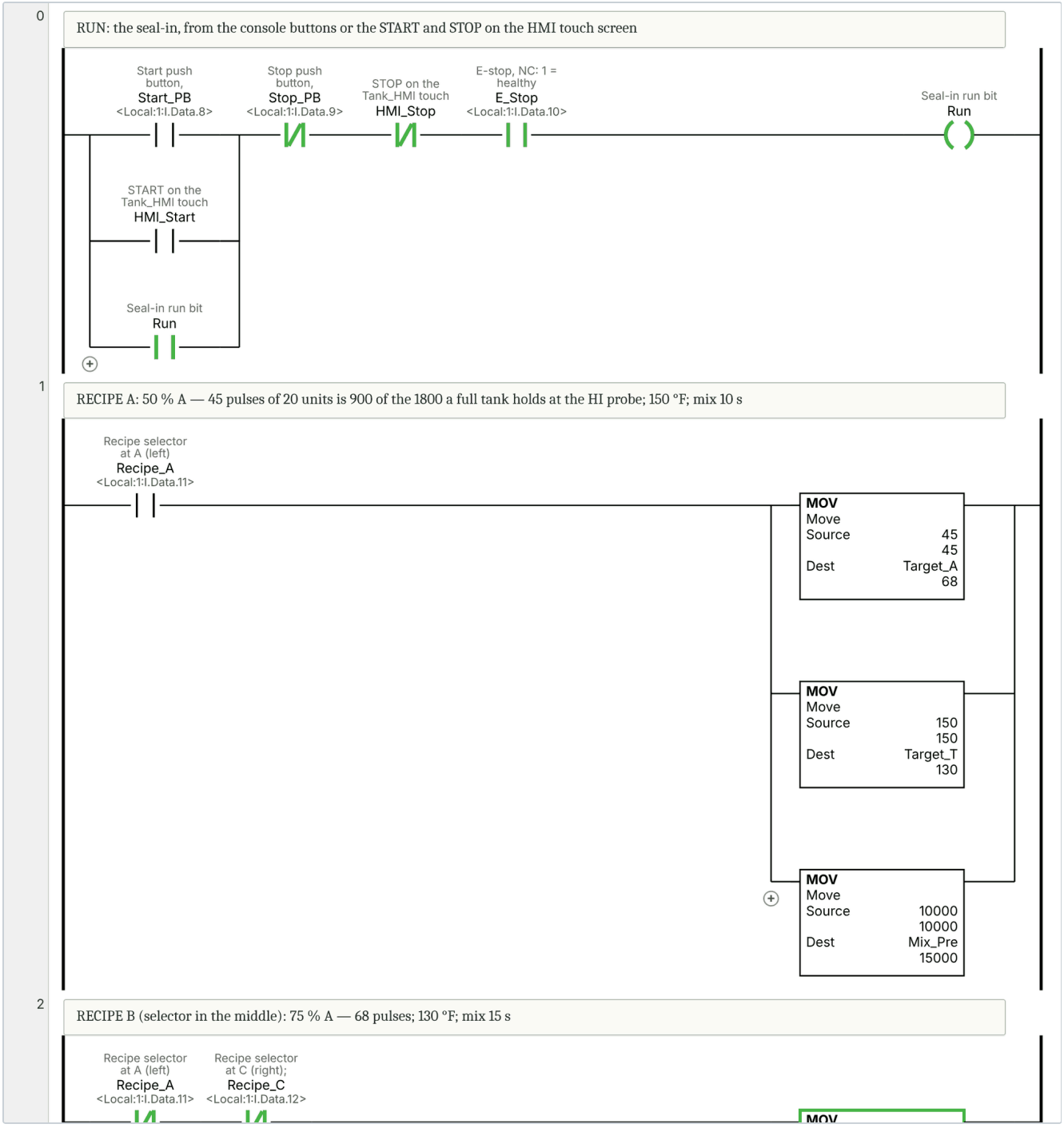


Figure 130. Level 7 online, the top of the routine.

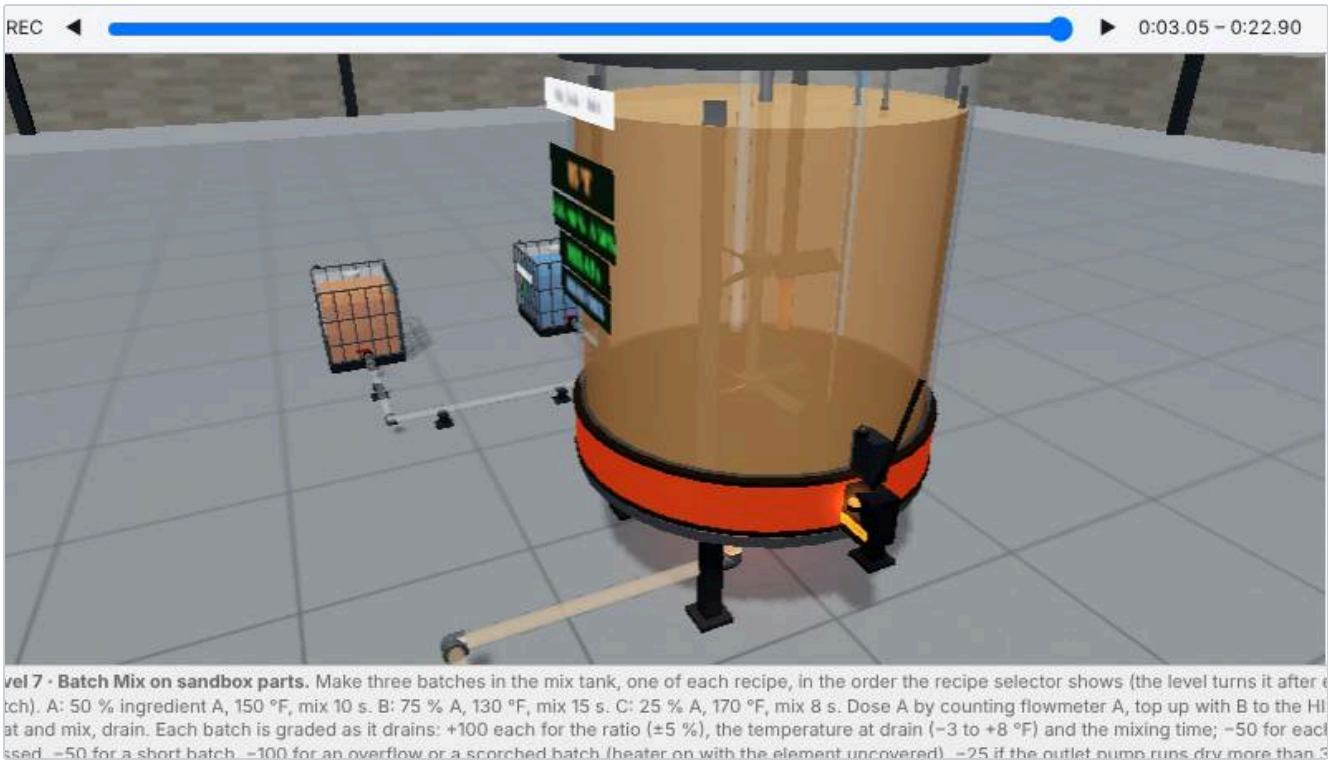


Figure 131. The mix tank in 3D while it heats and mixes: the product has gone from clear to cloudy, the heating band glows, the impellers turn behind the displays; Supply_A and Supply_B stand behind on the left.

TRY IT

The reference program wastes most of its time heating after the tank is full. Start the heater as soon as Lo_Level is covered — in step 1 — and watch two things happen: the batch is ready much sooner, and every slug of cold ingredient pulls the temperature down (the tank mixes it in), so the heater works against the fill. Beat 150 s. Then take `XIC(Lo_Level)` out of the heater rung and start a batch: the element comes on before the liquid covers it, and the batch is scorched. Last, give the heater no band at all — `LES(Tank_Temp,Target_T)OTE(Heater)` — and watch it chatter on and off at the setpoint in Monitor Tags.

TRY IT — RUN A TOTE DRY

Set Supply_B's *Starts at* to 15 % (1,500 units) and run the reference program. Batches 1 and 2 take about 1,350 units of B between them, so in batch 3 the tote empties while pump B is topping up. Pump B runs dry, the HI probe never makes, and the program sits in step 2 forever with the Run lamp on. A real line would be exactly as stuck. Make two fixes. First, a watchdog: `XIC(Run)EQU(Step,2)TON(B_Fill_T,20000,0)`, and when it is done, light Lamp_Fault and stop. Second, use the tote's own **LowLevel** switch, a digital input like any other (alias it as `Tote_B_Low`): do not start a batch while it is on. The watchdog catches the failure; the low switch stops you walking into it. In Run mode, clicking an empty tote swaps in a full one, the way a forklift would.

A mix tank on the bottle line

The Mix Tank pipes to a filler the way the Reservoir does: put one in place of Syrup_Tank and pick it in the Filler's *Fed from*, which lists every tote, tank, reservoir and mix tank in the scene. A Tank can also be fed from the Mix Tank (its *Inlet fed from*); it then fills only while the mix tank's outlet pump runs and its own inlet valve is open. The filler then only gets product while the tank's outlet pump runs, so the program has to make a batch first and then hold Pump_Out on while the filler draws — a batch-to-line handoff that a supply tank never asked for.

The seven levels, solved

Level	Program	Bottles	Par	Time	Score
1 Height sort	latch, one at a time, 16 rungs	10	150 s	94 s	1200
2 Fill by label	latch, one at a time, 20 rungs	10	170 s	104 s	1200
3 Color sort	latch, one at a time, 20 rungs	12	190 s	125 s	1400
4 Two on the line	zone queues, 43 rungs	14	110 s	85 s	1600
5 Full sort	zone queues, 53 rungs	16	120 s	95 s	1800
6 Pack it	zone queues and a case lane, 39 rungs	16	140 s	111 s	1800
7 Batch Mix	step sequencer, 24 rungs	3 batches	240 s	179 s	1100

Every row is a run of the program printed in its chapter, made by the page's own test suite on this version. The zone-queue program solves Levels 1-3 as well, faster; File → Load Reference Solution installs the right one for any level.

Part VI — Beyond the bottle line

39 Save, share and replay

Three things outlive a run: the project file, a link that carries the project to someone else, and the recording of the last two minutes of the machine. This chapter is about all three, and about using the recording to debug. Being able to wind a machine back and watch the scan where it went wrong is something no real line gives you.

The project file

File → Save (Ctrl+S) writes one `.json` file holding everything: the scene and every device's settings, the tags, the routines, the I/O configuration and, if you tick it, the network settings. **File → Open** reads it back. The page also keeps your work by itself between visits, so the file is for versions and for moving a project to another PC.

A share link

File → Share Link… packs the same project into the address itself. The JSON is deflated and written after `#lc=`, so nothing is uploaded anywhere: whoever has the link has the project. The bottle line with its twelve routines makes a link of about 17 KB, a third of the file.

- **Opens at** is the address of the web copy of Logix Connect, your GitHub Pages site. On the site it fills itself in. In the desktop app, type it once and it is remembered. The desktop app runs from your own PC, so a link pointing at it would open nowhere else.
- **Include network settings** adds module IP addresses, connections and the bridge address. Leave it off when you send a project to someone on another network.
- **Include the score card** is offered once a level is finished. The link then carries the level, the score, the correct and wrong counts, the time and whether it beat par, and it shows when the link is opened.

Opening a link asks first, with a summary of what is in it, because it replaces the project you have open. Your programs for the fixed-machine levels are kept. In the desktop app, **File → Open Link…** takes a pasted link, or the whole message it came in.

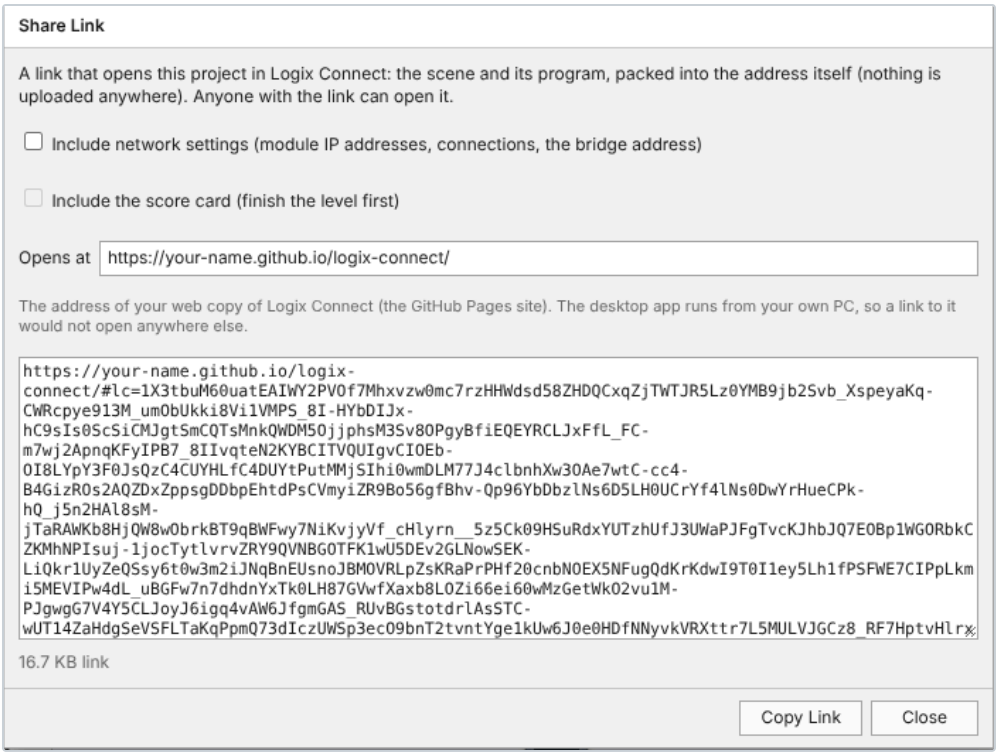


Figure 132. File → Share Link… on the bottle line: the web address it opens at, the two options and the link itself, 16.7 KB.

> **WATCH FOR**
>
> Some mail and chat apps cut very long links short. A link cut short is refused with a clear message rather than opening half a project. For a big scene (over about 60 KB of link, which the dialog warns about) send the file instead.

The scene recorder

While a sandbox scene runs, the recorder keeps a frame every 0.25 s of machine time for the last two minutes (one minute on a phone). A frame holds every part on the floor, every device's state (a timer half run, a cylinder half out, a tank's level and temperature, a case on the packer's lane), the simulator's counters and log, the machine faults you injected, the random generator that picks the part mix, and every tag in the controller.

The bar under the simulator toolbar shows the span recorded, **REC 0:00.05 – 0:40.38**. Drag the slider back, or press ◀, and the scene goes into **REPLAY**:

- All three views, the I/O panel, Quick Watch and the data table show that moment, read-only. Nothing steps.
- The ladder shows the power flow of that moment. The page runs one scan of no time over the recorded tags to light the rungs, then puts the tags back exactly as they were recorded, so looking changes

nothing.

- ▶ steps forward a frame. **Live** puts the run back exactly as you left it. The Run Mode button does the same.
- **Run from here** keeps that moment and runs on from it, with the program as it is *now*. The frames after it are dropped, and the log notes the rewind.

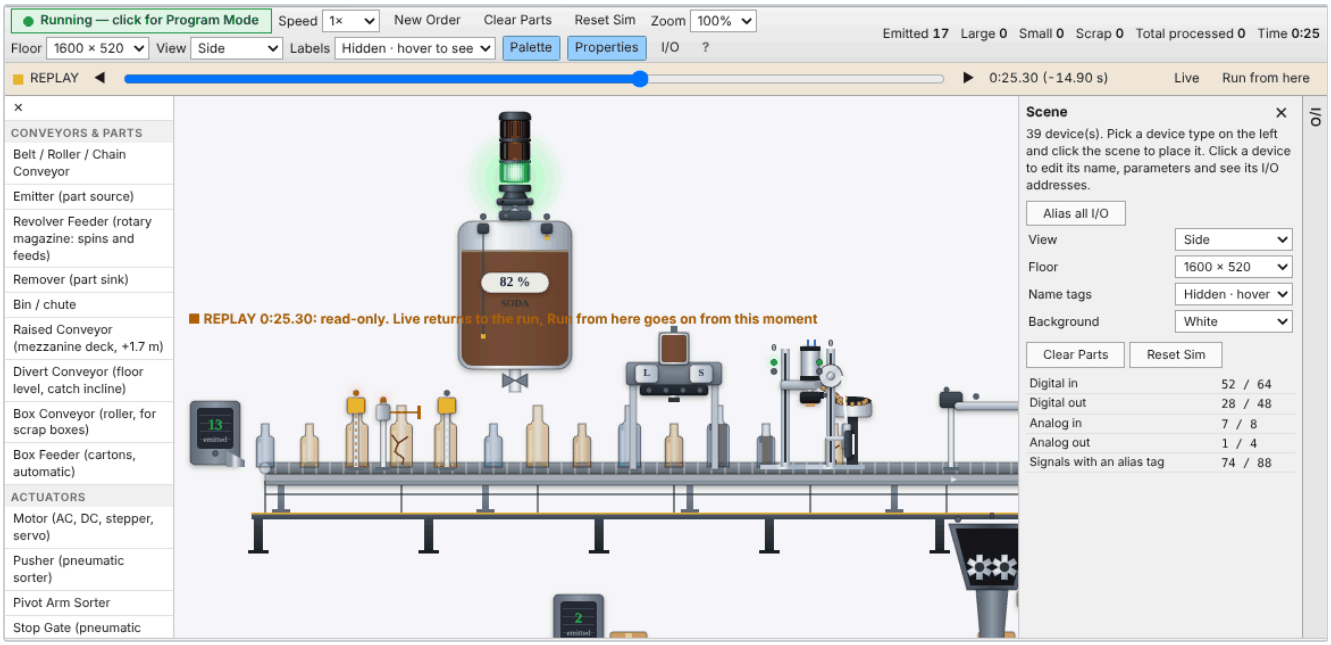


Figure 133. Replay on the bottle line, 14.9 s back: the bar is amber, the scene says REPLAY and is frozen at 0:25.30, and Live and Run from here are offered.

Because the random generator is rewound with everything else, running on from a frame with the same program gives the same run, bottle for bottle. Change a rung and run on from the same frame, and the only difference between the two runs is your change. That is the most useful thing the recorder does.

TRY IT — CATCH A JAM AND READ THE SCAN

Run the bottle line. After a few bottles, use Tools → Inject Machine Fault… → *Jam*. The jammed bottle stops, LS11 does not see it where the registers say it should be, Alarm_Word bit 2 sets and the line stops. Switch to Program mode, open `S11_Alarms` and drag the slider back a second at a time to the frame where `EOL_Missing` first goes true. The rung above it shows why: `LS11` dark while the Exist and Large bits said a bottle was due. Now drag back to before you injected the jam and press **Run from here**. The jam is gone, because the fault was injected after that moment and the recorder rewinds faults with the rest. The line runs as if it had never happened.

TRY IT — PROVE A CHANGE ON THE SAME RUN

On Level 6, run until the first case creeps onto the mark under the packer head, then scrub back to the frame where `Case_Slow` makes. Note where the case stops: on the mark. Change the creep rung from `MOV(15,Case_Hz)` to `MOV(30,Case_Hz)`, accept the edit and press **Run from here**. The same case, on the same frame, now runs past the head before `Case_In_Place` can stop it, because at 30 Hz the drive's 0.5 s decel ramp carries it beyond the ±6 px window. With no case under the head nothing is packed, and the case goes out empty for −25. Put 15 back, scrub to the same frame, run from here again, and it lands. Two runs that differ only in one number.

What is and is not rewound

Rewound: the parts, the devices, the controller's tags (timers, counters, ONS bits, arrays, UDTs), its periodic task schedule, the machine faults and the random part mix. Not rewound: your program and the scene layout (they are what you are testing), the trend charts, and anything outside the page: a real device on the bridge, or an EtherNet/IP module. Reset Sim, New Order or opening another project starts a new recording.

40 Where to go next

Build cells of your own

Every part in Part III works in an empty sandbox the way it does on the template, and the program's test suite builds a scrap cell from the parts list in an empty floor every release: a Raised Conveyor with a Bottle Feeder, a Broken-bottle Eye, a pivot gate over a Grinder, a Box Conveyor with a Box Feeder and Box Photoeye, a second gate over a Divert Conveyor, and six rungs — a run bit, the belts, a gate on its own photoeye, the box conveyor on LS, the shredder on an empty box. It runs. Start there, then:

- **A continuous line.** No indexing, no shift registers: every station acts on its own photoeye (`Detect` on a gate is the eye just ahead of the arm). Compare the two styles.
- **A second labeler.** The line's labeler (step 10) is RH with a wrap-around roller. Put a second one on the other side (LH, tamp pad) two stops further and give it its own routine — the two handshakes are identical; only the register bits move. Then a photoeye set to *labeled bottles* at the end of the line counts the finished product.
- **A Revolver Feeder cell.** Feed one part per pulse from the rotary magazine: `XIO(T1.DN)TON(T1,1000,0); XIC(T1.DN)XIC(Rev_PartReady)OTE(Rev_Index);`, run it dry, let it reload.
- **A sorter.** Inductive and capacitive sensors tell metal from plastic, a colour sensor tells red from blue, a light array measures height, a weighing conveyor weighs; pushers and bins sort. The levels are this with a score.
- **A filling cell with a Dispenser** and a Fill Level Sensor instead of the fill station: the level sensor tells you when to close the valve.
- **Vision and RFID.** The Vision Sensor can be a camera module from an EDS file (I/O Configuration → Ethernet) with trigger, results and inspection-complete tags; the RFID reader reads and writes a tag's memory on each part.

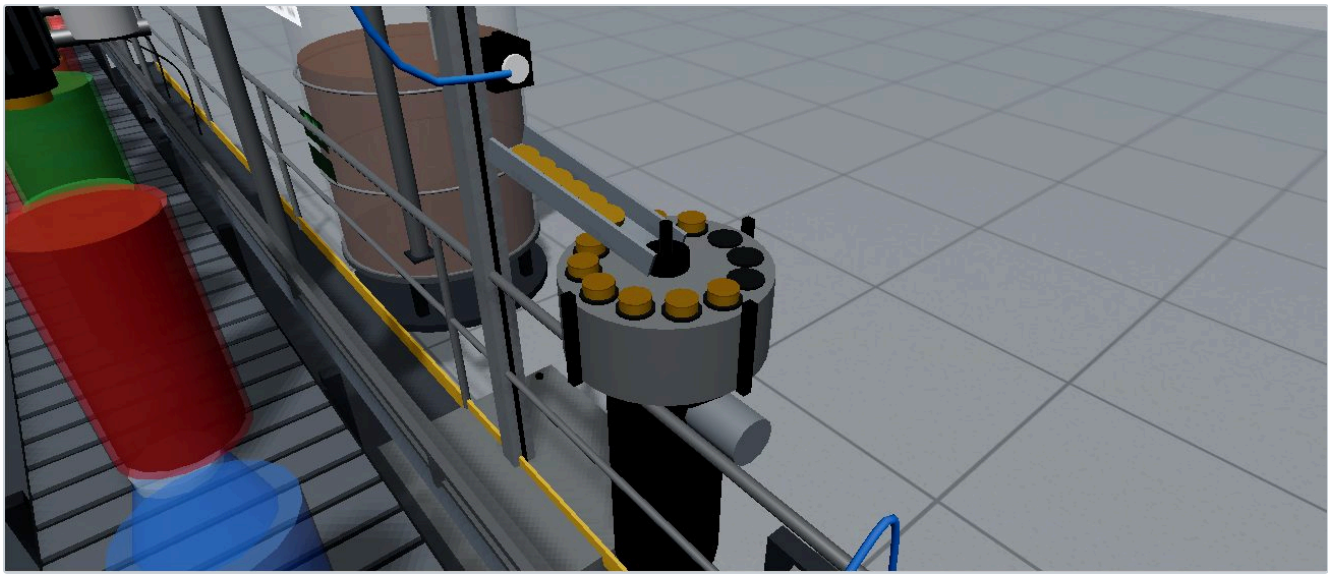


Figure 134. The rotary cap magazine, loaded, indexing a pocket per cap: the same mechanism is the Revolver Feeder part, which feeds any part one per pulse.

Real hardware

Bridge & Tools runs a small Python bridge on your PC that connects the page to the outside: a TCP socket a vision system or a test bench talks to (the *TCP Client Socket* part), and EtherNet/IP devices. Communications → Start Bridge, then the socket part's settings. Help → Bridge & Tools covers it.

Studio 5000

Export L5X and import the file into Studio 5000: the tags, aliases and routines come across, and the program that ran the simulated line is the program you would download to a CompactLogix driving the real one — the I/O addresses are already in the right shape. Going the other way, Import L5X brings a real project's routines in to run against a sandbox machine you lay out to match.

Appendix A — The bottle line tag list

Tag	Type	Alias for	Description
`Start`	BOOL	`Local:3:I.Data.2`	LogixPro I:1/0 · Start push button, NO
`Stop`	BOOL	`Local:3:I.Data.3`	LogixPro I:1/1 · Stop push button, NC (1 until pressed)
`LS1`	BOOL	`Local:1:I.Data.4`	LogixPro I:1/6 · Exist sensor: a bottle at the start of the line
`LS2`	BOOL	`Local:1:I.Data.5`	LogixPro I:1/7 · Size check: beam set so only a large bottle breaks it
`LS3`	BOOL	`Local:1:I.Data.6`	LogixPro I:1/8 · Broken check: sees broken bottles only
`LS4`	BOOL	`Local:1:I.Data.9`	LogixPro I:1/9 · Fill tube down
`LS5`	BOOL	`Local:1:I.Data.10`	LogixPro I:1/10 · Large charge delivered
`LS6`	BOOL	`Local:1:I.Data.11`	LogixPro I:1/11 · Small charge delivered
`LS7`	BOOL	`Local:1:I.Data.13`	LogixPro I:1/12 · Cap ram down
`LS8`	BOOL	`Local:2:I.Data.7`	LogixPro I:1/13 · Bottle at the scrap gate
`LS9`	BOOL	`Local:2:I.Data.11`	LogixPro I:1/14 · Bottle at the divert gate
`LS10`	BOOL	`Local:3:I.Data.0`	LogixPro I:1/15 · Scrap box under the grinder chute
`Main_Conveyor`	BOOL	`Local:4:O.Data.1`	LogixPro O:2/0 · Main conveyor motor
`Scrap_Conveyor`	BOOL	`Local:5:O.Data.4`	LogixPro O:2/1 · Box conveyor under the grinder
`Divert_Conveyor`	BOOL	`Local:5:O.Data.3`	LogixPro O:2/2 · Divert conveyor for large bottles
`Grinder`	BOOL	`Local:5:O.Data.1`	LogixPro O:2/3 · Grinder motor
`Scrap_Gate`	BOOL	`Local:5:O.Data.0`	LogixPro O:2/4 · Scrap gate: swings a bottle into the grinder
`Divert_Gate`	BOOL	`Local:5:O.Data.2`	LogixPro O:2/5 · Divert gate: swings a bottle onto the divert conveyor
`Fill_Tube`	BOOL	`Local:4:O.Data.6`	LogixPro O:2/6 · Lower the fill tube
`Large_Fill`	BOOL	`Local:4:O.Data.7`	LogixPro O:2/7 · Large fill valve
`Small_Fill`	BOOL	`Local:4:O.Data.8`	LogixPro O:2/8 · Small fill valve
`Cap_Ram`	BOOL	`Local:4:O.Data.9`	LogixPro O:2/9 · Cap ram
`Run`	BOOL	`Local:5:O.Data.5`	LogixPro O:2/10 · Run lamp

Tag	Type	Alias for	Description
`Cap_Feeder`	BOOL	`Local:4:O.Data.10`	Capper Feeder: runs the vibratory cap bowl, refilling the chute
`Label_Apply`	BOOL	`Local:4:O.Data.13`	Labeler Start: the applicator comes in and presses a label on
`Label_PadIn`	BOOL	`Local:2:I.Data.0`	Labeler applicator at the end of its stroke (label pressed on)
`Label_Ready`	BOOL	`Local:2:I.Data.1`	Labeler ready: web loaded, head home, no fault
`Label_Error`	BOOL	`Local:2:I.Data.2`	Labeler fault: end of roll, or the last cycle found no label
`Label_Low`	BOOL	`Local:2:I.Data.3`	Labeler roll low
`Label_Present`	BOOL	`Local:2:I.Data.4`	A label is peeled and waiting at the dispensing edge
`Label_Done`	BOOL	`Local:2:I.Data.5`	The bottle under the applicator already carries a label
`Label_Green`	BOOL	`Local:4:O.Data.14`	Labeler green indicator
`Label_Amber`	BOOL	`Local:4:O.Data.15`	Labeler amber indicator
`Cap_Green`	BOOL	`Local:4:O.Data.11`	Capper green indicator (column)
`Cap_Amber`	BOOL	`Local:4:O.Data.12`	Capper amber indicator (column)
`Cap_Low`	BOOL	`Local:1:I.Data.14`	Capper CapLow: the chute is down to its low count
`Cap_Present`	BOOL	`Local:1:I.Data.15`	Capper CapPresent: a cap is waiting at the chuck
`Cap_Fill_T`	TIMER		Cap feeder run-on after CapLow clears
`Thumbwheel`	INT	`Local:9:I.Ch0Data`	LogixPro I:3 · Thumbwheel switch, read as a number (BCD on the real one)
`Enter`	BOOL	`Local:3:I.Data.6`	Thumbwheel Enter button
`LS11`	BOOL	`Local:3:I.Data.4`	End-of-line eye at 1240: verifies the tracking (only large, whole bottles reach it)
`Main_Conv_Running`	BOOL	`Local:1:I.Data.1`	Main conveyor running feedback (motor contactor)
`Divert_Conv_Running`	BOOL	`Local:2:I.Data.12`	Divert conveyor running feedback
`Fault_Lamp`	BOOL	`Local:5:O.Data.6`	Red fault lamp on the console: flashing = unacknowledged alarm, steady = alarm still active
`Reset_PB`	BOOL	`Local:3:I.Data.5`	Blue Reset button: acknowledge alarms and clear the ones whose cause is gone
`Case_Run`	BOOL	`Local:5:O.Data.9`	Case lane drive: run (two-wire)
`Case_VFD_Reset`	BOOL	`Local:5:O.Data.11`	Case lane drive: fault reset (rising edge)

Tag	Type	Alias for	Description
`Case_Speed_Ref`	INT	`Local:8:O.Ch0Data`	Case lane drive speed reference: 0-10000 = 0-10 V = 0-60 Hz
`Case_VFD_Ready`	BOOL	`Local:10:I.Data.0`	Case lane drive ready (not tripped)
`Case_VFD_Running`	BOOL	`Local:10:I.Data.1`	Case lane drive output frequency above zero
`Case_At_Speed`	BOOL	`Local:10:I.Data.2`	Case lane drive at the commanded speed
`Case_VFD_Fault`	BOOL	`Local:10:I.Data.3`	Case lane drive tripped
`Case_Hz_Fbk`	INT	`Local:9:I.Ch2Data`	Case lane drive output frequency: 0-10000 = 0-10 V = 0-60 Hz
`Case_Slow`	BOOL	`Local:3:I.Data.15`	Slow-down eye on the case lane, 100 px before the packer head
`Case_In_Place`	BOOL	`Local:3:I.Data.10`	A case is centred under the packer head
`Case_Full`	BOOL	`Local:3:I.Data.11`	The case under the head has been packed
`Group_Ready`	BOOL	`Local:3:I.Data.8`	The packer infeed holds a case worth (4)
`Infeed_Full`	BOOL	`Local:3:I.Data.9`	Packer infeed buffer full: bottles arriving now wait on the belt
`Infeed_Count`	INT	`Local:9:I.Ch1Data`	Bottles in the packer infeed
`Packer_Ready`	BOOL	`Local:3:I.Data.7`	Packer head home, not cycling
`Pack_Drop`	BOOL	`Local:5:O.Data.7`	Packer: pick a case worth from the infeed and place it in the case (rising edge)
`Large_Bottle_Count`	DINT		LogixPro O:4/0 · Large bottle count, shown on the LARGE display
`Small_Bottle_Count`	DINT		LogixPro O:4/1 · Small bottle count, shown on the SMALL display
`Scrap_Bottle_Count`	DINT		LogixPro O:4/2 · Scrap bottle count, shown on the SCRAP display
`Scrap_Box_Count`	DINT		LogixPro O:4/3 · Scrap box count, shown on the BOXES display
`Main_Conv_A`	BOOL	`Local:1:I.Data.2`	Main_Conv encoder channel A: one rising edge per 40 px pitch
`Bottle_Feed_Emit`	BOOL	`Local:4:O.Data.0`	Bottle_Feed Emit: one bottle per rising edge
`Line_Run`	BOOL		Seal-in run bit (LogixPro B3:0/0)
`Conv_Index`	BOOL		Latched while the belt steps one pitch
`A_ONS`	BOOL		One-shot storage for the encoder edge
`Dwell`	TIMER		Belt stopped: sensors read, feeder drops a bottle

Tag	Type	Alias for	Description
`Exist`	DINT[2]		Bottle present, one bit per stop (LogixPro B3:10). During the dwell: filler 6, capper 9, scrap gate 13, divert gate 20; after the shift (Dwell.DN) one bit higher
`Large`	DINT[2]		Large bottle (LogixPro B3:20). During the dwell: filler 5, capper 8, scrap gate 12, divert gate 19; after the shift one bit higher
`Broken`	DINT[2]		Broken bottle (LogixPro B3:30). During the dwell: filler 4, capper 7, scrap gate 11, divert gate 18; after the shift one bit higher
`Exist_Ctl`	CONTROL		LogixPro R6:1
`Large_Ctl`	CONTROL		LogixPro R6:2
`Broken_Ctl`	CONTROL		LogixPro R6:3
`Fill_Req`	BOOL		A bottle stands under the filler and still needs its charge
`Fill_Done`	BOOL		Latched when the charge is in; cleared when the belt steps
`Cap_Req`	BOOL		A bottle stands under the cap ram and has not been capped
`Cap_Done`	BOOL		Latched when the ram has been down; cleared when the belt steps
`Label_Req`	BOOL		Latched while the labeler is asked to apply; cleared at PadIn
`Label_Applied`	BOOL		Latched when the label is pressed on; cleared when the belt steps
`Label_Fault`	BOOL		Latched on the labeler Error input (end of roll or a missed label); Enter clears it
`Alarm_Word`	DINT		Latched alarms, one bit each: 0 main conveyor overload, 1 divert conveyor overload, 2 end-of-line check failed (a jam, a lost bottle, tracking out of step), 3 LS1 dark too long (dirty lens or feeder empty), 4 labeler fault, 5 reservoir inlet open but level not rising, 6 case lane drive tripped, 7 packer infeed full
`Alarm_Unack`	BOOL		New alarm, not yet acknowledged (Reset)
`Main_OL_T`	TIMER		Main conveyor overload timer (no running feedback)

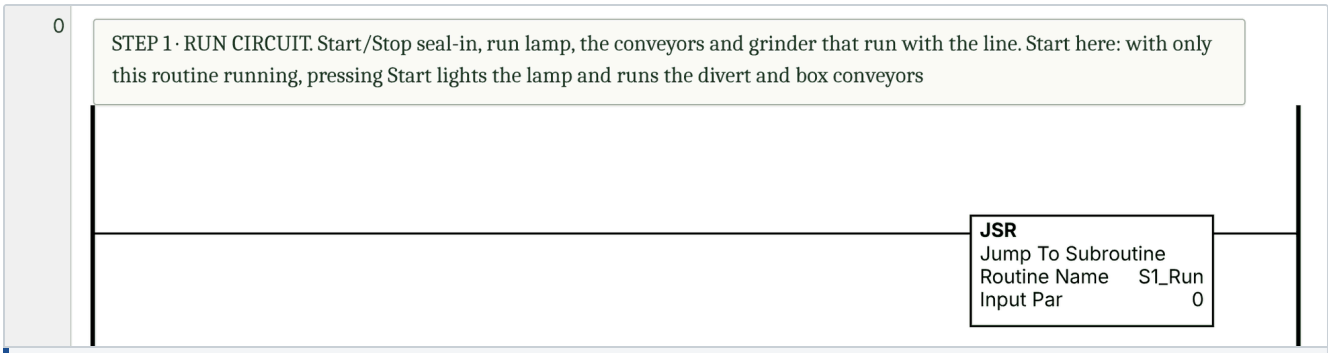
Tag	Type	Alias for	Description
`Div_OL_T`	TIMER		Divert conveyor overload timer (no running feedback)
`EOL_Missing`	BOOL		LS11 dark when the registers said a large, whole bottle was there
`EOL_Extra`	BOOL		LS11 saw a bottle the registers did not expect
`EOL_Inhibit`	BOOL		Tracking cleared by Reset: the end-of-line check waits until the first bottle tracked since then reaches LS11
`EOL_ONS`	BOOL		
`LS1_Dark_T`	TIMER		LS1 dark too long timer
`Inlet_T`	TIMER		Inlet not filling timer
`Flash_T`	TIMER		Fault lamp flasher
`OL_ONS`	BOOL		
`DOL_ONS`	BOOL		
`BLK_ONS`	BOOL		
`Reg_Clear`	BOOL		
`DRK_ONS`	BOOL		
`LBL_ONS`	BOOL		
`INL_ONS`	BOOL		
`RST_ONS`	BOOL		
`VFD_ONS`	BOOL		
`INF_ONS`	BOOL		
`Case_Hz`	REAL		Case lane speed the program wants, in Hz
`Case_Hz_Actual`	REAL		Case lane drive output frequency read back, in Hz
`Case_Count`	DINT		Cases packed, shown on the CASES display
`Case_Creep`	BOOL		Latched from the slow-down eye until an empty case stands under the head
`Drop_ONS`	BOOL		
`Case_ONS`	BOOL		
`Box_Full`	BOOL	`Local:2:I.Data.9`	Grinder BoxFull: the box under the scrap drop holds its share

Tag	Type	Alias for	Description
`Box_Level`	INT	`Local:7:I.Ch3Data`	Grinder BoxLevel: bottles in the box under the chute (the IN BOX display)
`Scrap_Waiting`	BOOL	`Local:2:I.Data.8`	Grinder Detect: a broken bottle is waiting in the hopper
`Scrap_ONS`	BOOL		
`Divert_ONS`	BOOL		
`Small_ONS`	BOOL		
`Large_ONS`	BOOL		
`Box_ONS`	BOOL		
`Enter_ONS`	BOOL		
`Batch_Done`	BOOL		The large-bottle count has reached the thumbwheel preset
`Tank_Level`	INT	`Local:7:I.Ch2Data`	Filler head tank, 0-100 %
`Tank_Low`	BOOL	`Local:1:I.Data.12`	Filler head tank below 15 %
`Syrup_Low`	BOOL	`Local:1:I.Data.8`	Syrup_Tank low float
`Syrup_High`	BOOL	`Local:1:I.Data.7`	Syrup_Tank high float
`Syrup_Inlet`	BOOL	`Local:4:O.Data.2`	Syrup_Tank inlet valve
`Syrup_Level`	INT	`Local:7:I.Ch1Data`	Syrup_Tank level, 0-100 %
`Tank_Red`	BOOL	`Local:4:O.Data.3`	Tank_Light red: reservoir at the low float (flashes)
`Tank_Amber`	BOOL	`Local:4:O.Data.4`	Tank_Light amber: inlet open, refilling
`Tank_Green`	BOOL	`Local:4:O.Data.5`	Tank_Light green: level healthy

Appendix B — The complete program

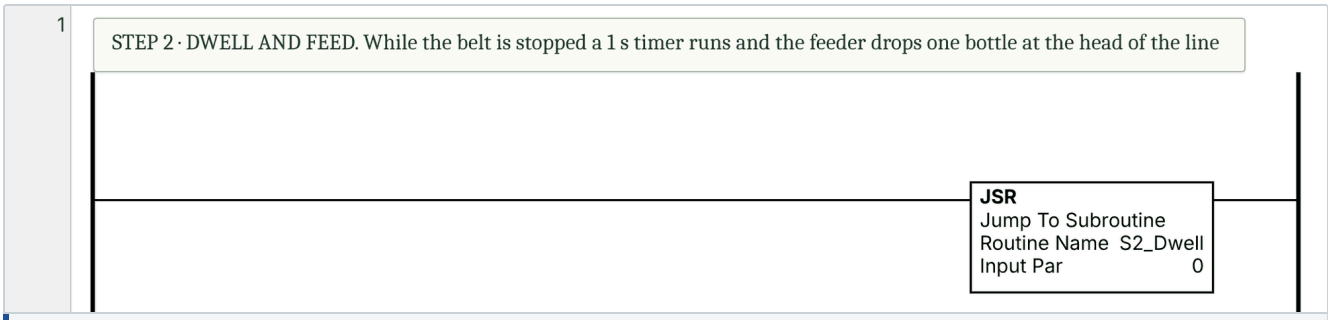
MainRoutine

RUNG 0



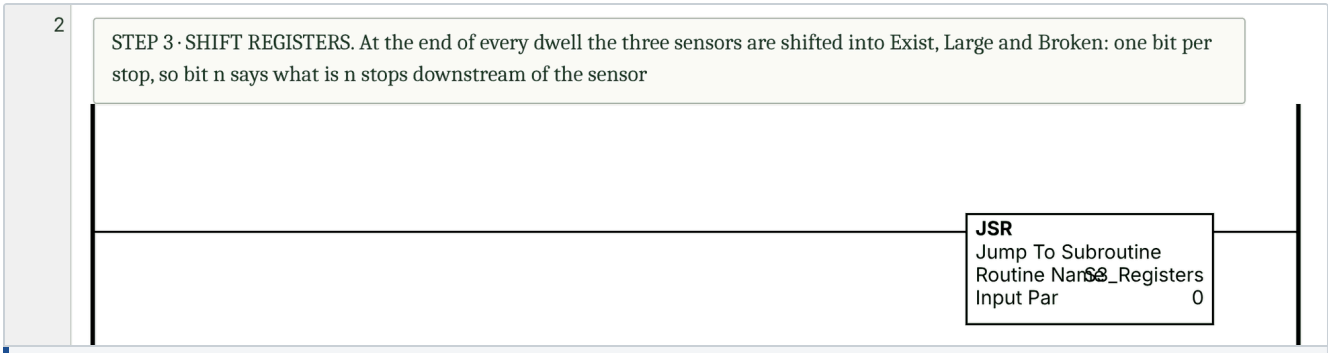

```
JSR(S1_Run);
```

RUNG 1




```
JSR(S2_Dwell);
```

RUNG 2




```
JSR(S3_Registers);
```

RUNG 3

3

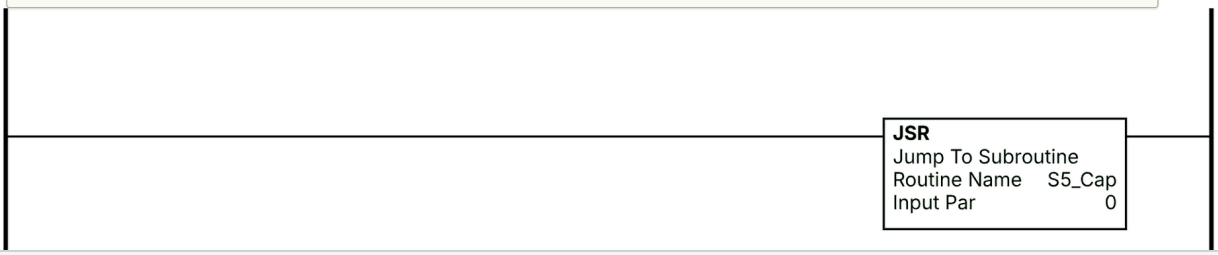
STEP 4 · FILL. A bottle under the nozzle (Exist bit 7 after the shift) gets its large or small charge from the head tank before the belt is allowed to step

```
JSR(S4_Fill);
```

RUNG 4

4

STEP 5 · CAP. A bottle under the chuck (Exist bit 10 after the shift) gets a cap from the chute; the cap feeder keeps the chute full and the two column lights say what the capper is doing

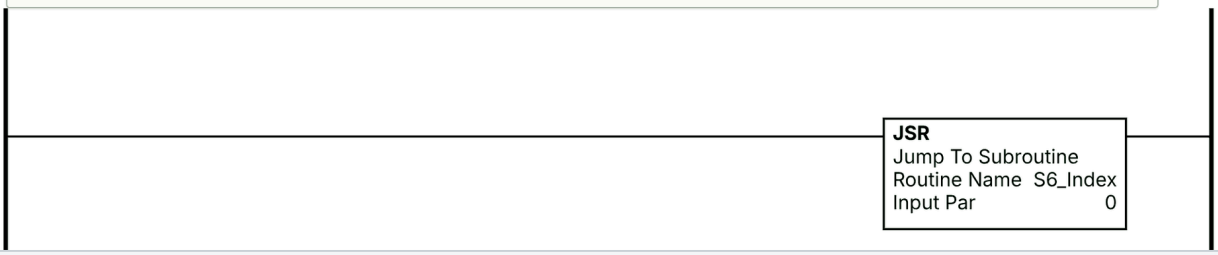



```
JSR(S5_Cap);
```

RUNG 5

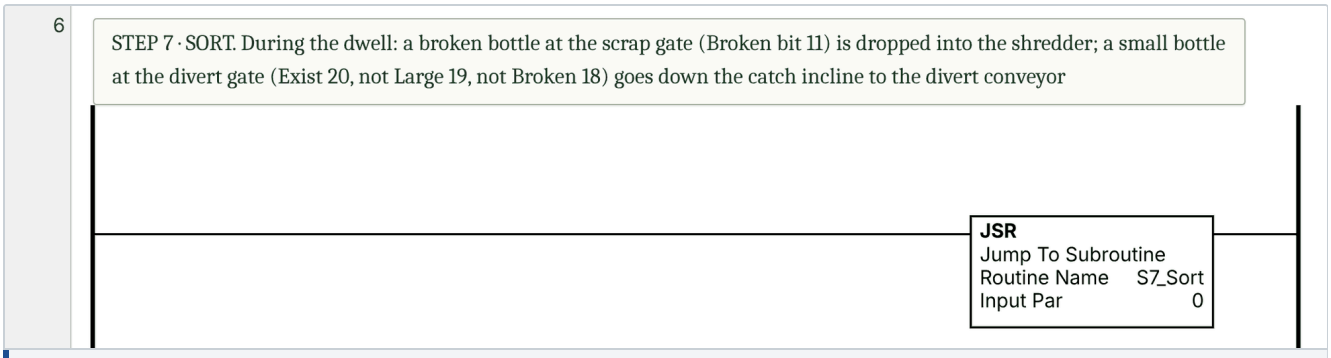
5

STEP 6 · INDEX. When the dwell is done and nothing is holding the belt, step exactly one pitch on the encoder. This runs after Fill and Cap so their requests are seen in the same scan



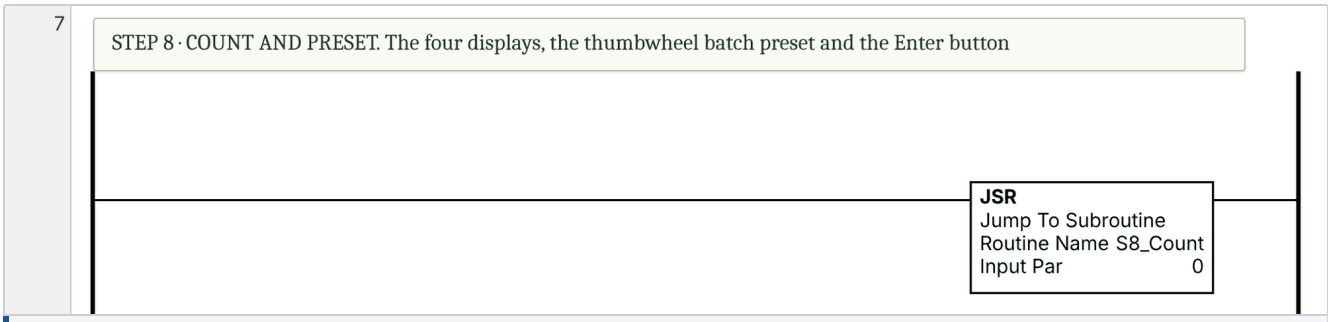

```
JSR(S6_Index);
```

RUNG 6



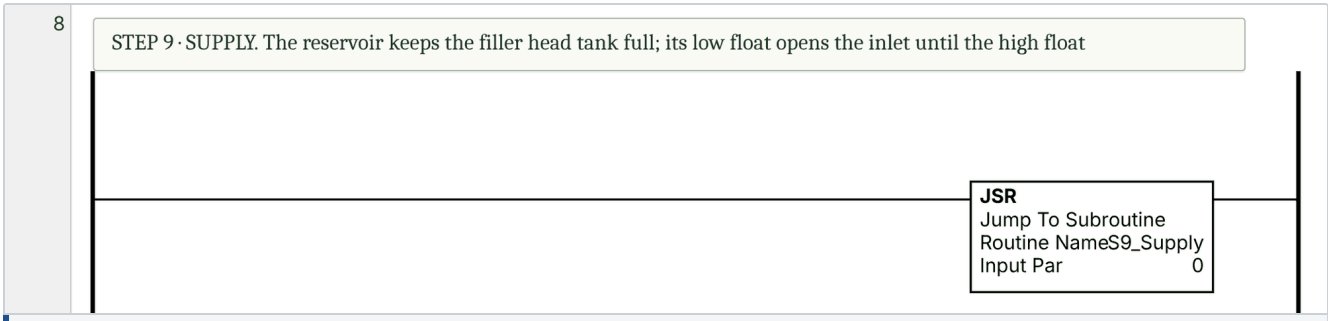

```
JSR(S7_Sort);
```

RUNG 7




```
JSR(S8_Count);
```

RUNG 8

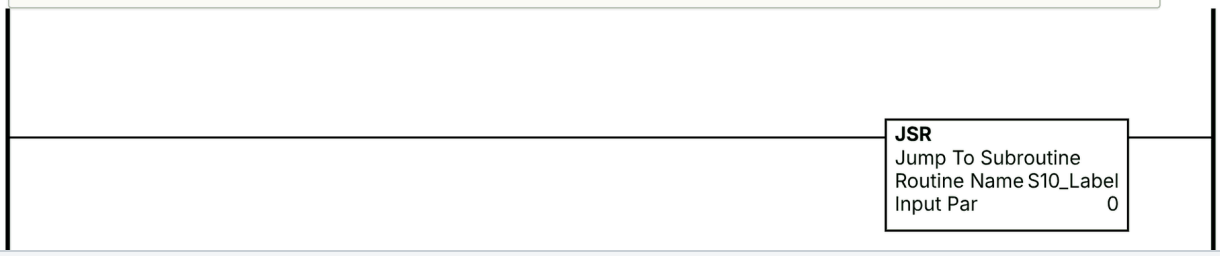



```
JSR(S9_Supply);
```

RUNG 9

9

STEP 10 · LABEL. One stop past the capper (Exist bit 11 after the shift) an FLS-style dispenser presses a label on every whole, capped bottle: start only when Ready, done at PadIn, fault latched on Error and cleared with Enter




```
JSR(S10_Label);
```

RUNG 10

10

STEP 11 · ALARMS. Machine faults your ladder has to notice: a motor commanded with no running feedback, a sensor blocked or dark far longer than normal, the labeler fault, an inlet that does not fill. Each latches a bit in Alarm_Word; the red lamp flashes until Reset; the faults that make the line unsafe to run stop it

```
JSR(S11_Alarms);
```

RUNG 11

11

STEP 12 · PACKING. Large bottles at the end of the line go into the case packer's infeed; four at a time are packed into a case. The case lane runs on a VFD: fast, then a creep speed from the slow-down eye so the case stops under the head, scaled from Hz to the 0-10 V analog output with SCP

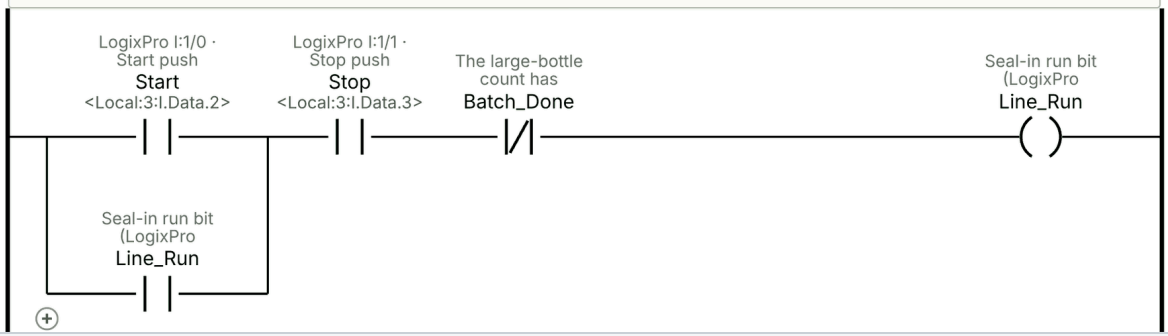
```
JSR(S12_Pack);
```

S1_Run

RUNG 0

0

SEAL-IN: Start latches Line_Run through its own contact; Stop is a normally closed button (1 until pressed) so it breaks the seal; Batch_Done (step 8) also stops the line

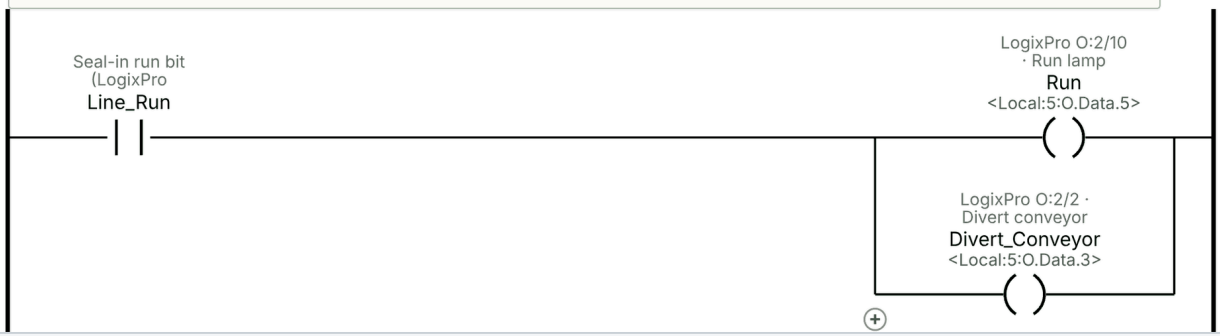



```
[XIC(Start),XIC(Line_Run)]XIC(Stop)XIO(Batch_Done)OTE(Line_Run);
```

RUNG 1

1

Run lamp and divert conveyor run whenever the line runs. The main conveyor steps (step 6); the box conveyor and shredder have their own rules (step 7)




```
XIC(Line_Run)[OTE(Run),OTE(Divert_Conveyor)];
```

RUNG 2

2

Line stopped: drop a label request in progress so the head goes home. Nothing else is forgotten on purpose: a step in progress (Conv_Index) and the done bits stay as they are, because the bottles have not moved, so Start carries on exactly where Stop left off and the registers stay in step with the bottles. Step 11's end-of-line check would catch it if they did not

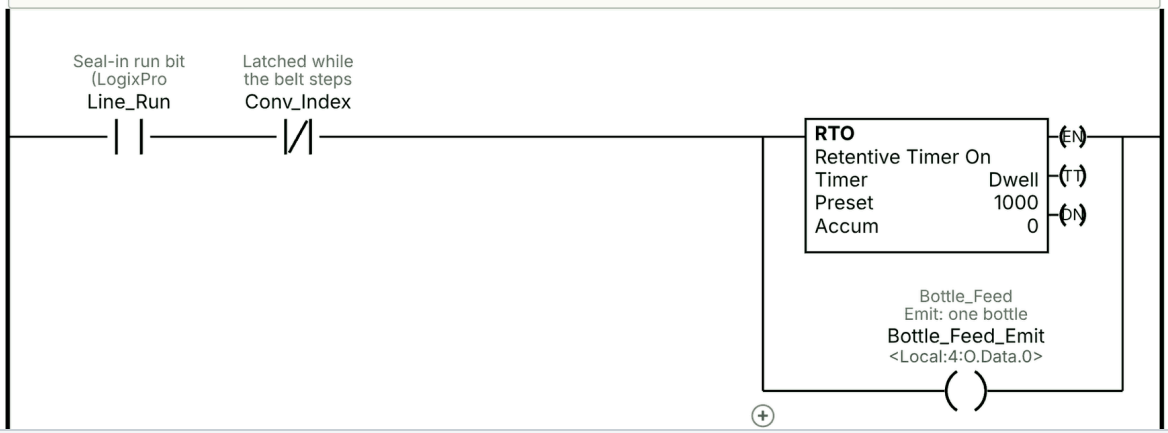
```
XIO(Line_Run)OTU(Label_Req);
```

S2_Dwell

RUNG 0

0

BELT STOPPED: a 1 s dwell runs and the feeder output is held on. Bottle_Feed is in edge mode, so it drops exactly one bottle per rising edge, that is one per stop. RTO, not TON: if the line is stopped in the middle of a dwell the time already run is kept (and a finished dwell stays done), so Start carries on and the registers do not shift twice for one stop. A new bottle is not dropped either: the feeder only drops when its exit is clear




```
XIC(Line_Run)XIO(Conv_Index)[RTO(Dwell,1000,0),OTE(Bottle_Feed_Emit)];
```

RUNG 1

1

The belt is stepping: reset the dwell for the next stop. A retentive timer only clears with RES

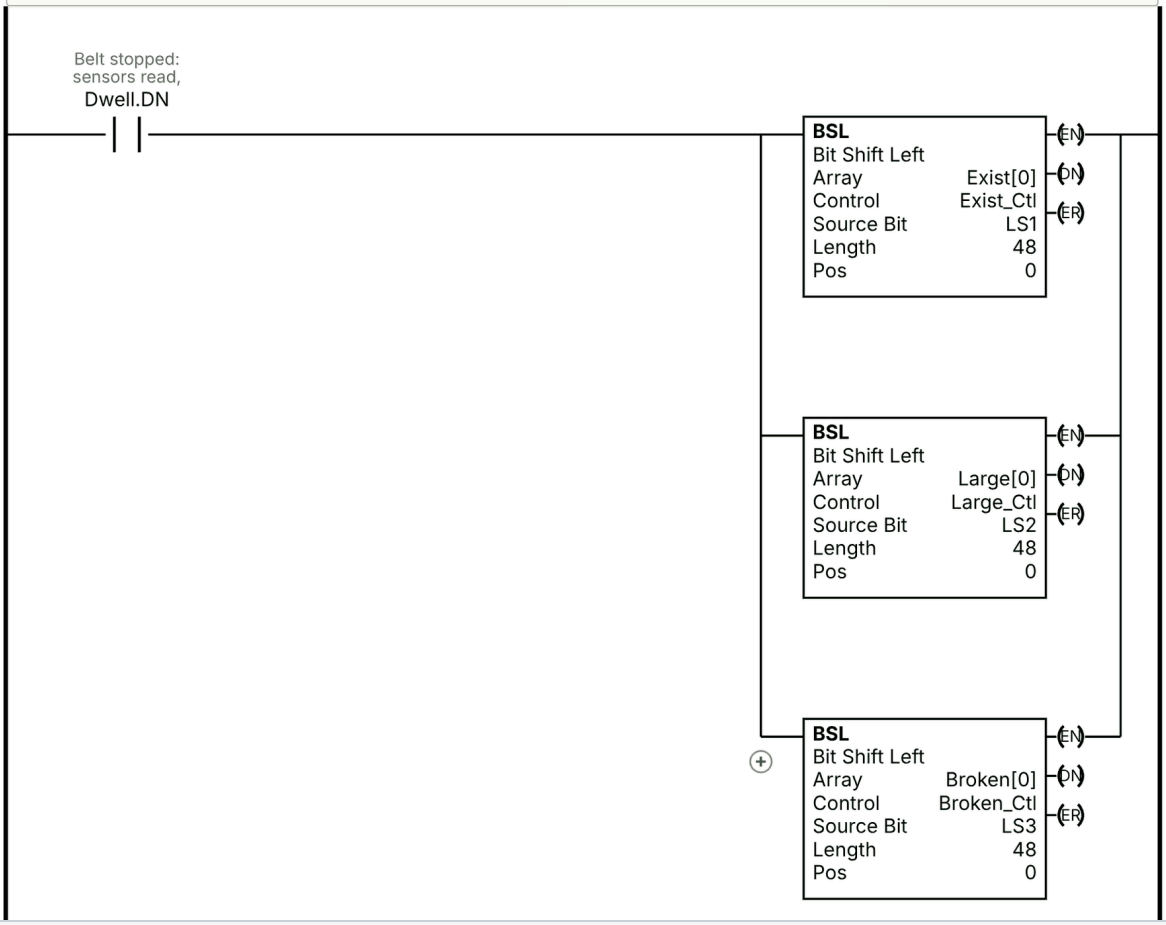
```
XIC(Conv_Index)RES(Dwell);
```

S3_Registers

RUNG 0

0

SHIFT ONCE PER STOP, while the bottles stand still: LS1 (bottle present) into Exist, LS2 (large) into Large, LS3 (broken) into Broken. Watch Exist[0] in Binary style in Controller Tags and see the bits walk down the line one place per stop



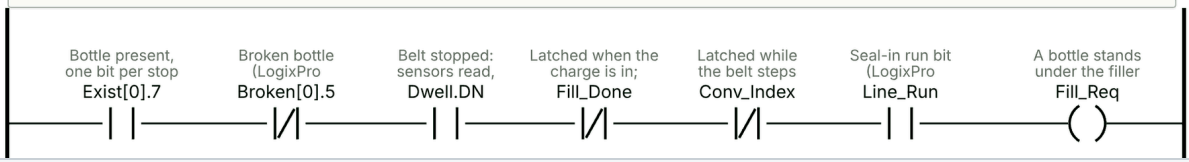

```
XIC(Dwell.DN)
[BSL(Exist[0],Exist_Ctl,LS1,48),BSL(Large[0],Large_Ctl,LS2,48),BSL(Broken[0],Br
oken_Ctl,LS3,48)];
```

S4_Fill

RUNG 0

0

FILL REQUEST: the registers have just shifted (Dwell.DN), Exist bit 7 says a bottle stands under the nozzle and Broken bit 5 says it is whole; it has not had its charge yet. The shift happens at the END of the dwell, so a rung that runs after it reads one bit higher than a rung that runs during the dwell. A broken bottle gets no charge: it would only run out onto the belt

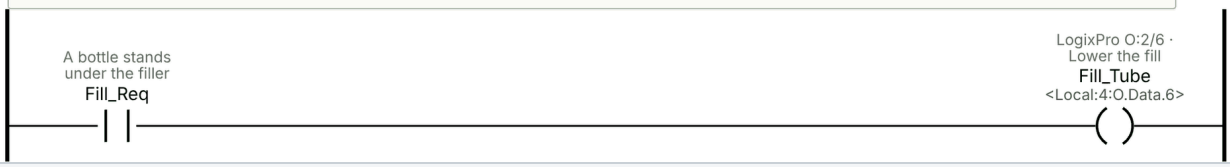



```
XIC(Exist[0].7)XIO(Broken[0].5)XIC(Dwell.DN)XIO(Fill_Done)XIO(Conv_Index)XIC(Li
ne_Run)OTE(Fill_Req);
```

RUNG 1

1

Tube down while a fill is requested; it comes back up the moment Fill_Done latches

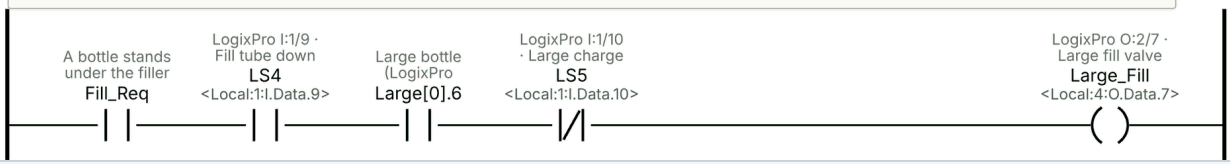



```
XIC(Fill_Req)OTE(Fill_Tube);
```

RUNG 2

2

LARGE BOTTLE: open the large valve once the tube is down (LS4) until the large charge is in (LS5). Large[0].6, not .9: LS2 sits 40 px past LS1 and sees the bottle earlier, so the Large register runs three bits ahead of Exist at the same stop

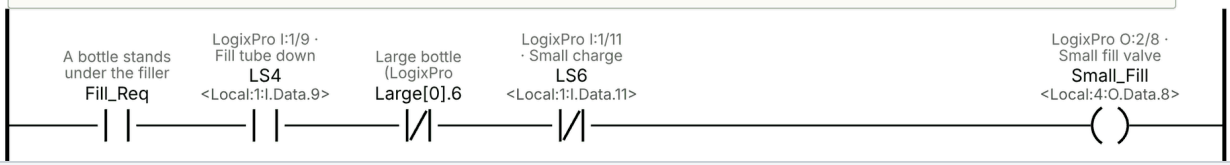



```
XIC(Fill_Req)XIC(LS4)XIC(Large[0].6)XIO(LS5)OTE(Large_Fill);
```

RUNG 3

3

SMALL BOTTLE: the small valve until the small charge is in (LS6)

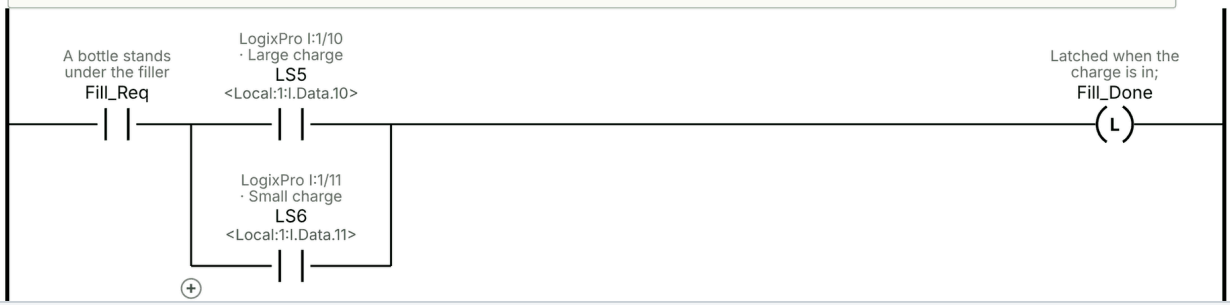



```
XIC(Fill_Req)XIC(LS4)XIO(Large[0].6)XIO(LS6)OTE(Small_Fill);
```

RUNG 4

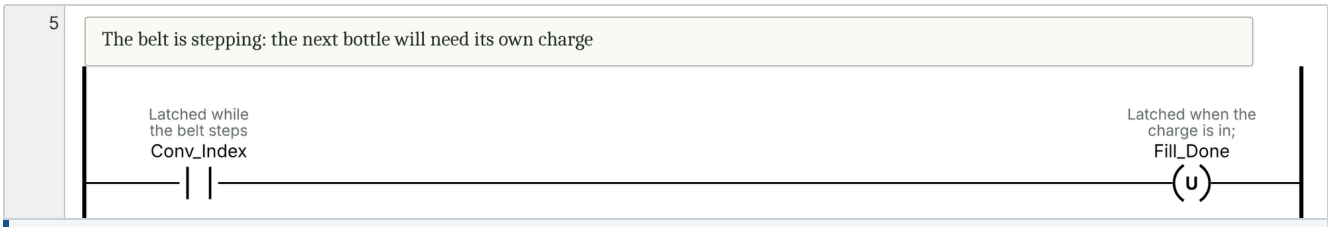
4

Charge delivered: latch Fill_Done, which drops Fill_Req, closes the valve and lifts the tube




```
XIC(Fill_Req)[XIC(LS5),XIC(LS6)]OTL(Fill_Done);
```

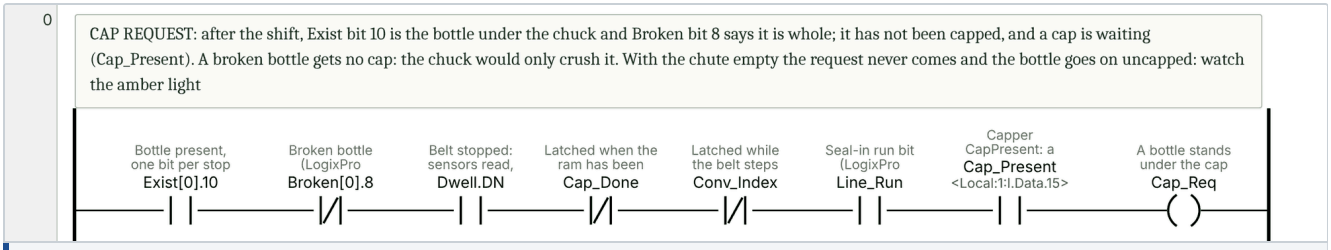
RUNG 5




```
XIC(Conv_Index)OTU(Fill_Done);
```

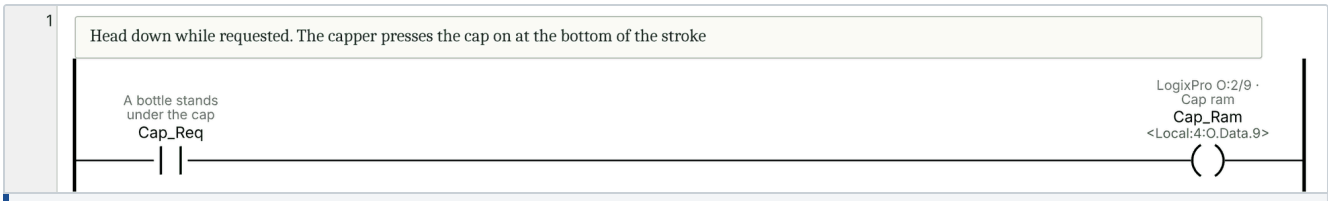
S5_Cap

RUNG 0



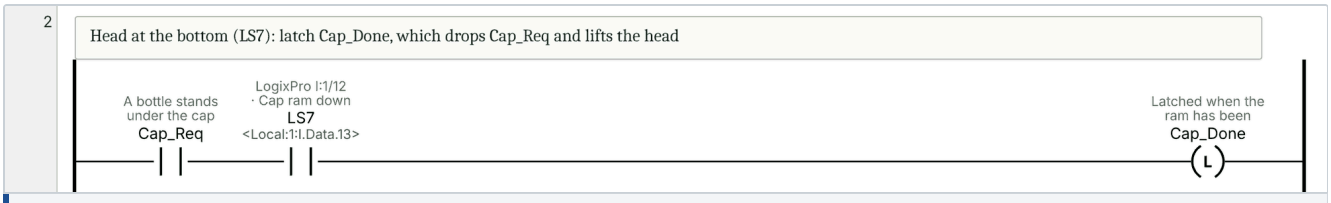

```
XIC(Exist[0].10)XIO(Broken[0].8)XIC(Dwell.DN)XIO(Cap_Done)XIO(Conv_Index)XIC(Line_Run)XIC(Cap_Present)OTE(Cap_Req);
```

RUNG 1



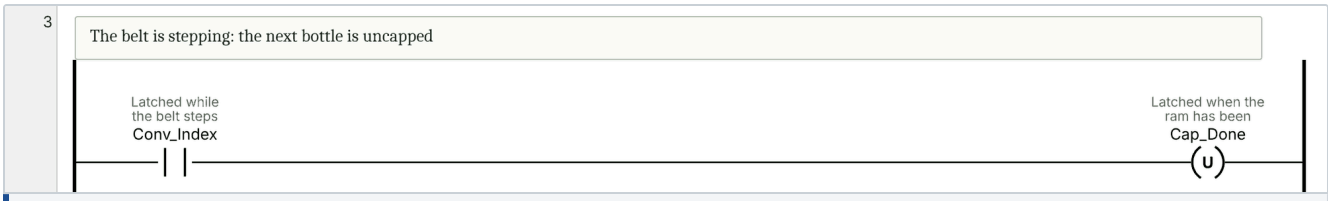

```
XIC(Cap_Req)OTE(Cap_Ram);
```

RUNG 2



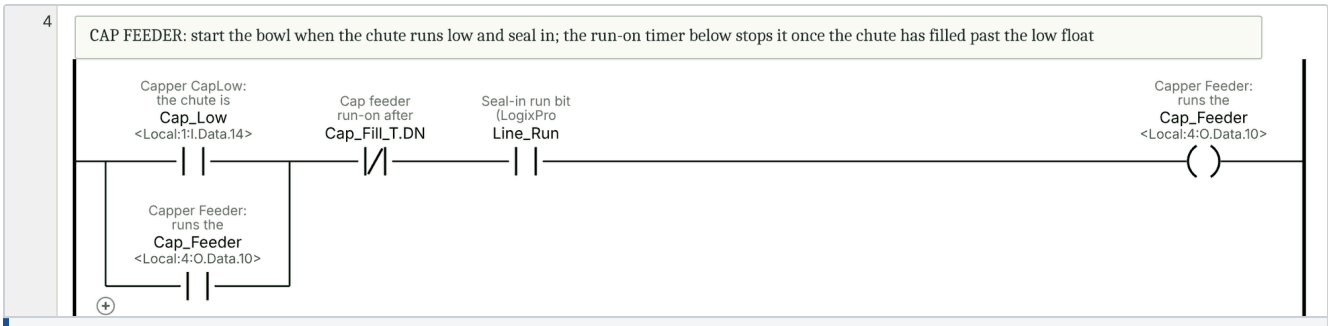

```
XIC(Cap_Req)XIC(LS7)OTL(Cap_Done);
```

RUNG 3



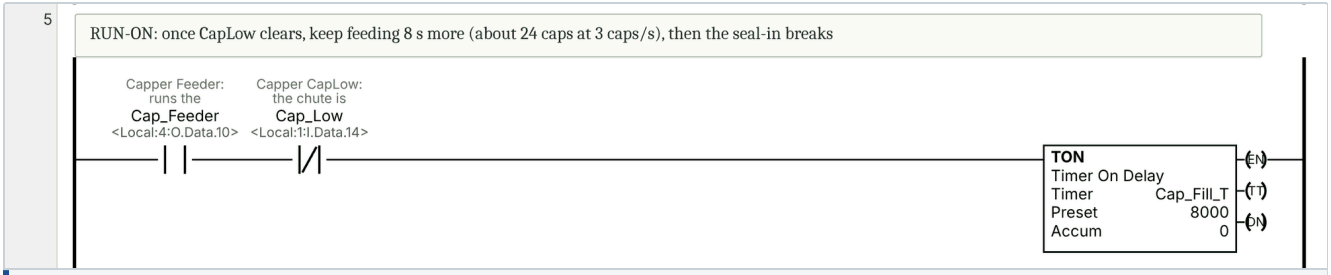

```
XIC(Conv_Index)OTU(Cap_Done);
```

RUNG 4



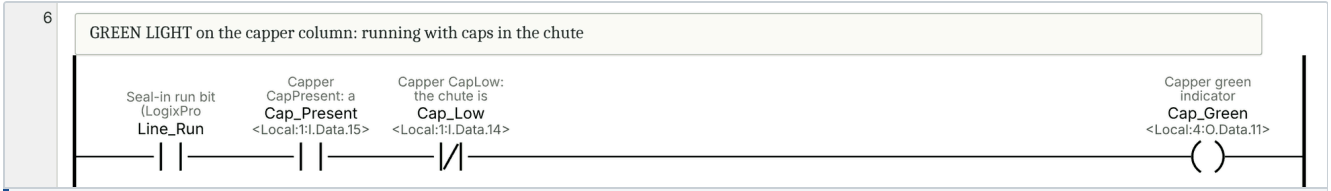

```
[XIC(Cap_Low),XIC(Cap_Feeder)]XIO(Cap_Fill_T.DN)XIC(Line_Run)OTE(Cap_Feeder);
```

RUNG 5



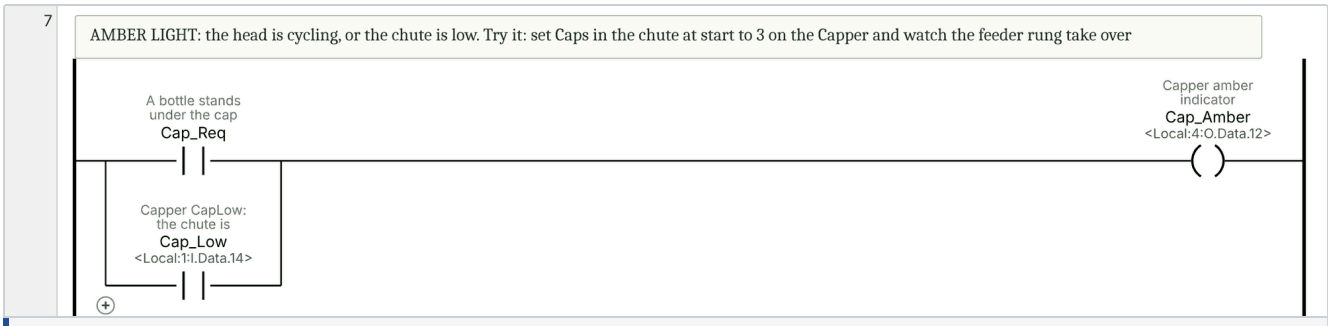

```
XIC(Cap_Feeder)XIO(Cap_Low)TON(Cap_Fill_T,8000,0);
```

RUNG 6




```
XIC(Line_Run)XIC(Cap_Present)XIO(Cap_Low)OTE(Cap_Green);
```

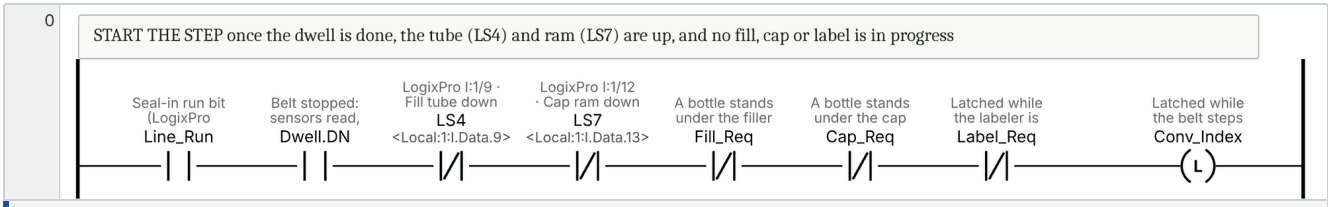
RUNG 7




```
[XIC(Cap_Req),XIC(Cap_Low)]OTE(Cap_Amber);
```

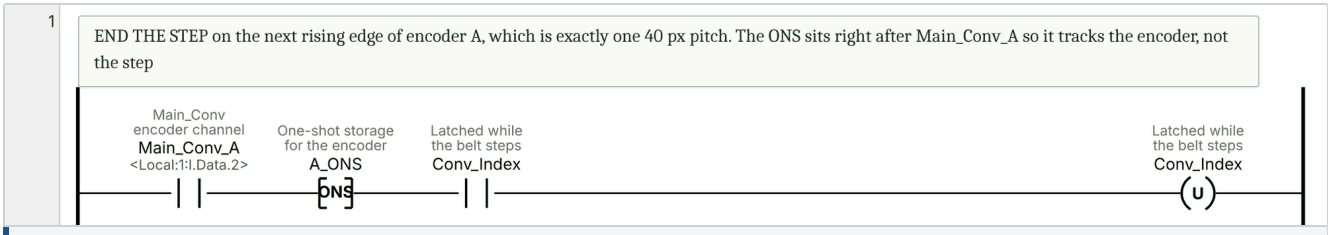
S6_Index

RUNG 0



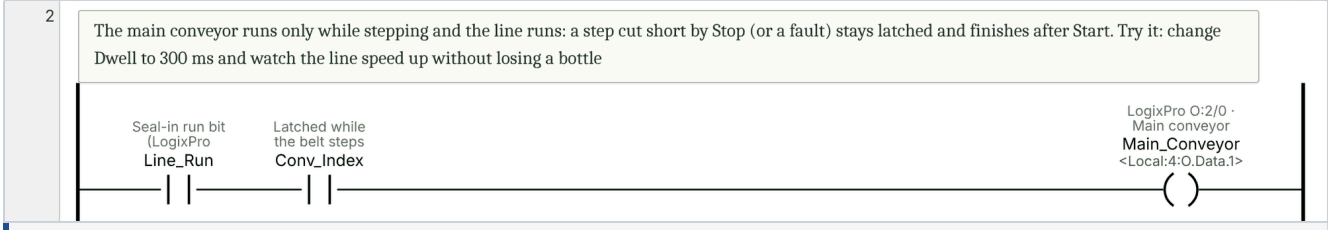

```
XIC(Line_Run)XIC(Dwell.DN)XIO(LS4)XIO(LS7)XIO(Fill_Req)XIO(Cap_Req)XIO(Label_Req)OTL(Conv_Index);
```

RUNG 1




```
XIC(Main_Conv_A)ONS(A_ONS)XIC(Conv_Index)OTU(Conv_Index);
```

RUNG 2




```
XIC(Line_Run)XIC(Conv_Index)OTE(Main_Conveyor);
```

S7_Sort

RUNG 0

0

SCRAP: while the belt stands still (Dwell timing) Broken bit 11 is the bottle at the scrap gate; the arm swings it off the raised belt and it drops into the shredder hopper below. The gate works during the dwell on purpose: the registers shift at the end of the dwell, and an arm that fired after the shift would push the bottle still standing at the gate, not the one the register names

```
XIC(Dwell.TT)XIC(Broken[0].11)OTE(Scrap_Gate);
```

RUNG 1

1

DIVERT: a small, unbroken bottle at the divert gate (Exist bit 20, not Large 19, not Broken 18, all in the dwell frame) is swung off the raised belt and rolls down the catch incline onto the divert conveyor. Large bottles carry on to the end of the line. The Broken bit matters: a broken bottle was already scrapped, but its Exist bit is still set, so without it the arm would fire at an empty slot

```
XIC(Dwell.TT)XIC(Exist[0].20)XIO(Large[0].19)XIO(Broken[0].18)OTE(Divert_Gate);
```

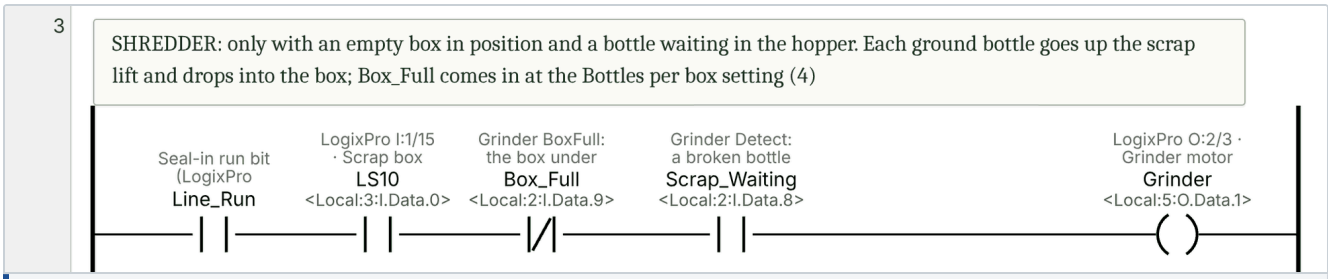
RUNG 2

2

BOX CONVEYOR: run until a box reaches LS10, which puts it under the scrap drop; run again once that box is full to carry it out and bring the next one in

```
XIC(Line_Run)[XIO(LS10),XIC(Box_Full)]OTE(Scrap_Conveyor);
```

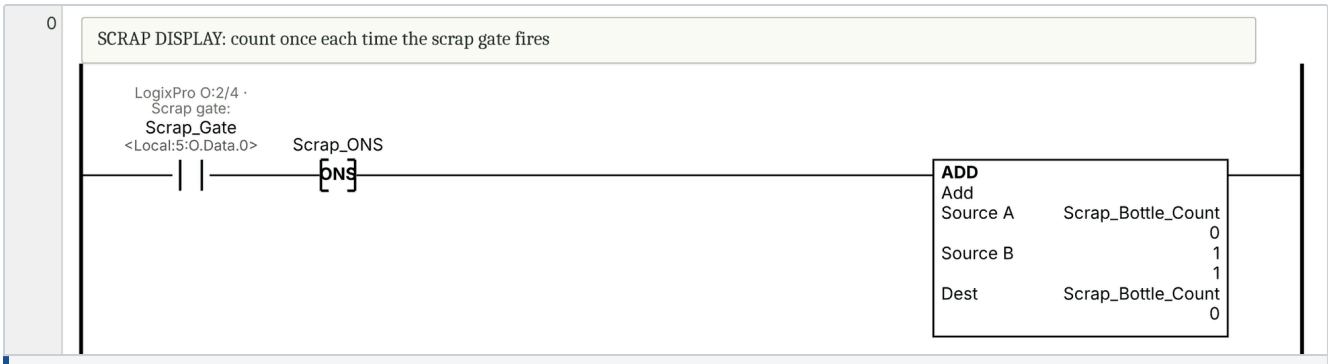
RUNG 3




```
XIC(Line_Run)XIC(LS10)XIO(Box_Full)XIC(Scrap_Waiting)OTE(Grinder);
```

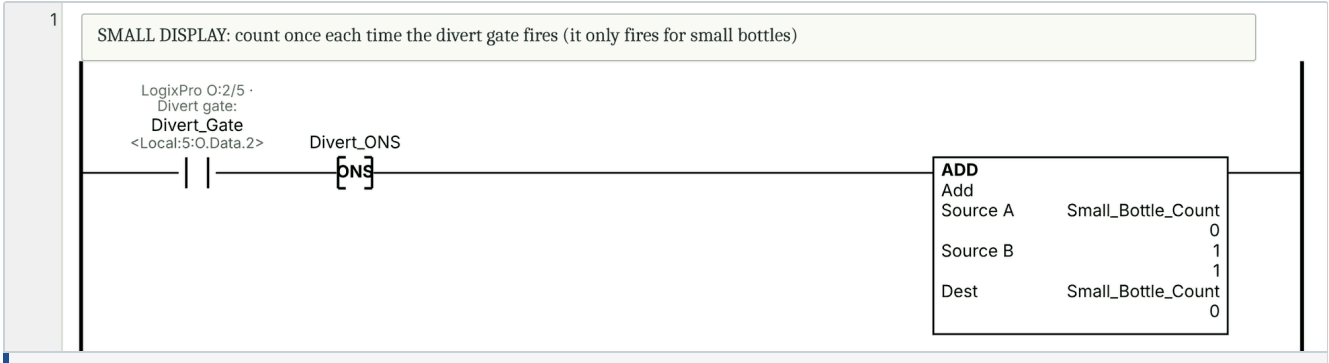
S8_Count

RUNG 0




```
XIC(Scrap_Gate)ONS(Scrap_ONS)ADD(Scrap_Bottle_Count,1,Scrap_Bottle_Count);
```

RUNG 1

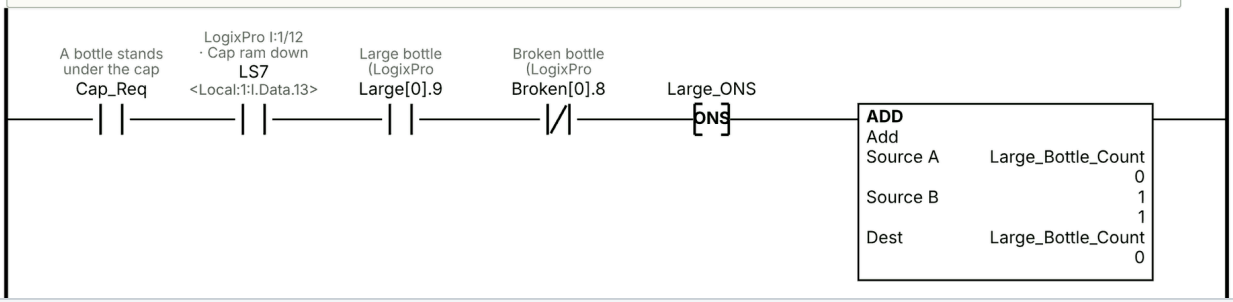



```
XIC(Divert_Gate)ONS(Divert_ONS)ADD(Small_Bottle_Count,1,Small_Bottle_Count);
```

RUNG 2

2

LARGE DISPLAY: a large, unbroken bottle gets capped (ram at the bottom, Large bit 9 / Broken bit 8 after the shift): one count




```
XIC(Cap_Req)XIC(LS7)XIC(Large[0].9)XIO(Broken[0].8)ONS(Large_ONS)ADD(Large_Bott
le_Count,1,Large_Bottle_Count);
```

RUNG 3

3

BOXES DISPLAY: one count each time a box fills

```
XIC(Box_Full)ONS(Box_ONS)ADD(Scrap_Box_Count,1,Scrap_Box_Count);
```

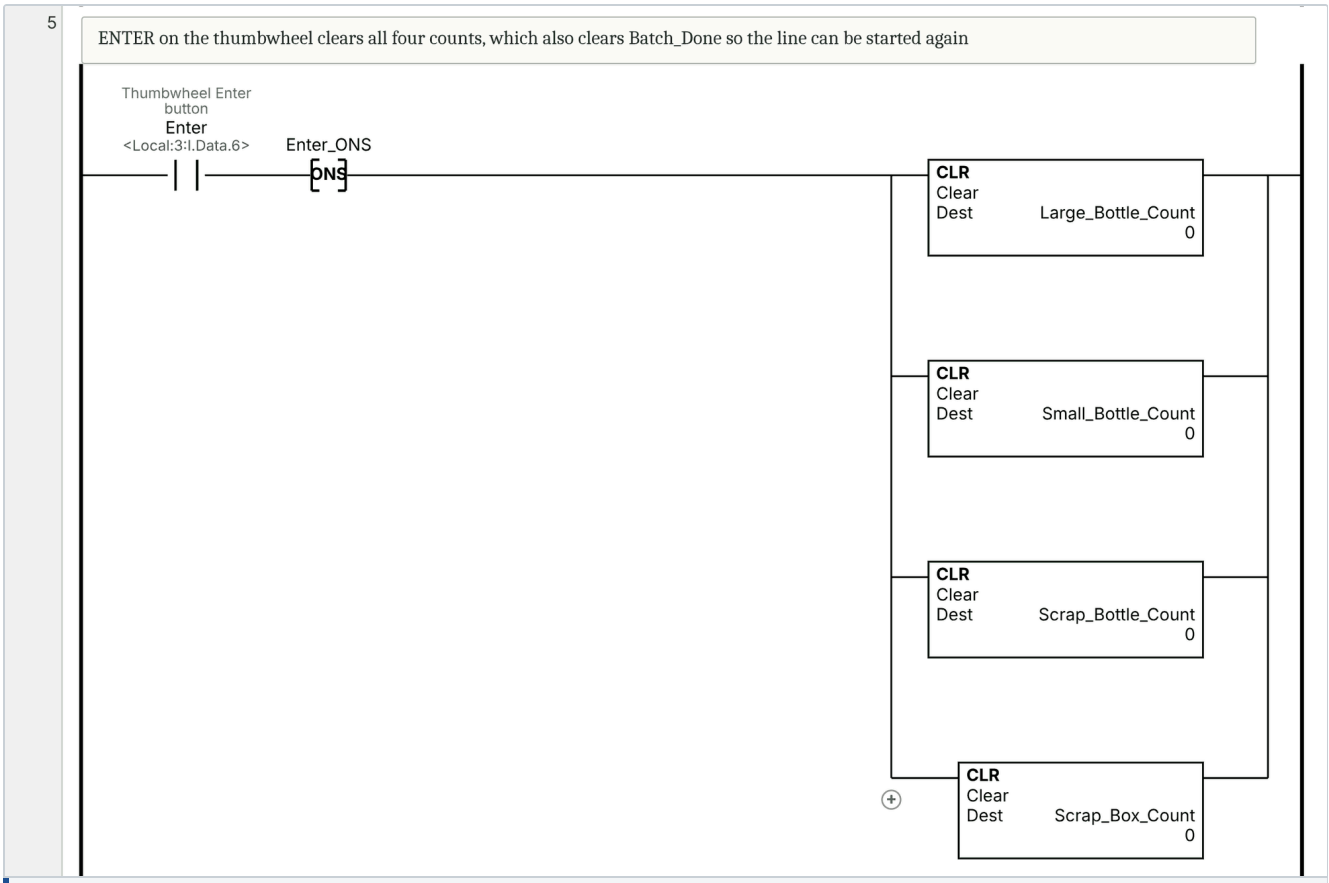
RUNG 4

4

BATCH PRESET: dial a number on the thumbwheel and the line stops itself when that many large bottles have been capped (0 = no limit). Batch_Done breaks the seal-in in step 1

```
GRT(Thumbwheel,0)GEQ(Large_Bottle_Count,Thumbwheel)OTE(Batch_Done);
```

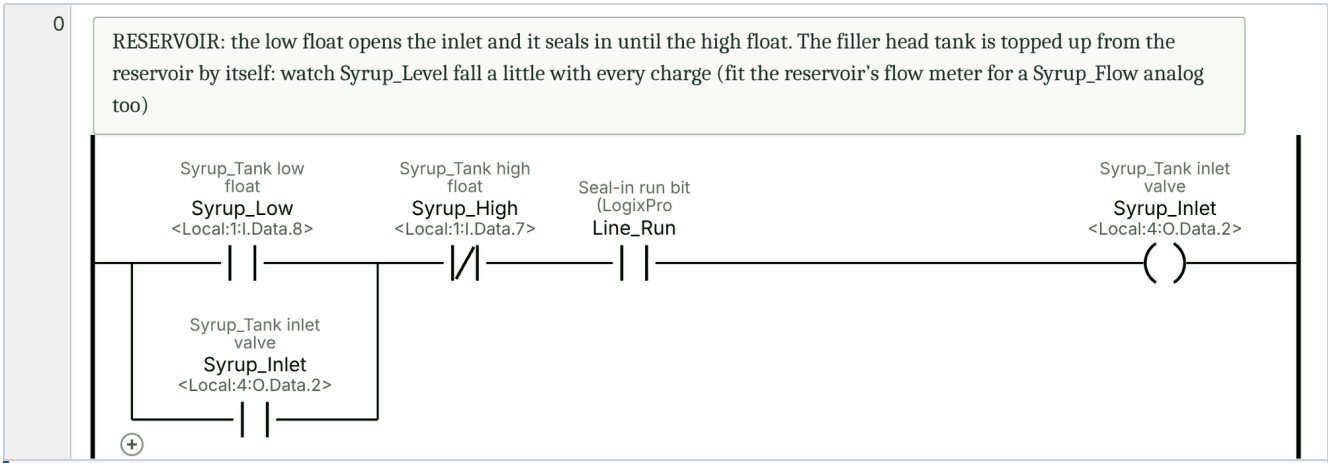
RUNG 5




```
XIC(Enter)ONS(Enter_ONS)
[CLR(Large_Bottle_Count),CLR(Small_Bottle_Count),CLR(Scrap_Bottle_Count),CLR(Scrap_Box_Count)];
```

S9_Supply

RUNG 0




```
[XIC(Syrup_Low),XIC(Syrup_Inlet)]XIO(Syrup_High)XIC(Line_Run)OTE(Syrup_Inlet);
```

RUNG 1

1

PROTECT THE FILL: if the head tank runs low (reservoir empty, inlet too slow) stop the line rather than pour short charges. Try it: set the Syrup_Tank Inlet flow to 10 and see this rung trip

```
XIC(Tank_Low)XIC(Line_Run)OTU(Line_Run);
```

RUNG 2

2

TANK STACK LIGHT, RED: the reservoir is at its low float (the light is set to flash its red segment)

```
XIC(Syrup_Low)OTE(Tank_Red);
```

RUNG 3

3

AMBER: the inlet is open, the reservoir is refilling

```
XIC(Syrup_Inlet)OTE(Tank_Amber);
```

RUNG 4

4

GREEN: level healthy and not refilling. Try it: make amber mean head-tank low instead

```
XIO(Syrup_Low)XIO(Syrup_Inlet)OTE(Tank_Green);
```

Appendix C — The register bit map

Station	x	Stops after LS1 / LS2 / LS3	Exist bit	Large bit	Broken bit	Frame
Filler	440	7 / 6 / 5	7	6	5	after the shift (Dwell.DN)
Capper	560	10 / 9 / 8	10	9	8	after the shift
Labeler	600	11 / 10 / 9	11	10	9	after the shift
Scrap gate	720	14 / 13 / 12	13	12	11	during the dwell (Dwell.TT)
Divert gate	1000	21 / 20 / 19	20	19	18	during the dwell
LS11 (end-of-line check)	1240	27 / 26 / 25	26	25	24	during the dwell, ACC ≥ 500

During the dwell a register describes the stop one pitch behind its count (it was last shifted before the previous step); after the shift it describes the count itself. Pitch 40 px; LS1 at 160, LS2 at 200, LS3 at 240; the registers are 48 bits long.

Appendix D — Sensor reference

Sensors detect parts, measure distance and tell part types apart. Every sensor is placed over a conveyor and looks at the lane beneath it; **60 px = 1 m**, so the default ranges follow the real devices they are modelled on. Rotate a sensor with its right-click menu or R; drag it along a conveyor and it snaps to the lane. Each sensor below lists its LEDs, what it detects, its range, and a table of the controller tags for every configuration (the Output setting in the property sheet).

Weighing Conveyor

- **Settings:** Speed (mm/s)

Tag	Controller I/O	Type	Description
Device_Fwd	Output	BOOL	Motor forward
Device_Weight	Input	INT	Weight on the scale, kg
Device_Running	Input	BOOL	Motor aux contact

Fill Level Sensor (liquid in the bottle)

- **Settings:** Fill line (px above the belt; a small bottle is 40 tall, a large 56; the liquid tops out at about 80 % of that); Sensor; Analog output (adds a Percent analog input: fill of the bottle under the sensor, 0-100 % of its label)

Tag	Controller I/O	Type	Description
Device_Detect	Input	BOOL	Liquid at or above the fill line in the bottle under the sensor

Diffuse Sensor (photoelectric)

Diffuse photoelectric sensor: its own light reflects off the part, so no reflector is needed. The shaded band on the lane shows the sensing range.

- **LED:** Red: detecting.
- **Detectable:** Solids: any part.
- **Sensing range:** 0 – 100 px, default 20 px (≈ 0.3 m). Factory I/O: 0 – 1.6 m.
- **Settings:** Sensing range (px; 60 px = 1 m); Sensitivity (%): the gain. 100 sees every part at full range; lower it and the range shrinks and dull targets (glass bottles, broken glass, dark cartons) drop out before bright ones (metal, cans, white and yellow cartons); Beam; Sees

Tag	Controller I/O	Type	Description
Device_Detect	Input	BOOL	Detecting: a solid part is within range

Retroreflective Sensor and Reflector

The sensor and its reflector face each other across the conveyor. **Detect** is 1 while the beam is interrupted. Use the beam height to ignore low parts: a lid (8 px) passes under a 12 px beam, a carton does not.

- **LED:** Green: aligned with the reflector. Yellow: beam not interrupted (goes out while a part blocks it).
- **Detectable:** Solids taller than the beam height.
- **Sensing range:** Across the lane (≈ 1 m here; the real device reaches 6 m).
- **Settings:** Beam height (px above the belt)

Tag	Controller I/O	Type	Description
Device_Detect	Input	BOOL	Light beam interrupted

Light Array (8 beams)

Eight parallel beams form a light curtain. Beam 1 is lowest. **Digital** gives one BOOL per beam, **Numerical** packs the beams into one INT (bit 0 = beam 1), **Analog** gives 10 V × interrupted beams ÷ 8 as hundredths of a volt. With the default 6 px spacing a small carton (22 px) breaks beams 1-4, a medium one 5, a large one 7, a crate all 8, a lid only beam 1.

- **LED:** Green: emitter and receiver synchronised. One red LED per interrupted beam.
- **Detectable:** Solids; measures how tall a part is.
- **Sensing range:** Across the lane; 8 beams spaced by the beam-spacing setting.
- **Settings:** Output; Beam spacing (px of part height)

Configuration: digital

Tag	Controller I/O	Type	Description
Device_Beam1	Input	BOOL	Beam 1 interrupted (3 px above the belt)
Device_Beam2	Input	BOOL	Beam 2 interrupted (9 px above the belt)
Device_Beam3	Input	BOOL	Beam 3 interrupted (15 px above the belt)
Device_Beam4	Input	BOOL	Beam 4 interrupted (21 px above the belt)
Device_Beam5	Input	BOOL	Beam 5 interrupted (27 px above the belt)
Device_Beam6	Input	BOOL	Beam 6 interrupted (33 px above the belt)
Device_Beam7	Input	BOOL	Beam 7 interrupted (39 px above the belt)
Device_Beam8	Input	BOOL	Beam 8 interrupted (45 px above the belt)

Configuration: numerical

Tag	Controller I/O	Type	Description
Device_Value	Input	INT	Each bit is a beam: bit 0 = beam 1 (lowest) … bit 7 = beam 8; 1 = interrupted

Configuration: analog

Tag	Controller I/O	Type	Description
Device_Volts	Input	INT	10 V × interrupted beams / 8, as hundredths of a volt (0–1000)

Inductive Sensor (metal)

Proximity sensor for metal. **Digital** gives Detect. **Analog** gives Volts: 0 V with the part at the sensor, rising to 10 V (1000) at the edge of the range, and 1000 when nothing metal is in range.

- **LED:** Red: detecting.
- **Detectable:** Conductive parts only: metal blocks, metal stock, metal bases and lids, cans.
- **Sensing range:** 0 – 60 px, default 10 px (≈ 0.17 m). Factory I/O: 0 – 0.1 m.
- **Settings:** Output; Sensing range (px; 60 px = 1 m)

Configuration: digital

Tag	Controller I/O	Type	Description
Device_Detect	Input	BOOL	Detecting: a conductive (metal) part is within range

Configuration: analog

Tag	Controller I/O	Type	Description
Device_Volts	Input	INT	0–10 V as hundredths of a volt: distance from the sensor to the nearest metal part, 1000 when none in range

Capacitive Sensor (any material)

Short-range proximity sensor for anything. **Digital** gives Detect. **Analog** gives Volts exactly like the inductive sensor, without the material filter.

- **LED:** Green: detecting.
- **Detectable:** Any material: solids (and liquids on the real device).
- **Sensing range:** 0 – 60 px, default 12 px (≈ 0.2 m), same as the real sensor.
- **Settings:** Output; Sensing range (px; 60 px = 1 m)

Configuration: digital

Tag	Controller I/O	Type	Description
Device_Detect	Input	BOOL	Detecting: any solid part is within range

Configuration: analog

Tag	Controller I/O	Type	Description
Device_Volts	Input	INT	0–10 V as hundredths of a volt: distance from the sensor to the nearest part, 1000 when none in range

Color Sensor (R/B/G/Y banks)

Four colour banks, one BOOL each.

- **LED:** The head shows the colour it sees.
- **Detectable:** Coloured plastic parts: red, blue, green, yellow. Metal gives no bank.
- **Sensing range:** Directly beneath the head.

Tag	Controller I/O	Type	Description
Device_Red	Input	BOOL	Bank 1: red
Device_Blue	Input	BOOL	Bank 2: blue
Device_Green	Input	BOOL	Bank 3: green
Device_Yellow	Input	BOOL	Bank 4: yellow

Vision Sensor

Recognises the part under it. **Digital**: one BOOL per ticked part. **Bits**: the part code as five binary inputs Bit0-Bit4 (Factory I/O's "All Digital"). **Numerical**: one INT with the part code (the number next to each part in the list; 0 = none or not ticked). **ID**: the serial number of the part in view plus its code, usable like a barcode. **Size**: the older Small/Medium/Large/Metal outputs. Tick **Triggered** for a Trigger output with Busy and Complete, like a real smart camera: the result then holds until the next trigger.

- **LED:** Green: a part in view. Amber: result valid for a selected part. The ring light flashes on a trigger.
- **Detectable:** Every part in the catalogue; tick the ones to detect.
- **Sensing range:** Directly beneath the camera.
- **Settings:** Output; Camera module (its name in I/O Configuration; blank = wired signals only); Housing; Parts to Detect; Triggered (adds Trigger input, Busy and Complete); Inspection time (ms)

Configuration: digital

Tag	Controller I/O	Type	Description
Device_Detect	Input	BOOL	A part is in the field of view
Device_Pass	Input	BOOL	The part is one of the selected parts
Device_CartonS	Input	BOOL	Carton (S) in view
Device_CartonM	Input	BOOL	Carton (M) in view
Device_CartonL	Input	BOOL	Carton (L) in view
Device_MetalM	Input	BOOL	Metal Block (M) in view

Configuration: bits

Tag	Controller I/O	Type	Description
Device_Detect	Input	BOOL	A part is in the field of view
Device_Pass	Input	BOOL	The part is one of the selected parts
Device_Bit0	Input	BOOL	Part code bit 0 (the code of the part in view, 1..21, as 5 binary digits; all 0 = none or not selected)
Device_Bit1	Input	BOOL	Part code bit 1 (the code of the part in view, 1..21, as 5 binary digits; all 0 = none or not selected)
Device_Bit2	Input	BOOL	Part code bit 2 (the code of the part in view, 1..21, as 5 binary digits; all 0 = none or not selected)

Device_Bit3	Input	BOOL	Part code bit 3 (the code of the part in view, 1..21, as 5 binary digits; all 0 = none or not selected)
Device_Bit4	Input	BOOL	Part code bit 4 (the code of the part in view, 1..21, as 5 binary digits; all 0 = none or not selected)

Configuration: numerical

Tag	Controller I/O	Type	Description
Device_Detect	Input	BOOL	A part is in the field of view
Device_Pass	Input	BOOL	The part is one of the selected parts
Device_Code	Input	INT	Part code: 1 = Carton (S), 2 = Carton (M), 3 = Carton (L), 7 = Metal Block (M); 0 = none or not selected

Configuration: id

Tag	Controller I/O	Type	Description
Device_Detect	Input	BOOL	A part is in the field of view
Device_Pass	Input	BOOL	The part is one of the selected parts
Device_ID	Input	INT	Serial number of the part in view (every part gets its own; 0 = none)
Device_Code	Input	INT	Part code, as in Numerical mode

Configuration: size

Tag	Controller I/O	Type	Description
Device_Detect	Input	BOOL	A part is in the field of view
Device_Pass	Input	BOOL	The part is one of the selected parts
Device_Small	Input	BOOL	Small part
Device_Medium	Input	BOOL	Medium part
Device_Large	Input	BOOL	Large part
Device_Metal	Input	BOOL	Metal part

Configuration: camera tags only

Tag	Controller I/O	Type	Description

RFID Reader

Single-trigger command interface: set **Command**, pulse **Execute**; when **CommandID** increments the command has finished and **Status** says how it went (0 = success). Command 0 only checks for a tag, 1 reads the serial into ReadData, 2 reads the word at MemoryIndex, 3 writes WriteData there, 4 clears the memory. Two tags in range give status 2. Write a route code into a part at one station and sort on it at the next.

- **LED:** Green: idle. Orange: executing a command. Red: the last command failed.
- **Detectable:** RFID tags. Every part carries one except raw stock, housing bases and housing lids. Each tag has a unique serial number and 16 words of memory.

- **Sensing range:** 0 – 60 px, default 16 px.
- **Settings:** Reading range (px); Command time (ms)

Tag	Controller I/O	Type	Description
Device_Command	Output	INT	Command to execute: 0 = is a tag available?, 1 = read serial number, 2 = read data, 3 = write data, 4 = clear all data
Device_Execute	Output	BOOL	Rising edge executes the command
Device_MemoryIndex	Output	INT	Tag memory index for read/write (0-15)
Device_WriteData	Output	INT	Data written by command 3
Device_CommandID	Input	INT	Number of commands executed so far; changes when the current one has finished
Device_Status	Input	INT	0 = success, 1 = no tag, 2 = too many tags, 3 = invalid index, 4 = invalid command
Device_ReadData	Input	INT	Result of command 1 (serial) or 2 (data)
Device_TagPresent	Input	BOOL	Exactly one tag is within range

Appendix E — Keyboard shortcuts

Insert	Add a rung after the selected one	Enter	Edit the selected rung
Delete	Delete the selected rung (or the selected tag in Controller Tags)	Esc	Cancel the edit
↑ / ↓	Select the previous / next rung	Ctrl+↑ / Ctrl+↓	Move the rung
Drag a tab	Rearrange the document tabs; the order is remembered	Right-click a tab	Move Left / Right / to Start / to End, Reset Tab Order, Close Others
Ctrl+D	Add or edit the comment on the selected rung (also the Comment button; double-click a comment to edit it)	Enter / Shift+Enter / Esc	In a comment: save / new line / cancel
Ctrl+C / Ctrl+X	Copy / cut the selected instruction or branch (or the whole rung when no instruction is selected)	Ctrl+V / Ctrl+Shift+V	Paste after / before the selection. Works across rungs, routines, programs and projects
Ctrl+Enter	In a Structured Text routine: accept the text	Tab	In a Structured Text routine: indent
Alt+3	Open or close the Quick Watch window		
Ctrl+E	Cross Reference for the selected instruction or tag	Ctrl+G	Go To a routine and rung
Ctrl+F / F3 / Shift+F3	Find / find next / find previous (on this page Ctrl+F goes to the search box)	Ctrl+H	Find and Replace
Ctrl+Z / Ctrl+Y	Undo / redo	Ctrl+T	Toggle the selected bit
Ctrl+S / Ctrl+O	Save the project file / open one	Double-click a tab	Maximize / restore the document

Appendix F — Glossary

Accumulator (.ACC)

The running value of a timer (milliseconds elapsed) or a counter (counts so far). It is a DINT, so it shows up for compare and math instructions, not for XIC.

Adapter

The EtherNet/IP device that is owned and answers the scanner: a camera, a drive, a rack of remote I/O.

Add-On Instruction (AOI)

An instruction you define yourself: parameters, local tags and a ladder routine, packaged under one name and used like TON or MOV. Every call needs its own backing tag. See the Add-On Instructions section.

Alias tag

A second name for another tag or an I/O point. Start_PB aliased to Local:1:I.Data.0 is that input, under a readable name. Cross Reference treats the alias and its target as the same thing.

Analog I/O

A channel that carries a number instead of on/off: a level, a weight, a speed reference. Here it is an INT on the 1769-IF4 and 1769-OF4 cards.

Array

A tag with a Dim: Name[0] to Name[Dim-1], all of one data type. Used by the file, shift, FIFO and sequencer instructions.

Assemble (edits)

The last step of an online edit: the accepted edits become the program and the original rungs are removed.

Assembly (CIP)

A block of bytes an EtherNet/IP device exposes as one object, class 4. The input assembly is what the device produces, the output assembly is what it consumes. The EDS file gives the instance numbers and sizes.

Backing tag

The tag of an Add-On Instruction's own data type that goes with one call. It holds that call's parameters and local tags, so two calls with two backing tags run independently.

Backplane

The bus the controller and its local cards sit on. Local:3:O.Data.0 means slot 3 on the backplane.

Base tag

The real tag or I/O address behind an alias.

BOOL

One bit: 0 or 1. What XIC, XIO and OTE work on.

Branch

A parallel path in a rung, written [leg , leg]. Power gets through if any leg is true.

Bridge

The small server (TcpBridge, built into LogixConnect.exe) that gives the page real TCP and UDP sockets. Everything that talks to hardware goes through it.

Bumpless transfer

Switching a PID loop between manual and automatic without a jump in the output.

CIP

Common Industrial Protocol: the object model (classes, instances, attributes, services) that EtherNet/IP carries.

Class 1 connection

A cyclic I/O connection: after a Forward Open, both ends send their data over UDP 2222 every RPI. Also called implicit messaging.

CompactLogix

The Allen-Bradley controller family this workstation is modelled on (1769-L33ER).

Consumed tag

A controller tag whose value is read from another controller at a fixed interval (RPI).

Continuous task

The task that runs over and over as fast as it can. Here MainTask scans every 20 ms.

CONTROL

The structure the file instructions use to keep their place: .LEN, .POS and the status bits .EN .DN .ER .EM .UL.

Control variable (CV)

The output a control loop moves: a valve position, a heater power, a speed reference.

Controller Organizer

The tree on the left: controller tags, tasks, programs and routines, Add-On Instructions, data types, I/O configuration.

Controller tags

Tags every program can use. All I/O and alias tags are controller tags. Program tags belong to one program.

Counter

CTU and CTD count rung transitions into .ACC and set .DN when .ACC reaches .PRE. RES clears it.

Cross Reference

A list of every instruction that reads or writes a tag, across all routines, with the ones that write it marked Destructive. Ctrl+E.

Data context

Which call of an Add-On Instruction you are looking at when its logic is open: the definition, or one backing tag's live values.

Data type

What a tag holds: BOOL, SINT, INT, DINT, REAL, STRING, a structure such as TIMER, a user-defined type or an Add-On Instruction's type.

Deadband

A range around the setpoint inside which the error is treated as zero, so the output does not chase noise.

Destructive reference

A place where an instruction writes a tag (OTE, MOV destination, a timer instruction on its timer). Cross Reference marks these with Y.

Destructive reference

An instruction that writes the tag (OTE, MOV destination, TON on its timer) rather than only reading it. Two destructive references to one bit is the classic double-coil bug.

DINT

A 32-bit signed integer. The default number type in Logix.

Done bit (.DN)

Set by a timer when .ACC reaches .PRE, by a counter when the count is reached, by a message when it completes.

EDS file

Electronic Data Sheet: the text file a vendor ships that describes an EtherNet/IP device. The identity, the assemblies, the connections and their sizes come from it.

Enable bit (.EN)

True while the instruction's rung-in condition is true (timers, messages, file instructions).

EnableIn / EnableOut

The two built-in parameters of every Add-On Instruction: the rung-in condition it was called with, and the rung-out condition it passes on.

EtherNet/IP

CIP carried over standard Ethernet: explicit messages on TCP 44818 and cyclic I/O on UDP 2222.

Event task

A task that runs when something happens: an EVENT instruction, a trigger tag going true or changing, or a timeout with no trigger.

Exclusive owner

The one scanner allowed to write a device's outputs. A second one is refused with an ownership conflict.

Explicit messaging

Request and reply, one at a time, on TCP: read this attribute, write that tag. The MSG instruction does this.

Expression

A formula written out, as in CPT, CMP, FAL, FSC and Structured Text: Level > 50 AND NOT Fault, or Src[C.POS] * 2 + 1.

Fault handler

A program under Controller Fault Handler that runs when a major fault happens. Here it runs once and the fault is cleared.

Fault, major

An error that stops the controller: mode shows Faulted and logic stops until Clear Faults, unless a fault handler program clears it. Integer divide by zero is one.

Fault, minor

An error that is logged while the scan carries on, such as arithmetic overflow.

FBD_TIMER, FBD_COUNTER, FBD_ONESHOT

The structures the Structured Text timers, counter and one-shots work on (TONR, TOFR, RTOR, CTUD, OSRI, OSFI). You set their inputs, such as .TimerEnable and .PRE, then call the instruction.

FIFO

First in, first out: FFL loads a value into an array, FFU unloads the oldest one.

File instruction

An instruction that works through an array one element at a time using a CONTROL tag: FAL, FSC, AVE, SRT, STD, COP, FLL.

Force

An override on an I/O point from the programming software, on top of the device (inputs) or the logic (outputs). Forces act only while enabled.

Forward Open

The CIP request that opens a connection. The scanner asks for sizes and an RPI, the device answers with the connection IDs, then cyclic data starts.

FreqCommand / OutputFreq

A drive's speed reference and its actual output frequency, in hundredths of a hertz: 6000 is 60.00 Hz.

GSV / SSV

Get / Set System Value: read or write controller data such as the wall clock, task scan times or a module's EntryStatus.

I/O Configuration

The part of the organizer listing the local cards and the Ethernet modules the controller owns.

I/O Monitor

The live list of every input and output point with its value, the device wired to it and the rungs that use it.

Implicit messaging

Cyclic I/O data with no request: both ends just send at the RPI. See Class 1 connection.

In-Sight

Cognex's vision system family. Over EtherNet/IP it exposes trigger and status bits and an inspection results area.

Indirect addressing

Choosing an array element with a tag instead of a number: Recipe[Step]. The element is picked each time the instruction runs. A subscript outside the array is a major fault.

Inhibit

Configured but switched off: an inhibited task or module stays in the project and does not run.

Input / Output / InOut (parameter)

How an Add-On Instruction parameter is passed. Input: the value is copied in. Output: the value is copied out. InOut: the logic works on the caller's tag directly.

Instruction

One element of a rung: a contact, a coil or a box such as TON, MOV or an Add-On Instruction.

INT

A 16-bit signed integer.

Interlock

A condition in series that must be true before an output may come on.

JSR / SBR / RET

Jump To Subroutine, and the Subroutine and Return instructions that receive and hand back its parameters.

L5X

The XML file Studio 5000 uses to export and import a project or a piece of one. File → Import L5X and Export L5X read and write it.

Ladder Diagram (LD)

The relay-style language of rungs between two power rails. The only language here.

Latch / Unlatch

OTL sets a bit and it stays set until OTU clears it, whatever the rungs do in between.

LIFO

Last in, first out: LFL loads a value into an array, LFU unloads the newest one.

Listen-only

A connection that receives a device's inputs without owning its outputs. In-Sight cameras do not offer one.

Local tag

A tag that belongs to an Add-On Instruction and exists once per backing tag: its timers, counters and working values.

Main routine

The routine a program runs each scan. Other routines run only when a JSR calls them.

MCR zone

The rungs between two MCR instructions. While the first MCR rung is false, every rung in the zone is scanned as false, so ordinary outputs drop. It is not a substitute for a hard-wired safety relay.

Module

A device in the I/O configuration: a local card, or an EtherNet/IP device created from an EDS file.

MSG

The Message instruction: one explicit request to a device or another controller, with .EN .DN .ER status bits.

Neutral text

The text form of a rung, for example XIC(Start)OTE(Motor); . Press Enter on a rung to edit it this way.

One-shot

ONS, OSR, OSF: true for exactly one scan when the rung goes true (or false), so something happens once per event.

Online

Connected to the controller, so you see live values. Here the workstation is always online with its own controller unless you go offline.

Online edit

Changing logic while the controller runs. The change is pending (i), then accepted (I), then tested, then assembled. See the Online edits section.

Operand

A value or tag an instruction works on: the bit of an XIC, the source and destination of a MOV.

OTE

Output Energize: the coil. The bit follows the rung, every scan.

Overlap

A task triggered again before its previous run had started or finished. Counted in GSV(Task,name,OverlapCount).

Ownership conflict

The error a device returns when a second scanner asks for a connection it has already given to another.

Pen

One tag plotted on a trend. A trend holds up to 8, each with its own colour.

Pending edit

An online edit that is still only in the editor: the i rung. The controller has not been given it yet.

Periodic task

A task that runs at a fixed period in milliseconds, for logic that must run at a known rate.

PID

Proportional, Integral, Derivative: the standard closed-loop control instruction. It compares the process variable with the setpoint and moves the control variable to close the gap.

PLC

Programmable Logic Controller: the industrial computer that reads inputs, runs the logic and writes outputs, over and over.

Power flow

Whether the rung is true up to a given point. The green highlighting on contacts and wires.

Pre-emption

A higher-priority task interrupting a lower one part-way through. Here it happens between rungs; the interrupted task then carries on from where it stopped.

Preset (.PRE)

The target of a timer (milliseconds) or counter (counts).

Priority (task)

1 is highest, 15 lowest. Periodic and event tasks all outrank the continuous task.

Process variable (PV)

The measured value a control loop is controlling: a level, a temperature, a speed.

Produced / consumed tag

A tag one controller broadcasts and another receives cyclically. Not supported here.

Produced tag

A controller tag that is sent to another controller at a fixed interval (RPI).

Program

A group of routines with its own tags, scheduled in a task.

Program mode

Controller mode where inputs update but no logic runs and outputs are off.

Program tags (local tags)

Tags that belong to one program, listed under Parameters and Local Tags. Only that program's routines can use them, and inside that program they win over a controller tag of the same name.

Quick Watch

Your own list of tags with live values that you can change, shown in a floating window (Alt+3) and in the Watch tab of the output window. Saved with the project.

REAL

A 32-bit floating-point number.

Remote controller

Another controller this one shares produced and consumed tags with: simulated inside the project, or a real one reached through the bridge.

Remote I/O

Inputs and outputs on a block out on the network instead of in the controller chassis. Their tags are named after the module, such as RIO1:I.Pt00.

Requested Packet Interval (RPI)

How often, in milliseconds, each end of a cyclic connection sends its data.

Retentive

Keeps its value when the rung goes false. RTO is a retentive timer; only RES clears it.

Routine

A list of rungs. Each program has a main routine and may have more.

Run mode

Controller mode where logic scans and outputs are driven.

Rung

One line of ladder: conditions on the left, outputs on the right.

Rung comment

Free text above a rung saying what it does. Ctrl+D.

Sandbox

The free-form project where you place your own devices and wire them to I/O.

Scan

One pass: read inputs, run the tasks, write outputs. 20 ms here.

Scanner

The EtherNet/IP device that opens connections and owns the adapters: normally the PLC, here the workstation.

Scope

Where a tag lives and who can see it: the controller (everyone) or one program (only its routines). The Scope box in the tag editor switches between them.

Seal-in

A branch around a start contact using the output's own bit, so a momentary start keeps the output on until a stop breaks the rung.

Sequencer

SQO steps through an array of words and copies each to an output through a mask.

Setpoint (SP)

The value a control loop is trying to hold.

SINT

An 8-bit signed integer.

Slot

A position on the backplane. In a route, slot 0 is the CompactLogix controller.

STRING

Text of up to 82 characters with a .LEN and the characters in .DATA.

Structure

A tag made of members: TIMER, COUNTER, CONTROL, MESSAGE, a user-defined type or an Add-On Instruction tag.

Structured Text (ST)

A text language for routines: assignments with :=, IF, CASE, FOR, WHILE and REPEAT. Good for math, loops and decisions that are clumsy in ladder. See the Structured Text section.

Tag

A named piece of controller memory. Logix addresses everything by tag name.

Task

What schedules programs: continuous, periodic or event. Each has a priority; a higher-priority task interrupts a lower one.

TCP Client Socket

A sandbox device that opens a plain TCP connection through the bridge to send and receive text, for equipment that speaks ASCII.

Test (edits)

Running the accepted edits in place of the original rungs without committing to them. Untest goes back.

Test mode

Controller mode where logic scans but outputs stay off.

Timer

TON, TOF, RTO: time a condition in milliseconds into .ACC and set .DN at .PRE. .TT is true while timing.

Timing bit (.TT)

True while a timer is counting: enabled and not done yet.

Trend

A chart of tag values against controller time. Each plotted tag is a pen. See the Trends section.

Trigger

The signal that makes a camera acquire an image. Over EtherNet/IP: set Trigger Enable, pulse Trigger, wait for the acknowledge and Results Valid.

UCMM

Unconnected Message Manager: explicit messages sent without opening a connection first.

UID / UIE

User Interrupt Disable and Enable. Rungs between them cannot be interrupted by another task.

User-Defined Data Type (UDT)

A structure you define: named members of any type. A tag of that type gets Tag.Member for each.

Variable frequency drive (VFD)

A drive that runs an AC motor at a commanded frequency. The built-in one uses the PowerFlex 525 tags: Start, Stop, FreqCommand out; Ready, Active, Faulted, OutputFreq in.

Verify

Check every rung for errors without running anything. The results are in the Errors window.

Watchdog

The limit on how long a task may run. Here a loop that never ends is stopped with a major fault (type 6) instead of freezing the controller.

XIC

Examine If Closed: a contact that is true when the bit is 1.

XIO

Examine If Open: a contact that is true when the bit is 0.

Zone markers (i I r R d D)

The letters in the rung margin during an online edit. Lower case is pending, upper case is in the controller: i/I the new rung, r/R the original it replaces, d/D a rung being deleted.

Appendix G — Instruction set

Instructions the rung editor accepts (parameter counts as in Logix): `XIC`, `XIO`, `OTE`, `OTL`, `OTU`, `ONS`, `OSR`, `OSF`, `TON`, `TOF`, `RTO`, `CTU`, `CTD`, `RES`, `MOV`, `ADD`, `SUB`, `EQU`, `NEQ`, `GRT`, `GEQ`, `LES`, `LEQ`, `LIM`, `MEQ`, `JSR`, `SBR`, `RET`, `JMP`, `LBL`, `AFI`, `NOP`, `MSG`, `GSV`, `SSV`, `COP`, `CPS`, `FLL`, `MUL`, `DIV`, `MOD`, `SQR`, `NEG`, `ABS`, `CLR`, `CPT`, `DTOS`, `STOD`, `RTOS`, `STOR`, `CONCAT`, `UPPER`, `LOWER`, `MID`, `FIND`, `DELETE`, `INSERT`, `BSL`, `BSR`, `FFL`, `FFU`, `SQO`, `MVM`, `BTD`, `SWPB`, `AND`, `OR`, `XOR`, `NOT`, `CMP`, `XPY`, `TRN`, `SCP`, `SIN`, `COS`, `TAN`, `ASN`, `ACS`, `ATN`, `LN`, `LOG`, `DEG`, `RAD`, `FAL`, `FSC`, `AVE`, `STD`, `SRT`, `SIZE`, `LFL`, `LFU`, `SQI`, `SQL`, `PID`, `EVENT`, `UID`, `UIE`, `TND`, `MCR`, `BRK`, `FOR`, `TONR`, `TOFR`, `RTOR`, `CTUD`, `OSRI`, `OSFI`.

Appendix H — Walkthrough log

For the first reader. Work through Part IV in order; after each lesson's *Run it*, tick the row if the check passed as written, and note anything that was unclear, out of order, or did not behave as the text says — a sentence is enough. The log is what the next revision of this book is written from.

Reader: _____ Version of the page: _____ Date started: _____

#	Lesson	The check	Passed	Time taken	Where I stalled / what was unclear
1	Lay out the machine (ch. 15)	39 devices; Scene sheet reads ≈ 52 DI, 29 DO, 7 AI in use; pipe, incline and latch doors visible in Side / 3D	☐		
2	The tag list (ch. 16)	Monitor Tags: Start goes to 1 when pressed; Stop is already 1; toggling Bottle_Feed_Emit drops a bottle and LS1 goes to 1	☐		
3	Step 1 — run circuit (ch. 17)	Start: Run lamp on, divert conveyor turning, Line_Run = 1. Stop drops it. Start while holding Stop: nothing	☐		
4	Step 2 — dwell and feeder (ch. 18)	One bottle drops at the head; Dwell.ACC counts to 1000 and holds; Dwell.DN stays on	☐		
5	Step 3 — registers (ch. 19)	Exist bit 0 = 1 after the first dwell; toggling Conv_Index twice walks it to bit 1	☐		
6	Step 4 — filling (ch. 20)	Verifies clean; filler head tank reads 100 %; lamps dark (nothing fills yet)	☐		
7	Step 5 — capping (ch. 21)	Verifies clean; green column lamp on when the line runs	☐		
8	Step 6 — index (ch. 22)	Belt steps one pitch per dwell; bottles fill at stop 7 and cap at stop 10; Simulator Log shows fills and caps	☐		
9	Step 7 — sort (ch. 23)	Broken bottles into the shredder; a box under the chute; IN BOX counts to 4; small bottles down the incline; no "lost", no "no box" log lines	☐		
10	Step 8 — count and preset (ch. 24)	Dial 0020: line stops at LARGE = 20; Enter zeroes; Start runs again	☐		

#	Lesson	The check	Passed	Time taken	Where I stalled / what was unclear
11	Step 9 — supply (ch. 25)	Tank falls to 20 %, red flashes, inlet opens, level climbs to 90 %	☐		
12	Step 10 — labeler (ch. 26)	Every capped bottle leaves labeled; with 3 labels on the roll: fault latched, amber, line keeps running; click reloads, Enter clears	☐		
13	Step 11 — alarms (ch. 27)	Three minutes and random Stop / Start: Alarm_Word 0. Each injected fault caught by its bit; lamp flashes, steady after Reset; Clear, Reset, Start recovers	☐		
14	Step 12 — packing (ch. 28)	Cases come in fast, creep onto the mark, are packed with four and leave; Case_Speed_Ref 10000 / 2500; CASES counts; a drive trip latches bit 6 and Reset clears it	☐		
15	Commissioning (ch. 29)	All checklist items pass; compare with the template via Export / Copy as Text	☐		
16	Levels 1–5 (ch. 31–35)	Each level completes with every bottle in the right bin, under par	☐		
17	Level 6 — Pack it (ch. 36)	Complete, every good bottle packed and the broken ones in the scrap bin, under 140 s	☐		
18	Level 7 — Batch Mix (ch. 37)	Three batches A, B, C on spec (ratio, temperature, mixing time), no overflow or scorch, under 240 s	☐		

Overall. Which chapter of Part II did you have to re-read before Part IV made sense? _____
Which lesson was the hardest? _____

What would you add, cut or move?